System Design

High-Level and Low-Level Design in Java

Scalability fundamentals, data and caching patterns, high-level designs worked end to end, low-level design with complete Java code, with estimates, diagrams, trade-offs, failure scenarios with fixes, and interview preparation.

Estimation · Load Balancing · Caching · Sharding · Consistent Hashing · Queues
Rate Limiter · News Feed · Chat · Parking Lot · LRU · SOLID · Design Patterns

```
$ estimate --users 100M --writes 1/day --reads 20/day
  write QPS   ~1.2 K   peak ~3.5 K
  read  QPS   ~23 K    peak ~70 K
  storage     ~180 TB / 5 years
  cache hit   95% -> DB reads ~3.5 K QPS
  shards      16  (replication factor 3)
```


Maruthi Sreeram

Prepared for Java full stack interview preparation

Maruthi Sreeram

Version basis

Java code and simulations were run on JDK 21 on the build machine; figures are estimates
Real systems' internals are described from public documentation and labelled as such
Prepared for Maruthi Sreeram

How to use this guide

- **Who this is for.** A Java full stack developer who must explain, and diagnose, what the operating system does beneath a running service: process and thread management, memory, files, I/O, containers and synchronisation.
- **Every chapter follows the same steps:** (1) the concept and the problem it solves, (2) the mechanism, shown with diagrams, (3) a worked trace or calculation, (4) a hands-on experiment with real commands, C or Java code, (5) **real-world incidents**, each presented as symptoms, diagnosis, root cause, fix and prevention, and (6) the questions interviewers ask, with answers.
- **How to study.** Read each diagram before the prose and narrate it aloud. Reproduce every experiment on a Linux machine or virtual machine (never on a production host). Treat each incident as an exercise: cover the diagnosis, decide which command to run first, then compare with the text.
- **Running the code.** Save a Java listing as `Main.java` and run `java Main.java`; compile C with `gcc -O0 -o demo demo.c`. Terminal sessions begin with `$`. Output shown was produced on the build machine unless labelled typical.
- **Scope.** The focus is Linux, the platform on which most Java services run, together with the JVM and container behaviour that interacts with it. Windows and macOS are mentioned only where the difference matters.

Contents

Part 1: System Design Fundamentals 4

1.1 How to Approach a System Design Interview 4
1.2 Back-of-the-Envelope Estimation 10
1.3 Scalability and Performance Basics 17
1.4 Load Balancing and Networking for Design 26
1.5 Availability and Reliability 37
1.6 Choosing the Data Store 48
1.7 Caching 55
1.8 Replication, Partitioning and Consistent Hashing 63
1.9 Consistency and Distributed Coordination 73
1.10 Asynchronous Processing and Messaging 86
1.11 Common Infrastructure Components 94

Part 2: High-Level Design Case Studies 106

2.1 Design: URL Shortener 106
2.2 Design: Pastebin 114
2.3 Design: Distributed Rate Limiter 118
2.4 Design: Unique ID Generator 127
2.5 Design: Distributed Key-Value Store 133
2.6 Design: Distributed Cache 143
2.7 Design: Notification System 151
2.8 Design: Web Crawler 159
2.9 Design: Distributed Task Scheduler and Job Queue 167
2.10 Design: News Feed 176
2.11 Design: Chat System 183
2.12 Design: Video Streaming Platform 191
2.13 Design: Ride Hailing and Proximity Service 198

sreram

2.14 Design: Ticket Booking System . . . 205
2.15 Design: E-Commerce Checkout and Inventory . . . 214
2.16 Design: Search Autocomplete . . . 222
2.17 Design: File Storage and Sync (Dropbox, Google Drive) . . . 230
2.18 Design: Payment System and Wallet . . . 239

Part 3: Low-Level Design in Java . . . 249

3.1 Object-Oriented Design Principles . . . 249
3.2 Design Patterns for LLD . . . 255
3.3 The LLD Interview Process . . . 265
3.4 LLD: Parking Lot . . . 268
3.5 LLD: LRU and LFU Cache . . . 273
3.6 LLD: Elevator System . . . 278
3.7 LLD: Splitwise, Expense Sharing . . . 283
3.8 LLD: Vending Machine . . . 288
3.9 LLD: Rate Limiter . . . 293
3.10 LLD: Logger Framework . . . 298
3.11 LLD: Movie Ticket Booking, Seat Hold . . . 303
3.12 LLD: Tic-Tac-Toe . . . 309
3.13 LLD: Pub-Sub Event Bus . . . 314
3.14 LLD: Task Scheduler . . . 318
3.15 LLD: Connection Pool . . . 323

Part 4: Operating at Scale and Interview Preparation . . . 329

4.1 Observability and Operations at Scale . . . 329
4.2 Security in System Design . . . 339
4.3 Architecture Styles and Evolution . . . 344
4.4 Failure Case Studies . . . 348
4.5 Interview Preparation . . . 358
4.6 Cheat Sheets and Study Plan . . . 366
4.7 Putting It Together: Designing a Complete Product End to End . . . 371

sreeram

PART 1

System Design Fundamentals

Vocabulary, estimation, scaling, load balancing and reliability: the toolkit that every later design reuses.

In this part

- **1.1** How to Approach a System Design Interview
- **1.2** Back-of-the-Envelope Estimation
- **1.3** Scalability and Performance Basics
- **1.4** Load Balancing and Networking for Design
- **1.5** Availability and Reliability
- **1.6** Choosing the Data Store
- **1.7** Caching
- **1.8** Replication, Partitioning and Consistent Hashing
- **1.9** Consistency and Distributed Coordination
- **1.10** Asynchronous Processing and Messaging
- **1.11** Common Infrastructure Components

1.1 How to Approach a System Design Interview

A system design problem is deliberately under-specified: "design a news feed" or "design a ticket-booking service" admits thousands of valid answers and no single correct one. The evaluation therefore concentrates on the process by which a candidate narrows the problem, quantifies it, selects components and defends trade-offs. A candidate who begins drawing boxes within the first minute usually builds a design for a problem that was never posed. A candidate who follows a fixed framework spends the available time on the decisions that matter and leaves a written trail that the interviewer can steer.

This chapter defines a seven-step framework, assigns a time budget to each step for a 45-minute session, separates functional from non-functional requirements, lists the mistakes that cost the most points, and applies the framework to a small worked example. The case-study chapters that follow in later Parts use the same section order, so the framework also serves as a reading guide.

1.1.1 The seven-step framework

The framework consists of seven steps. Each step produces a concrete artefact that is written on the board, which keeps the discussion anchored and allows the interviewer to correct a wrong assumption early, when the correction is cheap.

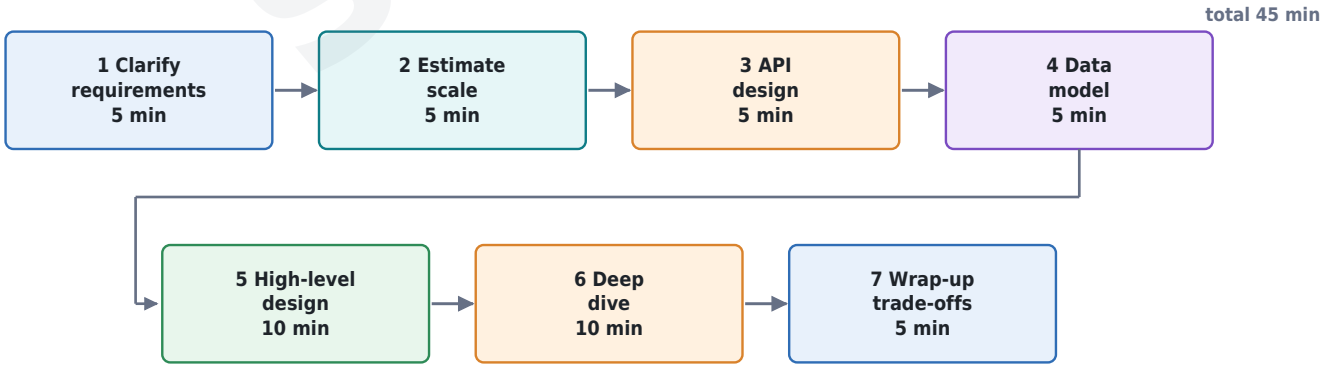


Figure 1.1. The seven steps in order with the time allocated to each; the first four steps (20 minutes) define the problem, and the last three (25 minutes) solve it.

Step	Activities	Artefact on the board
1 Clarify	Ask about users, core features, scale, platforms, latency and consistency expectations; state out-of-scope items	Functional list, non-functional list, assumptions

Step	Activities	Artefact on the board
2 Estimate	Derive request rates, storage, bandwidth and memory from assumed user counts	Table of numbers with arithmetic
3 API	Define the externally visible operations, identifiers, pagination, idempotency, error model	Endpoint list with request and response shapes
4 Data model	Choose entities, keys, relationships, access patterns, partitioning key, indexes	Schema sketch plus the queries it must serve
5 High-level design	Place clients, load balancers, services, caches, stores, queues, workers; trace the main flows	Architecture diagram and numbered request flows
6 Deep dive	Pick the two or three hardest problems (hot keys, fan-out, consistency, ordering) and compare alternatives	Decision with the assumption that drives it
7 Wrap-up	Review bottlenecks, failure modes, monitoring, extensions; summarise trade-offs	Risk list and next steps

Steps, activities and artefacts.

Step 1: clarify requirements

Clarification converts a vague prompt into a scoped problem. The questions fall into five groups:

- **Users and use cases.** Who uses the system (consumers, internal staff, other services), and which three or four features are essential? The candidate proposes a minimal feature set and asks for confirmation instead of enumerating every conceivable feature.
- **Scale.** Daily active users, peak concurrency, growth horizon, read-to-write ratio, geographic distribution.
- **Quality attributes.** Latency expectations, availability target, consistency requirements, durability expectations, data retention.
- **Constraints.** Existing technology, team size, regulatory requirements, budget, build versus buy.
- **Out of scope.** Explicitly stated exclusions such as billing, administration consoles, or machine-learning ranking, so that the interviewer can confirm that the exclusion is acceptable.

Requirements are classified into two kinds. A **functional requirement** states what the system does ("a user can post a message; followers see it in a feed"). A **non-functional requirement** (also called a quality attribute) states how well the system does it ("a feed loads in under 200 ms at the 99th percentile; the service is available 99.95 percent of the time"). Functional requirements determine which components exist; non-functional requirements determine how those components are arranged, replicated and scaled.

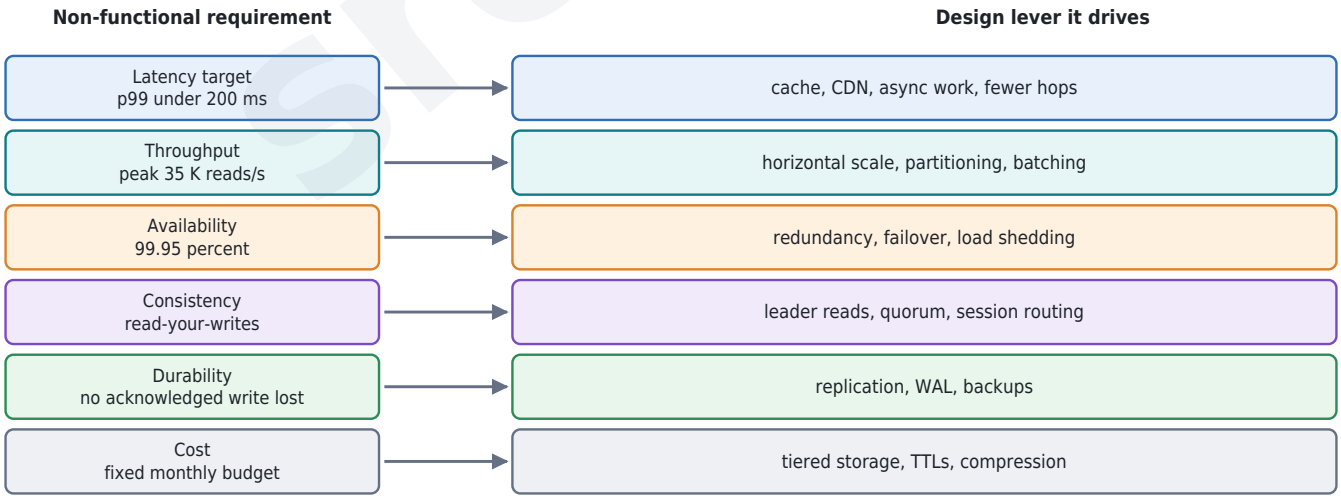


Figure 1.2. Each non-functional requirement maps to a family of design levers; stating the requirement with a number makes the corresponding lever selectable.

A non-functional requirement without a number cannot drive a decision. "Fast" is not a requirement; "p99 read latency below 200 ms" is. When the interviewer does not supply a number, the candidate proposes one, labels it as an assumption and continues.

Step 2: estimate

Back-of-the-envelope estimation (the next chapter treats the technique in detail) establishes the order of magnitude of the load. The purpose is not accuracy but the selection of a design regime: 100 requests per second fit on one server; 100 000 per second do not. Three results are almost always needed: peak requests per second, total storage over the retention period, and peak network bandwidth. A fourth, working-set memory, decides whether a cache is useful.

Step 3: API design

The API fixes the contract between clients and the system. For each core use case the candidate writes one operation with its method and path (or RPC name), parameters, response, and error behaviour. Four properties are discussed whenever they are relevant: **pagination** for list endpoints (cursor-based pagination is preferred for large or changing collections, because offset pagination degrades and returns duplicates under concurrent writes); **idempotency** for operations with side effects (a client-supplied idempotency key makes retries safe); **authentication and authorisation** (stated, not designed in detail unless the prompt targets it); and **versioning**.

Step 4: data model

The data model is derived from the access patterns, not from the entities alone. For each query that the API implies, the candidate states which key is used to find the data, how the data is partitioned across nodes, and which index supports the query. The choice between a relational store and a NoSQL store follows from those patterns and from the consistency requirements, as covered in the databases material; here the decision is justified in one sentence per store.

Step 5: high-level design

The high-level design is a block diagram plus two or three numbered request flows (for instance, the write path and the read path). Components are added only when a requirement or a number justifies them: a cache appears because a read-to-write ratio of 100 to 1 was established in step 2, a queue appears because a write must be acknowledged before slow processing finishes. Every arrow in the diagram is labelled with the protocol or the data it carries.

Step 6: deep dive

The interviewer typically selects one or two components for depth, or accepts the candidate's proposal. A deep dive follows a fixed pattern: state the problem and its numbers, list two or three alternatives, compare them in a table against the relevant criteria, select one, and state the assumption that drives the choice together with the condition under which the choice would be reversed. The second and third parts of this book apply the pattern to more than twenty systems.

Step 7: wrap-up

The wrap-up addresses what the design does not yet cover: single points of failure, behaviour under overload, monitoring (which metrics and alerts would show the design failing), security, and the next two extensions. A candidate who names the weakest part of the own design before the interviewer does is evaluated more favourably than one who presents the design as flawless.

1.1.2 Time management for a 45-minute interview

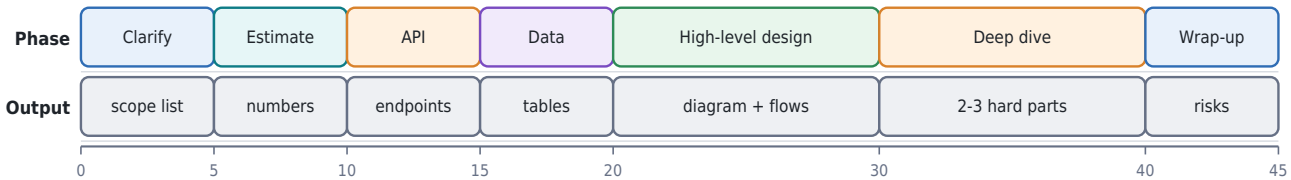


Figure 1.3. Time allocation for a 45-minute session; the high-level design and deep dive together take 20 of the 45 minutes, and every step ends with a visible artefact.

The allocation above is a guideline, not a rule. Two adjustments are common. When the prompt is a familiar one (a URL shortener), steps 1 to 4 compress to 12 to 15 minutes, which leaves more time for the deep dive. When the interviewer interrupts with questions, the time belongs to the interviewer's agenda, and the candidate resumes the framework at the step that was interrupted. In a 60-minute session the extra 15 minutes belong to the deep dive and the wrap-up.

A short checkpoint at the end of each step keeps the session on schedule: "The scope is: post, follow, feed. Scale is 100 million daily users. Shall the estimation proceed?" The checkpoint costs ten seconds and prevents a long divergence.

1.1.3 Worked example: a page-view counter service

The example is deliberately small so that every step fits in a few lines. The prompt is: "Design a service that counts page views for a website and shows the count on each page."

Step 1: clarification (answers assumed)

- Functional: record a view of a page; return the view count of a page; counts need not be exact to the last view.
- Non-functional: record path acknowledged in under 50 ms at p99; read path under 100 ms at p99; availability 99.95 percent; counts may lag real time by up to 5 seconds (eventual consistency is acceptable); no view is lost once acknowledged.
- Scale (assumed): 1 billion page views per day, 50 million distinct pages, 5 years of retention of daily totals.
- Out of scope: unique-visitor counting, bot filtering, per-user analytics, dashboards.

Step 2: estimation

Quantity	Arithmetic	Result
Average write rate	1 000 000 000 / 86 400 s	11 574 per second, about 11.6 K/s
Peak write rate	11.6 K/s x 3 (assumed peak factor)	about 35 K/s
Read rate	1 read per view (page render shows the count)	same as writes, about 35 K/s at peak
Event size	page id (8 B) + timestamp (8 B) + overhead (about 84 B)	about 100 B
Raw event volume per day	1 000 000 000 x 100 B	100 GB per day
Counter state	50 000 000 pages x 16 B (id + count)	about 0.8 GB: fits in memory of one node

Estimates for the counter service (all inputs assumed).

The two numbers that shape the design are 35 K writes per second at peak, which is more than a single relational row-update path should absorb for hot pages, and the 0.8 GB counter state, which is small enough to cache entirely.

Step 3: API

Counter API

```
POST /v1/views
     body: {"pageId": "p-123"}
     -> 202 Accepted
GET  /v1/pages/{pageId}/views
     -> 200 {"pageId": "p-123", "views": 91823}
GET  /v1/pages/{pageId}/views?bucket=day&from=2026-10-01&to=2026-10-07
     -> 200 [{"day": "2026-10-01", "views": 120}, {"day": "2026-10-02", "views": 133}]
```

The write returns 202 (accepted for processing) rather than 200, which states honestly that the count is updated asynchronously. No idempotency key is required because the semantics are "at least one view recorded"; if exactly-once counting were required, the request would carry an event identifier and the aggregator would deduplicate.

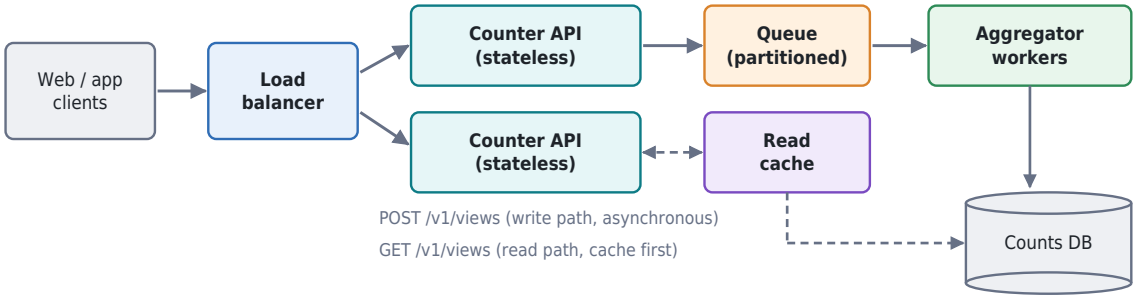
Step 4: data model

Counter tables SQL

```sql
-- current total, hot, read often
CREATE TABLE page_total  (page_id BIGINT PRIMARY KEY, views BIGINT NOT NULL);
-- history, written once per page per day
CREATE TABLE page_daily  (page_id BIGINT, day DATE, views BIGINT NOT NULL,
                          PRIMARY KEY (page_id, day));
```

The partitioning key is the page identifier, so that all updates of one page reach one partition and the read for a page touches one partition.

Steps 5 and 6: high-level design and deep dive



Write path: client, LB, API, queue, aggregator (batch sums per page), DB.

Read path: client, LB, API, cache; on miss, counts DB; result cached for a few seconds.

Figure 1.4. Write path (top row): API servers append events to a partitioned queue, and aggregator workers apply batched sums to the counts store; read path: cache first, store on miss.

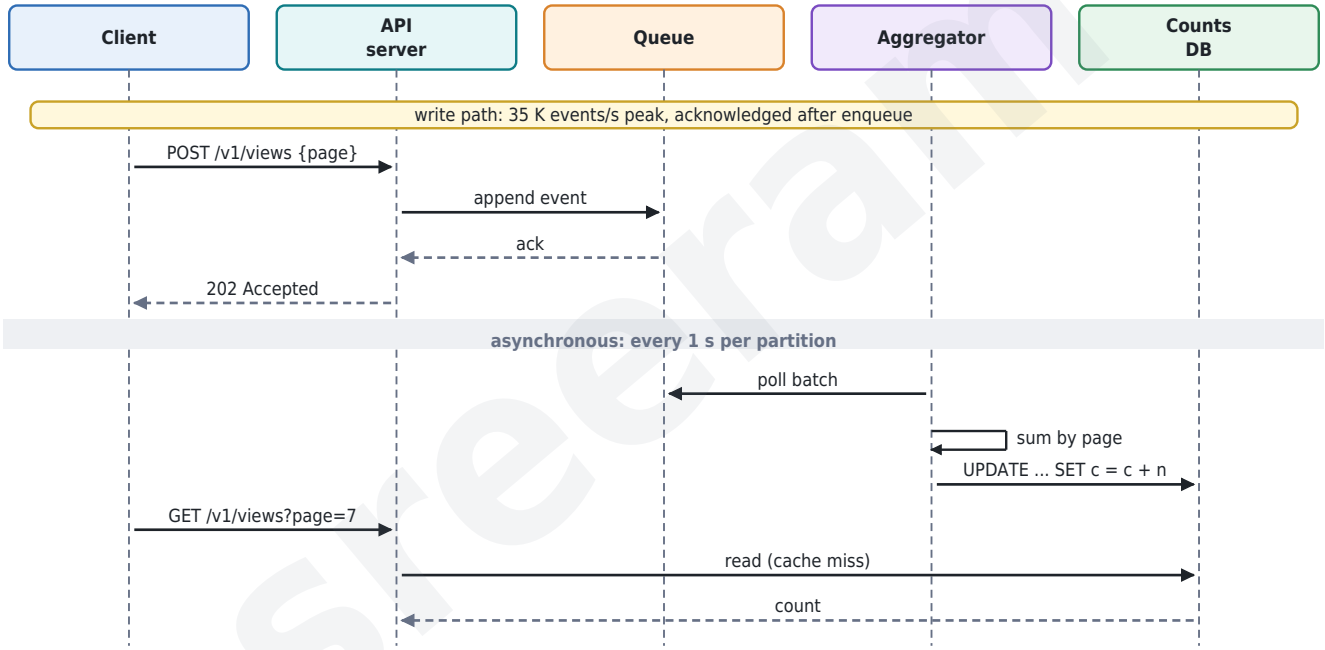


Figure 1.5. Sequence of the write path (acknowledged after the queue append) and of a read that misses the cache.

The deep dive concerns one problem: 35 K increments per second must not become 35 K row updates per second, because a popular page would then serialise on a single row. Three alternatives are compared:

Alternative	Behaviour	Advantage	Disadvantage
Synchronous UPDATE per view	Each request executes "views = views + 1"	Simplest; count always current	Row-lock contention on hot pages; write latency tied to the database
In-memory counter per API node, periodic flush	Each node sums locally and flushes every second	Very cheap; reduces writes by the batch factor	A node crash loses up to one second of counts (violates the no-loss requirement)
Queue, then aggregator batch	Events are appended durably, then summed per page per window	Durable once acknowledged; batch factor reduces database writes; absorbs spikes	One more component; counts lag by the window (1 to 5 s)

Alternatives for the write path.

The third alternative is selected because the requirement "no acknowledged view is lost" excludes the second, and the tolerance of a few seconds of lag permits the third. The assumption that drives the decision is the lag tolerance; if exact real-time counts were required, the design would return to synchronous updates with sharded counters (one logical counter split across several rows that are summed on read).

Step 7: wrap-up

- Single points of failure: the queue is replicated (three replicas, assumed); aggregator workers are stateless and restart from the last committed offset.
- Overload: the API applies admission control and returns 429 when the queue append latency exceeds a threshold (see the chapter on availability and reliability in this Part).
- Monitoring: consumer lag (seconds behind), queue append p99, aggregator batch size, cache hit ratio, database write rate.
- Extensions: unique visitors via a probabilistic structure (HyperLogLog), per-region counts, bot filtering upstream.

1.1.4 Mistakes that cost the most points

Mistake	Correction
Drawing the architecture before stating requirements	Spend the first five minutes on scope; write the list on the board
Numbers without arithmetic, or arithmetic without assumptions	Show each multiplication and label every input as an assumption
Naming technologies instead of properties ("use Kafka")	State the property needed (durable, ordered, replayable log), then name a technology that provides it
Adding components that no requirement justifies	Tie every component to a number or a requirement
A single design with no alternatives	For each major decision, name at least one alternative and the reason for rejecting it
Ignoring failure	For every stateful component, state what happens when it fails or lags
Treating the interviewer as an examiner instead of a collaborator	Offer checkpoints; adopt hints; follow the interviewer's chosen deep dive
Over-engineering for planet scale when the numbers are modest	Let the estimate choose the regime: one database with a replica is correct for 200 requests per second
Silence while thinking	State the decision under consideration, for example "choosing between cache-aside and write-through"

Frequent mistakes and corrections.

INTERVIEW ANGLE

How is a system design interview evaluated? Typical rubrics weigh four areas: problem exploration (requirements, scope, assumptions), technical depth (data model, component choices, correctness of the reasoning), trade-off analysis (alternatives and the conditions under which each wins), and communication (structure, use of numbers, response to hints). Correct component names carry little weight on their own; a justified decision with a stated assumption carries much more.

1.1.5 Questions to expect

1. Why are requirements clarified before any design?

The same prompt yields different designs for 1 000 and for 100 million users, and for strong versus eventual consistency. Clarification fixes those parameters, prevents the design of a system that was not requested, and demonstrates the engineering habit of scoping before building.

2. How is a non-functional requirement turned into a design decision?

By attaching a number and then mapping the number to a lever: a p99 latency target of 200 ms limits the number of sequential network hops and suggests caching; an availability target of 99.95 percent (about 4.4 hours of downtime per year) requires redundancy across failure domains; a durability requirement excludes designs that acknowledge a write before it is replicated.

3. What is to be done when the interviewer provides no numbers?

Propose round, defensible assumptions, state them aloud, and ask whether they are acceptable. For example: "I assume 100 million daily active users, 10 reads and 0.1 writes per user per day." The estimate remains valid because every later number derives from the stated assumptions and can be recomputed if the interviewer changes one.

4. How does a candidate choose which component to deep-dive?

The deep dive targets the component whose failure or scaling limit would invalidate the design: the hottest data path, the component with the weakest consistency story, or the one the estimate shows to be at the edge of a single node. The interviewer's interest overrides the candidate's choice.

5. What is the difference between a design that "works" and one that is "good"?

A working design satisfies the functional requirements at the stated scale. A good design also states which requirement each component serves, identifies its own bottlenecks and failure modes, gives alternatives with reasons for rejection, and remains simple enough that the stated team could operate it.

6. How is time recovered when the discussion runs long?

By summarising the current step in one sentence, recording any open question on the board as a follow-up item, and moving to the next step. Skipping the wrap-up is the most common and most damaging way to run out of time; it is protected by reserving the last five minutes.

7. Is it acceptable to name a specific product?

Yes, provided the property that motivates the choice is stated and an alternative is acknowledged. "A partitioned, replicated log such as Kafka, because the aggregator needs durable, replayable ordered input; a managed queue would also work if replay is not required" is a complete statement.

KEY POINTS

- The seven steps are: clarify, estimate, API, data model, high-level design, deep dive, wrap-up; each produces a visible artefact.
- Functional requirements decide which components exist; numeric non-functional requirements decide how they are arranged and scaled.
- Every number is labelled as an assumption and shown with its arithmetic; every component is tied to a requirement or a number.
- A deep dive compares alternatives in a table, selects one, and states the assumption that drives the choice.
- A 45-minute budget is 5, 5, 5, 5, 10, 10 and 5 minutes; the wrap-up is protected.
- A design that names its own weaknesses is evaluated above one that claims none.

1.2 Back-of-the-Envelope Estimation

Capacity estimation answers one question before any component is drawn: which regime does the system belong to? A service that peaks at 200 requests per second and stores 50 GB is served by one application server, one primary database and one replica. A service that peaks at 400 000 requests per second and stores 600 TB requires partitioning, caching tiers and a content delivery network. The architecture differs in kind, not only in degree, and the estimate selects between the two.

The estimate is intentionally coarse. A result within a factor of two to three of reality is sufficient, because the design decisions that follow depend on orders of magnitude. The discipline consists of three habits: state every input as an assumption, show every multiplication, and round aggressively to keep the arithmetic mental.

1.2.1 Concept and mechanism

Numbers to memorise

Storage and traffic estimates rely on a small set of constants.

Power	Exact value	Approximation	Common name
2^10	1 024	1 thousand	1 KB
2^20	1 048 576	1 million	1 MB
2^30	1 073 741 824	1 billion	1 GB
2^40	1 099 511 627 776	1 trillion	1 TB
2^50	about 1.13 x 10^15	1 quadrillion	1 PB

Power	Exact value	Approximation	Common name
2^32	4 294 967 296	4.3 billion	number of IPv4 addresses; 32-bit unsigned limit
2^63 - 1	about 9.22 x 10^18	9.2 quintillion	Java long maximum

Powers of two and their decimal approximations.

Time conversions used continually:

Quantity	Value	Rule of thumb
Seconds per day	86 400	round to 100 000 (10^5) for mental arithmetic; error 16 percent
Seconds per month (30 days)	2 592 000	about 2.5 million
Seconds per year	31 536 000	about 3 x 10^7 (pi x 10^7 is close)
1 million events per day	11.6 events per second	about 12 per second
1 billion events per day	11 574 events per second	about 12 K per second
1 event per second	86 400 events per day	about 100 K per day

Time and rate conversions.

Latency numbers

The relative cost of operations determines where a design places data. The following figures are the classical list attributed to Jeff Dean and reproduced by Peter Norvig (norvig.com/21-days.html, "Approximate timing for various operations on a typical PC"). They date from about 2009 and describe orders of magnitude: absolute values have improved (solid-state storage and faster networks), but the ratios between memory, storage and network remain instructive.

Operation	Time	Relative to a memory reference
Fetch from L1 cache	0.5 ns	1/200
Branch misprediction	5 ns	1/20
Fetch from L2 cache	7 ns	1/14
Mutex lock and unlock	25 ns	1/4
Fetch from main memory	100 ns	1
Send 2 KB over a 1 Gbit/s network	20 000 ns (20 us)	200
Read 1 MB sequentially from memory	250 000 ns (250 us)	2 500
Disk seek (new disk location)	8 000 000 ns (8 ms)	80 000
Read 1 MB sequentially from disk	20 000 000 ns (20 ms)	200 000
Packet round trip, US to Europe	150 ms	1 500 000

Approximate timing of operations, per Norvig's list.

Two further values appear in most versions of the list that circulate among engineers (for example the interactive tables derived from Dean's 2009 talk); they are quoted here as orders of magnitude and not verified for current hardware: a round trip within one data center is about 0.5 ms, and a random 4 KB read from a solid-state drive is about 0.1 to 0.15 ms. Cloud network round trips between availability zones of one region are commonly in the range of one to a few milliseconds; the exact value must be measured on the platform in use.

sreram

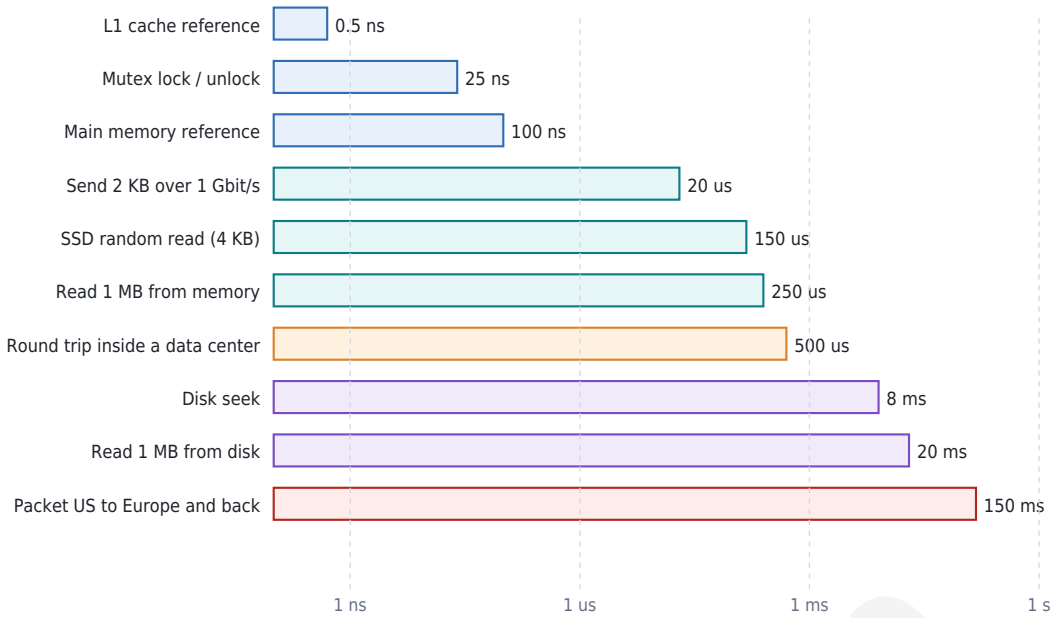


Figure 1.6. Operation latencies on a logarithmic axis: each gridline is a factor of one thousand; a cross-continent round trip costs about a million memory references, which is why designs keep data close to the computation and avoid sequential remote calls.

Four conclusions follow from the table:

- Memory is roughly three orders of magnitude faster than a network round trip within a data center, and five orders faster than a cross-continent one. Caching saves a network trip, not only a disk access.
- Sequential access beats random access by a large factor on rotating disks (reading 1 MB sequentially costs 20 ms; each seek costs 8 ms). Log-structured and append-only designs exploit this.
- Compressing data before it crosses a slow link is usually profitable, because computation is far cheaper than transmission.
- Round trips accumulate: a request that performs 20 sequential calls inside one data center spends about 10 ms in the network alone.

The estimation formulas

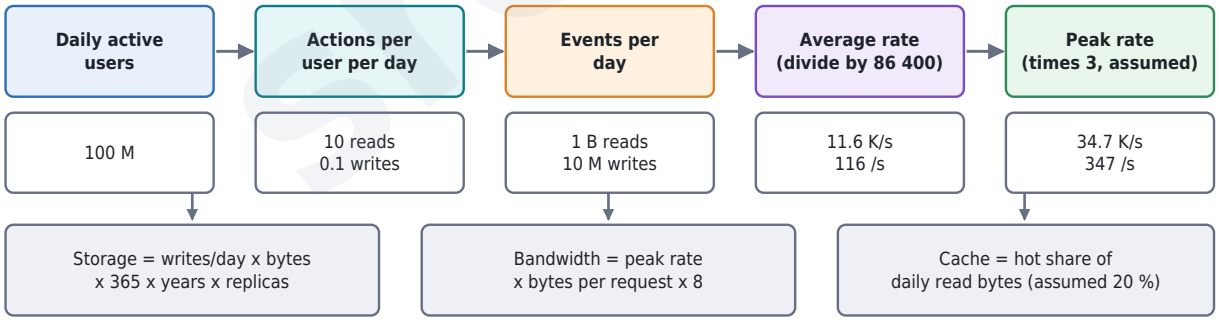


Figure 1.7. The pipeline from users to peak rate (top) and the derived quantities (bottom), using the URL-shortener inputs from the worked example.

Let D be daily active users, w and r the writes and reads per user per day, s_w and s_r the bytes per write and per read, Y the retention in years, R the replication factor and P the peak factor.

Quantity	Formula
Average write rate	`D x w / 86 400`
Average read rate	`D x r / 86 400`
Peak rate	`average rate x P (P is commonly 2 to 5 for consumer traffic; assumed per system)`
Daily new data	`D x w x s_w`

Quantity	Formula
Total storage	`daily new data x 365 x Y x R (add 20 to 50 percent for indexes and metadata)`
Peak ingress or egress bandwidth	`peak rate x bytes per request x 8 bits`
Cache size	`daily read bytes x hot fraction (often 20 percent by the 80/20 rule; an upper bound, because repeated reads of the same object count once in the working set)`
Servers required	`peak rate / (per-server capacity x target utilisation); target utilisation about 0.5 to 0.7`

Core formulas.

The peak factor deserves attention. Traffic is not uniform: a daily cycle typically yields a peak-hour rate two to three times the daily average, and events (a sale, a live match) produce short bursts of ten times or more. The system is sized for the peak plus headroom, not for the average.

The read-to-write ratio is the second parameter that shapes a design. Ratios of 100 to 1 and above (a URL shortener, a content site) favour caching and replicas; ratios near 1 to 1 or below (metrics ingestion, logging) favour append-optimised, partitioned storage.

Per-server capacity is itself an assumption that must be measured for the real application. Planning values often quoted as orders of magnitude, not guarantees: a stateless application server performing little work per request handles hundreds to a few thousand requests per second per core-pair, an in-memory key-value store handles on the order of 100 000 simple operations per second per instance, and a single relational database node handles thousands to tens of thousands of simple queries per second depending on the query and the hardware. These values guide the first sketch and are replaced by measurements (see load testing in a later chapter).

1.2.2 Worked example

Three systems of very different shape are estimated with the same formulas. Every input is an assumption chosen for illustration; none is a measured figure of any real service.

Input	URL shortener	Social timeline	Video streaming
Daily active users	100 million	200 million	50 million
Writes per user per day	0.1	2	0.002 (uploads)
Reads per user per day	10	100	5 (views)
Bytes per write	500 B	300 B	300 MB
Bytes per read	500 B	1 000 B	50 MB
Retention	5 years	5 years	5 years
Peak factor	3	2	2
Replication	3	3	1.5 (erasure coding, assumed)

Inputs (all assumed).

URL shortener, step by step

- Writes per day: 100 000 000 x 0.1 = 10 000 000. Average write rate: 10^7 / 86 400 = 115.7 per second; peak 3 x 115.7 = 347 per second.
- Reads per day: 100 000 000 x 10 = 1 000 000 000. Average read rate: 11 574 per second; peak 34.7 K per second. The read-to-write ratio is 100 to 1.
- New data per day: 10^7 x 500 B = 5 GB. Over 5 years with three replicas: 5 GB x 365 x 5 x 3 = 27.4 TB.
- Peak egress: 34.7 K x 500 B x 8 = 139 Mbit/s: negligible. The system is limited by request rate, not by bandwidth.
- Cache at 20 percent of daily read bytes: 10^9 x 500 B x 0.2 = 100 GB, an upper bound; the number of distinct popular links is far smaller, so a few gigabytes per cache node suffice.

Conclusion: 35 K reads per second at peak requires several application servers and a cache tier, but 347 writes per second and 27 TB of storage fit a small sharded or even a single replicated relational deployment. The design regime is "read-heavy, cache-first".

Social timeline and video streaming

The timeline workload produces 231 K reads per second on average (463 K at peak) and 657 TB of replicated storage: partitioning of the data store and a cache measured in terabytes are required, and the read path (not the write path) determines the architecture. The video workload has negligible request rates (6 K views per second at peak) but 2.3 Tbit/s of peak egress and 82 PB of storage: the problem is bandwidth and bytes, which places a content delivery network and object storage at the centre. The same formulas, applied to different inputs, select three different architectures.

1.2.3 Hands-on: an estimator in Java

The following program implements the formulas above for any workload description. It was compiled and run on JDK 21 with the single-file launcher.

Estimator.java JAVA

```java
import java.util.Locale;

public class Estimator {

    /** All inputs are assumptions supplied by the caller. */
    record Workload(String name, double dau, double writesPerUserDay, double readsPerUserDay,
                    double bytesPerWrite, double bytesPerRead, double retentionYears,
                    double peakFactor, double cacheShare, double replication) { }

    static final double SECONDS_PER_DAY = 86_400;

    static String bytes(double v) {
        String[] unit = {"B", "KB", "MB", "GB", "TB", "PB"};
        int i = 0;
        while (v >= 1000 && i < unit.length - 1) { v /= 1000; i++; }
        return String.format(Locale.ROOT, "%.1f %s", v, unit[i]);
    }

    static String bits(double v) {
        String[] unit = {"bit/s", "Kbit/s", "Mbit/s", "Gbit/s", "Tbit/s"};
        int i = 0;
        while (v >= 1000 && i < unit.length - 1) { v /= 1000; i++; }
        return String.format(Locale.ROOT, "%.1f %s", v, unit[i]);
    }

    static String rate(double v) {
        if (v >= 1e3) return String.format(Locale.ROOT, "%.1f K/s", v / 1e3);
        return String.format(Locale.ROOT, "%.1f /s", v);
    }

    static void line(String label, String value) {
        System.out.printf(Locale.ROOT, "%-24s: %s%n", label, value);
    }

    static void report(Workload w) {
        double writesDay = w.dau() * w.writesPerUserDay();
        double readsDay = w.dau() * w.readsPerUserDay();
        double avgWrite = writesDay / SECONDS_PER_DAY, avgRead = readsDay / SECONDS_PER_DAY;
        double peakWrite = avgWrite * w.peakFactor(), peakRead = avgRead * w.peakFactor();
        double storageDay = writesDay * w.bytesPerWrite();
        double storageTotal = storageDay * 365 * w.retentionYears() * w.replication();
        System.out.println("== " + w.name() + " ==");
        line("writes per day", String.format(Locale.ROOT, "%,.0f", writesDay));
        line("reads per day", String.format(Locale.ROOT, "%,.0f", readsDay));
        line("write rate avg / peak", rate(avgWrite) + " / " + rate(peakWrite));
        line("read rate avg / peak", rate(avgRead) + " / " + rate(peakRead));
        line("read : write", String.format(Locale.ROOT, "%.0f : 1", readsDay / writesDay));
        line("new data per day", bytes(storageDay));
        line("total storage", bytes(storageTotal) + " (" + w.retentionYears() + " years, x"
                + w.replication() + " replicas)");
        line("peak ingress", bits(peakWrite * w.bytesPerWrite() * 8));
        line("peak egress", bits(peakRead * w.bytesPerRead() * 8));
        line("cache (" + (int) (w.cacheShare() * 100) + "% of read bytes)",
                bytes(readsDay * w.bytesPerRead() * w.cacheShare()));
        System.out.println();
    }

    public static void main(String[] args) {
```

Estimator.java (continued) JAVA

```java
        // name, DAU, writes/user/day, reads/user/day, B/write, B/read, years, peak, cache, replicas
        report(new Workload("URL shortener", 100e6, 0.1, 10, 500, 500, 5, 3, 0.2, 3));
        report(new Workload("Social timeline", 200e6, 2, 100, 300, 1_000, 5, 2, 0.2, 3));
        report(new Workload("Video streaming", 50e6, 0.002, 5, 300e6, 50e6, 5, 2, 0.02, 1.5));
    }
}
```

Run the estimator TERMINAL

```
$ java Estimator.java
== URL shortener ==
writes per day          : 10,000,000
reads per day           : 1,000,000,000
write rate avg / peak   : 115.7 /s / 347.2 /s
read rate avg / peak    : 11.6 K/s / 34.7 K/s
read : write            : 100 : 1
new data per day        : 5.0 GB
total storage           : 27.4 TB (5.0 years, x3.0 replicas)
peak ingress            : 1.4 Mbit/s
peak egress             : 138.9 Mbit/s
cache (20% of read bytes): 100.0 GB

== Social timeline ==
writes per day          : 400,000,000
reads per day           : 20,000,000,000
write rate avg / peak   : 4.6 K/s / 9.3 K/s
read rate avg / peak    : 231.5 K/s / 463.0 K/s
read : write            : 50 : 1
new data per day        : 120.0 GB
total storage           : 657.0 TB (5.0 years, x3.0 replicas)
peak ingress            : 22.2 Mbit/s
peak egress             : 3.7 Gbit/s
cache (20% of read bytes): 4.0 TB

== Video streaming ==
writes per day          : 100,000
reads per day           : 250,000,000
write rate avg / peak   : 1.2 /s / 2.3 /s
read rate avg / peak    : 2.9 K/s / 5.8 K/s
read : write            : 2500 : 1
new data per day        : 30.0 TB
total storage           : 82.1 PB (5.0 years, x1.5 replicas)
peak ingress            : 5.6 Gbit/s
peak egress             : 2.3 Tbit/s
cache (2% of read bytes): 250.0 TB
```

What to notice: the three outputs differ by orders of magnitude in different columns (request rate for the timeline, storage for the video platform, neither for the URL shortener); the `bits` helper converts bytes to bits to avoid the common error of comparing megabytes per second with megabits per second; and the cache line reports an upper bound, as explained above.

1.2.4 Real-world problems and solutions

The scenarios below are composite illustrations of failure classes; the figures in them are illustrative and do not report a specific incident.

Problem 1: capacity sized on the average rate

Scenario. A service is sized for 2 000 requests per second, its daily average, with each node loaded to about 80 percent.

Symptoms. Latency rises every evening between 19:00 and 22:00; error rates reach several percent; scaling out after the fact takes 15 minutes.

Diagnosis. The request-rate graph shows an evening peak of 3.5 times the daily average, and a campaign spike reaching 8 times.

Root cause. The estimate omitted the peak factor, and per-node utilisation was planned at 80 percent, which leaves no headroom for variance (see the utilisation curve in the next chapter).

Fix. Size for the observed peak hour at 50 to 60 percent utilisation, and add autoscaling with a scale-out threshold below saturation.

Prevention. Record the peak factor as an explicit assumption in the design document; review it against production metrics each quarter.

Problem 2: storage underestimated by replication, indexes and metadata

Scenario. A table of 5 TB of raw records is provisioned with 6 TB of disk.

Symptoms. The disk fills within two months; write performance degrades before the disk is full.

Diagnosis. The database reports 5 TB of table data, 1.8 TB of indexes, three replicas, and write-ahead log and backup retention on the same volume.

Root cause. The estimate counted payload bytes only. Indexes (20 to 50 percent), replication (three copies), write-ahead logs, snapshots and the free-space reserve needed for compaction or vacuuming were omitted.

Fix. Provision at least 2 times the logical size per copy, multiplied by the replica count; alert at 70 percent usage.

Prevention. The estimate table contains separate rows for payload, index overhead, replication and operational reserve.

Problem 3: bits and bytes confused in bandwidth planning

Scenario. A media endpoint is planned for a 1 Gbit/s link on the basis of "100 MB per second of peak traffic".

Symptoms. The link saturates at 125 MB per second of payload, but protocol overhead and retransmissions reduce usable throughput to about 110 MB per second; downloads stall at peak.

Diagnosis. Interface counters show sustained 940 Mbit/s.

Root cause. The plan treated 100 MB per second as 100 Mbit per second in one place and as 1 Gbit per second in another, and ignored overhead. 100 MB per second is 800 Mbit per second, which leaves almost no headroom on a 1 Gbit/s link.

Fix. Convert every figure to one unit (the estimator multiplies bytes by 8 exactly once) and plan for at most 60 to 70 percent of the nominal link rate.

Prevention. Record units in every row of the estimate table.

1.2.5 Questions to expect

1. Estimate the storage for 100 million users who each upload one 200 KB photo per day for five years.

Daily: 10^8 x 200 KB = 20 TB. Five years: 20 TB x 365 x 5 = 36.5 PB. With three replicas: about 110 PB. Thumbnails and variants add perhaps 20 to 30 percent. The answer states the assumptions (one upload per user per day is high; a typical figure may be 0.1 to 0.5) and shows that the result is dominated by retention and replication.

2. Why is the peak factor applied, and how is it chosen?

Systems fail at the peak, not on average. The factor is taken from comparable systems or from measured data: a daily cycle usually gives two to three, event-driven services five or more. When no data exists, 2 to 3 is assumed and labelled.

3. How many servers are required for 35 000 requests per second?

If a server sustains 1 000 requests per second at saturation and is run at 60 percent utilisation, each serves 600; 35 000 / 600 is about 58 servers, plus N+1 or N+2 redundancy and a zone failure margin, which brings the figure to about 75. The per-server capacity is an assumption to be replaced by a load-test result.

4. When is a cache useful, judged from the estimate?

When the read-to-write ratio is high, when the hot working set fits in memory, and when the saving in latency or database load justifies the consistency complexity. The 80/20 observation suggests that caching 20 percent of the data may serve 80 percent of reads, which the estimate checks against the cache memory budget.

5. What does a latency table teach about design?

That network round trips and disk seeks dominate, so designs minimise sequential remote calls, batch requests, keep data in memory when access is frequent, place data near users, and prefer sequential to random I/O.

6. How is bandwidth estimated for a video service?

Average bitrate times concurrent viewers: for example 5 Mbit/s x 1 000 000 concurrent streams = 5 Tbit/s. The result exceeds what an origin can serve, which establishes the need for a CDN and explains why the origin sees only cache-miss traffic.

7. Why are estimates rounded so aggressively?

Inputs such as daily active users are themselves uncertain by a factor of two. Precision beyond one significant digit is false precision, and rounding (86 400 to 10^5) keeps the arithmetic fast and the structure of the argument visible.

KEY POINTS

- Estimation selects the design regime; a factor-of-two error rarely changes the architecture, an order-of-magnitude error does.
- Rates derive from users x actions per day / 86 400, multiplied by an assumed peak factor of 2 to 5.
- Storage = daily bytes x 365 x years x replicas, plus index, log and reserve overhead.
- Bandwidth uses bits: bytes per second x 8; plan for 60 to 70 percent link utilisation.
- Latency ratios (memory, data-center network, disk, intercontinental) explain why caches, batching and locality dominate designs; quote the table as approximate and sourced.
- Every input is an assumption and appears with its arithmetic and units.

1.3 Scalability and Performance Basics

Scalability is the ability of a system to maintain its performance targets as load grows, by adding resources at a cost that grows roughly in proportion to the load. Performance is a separate property: a system can be fast for one user and collapse at one thousand. This chapter defines the vocabulary used in every later design (throughput, latency, percentiles, utilisation), states the three quantitative laws that predict behaviour (Amdahl, Little, and the queueing utilisation curve), and demonstrates the two effects that most often surprise engineers: tail latency amplification under fan-out and the non-linear growth of response time near saturation.

1.3.1 Concept and mechanism

Vertical and horizontal scaling

Vertical scaling (scale up) replaces a node with a larger one: more cores, memory or faster disks. It requires no application change, keeps data in one place and avoids distributed-systems complexity. It is bounded by the largest available machine, its cost per unit of capacity rises steeply at the top end, and a single node remains a single point of failure. **Horizontal scaling** (scale out) adds nodes behind a load balancer or partitions data across nodes. Capacity grows by adding machines, a node failure removes only a fraction of capacity, and the ceiling is far higher. The price is complexity: the application must tolerate several instances, data must be partitioned or replicated, and operations such as joins and transactions become distributed.

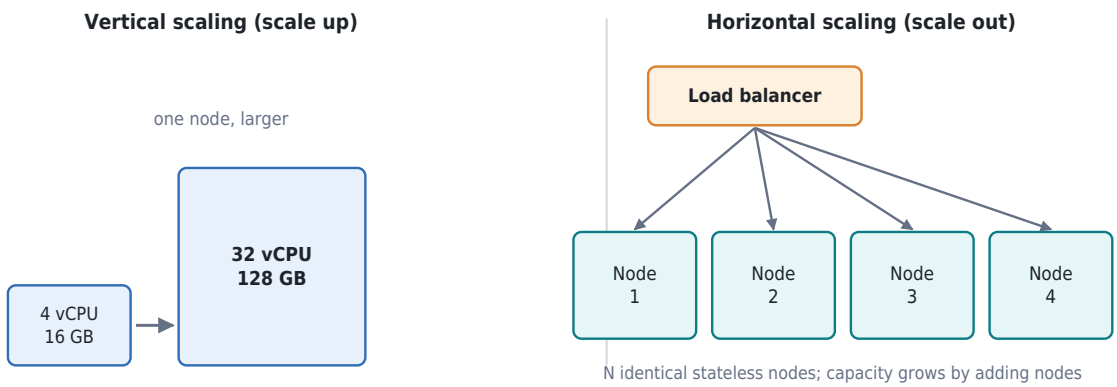


Figure 1.8. Scale up enlarges one node; scale out adds identical nodes behind a load balancer, which is possible only when requests are independent of which node serves them.

A practical rule is to scale vertically while it is cheap and simple, and to design the application so that scaling out remains possible: stateless application tiers first, then partitioned data when a single database node is exhausted.

Scaling decomposes along three independent axes, described as the scale cube in the literature of Abbott and Fisher: cloning identical instances behind a balancer (x axis), splitting functionality into services (y axis), and splitting data by key into partitions (z axis). Stateless web tiers use the x axis; microservices use the y axis; sharded databases use the z axis. Most systems apply all three at different layers.

Stateless services

A service is **stateless** when the response to a request depends only on the request and on external shared state, never on data held in the memory of a particular instance from earlier requests. Statelessness is the property that makes horizontal scaling of the application tier trivial: any instance can serve any request, so instances can be added, removed and replaced at will, and a failed instance loses nothing that cannot be recovered.

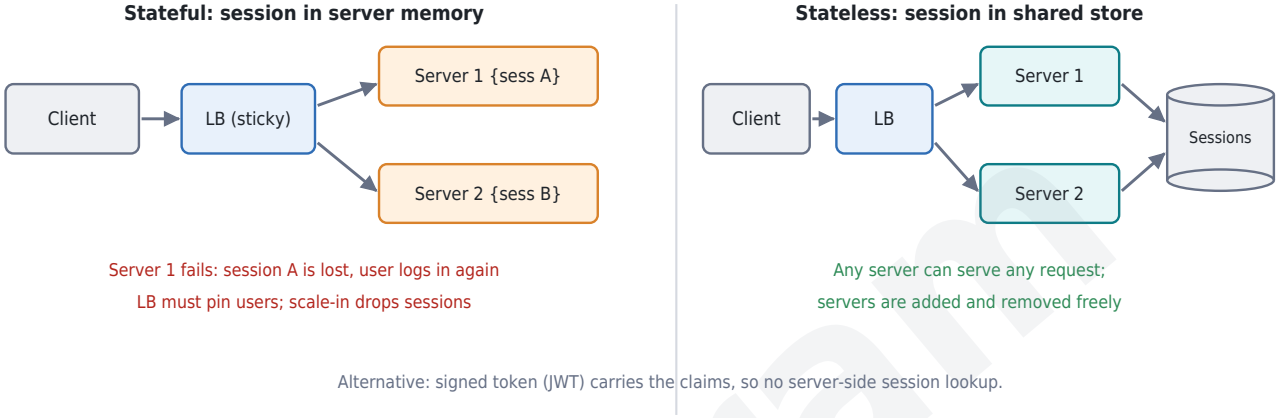


Figure 1.9. Left: sessions held in server memory force affinity and are lost when a server fails. Right: sessions externalised to a shared store (or carried in a signed token) leave the servers interchangeable.

Session state is the common offender. Three remedies exist: store the session in a shared, replicated store (an in-memory data store is typical), carry the state in the client in a signed or encrypted token (a JSON Web Token, as covered in the Spring Security material), or keep the server stateful and route by affinity, which accepts the failure and rebalancing problems described in the next chapter. Other instance-local state to be removed includes uploaded files on local disk, in-memory caches that must be consistent across instances, and scheduled jobs that run once per instance instead of once per cluster.

Throughput, latency and response time

Throughput is the number of operations completed per unit of time (requests per second, megabytes per second). **Latency** is the time one operation takes. **Response time**, as the client observes it, equals service time (the time the resource spends working on the request) plus waiting time (queueing) plus network time. Throughput and latency are related but distinct: a pipeline can have high throughput and high latency (a batch job), or low latency and low throughput (a single request on an idle machine). The targets in a design are stated for both, for example "40 K requests per second at p99 below 200 ms".

Percentiles and why averages mislead

The mean of a latency distribution hides its tail. A service with a mean of 50 ms may answer 99 percent of requests in 30 ms and one percent in 2 seconds. Percentiles state the distribution directly: **p50** (the median) is the typical case, **p95** and **p99** are the experience of the slowest 5 and 1 percent of requests, and **p99.9** captures rarer events. The Google SRE book states a general preference for percentiles over the mean, because averages hide long-tail behaviour while high percentiles show the plausible worst case.

Percentiles of different requests do not combine by averaging, and percentiles per node cannot be averaged into a cluster percentile; the underlying histograms must be merged. A single page view typically issues many requests, so a user meets a slow response far more often than the per-request percentile suggests: with 20 requests per page and 1 percent of requests slow, the probability that a page includes at least one slow request is 1 - 0.99^20 = 18 percent.

Tail latency amplification

The effect above compounds in services that fan out. When one request needs the answers of N backend calls issued in parallel, the request completes when the slowest call completes. If each call is slow with probability q, the probability that at least one of N calls is slow is 1 - (1 - q)^N. Dean and Barroso describe the case of a server that answers within one second 99 percent of the time: when a request must collect responses from 100 such servers, about 63 percent of user requests take more than one second ("The Tail at Scale", Communications of the ACM, 2013; the figure follows from 1 - 0.99^100).

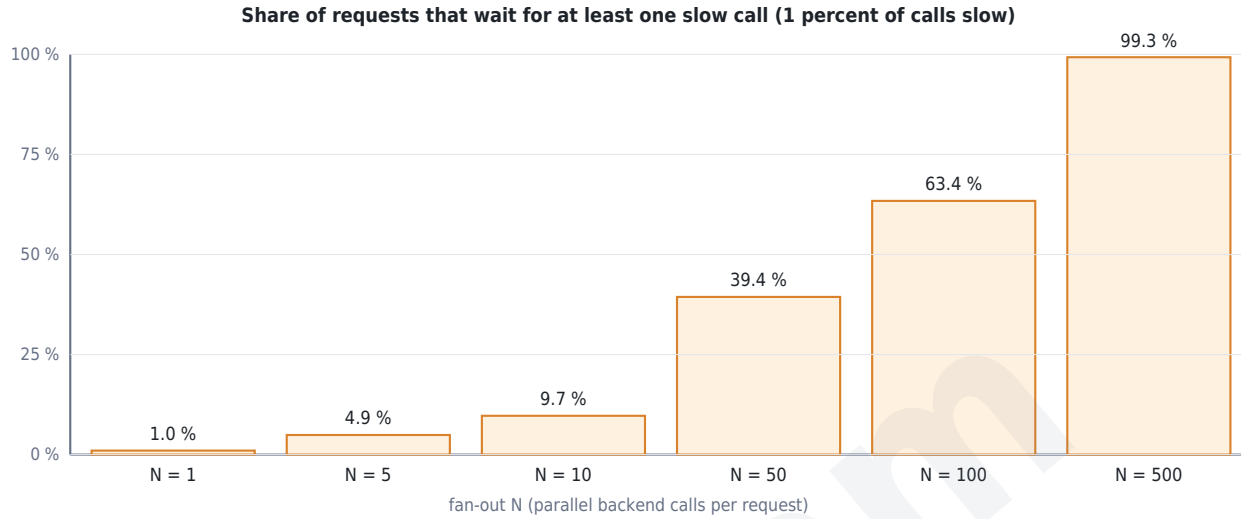


Figure 1.10. Probability that a request waits for at least one slow call, from a simulation of 200 000 requests per fan-out width (hands-on section); it matches 1 - 0.99^N.

Mitigations recorded in the same paper and in common practice are listed below.

Technique	Mechanism	Cost or limit
Reduce fan-out	Fewer, larger backend calls; hierarchical aggregation; pre-computed results	Design effort; may reduce parallelism
Timeouts with partial results	Return what has arrived by the deadline	Incomplete answers must be acceptable
Hedged requests	Send a second copy of the request to another replica if no reply arrives within the p95 latency; use the first reply	About 5 percent extra load when deferred to p95 (per Dean and Barroso)
Tied requests	Send to two replicas at once, each told of the other, so the second cancels when the first starts	Requires cancellation support; reported 40 percent reduction at p99.9 in one Google system (per the same paper)
Isolation and priorities	Separate queues for interactive and batch work	Capacity partitioning
Avoid shared slow paths	Remove synchronous calls to stragglers (garbage-collection pauses, compactions)	Operational discipline

Mitigations for tail latency.

Amdahl's law

Amdahl's law bounds the speedup obtainable by parallelising a task. If a fraction p of the work can run in parallel and the rest (1 - p) is serial, then the speedup on n workers is 1 / ((1 - p) + p / n), and the limit as n grows is 1 / (1 - p). A task that is 95 percent parallel can never run more than 20 times faster, regardless of the number of workers, and 32 workers achieve only 12.6 times.

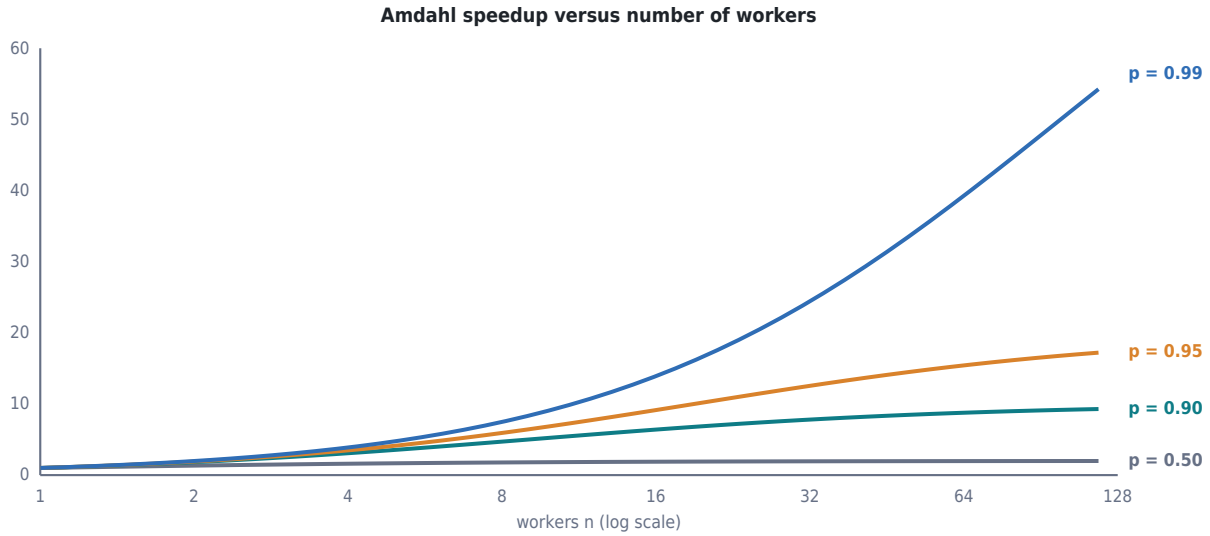


Figure 1.11. Speedup curves for four parallel fractions; the serial fraction (a lock, a single database primary, a shared counter) sets the plateau, so removing the serial portion matters more than adding workers.

In system design the serial fraction appears as any resource that every request must pass through: a single database primary, a global lock, a leader that sequences writes, a shared counter. The law also underestimates the real cost, because adding workers introduces coordination overhead (contention and coherence delays); the Universal Scalability Law of Neil Gunther extends the model with a term for it, which explains why throughput can decrease when too many workers contend for one resource.

Little's law

Little's law states that in a stable system the average number of items in the system equals the arrival rate multiplied by the average time each item spends in the system:

Little's law

```
L = lambda x W
L      average number of requests in the system (in flight)
lambda average arrival rate (requests per second)
W      average time in the system (seconds), queueing plus service
```

The law holds for any stable system regardless of the arrival or service distributions, which makes it the most useful sizing identity in practice. It converts a latency and a rate into a concurrency, and concurrency determines thread counts, connection pool sizes and memory.

Queueing and the utilisation curve

Requests wait when they arrive faster than they are served. For the simplest model of one server with random (Poisson) arrivals and exponentially distributed service times, called M/M/1, the mean time in the system is W = S / (1 - rho), where S is the mean service time and rho is the utilisation (arrival rate divided by capacity). The mean response time is 2 S at 50 percent utilisation, 5 S at 80 percent, 10 S at 90 percent, and 100 S at 99 percent. Response time does not grow in proportion to load; it grows without bound as utilisation approaches 100 percent.

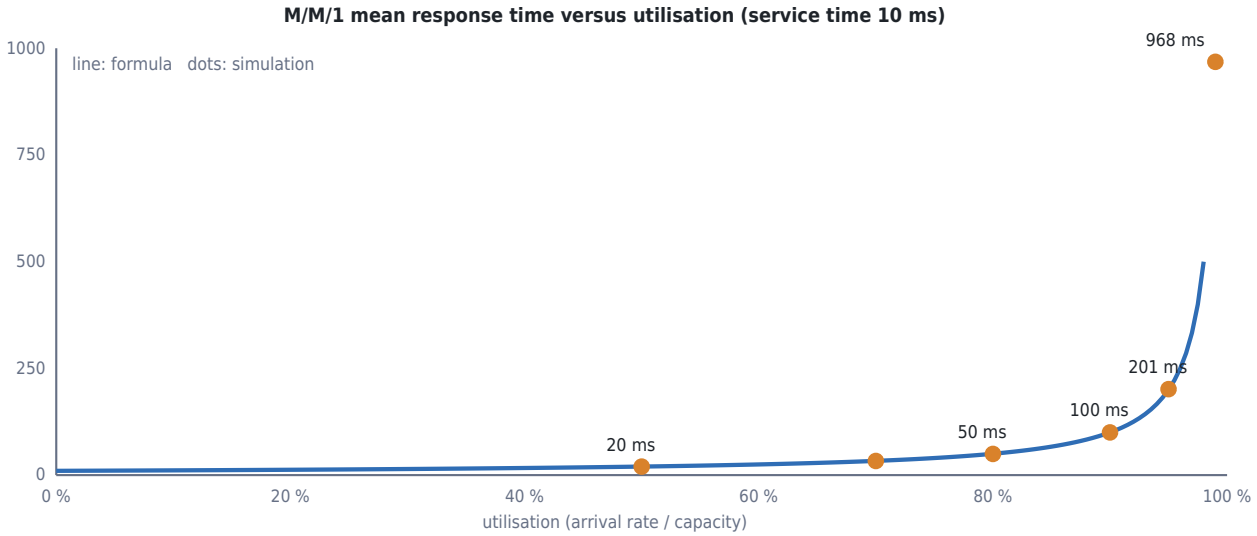


Figure 1.12. Mean response time of an M/M/1 queue with 10 ms service time; the knee begins near 70 to 80 percent utilisation, so capacity planning targets 50 to 70 percent at peak.

Real systems are not M/M/1 (service times have heavier tails, requests arrive in bursts, there are several servers per queue), so the curve is a qualitative guide. The conclusions are robust: a system close to saturation has high and unstable latency, a small increase in load produces a large increase in response time, and capacity must include headroom. Several servers sharing one queue (M/M/c) wait less than the same capacity split into separate queues, which is an argument for balancing requests to the least loaded node.

1.3.2 Worked example

The following computations use only the formulas above, with assumed inputs.

Sizing concurrency with Little's law

A service receives 35 000 requests per second at peak (the page-view counter of the previous chapter) and each request takes 50 ms on average. In-flight requests: L = 35 000 x 0.050 = 1 750. If one node runs 100 concurrent request handlers (assumed), 1 750 / 100 = 17.5, so 18 nodes at full concurrency and 30 nodes at 60 percent target utilisation. If each request holds a database connection for 5 ms, the average number of busy connections across the fleet is 35 000 x 0.005 = 175; a pool of 20 connections per node over 30 nodes (600 connections) is sufficient on average, but the database must accept 600 connections, which is itself a constraint to check.

A converse use: a database client pool of 20 connections and a mean query time of 5 ms cannot sustain more than 20 / 0.005 = 4 000 queries per second, whatever the number of application threads. Raising the thread count without raising the pool only lengthens the queue.

Predicting the benefit of parallelism

A report job takes 60 minutes; 10 percent of it (reading the input, writing the final file) is serial. With 8 workers the time is 60 x (0.10 + 0.90 / 8) = 60 x 0.2125 = 12.75 minutes, a speedup of 4.7 (the table value for p = 0.90, n = 8). With 32 workers: 60 x (0.10 + 0.9 / 32) = 7.7 minutes, a speedup of 7.8. Quadrupling the workers from 8 to 32 saves five minutes; halving the serial part from 10 to 5 percent saves 2.6 minutes at 8 workers (12.75 to 10.1 minutes) without adding any worker.

1.3.3 Hands-on: tail latency and queueing, simulated

Both programs below were run on JDK 21 (single-file launcher). They are single-machine simulations of the formulas, not measurements of any real service.

Tail amplification

Each simulated backend call has a lognormal body (median 10 ms) and a 1 percent chance of an additional stall of 100 to 300 ms. A request issues N calls in parallel and completes with the slowest. 200 000 requests are simulated per N.

TailAmplification.java JAVA

```java
import java.util.Arrays;
import java.util.Locale;
import java.util.Random;

public class TailAmplification {

    // One backend call: lognormal body (median 10 ms) plus a 1 % chance of a 100-300 ms stall.
    static double call(Random r) {
        double ms = Math.exp(Math.log(10) + 0.4 * r.nextGaussian());
        if (r.nextDouble() < 0.01) {
            ms += 100 + 200 * r.nextDouble();
        }
        return ms;
    }

    static double pct(double[] sorted, double p) {
        return sorted[(int) Math.min(sorted.length - 1, Math.ceil(p * sorted.length) - 1)];
    }

    public static void main(String[] args) {
        Random r = new Random(42);
        int trials = 200_000;
        System.out.println("fan-out | p50 ms | p99 ms | p99.9 ms | P(slow call): sim / formula");
        for (int n : new int[] {1, 5, 10, 50, 100, 500}) {
            double[] lat = new double[trials];
            int slow = 0;
            for (int t = 0; t < trials; t++) {
                double max = 0;
                boolean anySlow = false;
                for (int i = 0; i < n; i++) {
                    double c = call(r);
                    if (c > 100) anySlow = true;
                    max = Math.max(max, c);
                }
                lat[t] = max;
                if (anySlow) slow++;
            }
            Arrays.sort(lat);
            System.out.printf(Locale.ROOT, "%7d | %6.1f | %6.1f | %8.1f | %.3f / %.3f%n",
                    n, pct(lat, 0.50), pct(lat, 0.99), pct(lat, 0.999),
                    slow / (double) trials, 1 - Math.pow(0.99, n));
        }
    }
}
```

Run (a seeded generator makes the output reproducible) TERMINAL

```
$ java TailAmplification.java
fan-out | p50 ms | p99 ms | p99.9 ms | P(slow call): sim / formula
      1 |   10.0 |   41.9 |    289.4 | 0.010 / 0.010
      5 |   16.0 |  269.6 |    307.0 | 0.049 / 0.049
     10 |   18.8 |  290.5 |    310.3 | 0.097 / 0.096
     50 |   29.1 |  307.0 |    316.5 | 0.394 / 0.395
    100 |  172.6 |  310.0 |    318.6 | 0.634 / 0.634
    500 |  283.1 |  316.4 |    325.4 | 0.993 / 0.993
```

What to notice: with N = 1 the p99 is 42 ms, yet with N = 10 the p99 is already 290 ms, which is the stall time itself; with N = 100 even the median is 173 ms, because 63 percent of requests contain at least one stalled call; the simulated probability matches 1 - 0.99^N in every row. The backend that is "fast 99 percent of the time" produces a service that is slow most of the time at a fan-out of 100.

Queueing and Amdahl

QueueSim.java JAVA

```java
import java.util.Arrays;
import java.util.Locale;
import java.util.Random;

public class QueueSim {

    public static void main(String[] args) {
        double serviceMs = 10.0;           // mean service time, exponential
        int n = 2_000_000;
        System.out.println("M/M/1 queue, mean service 10 ms (capacity 100 req/s)");
        System.out.println("util | mean ms sim / theory | p99 ms sim / theory | L sim / theory");
        for (double rho : new double[] {0.50, 0.70, 0.80, 0.90, 0.95, 0.99}) {
            Random r = new Random(7);
            double lambda = rho / serviceMs;          // arrivals per ms
            double[] resp = new double[n];
            double wait = 0;                           // Lindley recursion
            double totalResp = 0;
            for (int i = 0; i < n; i++) {
                double service = -serviceMs * Math.log(1 - r.nextDouble());
                double interarrival = -Math.log(1 - r.nextDouble()) / lambda;
                resp[i] = wait + service;
                totalResp += resp[i];
                wait = Math.max(0, wait + service - interarrival);
            }
            Arrays.sort(resp);
            double mean = totalResp / n;
            double p99 = resp[(int) (0.99 * n)];
            double theoryMean = serviceMs / (1 - rho);
            double theoryP99 = theoryMean * Math.log(100);
            // Little's law: average number in system L = lambda * W
            double lSim = lambda * mean;
            double lTheory = rho / (1 - rho);
            System.out.println(String.format(Locale.ROOT,
                    "%.2f | %7.1f / %7.1f      | %7.1f / %7.1f     | %6.2f / %6.2f",
                    rho, mean, theoryMean, p99, theoryP99, lSim, lTheory));
        }
        System.out.println();
        System.out.println("Amdahl: speedup = 1 / ((1 - p) + p / n)");
        System.out.println("parallel fraction p | n=2 | n=8 | n=32 | n=128 | limit 1/(1-p)");
        for (double p : new double[] {0.50, 0.90, 0.95, 0.99}) {
            StringBuilder sb = new StringBuilder(String.format(Locale.ROOT, "%19.2f", p));
            for (int cores : new int[] {2, 8, 32, 128}) {
                sb.append(String.format(Locale.ROOT, " | %5.2f", 1 / ((1 - p) + p / cores)));
            }
            sb.append(String.format(Locale.ROOT, " | %5.1f", 1 / (1 - p)));
            System.out.println(sb);
        }
    }
}
```

Run TERMINAL

```
$ java QueueSim.java
M/M/1 queue, mean service 10 ms (capacity 100 req/s)
util | mean ms sim / theory | p99 ms sim / theory | L sim / theory
0.50 |    19.9 /    20.0      |    90.8 /    92.1     |   0.99 /   1.00
0.70 |    33.1 /    33.3      |   152.4 /   153.5     |   2.32 /   2.33
0.80 |    49.6 /    50.0      |   228.7 /   230.3     |   3.97 /   4.00
0.90 |   100.0 /   100.0      |   477.0 /   460.5     |   9.00 /   9.00
0.95 |   201.4 /   200.0      |  1051.9 /   921.0     |  19.13 /  19.00
0.99 |   968.2 /  1000.0      |  4418.3 /  4605.2     |  95.86 /  99.00

Amdahl: speedup = 1 / ((1 - p) + p / n)
parallel fraction p | n=2 | n=8 | n=32 | n=128 | limit 1/(1-p)
               0.50 |  1.33 |  1.78 |  1.94 |  1.98 |   2.0
               0.90 |  1.82 |  4.71 |  7.80 |  9.34 |  10.0
               0.95 |  1.90 |  5.93 | 12.55 | 17.41 |  20.0
               0.99 |  1.98 |  7.48 | 24.43 | 56.39 | 100.0
```

What to notice: the simulated mean and p99 follow the formulas W = S / (1 - rho) and p99 = W x ln(100) (the response time of an M/M/1 queue is exponentially distributed); at 90 percent utilisation the mean is already 100 ms for a 10 ms service time; the simulated value at 99 percent is below the formula because a finite sample of two million requests

does not fully reach the slowly converging steady state; and the last column confirms Little's law (L = lambda x W) in every row. The Amdahl table reproduces the speedups quoted above.

1.3.4 Real-world problems and solutions

The scenarios below are composite illustrations of failure classes; the figures in them are illustrative and do not report a specific incident.

Problem 1: tail latency from fan-out

Scenario. A product page assembles its content from 40 backend calls (pricing, inventory, reviews, recommendations and others) issued in parallel. Each backend meets its own objective of p99 below 100 ms.

Symptoms. The page p50 is acceptable, but about one page in three exceeds one second; backend dashboards show every individual service healthy.

Diagnosis. Per-service p99 is within objective. The page latency histogram has a long right tail. With 40 calls and a 1 percent slow probability per call, the expected fraction of pages containing a slow call is 1 - 0.99^40 = 33 percent.

Root cause. Latency of a fan-out request is the maximum of its calls, so per-service percentiles understate page percentiles; the objective was applied at the wrong level.

Fix. Set per-call objectives from the page objective (the per-call tail must be much rarer than 1 percent); add a per-call deadline with a degraded default for non-essential calls (render without recommendations); hedge idempotent reads after the p95 latency; reduce the number of calls through aggregation.

Prevention. Define SLOs at the user-visible request level, and alert on end-to-end percentiles, not on per-service averages.

Problem 2: the application tier scales, the database does not

Scenario. Application servers are autoscaled from 10 to 40 nodes during a traffic increase.

Symptoms. Throughput stops increasing at 25 nodes; latency rises; the database reports a large number of waiting connections and CPU near 95 percent; application logs show connection acquisition timeouts.

Diagnosis. The database primary is saturated; each added application node adds connections and load to it, and the connection limit is reached (40 nodes x 30 connections = 1 200 against a limit of 500).

Root cause. The shared stateful tier is the serial fraction of Amdahl's law; adding stateless nodes cannot raise throughput beyond what the database sustains and may lower it through connection overhead.

Fix. Cap total connections with a pooler placed in front of the database, add read replicas for read traffic, cache frequent reads, and batch writes; raise the database capacity or partition it when the single node is the limit.

Prevention. Load-test the whole path, locate the first saturating resource, and compute connection budgets per tier from Little's law before scaling out.

Problem 3: local state blocks scale-in and failover

Scenario. An application stores shopping-cart contents in the memory of the node that created the session; the balancer uses cookie affinity.

Symptoms. Users report empty carts after deployments and after the autoscaler removes nodes at night; one node carries a disproportionate share of long sessions.

Diagnosis. Carts disappear exactly when the owning node is replaced or removed; per-node memory differs by a factor of three.

Root cause. Session data lives in instance memory, so the node cannot be removed without destroying state and affinity distributes load by session count, not by work.

Fix. Move the cart to a replicated shared store keyed by session or user identifier, with a time-to-live; keep nodes stateless.

Prevention. A design review rule: no durable or semi-durable state in an application instance; instances must be disposable.

sreram

Problem 4: averages hide an overloaded service

Scenario. An alert is configured on mean latency above 300 ms; the mean is 120 ms.

Symptoms. Customer complaints about timeouts continue while dashboards are green.

Diagnosis. The p99 latency is 4.5 seconds; 2 percent of requests time out at the client.

Root cause. The mean is dominated by the fast majority and conceals a slow tail caused by queueing at a saturated shard.

Fix. Alert on p99 (and on error and timeout rates), segment latency by shard and endpoint, and add load shedding on the saturated shard.

Prevention. SLIs defined as percentiles over a fixed window (see the availability chapter).

1.3.5 Questions to expect

1. Compare vertical and horizontal scaling.

Vertical scaling enlarges one node: simple, no distribution problems, but bounded, expensive at the top end and a single point of failure. Horizontal scaling adds nodes: higher ceiling, fault tolerant, but requires statelessness, partitioning and coordination. A typical path is vertical first, then stateless horizontal scaling of the application tier, then read replicas and partitioning of the database.

2. What makes a service stateless, and why does it matter?

No request depends on instance-local memory from earlier requests; durable and session state live in shared stores or in client-held tokens. Any instance can handle any request, so instances can be added, removed, restarted and load-balanced without coordination or data loss.

3. Why are percentiles preferred to the average?

The average conceals the tail and mixes populations; percentiles describe what a given fraction of users experience. Users experience many requests per session, so the tail is encountered often. Percentiles must be computed from merged histograms and cannot be averaged across nodes.

4. A request fans out to 50 services, each with p99 of 100 ms. What is the probability the request exceeds 100 ms?

If calls are independent and each exceeds 100 ms with probability 0.01, the request exceeds 100 ms when at least one call does: 1 - 0.99^50 = 39.5 percent. Mitigations are fewer calls, deadlines with partial results, and hedged requests.

5. State Little's law and give a use.

L = lambda x W: the average number in the system equals the arrival rate times the average time in the system. Use: 10 000 requests per second at 100 ms average latency implies 1 000 requests in flight, which sizes thread pools and connection pools.

6. Why does latency rise sharply as utilisation approaches 100 percent?

In a queue, arrivals sometimes bunch together and the server cannot catch up; the waiting time grows as 1 / (1 - utilisation) in the M/M/1 model, so going from 90 to 99 percent utilisation multiplies the mean response time by ten. Capacity plans keep peak utilisation near 50 to 70 percent.

7. What does Amdahl's law imply for scaling a service?

The serial portion (one database primary, a global lock, a sequencer) bounds the benefit of adding workers: with 5 percent serial work the speedup is capped at 20. Effort is better spent shrinking the serial portion (partitioning the data, removing the lock) than adding nodes.

8. What is a hedged request and when is it safe?

A hedged request sends a duplicate of an in-progress request to a second replica after a delay near the p95 latency and uses whichever answer arrives first. It is safe for idempotent reads, adds a small amount of load (about 5 percent when the delay is p95), and sharply reduces tail latency. Non-idempotent writes need an idempotency key first.

sreram

KEY POINTS

- Scale out when the application is stateless; the shared stateful tier then becomes the limit (Amdahl) and is scaled by replicas, caches and partitioning.
- State targets as throughput plus a percentile latency; use merged histograms, never averages of percentiles.
- Fan-out multiplies tail risk: P(slow) = 1 - (1 - q)^N; reduce fan-out, set deadlines, hedge idempotent calls.
- Little's law L = lambda x W sizes concurrency, thread pools and connection pools.
- Queueing delay grows as 1 / (1 - utilisation); plan 50 to 70 percent utilisation at peak.
- Simulations on one machine illustrate formulas; real capacity is established by load testing the real system.

1.4 Load Balancing and Networking for Design

A load balancer distributes incoming work across a pool of servers so that no single server is overloaded, failed servers are bypassed, and capacity can be added or removed without clients noticing. Together with DNS, content delivery networks, reverse proxies and API gateways it forms the network edge of almost every system in this book. The problems it solves are three: capacity (one server cannot serve the load), availability (one failed server must not cause an outage) and operability (servers must be replaceable during deployment without dropping requests).

This chapter covers the layers at which balancing happens, the algorithms and their measured behaviour in a simulation, health checking, session affinity, connection draining, and the surrounding components (DNS, GeoDNS, CDN, anycast, reverse proxy, gateway). The operating-system and TCP details that underlie these components are covered in the computer networks material and are not repeated.

1.4.1 Concept and mechanism

Where balancing happens

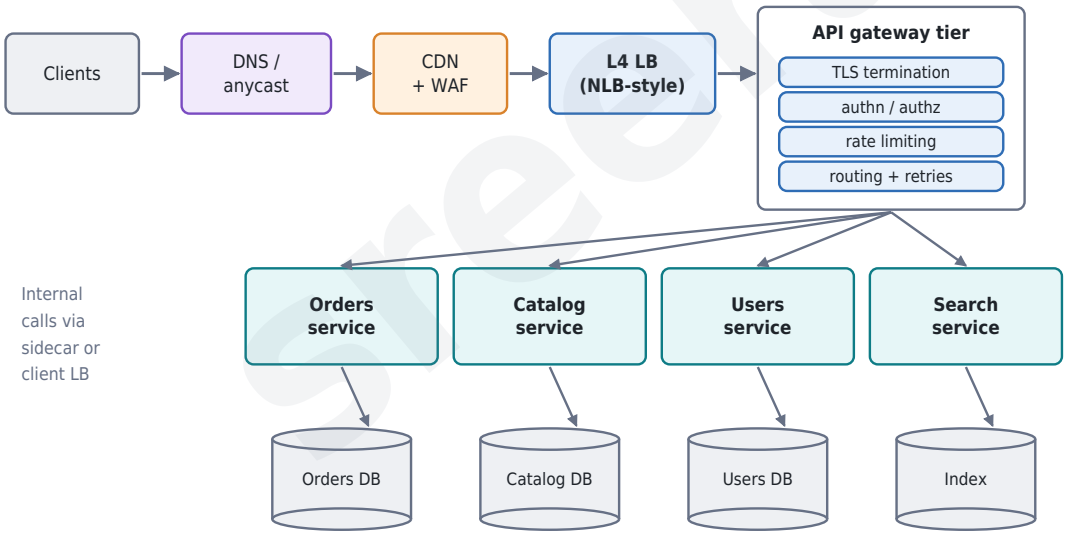


Figure 1.13. A typical request path: DNS or anycast steers the client to an edge; a CDN and web application firewall absorb static and hostile traffic; a layer 4 balancer spreads connections over a gateway tier that authenticates, rate-limits and routes to services, each owning its data.

Balancing occurs at several layers, and large systems use several at once:

- **DNS** returns different addresses for the same name (round robin, weighted, or geography-aware answers).
- **Network edge**: anycast addresses and CDN points of presence terminate client connections near the user.
- **Layer 4 balancer** forwards TCP or UDP connections without interpreting the application protocol.
- **Layer 7 balancer, reverse proxy and API gateway** terminate TLS and HTTP and route by request contents.
- **Client-side or sidecar balancing** places the decision inside the calling service (a client library or a service-mesh proxy), which removes one network hop for internal calls; Spring Cloud LoadBalancer is the Java example covered in the Spring microservices material.

Layer 4 versus layer 7

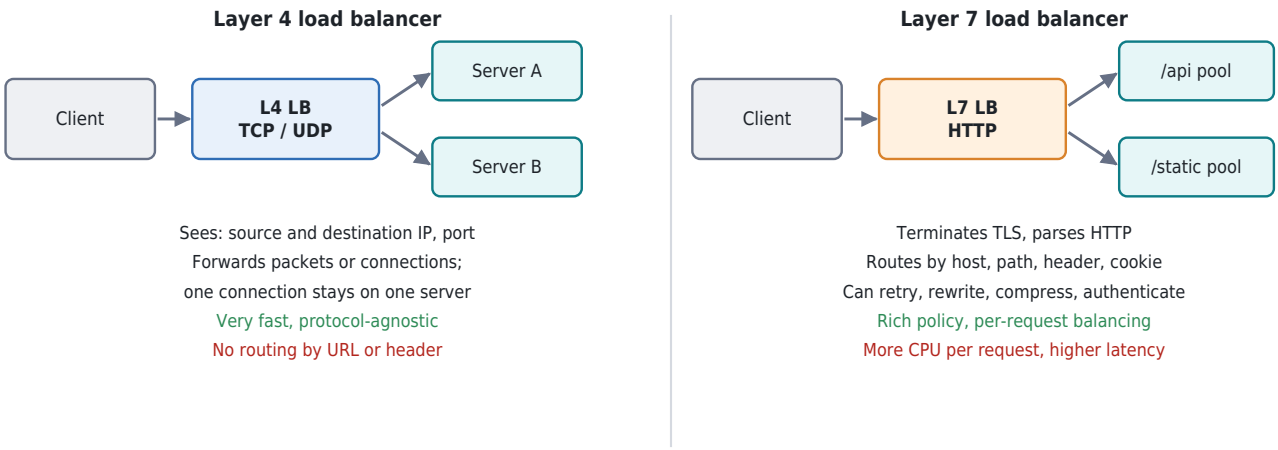


Figure 1.14. A layer 4 balancer decides per connection from addresses and ports; a layer 7 balancer terminates the protocol and decides per request from host, path, headers and cookies.

A layer 4 balancer operates on the transport layer. It selects a backend when a connection starts and then forwards every packet of that connection to it, so one TCP connection always reaches one server. It is fast, handles any protocol on top of TCP or UDP (databases, message brokers, TLS passthrough), and needs little CPU per connection. A layer 7 balancer operates on the application layer. It terminates the client connection, parses HTTP, selects a backend per request and opens or reuses a separate connection to that backend. It can route by hostname or path, terminate TLS, retry idempotent requests, add headers, compress, enforce authentication and balance the many requests that share one long-lived HTTP/2 connection.

Aspect	Layer 4	Layer 7
Decision unit	Connection (addresses, ports)	Request (URL, headers, cookies, method)
Balancing granularity	One backend per connection for its lifetime	One backend per request, even on a shared connection
TLS	Passthrough, or terminated at the balancer by some products	Terminated at the balancer; backends may receive plain or re-encrypted traffic
Typical features	Very high connection rate, low latency, static client address preserved by some products	Path and host routing, retries, rate limiting, authentication, header manipulation, observability per route
Cost	Low CPU	Higher CPU and latency per request
Examples	Cloud network balancers; HAProxy in TCP mode; IPVS	Cloud application balancers; NGINX; Envoy; HAProxy in HTTP mode

Layer 4 compared with layer 7.

The AWS documentation illustrates the distinction. The Network Load Balancer selects a target by a flow hash over protocol, source and destination address and port, and TCP sequence number, and routes each TCP connection to a single target for its lifetime. The Application Load Balancer evaluates listener rules in priority order and then selects a target from the chosen target group, with round robin as the default algorithm (per the AWS Elastic Load Balancing documentation).

The consequence for design: an HTTP/2 or gRPC client typically keeps one long-lived connection and multiplexes many requests on it. A layer 4 balancer sees one connection and therefore sends all the requests of that client to one backend; a layer 7 balancer can spread them. This matters for gRPC services in particular.

Balancing algorithms

Algorithm	Mechanism	Suitable when
Round robin (weighted)	Cycle through backends; weights give larger nodes more turns	Homogeneous backends, short uniform requests
Random	Pick a backend uniformly	Many independent balancers; simple, no shared state
Least connections (least outstanding requests)	Send to the backend with the fewest in-flight requests	Requests of variable duration; heterogeneous or degrading backends
Power of two choices	Sample two backends at random; send to the one with fewer in-flight requests	Many balancers without a global view; near-optimal balancing with O(1) work
Hash on a key (source address, session, URL, user)	The key determines the backend; consistent hashing keeps most assignments stable when the pool changes	Cache affinity, session locality, sharded state
Latency-aware (peak EWMA, least time)	Prefer the backend with lower measured response time	Backends with variable speed; requires measurement

Common algorithms.

Least connections reacts to actual load, because a slow server accumulates in-flight requests and receives fewer new ones. Its weakness is that an accurate count requires a single balancer (or shared state), and a failing server that returns errors instantly appears to have few connections and attracts traffic.

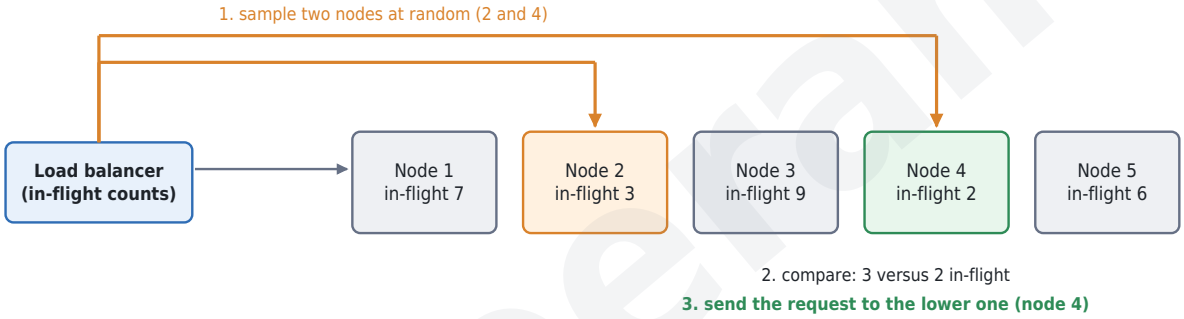


Figure 1.15. Power of two choices: two random candidates are compared and the less loaded one is used; the cost is constant regardless of pool size, and the maximum load is exponentially better than a single random choice (Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing").

Power of two choices (P2C) obtains most of the benefit of least connections without a global scan. Envoy documents this as its default least-request behaviour when host weights are equal: it selects N random available hosts (2 by default) and picks the one with the fewest active requests. NGINX offers the same idea as `random two least_conn`, which randomly picks two servers and then applies `least_conn` between them (both per the respective documentation).

Hash-based balancing maps a key to a backend. With simple modulo hashing, changing the number of backends from N to N+1 moves almost all keys. **Consistent hashing** places both backends and keys on a ring so that a change moves only about 1/(N+1) of the keys, which preserves cache contents and session locality; NGINX documents `hash key consistent` as ketama consistent hashing, and Envoy offers ring hash and Maglev policies. The ring construction, virtual nodes and rebalancing are treated in the chapter on replication, partitioning and consistent hashing; the demonstration in this chapter measures only the remapping effect. A hash policy balances well only if the keys are spread evenly: popular keys concentrate on one backend (see the hot instance problem below).

Health checks

A balancer must stop sending traffic to a failed backend. Two mechanisms are combined. **Passive checks** observe real traffic: after a number of failed requests within a window the backend is marked unavailable for a period (NGINX open source: `max_fails` defaults to 1 and `fail_timeout` to 10 seconds, per its documentation). **Active checks** probe a health endpoint periodically; NGINX documents periodic active health checks as part of its commercial subscription, while HAProxy, Envoy and the cloud balancers provide them natively.

The design of the health endpoint matters more than the mechanism:

- A **liveness** check answers "is the process running". A **readiness** check answers "can this instance serve traffic now" (dependencies connected, caches warmed, migrations finished). Balancers use readiness; restart logic uses liveness. Kubernetes liveness and readiness probes encode the same distinction.
- A **shallow** check (return 200) detects a crashed process only. A **deep** check that queries the database detects a broken dependency but can make every instance appear unhealthy at once when the shared dependency fails, so the balancer removes the whole pool and converts a partial degradation into a total outage. A common compromise is a deep check for dependencies local to the instance and a shallow check for shared ones, with the shared dependency failure handled by the application (degradation, circuit breaker).
- Thresholds trade detection speed for stability: for example mark down after 3 consecutive failures at 5-second intervals (about 15 seconds), and mark up after 2 consecutive successes. Aggressive thresholds cause flapping.

Session affinity (sticky sessions)

Affinity pins a client to one backend, by source address hash, by a cookie inserted by the balancer, or by an application session identifier. It is used when instances hold session state or local caches. The costs are uneven load (heavy users stay on one node), loss of the session when that node fails or is replaced, and difficulty in scaling in. Statelessness (see the previous chapter) is the preferred remedy; affinity remains legitimate for connection-oriented protocols such as WebSockets, where the connection itself is the unit of stickiness, and for cache locality, where a soft preference through consistent hashing is preferable to a hard pin.

Connection draining

Removing a backend must not break requests in flight. **Connection draining** (also called deregistration delay or graceful shutdown) marks the backend as no longer eligible for new requests while it completes the existing ones, up to a timeout, after which it is terminated.

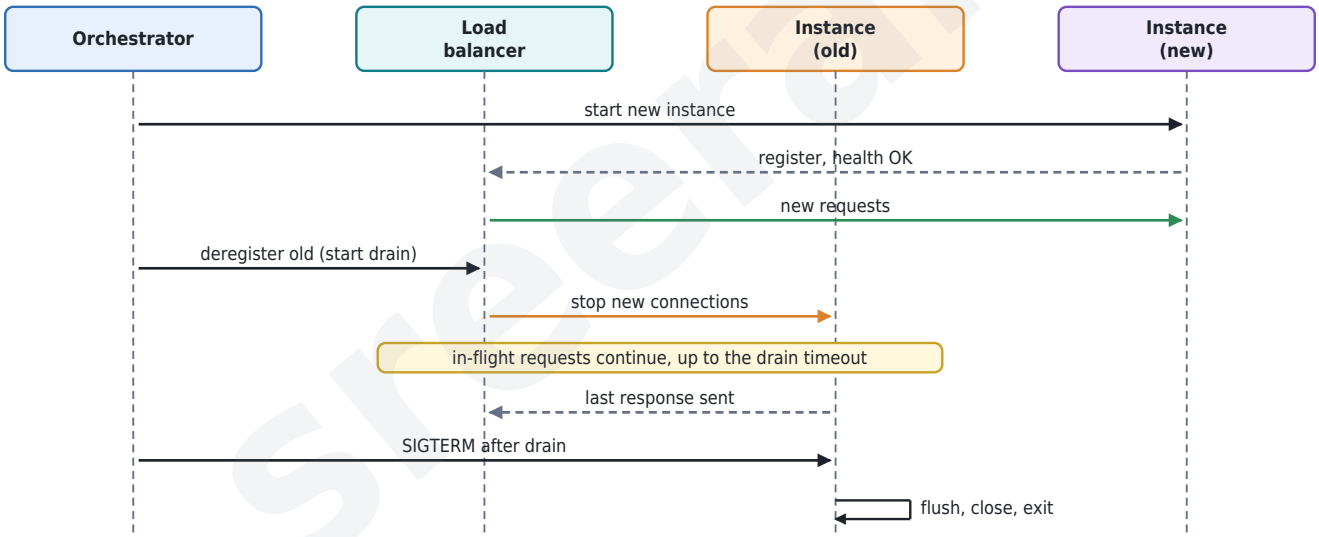


Figure 1.16. Rolling replacement of one instance: the new instance is registered before the old one is deregistered; the old one receives no new connections, completes its in-flight requests, and is stopped only after the drain period.

The sequence has three timing requirements. The drain timeout must exceed the longest normal request (otherwise long requests are cut). The application must handle SIGTERM by stopping its listener and finishing in-flight work, which Spring Boot supports through graceful shutdown (`server.shutdown=graceful`, with `spring.lifecycle.timeout-per-shutdown-phase` for the grace period). The orchestrator must not send SIGKILL before the application has finished, so its termination grace period exceeds drain plus shutdown work.

DNS, GeoDNS, CDN, reverse proxy, gateway and anycast

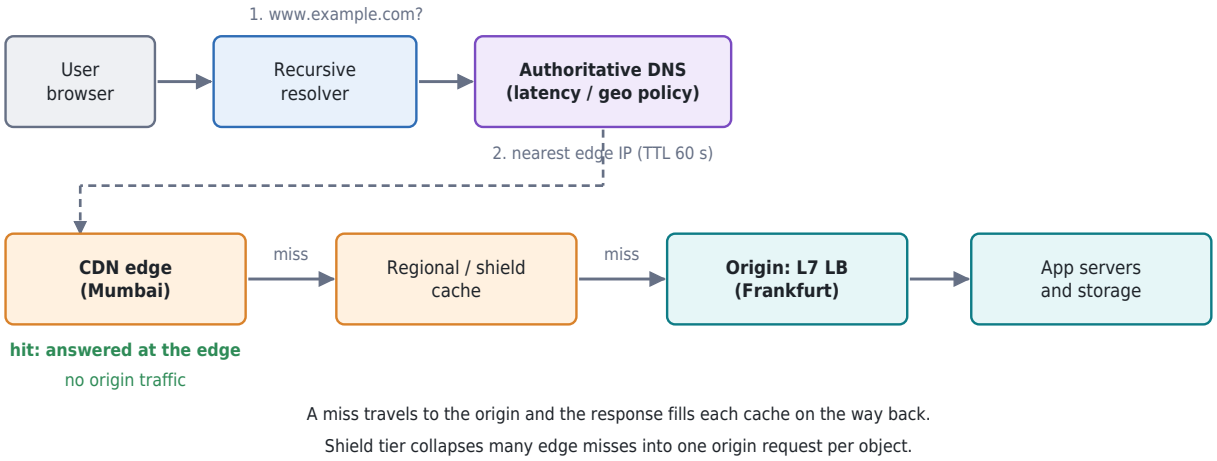


Figure 1.17. Resolution and delivery path: the authoritative DNS server returns the address of the nearest edge; a cache hit at the edge ends the request there; a miss travels through the shield tier to the origin and fills the caches on the way back.

DNS-based balancing returns several addresses for one name, or different addresses by policy. It is the only mechanism that works before a connection exists, and it directs users to a region. Its limits are caching (resolvers and clients keep answers for the record time-to-live, so a failover takes at least one TTL plus the behaviour of non-compliant resolvers), no knowledge of connection load, and coarse granularity. TTLs of 30 to 60 seconds are common for records that must fail over, at the price of more resolver traffic. **GeoDNS** (geolocation or latency-based routing) answers according to the resolver location or, where supported, the client subnet, so that users reach the nearest region; the decision is based on the resolver, which may be far from the user.

Anycast announces the same IP address from several locations through the routing protocol (BGP); packets reach the topologically nearest location. It is used by public DNS services and CDNs. A routing change can move a client to another location in the middle of a TCP connection, so anycast suits short-lived connections and stateless protocols best, or must be combined with connection handling at the edge.

A **CDN** caches content at points of presence near users. Static assets and cacheable API responses are served from the edge, which removes both latency (the round trip to the origin) and origin load; the cache header details belong to the caching chapter. A **shield** or regional tier collapses misses from many edges into one request to the origin.

A **reverse proxy** sits in front of servers and forwards client requests to them, adding TLS termination, compression, caching, connection pooling to backends and request buffering that protects slow application servers from slow clients. An **API gateway** is a reverse proxy with API-management features: authentication and authorisation, rate limiting, request routing and transformation, versioning, and aggregated telemetry. It is the usual single entry point of a microservice system; the corresponding Spring component is Spring Cloud Gateway. A gateway centralises cross-cutting concerns, at the price of becoming a critical component that must itself be redundant and kept free of business logic.

1.4.2 Worked example

A service must handle 35 000 requests per second at peak, with a mean response of 50 ms (the sizing of the previous chapter, 1 750 requests in flight). The edge is designed as follows (all capacity figures assumed).

Tier	Reasoning	Result
Layer 4 balancer	35 K requests per second, assume 3 requests per connection at peak: about 12 K new connections per second; a single managed or software balancer handles this class of load	2 balancers (active-active) for redundancy
Layer 7 gateway	Assume 5 000 requests per second per gateway node with TLS termination and auth at 60 percent target utilisation: 35 000 / 3 000 = 11.7 nodes	12 nodes, plus 2 for zone failure: 14
Application servers	1 750 in flight, 100 concurrent per node at 60 percent: 1 750 / 60	30 nodes across 3 zones (10 per zone)

Tier	Reasoning	Result
Zone failure margin	Losing one zone removes 33 percent of capacity: the remaining 20 nodes must carry 35 000 requests per second at 100 concurrency each = 2 000 in flight (above 1 750)	Sizing is adequate, with no headroom; add 3 to 5 nodes

Edge sizing (assumed capacities).

The last row demonstrates the rule used throughout this book: capacity is checked for the failure of the largest failure domain, not only for the healthy case.

1.4.3 Hands-on: balancing algorithms simulated

The simulation models ten backends. Each backend is a single-server FIFO queue; request service times are lognormal (median 8 ms, mean about 11 ms); arrivals are Poisson. 400 000 requests are simulated per algorithm. Scenario A has ten equal backends at 700 requests per second (the pool capacity is about 900 per second). Scenario B has one backend (node 0) three times slower, at 500 requests per second. Hash-based routing uses 10 000 keys with a Zipf popularity distribution (exponent 1), a realistic shape for popular content, on a ring of 150 virtual nodes per backend. This is a one-process simulation of queueing behaviour, not a benchmark of any real balancer.

LbSim.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Locale;
import java.util.Random;
import java.util.TreeMap;
import java.util.function.IntSupplier;

public class LbSim {

    static final int BACKENDS = 10;
    static final int REQUESTS = 400_000;
    static final int KEYS = 10_000;

    /** One FIFO single-server queue per backend; in-flight = requests not yet completed. */
    static class Backend {
        final double speed;            // service-time multiplier
        double freeAt = 0;
        long served = 0;
        final ArrayDeque<Double> completions = new ArrayDeque<>();
        Backend(double speed) { this.speed = speed; }
        int inFlight(double now) {
            while (!completions.isEmpty() && completions.peekFirst() <= now) {
                completions.pollFirst();
            }
            return completions.size();
        }
        double submit(double now, double service) {
            double start = Math.max(now, freeAt);
            freeAt = start + service * speed;
            completions.addLast(freeAt);
            served++;
            return freeAt - now;       // response time
        }
    }

    static int zipfKey(Random r, double[] cdf) {
        double u = r.nextDouble();
        int lo = 0, hi = cdf.length - 1;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (cdf[mid] < u) lo = mid + 1; else hi = mid;
        }
        return lo;
    }

    static long mix(long x) {
        x ^= x >>> 33; x *= 0xff51afd7ed558ccdL;
        x ^= x >>> 33; x *= 0xc4ceb9fe1a85ec53L;
        x ^= x >>> 33;
        return x;
    }
```

LbSim.java (continued) JAVA

```
    static TreeMap<Long, Integer> ring(int vnodes) {
        TreeMap<Long, Integer> ring = new TreeMap<>();
        for (int b = 0; b < BACKENDS; b++)
            for (int v = 0; v < vnodes; v++) ring.put(mix(b * 1_000_003L + v), b);
        return ring;
    }

    static void run(String scenario, double ratePerSec, double slowFactor) {
        System.out.println("-- " + scenario);
        System.out.println("algorithm              | max/mean load | p50 ms | p99 ms | node 0");
        double[] cdf = new double[KEYS];
        double sum = 0;
        for (int i = 0; i < KEYS; i++) { sum += 1.0 / (i + 1); cdf[i] = sum; }
        for (int i = 0; i < KEYS; i++) cdf[i] /= sum;
        TreeMap<Long, Integer> ring = ring(150);
        for (String algo : new String[] {"round robin", "random", "least connections",
                "power of two choices", "consistent hash (zipf)"}) {
            Random r = new Random(11);
            Backend[] bs = new Backend[BACKENDS];
            for (int i = 0; i < BACKENDS; i++) bs[i] = new Backend(i == 0 ? slowFactor : 1.0);
            double[] resp = new double[REQUESTS];
            double now = 0;
            int rr = 0;
            for (int q = 0; q < REQUESTS; q++) {
                now += -1000.0 / ratePerSec * Math.log(1 - r.nextDouble());       // ms
                // lognormal service time, median 8 ms, mean about 11 ms
                double service = Math.exp(Math.log(8.0) + 0.8 * r.nextGaussian());
                final double t = now;
                int pick;
                switch (algo) {
                    case "round robin" -> pick = rr++ % BACKENDS;
                    case "random" -> pick = r.nextInt(BACKENDS);
                    case "least connections" -> {
                        int start = r.nextInt(BACKENDS);
                        pick = start;
                        for (int k = 1; k < BACKENDS; k++) {
                            int i = (start + k) % BACKENDS;
                            if (bs[i].inFlight(t) < bs[pick].inFlight(t)) pick = i;
                        }
                    }
                    case "power of two choices" -> {
                        int a = r.nextInt(BACKENDS), b = r.nextInt(BACKENDS);
                        pick = bs[a].inFlight(t) <= bs[b].inFlight(t) ? a : b;
                    }
                    default -> {
                        long h = mix(zipfKey(r, cdf) * 7919L + 13);
                        var e = ring.ceilingEntry(h);
                        pick = (e != null ? e : ring.firstEntry()).getValue();
                    }
                }
                resp[q] = bs[pick].submit(t, service);
            }
            long max = 0, tot = 0;
            for (Backend b : bs) { max = Math.max(max, b.served); tot += b.served; }
            if (algo.startsWith("consistent")) {
                StringBuilder sb = new StringBuilder("   per-node share of requests (%):");
                for (Backend b : bs) {
                    sb.append(String.format(Locale.ROOT, " %.1f", 100.0 * b.served / tot));
                }
                System.out.println(sb);
            }
            Arrays.sort(resp);
            System.out.printf(Locale.ROOT, "%-22s | %13.2f | %6.1f | %6.1f | %5.1f%%%n",
                    algo, max / (tot / (double) BACKENDS), resp[REQUESTS / 2],
                    resp[(int) (0.99 * REQUESTS)], 100.0 * bs[0].served / tot);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        run("A: 10 equal nodes, 700 req/s (capacity about 900 req/s)", 700, 1.0);
        run("B: node 0 runs 3x slower, 500 req/s", 500, 3.0);
    }
}
```

sroram

Run (seeded, reproducible; per-node shares are printed before the hash row) TERMINAL

```
$ java LbSim.java
-- A: 10 equal nodes, 700 req/s (capacity about 900 req/s)
algorithm             | max/mean load | p50 ms | p99 ms | node 0
round robin           |          1.00 |   16.7 |  143.2 |  10.0%
random                |          1.01 |   30.5 |  222.6 |  10.0%
least connections     |          1.00 |    9.9 |   64.0 |  10.0%
power of two choices  |          1.01 |   15.6 |   88.2 |  10.0%
   per-node share of requests (%): 19.7 6.2 7.6 7.4 17.2 5.8 10.7 6.5 8.6 10.2
consistent hash (zipf) |          1.97 |   51.0 | 276761.6 |  19.7%

-- B: node 0 runs 3x slower, 500 req/s
algorithm             | max/mean load | p50 ms | p99 ms | node 0
round robin           |          1.00 |   11.6 | 468888.2 |  10.0%
random                |          1.01 |   18.2 | 461624.6 |  10.0%
least connections     |          1.12 |    8.8 |   69.0 |   4.8%
power of two choices  |          1.06 |   12.0 |  112.4 |   5.4%
   per-node share of requests (%): 19.7 6.2 7.6 7.4 17.2 5.8 10.7 6.5 8.6 10.2
consistent hash (zipf) |          1.97 |   26.7 | 1698969.5 |  19.7%
```

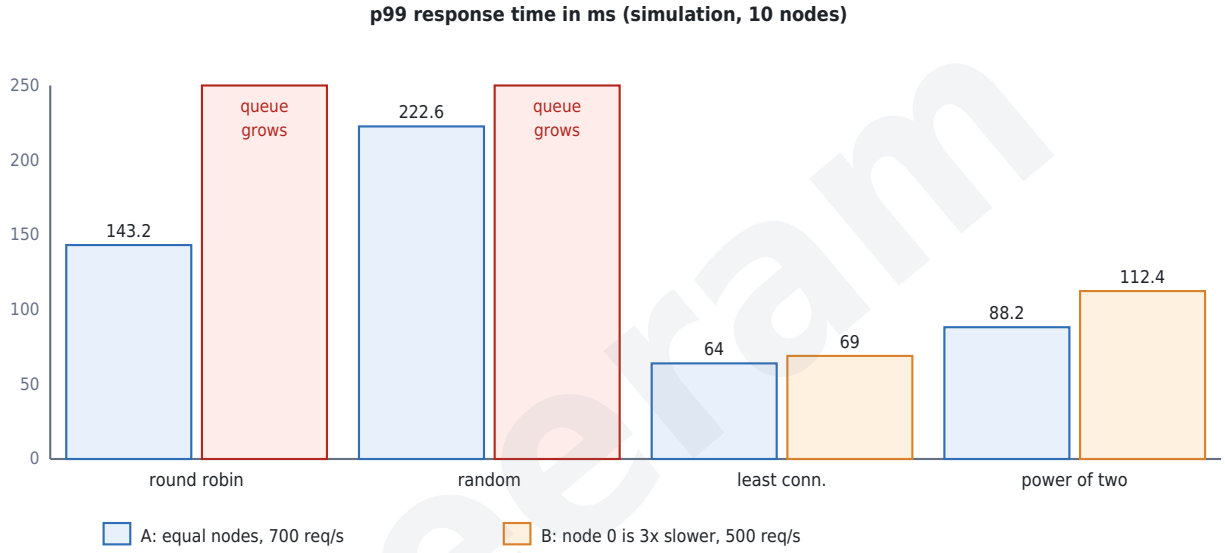


Figure 1.18. The p99 response times of the four load-based algorithms in both scenarios; round robin and random cannot adapt to the slow node, whose queue grows for the whole run.

What to notice:

- In scenario A, all four load-based algorithms spread requests evenly (node 0 receives 10 percent), but the p99 differs: 64 ms for least connections, 88 ms for power of two choices, 143 ms for round robin and 223 ms for random. Random placement creates transient imbalance; round robin removes count imbalance but ignores the variable duration of requests.
- In scenario B, round robin and random send 10 percent of the traffic (50 requests per second) to a node whose capacity is about 30 per second, so its queue grows without bound; the p99 in the output (hundreds of seconds) is a property of the run length and only shows that the system is unstable. Least connections reduces the share of the slow node to 4.8 percent and power of two choices to 5.4 percent, and both keep the p99 near 70 to 110 ms.
- The key-hash policy concentrates traffic: with Zipf-skewed keys two nodes receive 19.7 and 17.2 percent of the requests, the busiest node receives 1.97 times the mean, and the p99 explodes. Hash routing is appropriate for cache or state locality, not as a general-purpose balancing algorithm for skewed keys.

sreeram

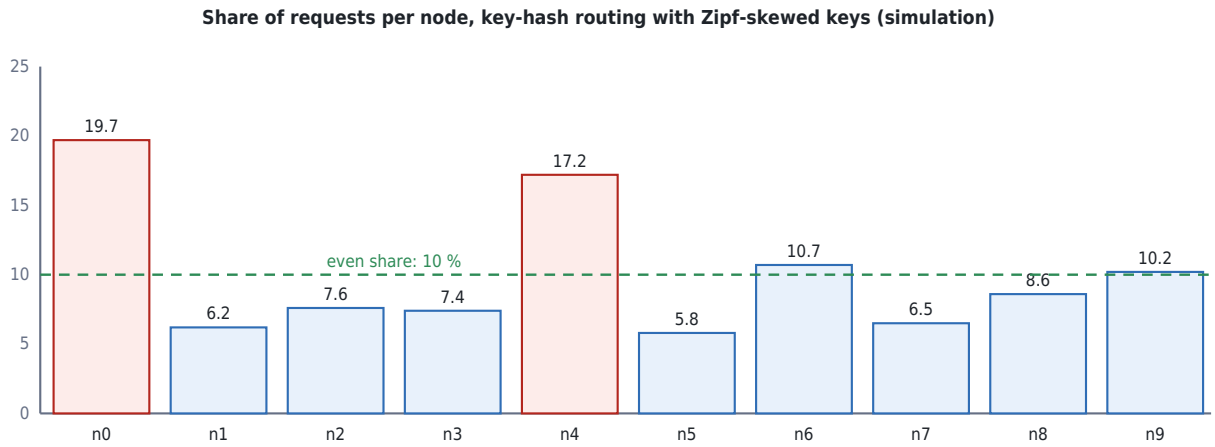


Figure 1.19. Per-node request share under key-hash routing with skewed keys: two nodes carry 19.7 and 17.2 percent of the traffic against an even share of 10 percent.

The second demonstration measures the remapping effect that justifies consistent hashing: 200 000 keys are assigned to N nodes, a node is added, and the share of keys that change owner is counted.

RemapDemo.java JAVA

```java
import java.util.Locale;
import java.util.TreeMap;

public class RemapDemo {

    static long mix(long x) {
        x ^= x >>> 33; x *= 0xff51afd7ed558ccdL;
        x ^= x >>> 33; x *= 0xc4ceb9fe1a85ec53L;
        x ^= x >>> 33;
        return x;
    }

    static TreeMap<Long, Integer> ring(int nodes, int vnodes) {
        TreeMap<Long, Integer> ring = new TreeMap<>();
        for (int n = 0; n < nodes; n++) {
            for (int v = 0; v < vnodes; v++) ring.put(mix(n * 1_000_003L + v), n);
        }
        return ring;
    }

    static int owner(TreeMap<Long, Integer> ring, long key) {
        var e = ring.ceilingEntry(mix(key * 7919L + 13));
        return (e != null ? e : ring.firstEntry()).getValue();
    }

    public static void main(String[] args) {
        int keys = 200_000;
        for (int n : new int[] {5, 10, 20}) {
            int movedMod = 0, movedRing = 0;
            var before = ring(n, 150);
            var after = ring(n + 1, 150);
            for (long k = 0; k < keys; k++) {
                if (Math.floorMod(mix(k), n) != Math.floorMod(mix(k), n + 1)) movedMod++;
                if (owner(before, k) != owner(after, k)) movedRing++;
            }
            String fmt = "%2d -> %2d nodes: mod N %5.1f%%, ring %5.1f%%, ideal %.1f%%%n";
            System.out.printf(Locale.ROOT, fmt,
                    n, n + 1, 100.0 * movedMod / keys, 100.0 * movedRing / keys, 100.0 / (n + 1));
        }
    }
}
```

Run TERMINAL

```
$ java RemapDemo.java
 5 ->  6 nodes: mod N  83.5%, ring  16.8%, ideal 16.7%
10 -> 11 nodes: mod N  90.9%, ring   9.0%, ideal 9.1%
20 -> 21 nodes: mod N  95.3%, ring   4.8%, ideal 4.8%
```

sreeram

Modulo hashing moves 84 to 95 percent of the keys when one node is added, while the ring moves almost exactly the ideal fraction 1/(N+1). For a cache tier this is the difference between losing a sixth of the cached data and losing nearly all of it, which would send the full read load to the database.

1.4.4 Real-world problems and solutions

The scenarios below are composite illustrations of failure classes; the figures in them are illustrative and do not report a specific incident.

Problem 1: sticky-session imbalance

Scenario. A web application uses cookie-based affinity with 6 nodes. Users stay logged in for hours; a small fraction are power users with many requests.

Symptoms. One node runs at 90 percent CPU while the others are at 35 percent; p99 latency is high for the users pinned to that node; the autoscaler adds nodes, but the new nodes receive few sessions.

Diagnosis. Per-node request counts diverge; the busy node holds the longest-lived and heaviest sessions; new nodes only receive newly created sessions.

Root cause. Affinity assigns sessions, not work, at login time and never rebalances; session length and request rate vary by orders of magnitude.

Fix. Externalise session state so that any node can serve any request and disable affinity; if affinity is required for caches, use consistent hashing with a bounded load (a node above a threshold sheds to the next one); cap session lifetime so that sessions redistribute.

Prevention. Alert on the ratio of the maximum to the mean per-node load; require a design justification for any hard affinity.

Problem 2: an unhealthy node still receives traffic

Scenario. One of eight nodes loses its connection to the database but its process keeps running; the health endpoint returns 200 because it checks only the process.

Symptoms. About one request in eight fails with a 500 error; the error rate persists for hours; the node's CPU is low (it fails quickly).

Diagnosis. Errors cluster on one instance in logs and traces; the balancer shows all targets healthy.

Root cause. The health check does not reflect the ability to serve; additionally, with least connections the node attracts more traffic than its share because failed requests finish fast and keep its in-flight count low.

Fix. Use a readiness check that verifies the dependencies the instance owns; add passive outlier detection that ejects a node whose error rate exceeds a threshold (with a cap on the ejected fraction so that a global failure does not eject everything); prefer algorithms that account for errors, such as peak EWMA with failure penalties.

Prevention. Chaos tests that break one instance's dependency and verify its removal within the intended time.

Problem 3: hot instance under skewed keys

Scenario. A cache tier routes by hash of the key. One celebrity profile receives 8 percent of all reads.

Symptoms. One cache node saturates its network interface and CPU; cache latency rises for all keys on that node; the hit rate of the pool appears normal.

Diagnosis. Per-node request rate shows one node at roughly two times the mean (as in the simulation above: 1.97); a key-level top-N report identifies a single key.

Root cause. Hash routing assigns every request for a key to one node, so popularity skew becomes load skew; adding nodes does not divide a single key.

Fix. Replicate hot keys across several nodes and read from a random replica; add a small local in-process cache with a short TTL in front for the hottest keys; split the key (append a random suffix on write to several sub-keys, read from any).

Prevention. Monitor top-key frequency; load-test with skewed key distributions, not only uniform ones.

sreram

Problem 4: thundering herd after restart

Scenario. After a deployment or an incident, every node of a service restarts at about the same time. Caches are cold, connection pools are empty and the JVM has not yet compiled hot paths.

Symptoms. Immediately after the restart the first nodes marked healthy receive their full share, or more (a least-connections balancer favours a node with zero connections); latency and errors spike; nodes crash again and the cycle repeats.

Diagnosis. Error rate correlates with node start time; the database sees a burst of identical queries and connection attempts.

Root cause. Newly started nodes receive full traffic instantly although their effective capacity is a fraction of normal, and cold caches turn every request into a database query.

Fix. Ramp traffic up gradually (slow start: the balancer raises the weight of a new node over a period; NGINX documents `slow_start`, noting it as a commercial feature; Envoy and cloud balancers offer comparable options), warm up caches and connections before reporting ready, stagger restarts (rolling updates with a small surge), and add jitter to client reconnect and refresh timers.

Prevention. Rolling deployments with readiness gates and a maximum unavailable percentage; load-test the cold-start path.

Problem 5: the balancer as a single point of failure

Scenario. All traffic enters through one load balancer instance.

Symptoms. A host failure, a software upgrade or a saturated network interface takes the whole service offline.

Diagnosis. The balancer appears at the root of every dependency graph; no standby exists or failover is manual.

Root cause. Redundancy was applied to the servers but not to the component that fronts them.

Fix. Run at least two balancers in active-active (with DNS or anycast spreading clients) or active-passive with a floating address and a health-checked failover; use a managed balancer that is itself distributed across zones.

Prevention. A design checklist of every component on the request path, each with its failure behaviour (see the chapter on availability and reliability).

1.4.5 Questions to expect

1. What is the difference between a layer 4 and a layer 7 load balancer?

Layer 4 balances connections using addresses and ports, is fast and protocol-agnostic, and pins a connection to one backend. Layer 7 terminates the application protocol and balances individual requests by URL, header or cookie, enabling path routing, retries, TLS termination and authentication, at higher cost. For multiplexed protocols such as HTTP/2 and gRPC, layer 7 is needed to spread the requests of one connection.

2. Which algorithm is selected for requests of very different duration?

Least connections (least outstanding requests) or power of two choices, because they react to actual load. Round robin ignores the duration and lets slow requests accumulate on a node. P2C obtains nearly the same result as least connections without needing a global view across multiple balancers.

3. Why does consistent hashing matter for a cache tier?

Modulo hashing remaps almost all keys when the node count changes (84 to 95 percent in the measurement above), flushing the effective cache and flooding the database. Consistent hashing remaps about 1/(N+1) of the keys.

4. How are failed servers detected, and what are the pitfalls?

By passive checks (observed errors) and active probes of a health endpoint. Pitfalls: a shallow check that misses broken dependencies; a deep check that removes the whole pool when a shared dependency fails; flapping from aggressive thresholds; and no cap on ejection.

5. When are sticky sessions acceptable?

For connection-oriented protocols (WebSockets), and for cache locality where a soft preference suffices. For HTTP sessions, externalising state is preferable because affinity causes imbalance and loses sessions on failure.

sreeram

6. What happens to in-flight requests when an instance is removed during deployment?

With connection draining, the balancer stops sending new requests, the instance completes in-flight work within a timeout, and only then terminates. Without it, in-flight requests are cut and clients see errors. The application must handle SIGTERM gracefully and the grace period must exceed the longest request.

7. How does DNS-based failover behave, and why is it slow?

The authoritative server stops returning the failed address, but resolvers and clients cache the old answer for the TTL, and some ignore low TTLs. Failover therefore takes at least the TTL, often longer. Faster failover uses anycast, a floating address or a balancer in front, with DNS used for regional steering.

8. What is the role of an API gateway, and what are its risks?

It centralises authentication, rate limiting, routing, request transformation and telemetry at the entry point. Risks: it becomes a critical, high-traffic component requiring redundancy; it may accumulate business logic; and it adds a network hop. It is kept thin and stateless.

KEY POINTS

- Layer 4 balances connections quickly; layer 7 balances requests with rich policy; multiplexed protocols need layer 7 to spread load.
- Least connections and power of two choices adapt to variable work and slow nodes (simulated p99 of 64 and 88 ms against 143 and 223 ms for round robin and random); hash routing serves locality, not even balance.
- Consistent hashing remaps about 1/(N+1) of keys on a pool change; modulo hashing remaps 84 to 95 percent in the measurement.
- Health checks must reflect the ability to serve without letting a shared dependency empty the pool; draining and graceful shutdown protect in-flight requests.
- DNS failover is bounded by the TTL; anycast and floating addresses fail over faster; every component on the request path, including the balancer, needs redundancy.
- Capacity is verified for the loss of a whole failure domain, not only for the healthy case.

1.5 Availability and Reliability

A system that is fast but frequently unavailable has no value to its users, and a system that is available but returns wrong results has negative value. **Availability** is the fraction of time (or of requests) for which a service performs its function correctly. **Reliability** is the probability that the service performs correctly for a stated period under stated conditions; it includes correctness and durability as well as uptime. Designs achieve both by assuming that every component fails, in every possible way, and by arranging redundancy, isolation and graceful behaviour so that individual failures do not become user-visible outages.

This chapter defines the vocabulary of service-level objectives, derives the arithmetic of redundancy, describes failure domains and failover, summarises the defensive techniques that Spring and the resilience material treat in code (timeouts, retries, circuit breakers), and adds the two mechanisms that protect a service from overload: backpressure and load shedding. Two simulations quantify the arithmetic of series and parallel components and the amplification caused by retries.

1.5.1 Concept and mechanism

SLI, SLO and SLA

The Google SRE book defines the terms as follows (chapter "Service Level Objectives"). A **service level indicator (SLI)** is a carefully defined quantitative measure of some aspect of the level of service that is provided. A **service level objective (SLO)** is a target value or range of values for a service level that is measured by an SLI. A **service level agreement (SLA)** is an explicit or implicit contract with users that includes consequences of meeting or missing the SLOs it contains, often financial. The relation is: SLIs are measured, SLOs are targets set internally (stricter than the SLA), and SLAs are commitments with penalties.

Typical SLIs are the fraction of requests that succeed (availability SLI), the fraction served faster than a threshold (latency SLI, for example "99 percent of requests complete in under 300 ms"), and for data systems the fraction of data that is fresh or durable. Measuring availability as the ratio of good requests to total requests over a window is more informative than measuring server uptime, because a server can be "up" and still fail every request.

An **error budget** is the allowed unreliability, 1 minus the SLO. An SLO of 99.9 percent over 30 days permits 0.1 percent of requests (or 43.2 minutes of full outage) to fail. The SRE book recommends tracking the budget daily or weekly and letting its consumption inform release decisions: while budget remains, the team ships features; when it is exhausted, the team prioritises reliability work. The mechanism turns reliability from a feeling into a quantity that product and engineering share.

The nines

Availability	Per year	Per 30-day month	Per day	Typical meaning
99 percent	3.65 days	7.2 hours	14.4 minutes	Two nines: internal tools, batch systems
99.9 percent	8.76 hours	43.2 minutes	86.4 seconds	Three nines: common target for business services
99.95 percent	4.38 hours	21.6 minutes	43.2 seconds	Between three and four nines
99.99 percent	52.6 minutes	4.32 minutes	8.64 seconds	Four nines: requires automated failover
99.999 percent	5.26 minutes	25.9 seconds	0.86 seconds	Five nines: requires multi-region design and no manual step

Allowed downtime for each availability level (365-day year, 30-day month).

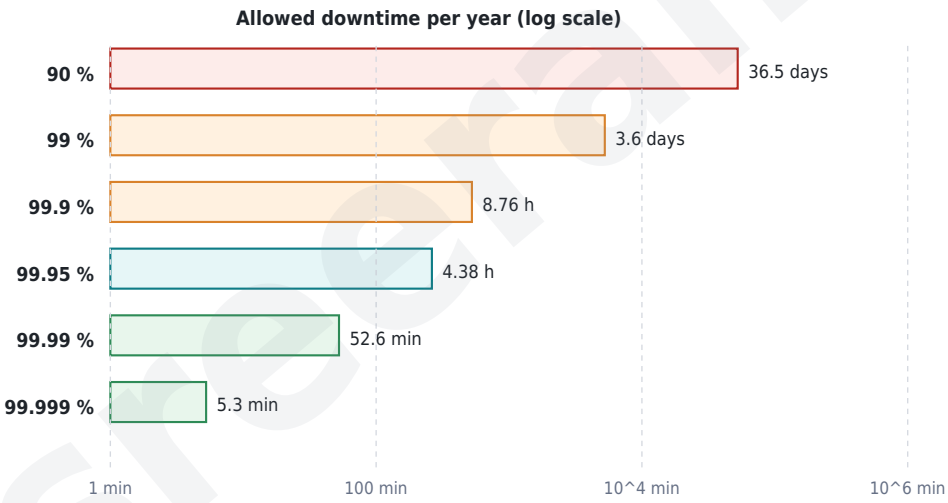


Figure 1.20. Allowed downtime per year on a logarithmic axis; each added nine divides the budget by ten and removes the possibility of any manual recovery step.

Each additional nine shortens the budget tenfold. Between three and four nines the practical difference is that human detection and response (several minutes at best) no longer fit in the budget, so detection, failover and rollback must be automated. Availability promised to customers should be lower than what the architecture can deliver: the measurable availability of a service never exceeds the product of the availabilities of the components it depends on in series.

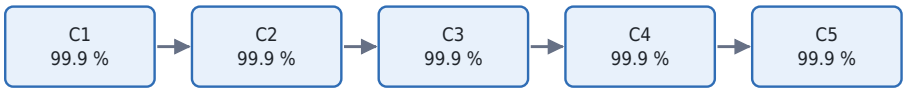
The arithmetic of availability

Availability of a repairable component follows from its mean time between failures (MTBF) and mean time to repair (MTTR): A = MTBF / (MTBF + MTTR). A component that fails once a month (MTBF about 720 hours) and takes 30 minutes to recover is available 720 / 720.5 = 99.93 percent of the time. The formula shows two levers: fail less often, or recover faster; for most systems shortening recovery time is cheaper than preventing every failure.

Components combine in two basic ways.

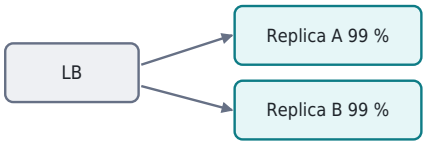
- **Series** (every component is needed): A = A1 x A2 x ... x An. Availability is lower than that of the weakest component, and falls with each added dependency. Five components of 99.9 percent give 0.999^5 = 99.50 percent (about 43.7 hours of downtime per year); ten give 99.0 percent.
- **Parallel** (any one component suffices): A = 1 - (1 - A1)(1 - A2)...(1 - An). Two independent replicas of 99 percent each give 1 - 0.01^2 = 99.99 percent, three give 99.9999 percent.

Series: every component is needed



0.999^5 = 0.99501 -> 99.50 %, about 43.7 hours of downtime per year

Parallel: any one replica suffices (independent failures)



1 - (1 - 0.99)^2 = 0.9999

99.99 %: 52.6 minutes per year

Common-cause failure (shared zone, shared database, shared deploy) caps the parallel gain: two replicas in one zone with 99.9 % availability cannot exceed 99.9 % overall.

Figure 1.21. Series chains multiply availabilities and lose; independent parallel replicas multiply unavailabilities and gain; shared causes break the independence assumption and cap the gain.

The parallel formula assumes independent failures. Real replicas share causes: a power feed, a network switch, a software version, a configuration push, a dependency. If both replicas sit in one zone with availability 99.9 percent, the combined availability cannot exceed 99.9 percent however many replicas exist. This is the argument for spreading replicas across independent failure domains, and for diversity in deployment (staged rollout) so that one bad change does not hit every replica simultaneously. A further condition on the parallel formula is capacity: if the survivors cannot carry the full load, the replicas are not truly redundant. The standard notation is N+1 (one spare) or N+2.

Failure domains

A **failure domain** is a set of components that can fail together because they share a dependency. Typical nesting, from small to large: process, host, rack (power and top-of-rack switch), availability zone (independent power, cooling and network within a region), region, and provider. Larger domains fail more rarely but with larger impact; a design chooses the largest domain whose loss it will tolerate automatically, and documents the rest as accepted risk. Most services target zone-level tolerance by default and add region-level tolerance only when the SLO or the business impact justifies the cost and complexity of multi-region data replication (see the chapters on replication and on multi-region designs).

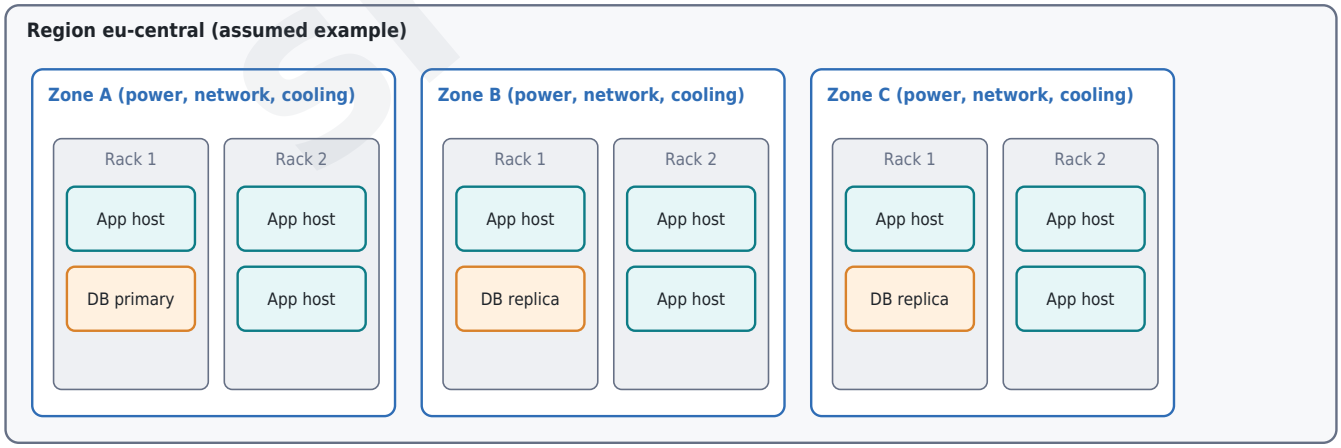


Figure 1.22. Application hosts are spread over three zones and two racks per zone, and the database has a primary and replicas in different zones; the loss of any single host, rack or zone leaves the capacity needed (if sized N+1 for the largest domain) to continue.

Failover: active-passive and active-active

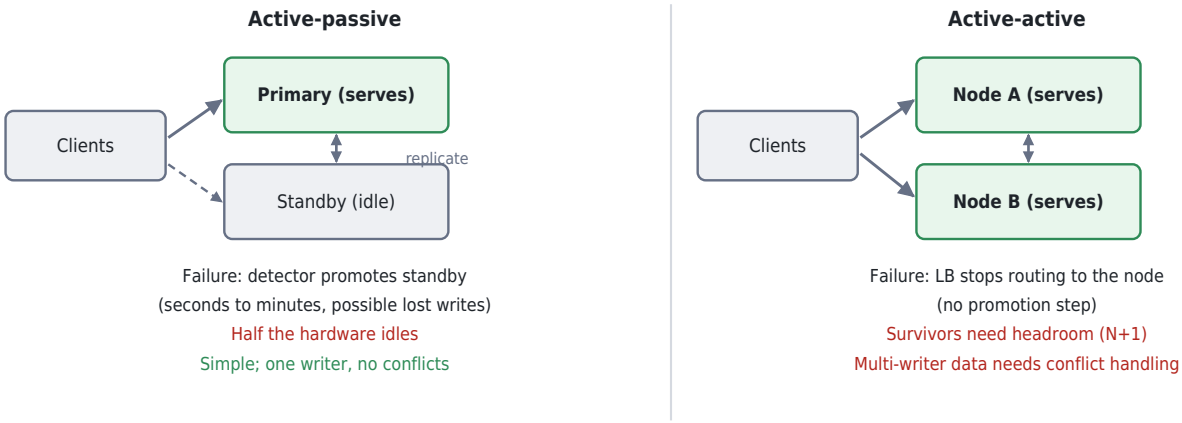


Figure 1.23. Active-passive keeps a standby that is promoted on failure; active-active serves from all nodes and removes the failed node from rotation.

In **active-passive** (hot standby) one node serves and a standby replicates its state; a detector promotes the standby on failure. It is simple and suits systems with one writer (a relational primary). Its costs are idle capacity, a detection-and-promotion delay (seconds to minutes) and the risk that asynchronous replication loses recently acknowledged writes (the databases material covers synchronous and asynchronous replication and the RPO consequence). In **active-active** all nodes serve traffic simultaneously; a failure removes a node from the balancer's pool, which is fast and needs no promotion. It requires that the survivors hold spare capacity and that shared data be safe to write from several places: stateless tiers are trivially active-active, while multi-writer databases need conflict resolution.

Two further properties decide whether failover works. Detection must be reliable and fast (a health check interval and failure threshold of 5 s x 3 gives 15 s), and false positives must be rare because an unnecessary failover is itself an outage. The standby path must be exercised regularly: an untested failover is a hypothesis, not a capability. A risk specific to failover is split brain, where two nodes both believe themselves primary; the fencing and leader-election techniques that prevent it belong to the chapter on consistency and coordination.

Graceful degradation

When a dependency fails or load exceeds capacity, a service can return a reduced result instead of an error: serve cached or stale data, hide the recommendations panel, accept writes into a queue and confirm later, or fall back to a default. Degradation must be designed: each feature is classified as essential (checkout payment) or optional (recommendations), optional features are isolated behind timeouts and breakers, and the degraded behaviour is tested. The benefit is that partial failure of an optional component does not consume the availability budget of the essential function.

Timeouts, retries, circuit breakers and idempotency

These techniques are implemented in Spring with Resilience4j and covered in the Spring microservices resilience material; the design-level rules are summarised here.

- **Timeouts**: every remote call has a timeout, derived from the downstream p99 plus a margin and bounded by the caller's own deadline. A call without a timeout can hold a thread indefinitely. Timeouts must decrease along the call chain (the client waits 2 s, the gateway 1.5 s, the service 1 s), otherwise a caller abandons a request while downstream work continues, wasting capacity. Propagating a deadline with the request removes the guesswork.
- **Retries** convert transient failures into successes, but only for idempotent operations and only with exponential backoff and random jitter, a limited number of attempts, and a retry budget (for example, retries may add at most 10 percent to the request volume). Unbounded or synchronised retries cause the retry storm demonstrated below.
- **Circuit breakers** stop calling a dependency that is failing so that callers fail fast and the dependency can recover.

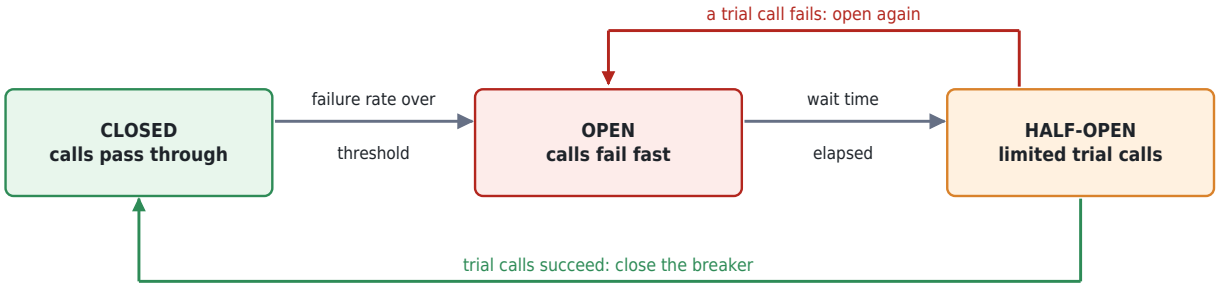


Figure 1.24. A circuit breaker has three states: closed (calls pass), open (calls fail immediately for a wait period), and half-open (a few trial calls decide whether to close or reopen).

- **Idempotency** makes a repeated request equivalent to a single one. Reads and PUT-style replacements are naturally idempotent; creating an order or charging a card is not, and needs an idempotency key: the client generates a unique key per logical operation, the server stores the key with the result, and a repeated request returns the stored result instead of repeating the side effect. Idempotency is what makes retries, queue redelivery and client re-submission safe.

Backpressure and load shedding

When offered load exceeds capacity, the work that cannot be served must be refused somewhere. **Backpressure** pushes the signal upstream so that producers slow down: a bounded queue that blocks or rejects, a credit-based protocol, HTTP 429 or 503 with a Retry-After header, TCP flow control. **Load shedding** deliberately drops excess work, preferably the least valuable first (batch before interactive, anonymous before authenticated, retries before first attempts), so that the work that is accepted completes within its deadline.



Rules: reject early (cheap) rather than fail late (expensive); prefer low-priority traffic as the first to be shed; bound queue length so waiting time stays below the client timeout; propagate pressure upstream (HTTP 429, gRPC RESOURCE_EXHAUSTED) so callers back off.

Figure 1.25. Admission control rejects excess requests immediately with 429 or 503; a bounded queue prevents waiting beyond the client timeout; accepted work completes within its deadline.

Queue bounds follow from Little's law. A service that completes 1 000 requests per second and whose clients time out after 1 second can usefully hold at most about 1 000 requests in its queue: a request admitted behind more than 1 000 others waits longer than the client will, and is served after the client has given up, which is wasted work that also delays later requests. An unbounded queue therefore converts overload into universal timeout, while a bounded queue with rejection preserves the throughput of the accepted share.

Retry storms and cascading failures

A **retry storm** occurs when failures trigger retries, retries add load, and the added load causes more failures. With layered retries the amplification multiplies: if three layers each make up to three attempts, one user request can produce 3 x 3 x 3 = 27 calls to the innermost dependency while it is failing, precisely when it is least able to serve them.

A **cascading failure** is a failure that spreads: the loss of a node shifts its load onto the survivors, which become overloaded and fail in turn.

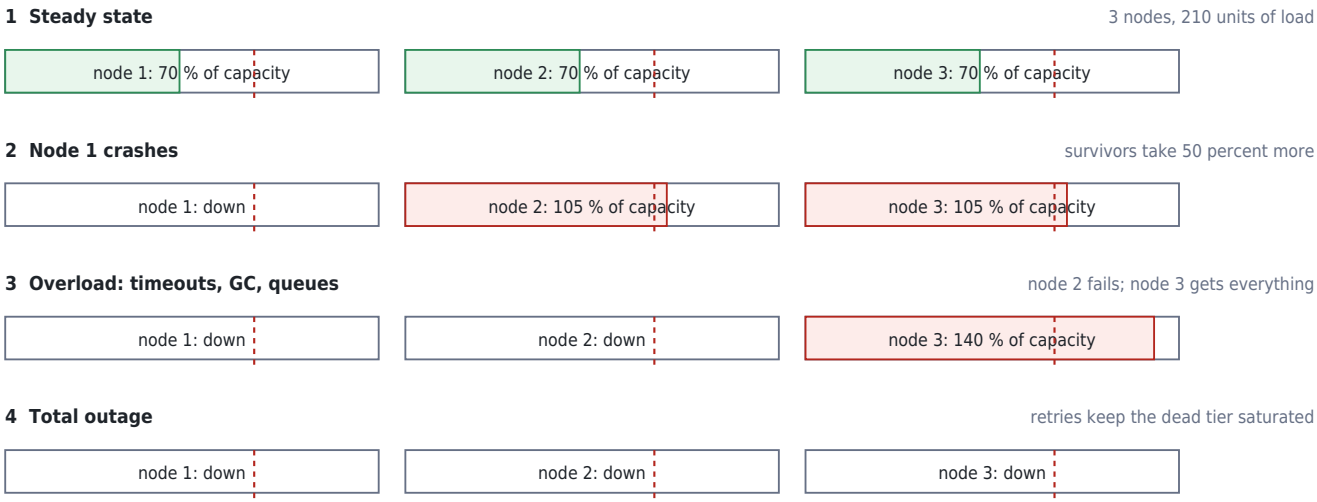


Figure 1.26. Three nodes at 70 percent load; the loss of one raises each survivor to 105 percent, beyond capacity; timeouts, garbage collection and queueing then take down the second node, and the remaining node receives all the load.

The arithmetic is simple. With N nodes at utilisation u, the loss of one node raises the utilisation of the survivors to u x N / (N - 1). Three nodes at 70 percent become 105 percent; ten nodes at 70 percent become 77.8 percent. Small clusters therefore need lower steady-state utilisation, and the margin to be maintained is that the loss of the largest failure domain must leave the survivors below saturation.

1.5.2 Worked example

A request path consists of a CDN and balancer pair (99.99 percent), a gateway tier (99.99 percent), a stateless service tier of three instances in three zones (each 99.5 percent), and a relational database with a primary and a synchronous replica in different zones (99.95 percent with automated failover; assumed values). The availabilities in series are multiplied; the three instances in parallel contribute 1 - 0.005^3 = 99.99999 percent if failures were independent:

Composite availability

```
0.9999 x 0.9999 x (1 - 0.005^3) x 0.9995 = 0.99930
Downtime per year: (1 - 0.9993) x 525 600 minutes = 368 minutes = 6.1 hours
```

Three conclusions follow. The result, 99.93 percent, is lower than any single tier except the database, because dependencies in series accumulate. The database dominates: of 368 minutes of expected downtime, 263 minutes come from the database tier, so spending effort on a fourth service instance has no effect, while improving failover of the database does. And the 99.99999 percent of the parallel tier is an upper bound that assumes independent instances and enough capacity in two zones to carry the load; the shared deploy pipeline, the shared configuration and the shared database connection pool make the true figure lower, which is why such calculations are used to find the weakest tier and not to promise a number.

A service that depends synchronously on five other services of 99.9 percent each cannot exceed 0.999^5 = 99.5 percent unless it degrades gracefully when some of them fail; this is the quantitative argument for isolating optional dependencies.

1.5.3 Hands-on: availability and retry simulations

Both programs run on JDK 21 with the single-file launcher and are one-process simulations of the stated models.

Series and parallel availability

The first program draws, for every simulated minute, whether each component is up (independent Bernoulli trials) and counts the minutes in which the topology works, over 60 simulated years per row. The row with a shared zone adds a common cause: a zone that is down 0.1 percent of the minutes takes both replicas with it.

Availability.java JAVA

```java
import java.util.Locale;
import java.util.Random;

public class Availability {

    static final int MINUTES = 525_600;      // one year in one-minute slots
    static final int YEARS = 60;             // independent simulated years per topology

    /** Decides from the component states (and a shared zone) whether the service is up. */
    interface Topology { boolean up(boolean[] c, boolean zoneDown, Random r); }

    static void simulate(String name, int components, double a, double zoneDown,
                         Topology topology, double formula) {
        Random r = new Random(5);
        boolean[] c = new boolean[components];
        long up = 0, total = (long) MINUTES * YEARS;
        for (long m = 0; m < total; m++) {
            for (int i = 0; i < components; i++) c[i] = r.nextDouble() < a;
            boolean zone = zoneDown > 0 && r.nextDouble() < zoneDown;
            if (topology.up(c, zone, r)) up++;
        }
        double availability = up / (double) total;
        System.out.printf(Locale.ROOT, "%-40s | %9.5f%% | %9.5f%% | %8.1f%n", name,
                100 * availability, 100 * formula, (1 - availability) * MINUTES);
    }

    public static void main(String[] args) {
        String h = "%-40s | %-10s | %-10s | %s%n";
        System.out.printf(h, "topology", "simulated", "formula", "min/yr");
        simulate("1 component, 0.999", 1, 0.999, 0, (c, z, r) -> c[0], 0.999);
        simulate("5 in series, each 0.999", 5, 0.999, 0,
                (c, z, r) -> c[0] && c[1] && c[2] && c[3] && c[4], Math.pow(0.999, 5));
        simulate("10 in series, each 0.999", 10, 0.999, 0, (c, z, r) -> {
            for (boolean b : c) if (!b) return false;
            return true;
        }, Math.pow(0.999, 10));
        simulate("2 parallel, each 0.99, independent", 2, 0.99, 0,
                (c, z, r) -> c[0] || c[1], 1 - Math.pow(0.01, 2));
        simulate("3 parallel, each 0.99, independent", 3, 0.99, 0,
                (c, z, r) -> c[0] || c[1] || c[2], 1 - Math.pow(0.01, 3));
        // both replicas share one zone that is down 0.1 % of the time (common-cause failure)
        simulate("2 parallel 0.99 in one zone (0.999)", 2, 0.99, 0.001,
                (c, z, r) -> !z && (c[0] || c[1]), 0.999 * (1 - Math.pow(0.01, 2)));
        // load balancer 0.9999, then two parallel app servers 0.99, then database 0.999
        simulate("LB 0.9999 -> 2 app 0.99 -> DB 0.999", 2, 0.99, 0,
                (c, z, r) -> r.nextDouble() < 0.9999 && (c[0] || c[1])
                        && r.nextDouble() < 0.999,
                0.9999 * (1 - 0.0001) * 0.999);
    }
}
```

Run (about 25 seconds on the build machine) TERMINAL

```
$ java Availability.java
topology                                 | simulated  | formula    | min/yr
1 component, 0.999                       |  99.90007% |  99.90000% |    525.2
5 in series, each 0.999                  |  99.50123% |  99.50100% |   2621.6
10 in series, each 0.999                 |  99.00350% |  99.00449% |   5237.6
2 parallel, each 0.99, independent       |  99.98991% |  99.99000% |     53.0
3 parallel, each 0.99, independent       |  99.99992% |  99.99990% |      0.4
2 parallel 0.99 in one zone (0.999)      |  99.88942% |  99.89001% |    581.2
LB 0.9999 -> 2 app 0.99 -> DB 0.999      |  99.87995% |  99.88002% |    631.0
```

What to notice: the simulated values agree with the formulas to within sampling error; ten 99.9 percent components in series lose 5 238 minutes a year (3.6 days), nearly ten times the loss of one component; two independent 99 percent replicas reach 99.99 percent; and when both replicas share a zone with 99.9 percent availability, the result is 99.89 percent, no better than the zone itself, despite the replica pair. The last row confirms the composition rule for a chain with a parallel stage.

Retry storm

The second program models a service of capacity 1 000 requests per second that receives 800 new requests per second (80 percent utilisation). It is fully unavailable for 5 seconds (ticks 10 to 14). Every request that is not served in its tick may be retried, up to 5 attempts in total. Three client policies are compared: immediate retry in the next second, exponential backoff with full jitter, and backoff with jitter plus a retry budget of 10 percent of the new-request rate (retries above the budget are abandoned, and those requests fail). One tick is one second and every request is treated as an independent unit.

RetryStorm.java JAVA

```
import java.util.Locale;
import java.util.Random;

public class RetryStorm {

    static final int TICKS = 40;                        // one tick = one second
    static final int OUTAGE_START = 10, OUTAGE_END = 15;   // capacity is 0 in ticks 10..14
    static final double CAPACITY = 1000, NEW_LOAD = 800;
    static final int MAX_ATTEMPTS = 5;

    /** pending[t][k]: requests whose k-th attempt (k = 0 is the first try) happens in tick t. */
    static double[] simulate(boolean backoff, boolean budget, Random rnd, double[] offered) {
        double[][] pending = new double[TICKS + 64][MAX_ATTEMPTS];
        double[] served = new double[TICKS];
        for (int t = 0; t < TICKS; t++) {
            pending[t][0] += NEW_LOAD;
            double total = 0;
            for (double v : pending[t]) total += v;
            offered[t] = total;
            double cap = (t >= OUTAGE_START && t < OUTAGE_END) ? 0 : CAPACITY;
            double fraction = total == 0 ? 1 : Math.min(1, cap / total);
            served[t] = total * fraction;
            double retryBudget = budget ? 0.10 * NEW_LOAD : Double.MAX_VALUE;  // retries per tick
            for (int k = 0; k < MAX_ATTEMPTS - 1; k++) {
                double failed = pending[t][k] * (1 - fraction);
                double allowed = Math.min(failed, retryBudget);
                retryBudget -= allowed;
                if (allowed <= 0) continue;
                if (!backoff) {
                    pending[t + 1][k + 1] += allowed;          // immediate retry in the next tick
                } else {
                    int window = (int) Math.pow(2, k + 1);       // exponential window, full jitter
                    for (int i = 0; i < 100; i++) {
                        pending[t + 1 + rnd.nextInt(window)][k + 1] += allowed / 100;
                    }
                }
            }
        }
        return served;
    }

    public static void main(String[] args) {
        System.out.println("capacity 1000/s, new load 800/s, outage in ticks 10-14, 5 attempts");
        double[][] off = new double[3][TICKS];
        double[][] srv = new double[3][];
        boolean[] backoff = {false, true, true};
        boolean[] budget = {false, false, true};
        for (int i = 0; i < 3; i++) srv[i] = simulate(backoff[i], budget[i], new Random(1), off[i]);
        System.out.println("tick | immediate retry | backoff+jitter | backoff+jitter+budget");
        System.out.println("     | offered/served  | offered/served | offered/served");
        for (int t = 8; t < 30; t++) {
            System.out.printf(Locale.ROOT, "%4d | %7.0f/%6.0f | %7.0f/%6.0f | %7.0f/%6.0f%n", t,
                    off[0][t], srv[0][t], off[1][t], srv[1][t], off[2][t], srv[2][t]);
        }
        String[] names = {"immediate retry", "backoff+jitter", "backoff+jitter+budget"};
        for (int i = 0; i < 3; i++) {
            double peak = 0, extra = 0;
            for (int t = OUTAGE_END; t < TICKS; t++) {
                peak = Math.max(peak, off[i][t]);
                extra += Math.max(0, off[i][t] - NEW_LOAD);
            }
            System.out.printf(Locale.ROOT, "%-22s peak %.0f/s (%.2fx), extra requests %.0f%n",
```

RetryStorm.java (continued) JAVA

```
                    names[i], peak, peak / NEW_LOAD, extra);
        }
    }
}
```

Run TERMINAL

```
$ java RetryStorm.java
capacity 1000/s, new load 800/s, outage in ticks 10-14, 5 attempts
tick | immediate retry | backoff+jitter | backoff+jitter+budget
     | offered/served  | offered/served | offered/served
   8 |     800/   800 |     800/   800 |     800/   800
   9 |     800/   800 |     800/   800 |     800/   800
  10 |     800/     0 |     800/     0 |     800/     0
  11 |    1600/     0 |    1240/     0 |     844/     0
  12 |    2400/     0 |    1651/     0 |     878/     0
  13 |    3200/     0 |    1951/     0 |     870/     0
  14 |    4000/     0 |    2117/     0 |     892/     0
  15 |    4000/  1000 |    2440/  1000 |     882/   882
  16 |    3200/  1000 |    2283/  1000 |     835/   835
  17 |    2588/  1000 |    2181/  1000 |     800/   800
  18 |    2134/  1000 |    1997/  1000 |     800/   800
  19 |    1800/  1000 |    1733/  1000 |     800/   800
  20 |    1540/  1000 |    1750/  1000 |     800/   800
  21 |    1312/  1000 |    1673/  1000 |     800/   800
  22 |    1103/  1000 |    1689/  1000 |     800/   800
  23 |     901/   901 |    1572/  1000 |     800/   800
  24 |     800/   800 |    1498/  1000 |     800/   800
  25 |     800/   800 |    1371/  1000 |     800/   800
  26 |     800/   800 |    1345/  1000 |     800/   800
  27 |     800/   800 |    1252/  1000 |     800/   800
  28 |     800/   800 |    1167/  1000 |     800/   800
  29 |     800/   800 |    1088/  1000 |     800/   800
immediate retry        peak 4000/s (5.00x), extra requests 11378
backoff+jitter         peak 2440/s (3.05x), extra requests 13864
backoff+jitter+budget  peak 882/s (1.10x), extra requests 117
```

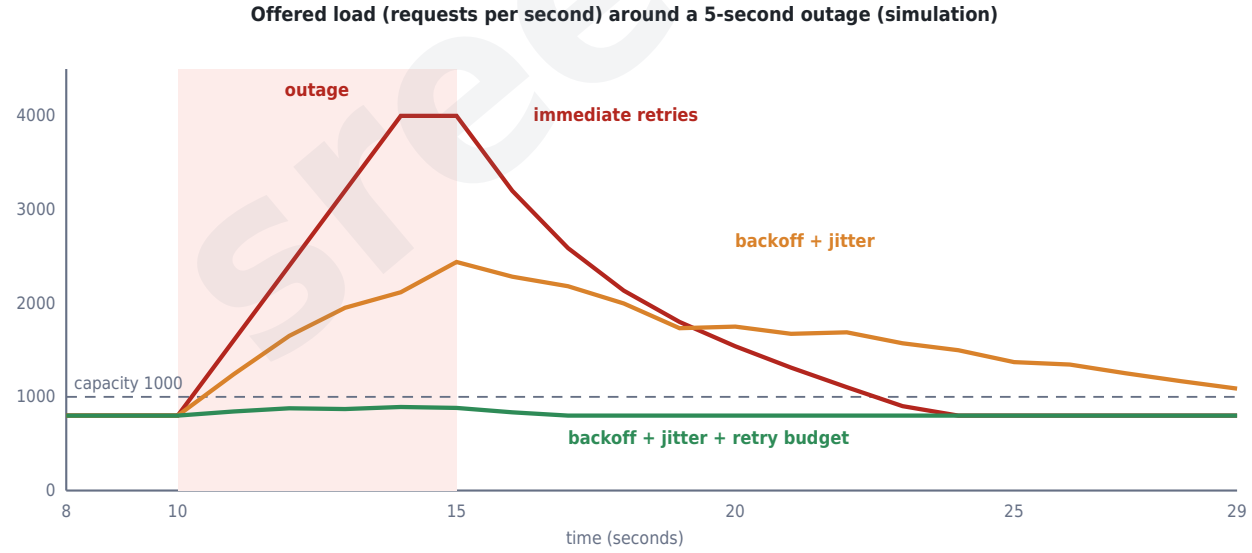


Figure 1.27. Offered load around a 5-second outage: immediate retries pile up to five times the normal load and keep the recovering service above capacity for 8 seconds; backoff with jitter spreads the load but still holds it above capacity fifteen seconds after the outage; the retry budget keeps the offered load below 900 requests per second.

What to notice:

- With immediate retries the offered load rises to 4 000 requests per second at the end of the outage (five times normal, four times the capacity), and the service runs at its capacity limit for 8 more seconds (ticks 15 to 22), recovering at tick 23 although the fault ended at tick 14.
- Backoff with jitter lowers the peak to 2 440 requests per second, but the retries are spread over a long period, so the offered load is still above the capacity of 1 000 at tick 29 (1 088), fifteen seconds after the outage ended, and

the total number of extra requests is larger (13 864 against 11 378). Backoff alone does not bound the load; it only spreads it.

- The retry budget limits the offered load to at most 892 requests per second (1.12 times normal) and the service serves everything from tick 15. The price is that requests whose retries are refused fail permanently: the budget trades a bounded recovery for lost requests, which is acceptable only where the caller can report the failure or the work can be requeued.
- The model has no capacity-dependent collapse (served throughput stays at the capacity of 1 000 per second). Real services often lose throughput under overload through thread contention, garbage collection and timeouts, which makes the real storm worse than the simulation shows.

1.5.4 Real-world problems and solutions

The scenarios below are composite illustrations of failure classes; the figures in them are illustrative and do not report a specific incident.

Problem 1: single point of failure

Scenario. A service has stateless application nodes in three zones, but all instances use one configuration service, one primary database in one zone, and a single message broker node.

Symptoms. Loss of the broker node halts all asynchronous processing although every application node is healthy; a zone failure containing the database primary takes the service down for 20 minutes during a manual promotion.

Diagnosis. A dependency diagram of the request path shows components with a single instance; the failover for the database is manual.

Root cause. Redundancy was applied to the visible tier only; the dependency chain contains components with availability equal to a single node and with no automated recovery.

Fix. Replicate each stateful component across zones (broker cluster with replication, database with synchronous replica and automated promotion, configuration with a cached last-known value in each instance so that its outage does not stop the service).

Prevention. For every component on the request path record the number of instances, the failure domain and the recovery action; review the list at each design change; run game days in which a component is killed.

Problem 2: retry storm after a brief outage

Scenario. A downstream service is unavailable for 30 seconds. Three layers of the caller each retry failed calls three times immediately.

Symptoms. After the downstream service restarts, it is overloaded again within seconds and stays unavailable for ten minutes; request volume at the downstream is five to ten times normal; callers' threads are exhausted.

Diagnosis. Request-rate graphs show a spike beginning when the dependency recovers; traces show multiple attempts per user request; the downstream fails with timeouts and not with business errors.

Root cause. Layered, immediate, unbudgeted retries multiply load (3 x 3 x 3 = 27 attempts per user request in the worst case) and synchronise clients, so the recovering service is hit by the entire backlog at once.

Fix. Retry in one layer only (typically the edge closest to the user); use exponential backoff with full jitter; enforce a retry budget; open a circuit breaker so that failures fail fast; let the downstream shed load with 429 and Retry-After so that callers back off.

Prevention. Retry policy is part of the design review; chaos tests inject a 30-second dependency outage and verify recovery without manual action.

Problem 3: cascading failure after the loss of one node

Scenario. A cluster of four nodes runs at 80 percent utilisation at peak. One node fails during the evening peak.

Symptoms. The three survivors rise to 107 percent, latency climbs, health checks time out, and the balancer removes a second node; the remaining two nodes receive 160 percent of capacity each and the service is down within minutes.

Diagnosis. Load per node shows a staircase as nodes disappear; garbage-collection pauses and queue depth grow before each failure.

Root cause. Capacity was planned for the healthy state: 80 percent x 4 / 3 = 107 percent after one failure. Overloaded nodes slow down, fail health checks and are ejected, which transfers still more load.

Fix. Plan peak utilisation so that the loss of the largest failure domain leaves less than 70 to 80 percent (here: at most 60 percent with four nodes); shed load at the edge instead of queueing; limit the share of nodes that health checks may eject (a panic threshold); add autoscaling on queue depth.

Prevention. Capacity review for N+1 or N+2 at peak; load tests that remove a node under peak load.

Problem 4: failover that was never tested

Scenario. A database has a standby that has never been promoted in production. The primary fails.

Symptoms. Promotion takes 40 minutes; applications still point to the old address; the standby is found to be several minutes behind and missing a required extension.

Diagnosis. The runbook is out of date; connection strings are hard-coded; replication lag alarms were disabled after earlier noise.

Root cause. The failover path had no automation and no routine exercise, so latent faults accumulated unnoticed.

Fix. Automate promotion and endpoint switching (virtual address, proxy or DNS with a short TTL), monitor replication lag with an alert tied to the recovery-point objective, and keep the standby configuration identical to the primary.

Prevention. Perform scheduled failover drills, at least quarterly, and treat a drill failure as an incident.

Problem 5: no timeout on a remote call

Scenario. A service calls an internal API whose latency usually is 20 ms. The HTTP client has no read timeout.

Symptoms. When the API starts to hang (accepting connections but not responding), the caller's thread pool fills with blocked threads within a minute; the caller stops serving every endpoint, including those that do not use the API.

Diagnosis. A thread dump shows all worker threads waiting in socket reads; CPU is low.

Root cause. A missing timeout allows a single slow dependency to consume every thread of the caller (resource exhaustion by waiting).

Fix. Set connect and read timeouts, bound the thread or connection pool per dependency (bulkhead) so that one dependency cannot exhaust the shared pool, and add a circuit breaker.

Prevention. A client wrapper that makes the timeout mandatory; a lint or review rule for any outbound call.

1.5.5 Questions to expect

1. Define SLI, SLO and SLA and the relation between them.

An SLI is a measured quantity (for example the proportion of requests that succeed within 300 ms); an SLO is a target for that quantity (99.9 percent over 30 days); an SLA is a contract with consequences if a stated objective is missed. The SLO is set stricter than the SLA so that the team reacts before a penalty applies. The error budget is 1 minus the SLO.

2. What is the availability of five services in series, each 99.9 percent, and of two replicas in parallel, each 99 percent?

Series: 0.999^5 = 99.50 percent (about 43.7 hours of downtime per year). Parallel: 1 - 0.01^2 = 99.99 percent, assuming independent failures and sufficient capacity in each replica. Shared failure causes reduce the parallel result.

3. Compare active-passive and active-active.

Active-passive: one serving node, one standby; simple, no write conflicts, but idle capacity, promotion delay and possible loss of recent writes under asynchronous replication. Active-active: all nodes serve; failover is removal from rotation and is fast, but survivors need headroom and shared state needs conflict handling.

4. How many nines can a design achieve with a single region?

Zone-level redundancy with automated failover commonly supports three to four nines, limited by region-wide events and by shared control planes and deployments. Higher targets require multi-region designs and disciplined change management; the target must be justified by the cost of downtime.

sreram

5. Why are retries dangerous, and how are they made safe?

Retries add load during failure and, when layered or synchronised, multiply it. They are made safe by retrying only idempotent operations, using exponential backoff with jitter, limiting attempts and enforcing a retry budget, retrying in one layer, and combining retries with circuit breakers and load shedding at the callee.

6. What is the difference between backpressure and load shedding?

Backpressure slows the producer (bounded queues, 429 with Retry-After, flow control) so that the system is not offered more than it can process. Load shedding discards excess work at the system boundary, choosing the least valuable first, when producers cannot or will not slow down. Both require bounded queues.

7. How does a design avoid a cascading failure?

It keeps enough spare capacity to absorb the loss of the largest failure domain, bounds queues and applies admission control so that overload becomes fast rejections and not slow timeouts, uses timeouts and breakers to isolate failing dependencies, limits automatic ejection of nodes, and isolates resource pools (bulkheads) per dependency.

8. What is graceful degradation?

Continuing to provide the essential function with reduced quality when a component fails: stale cache, omitted optional panel, queued writes, default values. It requires classifying features by importance in advance and testing the degraded paths.

9. What is an idempotency key and when is it required?

A client-generated unique identifier for one logical operation, stored by the server with the outcome. A repeated request with the same key returns the stored outcome instead of repeating the effect. It is required for non-idempotent operations (payments, order creation) whenever retries, redelivery or double submission can occur.

KEY POINTS

- SLIs are measured, SLOs are targets, SLAs are contracts; the error budget (1 - SLO) turns reliability into a shared quantity.
- Each added nine cuts the budget tenfold and removes manual recovery: 99.9 percent allows 8.76 hours a year, 99.99 percent allows 52.6 minutes.
- Series components multiply availabilities (ten at 99.9 percent give 99.0 percent); independent parallel replicas multiply unavailabilities; shared failure domains cap the gain.
- Redundancy needs independent failure domains, spare capacity (N+1) and a failover path that is automated and tested.
- Timeouts, budgeted retries with jitter, circuit breakers and idempotency keys keep failures local; layered immediate retries multiply load (3 x 3 x 3 = 27).
- Bounded queues, backpressure and load shedding turn overload into fast rejections; with N nodes at utilisation u, losing one raises survivors to u x N / (N - 1).

1.6 Choosing the Data Store

The data store is the component of a design that is hardest to change after launch. Application servers are stateless and replaceable; a database holds the only copy of the business state, and its data model, consistency behaviour and failure modes constrain every other component. A design that selects a store by popularity, rather than by access pattern, typically fails in one of two ways: the workload does not fit the store (a join-heavy reporting query on a key-value system), or the store is operated at a scale it was not designed for (a single relational node taking every write).

This chapter provides a decision framework for the choice. It assumes the internals of relational and NoSQL systems (indexes, MVCC, replication, sharding, the storage engines) as covered in the databases material, and concentrates on the question a design interview poses: given these requirements, which store, and why.

1.6.1 Concept and mechanism

Start from the access pattern, not the data shape

A store is selected by the operations that the system performs, ranked by frequency and by latency sensitivity. Four questions determine most of the decision.

1. **What are the dominant reads?** Lookup by primary key, lookup by secondary attribute, range scan, join across entities, full-text search, aggregation over many rows, or traversal of relationships.
2. **What are the dominant writes?** Single-row updates, multi-row transactions that must be atomic, append-only events, or bulk loads.
3. **What consistency is required?** Strong (every read observes the latest committed write), or eventual (a bounded or unbounded delay is acceptable). Some invariants, such as a non-negative account balance or a unique username, cannot be enforced under eventual consistency without extra machinery.
4. **What are the scale and growth?** Data volume, read and write rates, and the factor by which both are expected to grow. A workload that fits on one well-provisioned node for the next three years does not need a partitioned store.

Store families and their fit

Family	Strong at	Weak at	Typical use
`Relational (OLTP)`	joins, constraints, multi-row ACID transactions, ad hoc queries with indexes	horizontal write scaling needs sharding, which removes cross-shard joins and transactions	orders, payments, accounts, inventory
`Key-value`	single-key get and put at very low latency and high throughput	no secondary queries, no joins	sessions, caches, feature flags, counters
`Document`	variable-shape records read and written as a unit, secondary indexes on fields	multi-document transactions are limited or costly, joins are manual	product catalogue, user profiles, content
`Wide-column`	very high write rates, partitioned by key, tunable consistency	queries must follow the primary key design, no joins	activity feeds, messaging history, telemetry
`Search engine`	full-text, relevance ranking, faceted filters	not a system of record, eventual consistency, expensive updates	product search, log search, autocomplete
`Time-series`	append-heavy timestamped data, compression, downsampling, retention	general-purpose updates and joins	metrics, sensor data, monitoring
`Graph`	multi-hop relationship traversal	bulk aggregation, sharding a connected graph	social graph, fraud rings, recommendations
`Columnar warehouse (OLAP)`	scans and aggregates over billions of rows	single-row lookups, high-concurrency small writes	analytics, reporting, business intelligence
`Object storage`	cheap, durable storage of large immutable blobs	no queries, whole-object access, higher latency	images, video, backups, data lake files

Store families, typical fit and typical limitation.

The families overlap in practice. A relational database with a JSON column covers many document workloads; a document database with multi-document transactions covers many relational ones. The table states default strengths, not exclusive capabilities. The decision should therefore rest on the dominant access pattern and on the operational reality of the team, not on the label of the product.

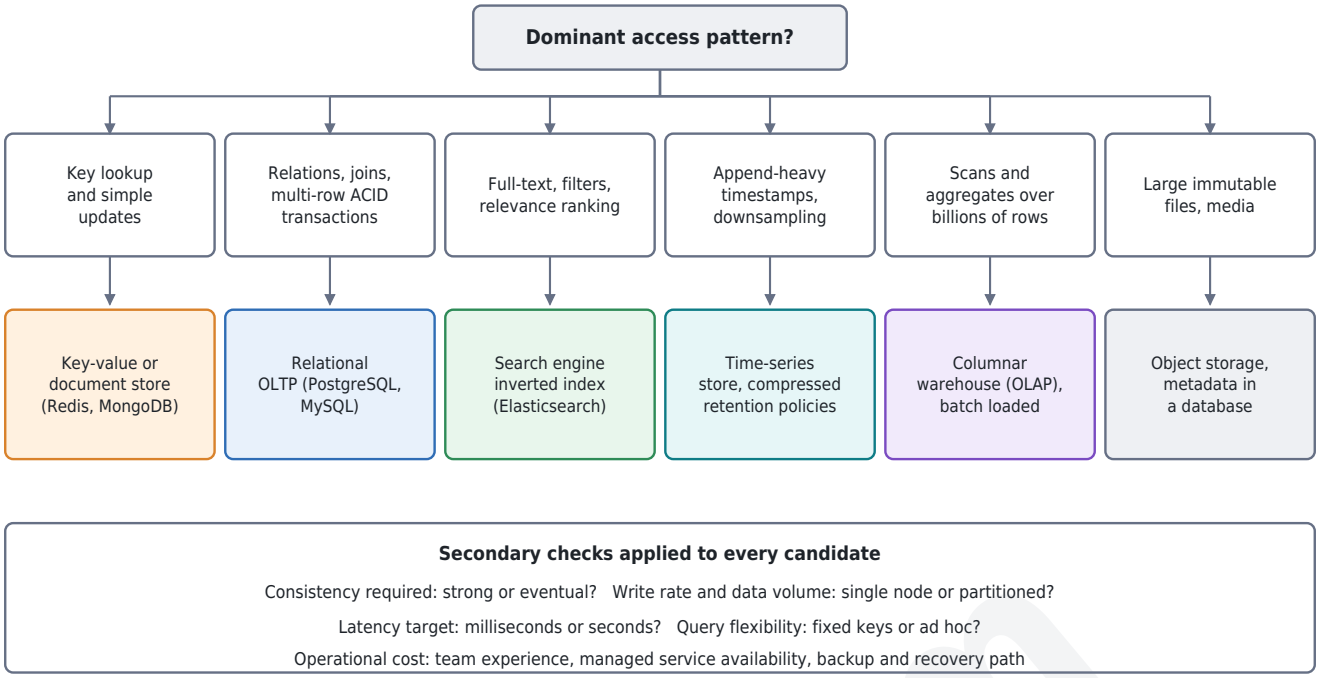


Figure 1.28. The access pattern selects the store family; the four secondary checks (consistency, scale, latency, operations) then filter the candidates and are applied to every branch.

SQL versus NoSQL: the decision in practice

The distinction is less about query language than about which guarantees the store maintains on behalf of the application. A relational store enforces schema, constraints and transactions in the database. A typical NoSQL store moves part of that responsibility into the application in exchange for horizontal scalability, flexible schema or a simpler operational model.

Relational is the default for the system of record unless a concrete requirement rules it out. The reasons: transactions remove entire classes of application bugs, ad hoc queries keep the data useful for questions that were not anticipated, and a single well-tuned node sustains thousands of writes per second and tens of thousands of indexed reads per second. A NoSQL store becomes the better choice under any of these conditions:

- The write rate or data volume exceeds what one relational primary with replicas can serve, and the access pattern is key-based, so sharding by key is natural (feed items, messages, events).
- The workload is a pure key lookup with a latency target under one millisecond (sessions, rate limiter counters).
- The record is naturally a self-contained aggregate that is read and written as one unit and has a variable shape (product pages with category-specific attributes).
- The availability requirement dictates multi-region writes with tolerance for conflicts, which leaderless and multi-leader stores support natively.

Row stores, column stores, OLTP and OLAP

An OLTP (online transaction processing) workload consists of many short transactions touching a few rows each; a row-oriented layout, which stores all fields of a row contiguously, serves it well because a transaction reads or writes whole rows. An OLAP (online analytical processing) workload consists of few queries scanning many rows but only a few columns; a column-oriented layout stores each column contiguously, which reads only the needed columns, compresses well (similar values adjacent) and enables vectorised execution.

Running analytical queries against the OLTP primary degrades both: long scans evict cache pages and hold resources, while the transactions that generate revenue slow down. The standard remedy is to copy data into a separate analytical store through change data capture (CDC, reading the database log) or batch export, accepting that analytics lag the source by seconds to hours.

Polyglot persistence and the single system of record

Polyglot persistence is the use of several store types in one system, each for the workload it serves best. The principle that keeps it manageable is a **single system of record** per entity: exactly one store holds the authoritative copy, and every other store holds a derived copy that can be rebuilt from the authoritative one or from an event log. Derived copies are updated asynchronously (CDC or events), so they are eventually consistent, and a divergence is repaired by rebuilding the copy, not by reconciling two authorities.

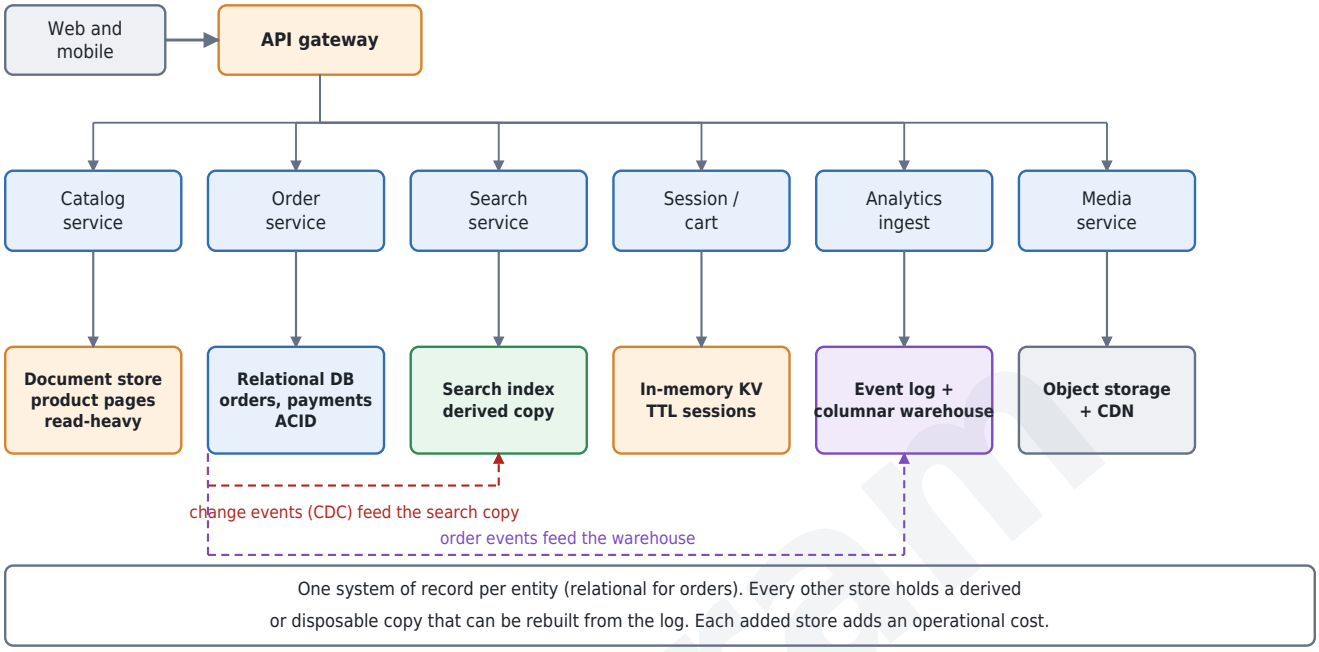


Figure 1.29. A polyglot e-commerce backend: the relational store is the system of record for orders, the search index and warehouse are derived copies fed by change events, and sessions and media use stores matched to their access pattern.

Each additional store carries a cost: another failure domain, another backup and recovery procedure, another consistency boundary and another set of skills. A design should justify each store by a concrete access pattern that the existing stores cannot serve, and should prefer an extension of an existing store (a JSON column, a full-text index in the relational database) when the volume is moderate.

Specialised stores

Search stores. An inverted index maps each term to the list of documents containing it, which supports full-text matching and relevance ranking that a B-tree index cannot provide (see the chapter on common infrastructure components). A search store is a derived copy: documents are indexed after the system of record commits, so search results lag by a refresh interval, commonly about one second, and a rebuild path must exist.

Object storage. Blobs (images, video, documents) are stored as immutable objects addressed by key, with durability achieved through replication or erasure coding across failure domains. The database holds only metadata and the object key. Clients upload and download directly through signed URLs, so that application servers do not carry the bytes. Object stores offer high throughput and low cost per gigabyte but higher per-request latency than a database and no partial update of an object.

Time-series stores. Measurements arrive in timestamp order, are rarely updated, and are queried by time range and aggregated. The stores exploit this with append-only segments, delta and run-length compression, automatic downsampling (for example one-second data rolled up to one-minute averages after a week) and retention policies that drop old data cheaply by dropping whole partitions.

Graph stores. A graph store keeps edges as first-class, directly traversable structures, so a query such as friends-of-friends who like a given item costs proportional to the neighbourhood visited, not to table sizes as repeated joins would. The trade-off is that large connected graphs partition poorly, so graph stores are chosen when traversal depth matters more than horizontal write scale. Many systems that appear to need a graph store are served by two relational tables and a bounded-depth query.

1.6.2 Worked example

The following selection is made for a ride-sharing style product with the assumed volumes shown. All figures are assumptions for the exercise, not measurements.

Data	Access pattern (assumed volume)	Requirement	Selected store and reason
Trips and payments	create, update status, query by rider or driver; about 2,000 trips per second at peak	atomic trip and payment state, no lost update	sharded relational store; transactions keep payment and trip consistent
Driver location pings	1,000,000 drivers, one update per 4 s = 250,000 writes per second; read latest position	low latency, loss of old pings acceptable	in-memory key-value store keyed by driver id, plus a geo index; history streamed to a log
Location history	append, query by driver and time range	cheap storage, retention 90 days	time-series or wide-column store, partitioned by driver and day
Trip search for support staff	full-text and filters over trips	seconds-level freshness	search index fed by CDC from the trip store
Receipts, driver documents	immutable files up to 10 MB	durable, cheap, served by CDN	object storage, metadata in the trip store
Business reporting	scans over months of trips	no impact on live traffic	columnar warehouse loaded nightly and by stream

Selection by workload for an assumed ride-hailing backend.

The arithmetic behind the location row is the decisive one: 1,000,000 drivers divided by a 4-second update interval gives 250,000 writes per second, a rate that a single relational primary is not expected to absorb as individual transactions, whereas an in-memory key-value store partitioned by driver id handles it with a modest cluster. Trips, at 2,000 per second, are two orders of magnitude lower and carry the transactional requirement, so they stay relational.

1.6.3 Hands-on

Two measurements illustrate why layout and index structure matter. The programs below run on the build machine; absolute numbers depend on the machine and on the Python interpreter overhead, so only the ratios are meaningful.

columnar.py PYTHON

```python
import time, random, sqlite3
from array import array
random.seed(1)
N = 2_000_000
price = [random.random() * 100 for _ in range(N)]
# row layout: one tuple per row with 8 fields; column layout: one packed array per field
rows = [(i, i % 1000, price[i], i * 2, "x", 1, 2, 3) for i in range(N)]
col_price = array("d", price)
def t(fn, reps=5):
    best = 1e9
    for _ in range(reps):
        a = time.perf_counter(); r = fn(); best = min(best, time.perf_counter() - a)
    return best, r
tr, r1 = t(lambda: sum(r[2] for r in rows))
tc, r2 = t(lambda: sum(col_price))
print(f"SUM(price) over {N:,} rows, 8 fields per row")
print(f"  row layout (tuples)   : {tr*1000:7.1f} ms")
print(f"  column layout (array) : {tc*1000:7.1f} ms  ({tr/tc:.1f}x faster)")
print("  same result:", abs(r1 - r2) < 1e-3)

# point lookup with and without an index (SQLite)
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE users(id INTEGER, email TEXT, name TEXT)")
con.executemany("INSERT INTO users VALUES(?,?,?)",
                ((i, f"user{i}@example.com", f"n{i}") for i in range(1_000_000)))
def q():
    sql = "SELECT name FROM users WHERE email = ?"
    return con.execute(sql, ("user987654@example.com",)).fetchone()
ts, _ = t(q, 3)
con.execute("CREATE INDEX idx_email ON users(email)")
```

columnar.py (continued) PYTHON

```python
ti, _ = t(q, 3)
print(f"Point lookup by email in 1,000,000 rows (SQLite, in memory)")
print(f"  full scan : {ts*1000:8.2f} ms")
print(f"  B-tree idx: {ti*1000:8.3f} ms  ({ts/ti:,.0f}x faster)")
```

Typical output (measured on the build machine) TERMINAL

```
$ python3 columnar.py
SUM(price) over 2,000,000 rows, 8 fields per row
  row layout (tuples)   :    48.6 ms
  column layout (array) :    12.3 ms  (4.0x faster)
  same result: True
Point lookup by email in 1,000,000 rows (SQLite, in memory)
  full scan :    27.52 ms
  B-tree idx:    0.002 ms  (15,542x faster)
```

The first result shows the column-store advantage for aggregation: the column layout reads only the price values, contiguous in memory, while the row layout visits every tuple. The ratio in a real columnar engine is larger on disk, where reading fewer bytes dominates, and smaller when the row store already has a covering index. The second result shows that the access pattern (lookup by email) is served by a matching index and not by the store family: a relational database serves it in microseconds once the index exists. A store that lacks an index on the queried attribute degenerates to a scan regardless of its category.

1.6.4 Real-world problems and solutions

Problem 1: key-value store chosen, then ad hoc queries are required

Scenario. A team stores orders in a key-value store because the first feature is lookup by order id at high volume. Six months later support needs orders by customer, by date range and by status. **Symptoms.** Each new report requires a full scan or a new hand-maintained index table; scan jobs time out; data is duplicated in several keys that drift apart. **Diagnosis.** Query logs show scans over the full keyspace; row counts differ between the main table and the hand-built index tables. **Root cause.** The store was chosen for the first access pattern only, and the system of record cannot answer questions that were not anticipated. **Fix.** Move the system of record to a relational store (or add global secondary indexes where the store supports them), keeping the key-value store as a cache for the id lookup. Backfill from the existing data through a bulk export. **Prevention.** List the foreseeable query patterns (support, reporting, search) before choosing the system of record, and default to relational when the list is open-ended.

Problem 2: analytics queries on the primary database

Scenario. A dashboard runs a monthly aggregation over the orders table on the primary database during business hours. **Symptoms.** Checkout latency rises from 40 ms to seconds at the top of each hour; connection pool exhaustion; replication lag increases. **Diagnosis.** The slow query log shows a sequential scan lasting minutes; the buffer cache hit ratio drops during the scan. **Root cause.** A long analytical scan competes with transactions for I/O, cache pages and locks or vacuum horizons. **Fix.** Route analytics to a read replica as a first step, and then to a warehouse loaded by CDC. Apply statement timeouts to the reporting role. **Prevention.** Separate OLTP and OLAP roles and endpoints from the first release, and set per-role resource limits.

Problem 3: search index treated as the source of truth

Scenario. Product data is written directly to the search cluster, which also serves the product page. **Symptoms.** After a cluster incident, recent updates are missing; a mapping change requires reindexing and there is no authoritative copy to reindex from. **Diagnosis.** The record count in the search index differs from the count expected from order history; no other copy contains the product edits. **Root cause.** A derived-data store was used as the system of record, and it lacks the durability and transactional guarantees expected of one. **Fix.** Restore the product data from the latest snapshot and logs where possible, then introduce a relational or document system of record and a CDC pipeline that feeds the index. **Prevention.** State the system of record of every entity in the design document, and treat search, cache and warehouse as rebuildable.

Problem 4: too many stores for a small team

Scenario. A product with 20,000 users runs five database technologies chosen for individual features. **Symptoms.** Incident response is slow because few engineers know every store; backup restores were never tested for three of the five; a version upgrade is postponed for a year. **Diagnosis.** An inventory of stores shows each below 5% utilisation and several holding under 10 GB. **Root cause.** Each store was justified by a theoretical advantage at scale that the actual volume does not reach, while the operational cost is incurred immediately. **Fix.** Consolidate onto the relational database with a JSON column, a full-text index and a cache, and retire stores whose volume does not justify them. **Prevention.** Require a quantified reason (volume, rate, latency) for each additional store, and review the inventory when the workload changes.

1.6.5 Questions to expect

1. **How is a database chosen in a design interview?** State the dominant access patterns, the consistency requirement and the expected scale with numbers, then map them to a store family. Name the system of record explicitly and list derived stores. Default to relational unless a quantified requirement rules it out.
2. **When is NoSQL a better choice than SQL?** When the write rate or volume exceeds a single relational primary and the access is key-based; when the latency target for key lookups is sub-millisecond; when the data is a self-contained aggregate with variable shape; or when multi-region writes are required. Otherwise the transactions and query flexibility of a relational store are valuable.
3. **Why not use one database for everything?** A single database is the correct starting point. It is split when a workload class (analytics, search, large blobs, very high write rates) harms the others or exceeds what the store can do. Splitting adds operational and consistency cost.
4. **What is polyglot persistence and what is its main risk?** Using multiple store types, each for the workload it suits. The main risk is inconsistency between copies; it is controlled with a single system of record and asynchronously derived copies that can be rebuilt.
5. **Why are columnar stores used for analytics?** Analytical queries read few columns across many rows. Column layout reads only the needed columns, compresses well and allows vectorised execution, so scans are cheaper by large factors.
6. **Why are images not stored in the database?** Large blobs inflate backups and replication, consume buffer cache and force the database to serve bytes. Object storage is cheaper and more durable for blobs, and signed URLs let clients transfer directly. The database stores the key and metadata.
7. **How is a search index kept consistent with the primary database?** By CDC or events emitted after the primary commits; the index is eventually consistent, with a lag measured in seconds, and a full reindex path from the system of record covers failures.
8. **When is a graph database justified?** When queries traverse several hops and the neighbourhood visited is small relative to the data (fraud rings, recommendations). For one or two hops, relational joins are usually sufficient.

KEY POINTS

- Select a store from the dominant access patterns, the required consistency and the quantified scale, not from product reputation.
- Relational is the default system of record; leave it only for a stated, quantified reason.
- Keep exactly one system of record per entity; search, cache and warehouse are derived and rebuildable.
- Separate OLTP from OLAP early; analytical scans on the primary degrade transactions.
- Each added store adds operational cost; justify it with volume, rate or latency numbers.

Sheeran

1.7 Caching

A cache stores a copy of data in a faster or closer location so that repeated requests avoid the cost of the original source. In a backend system the original source is usually a database, whose latency is of the order of milliseconds and whose capacity is bounded by CPU and I/O. A cache in memory answers in well under a millisecond and absorbs the majority of reads, which converts a database-bound system into one limited by the cache tier, a tier that scales horizontally and cheaply. Caching is therefore the first optimisation applied to a read-heavy design, and the source of a large share of its subtle failures: stale data, stampedes and cold-start outages.

This chapter covers the placement of caches, the five write and read strategies, eviction, invalidation, the three classic failure modes (stampede, penetration, avalanche), HTTP and CDN caching, and sizing. The Redis material covers the data structures and the server internals; the Spring material covers the `@Cacheable` abstraction. Both are referenced, not repeated.

1.7.1 Concept and mechanism

Why caching works

Caching is effective because request distributions are skewed: a small fraction of keys receives most of the traffic. For a Zipf distribution with exponent 1 over 100,000 keys, an LRU cache holding 20% of the keys reached an 80.6% hit ratio in the measurement later in this chapter, and a cache of 1% reached 50.6%. For uniformly distributed keys the same 20% cache reaches only about 20%. The hit ratio, and therefore the benefit, is a property of the workload, and a design must state which distribution it assumes.

With hit ratio h, cache latency L_c and database latency L_d, the expected latency is h x L_c + (1 - h) x (L_c + L_d), since a miss pays for the cache lookup and then the database. With assumed values L_c = 0.5 ms and L_d = 8 ms: at h = 0.80 the mean is 0.5 + 0.2 x 8 = 2.1 ms, and at h = 0.95 it is 0.5 + 0.05 x 8 = 0.9 ms. The database load falls in the same proportion: the database receives (1 - h) of the read traffic, so moving h from 0.80 to 0.95 cuts database reads by a factor of four.

Where caches sit

A request can be served from several layers, each with a different scope, latency and invalidation mechanism.

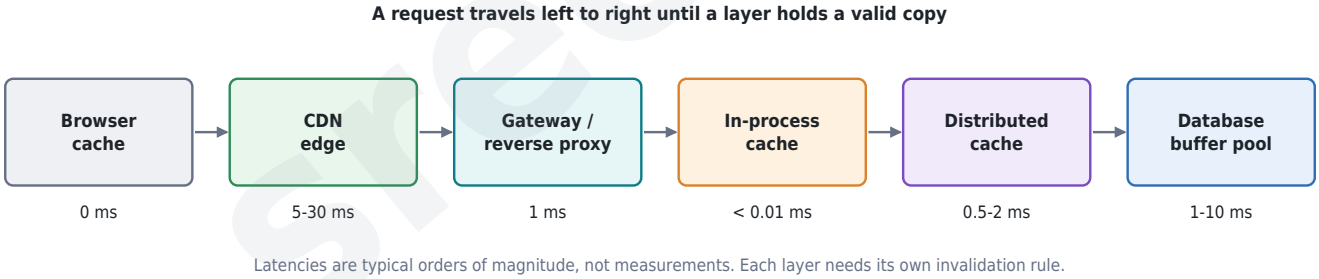


Figure 1.30. Cache layers from the client to the database; a request is answered by the first layer holding a valid copy, and every layer needs its own invalidation rule.

Layer	Scope	Strength	Main risk
Browser	one user	zero network cost	cannot be invalidated by the server before expiry
CDN edge	all users in a region	offloads origin bandwidth, low latency	stale assets, purge propagation delay
In-process (for example Caffeine)	one service instance	no network hop, nanosecond access	each instance holds a different copy; memory shared with the heap
Distributed (Redis, Memcached)	all instances	single shared copy, large capacity	network hop, a new dependency whose failure shifts load to the database
Database buffer pool	one database node	automatic	not controlled by the application

Cache layers and their properties.

An in-process cache and a distributed cache are frequently combined: a small local cache with a short TTL (a few seconds) in front of the distributed cache removes the hottest keys from the network entirely, at the price of bounded staleness between instances.

The five access strategies

The strategies differ in who talks to the database and when a write reaches it.

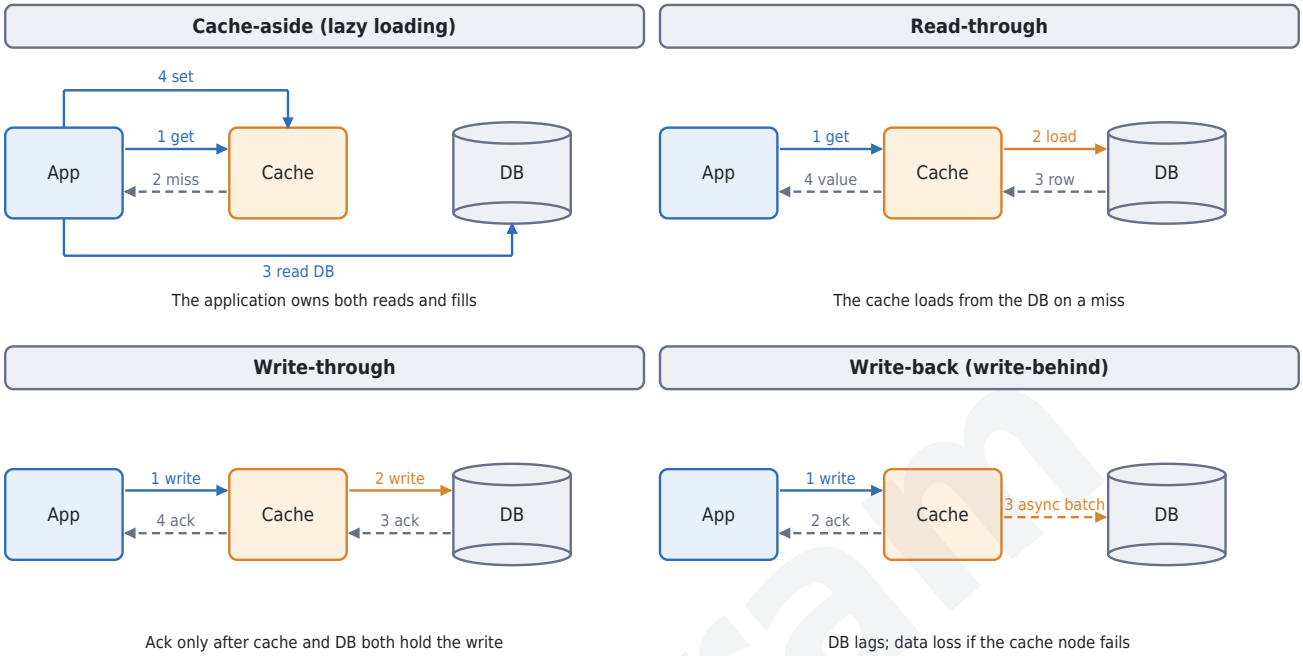


Figure 1.31. The four main strategies drawn as message flows; write-around (not drawn) writes to the database only and lets the next read populate the cache.

Strategy	Mechanism	Strength	Weakness
Cache-aside	application checks the cache, loads from the database on a miss and fills the cache; on write, updates the database and deletes the cache entry	simple; the cache holds only requested data; failure of the cache degrades to database reads	race between a read fill and a write (stale entry); first read after a miss is slow
Read-through	the cache component loads from the database itself	application code is simpler; loading logic in one place	requires a cache that supports loaders; the same stale-fill race exists
Write-through	each write goes to the cache and the database synchronously	cache is always consistent with acknowledged writes; reads after writes hit	write latency includes both; caches data that may never be read
Write-back (write-behind)	write to the cache, acknowledge, flush to the database asynchronously, often batched	lowest write latency; coalesces repeated writes to one key	data loss if the cache node fails before the flush; the database lags
Write-around	write to the database only; the cache is populated on the next read	avoids filling the cache with write-once data	a read soon after a write misses

Caching strategies compared.

Cache-aside with delete-on-write is the default choice for most services. Write-through is used when reads of recently written data dominate and the cache is a managed component. Write-back is reserved for workloads that tolerate loss within the flush interval, such as counters and telemetry, or where the cache itself is durable and replicated.

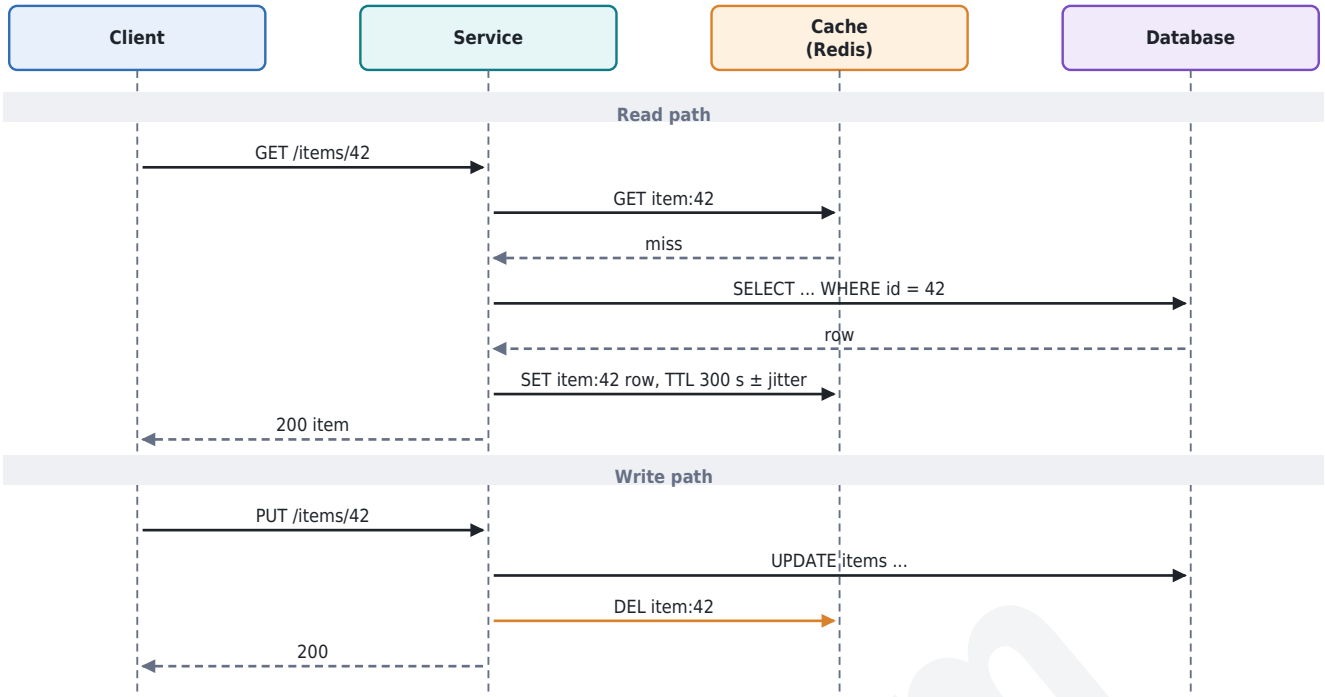


Figure 1.32. The cache-aside read and write paths: the read path fills the cache with a TTL that includes jitter, and the write path updates the database first and then deletes the entry.

Eviction

When the cache is full, an eviction policy selects the entries to remove.

- **LRU** (least recently used) evicts the entry unused for the longest time. It adapts to shifting popularity and is implemented with a hash map plus a doubly linked list in O(1) per operation. A single large scan can flush the whole cache.
- **LFU** (least frequently used) evicts the entry with the fewest accesses. It protects consistently hot keys and resists scans, but an entry that was hot long ago may persist unless counts decay. Modern implementations combine frequency and recency: Caffeine uses W-TinyLFU, and Redis offers `allkeys-lfu` with a decaying counter.
- **FIFO and random** are cheap and sometimes adequate; random eviction is the baseline in the literature.
- **TTL** (time to live) is not an eviction policy for capacity but a freshness bound: an entry expires after a fixed duration whether or not the cache is full.

Redis approximates LRU by sampling a small number of keys (the `maxmemory-samples` setting) and evicting the best candidate among them, which avoids the cost of maintaining a global list. The `maxmemory-policy` setting selects the policy, for example `allkeys-lru`, `allkeys-lfu` or `volatile-ttl`.

TTL and jitter

A TTL bounds staleness and reclaims memory without explicit invalidation. Its value is a trade-off: a long TTL raises the hit ratio and the staleness window; a short TTL does the reverse. Entries written at the same moment with the same TTL expire at the same moment, which produces synchronised misses. Adding random jitter (for example TTL = 300 s plus a random value between 0 and 60 s) spreads the expiries.

Invalidation strategies

Invalidation is the hard part of caching because the cache and the database cannot be updated atomically.

1. **TTL only.** Accept staleness up to the TTL. Appropriate for data where bounded staleness is a stated requirement (a product description may lag by 5 minutes).
2. **Delete on write.** The writer updates the database, then deletes the cache entry; the next read refills it. Deleting is safer than updating the cache, because concurrent updates can then write values in the wrong order.
3. **Event-driven.** A change event (CDC from the database log, or an event published by the service) triggers invalidation in every cache, including local caches in each instance. This covers writes made by paths other than the service that owns the cache.

4. **Versioned keys.** Include a version or content hash in the key (`item:42:v17`, `app.3f9a1c.js`). New data uses a new key; old entries age out. This removes the invalidation problem for immutable content and is the standard approach for static assets.

1.7.2 Worked example

The sizing below uses the 80/20 rule as a starting estimate, then replaces it with the measured curve where available. All inputs are assumptions.

Quantity	Value	Arithmetic
Items	10,000,000	assumed
Average value size	2 KB plus 64 B entry overhead	assumed, about 2.1 KB per entry
Full dataset in cache	about 21 GB	10,000,000 x 2.1 KB
Cache the hottest 20%	2,000,000 entries, 4.2 GB	20% x 21 GB
Memory with 30% headroom for fragmentation	about 5.5 GB	4.2 GB x 1.3
Peak read rate	40,000 per second	assumed
Expected hit ratio	about 80%	measured 80.6% for Zipf s = 1.0 at 20% (see Hands-on); depends on the real distribution
Database reads at peak	8,000 per second	40,000 x (1 - 0.80)
Database reads if the hit ratio were 95%	2,000 per second	40,000 x (1 - 0.95)
Cache nodes	2 primaries with replicas	memory fits one node; two for throughput and availability (assumed 100,000 simple operations per second per node)

Cache sizing for an assumed product catalogue.

The design consequence is that the database is sized for 8,000 reads per second at the expected hit ratio, but must survive a short period at 40,000 per second if the cache is lost; mitigation of that case (warm-up, load shedding, request coalescing) is discussed under cold-start outages below. A cache that is required for the database to survive is an availability dependency, not an optimisation.

1.7.3 Hands-on

An LRU cache and the effect of cache size

The program implements an LRU cache with `LinkedHashMap` in access-order mode, generates 1,000,000 requests from a Zipf distribution over 100,000 keys, and reports the hit ratio for several cache sizes.

LruDemo.java JAVA

```java
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Random;

public class LruDemo {
    /** LRU cache: a LinkedHashMap in access order evicts the eldest entry past capacity. */
    static final class LruCache<K, V> extends LinkedHashMap<K, V> {
        private final int capacity;

        LruCache(int capacity) {
            super(16, 0.75f, true);          // accessOrder = true
            this.capacity = capacity;
        }

        @Override
        protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
            return size() > capacity;
        }
    }

    /** Draws a key rank from a Zipf distribution using the cumulative distribution. */
    static int zipfKey(Random rnd, double[] cdf) {
        double u = rnd.nextDouble();
```

LruDemo.java (continued) JAVA

```
        int lo = 0, hi = cdf.length - 1;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (cdf[mid] < u) lo = mid + 1; else hi = mid;
        }
        return lo;
    }

    public static void main(String[] args) {
        int keys = 100_000, requests = 1_000_000;
        double[] cdf = new double[keys];
        double sum = 0;
        for (int i = 0; i < keys; i++) { sum += 1.0 / (i + 1); cdf[i] = sum; }   // Zipf, s = 1
        for (int i = 0; i < keys; i++) cdf[i] /= sum;

        System.out.println("keys=" + keys + " requests=" + requests + " zipf s=1.0");
        System.out.println("cache size (% of keys) | LRU hit ratio");
        for (double pct : new double[] {0.5, 1, 2, 5, 10, 20}) {
            int cap = (int) (keys * pct / 100.0);
            LruCache<Integer, Integer> cache = new LruCache<>(cap);
            Random rnd = new Random(42);
            long hits = 0;
            for (int i = 0; i < requests; i++) {
                int k = zipfKey(rnd, cdf);
                if (cache.get(k) != null) hits++; else cache.put(k, k);
            }
            System.out.printf("%6.1f%% (%6d entries) | %.1f%%%n",
                    pct, cap, 100.0 * hits / requests);
        }
        LruCache<Integer, Integer> cache = new LruCache<>(keys / 5);   // uniform keys, 20% cache
        Random rnd = new Random(42);
        long hits = 0;
        for (int i = 0; i < requests; i++) {
            int k = rnd.nextInt(keys);
            if (cache.get(k) != null) hits++; else cache.put(k, k);
        }
        System.out.printf("uniform keys, 20%% cache: %.1f%%%n", 100.0 * hits / requests);
    }
}
```

Run (JDK 21; the output is deterministic for the fixed seed) TERMINAL

```
$ java LruDemo.java
keys=100000 requests=1000000 zipf s=1.0
cache size (% of keys) | LRU hit ratio
   0.5% (   500 entries) | 44.0%
   1.0% (  1000 entries) | 50.6%
   2.0% (  2000 entries) | 57.3%
   5.0% (  5000 entries) | 66.4%
  10.0% ( 10000 entries) | 73.5%
  20.0% ( 20000 entries) | 80.6%
uniform keys, 20% cache: 19.9%
```

What to notice: the constructor argument `true` selects access order, so each `get` moves the entry to the most recent position, and `removeEldestEntry` evicts the least recently used one after an insertion. The curve rises quickly and then flattens: the first 1% of capacity yields 50.6% hits, the next 19% adds only 30 more points. The uniform case at 20% capacity gives 19.9%, which shows that the hit ratio comes from skew and not from the cache itself. This class is not thread-safe; a concurrent cache requires locking, striping or a library such as Caffeine.

A cache stampede and single-flight loading

The program starts 200 threads that request one expired key at the same instant, with a database load that takes 100 ms. The naive version lets every thread load the key; the single-flight version lets one thread load while the others wait for its result.

StampedeDemo.java JAVA

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
```

StampedeDemo.java (continued) JAVA

```java
import java.util.function.Function;

public class StampedeDemo {
    static final AtomicInteger dbLoads = new AtomicInteger();

    static String loadFromDb(String key) {
        dbLoads.incrementAndGet();
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        return "value-of-" + key;
    }

    // Naive cache-aside: every thread that observes a miss loads from the database.
    static final ConcurrentHashMap<String, String> naive = new ConcurrentHashMap<>();

    static String naiveGet(String key) {
        String v = naive.get(key);
        if (v == null) { v = loadFromDb(key); naive.put(key, v); }
        return v;
    }

    // Single flight: concurrent misses for one key share one in-flight load.
    static final ConcurrentHashMap<String, CompletableFuture<String>> inflight =
            new ConcurrentHashMap<>();
    static final ConcurrentHashMap<String, String> guarded = new ConcurrentHashMap<>();

    static String singleFlightGet(String key) {
        String v = guarded.get(key);
        if (v != null) return v;
        CompletableFuture<String> mine = new CompletableFuture<>();
        CompletableFuture<String> existing = inflight.putIfAbsent(key, mine);
        if (existing != null) return existing.join();       // wait for the leader's load
        try {
            v = loadFromDb(key);
            guarded.put(key, v);
            mine.complete(v);
            return v;
        } finally {
            inflight.remove(key);
        }
    }

    static int run(Function<String, String> getter) throws Exception {
        dbLoads.set(0);
        int threads = 200;
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        CountDownLatch start = new CountDownLatch(1), done = new CountDownLatch(threads);
        for (int i = 0; i < threads; i++) {
            pool.submit(() -> {
                try { start.await(); getter.apply("hot-key"); } catch (InterruptedException e) { }
                done.countDown();
            });
        }
        start.countDown();
        done.await();
        pool.shutdown();
        return dbLoads.get();
    }

    public static void main(String[] args) throws Exception {
        System.out.println("200 concurrent requests for one expired key, database latency 100 ms");
        System.out.println("naive cache-aside   : database loads = "
                + run(StampedeDemo::naiveGet));
        System.out.println("single-flight guard : database loads = "
                + run(StampedeDemo::singleFlightGet));
    }
}
```

Run (JDK 21) TERMINAL

```
$ java StampedeDemo.java
200 concurrent requests for one expired key, database latency 100 ms
naive cache-aside   : database loads = 200
single-flight guard : database loads = 1
```

What to notice: `putIfAbsent` on the in-flight map elects exactly one leader per key; the other threads block on the leader's `CompletableFuture`. The load count drops from 200 to 1. The pattern works within one process; across many application instances, each instance still issues one load, so 50 instances produce 50 loads. A distributed lock with a short lease, or the probabilistic early refresh described below, reduces that further.

1.7.4 Real-world problems and solutions

Problem 1: cache stampede on expiry of a hot key

Scenario. A product page for a promoted item receives 8,000 requests per second. Its cache entry has a 10-minute TTL. **Symptoms.** Every 10 minutes the database CPU spikes to saturation for several seconds, request latency rises from 3 ms to seconds, and a share of requests fail with timeouts. **Diagnosis.** The spike period equals the TTL; the database log shows hundreds of identical queries for the same key in the same second. **Root cause.** When the entry expires, all concurrent requests miss, and each independently loads the same row. The cost of one regeneration, multiplied by the request rate during regeneration, exceeds database capacity. **Fix.** Coalesce loads per key (single flight in the instance, and a short-lease lock in the cache for cross-instance coordination). Serve the stale value while one request refreshes it (stale-while-revalidate). Use probabilistic early expiration: each reader recomputes with a probability that rises as the entry approaches expiry, so the refresh happens before the entry disappears. Add TTL jitter for populations of keys. **Prevention.** Identify hot keys by monitoring, and give them refresh-ahead loading by a background job instead of expiry on demand.

Problem 2: cache and database disagree after a concurrent read and write

Scenario. Cache-aside with delete-on-write. A reader and a writer touch the same key at the same time. **Symptoms.** A user updates a profile, receives a success response, and sees the old value for minutes until the TTL expires. The database holds the new value. **Diagnosis.** Comparing the cache entry with the row shows the older version in the cache. The timeline in the log reveals the interleaving below. **Root cause.** The reader's fill is not atomic with the database read, so a stale value can be written to the cache after the writer's delete. **Fix.** Keep a TTL as a safety net on every entry. Apply a delayed second delete (delete again 500 ms to 1 s after the write, for example through a queue), versioned writes (store the row version in the entry and refuse to overwrite a newer version with an older one), or leases (the cache issues a token on a miss and rejects a fill whose token was invalidated by a delete). For data that must not be stale, read from the database. **Prevention.** Document the staleness bound for each cached entity, and do not cache data whose invariants cannot tolerate that bound.

Interleaving that leaves a stale entry

```
t1  Reader : cache miss for item:42
t2  Reader : SELECT price FROM items WHERE id = 42  -> 100 (old)
t3  Writer : UPDATE items SET price = 120 WHERE id = 42
t4  Writer : DEL item:42
t5  Reader : SET item:42 = 100            <- stale value written after the delete
t6  Later reads return 100 until the TTL expires
```

Problem 3: cache penetration by requests for nonexistent keys

Scenario. An endpoint `GET /users/{id}` is cached by id. A crawler requests random ids that do not exist. **Symptoms.** The cache hit ratio falls from 95% to 40%, and the database receives a high rate of lookups that all return no rows. **Diagnosis.** The cache miss log shows ids outside the valid range; the database queries return empty results. **Root cause.** A miss for a nonexistent key is never cached, so every such request reaches the database. **Fix.** Cache the negative result with a short TTL (30 to 60 seconds). For large or unbounded key spaces, place a Bloom filter of existing ids in front of the cache and reject ids the filter reports absent (the filter never produces a false negative; see the chapter on common infrastructure components). Validate id format and range at the edge, and rate limit by client. **Prevention.** Treat the not-found case as a first-class cached outcome and monitor the ratio of misses that resolve to empty results.

Problem 4: cache avalanche after a restart or mass expiry

Scenario. A cache cluster is restarted for a version upgrade, or 100,000 keys loaded together at deployment share one TTL. **Symptoms.** Immediately after the restart (or at the common expiry instant) the database receives the full read traffic, latency rises, the connection pool is exhausted, and the service enters a failure that persists after the cache starts to fill, because slow responses cause client retries. **Diagnosis.** The cache hit ratio is near zero at the start of the incident; database QPS equals the total read QPS. **Root cause.** The database was sized for the post-cache load; with the cache empty the load exceeds capacity, and retries amplify it. **Fix.** Shed load at the service (reject or degrade a share of requests) while the cache warms; warm the cache from the top keys before routing traffic; restart cache nodes one at a time so that only a fraction of keys is cold; replicate the cache so a node failure does not lose its keys; and randomise TTLs. **Prevention.** Load test the database at the cold-cache rate, and keep a documented procedure for warm-up.

Problem 5: a hot key overloads one cache node

Scenario. A single key (a celebrity profile, a flash-sale item) receives 60% of the traffic of the cache cluster. **Symptoms.** One cache node reaches 100% CPU or network saturation while the others are idle; latency on that node rises and keys hosted by it become slow. **Diagnosis.** Per-node metrics show an imbalance; key-level sampling (for example Redis `--hotkeys` with an LFU policy, or client-side counters) identifies the key. **Root cause.** A key maps to exactly one node, so its entire load lands there, and adding nodes does not help. **Fix.** Replicate the hot key into a small in-process cache with a short TTL on every application instance, or store it under several derived keys (`item:42#0` to `item:42#7`) and read a random one. Add read replicas for the owning shard. **Prevention.** Monitor key-level skew and plan for known hot items in advance (a promotion calendar).

1.7.5 HTTP caching and CDNs

HTTP caching moves the cache to the client and to intermediaries, controlled by response headers.

Header or directive	Meaning
`` `Cache-Control: max-age=N` ``	the response is fresh for N seconds in any cache
`` `Cache-Control: s-maxage=N` ``	like `max-age`, but applies to shared caches (CDN, proxy) only
`` `Cache-Control: no-cache` ``	the response may be stored but must be revalidated with the origin before each reuse
`` `Cache-Control: no-store` ``	the response must not be stored (sensitive data)
`` `Cache-Control: private` / `public` ``	`private` restricts storage to the user agent; `public` permits shared caches
`` `Cache-Control: immutable` ``	the content never changes during its freshness lifetime; no revalidation on reload
`` `Cache-Control: stale-while-revalidate=N` ``	after expiry, serve the stale response for up to N seconds while refreshing in the background
`` `ETag` / `If-None-Match` ``	validator; if the representation is unchanged the origin returns 304 Not Modified with no body
`` `Last-Modified` / `If-Modified-Since` ``	time-based validator with the same 304 behaviour
`` `Vary` ``	names request headers (for example `Accept-Encoding`) that select among stored variants

Common caching headers.

Static assets are served with a content hash in the file name and `Cache-Control: public, max-age=31536000, immutable`; a deployment changes the file name, which is the versioned-key invalidation strategy applied to HTTP. HTML and API responses use short `max-age` or `no-cache` with an ETag, so that revalidation costs one round trip and no body. A CDN adds the shared layer: `s-maxage` sets the CDN lifetime independently of the browser, and a purge API removes an object, with propagation delays that the design must tolerate. Personalised responses must carry `private` or `Vary` on the discriminating header, otherwise a shared cache may serve one user's response to another, a frequent source of data leaks.

1.7.6 Questions to expect

1. **Compare cache-aside and write-through.** In cache-aside the application loads and fills on a miss and deletes on write; the cache holds only requested data and a cache outage degrades to database reads. Write-through writes to both synchronously, so the cache is consistent with acknowledged writes but write latency is higher and unread data occupies memory.
2. **Why delete the cache entry on write instead of updating it?** Two concurrent writers can update the cache in the opposite order from the database, leaving an old value indefinitely; a delete lets the next read load the current row. A delete still has a race with a concurrent read fill, so a TTL remains as a safety net.
3. **What is a cache stampede and how is it prevented?** Many concurrent misses for one expired key all load from the database. Prevent it with single-flight loading per key, stale-while-revalidate, probabilistic early refresh and TTL jitter.
4. **Distinguish penetration, stampede and avalanche.** Penetration: requests for keys that do not exist bypass the cache. Stampede: one hot key expires and many requests regenerate it. Avalanche: many keys expire or the cache is lost at once, so the database receives the full load.
5. **How is the cache size estimated?** Estimate the working set (items x entry size), choose the share to hold (80/20 as a starting point, then the measured hit-ratio curve), add headroom for overhead, and compute the database load as the miss ratio times the read rate.
6. **LRU or LFU?** LRU adapts quickly to changing popularity but is vulnerable to scans; LFU retains consistently popular keys and resists scans but adapts slowly without decay. W-TinyLFU combines both and is a strong default for local caches.
7. **How are caches kept consistent across service instances?** Local caches use short TTLs and event-driven invalidation (a broadcast or CDC event); a distributed cache gives one shared copy. State the acceptable staleness explicitly.
8. **What should not be cached?** Data whose staleness violates an invariant (account balances during a transfer, inventory at purchase time), highly personalised responses with a low reuse rate, and data that is cheap to compute.
9. **What happens when the cache goes down?** Requests fall through to the database. The design must either keep the database able to serve that load, or shed load and degrade, and must avoid the retry amplification that turns a partial outage into a full one.

KEY POINTS

- Hit ratio is a property of the key distribution; measure or state it, and size the database for the miss traffic and for the cold-cache case.
- Cache-aside with delete-on-write and a TTL is the default; the read-fill race means every entry also needs a staleness bound.
- Stampede (one hot key), penetration (absent keys) and avalanche (mass expiry or loss) have distinct fixes: single flight, negative caching with a Bloom filter, and jitter plus warm-up plus load shedding.
- Hot keys defeat sharding; replicate them locally or split them into derived keys.
- HTTP caching relies on versioned URLs for assets and validators (ETag) for dynamic responses; mark personalised responses `private`.

1.8 Replication, Partitioning and Consistent Hashing

A single database node limits a system in three ways: it is a single point of failure, its read and write capacity is bounded, and its storage is bounded. **Replication** keeps copies of the same data on several nodes, which addresses availability and read capacity. **Partitioning** (sharding) divides the data set among nodes so that each holds a subset, which addresses write capacity and storage. Large systems use both: the data is partitioned, and each partition is replicated.

This chapter treats the placement problem as a design decision. The replication mechanics inside a particular database (the write-ahead log, logical decoding, replication slots) and the physical sharding features of particular products are covered in the databases material; the focus here is on topologies, the anomalies they introduce, the partitioning strategies, consistent hashing, and the operational problems of rebalancing and resharding.

1.8.1 Concept and mechanism

Replication topologies

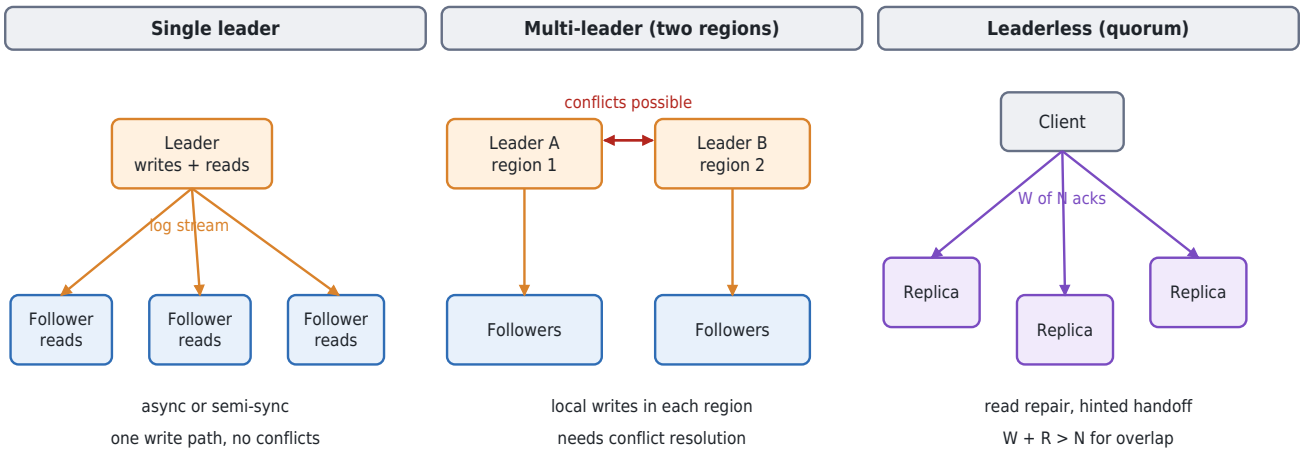


Figure 1.33. The three replication topologies: single leader (one write path), multi-leader (a write path per region, with conflicts) and leaderless (client-coordinated quorum writes).

Single leader (leader and followers). All writes go to the leader, which appends them to a replication log; followers apply the log in order. Reads may be served by followers. Replication is **synchronous** when the leader waits for the follower to confirm before acknowledging the client, **asynchronous** when it does not, and **semi-synchronous** when it waits for at least one follower. Synchronous replication removes the loss window but couples availability and latency to the slowest awaited follower; asynchronous replication keeps the leader fast but can lose acknowledged writes if the leader fails before the followers receive them.

Failover (promoting a follower after a leader failure) has three well-known hazards. First, an asynchronous follower may lack the last acknowledged writes, which are then lost or must be reconciled. Second, if the former leader returns and still accepts writes, two leaders exist (split brain; see the chapter on consistency and coordination). Third, a timeout that is too short triggers failover on a transient pause and causes more damage than it prevents.

Multi-leader. Each region has a leader that accepts local writes and replicates to the other leaders asynchronously. The benefits are low write latency in every region and survival of a region failure. The cost is that two leaders can accept conflicting writes to the same record. Conflict resolution strategies are last-writer-wins by timestamp (simple, discards data and depends on synchronised clocks), application-specific merge, conflict-free replicated data types (CRDTs) for structures with a mathematically commutative merge such as counters and sets, and avoiding conflicts by routing each record to one home region.

Leaderless. Any replica accepts a write. A client (or a coordinator) sends the write to N replicas and considers it successful after W acknowledgements; a read contacts R replicas and returns the newest version. If W + R > N, every read set intersects every write set (see the chapter on consistency and coordination for the quorum arithmetic). Replicas that missed a write are repaired by **read repair** (the reader writes the newest version back to stale replicas), **hinted handoff** (a node stores writes destined for an unavailable peer and delivers them later) and **anti-entropy** (background comparison of replicas, often with Merkle trees that identify differing key ranges cheaply). This is the model of the Dynamo paper and of systems derived from it.

Replication lag and its anomalies

With asynchronous replication a follower lags the leader by a variable delay: typically milliseconds, but seconds or more during load spikes, long transactions on the follower or network problems. A client that reads from followers can observe the following anomalies.

Anomaly	Description	Typical remedy
Read-your-writes violation	a user writes, then reads from a follower that has not applied the write, and sees the old value	route the user's reads to the leader for a window after a write, or track the write position and read only from a follower that has reached it
Monotonic reads violation	successive reads hit different followers, and the second returns older data than the first	pin a user or session to one follower

Anomaly	Description	Typical remedy
Consistent prefix violation	causally related writes (a question and its answer) are seen out of order because they were applied on different partitions	keep causally related writes in one partition, or carry causal metadata

Anomalies caused by replication lag.

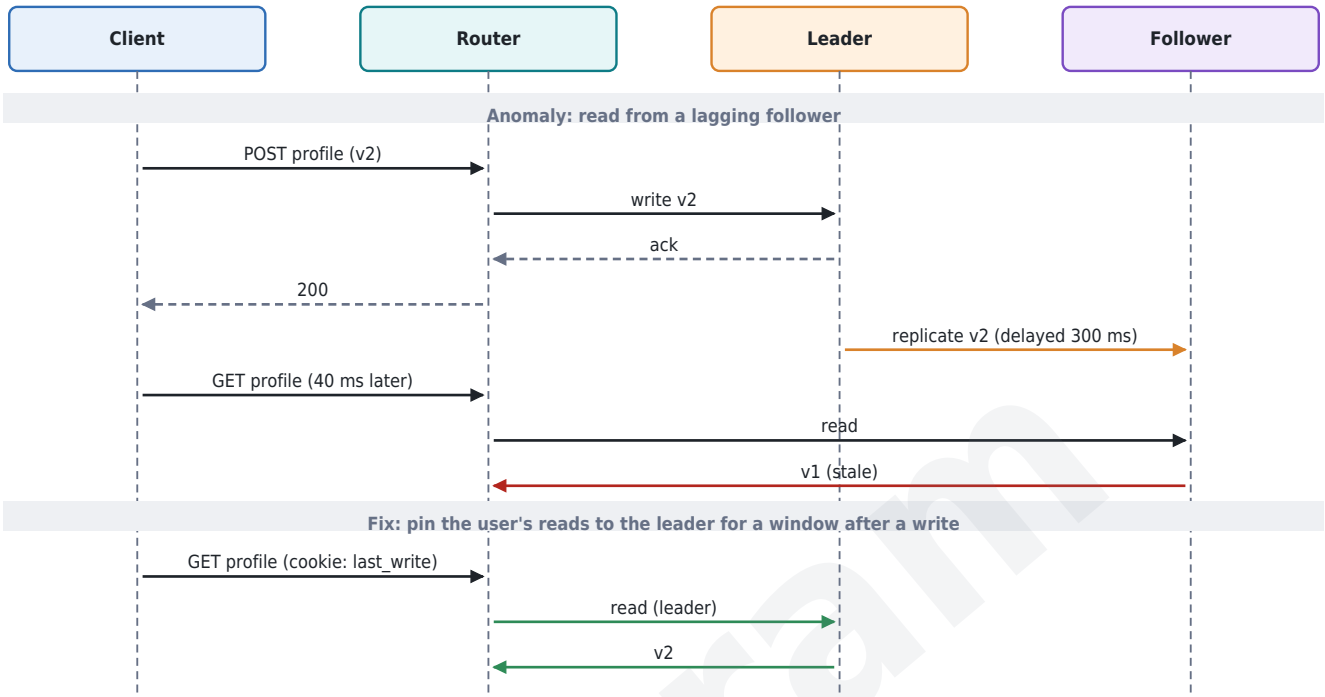


Figure 1.34. A read-your-writes anomaly: the follower returns the old version because replication is delayed; routing the user's reads to the leader after a write removes the anomaly.

Partitioning strategies

Partitioning assigns each record to one of several partitions by a **partition key**. The choice of key and of mapping function determines load balance, query cost and the ease of rebalancing.

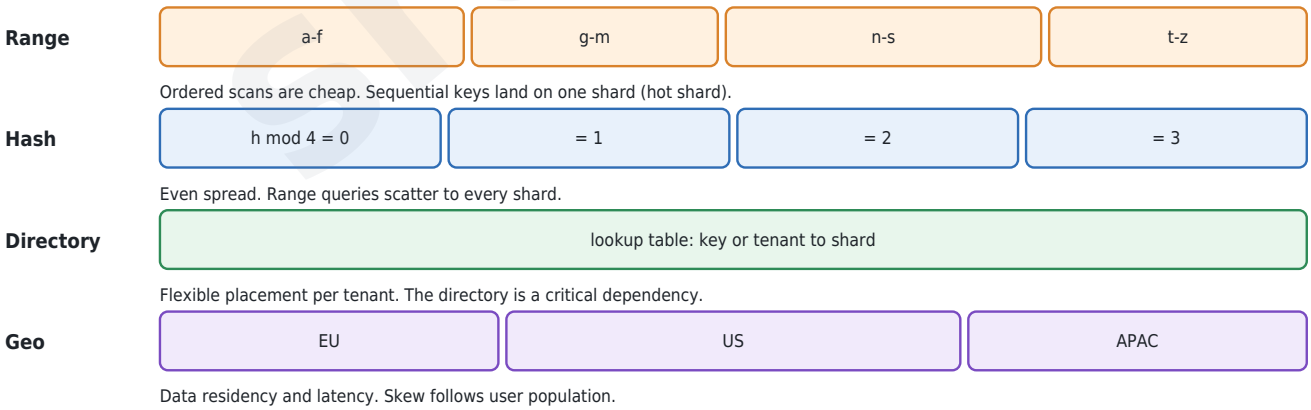


Figure 1.35. Four mappings from key to shard; each trades range query efficiency, load balance and operational flexibility differently.

- **Range partitioning** assigns contiguous key ranges to shards. Range scans and ordered traversal are efficient, and a range can be split when it grows. The risk is skew: sequential keys (timestamps, auto-increment ids) concentrate all current writes on the shard that owns the latest range.
- **Hash partitioning** applies a hash function to the key and maps the result to a shard. The distribution is even, but adjacent keys are scattered, so range queries must contact every shard.
- **Directory-based partitioning** keeps an explicit lookup table from key (or tenant) to shard. It permits arbitrary placement, such as moving one large tenant to a dedicated shard, at the cost of a highly available lookup service.

- **Geographic partitioning** assigns data to regions for latency and data residency rules. Load follows the distribution of users and can be skewed.

The partition key should satisfy four criteria: high cardinality (many distinct values), even access frequency, inclusion in the most frequent queries (so that a query targets one shard), and stability (a key that changes forces a move between shards). User id, tenant id and order id are typical choices; country, status and boolean flags are poor ones.

Consistent hashing

The simple mapping `shard = hash(key) mod N` fails when N changes: nearly every key maps to a different shard, so adding one node to a cluster of N forces about N/(N+1) of the data to move and invalidates nearly all cache entries at once. **Consistent hashing** limits the movement. Both nodes and keys are hashed onto the same circular space (for example 0 to 2^64 - 1). A key belongs to the first node found moving clockwise from the key's position. Adding a node takes over the arc preceding its position from one neighbour; removing a node hands its arc to the next node. On average only 1/N of the keys move when a node is added to a cluster of N - 1 nodes.

With one position per physical node, arcs have very unequal lengths and load is unbalanced. The standard remedy is **virtual nodes**: each physical node is hashed to many positions (tens to hundreds), so that it owns many small arcs, the load averages out, and a joining node takes small arcs from every existing node, which spreads the transfer work across the cluster. Heterogeneous nodes receive a number of virtual nodes proportional to their capacity.

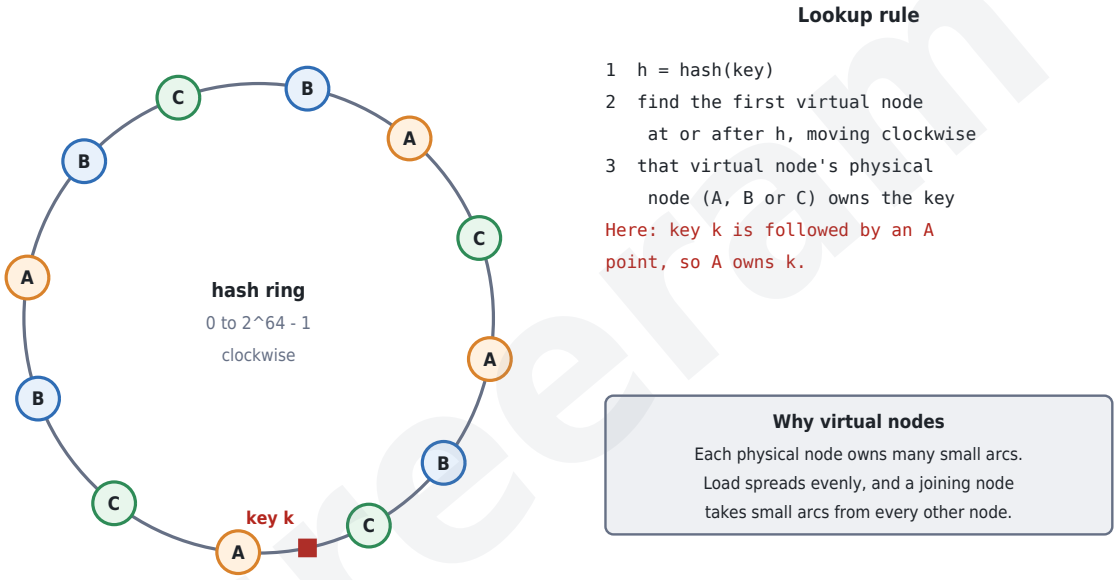


Figure 1.36. A hash ring with twelve virtual nodes on three physical nodes; a key is owned by the first virtual node clockwise from its hash position.

Related schemes serve the same purpose. **Rendezvous (highest random weight) hashing** scores every node for each key and selects the highest score; it needs no ring state and balances well, but the lookup costs O(N). **Fixed logical partitions** (used, for example, by Redis Cluster with 16,384 hash slots, and by Kafka partitions and Elasticsearch shards) hash keys to a fixed, large number of partitions and keep a table assigning partitions to nodes; rebalancing moves whole partitions, and the key-to-partition mapping never changes.

Rebalancing

Rebalancing moves data when nodes are added, removed or overloaded. Three approaches are common.

1. **Fixed number of partitions.** Create many more partitions than nodes (for example 1,024 partitions on 8 nodes). A new node steals whole partitions from existing nodes. The partition count must be chosen in advance, since it limits the maximum cluster size and the minimum partition size.
2. **Dynamic splitting.** Partitions split when they exceed a size threshold and merge when they shrink, as in range-partitioned stores. The number of partitions adapts to data volume, at the cost of more complex metadata.
3. **Proportional to nodes.** A fixed number of partitions per node, with new nodes splitting existing partitions randomly (the virtual node approach).

Rebalancing is expensive: it moves data across the network, competes with foreground traffic, and may run while a node is already overloaded. Automatic rebalancing triggered by a transient failure can aggravate the failure. Many systems therefore require operator confirmation, or throttle the transfer rate.

Cross-shard operations

When the data is partitioned, queries and transactions that span partitions become expensive.

- **Queries without the partition key** use scatter-gather: the request goes to every shard and the results are merged. Latency is that of the slowest shard, so the tail latency of the query is worse than that of any shard, and capacity does not scale with shard count because every query consumes work on every shard.
- **Secondary indexes** are either **local** (each shard indexes its own data; a query by the indexed attribute is scattered) or **global** (a separate index partitioned by the indexed attribute; a query contacts one index partition, but writes update an index on another shard, usually asynchronously).
- **Joins** across shards are avoided by denormalising (embedding the needed attributes), by co-locating related tables on the same partition key (all rows of a customer on one shard), or by broadcasting a small table to every shard.
- **Transactions** across shards require two-phase commit or a saga (see the chapter on consistency and coordination), both of which cost latency and availability. The better design chooses the partition key so that a business transaction stays within one shard.

Resharding without downtime

Changing the partitioning scheme or the number of shards while serving traffic follows a fixed sequence: copy, verify, cut over, clean up.

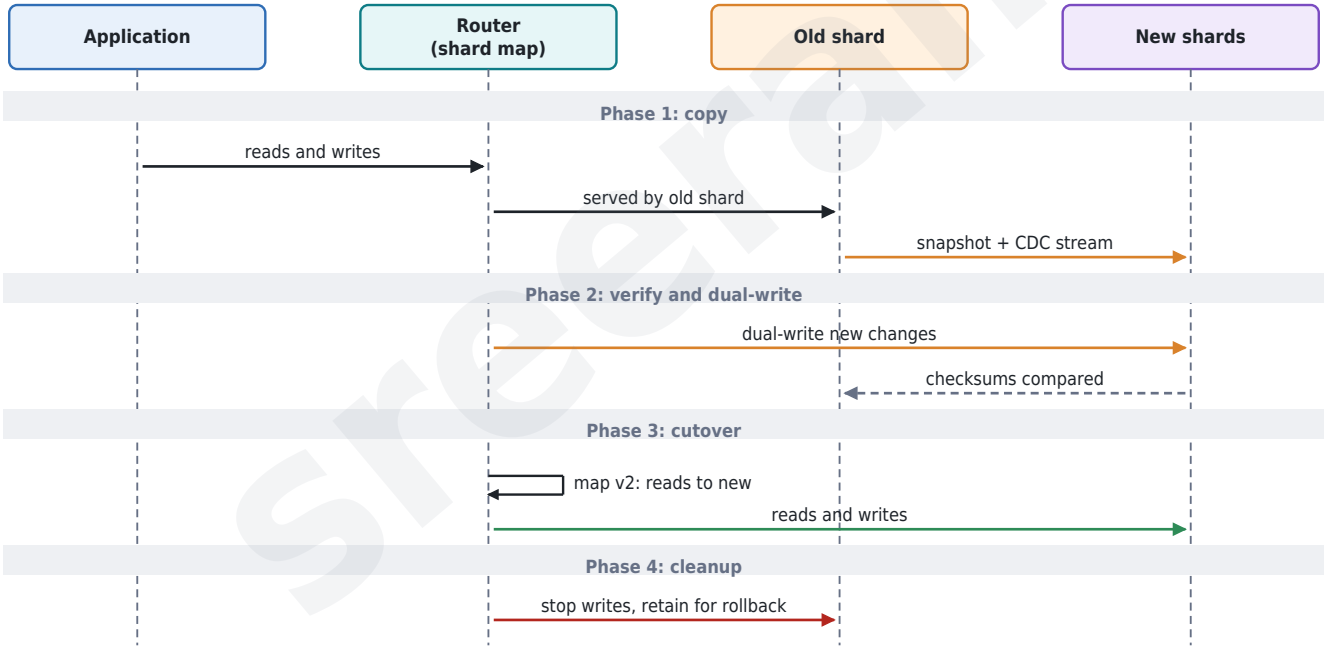


Figure 1.37. Online resharding in four phases: bulk copy plus change stream, dual-write and verification, a versioned cutover of the routing map, and delayed cleanup that keeps the old shard available for rollback.

The router holds a versioned shard map, so the cutover is an atomic change of one record. Reads move first or together with writes, depending on whether the new shard has verified as complete. Rollback is possible until the old shard is decommissioned, which is why the cleanup is delayed.

1.8.2 Worked example

The shard count for an assumed order store is derived from capacity limits and growth, then rounded to a power of two of logical partitions.

Quantity	Value	Arithmetic
`Data today`	6 TB	assumed
`Data in three years`	18 TB	assumed 3x growth
`Comfortable data per node`	1 TB	assumed (backup and restore time, buffer cache fit)

Quantity	Value	Arithmetic
Nodes needed for storage in three years	18	18 TB / 1 TB
Peak writes today	12,000 per second	assumed
Peak writes in three years	36,000 per second	3x growth
Sustainable writes per primary	3,000 per second	assumed, with headroom
Nodes needed for writes in three years	12	36,000 / 3,000
Binding constraint	storage, 18 nodes	max(18, 12)
Logical partitions	64	next power of two above 18 x 3, so each node can later hold fewer partitions
Initial deployment	8 nodes x 8 partitions	6 TB / 8 nodes = 0.75 TB per node today

Shard sizing for an assumed order store.

The result shows the purpose of fixed logical partitions: the application sees 64 partitions from the first day, and growth to 18 or 32 nodes is a matter of moving whole partitions, not of changing the key mapping. Each node is also replicated (assumed one leader and two followers per partition group), so the physical node count is three times the shard count in a fully replicated deployment; that figure belongs in the cost estimate.

1.8.3 Hands-on

Consistent hashing with virtual nodes

The program places 10 nodes on a ring with a varying number of virtual nodes per node, assigns 100,000 keys, and reports the load balance. It then adds an 11th node and counts the keys that change owner, with and without consistent hashing.

RingDemo.java JAVA

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.*;

public class RingDemo {
    static long hash(String s) throws Exception {
        byte[] d = MessageDigest.getInstance("MD5").digest(s.getBytes(StandardCharsets.UTF_8));
        long h = 0;
        for (int i = 0; i < 8; i++) h = (h << 8) | (d[i] & 0xff);
        return h;
    }

    static TreeMap<Long, String> build(List<String> nodes, int vnodes) throws Exception {
        TreeMap<Long, String> ring = new TreeMap<>();
        for (String n : nodes)
            for (int v = 0; v < vnodes; v++) ring.put(hash(n + "#" + v), n);
        return ring;
    }

    static String owner(TreeMap<Long, String> ring, long h) {
        Map.Entry<Long, String> e = ring.ceilingEntry(h);
        return (e != null ? e : ring.firstEntry()).getValue();
    }

    public static void main(String[] args) throws Exception {
        int keys = 100_000;
        long[] keyHash = new long[keys];
        for (int i = 0; i < keys; i++) keyHash[i] = hash("user:" + i);
        List<String> nodes = new ArrayList<>();
        for (int i = 0; i < 10; i++) nodes.add("node" + i);

        System.out.println("10 nodes, 100000 keys, ideal load per node = 10000");
        System.out.println("vnodes/node | min    max    max/ideal | stddev/mean");
        for (int vn : new int[] {1, 10, 100, 500}) {
            TreeMap<Long, String> ring = build(nodes, vn);
            Map<String, Integer> load = new HashMap<>();
            for (long h : keyHash) load.merge(owner(ring, h), 1, Integer::sum);
            int min = Integer.MAX_VALUE, max = 0;
```

RingDemo.java (continued) JAVA

```java
            double mean = keys / 10.0, var = 0;
            for (String n : nodes) {
                int c = load.getOrDefault(n, 0);
                min = Math.min(min, c); max = Math.max(max, c);
                var += (c - mean) * (c - mean);
            }
            System.out.printf("%11d | %-6d %-6d %-9.2f | %.1f%%%n", vn, min, max, max / mean,
                    100 * Math.sqrt(var / 10) / mean);
        }

        // keys moved when node count changes from 10 to 11
        TreeMap<Long, String> before = build(nodes, 100);
        List<String> more = new ArrayList<>(nodes); more.add("node10");
        TreeMap<Long, String> after = build(more, 100);
        int moved = 0, movedMod = 0;
        for (long h : keyHash) {
            if (!owner(before, h).equals(owner(after, h))) moved++;
            if (Long.remainderUnsigned(h, 10) != Long.remainderUnsigned(h, 11)) movedMod++;
        }
        System.out.println("\nadding an 11th node to 10 nodes:");
        System.out.printf("ring, 100 vnodes: %d keys move (%.1f%%); ideal 1/11 = 9.1%%%n",
                moved, 100.0 * moved / keys);
        System.out.printf("hash mod N: %d keys move (%.1f%%)%n",
                movedMod, 100.0 * movedMod / keys);
    }
}
```

Run (JDK 21; deterministic because MD5 hashing is deterministic) TERMINAL

```
$ java RingDemo.java
10 nodes, 100000 keys, ideal load per node = 10000
vnodes/node | min    max    max/ideal | stddev/mean
          1 | 66     33828  3.38      | 102.7%
         10 | 5269   14259  1.43      | 32.3%
        100 | 8550   11684  1.17      | 9.1%
        500 | 9364   11315  1.13      | 6.2%

adding an 11th node to 10 nodes:
ring, 100 vnodes: 9074 keys move (9.1%); ideal 1/11 = 9.1%
hash mod N: 90708 keys move (90.7%)
```

What to notice: with one position per node the most loaded node holds 3.38 times the ideal share and one node holds 66 keys; with 100 virtual nodes the spread falls to a standard deviation of 9.1% of the mean, and with 500 to 6.2%, with diminishing returns as the number of virtual nodes grows. When the 11th node joins, the ring moves 9,074 keys, matching the ideal 1/11 = 9.1%, while `hash mod N` moves 90.7%. `TreeMap.ceilingEntry` is the clockwise lookup; the wrap-around to the first entry closes the ring.

Hot shards, rebalancing and read-your-writes

The two programs below are single-process simulations: they model the distribution of keys and replication delays; they do not measure a real database.

shard.py PYTHON

```python
import bisect, hashlib, random

N = 4
BOUNDS = [250_000, 500_000, 750_000]            # range shards over ids 0..999,999, split in advance


def h(key):
    return int(hashlib.md5(str(key).encode()).hexdigest(), 16)


def range_shard(k):
    return bisect.bisect_right(BOUNDS, k)


def hash_shard(k):
    return h(k) % N


# 1. Monotonic keys: the latest 20,000 auto-increment ids
```

shard.py (continued) PYTHON

```python
for name, f in [("range", range_shard), ("hash", hash_shard)]:
    counts = [0] * N
    for k in range(900_000, 920_000):
        counts[f(k)] += 1
    print(f"{name:5s} sharding, 20,000 sequential inserts -> writes per shard {counts}")

# 2. Rebalancing with hash mod N when a fifth shard is added
moved = sum(1 for k in range(200_000) if h(k) % 4 != h(k) % 5)
print(f"hash mod N, 4 -> 5 shards: {moved} of 200000 keys move ({100 * moved / 200000:.1f}%)")

# 3. Fixed logical partitions: only whole partitions move to the new node
P = 1024
assign = {p: p % 4 for p in range(P)}
stolen = 0
for p in range(P):
    if stolen < P // 5 and p % 5 == 0:
        assign[p] = 4
        stolen += 1
sizes = [sum(1 for v in assign.values() if v == n) for n in range(5)]
print(f"1024 fixed partitions, 4 -> 5 nodes: {stolen} move ({100 * stolen / P:.1f}%)")
print("  partitions per node afterwards:", sizes)

# 4. A hot key on hash shards, with and without splitting the key
random.seed(1)
reqs = ["celeb"] * 50_000 + [f"u{random.randrange(10_000)}" for _ in range(50_000)]
c = [0] * N
for r in reqs:
    c[hash_shard(r)] += 1
print("hot key, 100,000 reads, 50% to one key -> per shard", c)
c = [0] * N
for r in reqs:
    if r == "celeb":
        r = f"celeb#{random.randrange(8)}"          # split the hot key into 8 sub-keys
    c[hash_shard(r)] += 1
print("same load, hot key split into 8 sub-keys -> per shard", c)
```

Run (Python 3, deterministic seeds) TERMINAL

```
$ python3 shard.py
range sharding, 20,000 sequential inserts -> writes per shard [0, 0, 0, 20000]
hash  sharding, 20,000 sequential inserts -> writes per shard [5048, 4964, 4945, 5043]
hash mod N, 4 -> 5 shards: 159942 of 200000 keys move (80.0%)
1024 fixed partitions, 4 -> 5 nodes: 204 move (19.9%)
  partitions per node afterwards: [205, 205, 205, 205, 204]
hot key, 100,000 reads, 50% to one key -> per shard [12774, 12310, 62118, 12798]
same load, hot key split into 8 sub-keys -> per shard [25259, 18582, 24816, 31343]
```

What to notice: with range partitioning, the 20,000 newest sequential inserts all land on the last shard, while hash partitioning spreads them within about 1% of equal. Moving from 4 to 5 shards with `mod N` relocates 80% of keys (the general value is N/(N+1) for adding one shard, here 4/5); fixed logical partitions relocate 19.9%, close to the ideal 1/5. A hot key defeats hash partitioning: the shard owning it receives 62,118 of 100,000 reads; splitting the key into eight sub-keys reduces the maximum to 31,343.

repl.py PYTHON

```python
import random

random.seed(9)


class Cluster:
    """One leader and two followers; asynchronous replication with random lag (ms)."""

    def __init__(self, lag):
        self.lag, self.leader, self.fol, self.pending = lag, {}, [{}, {}], []

    def write(self, t, key, value):
        self.leader[key] = (value, t)
        for i in range(2):
            self.pending.append((t + random.randint(*self.lag), i, key, (value, t)))

    def tick(self, t):
        keep = []
```

repl.py (continued) PYTHON

```
        for due, i, key, val in self.pending:
            if due <= t:
                if key not in self.fol[i] or self.fol[i][key][1] <= val[1]:
                    self.fol[i][key] = val
            else:
                keep.append((due, i, key, val))
        self.pending = keep


def trial(policy):
    c, stale, reads, last_write = Cluster((5, 400)), 0, 0, None
    for t in range(2000):
        c.tick(t)
        if t % 100 == 0:                              # the user updates the profile
            c.write(t, "profile", f"v{t}")
            last_write = t
        if t % 50 == 25 and last_write is not None:   # the same user reads the profile
            reads += 1
            if policy == "random follower":
                src = c.fol[random.randrange(2)]
            elif policy == "leader for 500 ms after own write":
                src = c.leader if t - last_write < 500 else c.fol[random.randrange(2)]
            else:
                src = c.leader
            v = src.get("profile")
            if v is None or v[1] < last_write:
                stale += 1
    return stale, reads


for p in ["random follower", "leader for 500 ms after own write", "always leader"]:
    s, r = trial(p)
    print(f"{p:36s} stale reads of own write: {s:2d} of {r}")
```

Run (Python 3, deterministic seeds) TERMINAL

```
$ python3 repl.py
random follower                      stale reads of own write: 36 of 40
leader for 500 ms after own write    stale reads of own write:  0 of 40
always leader                        stale reads of own write:  0 of 40
```

What to notice: with random lag between 5 and 400 ms, a read 25 ms after a write usually reaches a follower that has not applied it (36 of 40 reads are stale). Routing a user's reads to the leader for 500 ms after the user's own write removes the anomaly at the cost of leader load that is proportional to the write rate, not to the read rate.

1.8.4 Real-world problems and solutions

Problem 1: hot shard from a monotonic partition key

Scenario. An events table is range-partitioned by `created_at` (or an auto-increment id) across 8 shards. **Symptoms.** One shard runs at 95% CPU and write latency of hundreds of milliseconds, while seven shards are idle. Adding shards does not help. **Diagnosis.** Per-shard write counters show that the newest range receives all inserts; the key distribution of recent writes is concentrated at the high end. **Root cause.** With a monotonic key, the current write traffic always falls into the last range, and range partitioning places that range on a single shard. **Fix.** Hash the key (or prefix it with a hash bucket, for example `bucket = hash(user_id) mod 16`, key `bucket:timestamp`), so writes spread; keep time-range queries by querying each bucket in parallel. Alternatively partition by `user_id` and sort by time within the partition. **Prevention.** Examine the write distribution for every proposed partition key, and avoid time or sequence as the leading component of a range-partitioned key.

Problem 2: rebalancing storm after adding a node

Scenario. A cluster of 12 nodes at 70% disk use gains a 13th node, and automatic rebalancing starts immediately. **Symptoms.** Network and disk I/O saturate, foreground latency doubles, some nodes are marked unhealthy because they miss heartbeats, and the failure detector triggers further rebalancing. **Diagnosis.** Transfer throughput equals the link capacity; the rebalancer log shows concurrent moves of many partitions per node; health checks time out during the transfer. **Root cause.** Unthrottled data movement competes with foreground traffic, and the failure detector interprets the slowdown as node failure, which creates more movement (a positive feedback loop). **Fix.** Throttle the rebalance rate and the number of concurrent partition moves per node, run it off-peak, keep the failure-detector timeout longer than the expected stall, and use fixed partitions so the movement is bounded to about 1/13 of the data. **Prevention.** Test node addition under load before production, and add capacity before disks reach 70% instead of at 90%.

Problem 3: users see old data after saving

Scenario. Profile pages are served from follower replicas; profile edits go to the leader. **Symptoms.** Users report that a saved change does not appear after redirect, then appears after a refresh; duplicate edits follow. **Diagnosis.** Replication lag metrics show lag between 100 ms and several seconds at peak; the redirected GET is served by a lagging follower. **Root cause.** The read-your-writes guarantee is not provided by plain follower reads. **Fix.** After a write, route that user's reads to the leader for a window longer than the typical lag (for example 5 seconds, tracked in a cookie or session), or carry the log position of the write and let only followers that have reached it serve the read. **Prevention.** Monitor lag as an SLI, and define in the design which reads require read-your-writes.

Problem 4: scatter-gather queries dominate latency

Scenario. Orders are sharded by `order_id`; the support tool searches orders by customer email. **Symptoms.** Each search contacts all 32 shards; p50 is 60 ms but p99 is 900 ms, and one slow shard dominates. **Diagnosis.** Query traces show fan-out to 32 shards with the slowest shard defining total latency. **Root cause.** The query does not include the partition key, and the tail latency of a fan-out is the maximum over the shards. **Fix.** Build a global secondary index (email to order ids) or maintain a search index fed by CDC; use hedged requests with a deadline for the remaining fan-out. **Prevention.** Enumerate the top queries before selecting the partition key, and verify that each targets one shard or has a purpose-built index.

Problem 5: a celebrity key overloads one shard

Scenario. A hash-sharded counter or profile store has one key that receives more than half of the traffic. **Symptoms.** One shard saturates while the average utilisation is low; its replicas lag. **Diagnosis.** Key-level access sampling shows a single key above 50% of requests, as in the simulation above. **Root cause.** Hashing balances keys, not requests; the load of one key cannot be divided by partitioning. **Fix.** Split the key (write to a random sub-key and aggregate on read for counters), replicate the hot key to many read replicas or local caches, and cache it at the CDN or application layer. **Prevention.** Identify entities with unbounded popularity in advance (celebrities, trending items) and design a special path for them.

1.8.5 Questions to expect

1. **Replication versus partitioning?** Replication copies the same data to several nodes for availability and read scaling; partitioning splits the data among nodes for write and storage scaling. Production systems combine them: each partition has its own replicas.
2. **Synchronous versus asynchronous replication?** Synchronous waits for follower confirmation, so no acknowledged write is lost, at the cost of latency and availability coupling. Asynchronous acknowledges first, so it is faster but can lose recent writes on leader failure. Semi-synchronous (one synchronous follower) is a common compromise.
3. **What is replication lag and which anomalies does it cause?** The delay between a write on the leader and its application on a follower. It causes read-your-writes, monotonic-read and consistent-prefix violations, addressed by leader reads after writes, session pinning and keeping related data in one partition.
4. **Why consistent hashing instead of hash mod N?** With `mod N`, a change in N remaps almost all keys (N/(N+1) on adding a node). Consistent hashing remaps about 1/N of the keys, so scaling and failures disturb a small part of the data or cache.
5. **What are virtual nodes?** Multiple ring positions per physical node. They smooth the load distribution, spread the rebalancing work across all nodes, and allow weighting by capacity.

6. **How is the shard key selected?** High cardinality, even load, presence in the frequent queries, and stability. User or tenant id is typical; time and low-cardinality attributes are poor leading keys.
7. **How are cross-shard joins and transactions handled?** Prefer a design that avoids them: co-locate related rows by partition key, denormalise, and keep a business transaction in one shard. Where unavoidable, use scatter-gather for reads and two-phase commit or a saga for writes.
8. **How is a shard added without downtime?** Copy a snapshot and stream changes, dual-write and verify, switch the versioned routing map, then keep the old shard for rollback before cleanup.
9. **What happens when the leader fails?** A follower is promoted after failure detection and election. Writes not yet replicated may be lost with asynchronous replication, and the old leader must be fenced to prevent split brain.
10. **When is multi-leader replication justified?** For multi-region write latency or region-failure tolerance with a conflict resolution strategy; otherwise single leader is simpler and safer.

KEY POINTS

- Replicate for availability and read scale; partition for write and storage scale; combine both.
- Asynchronous replication can lose acknowledged writes and serves stale reads; plan read-your-writes explicitly.
- Choose the partition key for cardinality, even load and query locality; avoid monotonic leading keys.
- Consistent hashing with virtual nodes moves about 1/N of the keys on a membership change; hash mod N moves almost all (measured 9.1% versus 90.7%).
- Use many fixed logical partitions so that growth moves whole partitions; throttle rebalancing to avoid a storm.
- Reshard online by copy, dual-write, versioned cutover and delayed cleanup.

1.9 Consistency and Distributed Coordination

Once data is replicated and partitioned, the system must define what a reader may observe, how concurrent actors agree on a single outcome (who is the leader, who holds a lock, in what order events occurred), and how a multi-step operation spanning several nodes completes or is undone. These questions have no free answers: every guarantee costs latency, availability or complexity, and the cost is paid on every request. This chapter presents the guarantees, the coordination primitives built on them, and the identifier schemes that avoid coordination altogether.

1.9.1 Concept and mechanism

CAP and PACELC

The CAP theorem (Brewer's conjecture, proved formally by Gilbert and Lynch in 2002) states that a distributed data store cannot simultaneously provide linearizable consistency and availability of every request in the presence of a network partition. The usable form is narrower than the slogan: partitions occur whether or not the design permits them, so the choice is made during a partition, between refusing some requests (consistency, CP) and answering with possibly stale or divergent data (availability, AP). When no partition exists the theorem imposes no constraint.

PACELC (Abadi) extends the statement: if there is a Partition, choose between Availability and Consistency; Else, even without a partition, choose between Latency and Consistency. The second clause is the one that applies on every request: synchronous replication across regions yields consistency at the price of a cross-region round trip. A design statement of the form "the store is PA/EL" or "PC/EC" summarises both choices.

Consistency models

Model	Guarantee	Cost and typical use
Linearizable	every operation appears to take effect atomically at one instant between its call and its return, and the order respects real time	requires coordination (consensus or a single leader) on every write and read; used for locks, leader records, uniqueness constraints
Sequential	all clients see operations in one total order consistent with each client's program order, not necessarily real time	slightly weaker than linearizable; rarely a design target by itself
Causal	operations that are causally related are seen in the same order by everyone; concurrent operations may be seen in different orders	achievable without blocking during partitions; used by collaborative and social systems
Read-your-writes, monotonic reads	session guarantees for one client	implemented by routing or version tracking; sufficient for most user-facing features
Eventual	if writes stop, all replicas converge to the same value; no bound on when, and no ordering guarantee meanwhile	lowest latency and highest availability; used for feeds, counters, caches

Consistency models from strongest to weakest.

The model is selected per operation, not per system. A single product typically combines linearizable operations (reserving the last seat, creating a unique username) with eventual ones (view counts, activity feeds).

Quorums

In a replicated store with N replicas, a write that waits for W acknowledgements and a read that contacts R replicas are guaranteed to overlap in at least one replica when W + R > N, because the two sets cannot be disjoint within N members. The overlapping replica holds the latest acknowledged write, and the reader selects the newest version by version number or timestamp. Common settings for N = 3 are W = 2, R = 2 (balanced), W = 3, R = 1 (fast reads, writes need all replicas) and W = 1, R = 3 (fast writes, slow reads). The condition W + R > N does not by itself make the store linearizable: concurrent writes, failed writes that reached fewer than W replicas, and sloppy quorums (writes accepted by substitute nodes during a failure) can still produce stale reads.

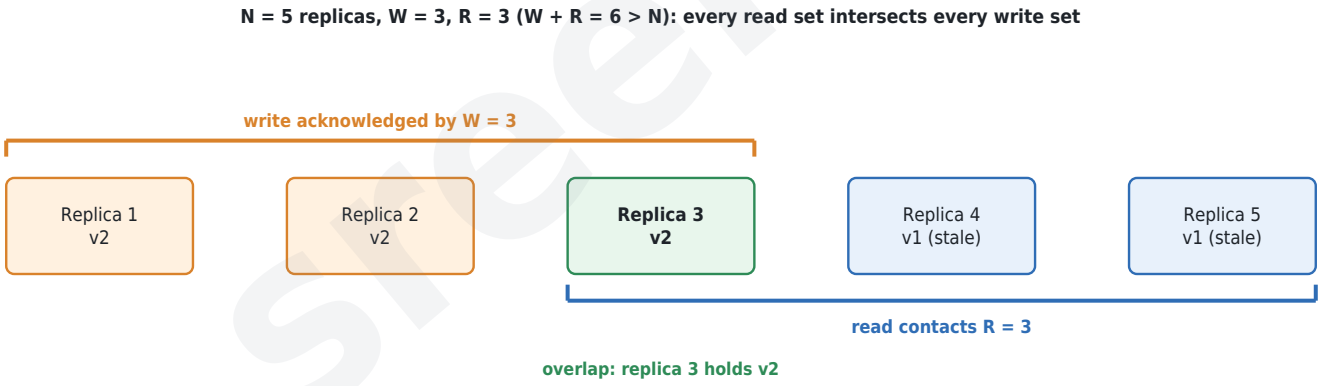


Figure 1.38. With N = 5, W = 3 and R = 3, replica 3 belongs to both the write set and the read set, so any read returns at least one replica holding the latest version.

Idempotency keys

Networks lose responses, so a client that does not receive an answer must retry, and the server may execute the same request twice. An operation is **idempotent** when executing it several times has the same effect as executing it once. Reads and PUT of a full resource are naturally idempotent; "add 100 to the balance" and "create an order" are not. The standard method makes them idempotent with a client-generated **idempotency key**: the client sends a unique key (a UUID) with each logical request and reuses it on every retry. The server records the key together with the result in the same transaction as the effect, protected by a unique constraint.

Idempotency table and atomic processing SQL

```sql
CREATE TABLE idempotency_keys (
    key          text PRIMARY KEY,
    request_hash text        NOT NULL,   -- detects reuse of a key with a different payload
    status       text        NOT NULL,   -- IN_PROGRESS, COMPLETED
    response     jsonb,
    created_at   timestamptz NOT NULL DEFAULT now()
);
```

Idempotency table and atomic processing (continued) SQL

```
-- Within one transaction:
-- 1. INSERT INTO idempotency_keys(key, request_hash, status) VALUES (?, ?, 'IN_PROGRESS');
--    a duplicate-key error means the request was seen: return the stored response
--    (or 409 if the status is still IN_PROGRESS)
-- 2. apply the business change (INSERT INTO payments ...)
-- 3. UPDATE idempotency_keys SET status = 'COMPLETED', response = ? WHERE key = ?;
-- 4. COMMIT
```

Keys are retained for a period longer than the longest client retry window (commonly 24 hours or more) and then removed. Payment APIs commonly require such a key on every write.

Distributed locks and fencing tokens

A distributed lock grants one client exclusive access to a resource for a period. The usual implementation stores a key with a **lease** (a time-limited ownership): in Redis, `SET lock:resource <random-value> NX PX 30000` acquires the lock only if absent, with an expiry of 30 seconds, and release deletes the key only if the stored value still matches the caller's random value (an atomic compare-and-delete, usually a Lua script). In ZooKeeper and etcd the equivalent is an ephemeral node or a lease-attached key whose removal follows the loss of the client session.

A lease protects against a crashed holder, but not against a **paused** one. A client may stall (a long garbage-collection pause, a virtual machine freeze, a slow network) for longer than the lease, resume, and act on the resource believing that it still holds the lock, while a second client holds the lock legitimately. Timing assumptions alone cannot exclude this. The remedy is a **fencing token**: the lock service issues a number that increases on every grant; the client presents it with each write; and the resource rejects any write whose token is lower than the highest it has accepted.

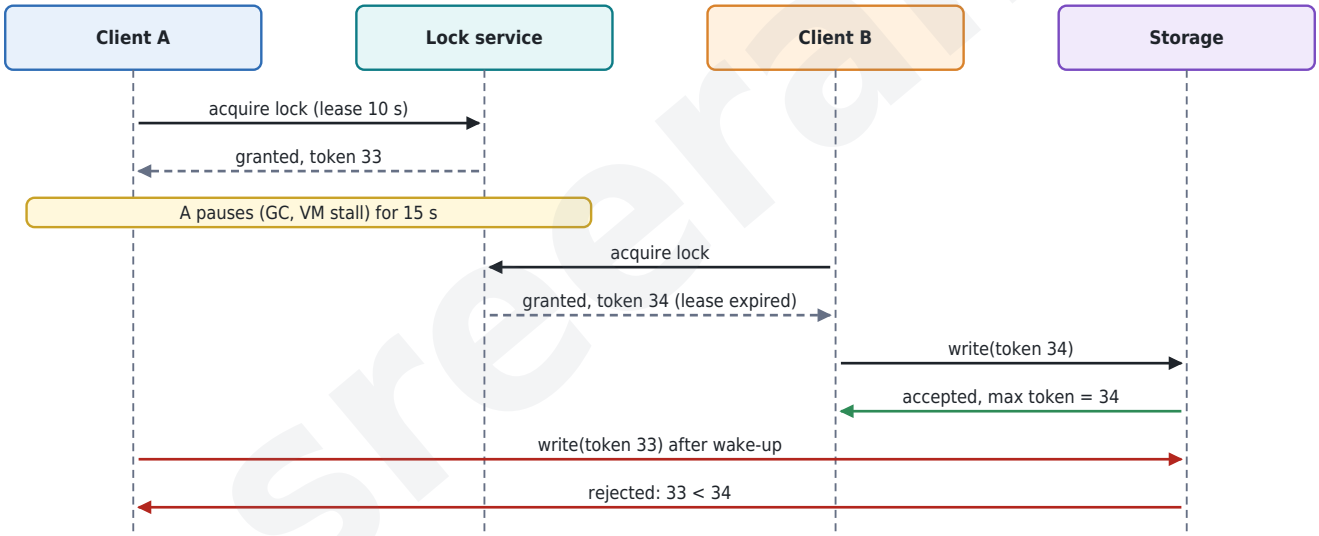


Figure 1.39. A fencing token protects the storage layer: client A resumes after a pause with token 33 and its write is rejected because token 34 was already accepted.

A lock built on a single Redis node is not safe against node failure with asynchronous replication (the replica may lack the key after promotion), and multi-node algorithms based on clock and timing assumptions have been criticised for the pause scenario above. For correctness-critical exclusion, a consensus-based service with fencing is the appropriate foundation; for efficiency-only locking (avoid duplicate work occasionally), a simple lease is acceptable because an occasional double execution is harmless.

Leader election and Raft

Leader election selects one node to perform a role (primary database, scheduler, shard owner). Common implementations acquire a lease or ephemeral node in a coordination service (ZooKeeper, etcd, Consul); the holder is the leader until the lease expires or is renewed. Building the election directly from consensus is the approach of Raft.

Raft keeps a replicated log on N nodes (typically 3 or 5) and tolerates the failure of a minority. The core ideas are as follows.

- Time is divided into **terms**. Each term has at most one leader.
- A follower that receives no heartbeat within a randomised **election timeout** becomes a candidate, increments the term and requests votes. A node votes for at most one candidate per term, and only for a candidate whose log is at

least as up to date as its own. A candidate with votes from a majority becomes leader. Randomised timeouts make split votes rare.

- The leader appends client commands to its log and replicates them with `AppendEntries`. An entry is **committed** when stored on a majority; the leader then applies it and answers the client. A committed entry is present in the log of every future leader, which follows from the voting rule above.
- A follower whose log diverges is overwritten to match the leader's log.

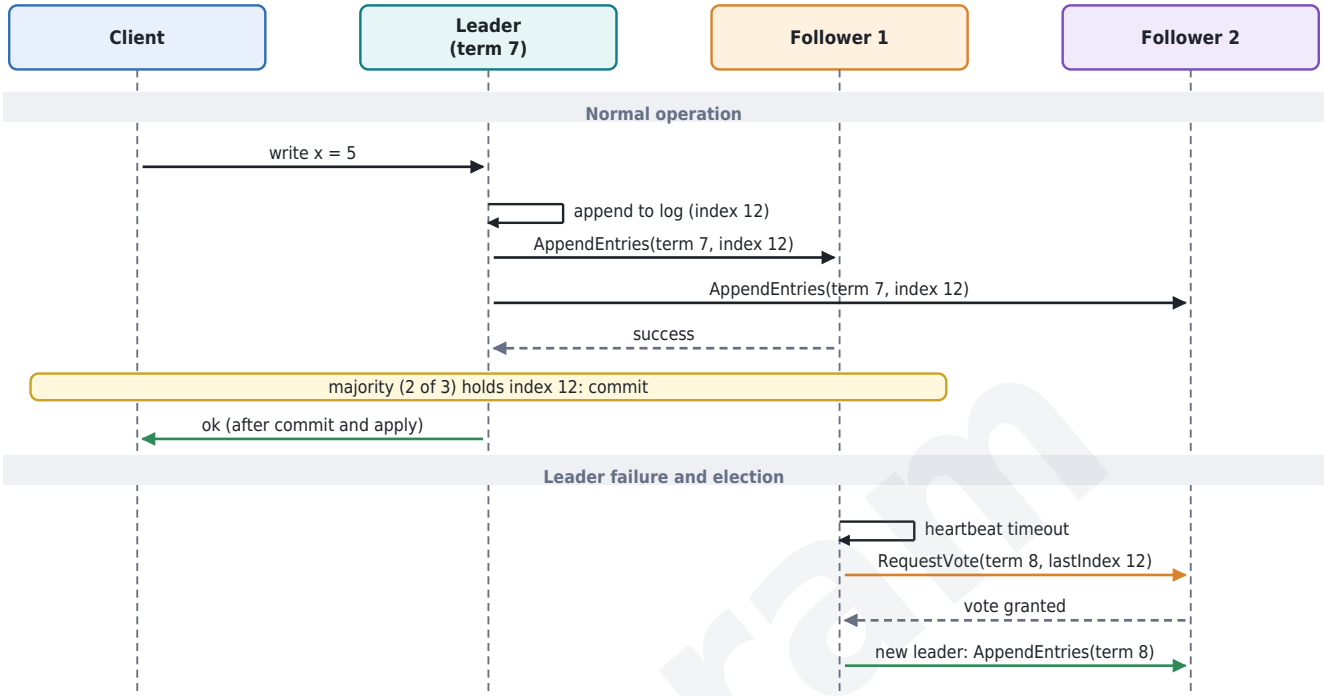


Figure 1.40. Raft in two phases: a write commits once a majority of three nodes holds the entry; after a leader failure a follower times out, wins a vote in a higher term, and continues replication.

The cost is that every write requires a round trip to a majority, and a minority partition cannot make progress (CP behaviour). Consensus is used for small, critical state (cluster membership, configuration, locks, metadata), not for bulk data; bulk data uses partitioned stores that call a consensus group for their metadata.

Two-phase commit and sagas

A transaction that spans several databases or services needs a coordination protocol.

Two-phase commit (2PC). A coordinator asks every participant to prepare (vote); each participant makes the transaction durable and holds its locks and answers yes or no; if all vote yes the coordinator logs the decision and instructs everyone to commit, otherwise to abort. The protocol gives atomicity, but it is blocking: a participant that has voted yes must wait for the decision, holding its locks, and if the coordinator crashes after collecting votes the participants cannot decide alone. Latency is two round trips plus durable writes, and availability is the product of the participants' availabilities.

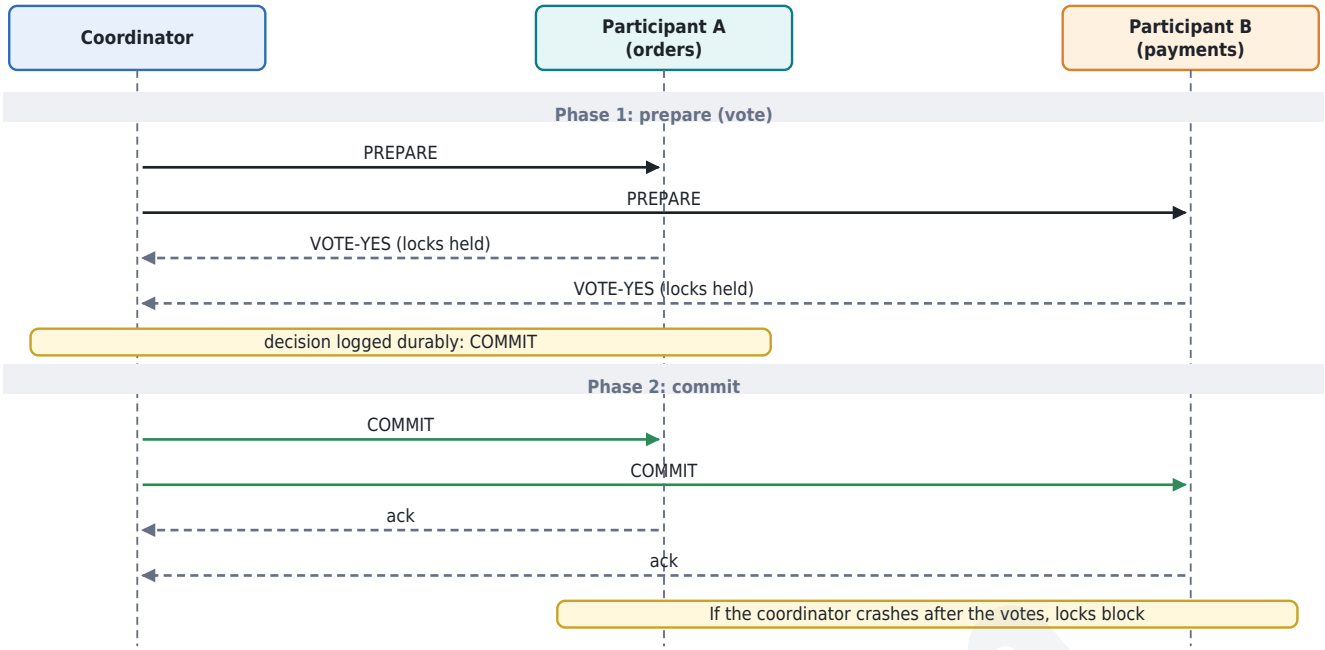


Figure 1.41. Two-phase commit: all participants vote yes and hold locks until the coordinator's durable decision arrives; a coordinator failure between the phases blocks the participants.

Saga. A saga splits the business transaction into a sequence of local transactions, each committed immediately, with a **compensating action** for each step that semantically undoes it. If step k fails, the compensations for steps k - 1 down to 1 run in reverse order. No global locks are held, so the saga scales and tolerates failures, but intermediate states are visible to other transactions (an order exists in a PENDING state before payment succeeds) and the design must tolerate that. Compensations must be idempotent and must themselves be retried until they succeed. A saga is either **orchestrated** (a coordinator service issues the steps and compensations, as in the diagram) or **choreographed** (each service reacts to events from the others, which has less central logic but is harder to follow and debug).

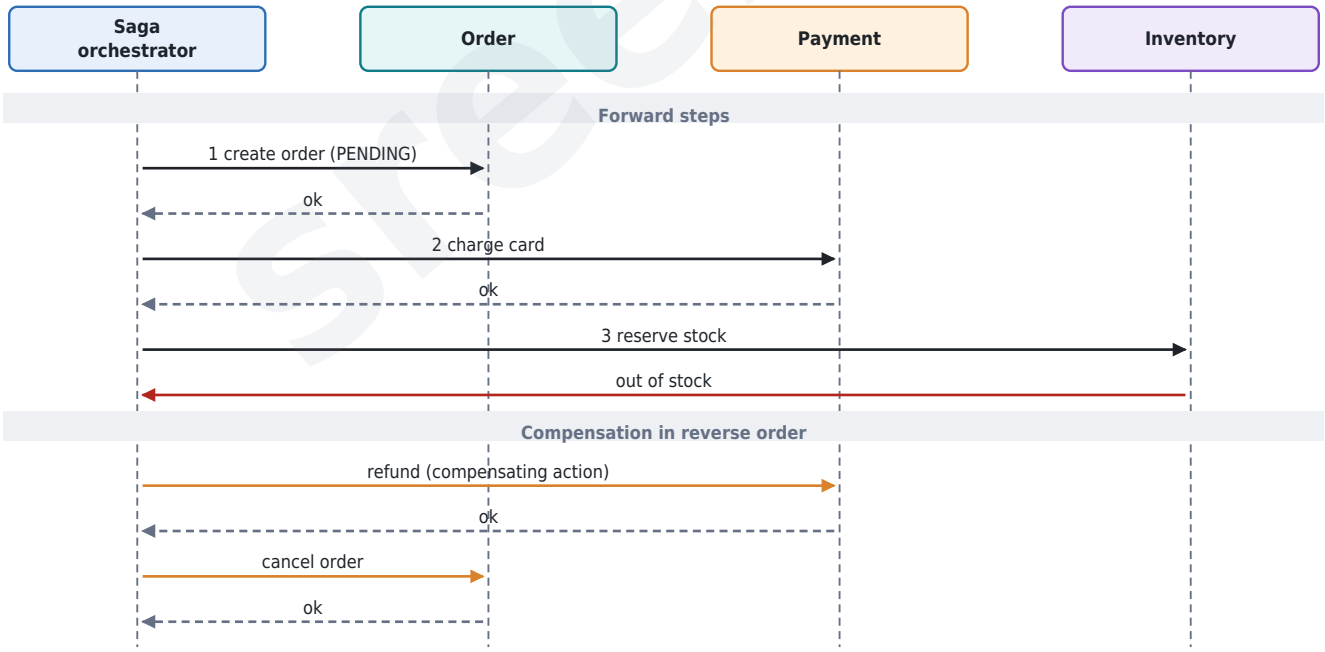


Figure 1.42. An orchestrated saga for order placement: stock reservation fails, so the orchestrator runs the compensating refund and cancellation in reverse order.

Criterion	2PC	Saga
Isolation	strong, locks held across participants	none across steps; intermediate states visible
Availability	all participants must be up	each step is local; tolerates temporary failures with retries
Latency	two round trips with locks	sum of local transactions, asynchronous possible

Criterion	2PC	Saga
Failure handling	the protocol aborts atomically	compensations, retries, manual intervention for failed compensations
Typical use	within one database cluster or between a few homogeneous resource managers	microservices and cross-organisation business processes

Choosing between 2PC and a saga.

Clocks and ordering

Physical clocks are unsuitable for ordering events across machines. Quartz clocks drift (tens of parts per million, so several milliseconds per few minutes without correction), NTP corrects by stepping or slewing and has typical error between a fraction of a millisecond and tens of milliseconds, and leap seconds and virtual machine migration add discontinuities. Two events that are 5 ms apart on different machines cannot be ordered by timestamps with confidence. Systems that use last-writer-wins on wall-clock timestamps lose data when a node's clock is ahead.

Lamport clocks. Each process keeps a counter: increment it on every event, attach it to every message, and on receipt set the counter to max(local, received) + 1. If event a happened before event b (causally), then L(a) < L(b). The converse does not hold: L(a) < L(b) does not imply that a happened before b, and concurrent events cannot be recognised.

Vector clocks. Each process keeps a vector with one counter per process. Comparison of two vectors yields before, after or **concurrent**, so concurrent updates are detected and can be handed to the application for merging (this is the mechanism of the Dynamo paper). The vector grows with the number of writers and must be pruned.

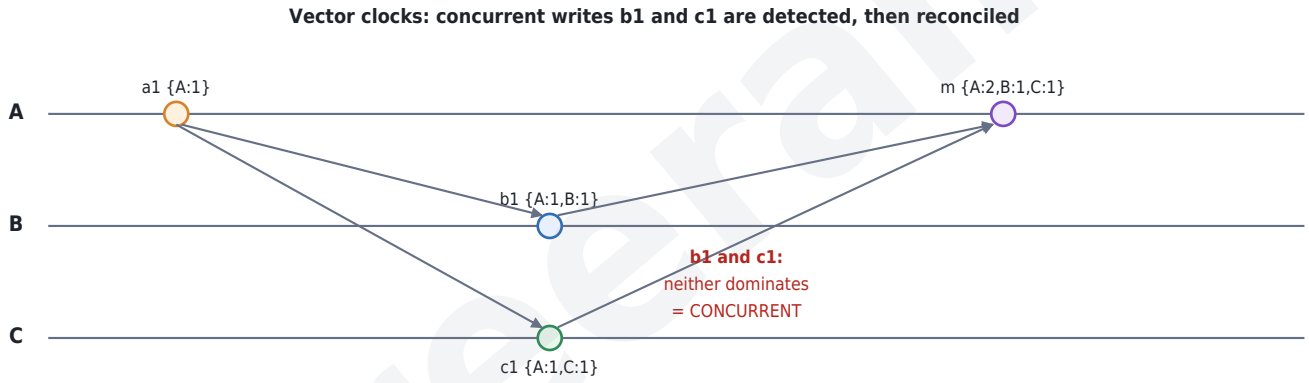


Figure 1.43. Vector clocks detect that b1 (by B) and c1 (by C) are concurrent, because each follows a1 but neither knows the other; the reconciled value m dominates both.

Hybrid logical clocks (HLC). An HLC combines the physical time with a logical counter, staying close to wall-clock time while preserving causal order (a received timestamp that is ahead of the local clock advances the local value). Timestamps are comparable across nodes and bounded in drift, which makes them a practical basis for multi-version concurrency control in distributed databases. **TrueTime**, described in the Spanner paper, takes a different approach: the API returns an interval [earliest, latest] that is guaranteed to contain the true time, and a transaction waits out the uncertainty (commit wait) before it becomes visible, which gives externally consistent ordering at the cost of the wait, kept small by GPS and atomic clock references.

Unique identifier generation

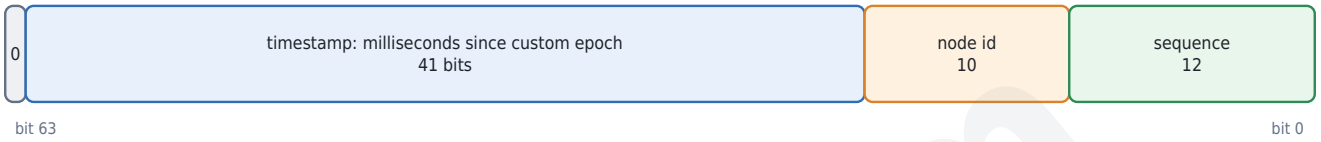
Distributed systems need identifiers that are unique without a central bottleneck, and often roughly ordered by time (for index locality and for sorting by recency).

Scheme	Properties	Strengths	Weaknesses
Database auto-increment	64-bit sequence from one primary	compact, strictly ordered, simple	single point of failure and write bottleneck; reveals volume; hard across shards (ranges or ticket servers with step N)
UUID version 4	122 random bits, 128-bit total	no coordination; collision probability negligible	16 bytes; random order fragments B-tree indexes; no time information

Scheme	Properties	Strengths	Weaknesses
UUID version 7 (RFC 9562)	48-bit Unix millisecond timestamp followed by random bits	no coordination; time-ordered, index-friendly	16 bytes; ordering within one millisecond is not guaranteed across generators unless a counter is used
ULID	48-bit timestamp, 80 random bits, 26-character base32 text	sortable, URL-safe text	16 bytes of binary; same-millisecond ordering relies on a monotonic variant
Snowflake	64-bit: 41-bit time, 10-bit node, 12-bit sequence	compact, k-sortable, 4,096 ids per node per millisecond, no coordination at runtime	needs unique node ids; depends on a monotonic clock; fails if the clock goes backwards

Identifier schemes.

Snowflake-style 64-bit identifier (layout used in the implementation below)



41 bits of milliseconds: 2^41 ms = 69.7 years of ids from the epoch

10 bits: 1,024 generator nodes 12 bits: 4,096 ids per node per millisecond

Ids sort by creation time (k-sortable): B-tree inserts stay near the right edge

Measured on the build machine: about 4,000,000 ids per second from one node

Figure 1.44. The 64-bit layout used in the implementation: 41 bits of milliseconds give about 69.7 years, 10 bits address 1,024 generators, and 12 bits allow 4,096 ids per millisecond on each generator.

The original Snowflake service, announced by Twitter in 2010, used this 1 + 41 + 10 + 12 bit layout (per the public description of the format on Wikipedia; the 10 bits were split into 5 bits for the data center and 5 for the worker in the first implementation). The node id must be assigned uniquely, for example from a coordination service lease or from a database table at start-up; the sequence counter resets every millisecond.

1.9.2 Worked example

The following computes the identifier capacity of the layout and checks it against a workload, then applies the quorum condition to a configuration.

Quantity	Value	Arithmetic
`Lifetime of 41 bits`	69.7 years	2^41 ms = 2,199,023,255,552 ms / (1000 x 86,400 x 365.25)
`Ids per node per millisecond`	4,096	2^12
`Ids per node per second`	4,096,000	4,096 x 1000
`Required rate for the service`	200,000 per second	assumed
`Nodes required for throughput`	1	200,000 / 4,096,000 = 0.05; at least 3 for availability
`Node id space`	1,024	2^10
`Quorum configuration`	N = 3, W = 2, R = 2	W + R = 4 > 3
`Tolerated replica failures for reads and writes`	1	N - W = 1 and N - R = 1

Capacity of the Snowflake layout and quorum choice (assumed workload).

The identifier is not the bottleneck: one generator provides about twenty times the assumed rate, so the number of generator nodes is dictated by availability, not throughput.

sreram

1.9.3 Hands-on

sreeram

A Snowflake generator

Snowflake.java JAVA

```java
import java.time.Instant;
import java.util.function.LongSupplier;

public class Snowflake {
    static final long EPOCH = 1_735_689_600_000L;       // 2025-01-01T00:00:00Z
    static final int NODE_BITS = 10, SEQ_BITS = 12;
    static final long MAX_SEQ = (1L << SEQ_BITS) - 1;

    private final long nodeId;
    private final LongSupplier clock;
    private long lastMs = -1, seq = 0;

    Snowflake(long nodeId, LongSupplier clock) {
        if (nodeId < 0 || nodeId >= (1L << NODE_BITS)) {
            throw new IllegalArgumentException("node id out of range");
        }
        this.nodeId = nodeId;
        this.clock = clock;
    }

    synchronized long next() {
        long now = clock.getAsLong();
        if (now < lastMs) {                             // refuse to go backwards
            throw new IllegalStateException("clock moved backwards by " + (lastMs - now) + " ms");
        }
        if (now == lastMs) {
            seq = (seq + 1) & MAX_SEQ;
            if (seq == 0) {                             // 4096 ids used: wait for the next ms
                while ((now = clock.getAsLong()) <= lastMs) { }
            }
        } else {
            seq = 0;
        }
        lastMs = now;
        return ((now - EPOCH) << (NODE_BITS + SEQ_BITS)) | (nodeId << SEQ_BITS) | seq;
    }

    static String decode(long id) {
        long ts = (id >>> (NODE_BITS + SEQ_BITS)) + EPOCH;
        long node = (id >>> SEQ_BITS) & ((1L << NODE_BITS) - 1);
        return Instant.ofEpochMilli(ts) + " node=" + node + " seq=" + (id & MAX_SEQ);
    }

    public static void main(String[] args) {
        Snowflake g = new Snowflake(7, System::currentTimeMillis);
        for (int i = 0; i < 3; i++) {
            long id = g.next();
            System.out.println(id + "  " + decode(id));
        }
        int n = 4_000_000;
        long[] ids = new long[n];
        long t0 = System.nanoTime();
        for (int i = 0; i < n; i++) ids[i] = g.next();
        double secs = (System.nanoTime() - t0) / 1e9;
        boolean sorted = true;
        for (int i = 1; i < n && sorted; i++) sorted = ids[i] > ids[i - 1];
        System.out.printf("%d ids in %.2f s (%.0f ids/s), strictly increasing: %b%n",
                n, secs, n / secs, sorted);

        long[] t = {1_000_000L + EPOCH};                // controllable clock
        Snowflake skewed = new Snowflake(1, () -> t[0]);
        skewed.next();
        t[0] -= 5;                                      // the clock is stepped back by 5 ms
        try {
            skewed.next();
        } catch (IllegalStateException e) {
            System.out.println("refused: " + e.getMessage());
        }
        long[] fixed = {2_000_000L + EPOCH};            // same node id configured twice
        long a = new Snowflake(3, () -> fixed[0]).next();
```

Sreeram

Snowflake.java (continued) JAVA

```java
        long b = new Snowflake(3, () -> fixed[0]).next();
        System.out.println("two generators with node id 3 produce the same id: " + (a == b));
    }
}
```

Typical output (the timestamps and the throughput vary between runs) TERMINAL

```
$ java Snowflake.java
237967010300981248  2026-10-19T15:55:56.469Z node=7 seq=0
237967010439393280  2026-10-19T15:55:56.502Z node=7 seq=0
237967010439393281  2026-10-19T15:55:56.502Z node=7 seq=1
4000000 ids in 1.02 s (3935197 ids/s), strictly increasing: true
refused: clock moved backwards by 5 ms
two generators with node id 3 produce the same id: true
```

What to notice: `next()` is `synchronized`, so one generator instance serialises ids and the throughput measured on this machine is about four million ids per second. The check `now < lastMs` makes clock regression an explicit error; the alternatives are to wait until the clock catches up (acceptable for a few milliseconds) or to refuse to serve and let another node take over. The last line shows the real operational risk: uniqueness depends entirely on every node having a distinct node id.

Fencing tokens, quorums and clocks

The three Python programs are single-process simulations of the protocols; they model the logic, not network behaviour.

fencing.py PYTHON

```python
class LockService:
    def __init__(self):
        self.token = 0

    def acquire(self):
        self.token += 1
        return self.token                 # monotonically increasing fencing token


class Storage:
    def __init__(self, check_tokens):
        self.check, self.max_token, self.value = check_tokens, 0, None

    def write(self, who, token, value):
        if self.check and token < self.max_token:
            print(f"  {who}: write REJECTED (token {token} < {self.max_token})")
            return
        self.max_token, self.value = max(self.max_token, token), value
        print(f"  {who}: write accepted with token {token}")


def scenario(check):
    lock, store = LockService(), Storage(check)
    token_a = lock.acquire()              # client A holds the lock, then stalls (GC pause)
    token_b = lock.acquire()              # the lease expired; client B acquires the lock
    store.write("client B", token_b, "B-data")
    store.write("client A", token_a, "A-stale")   # A wakes up and still believes it holds the lock
    print(f"token check {'ON ' if check else 'OFF'}: final value = {store.value}")


scenario(False)
scenario(True)
```

Run TERMINAL

```
$ python3 fencing.py
  client B: write accepted with token 2
  client A: write accepted with token 1
token check OFF: final value = A-stale
  client B: write accepted with token 2
  client A: write REJECTED (token 1 < 2)
token check ON : final value = B-data
```

sheeran

Without the check, the stale write from client A overwrites the newer data; with the check the storage layer rejects it. The protection exists in the storage layer, which is the only component able to observe both writes.

quorum.py PYTHON

```python
import random


def stale_read_rate(n, w, r, trials=20000, seed=3):
    rnd, stale = random.Random(seed), 0
    for _ in range(trials):
        replicas = [0] * n                          # version held by each replica
        for i in rnd.sample(range(n), w):           # a write just reached w replicas
            replicas[i] = 1
        if max(replicas[i] for i in rnd.sample(range(n), r)) == 0:
            stale += 1                              # no contacted replica had the new version
    return stale / trials


print("N W R  W+R>N  stale reads (simulation, random replica subsets)")
for n, w, r in [(3, 1, 1), (3, 2, 1), (3, 2, 2), (5, 3, 3), (5, 1, 1), (5, 2, 2)]:
    print(f"{n} {w} {r}  {str(w + r > n):5s}  {100 * stale_read_rate(n, w, r):5.1f}%")
```

Run (deterministic seed) TERMINAL

```
$ python3 quorum.py
N W R  W+R>N  stale reads (simulation, random replica subsets)
3 1 1  False   66.6%
3 2 1  False   33.1%
3 2 2  True     0.0%
5 3 3  True     0.0%
5 1 1  False   80.2%
5 2 2  False   30.1%
```

When W + R > N no stale read occurs; with W = R = 1 on N = 5, four of five reads miss the newest version (80.2%), which matches the expected probability that a random read replica is outside the write set. The simulation assumes the write has completed on exactly W replicas and that no further replication has occurred, which is the worst case for staleness.

clocks.py PYTHON

```python
def merge(a, b):
    return {k: max(a.get(k, 0), b.get(k, 0)) for k in set(a) | set(b)}


def compare(a, b):
    keys = set(a) | set(b)
    le = all(a.get(k, 0) <= b.get(k, 0) for k in keys)
    ge = all(a.get(k, 0) >= b.get(k, 0) for k in keys)
    return "equal" if le and ge else "before" if le else "after" if ge else "CONCURRENT"


a1 = {"A": 1}                          # node A writes
b1 = merge(a1, {"B": 1})               # B read a1 and wrote
c1 = merge(a1, {"C": 1})               # C also read a1 and wrote, without seeing b1
print("a1 vs b1:", compare(a1, b1), "| b1 vs c1:", compare(b1, c1))
m = merge(b1, c1)
m["A"] += 1                            # A reconciles both versions and increments its entry
print("merged:", dict(sorted(m.items())), "| vs b1:", compare(m, b1), "| vs c1:", compare(m, c1))


class Lamport:
    def __init__(self):
        self.t = 0

    def event(self):
        self.t += 1
        return self.t

    def receive(self, ts):
        self.t = max(self.t, ts) + 1
        return self.t


p, q = Lamport(), Lamport()
for _ in range(3):
```

sreeram

clocks.py (continued) PYTHON

```
    p.event()
sent = p.event()                    # P sends a message stamped 4
q.event()                           # Q has a local event at 1
print("P sends at", sent, "| Q at", q.t, "| Q receives ->", q.receive(sent))
```

Run TERMINAL

```
$ python3 clocks.py
a1 vs b1: before | b1 vs c1: CONCURRENT
merged: {'A': 2, 'B': 1, 'C': 1} | vs b1: after | vs c1: after
P sends at 4 | Q at 1 | Q receives -> 5
```

The vector comparison classifies b1 and c1 as concurrent, so a store using vector clocks keeps both versions as siblings until a reconciliation produces a vector that dominates both. The Lamport receive rule produces max(1, 4) + 1 = 5.

1.9.4 Real-world problems and solutions

Problem 1: split brain with two leaders

Scenario. A database primary is partitioned from its monitor, which promotes a replica. The old primary is still reachable by some clients. **Symptoms.** Two nodes accept writes; after the partition heals, the data sets have diverged, with duplicate order numbers and conflicting updates to the same rows. **Diagnosis.** Both nodes report the primary role in their metrics; write-ahead log positions diverge from a common point. **Root cause.** Failover was decided by a node that could not confirm that the old primary had stopped, and nothing prevented the old primary from continuing. **Fix.** Fence the old primary: revoke its storage or network access (power or cloud API), or make writes conditional on a fencing token or on holding a quorum-backed lease that the old primary loses when it cannot renew. Reconcile the divergence manually or by replaying the lost writes onto the new primary. **Prevention.** Use quorum-based election (an odd number of voters, majority required), require the leader to step down when it loses the quorum, and test partitions in staging.

Problem 2: clock skew breaks ordering and uniqueness

Scenario. A service resolves concurrent profile updates by comparing wall-clock timestamps, and generates Snowflake ids from the local clock. One node's clock is 3 seconds ahead after an NTP misconfiguration. **Symptoms.** Updates made on the skewed node always win, silently discarding newer writes from other nodes. After the clock is corrected, the generator produces ids lower than ids already issued, or refuses to start. **Diagnosis.** Comparing event times across nodes shows a systematic offset; the generator log reports a backwards clock step. **Root cause.** Wall-clock timestamps from different machines were treated as a global order, and the id generator assumed a monotonic clock. **Fix.** Use version numbers, vector clocks or hybrid logical clocks to order updates. In the id generator, refuse ids while the clock is behind the last timestamp (as the implementation above does) or wait for it to catch up; persist the last issued timestamp across restarts. **Prevention.** Monitor clock offset as a metric with alerting, use more than one time source, and design ordering that does not depend on synchronised clocks.

Problem 3: a lock holder pauses and corrupts the resource

Scenario. A batch job takes a Redis lock with a 30-second lease and updates a shared file. A garbage-collection pause of 40 seconds occurs in the middle of the update. **Symptoms.** Two instances of the job process the same partition and the output contains interleaved records. **Diagnosis.** Logs show the lease expired while the first holder was paused, and the second holder acquired it. **Root cause.** The lock was assumed to guarantee exclusion; a lease guarantees it only while the holder runs within the lease. **Fix.** Issue fencing tokens and have the storage layer reject stale tokens, or make the update conditional (compare-and-set on a version). Renew the lease from a background thread and abort work when renewal fails. **Prevention.** Classify each lock as correctness-critical or efficiency-only; for the former, require a fencing mechanism.

Problem 4: duplicate payment from a client retry

Scenario. A mobile client times out waiting for a payment response and retries. **Symptoms.** The customer is charged twice; the payment provider shows two transactions with different internal ids. **Diagnosis.** Server logs show two requests with identical payloads a few seconds apart; both succeeded. **Root cause.** The endpoint was not idempotent, and the first response was lost after the effect had been applied. **Fix.** Require an idempotency key on the endpoint, store it with the result in the same transaction as the charge (unique constraint), and return the stored result on a repeat. Refund the duplicate. **Prevention.** Make every non-idempotent write endpoint accept an idempotency key, and pass the same key to downstream providers.

Problem 5: saga left in an inconsistent state

Scenario. An order saga charges the card, then reserves stock; the refund compensation call fails because the payment service is down. **Symptoms.** Orders in a state CANCELLED, with money still captured, accumulate; customers complain. **Diagnosis.** The saga log shows compensation steps in a failed state with no retry. **Root cause.** Compensation was treated as best effort instead of a step that must eventually succeed. **Fix.** Persist the saga state, retry compensations with backoff until success, make them idempotent, and raise an alert and a manual task after a threshold. Reconcile periodically against the payment provider. **Prevention.** Design every forward step with an idempotent compensation, and test the failure of each step.

1.9.5 Questions to expect

1. **Explain CAP in practice.** During a network partition a system must choose between consistency (reject or delay requests that cannot be answered correctly) and availability (answer, possibly stale). Without a partition the trade-off is between latency and consistency (PACELC). The choice is made per operation.
2. **What is a quorum and why does W + R > N matter?** A quorum is the number of replicas that must acknowledge an operation. If W + R > N the read set and write set intersect, so a read sees at least one replica holding the latest acknowledged write. It does not give linearizability on its own, because of concurrent writes and sloppy quorums.
3. **How are duplicate requests handled?** With idempotency keys: the client sends a unique key on each logical request, the server stores key and result atomically with the effect under a unique constraint, and a repeat returns the stored result.
4. **How does a distributed lock work and what is its weakness?** A key with a lease acquired atomically. A holder that pauses beyond the lease still believes that it owns the lock. A fencing token checked by the resource closes the gap.
5. **How does Raft elect a leader and commit entries?** Followers that miss heartbeats become candidates in a higher term and need a majority of votes; voters accept only candidates with a log at least as current. The leader commits an entry when a majority stores it, so committed entries survive leader changes.
6. **2PC or saga?** 2PC gives atomic commit with locks and blocking, suited to a few homogeneous resources. A saga uses local transactions and compensations; it scales and tolerates failures but exposes intermediate states and requires idempotent compensations.
7. **Why are wall-clock timestamps a poor basis for ordering?** Clocks drift and are corrected in steps, so events from different machines cannot be ordered within the synchronisation error. Use version numbers, Lamport, vector or hybrid logical clocks.
8. **Compare UUID, auto-increment and Snowflake.** Auto-increment is compact and ordered but a bottleneck; UUIDv4 needs no coordination but is random and large; Snowflake is compact and time-ordered with no runtime coordination but needs node ids and a monotonic clock; UUIDv7 and ULID give time-ordering with 128 bits.
9. **How does a system avoid split brain?** Quorum-based election, leases that expire if not renewed, fencing of the old leader, and an odd number of voters so that at most one partition holds a majority.
10. **What does eventual consistency guarantee?** Only convergence when writes stop; it gives no bound on time and no ordering in the meantime, so the design needs conflict resolution and tolerance for stale reads.

KEY POINTS

- CAP applies during a partition; PACELC adds the latency versus consistency trade-off that applies on every request.
- W + R > N gives overlapping quorums, not linearizability by itself.
- Idempotency keys, stored atomically with the effect, make retries safe.
- A lease alone cannot guarantee mutual exclusion; fencing tokens checked by the resource can.
- Raft commits on a majority and survives leader change; use consensus for small critical state.
- Prefer sagas with idempotent compensations across services; use 2PC only where blocking is acceptable.
- Do not order events by wall clocks; use versions, vector clocks or hybrid logical clocks, and treat a backwards clock as an error in id generators.

1.10 Asynchronous Processing and Messaging

A synchronous call couples the caller to the availability and speed of the callee: if the callee is slow, the caller is slow, and if the callee is down, the caller fails. Asynchronous processing places a durable buffer between them. The producer records the work or the fact and continues; consumers process at their own rate and can be added, removed or restarted independently. The buffer absorbs traffic spikes, isolates failures, allows one event to reach many consumers, and enables retry of failed work. The price is added latency, eventual consistency between the stages, duplicate delivery, weaker ordering, and a new component that must itself be operated and monitored.

The Kafka material and the Spring messaging material cover the clients, the brokers and the configuration in detail. This chapter treats the design-level concepts: the queue and log models, delivery guarantees, ordering, failure handling, event-driven patterns, and the processing styles built on top.

1.10.1 Concept and mechanism

Queues and logs

Two models of a message broker dominate.

A **queue** (RabbitMQ, Amazon SQS, ActiveMQ) holds messages until a consumer acknowledges them. Consumers compete for messages, each message goes to one consumer, and it is removed after acknowledgement. Queues suit task distribution (send an email, resize an image), where every message is a unit of work to be done once.

A **log** (Apache Kafka, Pulsar, Kinesis) is an append-only, partitioned sequence of records retained for a configured time or size, regardless of consumption. Each consumer group maintains its own position (offset) per partition, so several independent groups read the same data, and a consumer can rewind to replay history. Logs suit event streams, where many consumers use the same facts for different purposes and replay is valuable.

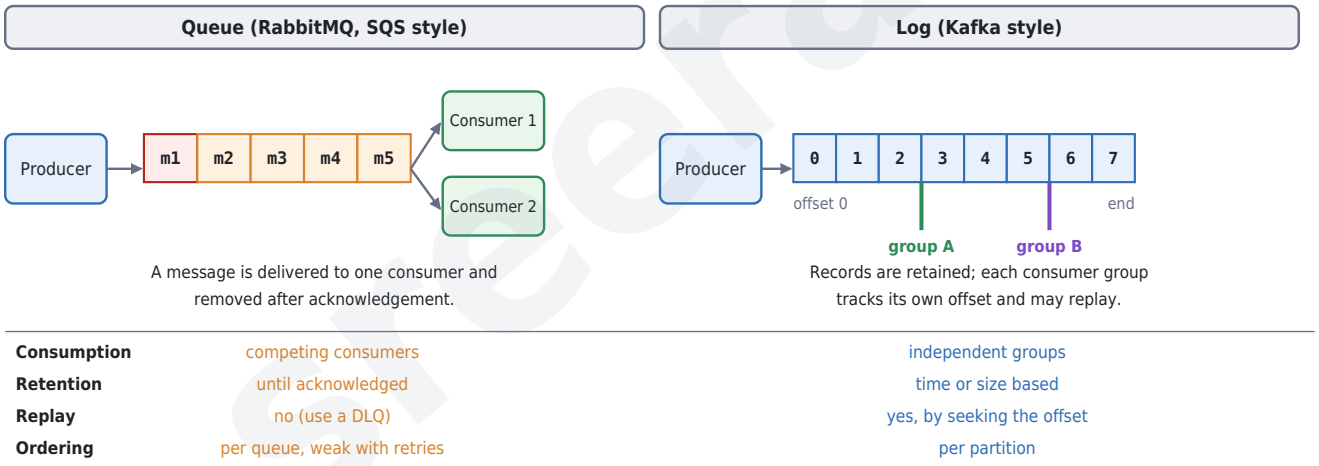


Figure 1.45. A queue delivers each message to one competing consumer and deletes it after acknowledgement; a log retains records and lets each consumer group keep its own offset.

Aspect	Queue	Log
Message lifetime	until acknowledged (or expired)	until retention expires
Consumers	competing consumers share the work	independent groups each read everything; within a group partitions are divided
Replay	not supported; a dead-letter queue holds failures	supported by resetting offsets
Ordering	per queue for one consumer; weakened by parallel consumers and retries	strict within a partition
Scaling consumers	add consumers freely	limited by the number of partitions per group
Typical use	background jobs, work distribution with per-message routing and priorities	event streaming, change data capture, audit trails, analytics feeds

Queue and log compared.

Managed queue services add features that shape the design. SQS standard queues deliver at least once with best-effort ordering; FIFO queues provide ordering per message group and a deduplication window (five minutes) based

on a deduplication id. A message that is received becomes invisible for a **visibility timeout**; if the consumer does not delete it before the timeout, the message reappears and is delivered again, which is the mechanism behind at-least-once delivery.

Delivery semantics

Guarantee	Mechanism	Consequence
At-most-once	acknowledge (or commit the offset) before processing; no redelivery	a crash after the acknowledgement loses the message; no duplicates
At-least-once	acknowledge after processing; redeliver when no acknowledgement arrives	a crash after the effect but before the acknowledgement causes a duplicate; no loss
Exactly-once (effect)	at-least-once delivery combined with an idempotent consumer, or a transaction that commits the effect and the offset atomically	each message affects the state once; the delivery itself still occurs more than once

Delivery guarantees.

A network cannot distinguish a lost request from a lost reply, so no protocol delivers a message exactly once end to end in the strict sense. What systems provide is **effectively-once processing**: at-least-once delivery plus idempotent handlers. Kafka supports exactly-once semantics inside Kafka (an idempotent producer, which deduplicates producer retries per partition, and transactions that write output records and consumer offsets atomically); an external effect, such as a call to a payment provider or a row written to another database, still requires an idempotent operation or a transactional outbox.

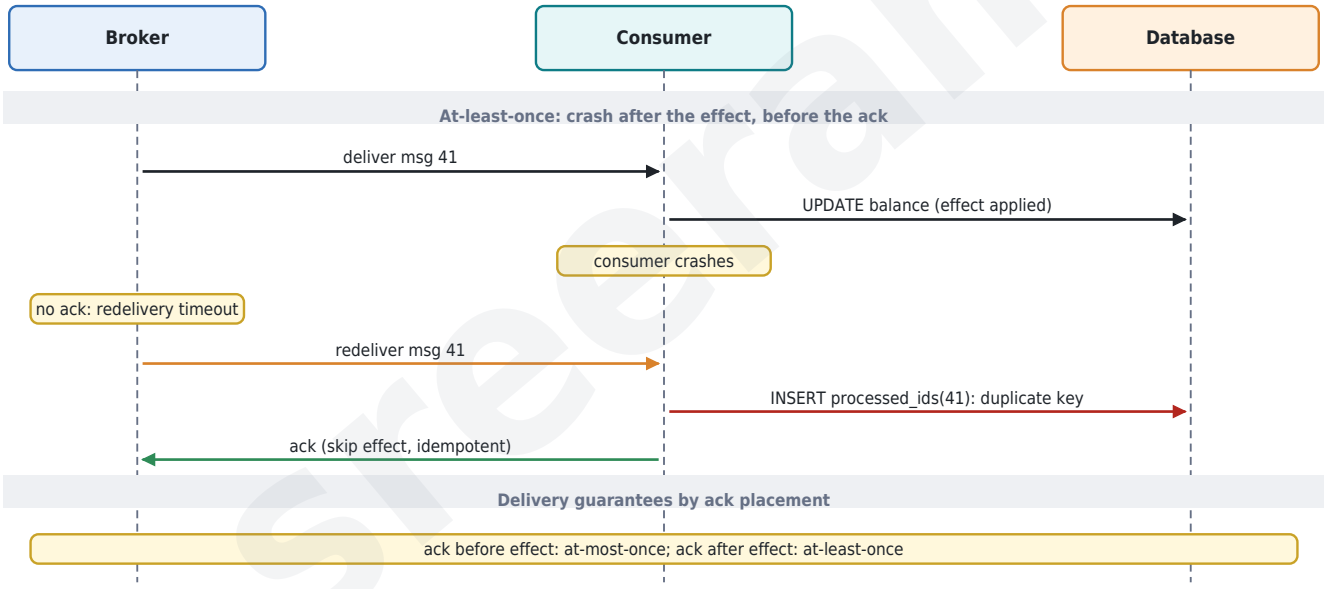


Figure 1.46. A consumer crashes after applying the effect and before the acknowledgement; the broker redelivers, and the processed-id check turns the duplicate into a no-op.

The consumer-side rule is: record the message id (or a business key) in the same transaction as the effect, protected by a unique constraint, and skip a message whose id is already recorded. When the natural operation is idempotent (set the status to SHIPPED), the check is unnecessary.

Consumer groups and partitions

In a log, a topic is divided into partitions, and each partition is an ordered sequence. The producer selects the partition by hashing the record key, so all records with one key go to one partition. Within a consumer group, each partition is assigned to exactly one consumer, which gives per-key ordering and parallelism equal to the number of partitions. When consumers join or leave, the group **rebalances** the assignment; during the rebalance consumption pauses for the affected partitions (modern cooperative protocols reduce the pause).

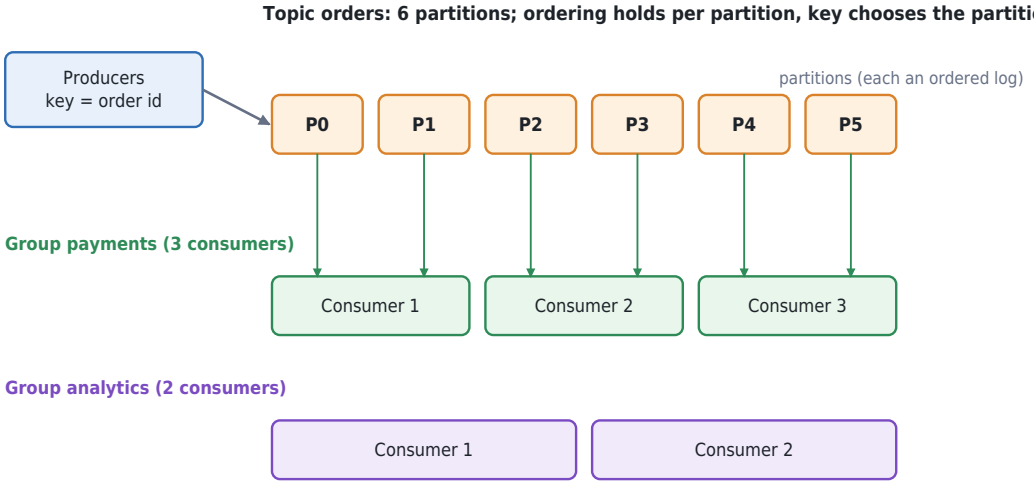


Figure 1.47. A topic with six partitions read by two groups: each partition has one consumer per group, three consumers use two partitions each, and the groups progress independently.

Three consequences follow. The maximum useful parallelism of a group equals the partition count, so the partition count must be chosen with the target throughput in mind and is hard to reduce later; increasing it changes the key-to-partition mapping for new records and disturbs per-key ordering across the change. A slow consumer causes **consumer lag**, the difference between the newest offset and the committed offset, which is the principal health metric of a streaming pipeline. A **hot key** (one key with much of the traffic) overloads one partition and one consumer.

Ordering

Order is guaranteed only within a partition (log) or a message group (FIFO queue). A global order across all messages requires a single partition and a single consumer, which limits throughput to that of one consumer. The practical design is per-entity ordering: choose the key so that all events that must be ordered share it (an order id, a user id), and accept that events of different entities are unordered.

Order can still be broken inside a partition. Producer retries can reorder records unless the producer is idempotent (in Kafka, `enable.idempotence=true` with at most five in-flight requests per connection preserves order across retries). Parallel processing of a partition's records inside one consumer (a thread pool) breaks order unless work is dispatched per key. A retry that republishes a failed message to a retry topic places it behind later messages. A consumer that needs strict order must either block on the failed message or tolerate and detect out-of-order events through a version or sequence number carried in the event.

Failure handling: retries, backoff and dead-letter queues

Failures divide into transient (a downstream timeout, a brief outage) and permanent (a malformed payload, a violated business rule). Transient failures are retried; permanent failures are not.

- **Retry with exponential backoff and jitter.** The delay before attempt k is a random value up to min(cap, base x 2^(k-1)) ("full jitter"). Backoff gives the dependency time to recover, and jitter prevents all consumers from retrying at the same instant.
- **Bounded retries.** After a maximum number of attempts the message is moved to a **dead-letter queue** (DLQ or dead-letter topic) with the error and metadata. The DLQ is monitored with an alert, inspected, and its contents are redriven after the cause is fixed.
- **Poison messages.** A message that always fails blocks a partition if the consumer retries it forever. Bounded retries with a DLQ keep the partition moving.
- **Retry topics.** In a log, delayed retries are implemented with separate topics per delay tier (retry-1m, retry-10m), since a partition cannot skip ahead without losing its position.

The transactional outbox and change data capture

A service that updates its database and then publishes an event has a dual-write problem: the two operations cannot be made atomic. If the process crashes between them, either the event is lost (database written, no event) or a false event is published (event sent, database transaction aborted).

The **transactional outbox** removes the dual write. The service writes the business change and an event row to an `outbox` table in one local transaction. A separate **relay** reads the unpublished rows (polling, or by reading the database log through change data capture, CDC) and publishes them to the broker, marking them as sent. The relay may publish a row twice if it crashes after publishing and before marking, so publication is at-least-once and consumers must be idempotent.

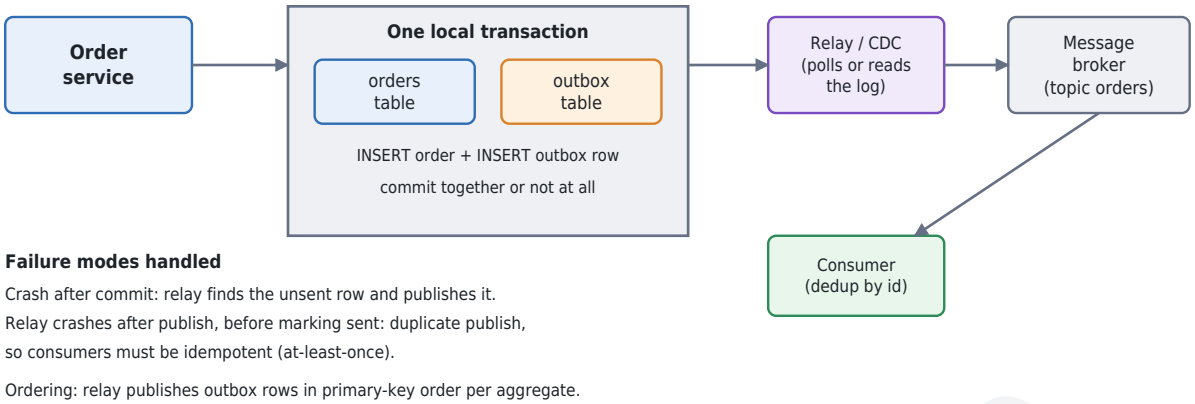


Figure 1.48. The outbox pattern: the order row and the outbox row commit together; a relay publishes the outbox rows, and consumers deduplicate by event id.

CDC tools read the database transaction log, so they observe every committed change without modifying the application and without polling load. The same mechanism feeds search indexes, caches and warehouses from the system of record.

Event-driven architecture, CQRS and event sourcing

In an event-driven architecture, services communicate by publishing events that describe facts that happened (`OrderPlaced`), and other services react. Two styles occur. **Event notification** carries a minimal payload (the id); consumers call back for details. **Event-carried state transfer** includes the data that consumers need, so they need not call the source and can maintain a local copy, at the price of larger events and duplicated data. A **command** (`ChargeCard`) is a request addressed to one service; an event is a statement addressed to no one in particular, and the distinction affects coupling.

CQRS (command query responsibility segregation) separates the model that handles writes from the models that serve reads, so each is shaped for its use: a normalised write model, and denormalised, query-specific read models (a search index, a cached view) updated from the events. **Event sourcing** stores the sequence of events as the source of truth and derives current state by folding over them; the log gives a complete audit trail and the ability to rebuild any read model.

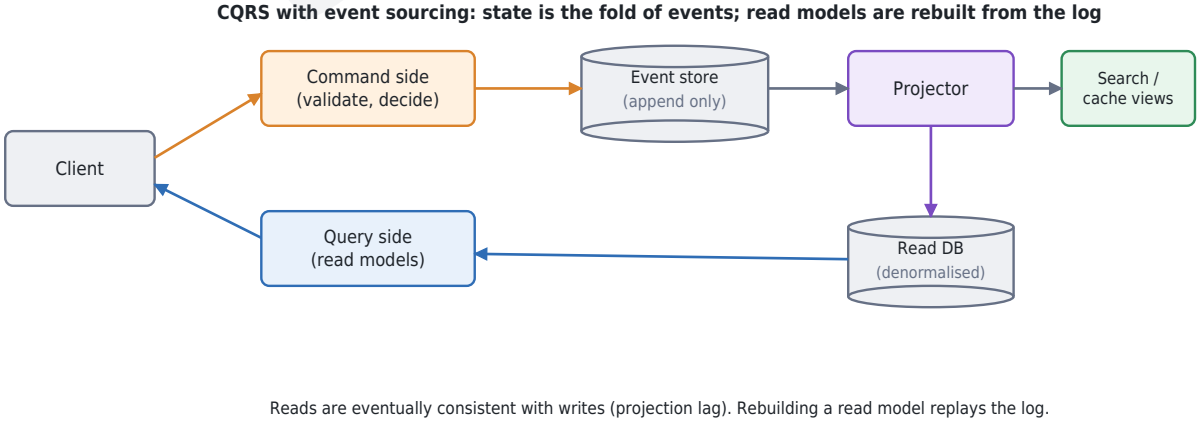


Figure 1.49. CQRS with event sourcing: commands append events to the event store, a projector builds the read model, and queries read only the projection; the read side lags the write side.

Both patterns add substantial complexity: eventual consistency between command and query sides, event schema evolution (versioned events and upcasting), the need for snapshots to bound replay time, and a harder mental model.

They are justified by audit requirements, by read and write workloads with very different shapes, or by the need for temporal queries; they are not a default.

Batch and stream processing

Batch processing runs a job over a bounded data set at intervals (nightly aggregation, a report, reconciliation). It is simple, efficient for large volumes, and easy to re-run, but latency is the batch interval. **Stream processing** (Kafka Streams, Flink, Spark Structured Streaming) processes an unbounded sequence of events continuously with latency of milliseconds to seconds, using windows (tumbling, sliding, session) and managed state. Stream processing must handle late and out-of-order events with event-time semantics and watermarks, and must recover its state after failure from checkpoints. A common architecture feeds both: the log is the single source, a stream job maintains low-latency views, and a periodic batch job recomputes and corrects them. A design should choose batch whenever the freshness requirement allows it.

Backpressure

When producers are faster than consumers, the backlog grows without bound unless a mechanism limits it. Options, in order of preference: a **pull-based** consumer that reads at its own pace (the log model); **bounded queues** that block or reject producers when full (a producer receives a clear failure instead of exhausting memory); **load shedding** that drops low-priority work; **autoscaling** consumers on lag; and **rate limiting** at the entry point. A bounded queue with a visible rejection is preferred over an unbounded one that hides the problem until memory or disk is exhausted and delays all later messages by the whole backlog.

1.10.2 Worked example

The following sizes a Kafka-style event log for an assumed order-events pipeline.

Quantity	Value	Arithmetic
Peak event rate	20,000 per second	assumed
Average event size	1 KB	assumed
Peak ingest bandwidth	20 MB/s	20,000 x 1 KB
Retention	7 days	assumed
Data retained (one copy)	about 12.1 TB	20 MB/s x 604,800 s, taking the peak as the average (upper bound)
Replication factor	3	assumed
Disk capacity needed	about 36 TB	12.1 TB x 3
Throughput of one consumer instance	500 events per second	assumed, including a database write
Consumers needed at peak	40	20,000 / 500
Partitions	64	next power of two above 40, leaving room to scale
Lag growth if consumers fall behind by 20%	4,000 events per second	20,000 x 0.20; 14.4 million events in one hour

Sizing of an event log (assumed workload).

The last row quantifies why consumer lag needs an alert: a 20% capacity shortfall accumulates a backlog of 14.4 million events per hour, and with a 500-per-second-per-consumer rate and 40 consumers the backlog clears only if capacity exceeds the arrival rate by a margin. Storage was computed at the peak rate for the full retention, which overstates the real figure; the average rate is used for the actual purchase.

1.10.3 Hands-on

The program below simulates at-least-once delivery with a consumer that crashes between the effect and the acknowledgement, compares a plain consumer with an idempotent one, prints a jittered backoff schedule, and shows per-key ordering under partitioning. It is a single-process simulation of the logic.

sreram

Delivery.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.ArrayList;
import java.util.Map;
import java.util.Random;
import java.util.Set;

public class Delivery {
    record Msg(long id, String key, int amount) { }

    /** The broker redelivers every message that is not acknowledged (at-least-once).
     *  The consumer crashes with probability crashProb after the effect, before the ack. */
    static int[] run(boolean idempotent, double crashProb, long seed) {
        Random rnd = new Random(seed);
        Deque<Msg> queue = new ArrayDeque<>();
        for (int i = 0; i < 10_000; i++) queue.add(new Msg(i, "acct" + (i % 50), 1));
        Map<String, Integer> balance = new HashMap<>();
        Set<Long> processed = new HashSet<>();           // stands in for a processed_ids table
        int deliveries = 0, duplicatesApplied = 0;
        while (!queue.isEmpty()) {
            Msg m = queue.poll();
            deliveries++;
            boolean seen = processed.contains(m.id());
            if (!(idempotent && seen)) {
                balance.merge(m.key(), m.amount(), Integer::sum);     // the side effect
                if (seen) duplicatesApplied++;
                processed.add(m.id());
            }
            if (rnd.nextDouble() < crashProb) queue.addFirst(m);     // ack lost: redelivery
        }
        int total = balance.values().stream().mapToInt(Integer::intValue).sum();
        return new int[] {deliveries, duplicatesApplied, total};
    }

    public static void main(String[] args) {
        System.out.println("10,000 messages, each adds 1 to an account; expected total = 10000");
        System.out.println("probability of a crash between effect and ack: 2%");
        int[] a = run(false, 0.02, 11), b = run(true, 0.02, 11);
        System.out.printf("plain consumer     : deliveries=%d duplicates applied=%d total=%d%n",
                a[0], a[1], a[2]);
        System.out.printf("idempotent consumer: deliveries=%d duplicates applied=%d total=%d%n",
                b[0], b[1], b[2]);

        System.out.println("\nRetry delays: base 100 ms, factor 2, cap 5000 ms, full jitter");
        Random r = new Random(5);
        for (int attempt = 1; attempt <= 8; attempt++) {
            long ceiling = Math.min(5000, 100L * (1L << (attempt - 1)));
            System.out.printf("attempt %d: ceiling %4d ms, chosen delay %4d ms%n",
                    attempt, ceiling, (long) (r.nextDouble() * ceiling));
        }

        System.out.println("\nOrdering: 12 events, 3 keys, 3 partitions, hash(key) mod 3");
        List<List<String>> parts =
                List.of(new ArrayList<>(), new ArrayList<>(), new ArrayList<>());
        String[] keys = {"orderA", "orderB", "orderC"};
        for (int i = 0; i < 12; i++) {
            String k = keys[i % 3];
            parts.get(Math.floorMod(k.hashCode(), 3)).add(k + ":e" + (i / 3 + 1));
        }
        for (int p = 0; p < 3; p++) System.out.println("partition " + p + ": " + parts.get(p));
    }
}
```

Run (JDK 21; deterministic for the fixed seeds) TERMINAL

```
$ java Delivery.java
10,000 messages, each adds 1 to an account; expected total = 10000
probability of a crash between effect and ack: 2%
plain consumer     : deliveries=10198 duplicates applied=198 total=10198
idempotent consumer: deliveries=10198 duplicates applied=0 total=10000
```

Run (JDK 21; deterministic for the fixed seeds) (continued) TERMINAL

```
Retry delays: base 100 ms, factor 2, cap 5000 ms, full jitter
attempt 1: ceiling  100 ms, chosen delay   73 ms
attempt 2: ceiling  200 ms, chosen delay   17 ms
attempt 3: ceiling  400 ms, chosen delay  195 ms
attempt 4: ceiling  800 ms, chosen delay  369 ms
attempt 5: ceiling 1600 ms, chosen delay  717 ms
attempt 6: ceiling 3200 ms, chosen delay 2232 ms
attempt 7: ceiling 5000 ms, chosen delay 1388 ms
attempt 8: ceiling 5000 ms, chosen delay 3799 ms

Ordering: 12 events, 3 keys, 3 partitions, hash(key) mod 3
partition 0: [orderC:e1, orderC:e2, orderC:e3, orderC:e4]
partition 1: [orderA:e1, orderA:e2, orderA:e3, orderA:e4]
partition 2: [orderB:e1, orderB:e2, orderB:e3, orderB:e4]
```

What to notice: with a 2% crash probability, 198 of 10,000 messages are delivered twice, and the plain consumer ends with a total of 10,198, an over-count of about 2%; the idempotent consumer reaches exactly 10,000 although the same 10,198 deliveries occurred. The backoff ceilings double from 100 ms to the 5,000 ms cap, and the chosen delays are spread uniformly below the ceiling. In the ordering section, all events of one key land in one partition in their original order (`e1` to `e4`), and the three keys are spread over three partitions; events of different keys have no defined relative order.

1.10.4 Real-world problems and solutions

Problem 1: duplicate processing

Scenario. A consumer applies a payment-ledger update for each message and commits offsets after processing. A deployment restarts the consumer during processing. **Symptoms.** Some customers are credited twice; the ledger total exceeds the sum of source events by about 0.1%. **Diagnosis.** The ledger contains two entries with the same event id; the consumer log shows a rebalance followed by reprocessing from the last committed offset. **Root cause.** At-least-once delivery re-delivers uncommitted work after a restart or rebalance, and the handler was not idempotent. **Fix.** Store the event id with a unique constraint in the same transaction as the ledger update, and ignore a repeat. Correct the existing duplicates with compensating entries. **Prevention.** Treat every consumer as a target of duplicate delivery; test by killing consumers mid-batch.

Problem 2: lost messages

Scenario. A producer sends with `acks=1`, and a consumer uses auto-commit of offsets every 5 seconds. **Symptoms.** After a broker failure and after a consumer crash, a few hundred events are missing downstream; no error was logged. **Diagnosis.** Comparing source row counts with consumed counts shows a gap near the failure times. Broker logs show a leader change; consumer offsets show commits ahead of processed records. **Root cause.** With `acks=1` the leader acknowledged before the followers copied the record, and the record was lost when the leader failed. Auto-commit advanced the offset for records still being processed, so a crash skipped them (at-most-once behaviour). **Fix.** Use `acks=all` with `min.insync.replicas=2` on a replication factor of 3, `enable.idempotence=true`, `unclean.leader.election.enable=false`, and `enable.auto.commit=false` with a manual commit after processing. Replay the missing range from the source of truth. **Prevention.** Reconcile counts end to end as a routine check, and keep the producer and consumer settings under review.

Problem 3: ordering broken by partitions and retries

Scenario. Order events (`created`, `paid`, `shipped`) are published without a key to a topic with 12 partitions. **Symptoms.** A consumer sometimes receives `shipped` before `paid`, and the order state machine rejects the transition or ends in a wrong state. **Diagnosis.** The three events of one order are located in different partitions, as shown by offsets and partition ids. **Root cause.** Without a key the producer spreads records over partitions, and ordering exists only within a partition. **Fix.** Use the order id as the record key; configure an idempotent producer; process each partition sequentially or dispatch by key; carry an event version so that the consumer can discard stale or out-of-order events. **Prevention.** State the ordering requirement with its scope (per order) in the design, and choose the key from it.

Problem 4: poison message blocks a partition

Scenario. A consumer throws an exception on a record with an unexpected field and retries forever. **Symptoms.** Lag on one partition grows steadily while the others are normal; the consumer log repeats one stack trace. **Diagnosis.** The committed offset of the lagging partition does not move; the same offset appears in every log line. **Root cause.** The failure is permanent but treated as transient, so the consumer never advances. **Fix.** Classify errors, limit retries (for example 3 attempts with backoff), publish the record to a dead-letter topic with the error details, commit the offset, and alert on the DLQ depth. Fix the parser and redrive. **Prevention.** Validate the schema at the producer, use a schema registry with compatibility rules, and include a DLQ in every consumer by default.

Problem 5: consumer lag grows and a rebalance storm follows

Scenario. A downstream database slows down, so batches take longer than `max.poll.interval.ms`. **Symptoms.** Consumers are repeatedly removed from the group, partitions are reassigned in a loop, throughput falls to near zero, and lag grows. **Diagnosis.** Group logs show frequent rebalances; consumers report that the poll interval was exceeded. **Root cause.** Processing time per poll exceeded the liveness limit, so the broker considered the consumer dead; each rebalance forces uncommitted batches to be reprocessed, which lengthens the work further. **Fix.** Reduce `max.poll.records`, raise `max.poll.interval.ms` to a safe margin, move slow work to a bounded worker pool with the polling thread kept responsive, and apply backpressure to the database calls. Scale consumers up to the partition count. **Prevention.** Alert on lag growth rate and rebalance frequency, and load test with a slow dependency.

1.10.5 Questions to expect

1. **Queue or log?** A queue distributes work to competing consumers and deletes messages after acknowledgement. A log retains an ordered, partitioned record that independent consumer groups read at their own offsets and can replay. Select a log for event streams and several consumers of the same data, a queue for task distribution.
2. **Can exactly-once delivery be guaranteed?** Not in the strict end-to-end sense. Systems provide at-least-once delivery combined with idempotent processing, or transactions that tie the output and the offset together, which gives exactly-once effects.
3. **How is ordering guaranteed?** Per partition or message group. Use the entity id as the key so that related events share a partition; use one consumer per partition and an idempotent producer. Global ordering requires one partition.
4. **What is a dead-letter queue?** A destination for messages that exceeded their retry limit or are permanently invalid. It keeps the main flow moving, preserves the failed data for analysis, and is redriven after the fix. It must be monitored.
5. **What problem does the outbox pattern solve?** The dual write of a database update and a message publication. The event is stored in the same transaction as the change and published by a relay, so neither is lost.
6. **How is a consumer scaled?** Add instances up to the partition count; beyond that, increase partitions (planning for the key-mapping effect) or process each partition's records in parallel by key. Autoscale on lag.
7. **What does consumer lag indicate?** The distance between the latest produced offset and the committed offset. A growing lag means the consumers are slower than the producers; the cause is investigated in processing time, downstream latency, rebalances or hot keys.
8. **When is event sourcing appropriate?** When the history itself is a requirement (audit, temporal queries), or when read models must be rebuilt. It costs complexity in schema evolution, snapshots and eventual consistency.
9. **How is backpressure applied?** Pull-based consumption, bounded queues with explicit rejection, load shedding of low-priority work, rate limits at the entry, and autoscaling on lag.
10. **Batch or stream?** Batch when the freshness requirement tolerates the interval; it is simpler and cheaper. Stream when latency of seconds is required, and then plan for late data, state and checkpointing.

KEY POINTS

- Asynchronous messaging decouples availability, absorbs spikes and enables fan-out, at the cost of eventual consistency, duplicates and weaker ordering.
- Delivery is at-least-once in practice; correctness comes from idempotent consumers, which is how effectively-once processing is achieved.
- Ordering exists per partition; choose the key from the entity whose events must stay ordered, and expect that hot keys overload one partition.
- Retry transient errors with backoff and jitter, send permanent failures to a monitored dead-letter queue, and never let a poison message block a partition.
- Use the transactional outbox or CDC instead of a dual write; monitor consumer lag as the primary health signal.
- CQRS and event sourcing are powerful but costly; adopt them for audit, divergent read and write shapes, or replay needs.

1.11 Common Infrastructure Components

Most system designs reuse a small set of building blocks: a limiter that protects a service from excess load, a compact probabilistic membership test, a search index, storage and upload of large files, a channel for pushing data to clients, a way for services to find each other, an entry point that applies common policies, and an index for geographic queries. Each has a standard design and a known set of failure modes. This chapter presents them at the level required to use them in a design and to explain their behaviour, with measurements from small implementations. Load balancing, DNS and CDN placement are treated in the chapter on load balancing and networking, and the data store choices in the chapter on choosing the data store.

1.11.1 Concept and mechanism

Rate limiting

A rate limiter bounds the number of requests that a client (user, API key, IP address, tenant) may make in a period. It protects the service from overload and abuse, enforces fair use between tenants, and caps cost. A rejected request receives HTTP status 429 (Too Many Requests), commonly with a `Retry-After` header and headers that report the limit and the remaining quota.

Algorithm	Mechanism	Strength	Weakness
Fixed window	a counter per window (for example per minute); reset at the boundary	minimal state (one counter), O(1)	a client can send twice the limit around a boundary
Sliding window log	keep the timestamp of each accepted request; count those within the last window	exact	memory proportional to the limit per client
Sliding window counter	weight the previous window's count by the overlap with the sliding window and add the current count	two counters per client, close to exact	an approximation that assumes uniform arrivals in the previous window
Token bucket	tokens accrue at a fixed rate up to a capacity; each request consumes one	permits bursts up to the capacity while holding the long-run rate; two values of state	burst can pass at full speed to the backend
Leaky bucket	requests enter a queue that drains at a constant rate; overflow is rejected	smooths the output to a constant rate	queued requests add latency; a burst fills the queue

Rate limiting algorithms.

The leaky bucket is used in two forms: as a queue that shapes traffic to a steady output rate, and as a meter (a counter that drains at a fixed rate and rejects on overflow), which behaves almost identically to a token bucket in the simulation below.

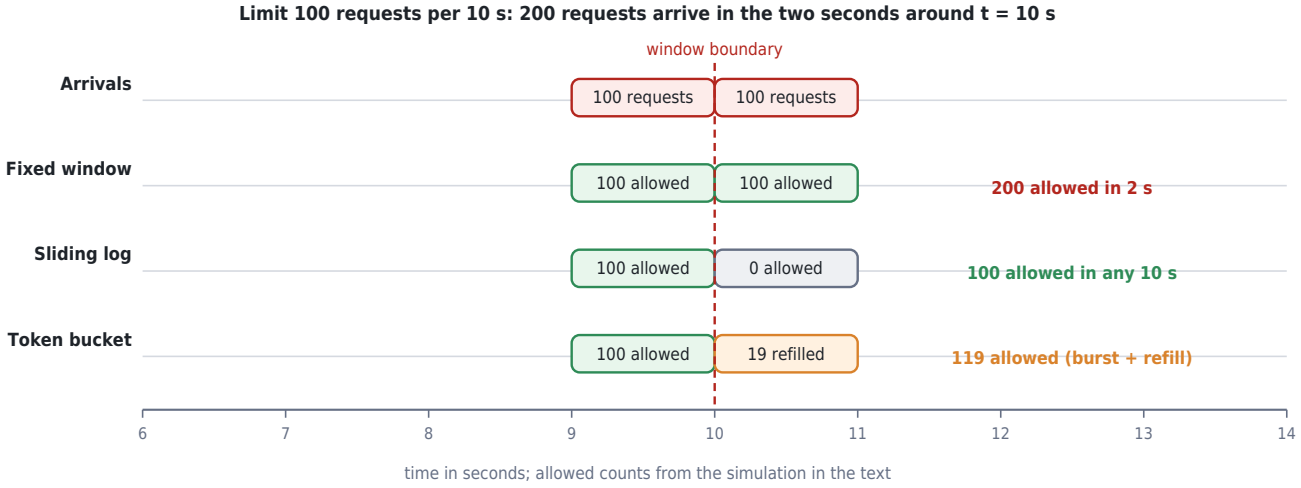


Figure 1.50. Two bursts of 100 requests that straddle a window boundary: a fixed window allows 200 requests in two seconds, a sliding log allows 100, and a token bucket allows 119 (the full bucket plus refill).

Distributed limiting. With several application instances, the counters must be shared or the limit is multiplied by the instance count. The usual design keeps the state in a central, fast store (Redis) and updates it with an atomic script, so that the read, the decision and the write cannot interleave with another instance. The decision costs one network round trip per request, which is acceptable for most APIs. Alternatives reduce that cost at the price of precision: each instance holds a local quota that is refreshed periodically from the central store, or each instance enforces limit / N locally (which is wrong when load is uneven across instances). Limiting at the API gateway, where all traffic is already aggregated, places the problem in one tier.

Token bucket as a Redis Lua script (not run here: no Redis server was available)

```
-- KEYS[1] = bucket key; ARGV[1] = refill rate (tokens per second); ARGV[2] = capacity
local rate, capacity = tonumber(ARGV[1]), tonumber(ARGV[2])
local t = redis.call('TIME')                          -- server clock: no skew between app instances
local now = t[1] * 1000 + math.floor(t[2] / 1000)  -- milliseconds
local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens, ts = tonumber(data[1]), tonumber(data[2])
if tokens == nil then tokens, ts = capacity, now end
tokens = math.min(capacity, tokens + (now - ts) * rate / 1000)
local allowed = 0
if tokens >= 1 then tokens = tokens - 1; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity / rate * 1000) * 2)   -- idle buckets expire
return allowed
```

Redis executes a script atomically, so concurrent requests for one key are serialised. Reading the clock inside the script avoids dependence on application clocks (calling `TIME` in a script is permitted in Redis 5 and later, where scripts replicate by effect).

Bloom filters

A Bloom filter answers "is x in the set" using a bit array of m bits and k hash functions. Adding an element sets the k bits selected by its hashes. A query checks the same k bits: if any bit is 0 the element is **definitely absent**; if all are 1 it is **possibly present**. The filter never returns a false negative, and it returns a false positive with probability approximately p = (1 - e^(-kn/m))^k after n insertions. For a target p, the required size is m/n = -ln p / (ln 2)^2 bits per element (about 9.6 bits for 1%, 14.4 bits for 0.1%), with the optimal number of hash functions k = (m/n) ln 2. The structure does not support deletion (a counting Bloom filter or a cuckoo filter does), and its false positive rate rises sharply when more than the designed number of elements is inserted.

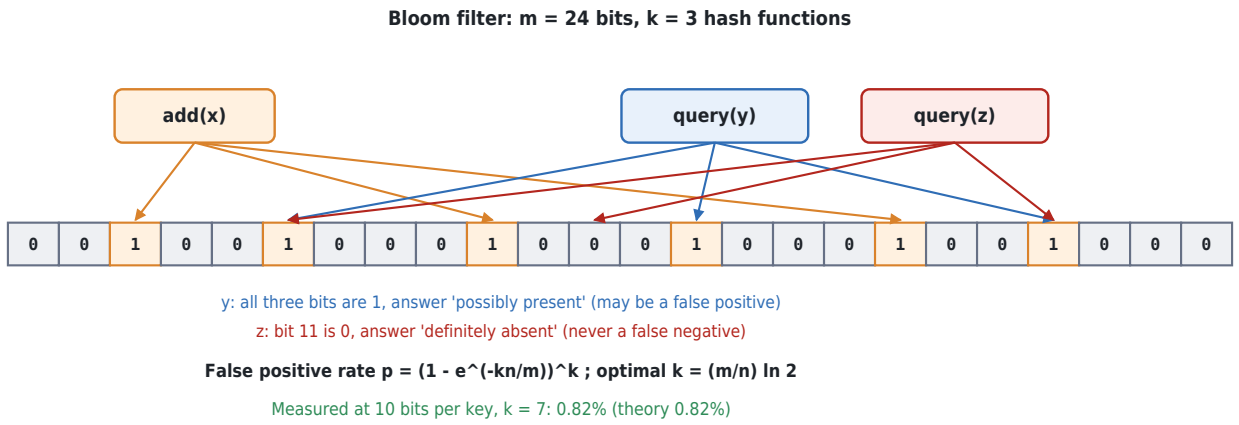


Figure 1.51. A Bloom filter with 24 bits and 3 hash functions: a query whose bits are all set is only possibly present, whereas a single zero bit proves absence.

Typical uses are to avoid expensive lookups for absent keys (a database or cache miss for nonexistent ids, the SSTable lookup in LSM-tree stores, a crawler that skips URLs already seen) and to test membership in sets too large for memory in exact form.

Search and the inverted index

A B-tree index finds rows by an exact or prefix key; it cannot find documents that contain a word in the middle of a text or rank them by relevance. A search engine builds an **inverted index**: for each term, a sorted list of the identifiers of documents containing it (the posting list), usually with term frequencies and positions. Indexing passes the text through an analyser (tokenisation, lower-casing, stop-word removal, stemming). A query is analysed in the same way, the posting lists of its terms are fetched, and they are intersected (AND) or merged (OR). The matches are ranked by a scoring function such as TF-IDF or BM25, which favours documents with frequent occurrences of rare query terms.

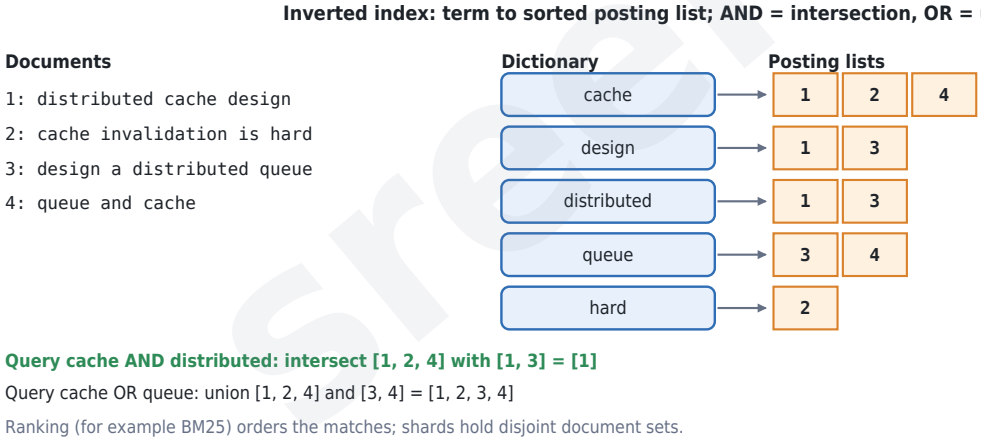


Figure 1.52. An inverted index over four documents: each term points to a sorted posting list, and a boolean query is answered by intersecting or merging the lists.

At scale the index is divided into **shards**, each holding a disjoint set of documents with its own inverted index, and each shard is replicated. A query is sent to one copy of every shard, each returns its top results, and a coordinator merges them (scatter-gather, with the tail-latency consequence discussed in the chapter on replication and partitioning). New documents become searchable after a **refresh** that makes the recent segment visible, commonly on a one-second interval, so search is near-real-time and never strongly consistent with the source. The index is a derived copy: the system of record is elsewhere, and a reindex path must exist.

Object storage and chunked upload

Object storage stores immutable blobs addressed by a bucket and key, with high durability from replication or erasure coding across failure domains. Applications avoid streaming large files through application servers. The server authorises the transfer and issues a time-limited **signed URL**; the client then uploads directly to the object store.

Large files are uploaded in **parts** (multipart or chunked upload): the client splits the file into parts of a fixed size (for example 16 MiB), uploads them in parallel, retries only the failed parts, and completes the upload with the list of part identifiers, whereupon the store assembles the object. This provides resumability (an interrupted upload resumes at

the first missing part), parallelism and bounded memory use. Each part carries a checksum, and the full-object checksum is verified at completion.

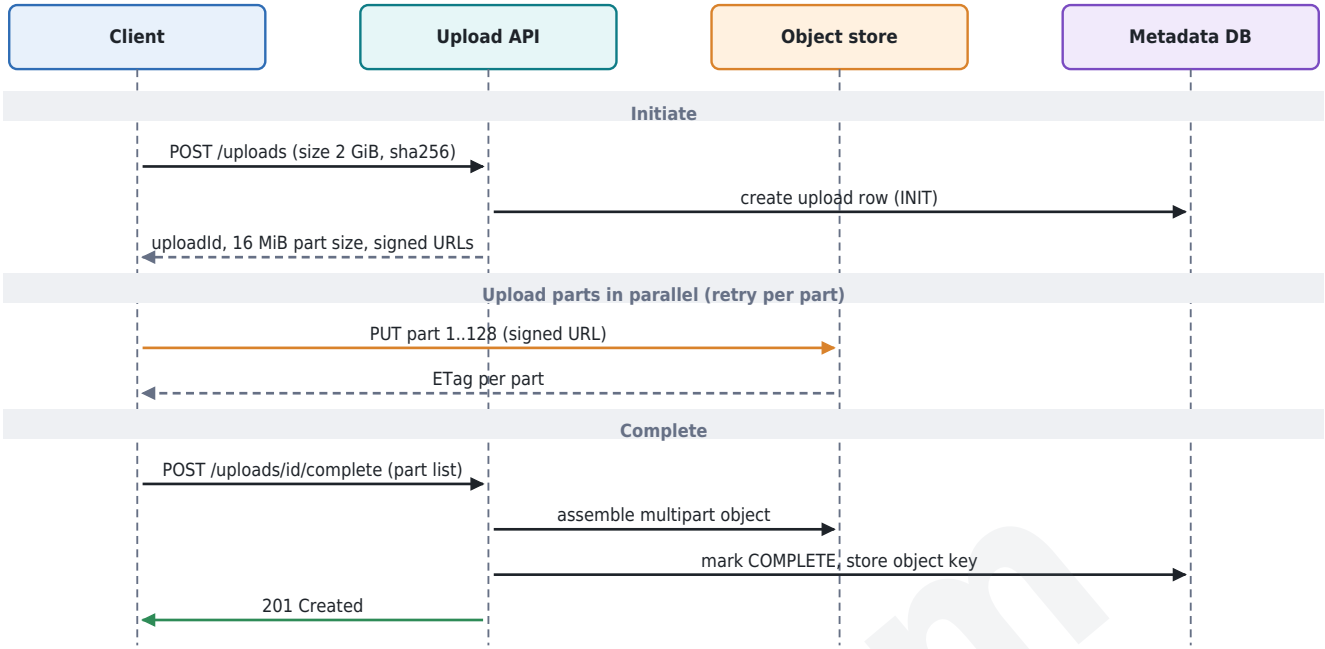


Figure 1.53. The chunked upload protocol: initiate and obtain signed URLs, upload parts in parallel with per-part retry, and complete with the part list so that the object is assembled and the metadata marked complete.

Metadata (owner, name, size, content type, object key, state) lives in a database, and an object becomes visible only after the metadata row reaches the COMPLETE state. Abandoned uploads are removed by a lifecycle rule that deletes incomplete parts after a number of days.

Real-time delivery to clients

Three mechanisms push data from the server to a client.

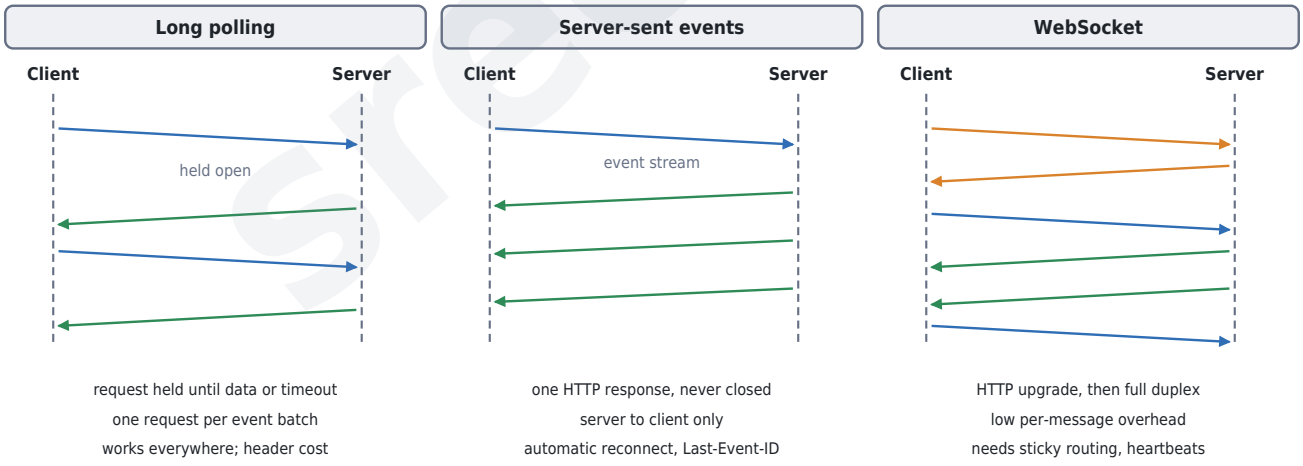


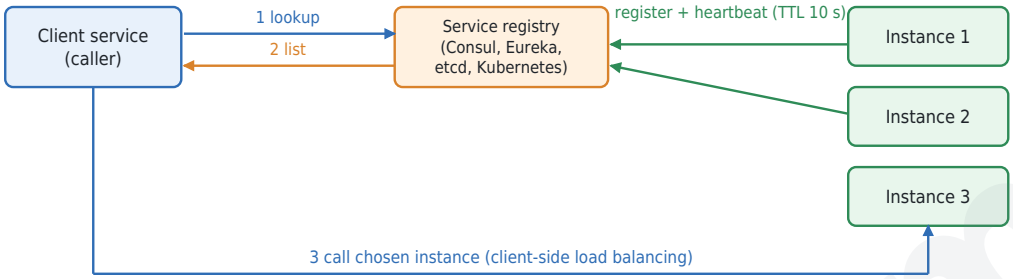
Figure 1.54. Long polling holds each request until data exists, server-sent events keep one response open for a one-way stream, and a WebSocket upgrades the connection to full duplex.

- **Long polling.** The client issues a request that the server holds until an event arrives or a timeout expires; the client then immediately issues the next request. It works through any proxy and needs no special support, but each event costs a full request cycle with headers.
- **Server-sent events (SSE).** One long-lived HTTP response streams events in a text format; the browser reconnects automatically and can resume from the last event id. It is one-way (server to client), simple and proxy-friendly.
- **WebSocket.** An HTTP request upgrades the connection to a persistent, full-duplex channel with small frames. It suits chat, collaboration and games, where the client also sends frequently.

A fleet of WebSocket servers holds connection state in memory: each connection occupies a socket and buffers (assumed 10 to 50 KB), so 1,000,000 concurrent connections need 10 to 50 GB across the fleet. To deliver a message to a user connected to an arbitrary server, the system keeps a mapping from user to gateway server (in a shared store with a TTL) and routes through a publish-subscribe channel or a message bus. Clients send heartbeats to detect dead connections, reconnect with jittered backoff, and resume from a sequence number so that missed messages are replayed.

Service discovery and the API gateway

Instances of a service appear and disappear (autoscaling, deployments, failures), so callers cannot rely on fixed addresses. A **service registry** holds the current list of healthy instances. Instances register themselves (or the platform registers them) and send periodic heartbeats; an entry expires when heartbeats stop.



Client-side discovery: caller caches the list and balances (low latency, library per language).

Server-side discovery: caller uses a stable name; a load balancer or proxy consults the registry.

Registry entries expire without heartbeats, so crashed instances leave the list after the TTL.

Stale entries are the main failure mode: callers need timeouts, retries and outlier ejection.

Figure 1.55. Client-side discovery: instances register with a heartbeat TTL, the caller looks up the list and balances across it; the principal risk is a stale entry for a crashed instance.

In **client-side discovery** the caller queries the registry, caches the list and balances requests itself; in **server-side discovery** the caller uses a stable name and a load balancer or proxy consults the registry (Kubernetes services and DNS work this way). DNS-based discovery is simple, but caching by clients and resolvers delays the removal of a failed address. Whatever the mechanism, callers need timeouts, bounded retries and ejection of instances that fail repeatedly (outlier detection).

An **API gateway** is the single entry point for external clients. It performs the cross-cutting work once instead of in every service: TLS termination, authentication and token validation, authorisation checks, rate limiting, request routing and versioning, request and response transformation, response caching, and logging, metrics and trace-id injection. It can also aggregate several backend calls into one client response. The costs are an extra network hop, a component that must be highly available and scaled, and the risk of accumulating business logic in it.

Geospatial indexing

A query such as "drivers within 2 km" cannot use a B-tree on latitude and longitude efficiently, because two independent ranges do not map to one sort order. Geospatial indexes map two-dimensional locations to one-dimensional keys or to a hierarchical structure.

Geohash. The globe is bisected alternately by longitude and latitude, and the resulting bit string is encoded in base 32. Each additional character refines the cell by a factor of 32. Points in the same cell share a prefix, so a prefix query on an ordinary index returns the points in a cell. A longer common prefix means a smaller shared cell, but the converse fails: two nearby points on opposite sides of a cell boundary can have different prefixes, so a search queries the target cell and its eight neighbours.

Sreeram

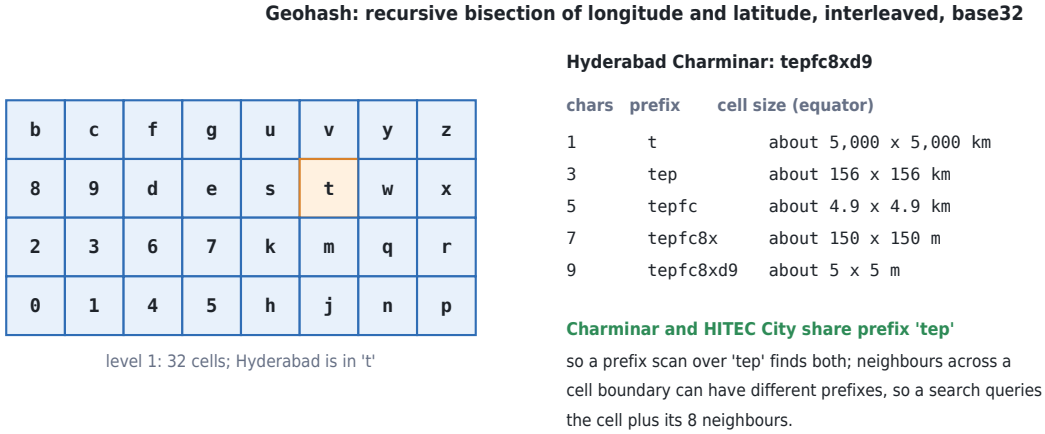


Figure 1.56. The geohash cell grid at the first level and the effect of prefix length on cell size, with the real encoding of a point in Hyderabad.

Quadtree. A tree in which each node covers a rectangle and splits into four children when it holds more than a threshold number of points (for example 100). Dense areas become deep, and sparse areas stay shallow, so cells adapt to density, unlike the fixed grid of a geohash. The tree usually lives in memory on a service and is rebuilt or updated as points move.

S2 and H3. S2 (Google) projects the sphere onto the faces of a cube and orders cells along a Hilbert curve, so that cells close in space are usually close in the one-dimensional id, and a region is covered by a small set of cell id ranges. H3 (Uber) uses hexagonal cells, which have uniform neighbour distances. Both provide hierarchical cell ids with covering and neighbour operations.

Frequently updated locations (drivers every few seconds) favour a simple in-memory structure or key-value store keyed by cell, with the cell id recomputed on each update; static points of interest favour a persistent index.

1.11.2 Worked example

Quantity	Value	Arithmetic
Rate limiter: active clients	10,000,000	assumed
State per client (hash with two fields plus key overhead)	about 100 B	assumed
Rate limiter memory	about 1 GB	10,000,000 x 100 B
Rate limiter operations	50,000 per second	one scripted call per request at 50,000 requests per second (assumed)
Bloom filter: URLs to track	1,000,000,000	assumed
Bits per element for 1% false positives	9.585	-ln(0.01) / (ln 2)^2
Bloom filter memory	about 1.2 GB	1,000,000,000 x 9.585 / 8 bytes
Hash functions	7	(m/n) ln 2 = 6.64, rounded
Exact set of the same URLs, 50 B per URL	50 GB	1,000,000,000 x 50 B
Upload: file size and part size	2 GiB, 16 MiB	assumed
Number of parts	128	2,048 MiB / 16 MiB
WebSocket connections	1,000,000	assumed
Memory per connection	20 KB	assumed
WebSocket fleet memory	20 GB	1,000,000 x 20 KB
Gateway servers at 50,000 connections each	20	1,000,000 / 50,000, plus spare capacity for failure

Sizing of rate limiter, Bloom filter, upload and WebSocket tiers (assumed workloads).

The Bloom filter uses one fortieth of the memory of an exact set (1.2 GB against 50 GB) at the price of a 1% false positive rate, which for a crawler means that about 1% of unseen URLs are skipped. The rate limiter fits in the memory of one Redis node, and its operation rate is the constraint that decides on replication or partitioning by client key.

1.11.3 Hands-on

Comparing the rate limiting algorithms

The program implements the five algorithms with simulated time and runs two scenarios: two bursts across a window boundary, and Poisson arrivals at 1.5 times the allowed rate.

ratelimit.py PYTHON

```python
"""Five rate limiting algorithms; limit = 100 requests per 10 s. Simulated time, no sleeping."""
import random
from collections import deque

LIMIT, WINDOW = 100, 10.0


class FixedWindow:
    def __init__(self):
        self.window, self.count = None, 0

    def allow(self, t):
        w = int(t // WINDOW)
        if w != self.window:
            self.window, self.count = w, 0
        if self.count < LIMIT:
            self.count += 1
            return True
        return False


class SlidingLog:
    def __init__(self):
        self.log = deque()

    def allow(self, t):
        while self.log and self.log[0] <= t - WINDOW:
            self.log.popleft()
        if len(self.log) < LIMIT:
            self.log.append(t)
            return True
        return False


class SlidingCounter:
    """Weights the previous window's count by the fraction of it still inside the sliding window."""
    def __init__(self):
        self.window, self.cur, self.prev = None, 0, 0

    def allow(self, t):
        w = int(t // WINDOW)
        if w != self.window:
            self.prev = self.cur if self.window == w - 1 else 0
            self.cur, self.window = 0, w
        estimate = self.prev * (1 - (t % WINDOW) / WINDOW) + self.cur
        if estimate < LIMIT:
            self.cur += 1
            return True
        return False


class TokenBucket:
    def __init__(self):
        self.tokens, self.last, self.rate = float(LIMIT), 0.0, LIMIT / WINDOW

    def allow(self, t):
        self.tokens = min(LIMIT, self.tokens + (t - self.last) * self.rate)
        self.last = t
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```

sreeram

ratelimit.py (continued) PYTHON

```python
class LeakyBucketMeter:
    """Bucket of capacity LIMIT that drains at LIMIT/WINDOW per second; a request adds one unit."""
    def __init__(self):
        self.level, self.last, self.rate = 0.0, 0.0, LIMIT / WINDOW

    def allow(self, t):
        self.level = max(0.0, self.level - (t - self.last) * self.rate)
        self.last = t
        if self.level + 1 <= LIMIT:
            self.level += 1
            return True
        return False


ALGOS = [("fixed window", FixedWindow), ("sliding log", SlidingLog),
         ("sliding counter", SlidingCounter), ("token bucket", TokenBucket),
         ("leaky bucket (meter)", LeakyBucketMeter)]

# Scenario A: 100 requests at 9.00-9.99 s, then 100 at 10.00-10.99 s (straddling a window boundary)
reqs = [9.0 + i * 0.01 for i in range(100)] + [10.0 + i * 0.01 for i in range(100)]
print("Scenario A: 100 requests at t=9.00..9.99 s, then 100 at t=10.00..10.99 s")
for name, cls in ALGOS:
    a = cls()
    print(f"  {name:22s} allowed {sum(a.allow(t) for t in reqs):3d} of 200 within 2 s")

# Scenario B: Poisson arrivals at 15 requests per second for 60 s (limit is 10 per second)
random.seed(7)
t, reqs = 0.0, []
while t < 60:
    t += random.expovariate(15.0)
    reqs.append(t)
print(f"\nScenario B: Poisson arrivals, mean 15 req/s for 60 s ({len(reqs)} requests)")
for name, cls in ALGOS:
    a = cls()
    ok = [t for t in reqs if a.allow(t)]
    worst, j = 0, 0
    for i, t in enumerate(ok):
        while ok[j] <= t - WINDOW:
            j += 1
        worst = max(worst, i - j + 1)
    print(f"  {name:22s} allowed {len(ok):4d}, most in any 10 s window: {worst}")
```

Run (Python 3, deterministic seed) TERMINAL

```
$ python3 ratelimit.py
Scenario A: 100 requests at t=9.00..9.99 s, then 100 at t=10.00..10.99 s
  fixed window           allowed 200 of 200 within 2 s
  sliding log            allowed 100 of 200 within 2 s
  sliding counter        allowed 110 of 200 within 2 s
  token bucket           allowed 119 of 200 within 2 s
  leaky bucket (meter)   allowed 119 of 200 within 2 s

Scenario B: Poisson arrivals, mean 15 req/s for 60 s (922 requests)
  fixed window           allowed  601, most in any 10 s window: 126
  sliding log            allowed  600, most in any 10 s window: 100
  sliding counter        allowed  600, most in any 10 s window: 105
  token bucket           allowed  700, most in any 10 s window: 171
  leaky bucket (meter)   allowed  700, most in any 10 s window: 171
```

What to notice: in scenario A the fixed window admits all 200 requests, double the limit, within two seconds, while the sliding log admits exactly 100 and the sliding counter 110 (its approximation error). The token bucket and leaky-bucket meter admit 119, because the bucket starts full and refills about 19 tokens in the interval. In scenario B the long-run admission is 600 requests (10 per second for 60 seconds) for the window algorithms and 700 for the bucket algorithms, the extra 100 being the initial burst capacity. The maximum in any 10-second window shows the burst behaviour: 100 for the sliding log, 126 for the fixed window, 171 for the buckets. The sliding log is exact but holds up to 100 timestamps per client, whereas the other algorithms hold two or three numbers.

A Bloom filter and its false positive rate

BloomDemo.java JAVA

```java
import java.util.BitSet;
import java.util.Random;

public class BloomDemo {
    final BitSet bits;
    final int m, k;                                   // m bits, k hash functions

    BloomDemo(int m, int k) {
        this.m = m;
        this.k = k;
        this.bits = new BitSet(m);
    }

    static long mix(long x) {                         // 64-bit finaliser (MurmurHash3 style)
        x ^= x >>> 33; x *= 0xff51afd7ed558ccdL;
        x ^= x >>> 33; x *= 0xc4ceb9fe1a85ec53L;
        return x ^ (x >>> 33);
    }

    // double hashing: the i-th index is h1 + i * h2, which simulates k independent hashes
    private int index(long h1, long h2, int i) {
        return (int) Long.remainderUnsigned(h1 + i * h2, m);
    }

    void add(long key) {
        long h1 = mix(key), h2 = mix(key ^ 0x9e3779b97f4a7c15L) | 1;
        for (int i = 0; i < k; i++) bits.set(index(h1, h2, i));
    }

    boolean mightContain(long key) {
        long h1 = mix(key), h2 = mix(key ^ 0x9e3779b97f4a7c15L) | 1;
        for (int i = 0; i < k; i++) if (!bits.get(index(h1, h2, i))) return false;
        return true;
    }

    public static void main(String[] args) {
        int n = 1_000_000;
        System.out.println("n = 1,000,000 inserted keys; 1,000,000 probes of keys never inserted");
        System.out.println("bits/key  k | memory   | measured FPR | theory (1-e^(-kn/m))^k");
        for (int[] cfg : new int[][] {{5, 3}, {10, 7}, {15, 10}, {10, 3}}) {
            int bitsPerKey = cfg[0], k = cfg[1];
            BloomDemo f = new BloomDemo(n * bitsPerKey, k);
            for (long i = 0; i < n; i++) f.add(i);
            int falsePositives = 0, probes = 1_000_000;
            Random r = new Random(1);
            for (int i = 0; i < probes; i++) {
                if (f.mightContain(n + 1L + r.nextInt(Integer.MAX_VALUE))) falsePositives++;
            }
            double theory = Math.pow(1 - Math.exp(-(double) k * n / f.m), k);
            System.out.printf("%8d %2d | %.2f MiB | %.4f%%     | %.4f%%%n", bitsPerKey, k,
                    f.m / 8.0 / 1048576, 100.0 * falsePositives / probes, 100 * theory);
        }
    }
}
```

Run (JDK 21; deterministic for the fixed seed) TERMINAL

```
$ java BloomDemo.java
n = 1,000,000 inserted keys; 1,000,000 probes of keys never inserted
bits/key  k | memory   | measured FPR | theory (1-e^(-kn/m))^k
       5  3 | 0.60 MiB | 9.2211%     | 9.1849%
      10  7 | 1.19 MiB | 0.8165%     | 0.8194%
      15 10 | 1.79 MiB | 0.0763%     | 0.0744%
      10  3 | 1.19 MiB | 1.7438%     | 1.7411%
```

What to notice: the measured false positive rate matches the formula to within sampling error (0.82% measured, 0.82% predicted at 10 bits per element and k = 7). Five bits per element give 9.2%, fifteen bits give 0.076%. The last row shows that a poor choice of k (3 instead of 7 at 10 bits per element) doubles the rate to 1.74%. Double hashing derives the k indexes from two hash values, a standard way to avoid computing k independent hashes.

Geohash and an inverted index

geo.py PYTHON

```python
B32 = "0123456789bcdefghjkmnpqrstuvwxyz"


def geohash(lat, lon, chars=9):
    """Interleave longitude and latitude bisection bits (longitude first), 5 bits per character."""
    lat_r, lon_r, bits, even = [-90.0, 90.0], [-180.0, 180.0], [], True
    while len(bits) < chars * 5:
        rng, value = (lon_r, lon) if even else (lat_r, lat)
        mid = (rng[0] + rng[1]) / 2
        if value >= mid:
            bits.append(1)
            rng[0] = mid
        else:
            bits.append(0)
            rng[1] = mid
        even = not even
    return "".join(B32[int("".join(map(str, bits[i:i + 5])), 2)] for i in range(0, len(bits), 5))


places = {"Hyderabad Charminar": (17.3616, 78.4747),
          "Hyderabad HITEC City": (17.4435, 78.3772),
          "Bengaluru MG Road": (12.9753, 77.6068),
          "London Eye": (51.5033, -0.1195),
          "Paris Louvre": (48.8606, 2.3376)}
for name, (lat, lon) in places.items():
    print(f"{name:22s} {geohash(lat, lon, 9)}")
print("check:", geohash(57.64911, 10.40744, 11), "(reference value u4pruydqqvj)")

print("\nchars | cell width x height at the equator (km)")
for c in range(3, 9):
    lon_bits, lat_bits = (c * 5 + 1) // 2, (c * 5) // 2
    print(f"{c:5d} | {360 / 2 ** lon_bits * 111.32:8.2f} x {180 / 2 ** lat_bits * 110.57:8.2f}")

# inverted index with boolean queries
docs = {1: "distributed cache design", 2: "cache invalidation is hard",
        3: "design a distributed queue", 4: "queue and cache"}
index = {}
for doc_id, text in docs.items():
    for term in text.split():
        index.setdefault(term, set()).add(doc_id)
print("\ncache        ->", sorted(index["cache"]))
print("distributed  ->", sorted(index["distributed"]))
print("cache AND distributed ->", sorted(index["cache"] & index["distributed"]))
print("cache OR queue        ->", sorted(index["cache"] | index["queue"]))
```

Run (Python 3) TERMINAL

```
$ python3 geo.py
Hyderabad Charminar    tepfc8xd9
Hyderabad HITEC City   tepepx2xc
Bengaluru MG Road      tdr1vfpd9
London Eye             gcpuvrbur
Paris Louvre           u09tvny1w
check: u4pruydqqvj (reference value u4pruydqqvj)

chars | cell width x height at the equator (km)
    3 |   156.54 x   155.49
    4 |    39.14 x    19.44
    5 |     4.89 x     4.86
    6 |     1.22 x     0.61
    7 |     0.15 x     0.15
    8 |     0.04 x     0.02

cache        -> [1, 2, 4]
distributed  -> [1, 3]
cache AND distributed -> [1]
cache OR queue        -> [1, 2, 3, 4]
```

What to notice: the encoding of the test point matches the published reference value `u4pruydqqvj`. The two Hyderabad locations, about 14 km apart, share only the three-character prefix `tep`, and a 3-character cell is about 156 km wide, so a prefix of 5 characters (4.9 km cells) is needed for a 5 km search, together with the neighbouring cells. The inverted index answers the boolean queries by set intersection and union of the posting lists.

1.11.4 Real-world problems and solutions

Problem 1: the rate limiter is inconsistent across nodes

Scenario. A limit of 100 requests per minute is enforced independently on each of 10 API instances behind a load balancer. **Symptoms.** Clients are able to make up to 1,000 requests per minute; other clients are rejected while sending only 40, because their requests concentrated on one instance. **Diagnosis.** Per-instance counters show different values for the same key; the accepted total per client exceeds the limit. **Root cause.** The counters are local, so the effective limit is the limit times the instance count, and uneven routing makes rejection arbitrary. **Fix.** Keep the state in a shared store and update it atomically (a Lua script or an atomic increment with expiry). Where the extra round trip is unacceptable, use local quotas leased from the shared store in batches, accepting bounded overshoot. Decide whether the limiter fails open or closed when the store is unavailable (open for protective limits that must not cause an outage; closed for security limits such as login attempts). **Prevention.** State the accuracy requirement of each limit, and test with multiple instances.

Problem 2: a Bloom filter saturates

Scenario. A filter was sized for 100 million ids at 1% false positives, and the data grew to 400 million. **Symptoms.** The share of requests that pass the filter and still miss in the database rises from 1% to about 40%; the filter no longer protects the database. **Diagnosis.** The fraction of set bits has grown from about 50% to over 90%; the measured false positive rate matches the formula with n = 400 million. **Root cause.** The false positive rate depends on the number of inserted elements relative to the size, and growth beyond the design capacity saturates the bit array. **Fix.** Rebuild a larger filter from the source data (or use a scalable Bloom filter that adds layers), and replace the old filter atomically. **Prevention.** Track the fill ratio and the element count, size for the growth horizon, and schedule rebuilds before the fill ratio passes 50%.

Problem 3: reconnect storm after a gateway restart

Scenario. A deployment restarts a WebSocket gateway that holds 50,000 connections. **Symptoms.** All 50,000 clients reconnect within a second, the remaining gateways reach their connection limits and CPU saturation from TLS handshakes, authentication calls overload the identity service, and the cascade removes more gateways. **Diagnosis.** Connection-rate graphs show a spike to the order of the whole connection population; handshake CPU and authentication latency rise together. **Root cause.** Clients reconnect immediately and at the same time, and the system has no admission control for new connections. **Fix.** Clients reconnect with exponential backoff and jitter; the gateway drains connections gradually on shutdown (closing them in batches with a randomised close time); new-connection rate limits and a token cache protect the identity service. **Prevention.** Roll deployments one node at a time with a drain period, and load test the reconnect of a full node.

Problem 4: nearby points missed at a geohash cell boundary

Scenario. A "nearby drivers" query uses the geohash prefix of the rider's location. **Symptoms.** Drivers 100 m away are missing from results when the rider stands close to a cell edge; the problem appears near specific streets. **Diagnosis.** Reproducing with the coordinates shows that rider and driver have different prefixes at the chosen precision. **Root cause.** Adjacent points across a cell boundary do not share a prefix; a single-cell query covers only a part of the search disc. **Fix.** Query the rider's cell and its eight neighbours, then filter the candidates by exact distance; choose a precision whose cell is at least as large as the search radius. **Prevention.** Test with points on both sides of cell boundaries and near the poles and the antimeridian, where cells distort.

Problem 5: stale registry entries send traffic to dead instances

Scenario. An instance is killed during a deployment, but its registry entry persists for the 30-second TTL. **Symptoms.** A share of requests (one in N) fail with connection errors for up to 30 seconds after each deployment. **Diagnosis.** Failed requests all target the addresses of terminated instances; the registry lists them as healthy until expiry. **Root cause.** Failure detection is bounded by the heartbeat TTL, and the instance did not deregister before it terminated. **Fix.** Deregister on shutdown and drain connections before termination; have callers retry on another instance after a connection failure and eject instances that fail repeatedly; shorten the TTL within the limits of registry load. **Prevention.** Include a graceful shutdown hook in every service and verify with a deployment test that error counts stay at zero.

1.11.5 Questions to expect

1. **Compare the rate limiting algorithms.** Fixed window is simple and allows a double burst at the boundary. Sliding log is exact but uses memory per request. Sliding counter approximates the sliding window with two counters. Token bucket allows controlled bursts at a fixed average rate. Leaky bucket smooths the output rate. Token bucket and sliding counter are common defaults.
2. **How is a rate limiter distributed?** Shared state in Redis updated by an atomic script, or local quotas leased in batches, applied at the API gateway; define fail-open or fail-closed behaviour when the store is unavailable.
3. **How does a Bloom filter work and when is it used?** k hashes set bits in an array; a query returns definitely absent or possibly present, with no false negatives and a tunable false positive rate. It protects expensive lookups for absent keys and deduplicates huge sets in small memory.
4. **How is the Bloom filter size computed?** Bits per element m/n = -ln p / (ln 2)^2, about 9.6 for 1%, and k = (m/n) ln 2. Total memory is n times that, divided by 8, in bytes.
5. **What is an inverted index?** A map from term to the sorted list of documents that contain it. Queries intersect or merge lists, and a scoring function such as BM25 ranks the results. It is sharded and replicated, and eventually consistent with the source.
6. **How is a large file uploaded reliably?** Direct to object storage through signed URLs in parts, in parallel, with per-part retry and checksums, and a completion call that assembles the object and marks the metadata complete.
7. **Long polling, SSE or WebSocket?** Long polling for compatibility and low event rates, SSE for one-way streams, WebSocket for bidirectional low-latency exchange. A fleet needs a user-to-server mapping, heartbeats and resumable sequence numbers.
8. **What does an API gateway do and what are its risks?** It centralises TLS, authentication, rate limiting, routing and logging. The risks are an additional hop, a critical dependency that needs high availability, and accumulation of business logic.
9. **How is proximity search implemented?** Index locations by geohash, quadtree or S2 cell; query the target cell and its neighbours; filter by exact distance. Choose the cell size relative to the radius and the update frequency.
10. **How does service discovery handle failures?** Heartbeats with TTL remove dead instances; clients use timeouts, retries on another instance and outlier ejection; services deregister and drain at shutdown.

KEY POINTS

- Token bucket and sliding window counter are the usual limiter defaults; fixed windows admit double bursts, and limiters across several instances need shared atomic state.
- A Bloom filter has no false negatives; size it at about 9.6 bits per element for 1% false positives and rebuild it before it saturates.
- A search index is a derived, sharded, near-real-time copy; the system of record lies elsewhere.
- Large files go to object storage directly in parallel parts with signed URLs; metadata becomes complete only after assembly.
- Choose long polling, SSE or WebSocket by direction and rate; plan reconnect with jitter and sequence-based resume.
- Geohash and S2 turn two-dimensional proximity into prefix or range queries; always search the neighbouring cells.
- Registries rely on heartbeats and TTLs; stale entries are the main failure, so combine graceful deregistration with client-side retries.

PART 2

High-Level Design Case Studies

Each chapter takes one system from requirements through estimation, API, data model, architecture, deep dive and failure analysis.

In this part

- **2.1** Design: URL Shortener
- **2.2** Design: Pastebin
- **2.3** Design: Distributed Rate Limiter
- **2.4** Design: Unique ID Generator
- **2.5** Design: Distributed Key-Value Store
- **2.6** Design: Distributed Cache
- **2.7** Design: Notification System
- **2.8** Design: Web Crawler
- **2.9** Design: Distributed Task Scheduler and Job Queue
- **2.10** Design: News Feed
- **2.11** Design: Chat System
- **2.12** Design: Video Streaming Platform
- **2.13** Design: Ride Hailing and Proximity Service
- **2.14** Design: Ticket Booking System
- **2.15** Design: E-Commerce Checkout and Inventory
- **2.16** Design: Search Autocomplete
- **2.17** Design: File Storage and Sync (Dropbox, Google Drive)
- **2.18** Design: Payment System and Wallet

2.1 Design: URL Shortener

A URL shortener maps a long URL to a short code and redirects requests for the short URL to the long one. The problem looks trivial, and its value as a first case study lies in the fact that every decision is driven by numbers: the code length follows from the number of URLs, the cache size from the read skew, and the id generation scheme from the requirement that several servers create codes without coordinating on every request. The design below follows the seven-step framework of the first part of the book and uses `assumed` figures that are labelled as such; none of them is a measurement of a real service.

2.1.1 Requirements and scope

Functional requirements

- Create a short URL for a long URL (`POST`), optionally with a custom alias and an expiry time.
- Redirect a request for a short URL to the original URL (`GET`), with low latency.
- Links expire at an optional deadline and can be deleted by their owner.
- Basic click analytics (count per link per day, coarse country) that must not slow down redirects.

Non-functional requirements

- Availability of the redirect path above 99.99% (assumed target): a dead link is visible to every end user of every page that embeds it.
- Redirect latency p99 below 100 ms inside the region (assumed), dominated by one cache lookup.
- Durability: an acknowledged short URL must never disappear.
- Codes must be short (at most 7 characters) and not trivially enumerable.
- Reads dominate writes by about 100 to 1 (assumed).

Out of scope are user accounts and billing, link-in-bio pages, QR code rendering, and malware scanning internals (only the hook is shown). **Assumptions**: one primary region with a second region for disaster recovery; the long URL is at most 2048 characters; links are immutable once created except for deletion.

sreram

2.1.2 Capacity estimation

All inputs are assumptions. The arithmetic is shown so that a different input can be substituted.

Quantity	Arithmetic	Result
New URLs per month	assumed	100 million
Write QPS (average)	100,000,000 / (30 x 86,400 s)	about 39 per second
Read QPS (average)	39 x 100 (read:write ratio)	about 3,900 per second
Read QPS (peak)	3,900 x 3 (assumed peak factor)	about 11,700 per second
Records after 5 years	100,000,000 x 12 x 5	6.0 billion
Code length	62^6 = 5.7e10 > 6.0e9; 62^7 = 3.5e12	7 characters (0.17% of the space used)
Row size	code 7 + URL about 100 + metadata about 150 + index overhead	about 500 bytes
Storage after 5 years	6.0e9 x 500 bytes	3.0 TB (9 TB with 3 replicas)
Redirect bandwidth	3,900 x about 500 bytes of response	about 2 MB/s (negligible)
Cache size	20% of daily reads: 3,900 x 86,400 x 0.2 = 67 million entries x 300 bytes	about 20 GB

Capacity estimate for the URL shortener (all inputs assumed).

Two conclusions follow. First, the data volume of 3 TB is small by distributed-database standards: sharding is chosen for write headroom and failure isolation more than for capacity. Second, 11,700 redirects per second is a modest load for a cache-backed service; the design effort concentrates on correctness, hot keys and the failure of the cache, not on raw throughput.

2.1.3 API design

REST API

```
POST /v1/urls
  body:   { "longUrl": "https://example.com/a/very/long/path?x=1",
            "alias": "spring-sale",        (optional, 4-20 chars [A-Za-z0-9_-])
            "expiresAt": "2027-01-01T00:00:00Z" }   (optional)
  header: Idempotency-Key: 7c2e...          (client generated, optional)
  201 Created  { "code": "aZ3k9Qx", "shortUrl": "https://sho.rt/aZ3k9Qx" }
  409 Conflict      alias already taken
  422 Unprocessable URL invalid or blocked
  429 Too Many Requests   per-user creation limit exceeded

GET /{code}
  302 Found   Location: https://example.com/a/very/long/path?x=1
  404 Not Found   unknown or deleted code
  410 Gone        code existed and has expired

DELETE /v1/urls/{code}                  (owner only)   204 No Content
GET    /v1/urls/{code}/stats?from=&to=  200  { "clicks": [ {"day": "...", "n": 12} ] }
GET    /v1/urls?ownerId=&pageToken=&limit=50   cursor-based pagination
```

The redirect uses 302 rather than 301 because a 301 may be cached by the browser indefinitely, which prevents click counting, expiry and deletion from taking effect. The trade-off is that every click reaches the service; this is acceptable at the estimated load. The `Idempotency-Key` header makes `POST` safe to retry: the service stores the key with the created code for 24 hours (assumed) and returns the same response for a repeated request. Pagination uses an opaque cursor (`pageToken`) instead of offsets, because offsets degrade on large partitions and shift when rows are inserted.

2.1.4 Data model

The access pattern is a point lookup by code on the read path and an insert on the write path. There are no joins and no range scans on the hot path, which suits a key-value layout. A sharded relational store or a wide-column store both fit; the choice made here is a sharded, replicated relational store as covered in the databases material, with `code` as the partition key, because it gives unique constraints for custom aliases and simple operations.

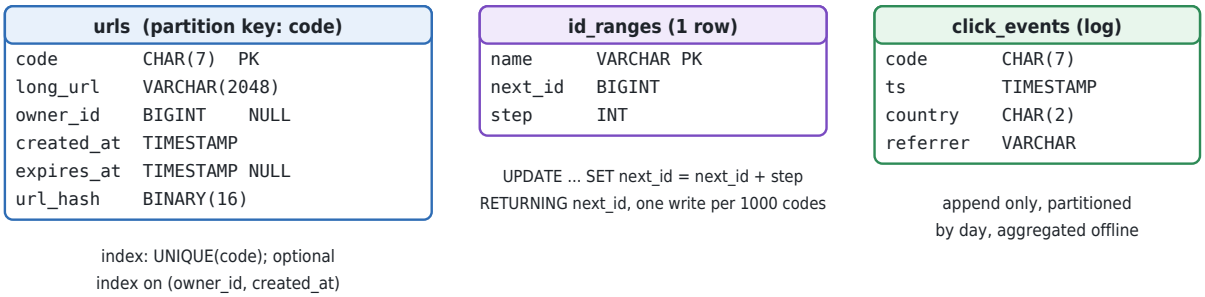


Figure 2.1. Three stores: the urls table partitioned by code, a single-row counter table used for range leases, and an append-only click log kept out of the redirect path.

schema.sql SQL

```sql
CREATE TABLE urls (
    code        CHAR(7)       PRIMARY KEY,      -- partition key: hash(code) selects the shard
    long_url    VARCHAR(2048) NOT NULL,
    owner_id    BIGINT,
    created_at  TIMESTAMP     NOT NULL DEFAULT now(),
    expires_at  TIMESTAMP,                      -- NULL means no expiry
    url_hash    BYTEA         NOT NULL          -- digest of the normalised long URL
);
CREATE INDEX urls_owner ON urls (owner_id, created_at DESC);   -- local to each shard

CREATE TABLE id_ranges (name TEXT PRIMARY KEY, next_id BIGINT NOT NULL, step INT NOT NULL);
-- one lease = one statement, executed once per 1000 created codes
UPDATE id_ranges SET next_id = next_id + step WHERE name = 'urls' RETURNING next_id - step;
```

The custom alias shares the `code` column and namespace; its length (4 to 20) differs from generated codes (7), and a reserved-word list prevents aliases such as `api` or `admin`. The owner index is local to each shard, so listing the links of one owner is a scatter-gather query over all shards, which is acceptable because it is a rare dashboard operation and not a redirect.

2.1.5 High-level architecture

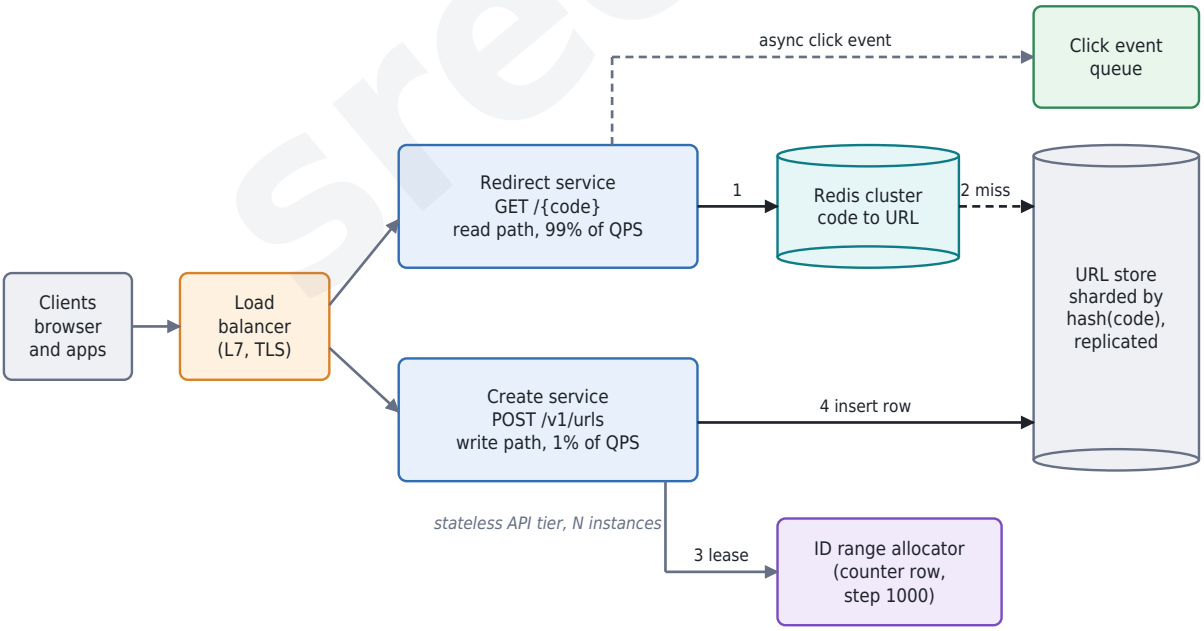


Figure 2.2. The read path (top) touches only the cache in the common case; the write path (bottom) leases id ranges rarely and inserts directly into the sharded store; click events leave the request path asynchronously.

Create flow. (1) The client sends `POST /v1/urls` through the load balancer. (2) The create service validates the URL (scheme `http` or `https`, length, blocklist) and normalises it. (3) If an alias was given, the service inserts it and maps a unique-key violation to 409. Otherwise it takes the next number from its in-memory range, permutes and encodes it as base62 (see the deep dive), and inserts the row. (4) The service writes the new entry into the cache and returns 201. The cache write is optional; it removes the cache miss of the first click.

Redirect flow. (1) The redirect service looks up `url:{code}` in the cache. (2) On a hit it returns 302 immediately. (3) On a miss it reads the row from the owning shard, checks `expires_at`, populates the cache with a time to live of at most the remaining lifetime, and returns 302 or 404/410. (4) A click event is published to a queue after the response has been written; failure to publish never fails the redirect.

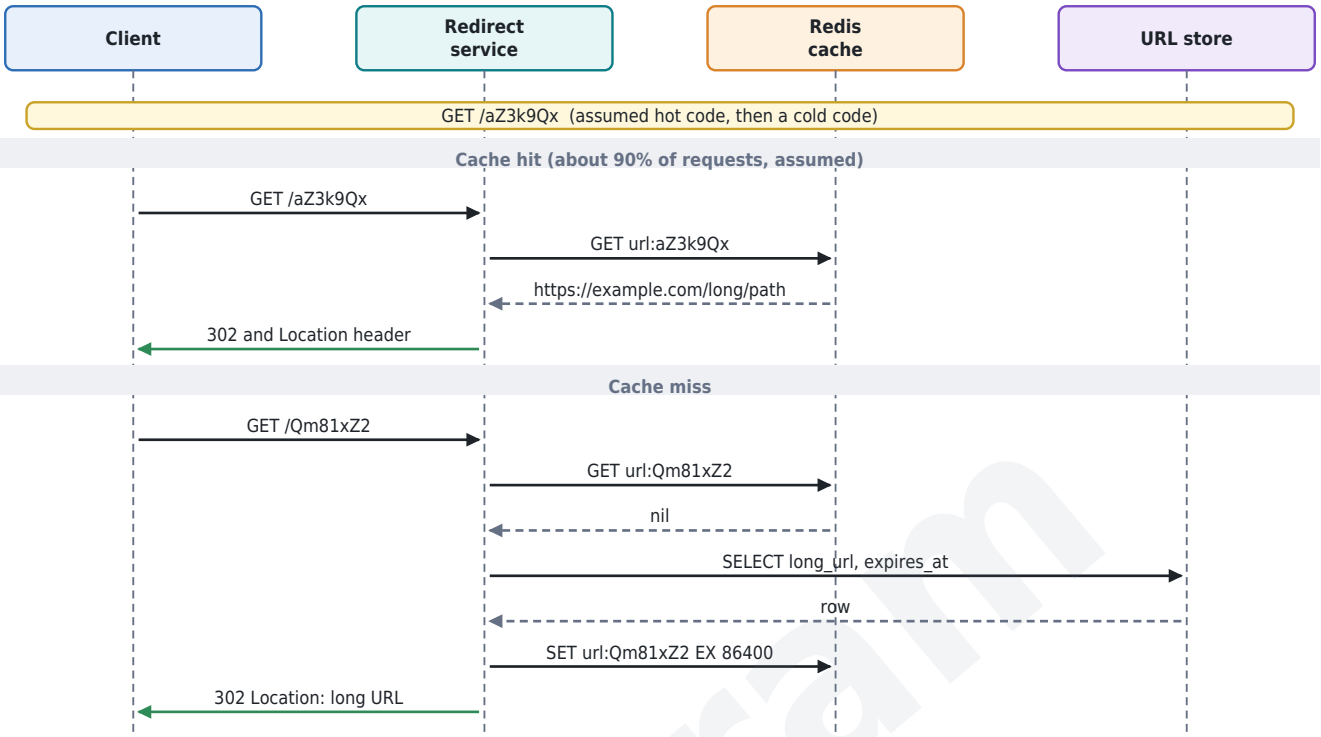


Figure 2.3. The cache hit path involves two network hops inside the data centre; the miss path adds one store read and one cache write.

2.1.6 Deep dive

Problem 1: generating unique short codes

The design question is how several create servers produce codes that never collide, without a network round trip per code and without predictable output.

Approach	How it works	Problems	Verdict
Hash and truncate	Take a digest of the long URL, encode, keep 7 characters	Birthday collisions need a check and retry; the same URL by two users gets the same code	Acceptable for dedup, weak as the main scheme
Random code and insert	Draw 7 random base62 characters; retry on unique-key violation	One failed insert per collision, rare at 0.17% fill; needs an insert round trip anyway	Simple and good at small fill
Global counter, base62	One atomic counter, encode the value	Counter is a bottleneck and a single point of failure; sequential codes are enumerable	Only with ranges
Range allocation	Each server leases a block of 1,000 ids and counts locally	Gaps after a crash; sequential within a server	Chosen, plus permutation
Snowflake-style id	Time, worker and sequence bits	64 bits encode to 11 base62 characters, longer than 7	Too long for this purpose

Code generation alternatives.

The chosen scheme leases ranges from a counter row and then applies a keyed permutation so that consecutive numbers do not produce consecutive codes. The lease cost is one `UPDATE` per 1,000 creates; at the estimated 39 creates per second the counter row receives about 0.04 writes per second, which makes a single replicated database row sufficient.

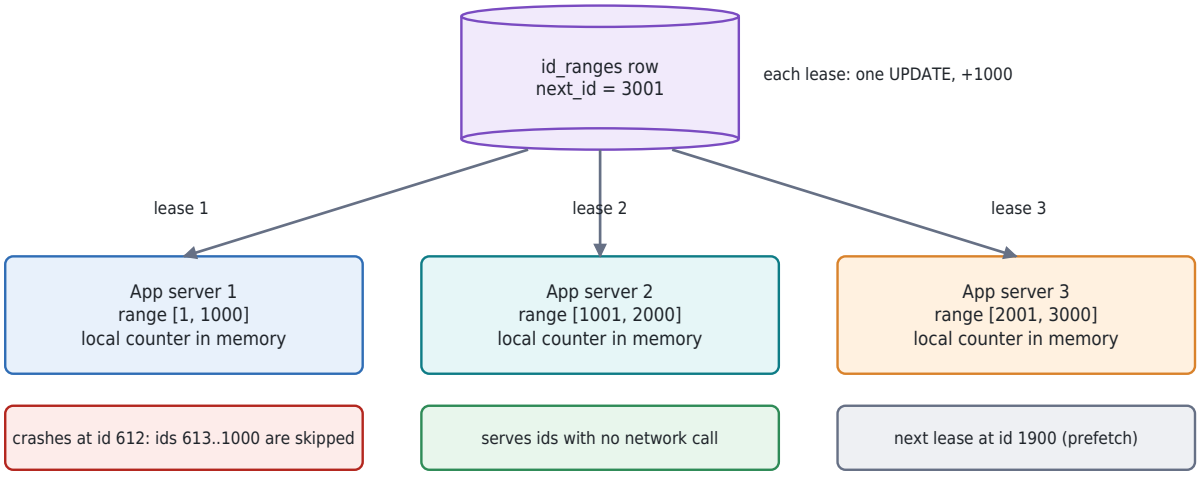


Figure 2.4. Each server serves ids from a private range and contacts the counter only when the range is nearly used; a crash wastes the unused part of a range but never produces a duplicate.

The following program implements base62 encoding, range leasing, a Feistel permutation over 40 bits, and a measurement of collisions for the hash-and-truncate alternative with random values.

ShortCode.java JAVA

```
import java.util.Arrays;
import java.util.Random;
import java.util.concurrent.atomic.AtomicLong;

public class ShortCode {
    static final String ALPHABET =
        "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";

    static String encode(long n) {
        if (n == 0) return "0";
        StringBuilder sb = new StringBuilder();
        while (n > 0) {
            sb.append(ALPHABET.charAt((int) (n % 62)));
            n /= 62;
        }
        return sb.reverse().toString();
    }

    static long decode(String s) {
        long n = 0;
        for (char c : s.toCharArray()) n = n * 62 + ALPHABET.indexOf(c);
        return n;
    }

    /** Feistel permutation over [0, 2^40) so that sequential ids give non-guessable codes. */
    static long permute(long x) {
        long l = x >>> 20, r = x & 0xFFFFF;
        for (int round = 0; round < 4; round++) {
            long f = ((r * 0x9E3779B1L + round * 0x85EBCA6BL) ^ (r >>> 7)) & 0xFFFFF;
            long nl = r, nr = l ^ f;
            l = nl;
            r = nr;
        }
        return (l << 20) | r;
    }

    /** Hands out id ranges the way a counter service would; each app server consumes a range. */
    static class RangeAllocator {
        private final AtomicLong next = new AtomicLong(1);
        long[] lease(int size) {
            long start = next.getAndAdd(size);
            return new long[] {start, start + size - 1};
        }
    }

    static int collisions(int bits, int n, long seed) {
        Random rnd = new Random(seed);
        long space = 1;
```

ShortCode.java (continued) JAVA

```java
        for (int i = 0; i < bits; i++) space *= 62;
        long[] v = new long[n];
        for (int i = 0; i < n; i++) v[i] = Math.floorMod(rnd.nextLong(), space);
        Arrays.sort(v);
        int c = 0;
        for (int i = 1; i < n; i++) if (v[i] == v[i - 1]) c++;
        return c;
    }

    public static void main(String[] args) {
        for (long n : new long[] {0, 61, 62, 3844, 1_000_000_000L, 3_521_614_606_207L}) {
            String s = encode(n);
            System.out.printf("%16d -> %-8s -> %d%n", n, s, decode(s));
        }
        System.out.println("62^6 = " + (long) Math.pow(62, 6)
            + ", 62^7 = " + (long) Math.pow(62, 7));
        RangeAllocator ra = new RangeAllocator();
        for (int i = 1; i <= 3; i++) {
            long[] r = ra.lease(1000);
            System.out.println("app server " + i + " leases [" + r[0] + ", " + r[1] + "]");
        }
        System.out.println("sequential ids 1000001..1000004 -> plain / permuted");
        for (long id = 1_000_001; id <= 1_000_004; id++) {
            System.out.println("  " + encode(id) + "  /  " + encode(permute(id)));
        }
        java.util.Set<Long> outs = new java.util.HashSet<>();
        for (long id = 0; id < 1_000_000; id++) outs.add(permute(id));
        System.out.println("1,000,000 sequential ids -> distinct permuted values: " + outs.size());
        System.out.println("hash-truncation simulation (random values, fixed seed):");
        System.out.printf("  5 chars (9.2e8 space), 100000 codes: %d collisions%n",
                          collisions(5, 100_000, 7));
        System.out.printf("  7 chars (3.5e12 space), 5000000 codes: %d collisions%n",
                          collisions(7, 5_000_000, 7));
        System.out.printf("  expected (n^2 / 2S): %.1f and %.1f%n",
            1e10 / (2 * 916_132_832.0), 25e12 / (2 * 3_521_614_606_208.0));
    }
}
```

Run ShortCode (JDK 21) TERMINAL

```
$ java ShortCode.java
               0 -> 0        -> 0
              61 -> z        -> 61
              62 -> 10       -> 62
            3844 -> 100      -> 3844
      1000000000 -> 15ftgG   -> 1000000000
   3521614606207 -> zzzzzzz  -> 3521614606207
62^6 = 56800235584, 62^7 = 3521614606208
app server 1 leases [1, 1000]
app server 2 leases [1001, 2000]
app server 3 leases [2001, 3000]
sequential ids 1000001..1000004 -> plain / permuted
  4C93  /  1MFTHZF
  4C94  /  8fq2LU1
  4C95  /  7gcqHuo
  4C96  /  7TCgmtu
1,000,000 sequential ids -> distinct permuted values: 1000000
hash-truncation simulation (random values, fixed seed):
  5 chars (9.2e8 space), 100000 codes: 6 collisions
  7 chars (3.5e12 space), 5000000 codes: 5 collisions
  expected (n^2 / 2S): 5.5 and 3.5
```

What to notice. The largest 7-character code `zzzzzzz` decodes to 62^7 - 1 = 3,521,614,606,207, so the space is exactly as large as computed in the estimate. The permuted codes of consecutive ids share no visible pattern and remain unique for 1,000,000 consecutive inputs, because a Feistel network is a bijection whatever the round function is. The permutation is obfuscation, not encryption: it prevents casual enumeration but does not resist an attacker who learns the round constants, and a real deployment keeps the key secret. The collision simulation matches the birthday estimate n squared over 2S (6 observed against 5.5 expected for 5 characters, 5 observed against 3.5 expected for 7 characters, each a single run with a fixed seed), which is why hash-and-truncate always needs a collision check.

Problem 2: serving redirects fast and cheaply

With 3,900 reads per second on average and a skewed popularity distribution, a cache absorbs most of the load. Assuming a hit ratio of 90% (assumed, to be measured), the store sees about 390 reads per second on average and about 1,170 at peak, which a replicated store handles easily; the cache is nevertheless required for latency. The cache key is the code, the value is the long URL plus the expiry, and the time to live is `min(24 h, expires_at - now)`, so an expired link can never be served from the cache longer than its remaining lifetime.

Problem 3: expiry and deletion

Expiry is enforced lazily at read time (compare `expires_at` with the clock) and reclaimed eagerly by a background sweeper that deletes expired rows in small batches by partition. Deleting a link must also delete its cache entry; if that delete is lost, the entry still expires by its time to live. Codes of deleted or expired links are not reused for a retention period (assumed 1 year), because a reused code would silently redirect old embedded links to a new destination.

2.1.7 Bottlenecks, failures and real-world problems

Viral link creates a hot key

Scenario. A link is shared by a popular account and receives 150,000 requests per second, about 13 times the estimated peak load of the whole service. **Symptoms.** One cache node shows saturated CPU and rising latency while the other nodes are idle; redirect p99 rises for every code that hashes to the same node.

Diagnosis. Per-key request counters or the cache's hot-key statistics show a single key dominating; the node's network card or single-threaded command loop is the limit. **Root cause.** Consistent hashing or slot assignment places each key on one node, so the capacity of a single key is the capacity of a single node, regardless of cluster size.

Fix. Add a small in-process cache (a few thousand entries, 1 to 5 seconds time to live) in each redirect instance so that at most one request per instance per second reaches the cache for the hot code; allow the CDN to cache the 302 for a few seconds for links flagged as hot (`Cache-Control: public, max-age=5`). **Prevention.** Keep the in-process layer enabled by default, track per-key rates, and load-test with a Zipf-skewed key distribution instead of a uniform one.

Cold cache after a cache cluster failure

Scenario. The cache cluster restarts after a failed deployment and every entry is gone. **Symptoms.** The store receives 3,900 reads per second instead of 390; its latency rises, the connection pool saturates, and redirects time out, which triggers client retries and more load.

Diagnosis. Cache hit ratio drops to zero in the dashboard at the same time as store read latency and queue depth rise. **Root cause.** The store was sized for the miss rate, not for the full read rate, so the system depended on the cache for availability and not only for latency.

Fix. Coalesce concurrent misses for the same code into one store read (single flight), cap concurrent store reads per instance and return 503 with `Retry-After` above the cap, and warm the cache from the most recent top-N list before putting a restarted cache back in rotation. **Prevention.** Size the store for the cold-start read rate or provide admission control that protects it, and rehearse a cache loss in a game day.

Counter row unavailable or range exhausted

Scenario. The database holding `id_ranges` fails over for 40 seconds while a server consumes its last ids. **Symptoms.** `POST /v1/urls` fails with 5xx on that server only; other servers are unaffected until their ranges also run out.

Diagnosis. Lease errors in the logs; the per-server remaining-range gauge reaches zero. **Root cause.** The server leased lazily, so a short outage of the allocator became an outage of creation.

Fix. Prefetch the next range when 10% of the current range remains, and keep a lease large enough to cover several minutes of peak traffic (1,000 ids last 25 seconds at 39 per second, so the step is raised to 100,000 for production, assumed). **Prevention.** Alert on remaining range per server, accept gaps in the code space, and never let the allocator be on the redirect path.

Abuse: phishing and spam links

Scenario. Attackers create thousands of short links that point to phishing pages. **Symptoms.** Spike in creation from few accounts; user reports; the domain appears on a blocklist of a browser vendor.

Diagnosis. Group creations by owner, IP range and destination domain; compare destinations against a reputation feed. **Root cause.** A shortener hides the destination, which makes it attractive for abuse, and creation had no per-user limit.

Fix. Apply creation rate limits per user and per IP (see the rate limiter design), check the destination against a reputation service before insert, allow takedown by setting a `blocked` flag that the redirect returns as an interstitial page. **Prevention.** Require sign-in above a creation threshold, and review the highest-volume destinations periodically.

2.1.8 Trade-offs and extensions

- **Multi-region.** Replicate the URL store asynchronously to a second region; reads are served locally, writes go to the primary region. A lag of a few seconds means a link created in one region may return 404 in the other for that time; mitigate by writing the new entry into the cache of both regions at creation.
- **Cache versus CDN.** A CDN can serve 302 responses at the edge and cut latency further, at the price of delayed deletion and lost click counts for cached responses; use it only for hot links.
- **Custom domains.** Add a `domain` column to the key (`domain + code`), and look up by host header.
- **Analytics accuracy.** The queue-based pipeline gives at-least-once delivery, so counts are approximate unless events carry unique ids and the aggregation deduplicates them.

2.1.9 Questions to expect

Why base62 and not base64?

Base64 includes `+` and `/`, which need escaping in URLs, and the URL-safe variant adds `-` and `_`, which look ambiguous in some fonts and are valid in aliases. Base62 uses only letters and digits, and its density (5.95 bits per character) is close to base64 (6 bits).

How many characters are required for 6 billion URLs?

62^6 = 5.7e10 already exceeds 6.0e9, so 6 characters suffice with 10.6% of the space used. Seven characters (3.5e12) leave a margin and make random generation collide rarely, and so 7 is chosen.

301 or 302?

302, because every click must pass through the service for analytics, expiry and deletion. A 301 reduces load but the browser may cache it permanently and later changes are invisible. 301 suits permanent, non-tracked redirects.

How are collisions handled with random codes?

The insert uses the primary key as the arbiter. A unique-key violation triggers a retry with a new random code, up to a small limit. At 0.17% fill the expected number of retries per create is 0.0017.

How does the design scale to ten times the traffic?

Redirects scale horizontally with more stateless instances and cache nodes; the store gains read replicas first and shards second. The allocator is not on the hot path, and its lease step increases from 1,000 to larger values.

What happens when two users shorten the same URL?

Each receives a different code by default, which keeps analytics and ownership separate. Deduplication by `url_hash` is an option for anonymous links, at the price of sharing deletion semantics.

How is the availability of the redirect path protected?

Stateless instances in several zones, a replicated cache with a local in-process layer, a replicated store with read replicas, and admission control that sheds load before the store collapses. Creation can fail without affecting redirects.

How would custom aliases change the design?

They share the namespace, so uniqueness is enforced by the same primary key; reserved words are blocked, and a creation returns 409 on conflict. Aliases are not generated, so they bypass the range allocator.

sreeram

KEY POINTS

- The numbers drive the design: 6.0 billion codes need 6 characters, 7 give a safe margin, and 3 TB of data fits a few shards.
- Range allocation removes the counter from the hot path; a crash causes gaps, never duplicates.
- A keyed permutation hides the sequence; it is obfuscation and not a security boundary.
- Use 302 when clicks, expiry and deletion matter; protect the store from a cold cache and the cache from a hot key.
- Keep analytics asynchronous so that a slow pipeline never slows a redirect.

2.2 Design: Pastebin

A pastebin stores text blobs and returns a URL for reading them. It reuses most of the URL shortener (a generated key, a read-heavy lookup, a cache) and adds three new concerns: payloads that are large and variable in size, expiry as a first-class feature that controls storage cost, and abuse by users who upload illegal or malicious content. The chapter therefore concentrates on where the bytes live and how they leave.

2.2.1 Requirements and scope

Functional. Create a paste (text up to 512 KB, optional title, language, expiry of 1 day, 7 days, 30 days or never, visibility public or unlisted); read a paste by its key; delete by the owner; view raw text. **Non-functional.** Read latency p99 below 150 ms (assumed); durability of unexpired pastes; storage cost bounded by expiry; abuse controls. **Out of scope**: search inside pastes, collaborative editing, syntax highlighting implementation (done in the browser), accounts. **Assumptions**: 1 million new pastes per day, a read to write ratio of 5 to 1 (assumed, pastes are read less than short links), and an expiry mix of 30% one day, 30% seven days, 20% thirty days and 20% never.

2.2.2 Capacity estimation

The expiry mix and the size distribution determine storage, and the program below computes the steady state with a log-normal size distribution (median 3 KB, assumed). The numbers are produced by a simulation of assumed inputs, not by measurement.

paste_sim.py — PYTHON

```python
import random

random.seed(5)
NEW_PER_DAY = 1_000_000                    # assumed
YEARS = 5
# assumed expiry mix: (share, lifetime in days; None = never)
MIX = [(0.30, 1), (0.30, 7), (0.20, 30), (0.20, None)]
# assumed paste size: log-normal, median 3 KB, capped at 512 KB
def size():
    return min(int(random.lognormvariate(8.0, 1.2)), 512 * 1024)

sample = [size() for _ in range(200_000)]
mean = sum(sample) / len(sample)
sample.sort()
print("size sample: mean %.1f KB, median %.1f KB, p99 %.1f KB"
      % (mean / 1024, sample[len(sample) // 2] / 1024, sample[int(len(sample) * .99)] / 1024))
inline = sum(1 for s in sample if s <= 4096) / len(sample)
print("pastes of at most 4 KB (could live inline in the row store): %.1f%%" % (inline * 100))

days = YEARS * 365
no_expiry = NEW_PER_DAY * days * mean
live = 0.0
for share, life in MIX:
    n = NEW_PER_DAY * share * (days if life is None else life)
    live += n * mean
print("stored after %d years, nothing ever deleted : %.2f TB (%.1f billion pastes)"
      % (YEARS, no_expiry / 1e12, NEW_PER_DAY * days / 1e9))
print("stored after %d years with the expiry mix    : %.2f TB" % (YEARS, live / 1e12))
for share, life in MIX:
    n = NEW_PER_DAY * share * (days if life is None else life)
    print("  %3d%% expire after %-5s -> %.2f TB live"
          % (share * 100, life or "never", n * mean / 1e12))
```

Run paste_sim.py (Python 3) TERMINAL

```
$ python3 paste_sim.py
size sample: mean 6.0 KB, median 2.9 KB, p99 47.6 KB
pastes of at most 4 KB (could live inline in the row store): 60.5%
stored after 5 years, nothing ever deleted : 11.13 TB (1.8 billion pastes)
stored after 5 years with the expiry mix    : 2.28 TB
   30% expire after 1     -> 0.00 TB live
   30% expire after 7     -> 0.01 TB live
   20% expire after 30    -> 0.04 TB live
   20% expire after never -> 2.23 TB live
```

Quantity	Arithmetic	Result
Write QPS	1,000,000 / 86,400	about 12 per second (peak about 36 with factor 3)
Read QPS	12 x 5	about 58 per second (peak about 175)
Average paste size	simulation mean	6.0 KB
Storage without expiry, 5 years	1.0e6 x 1,825 x 6.0 KB	11.1 TB
Storage with the expiry mix	simulation	2.3 TB, 98% of it the never-expire 20%
Key length	1.8e9 pastes over 5 years, 62^7 = 3.5e12	7 characters
Read bandwidth	58 x 6.0 KB	about 350 KB/s

Capacity estimate for the pastebin (inputs assumed, sizes from the simulation).

The central finding is that 20% of pastes without an expiry account for 98% of the stored bytes after five years (2.23 of 2.28 TB). The expiry policy is therefore the main cost control, and the option "never" deserves a size limit, an account requirement, or a different price.

2.2.3 API design

REST API

```
POST /v1/pastes
  body: { "content": "...", "title": "optional", "language": "java",
          "expiry": "1d" | "7d" | "30d" | "never", "visibility": "unlisted" }
  201 Created { "key": "k8Pq2mX", "url": "https://paste.example/k8Pq2mX",
                "expiresAt": "2026-10-20T10:00:00Z" }
  413 Payload Too Large (over 512 KB)    429 Too Many Requests
GET /v1/pastes/{key}          200 { "content", "title", "language", "createdAt", "expiresAt" }
GET /raw/{key}                200 text/plain
DELETE /v1/pastes/{key}       204   (requires the delete token returned at creation)
```

The creator receives a `deleteToken` in the response for anonymous pastes, which avoids accounts for the simple case. Keys use the same range allocator and permutation as the URL shortener.

2.2.4 Data model

Metadata and content are stored separately. Metadata is small (about 200 bytes) and queried by key; content is a blob that is written once and read many times. Pastes up to 4 KB, which are 60.5% of pastes in the simulation, may be stored inline in the metadata row to save an object store request; larger ones go to object storage as immutable objects named by key.

pastes.sql SQL

```sql
CREATE TABLE pastes (
    key         CHAR(7)      PRIMARY KEY,        -- partition key
    owner_id    BIGINT,
    title       VARCHAR(200),
    language    VARCHAR(20),
    size_bytes  INT          NOT NULL,
    inline_body TEXT,                            -- non-NULL when size_bytes <= 4096
    blob_ref    VARCHAR(100),                    -- object key when not inline
    content_sha CHAR(64)     NOT NULL,           -- for abuse blocklist and dedup of blobs
    visibility  SMALLINT     NOT NULL,
    created_at  TIMESTAMP    NOT NULL,
    expires_at  TIMESTAMP                        -- NULL for never
);
CREATE INDEX pastes_expiry ON pastes (expires_at) WHERE expires_at IS NOT NULL;
```

The partial index on `expires_at` lets the sweeper find due rows without scanning the table. Object storage keys include a short prefix hash (for example `ab/ab12.../key`) so that keys spread over partitions of the object store, which matters for request rate limits of some providers.

2.2.5 High-level architecture

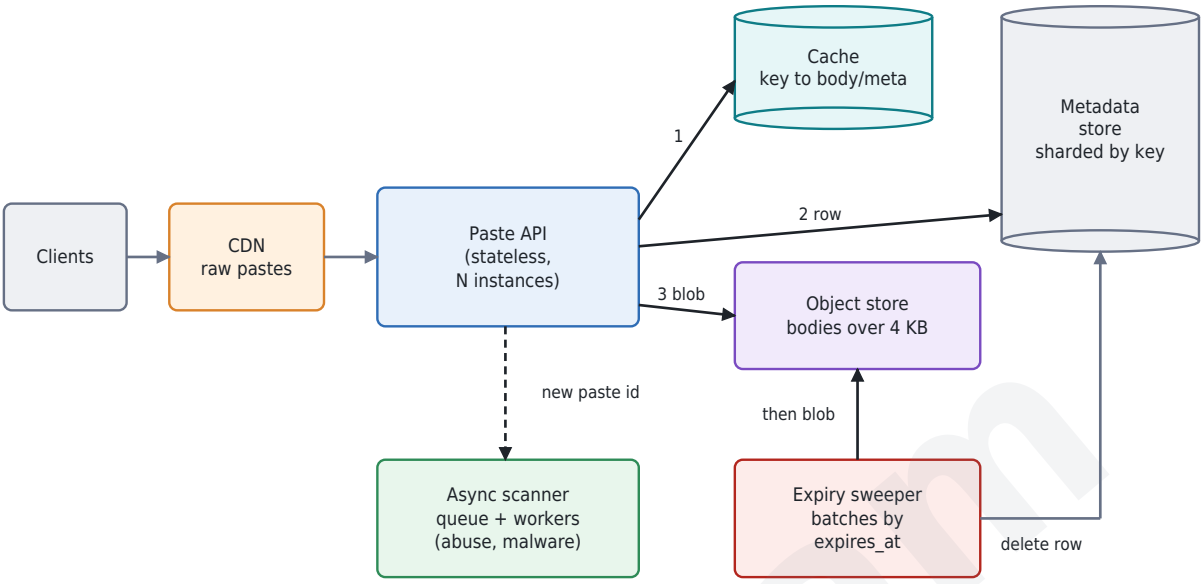


Figure 2.5. Metadata lives in a sharded store, large bodies in an object store, and a sweeper removes expired rows and objects in the background; a CDN serves hot raw pastes.

Create flow. The API validates the size and content type, computes the SHA-256 of the body, checks it against a blocklist of known-bad digests, writes the blob to object storage if it is larger than 4 KB, inserts the metadata row, writes a cache entry and returns the key. The blob is written before the row so that a crash leaves an orphan blob (collected by a periodic reconciliation job) rather than a row that points nowhere. **Read flow.** The service checks the cache, then the metadata row, verifies `expires_at`, and returns the inline body or streams the object. Raw pastes are cacheable at a CDN with a time to live no longer than the remaining lifetime.

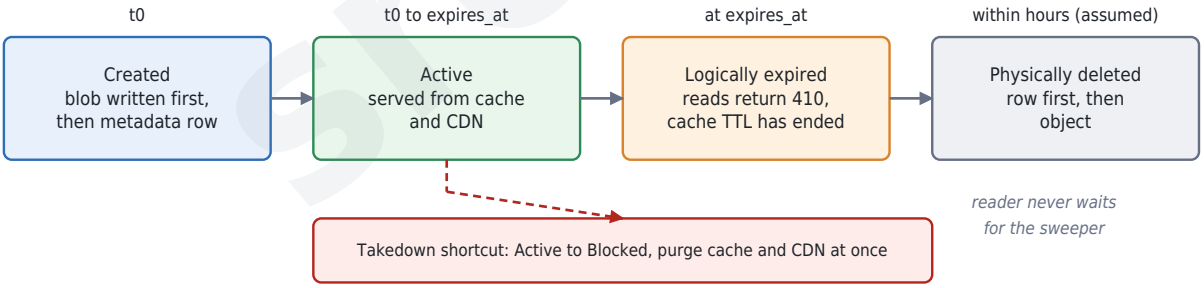


Figure 2.6. A paste passes through creation, active reads, logical expiry (immediately invisible) and physical deletion later by the sweeper, so that readers never depend on the sweeper's speed.

2.2.6 Deep dive

Problem 1: where the bytes live

Option	Advantages	Disadvantages	Verdict
All in the relational row	One read, transactional with metadata	Large rows bloat the store and the buffer cache; backups grow	Only up to a small size
Object store for all bodies	Cheap, durable, scales; CDN friendly	An extra request on every read; small objects are inefficient	Good for large bodies
Hybrid at a 4 KB threshold	Most reads need one query; large bodies stay out of the database	Two code paths	Chosen

Storage choices for paste bodies.

Problem 2: expiry at scale

With 1,000,000 pastes per day and 80% expiring within 30 days, about 0.8 million rows become due per day on average, roughly 9 per second. The sweeper deletes by `expires_at` in batches of a few thousand rows per shard, off-peak, and deletes the object after the row, with a retry list for failed object deletions. Readers check `expires_at` and treat an expired row as absent, so the lag of the sweeper has no visible effect. An alternative is to partition the table by creation day and drop whole partitions, which is cheaper but only fits expiry classes with similar lifetimes; a mix of 1, 7, 30 days and never suggests one partition set per class.

Problem 3: abuse

Public text hosting attracts malware, credential dumps and illegal content. Controls that fit the architecture: a per-IP and per-user creation rate limit, a digest blocklist checked on create, asynchronous scanning of new pastes with takedown (set `visibility` to blocked, purge cache and CDN), a report endpoint, and noindex headers so that unlisted pastes do not appear in search engines.

2.2.7 Bottlenecks, failures and real-world problems

Unbounded storage growth

Scenario. After a year the storage bill is three times the plan. **Symptoms.** Object store size grows linearly; the sweeper reports few deletions.

Diagnosis. Group stored bytes by `expiry` class; the never-expire class dominates, and a few clients upload 500 KB pastes in a loop. **Root cause.** "Never" was available to anonymous users without a quota.

Fix. Limit the never class to signed-in users with a quota, apply a default expiry of 30 days, and cap anonymous uploads per IP per day. **Prevention.** Track bytes per expiry class as a metric with an alert on growth.

Orphan blobs and dangling rows

Scenario. A crash occurs between the blob upload and the metadata insert, or between the row delete and the object delete. **Symptoms.** Object store contains objects that no row references; storage and the metadata count drift apart.

Diagnosis. A reconciliation job lists objects older than one hour and checks each key in the metadata store. **Root cause.** Two stores cannot be updated in one transaction.

Fix. Order operations so that failures leave garbage, never dangling references (write blob, then row; delete row, then blob), and run the reconciliation job daily. **Prevention.** Make object names deterministic from the key so the job can compute the owner of every object.

A popular paste overloads the origin

Scenario. A paste linked from a news page receives 5,000 reads per second. **Symptoms.** The object store request rate limit is reached and the service returns 503 for that key.

Diagnosis. Per-key request counters; object store throttling errors. **Root cause.** The raw endpoint set `Cache-Control: no-store`.

Fix. Serve immutable pastes with `Cache-Control: public, max-age` at the CDN; since a paste never changes after creation, caching is always correct until expiry or takedown. **Prevention.** Mark immutable resources as cacheable and use a purge call for takedown.

2.2.8 Trade-offs and extensions

- **Deduplication of bodies.** Storing blobs by content digest saves space for repeated pastes but complicates deletion (reference counts) and leaks the existence of identical content; usually not worth it below a large scale.
- **Private pastes and encryption.** Client-side encryption with the key in the URL fragment lets the server store only ciphertext, at the price of no server-side scanning.
- **Versioning.** Immutable pastes make forks trivial: a fork is a new paste with a `parent_key`.

sheram

2.2.9 Questions to expect

Why store small pastes inline?

It saves one object store request on the majority of reads (60.5% of pastes are at most 4 KB in the simulation) and keeps the metadata and body consistent in one row. The threshold is a tunable that balances row size against request count.

How do readers avoid seeing expired pastes before the sweeper runs?

Every read compares `expires_at` with the current time and returns 410, and the cache time to live never exceeds the remaining lifetime. Physical deletion is a space optimisation, not a correctness mechanism.

How would search be added?

Index the title and a text extraction asynchronously into a search store through a change event stream, and exclude unlisted and blocked pastes. The write path does not wait for indexing.

How is read-after-write guaranteed for the creator?

The create response is followed by a read, which must find the row. Reads go to the primary, or to a replica only after the cache has been written at creation; the cache write at creation covers the replication lag window.

KEY POINTS

- The expiry mix controls cost: in the estimate 20% never-expiring pastes hold 98% of the bytes.
- Split metadata from bodies; inline small bodies and store large ones in an object store.
- Order writes so that a crash leaves garbage that can be collected, not rows that point nowhere.
- Immutable content is safely cacheable at the CDN until expiry or takedown.

2.3 Design: Distributed Rate Limiter

A rate limiter bounds how many requests a client may make in a period. It protects services from overload and abuse, enforces fairness between tenants, and implements commercial plans. The algorithms were compared in the part on common infrastructure components; this chapter designs the service around them: where the check runs, where the counters live, how rules reach the checkers, and what happens when the counter store fails. The central difficulty is that a limit of 100 requests per minute for a user must hold even though the user's requests arrive at many gateway instances.

2.3.1 Requirements and scope

Functional requirements

- Decide for each request whether it is allowed, using rules that match on subject (user, API key, IP address, tenant), endpoint and method.
- Support several algorithms per rule (token bucket for burst-tolerant APIs, sliding window counter for strict windows).
- Return the decision with remaining quota and retry time so that clients can back off: `429 Too Many Requests`, `Retry-After`, and the `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` headers.
- Rules are changed at runtime without a deployment.

Non-functional requirements

- Added latency below 5 ms at p99 (assumed); the check is on the critical path of every request.
- High availability: the limiter must never become the reason an API is down.
- Accuracy: an over-admission of a few percent is acceptable; under-admission (blocking legitimate traffic) is worse.
- Memory efficiency per key, because the number of keys is large.

Out of scope are the billing integration, DDoS mitigation at the network layer (handled by the edge), and client-side throttling. **Assumptions**: the limiter runs in the API gateway tier; the counter store is an in-memory key-value store with atomic server-side scripts (a Redis cluster, as covered in the Redis material); keys are identified by an authenticated subject or by IP when anonymous.

2.3.2 Capacity estimation

Quantity	Arithmetic	Result
Peak gateway traffic	assumed	200,000 requests per second
Checks per request	2 rules (per user and per endpoint, assumed)	2
Peak check rate	200,000 x 2	400,000 checks per second
Capacity per counter shard	assumed, one Lua evaluation per check; must be benchmarked	80,000 checks per second
Shards required	400,000 / 80,000 = 5; add headroom to keep utilisation near 60%	8 shards (50,000 per shard, 63%)
Active keys	10 million subjects x 2 rules	20 million counters
Memory per token bucket	key about 40 bytes + 2 fields + overhead (assumed)	about 100 bytes, 2 GB in total
Memory per sliding log key	limit 1,000 per hour x 8 bytes per timestamp	8 KB, 160 GB in total
Network	400,000 x about 100 bytes each way	about 40 MB/s each way

Capacity estimate for the rate limiter (all inputs assumed).

The comparison of 2 GB against 160 GB is the reason sliding window logs are reserved for small limits, and the shard count follows from operation rate and not from memory.

2.3.3 API design

The limiter is consumed in two ways: embedded as a filter in the gateway, which calls the counter store directly, or as a standalone decision service that other services call. The decision API is small.

Decision API (gRPC style, shown as REST)

```
POST /v1/check
  { "ruleId": "orders-read", "subject": "user:42", "cost": 1 }
  200 { "allowed": true,  "remaining": 7, "limit": 10, "resetMs": 4200 }
  200 { "allowed": false, "remaining": 0, "limit": 10, "retryAfterMs": 600 }

PUT /v1/rules/orders-read          (admin, versioned)
GET /v1/rules?updatedAfter=...     (gateways poll or subscribe)
```

A denied decision still returns HTTP 200 from the decision API; the gateway turns it into a 429 response for the end client. A `cost` greater than 1 supports weighted endpoints (an export counts as 10).

2.3.4 Data model

rules.yaml YAML

```yaml
rules:
  - id: orders-read
    match: { path: "/v1/orders/**", method: GET }
    subject: user                  # user | api_key | ip | tenant
    algorithm: token_bucket
    capacity: 20                   # burst size
    refill_per_second: 10          # sustained rate
    on_store_failure: local        # open | closed | local
  - id: login
    match: { path: "/v1/login", method: POST }
    subject: ip
    algorithm: sliding_window_counter
    limit: 5
    window_seconds: 60
    on_store_failure: closed       # security rules fail closed
```

Counters live in the store as one hash per subject and rule: key `rl:{ruleId}:{subject}` with fields `tokens` and `ts` (last refill time in milliseconds), and an expiry set to twice the time needed to refill from empty, so that idle keys disappear on their own. The store is partitioned by key hash, and all fields of one key live on one shard, which is what makes a single-key atomic script possible. Rules are a small document set kept in a configuration store; gateways cache them and refresh every 30 seconds (assumed) or on a change notification.

sreram

2.3.5 High-level architecture

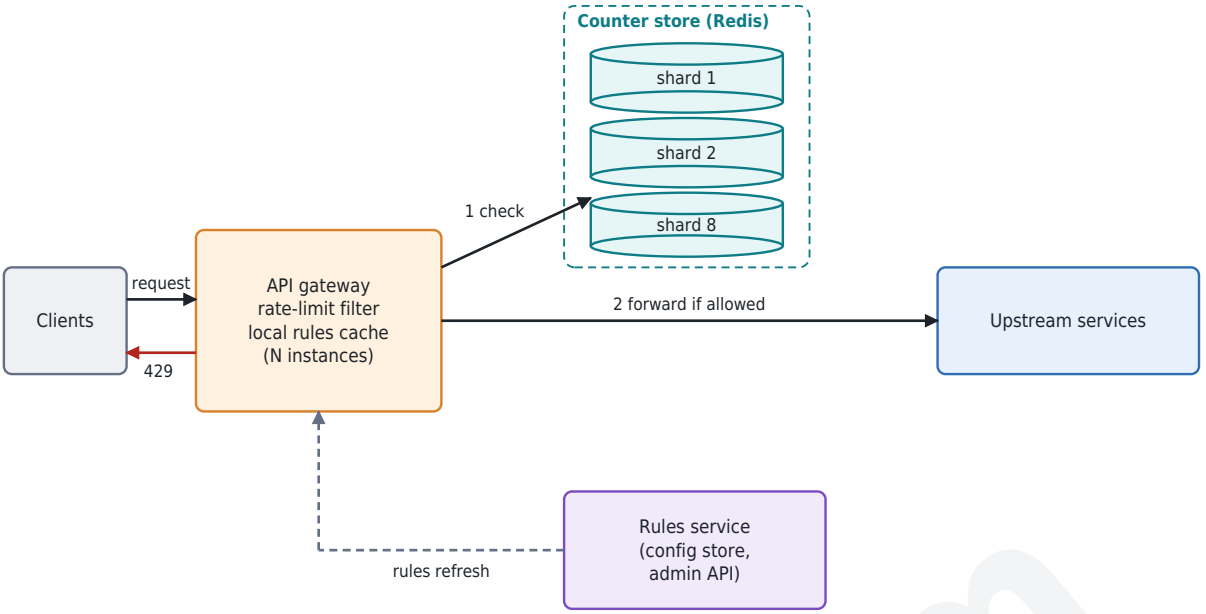


Figure 2.7. The gateway evaluates rules from its local cache and asks one counter shard per rule for a decision; rule changes flow from the rules service to the gateways, off the request path.

Request flow. (1) The gateway authenticates the request and derives the subject. (2) It selects the matching rules from its local rule cache. (3) For each rule it sends one script call to the shard that owns `rl:{ruleId}:{subject}`. (4) If all rules allow, the request is forwarded; otherwise the gateway returns 429 with `Retry-After`. Calls to different shards for the two rules are issued in parallel to keep the added latency at one round trip.

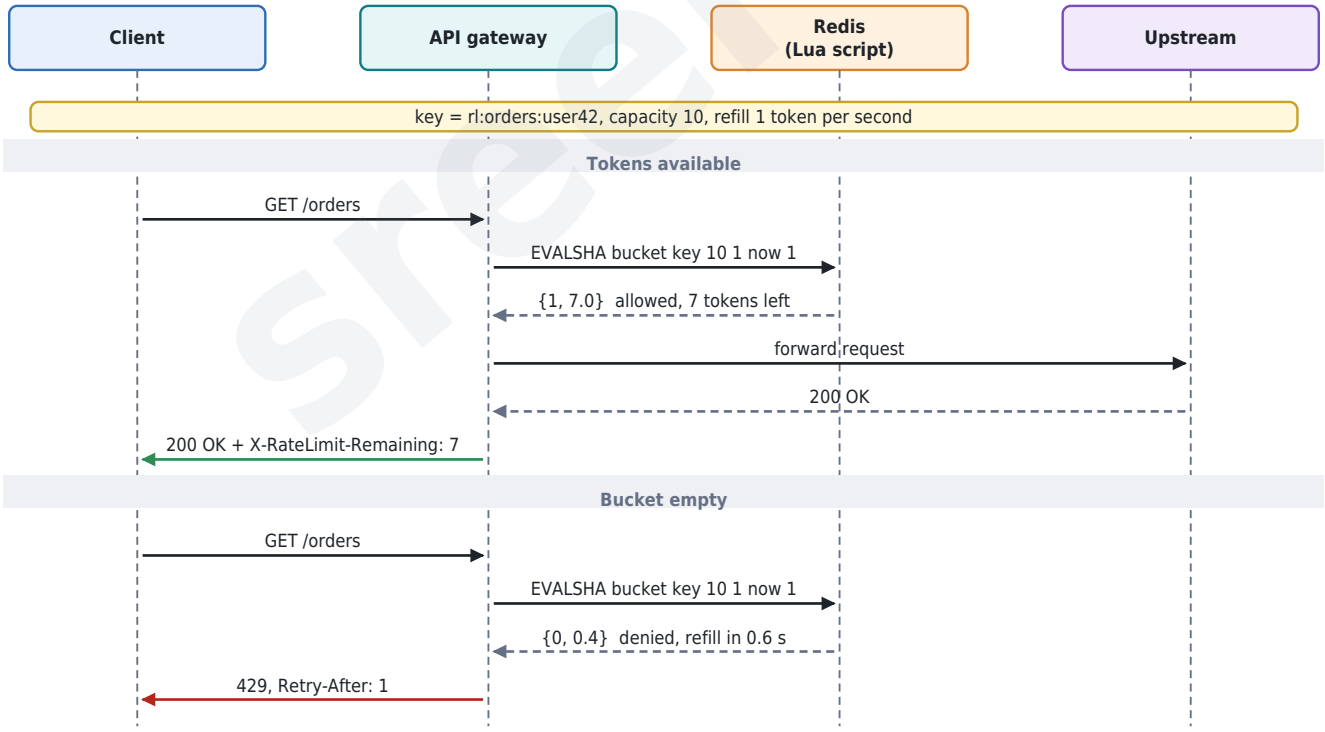


Figure 2.8. The decision is a single round trip that atomically refills the bucket, takes a token if one is available and returns the remaining count.

2.3.6 Deep dive

Problem 1: choosing the algorithm

Algorithm	Memory per key	Accuracy	Burst behaviour	Typical use
Fixed window counter	one integer	Poor at window edges (up to 2x the limit)	Allows double burst across a boundary	Coarse quotas where accuracy is unimportant
Sliding window log	one timestamp per request	Exact	None beyond the limit	Small limits (login attempts)
Sliding window counter	two integers	Approximate (assumes uniform spread in the previous window)	Smoothed	Large limits at low memory
Token bucket	two numbers	Exact for the model	Allows bursts up to capacity	General API limits
Leaky bucket	queue or counter	Exact output rate	Smooths bursts into a steady flow	Traffic shaping towards a fragile backend

Algorithm comparison.

The program below replays the same burst against four implementations: 10 requests in the last 500 ms of one 10-second window and 10 requests in the first 500 ms of the next, with a limit of 10 per 10 seconds.

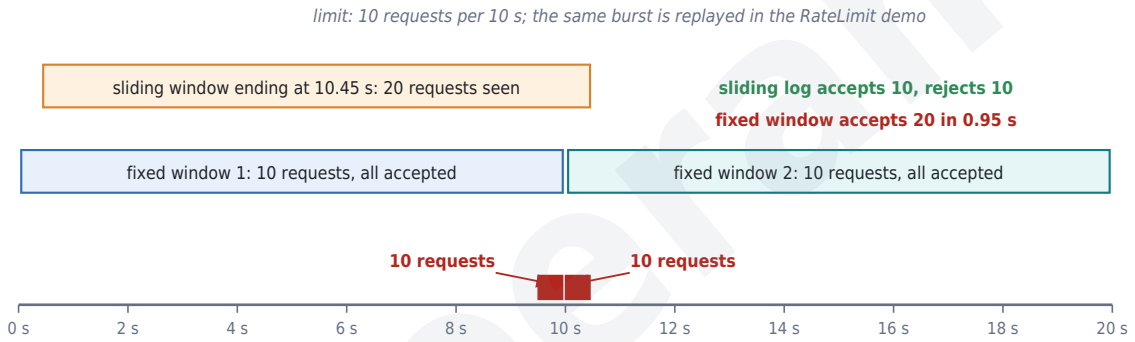


Figure 2.9. The fixed window accepts twenty requests within 0.95 seconds because the counter resets at the boundary; the sliding log, which looks back from the current time, accepts ten.

RateLimit.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Deque;

public class RateLimit {
    interface Limiter { boolean allow(long nowMs); }

    /** Fixed window: count per aligned window. */
    static class FixedWindow implements Limiter {
        final int limit; final long windowMs; long windowStart = -1; int count;
        FixedWindow(int limit, long windowMs) { this.limit = limit; this.windowMs = windowMs; }
        public boolean allow(long now) {
            long ws = now - now % windowMs;
            if (ws != windowStart) { windowStart = ws; count = 0; }
            return count < limit && ++count > 0;
        }
    }

    /** Sliding window log: exact, stores one timestamp per accepted request. */
    static class SlidingLog implements Limiter {
        final int limit; final long windowMs; final Deque<Long> log = new ArrayDeque<>();
        SlidingLog(int limit, long windowMs) { this.limit = limit; this.windowMs = windowMs; }
        public boolean allow(long now) {
            while (!log.isEmpty() && log.peekFirst() <= now - windowMs) log.pollFirst();
            if (log.size() >= limit) return false;
            log.addLast(now);
            return true;
        }
```

RateLimit.java (continued) JAVA

```
    }

    /** Sliding window counter: weighted sum of previous and current fixed windows. */
    static class SlidingCounter implements Limiter {
        final int limit; final long windowMs; long curStart = -1; int cur, prev;
        SlidingCounter(int limit, long windowMs) { this.limit = limit; this.windowMs = windowMs; }
        public boolean allow(long now) {
            long ws = now - now % windowMs;
            if (ws != curStart) {
                prev = (ws - curStart == windowMs) ? cur : 0;
                cur = 0; curStart = ws;
            }
            double weight = 1.0 - (now - ws) / (double) windowMs;
            double estimate = prev * weight + cur;
            if (estimate >= limit) return false;
            cur++;
            return true;
        }
    }

    /** Token bucket: capacity tokens, refilled at ratePerSec; tokens kept as a double. */
    static class TokenBucket implements Limiter {
        final double capacity, ratePerSec; double tokens; long last = -1;
        TokenBucket(double capacity, double ratePerSec) {
            this.capacity = capacity; this.ratePerSec = ratePerSec; this.tokens = capacity;
        }
        public boolean allow(long now) {
            if (last >= 0) tokens = Math.min(capacity, tokens + (now - last) / 1000.0 * ratePerSec);
            last = now;
            if (tokens < 1) return false;
            tokens -= 1;
            return true;
        }
    }

    static int run(Limiter l, long[] times) {
        int ok = 0;
        for (long t : times) if (l.allow(t)) ok++;
        return ok;
    }

    public static void main(String[] args) {
        // limit: 10 requests per 10 s. Client sends 10 requests in the last 500 ms of window 0
        // and 10 requests in the first 500 ms of window 1 (times in ms).
        long[] burst = new long[20];
        for (int i = 0; i < 10; i++) { burst[i] = 9_500 + i * 50; burst[10 + i] = 10_000 + i * 50; }
        System.out.println("burst of 20 requests across a window boundary, limit 10 per 10 s");
        System.out.println("  fixed window   accepted: " + run(new FixedWindow(10, 10_000), burst));
        System.out.println("  sliding log    accepted: " + run(new SlidingLog(10, 10_000), burst));
        System.out.println("  sliding counter accepted: "
            + run(new SlidingCounter(10, 10_000), burst));
        System.out.println("  token bucket   accepted: " + run(new TokenBucket(10, 1.0), burst));

        // steady 5 req/s for 60 s, then idle 20 s, then a burst of 30 requests within 100 ms
        TokenBucket tb = new TokenBucket(20, 5.0);
        int okSteady = 0;
        for (long t = 0; t < 60_000; t += 200) if (tb.allow(t)) okSteady++;
        int okBurst = 0;
        for (int i = 0; i < 30; i++) if (tb.allow(80_000 + i * 3)) okBurst++;
        System.out.println("token bucket (capacity 20, 5/s): steady accepted " + okSteady
            + " of 300, burst after idle accepted " + okBurst + " of 30");

        // centralised vs per-node split limit: global limit 90/s, 3 nodes, 80% of traffic on node 0
        int total = 90, nodes = 3;
        int[] offered = {80, 10, 10};
        int central = Math.min(total, offered[0] + offered[1] + offered[2]);
        int local = 0;
        for (int n = 0; n < nodes; n++) local += Math.min(offered[n], total / nodes);
        System.out.println("global limit 90/s, offered 80+10+10 = 100/s");
        System.out.println("  central counter accepts " + central + "/s");
        System.out.println("  3 local limiters of 30/s accept " + local
            + "/s (the limit allows 90/s)");
    }
}
```

Run RateLimit (JDK 21) TERMINAL

```
$ java RateLimit.java
burst of 20 requests across a window boundary, limit 10 per 10 s
  fixed window   accepted: 20
  sliding log    accepted: 10
  sliding counter accepted: 11
  token bucket   accepted: 10
token bucket (capacity 20, 5/s): steady accepted 300 of 300, burst after idle accepted 20 of 30
global limit 90/s, offered 80+10+10 = 100/s
  central counter accepts 90/s
  3 local limiters of 30/s accept 50/s (the limit allows 90/s)
```

What to notice. The fixed window admits twice the limit across the boundary. The sliding window counter admits 11 in this run because it estimates the previous window by weighting, an approximation that errs by a small amount in this adversarial pattern. The token bucket with capacity 10 and refill of 1 token per second admits exactly the initial 10 and then refills slowly; after 20 idle seconds it holds its capacity of 20 and admits 20 of an instant burst of 30. The last two lines show why a per-node split of a global limit is not equivalent to a central count: when the load balancer sends 80% of a user's traffic to one of three gateways, three local limiters of 30 per second accept only 50 per second although 90 are allowed.

Problem 2: atomicity of the check

A limiter that reads the counter, compares it with the limit and writes it back in separate network calls admits too many requests under concurrency, because two gateways read the same value before either writes. The atomic variant performs the whole decision in one server-side step. The simulation below reproduces the effect with threads: each check-then-set read is followed by a 20 microsecond pause, which stands in for a network round trip.

RaceDemo.java JAVA

```java
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.atomic.AtomicInteger;

public class RaceDemo {
    static final int LIMIT = 100, THREADS = 8, ATTEMPTS_PER_THREAD = 500;

    interface Counter { boolean tryAcquire(); }

    /** Read-modify-write in two steps: what GET followed by SET does against a shared store. */
    static class CheckThenSet implements Counter {
        private volatile int count = 0;
        public boolean tryAcquire() {
            int c = count;                  // read
            if (c >= LIMIT) return false;
            java.util.concurrent.locks.LockSupport.parkNanos(20_000);  // stands in for a round trip
            count = c + 1;                  // write based on a stale read
            return true;
        }
    }

    /** Single atomic step: what a Lua script or INCR with a check provides. */
    static class Atomic implements Counter {
        private final AtomicInteger count = new AtomicInteger();
        public boolean tryAcquire() {
            while (true) {
                int c = count.get();
                if (c >= LIMIT) return false;
                if (count.compareAndSet(c, c + 1)) return true;
            }
        }
    }

    static int run(Counter counter) throws InterruptedException {
        AtomicInteger admitted = new AtomicInteger();
        CountDownLatch start = new CountDownLatch(1);
        Thread[] ts = new Thread[THREADS];
        for (int i = 0; i < THREADS; i++) {
            ts[i] = new Thread(() -> {
                try { start.await(); } catch (InterruptedException e) { return; }
                for (int k = 0; k < ATTEMPTS_PER_THREAD; k++) {
                    if (counter.tryAcquire()) admitted.incrementAndGet();
                }
            });
            ts[i].start();
```

RaceDemo.java (continued) JAVA

```
        }
        start.countDown();
        for (Thread t : ts) t.join();
        return admitted.get();
    }

    public static void main(String[] args) throws Exception {
        System.out.println("limit = " + LIMIT + ", " + THREADS * ATTEMPTS_PER_THREAD + " attempts");
        for (int round = 1; round <= 3; round++) {
            int racy = run(new CheckThenSet());
            int safe = run(new Atomic());
            System.out.println("round " + round + ": check-then-set admitted " + racy
                + ", atomic admitted " + safe);
        }
    }
}
```

Run RaceDemo (JDK 21; timing dependent, typical output) TERMINAL

```
$ java RaceDemo.java
limit = 100, 4000 attempts
round 1: check-then-set admitted 797, atomic admitted 100
round 2: check-then-set admitted 799, atomic admitted 100
round 3: check-then-set admitted 762, atomic admitted 100
```

The check-then-set version admitted roughly eight times the limit, and the atomic version admitted exactly 100 in every round. The same applies to a store: use a single atomic command (`INCR` with an expiry set on the first increment) or a server-side script. The token bucket in Redis is implemented as a script (shown for illustration; it was not run, because no Redis server is available on the build machine):

token_bucket.lua (not run)

```
-- KEYS[1] = rl:{ruleId}:{subject}
-- ARGV: capacity, refill per second, now in ms (from the gateway), cost
local capacity = tonumber(ARGV[1])
local rate     = tonumber(ARGV[2])
local now      = tonumber(ARGV[3])
local cost     = tonumber(ARGV[4])
local d = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(d[1])
local ts     = tonumber(d[2])
if tokens == nil then tokens = capacity; ts = now end
tokens = math.min(capacity, tokens + math.max(0, now - ts) * rate / 1000)
local allowed = 0
if tokens >= cost then tokens = tokens - cost; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity / rate * 2000))
return { allowed, tostring(tokens) }
```

A script runs on the single command thread of one shard without interleaving, so the read-modify-write is atomic. The gateway supplies the time, so clocks of different gateways must be synchronised within a few milliseconds; an alternative is to read the store's own clock inside the script, at the cost of a dependency on the script replication mode of the server version in use.

Problem 3: distributed accuracy versus cost

Design	Accuracy	Cost and failure behaviour	Verdict
Central store, one call per request	Exact per key	One round trip per request; store failure needs a policy	Default
Local counters, limit divided by N	Poor when load is uneven (50 of 90 per second above)	No network; zero coordination	Only as degraded mode
Sticky routing by subject, local counters	Good while routing is stable	Breaks on instance loss and rebalancing	Special cases
Token leasing: gateways take batches of tokens centrally and spend locally	Over-admission bounded by (instances x batch size)	One call per batch; hot keys become cheap	For very hot subjects

Where the count lives.

The design uses the central store as the default and token leasing for subjects whose rate exceeds a threshold (for example 5,000 requests per second), where a single key would otherwise put the whole load of one subject on one shard.

2.3.7 Bottlenecks, failures and real-world problems

Counter store outage turns into a platform outage

Scenario. One counter shard fails over; for 30 seconds calls to it time out after the default 2 seconds. **Symptoms.** Gateway threads are blocked on the limiter, request latency rises to the timeout for every user whose key lives on that shard (one eighth of users), gateway connection pools fill and unrelated endpoints slow down.

Diagnosis. Latency histograms of the limiter call show a step at the timeout value; gateway thread dumps show threads waiting on the store client. **Root cause.** The limiter was treated as a mandatory dependency with a long timeout and no failure policy, which made a protection component into a source of cascading failure.

Fix. Set a 10 ms timeout without retries, open a circuit breaker after a few consecutive failures, and apply the per-rule policy: login and payment rules fail closed, general API rules fail to a local limiter (limit divided by instance count) so that the system degrades to approximate limiting instead of none. **Prevention.** Alert on breaker state and on the share of decisions made in degraded mode, and rehearse a store failure.

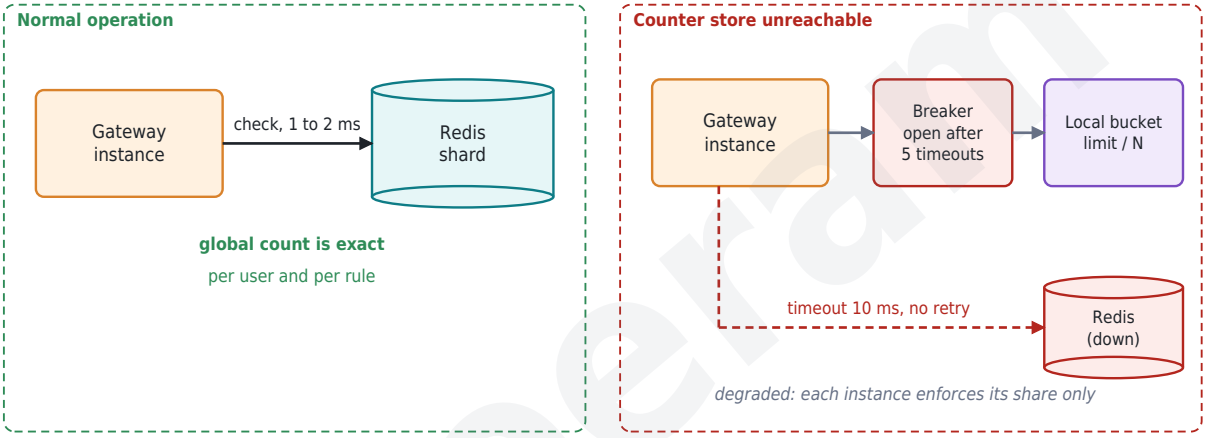


Figure 2.10. In normal operation every decision comes from the shared counter; when the store is unreachable the breaker opens quickly and each instance enforces its share locally.

Over-admission because of a non-atomic check

Scenario. A new implementation reads the counter with `GET`, compares it and writes with `SET`. **Symptoms.** Customers with bursts of parallel requests exceed their limit by a factor of several; the effect disappears in single-threaded tests.

Diagnosis. Replaying 8 parallel clients against a staging limiter reproduces over-admission (as in the RaceDemo run above). **Root cause.** Read-modify-write over the network is not atomic.

Fix. Use one atomic command or a server-side script for the complete decision. **Prevention.** Include a concurrent test in the build that asserts the admitted count never exceeds the limit.

One tenant saturates one shard

Scenario. A large customer sends 60,000 requests per second under one API key. **Symptoms.** The shard owning that key reaches 100% CPU on its command thread; other keys on the same shard see timeouts.

Diagnosis. Per-key operation counters (sampling of the command stream) show one key with 75% of the shard's operations. **Root cause.** All decisions for a key land on one shard, so one key is limited by the capacity of one shard, 80,000 checks per second in the estimate.

Fix. Switch the key to token leasing: each gateway instance obtains a batch of tokens (for example 5% of the per-second limit) and spends them locally, calling the store once per batch. **Prevention.** Detect keys above a rate threshold automatically and move them to leasing; accept that over-admission is bounded by instances times batch size.

False positives behind shared IP addresses

Scenario. A limit of 100 requests per minute per IP blocks every user in a company or on a mobile carrier gateway. **Symptoms.** Many users report 429 from the same address range while their individual usage is low.

Diagnosis. The blocked requests carry different user identifiers but the same source address. **Root cause.** The IP address was used as the identity for authenticated traffic, though many users can share one address.

Fix. Limit authenticated traffic by user or API key, and keep IP rules only for anonymous endpoints with high limits. **Prevention.** Choose the subject from the identity that the business wants to limit, and review IP rules for NAT effects.

2.3.8 Trade-offs and extensions

- **Fail open versus fail closed.** A limiter that fails open may let a flood through during an outage, one that fails closed turns its own failure into an outage; the decision belongs to each rule and is recorded in it.
- **Placement.** An edge or CDN limit protects the whole platform cheaply but sees little identity; a gateway limit knows the user; a service-level limit knows the cost of each operation. Layers are complementary.
- **Dynamic limits.** Adaptive concurrency limits that follow observed latency complement fixed quotas by protecting against overload that no fixed number predicts.
- **Multi-region.** Each region enforces its share of a global quota, with periodic rebalancing; exact global limits across regions cost a cross-region round trip per request and are rarely worth it.

2.3.9 Questions to expect

Which algorithm would be chosen for a public API and why?

A token bucket, because it permits short bursts that real clients produce and constrains the sustained rate, and because it needs two numbers per key. A sliding window counter is chosen when the contract is a strict count per window.

How is the limiter made accurate across many gateway instances?

All instances consult the same counter through an atomic script, so the count is exact per key. The cost is one round trip per request, which is reduced by leasing batches of tokens for very hot keys.

What happens when Redis is unavailable?

A short timeout and a circuit breaker prevent the limiter from slowing requests. Each rule defines its policy: fail closed for sensitive operations, otherwise degrade to local limits of limit divided by instance count, and alert.

How does the fixed window counter fail?

A client can send the full limit just before and just after the boundary, which is twice the limit within a short time. The sliding window counter or the token bucket removes the cliff.

How are rules changed without redeploying?

Gateways cache the rule set and refresh it on a timer or on a change notification. Changes are versioned so that a bad rule can be rolled back, and a new rule can be applied in shadow mode (decisions logged, not enforced) first.

How is clock skew between gateways handled?

Use the store's clock inside the script where the server version allows it, or keep the gateway clocks synchronised with NTP and make the algorithm tolerant by using `max(0, now - ts)` so that a small backwards step never produces negative refill.

How many shards are needed for 400,000 checks per second?

Divide by the benchmarked capacity per shard. With an assumed 80,000 checks per second, five shards are the minimum and eight keep utilisation near 60%, which leaves room for a shard failing over.

How would a limit per tenant across all its users be implemented?

Add a second rule with the tenant as the subject, evaluated in the same request. The tenant key is hot by construction, so it uses token leasing.

sreeram

KEY POINTS

- The decision must be one atomic step: a server-side script or a single command, never read-then-write.
- A token bucket suits most APIs; a fixed window can admit twice the limit across a boundary.
- A per-node split of a global limit under-admits when load is uneven; use a central count, plus token leasing for hot keys.
- Decide per rule whether the limiter fails open, closed or local, set a 10 ms timeout, and use a circuit breaker.
- Size the shard count from the operation rate (about 80,000 checks per second per shard, assumed), not from memory.

2.4 Design: Unique ID Generator

Distributed systems need identifiers that are unique without asking a single database for every one. A relational auto-increment column does not survive sharding, a random UUID is large and ruins index locality, and a global counter service is a bottleneck and a single point of failure. This chapter designs an id service for 64-bit identifiers that are unique across machines, roughly ordered by creation time, and generated without a network call per id. The embedded Snowflake implementation of the earlier chapter on distributed coordination is the starting point; here the focus is the service around it: assigning worker ids, handling clocks, the alternative of segment allocation, and the failures that cause duplicates.

2.4.1 Requirements and scope

Functional requirements

- Return globally unique 64-bit ids, one at a time or in batches.
- Ids are roughly time-ordered (k-sortable): ids created later are larger, within the clock accuracy.
- Ids fit in a signed 64-bit integer (a `BIGINT` column and a Java `long`).

Non-functional requirements

- Throughput of 100,000 ids per second in total (assumed peak), with spare capacity.
- Latency below 5 ms at p99 over the network (assumed), or zero when embedded as a library.
- No single point of failure; a node failure must not stop id generation.
- Uniqueness is mandatory; order is best effort.

Out of scope: gapless sequences (for example invoice numbers, which need a transactional counter), strict global ordering (which needs a consensus log), and ids that must be unguessable (use random tokens). **Assumptions**: node clocks are synchronised by NTP to within tens of milliseconds, and nodes are restarted from time to time.

2.4.2 Capacity estimation

Quantity	Arithmetic	Result
Peak demand	assumed	100,000 ids per second
Capacity of one worker	12 sequence bits: 4,096 per millisecond x 1,000	4.096 million per second
Workers for throughput	100,000 / 4,096,000	1 (3 to 5 nodes for availability)
Lifetime of 41 time bits	2^41 ms / (1,000 x 3,600 x 24 x 365.25)	69.7 years from the custom epoch
Worker id space	10 bits	1,024 workers
Id size	64 bits	8 bytes (UUID: 16 bytes)
Index size difference	10 billion rows x 8 bytes against 16 bytes	80 GB against 160 GB per index
Batch request	100 ids per call, 8 bytes each	800 bytes of payload

Capacity estimate for the id generator (inputs assumed).

Throughput is not the constraint: one node could serve the entire load. The number of nodes follows from availability and from the geographic spread of callers.

2.4.3 API design

API

```
GET /v1/ids?count=100
  200 { "ids": ["4194455552", "4194455553", ...] }     ids as decimal strings
gRPC: rpc NextIds(NextIdsRequest{count}) returns (NextIdsResponse{repeated int64 ids})
```

Ids are returned as strings over JSON because JavaScript numbers are IEEE 754 doubles that represent integers exactly only up to 2^53 (about 9.0e15), and a Snowflake id uses up to 63 bits. A client that parses the id as a number silently rounds it. The gRPC variant uses a native 64-bit integer.

2.4.4 Data model

The generator holds almost no persistent data, which is the point of the design. The only shared state is the assignment of worker ids.

Key or table	Content	Purpose
`/workers/{0..1023}`	node address, lease TTL 30 s, heartbeat every 10 s	Exclusive worker id for a running node (registry)
`last_ts (local file)`	highest timestamp issued	Refuse or wait after a restart when the clock is earlier
`id_segments`	biz_tag PK, max_id, step, updated_at	Segment allocation: one row per id space

State used by the two generation schemes.

2.4.5 High-level architecture

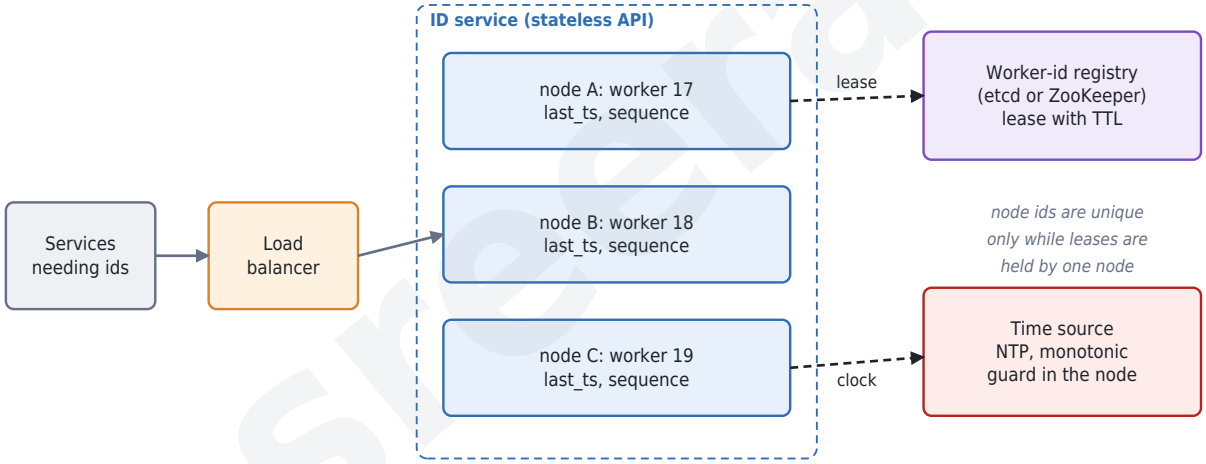
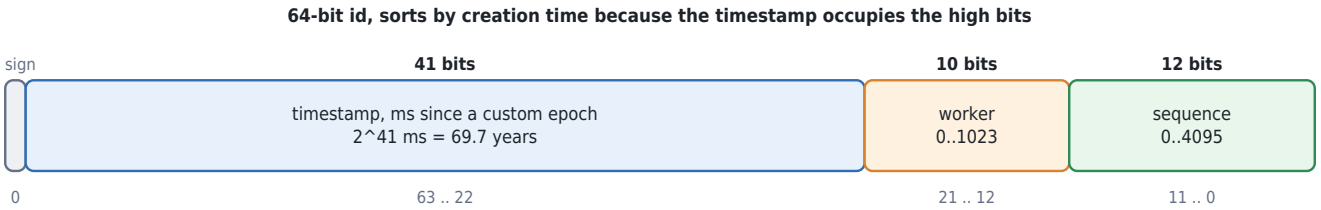


Figure 2.11. Each ID service node holds a leased worker id and generates ids locally from the clock; the registry and the time source are the only external dependencies, and neither is on the per-id path.

Start-up. A node acquires a lease on a free worker id in the registry (an ephemeral key with a time to live that the node refreshes), loads its persisted `last_ts` and refuses to serve until the clock has passed it. **Generation.** A request is handled entirely in memory: read the clock, compare with the last timestamp, increment the sequence or reset it, and compose the id. **Failure.** If a node loses its lease (network partition from the registry), it stops generating before the lease can expire for another holder; this is the rule that prevents two nodes from running with the same worker id.




```
id 4198649856 = (1001 << 22) | (37 << 12) | 0
timestamp = epoch + 1001 ms, worker = 37, sequence = 0
capacity: 4096 ids per millisecond per worker, 4.096 million per second
```

Figure 2.12. The 64 bits are split into a sign bit, 41 bits of time, 10 bits of worker and 12 bits of sequence; because time occupies the high bits, numeric order follows creation order.

2.4.6 Deep dive

Problem 1: choosing the scheme

Scheme	Properties	Problems	Verdict
Database auto-increment	Dense, ordered, 8 bytes	One writer; sharding needs offsets; the database is on the path of every insert	Single database only
Ticket server (two databases, odd and even)	Simple, 8 bytes	Servers are a dependency; order only within each server	Small systems
Segment allocation	Ranges leased from a table; local counting	Gaps after a crash; ids are not time-ordered across servers	Good when order is unimportant
Random UUID v4	No coordination, 128 bits	16 bytes; random inserts fragment B-tree indexes	When size is no issue
UUID v7 / ULID	128 bits, 48-bit millisecond timestamp, random remainder	16 bytes; no worker bits	When 128 bits are acceptable
Snowflake-style	64 bits, time-ordered, local generation	Needs worker ids and a trustworthy clock	Chosen

Id generation alternatives.

Problem 2: assigning worker ids safely

A static configuration file is simple but fails when a virtual machine image is cloned or a container is rescheduled with the same configuration; two nodes with worker id 17 produce identical ids whenever they generate in the same millisecond with the same sequence. A lease in a coordination service assigns each running node a unique worker id; the lease time to live must exceed the largest tolerated clock difference and pause (garbage collection, VM freeze), so that a node that was frozen cannot resume while another node already holds its id. A frozen node must check its lease before every batch, not only at start.

Problem 3: clocks

The timestamp must never decrease for a given worker. A clock can move backwards through an NTP step, a manual change, a virtual machine migration or a leap second. The safe policy is to remember the last issued timestamp, and when the clock reads less, either wait (for steps of a few milliseconds) or refuse (for large steps) and raise an alert, as the program below does. Prefer clock slewing to stepping in the NTP configuration, and use a monotonic clock for measuring elapsed time within the process while anchoring it once to the wall clock at start.

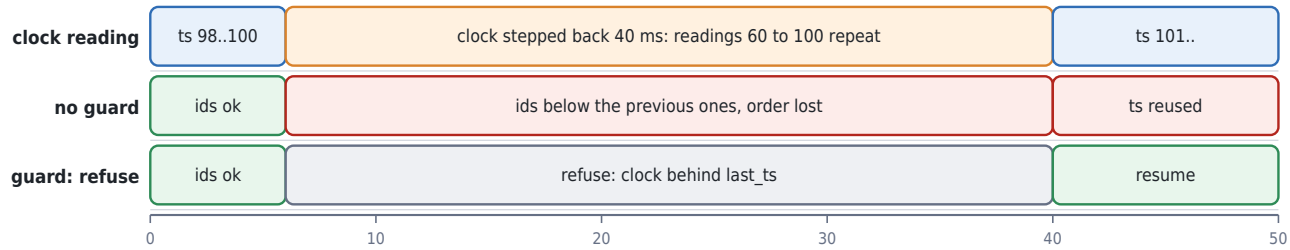


Figure 2.13. After a clock step backwards, a generator without a guard issues smaller timestamps and later repeats timestamps already used, which can duplicate ids; a guard refuses until the clock passes the last issued value.

The next program implements the generator with a fake clock so that rollback and overflow can be reproduced, and the segment allocator with prefetching.

IdGen.java JAVA

```
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.LongSupplier;

public class IdGen {
    static final long EPOCH = 1_700_000_000_000L;   // custom epoch, ms
    static final int WORKER_BITS = 10, SEQ_BITS = 12;
    static final long MAX_SEQ = (1L << SEQ_BITS) - 1;

    /** Snowflake-style generator: 41 bits time | 10 bits worker | 12 bits sequence. */
    static class Snowflake {
        final long worker; final LongSupplier clock;
        long lastMs = -1, seq = 0; long waits = 0, rollbacks = 0;
        Snowflake(long worker, LongSupplier clock) { this.worker = worker; this.clock = clock; }

        synchronized long next() {
            long now = clock.getAsLong();
            if (now < lastMs) {                       // clock moved backwards
                rollbacks++;
                if (lastMs - now > 5) throw new IllegalStateException(
                    "clock moved back " + (lastMs - now) + " ms, refusing to generate");
                now = lastMs;                         // tolerate a small step: stay on lastMs
            }
            if (now == lastMs) {
                seq = (seq + 1) & MAX_SEQ;
                if (seq == 0) {                       // 4096 ids used in this millisecond
                    waits++;
                    while (now <= lastMs) now = clock.getAsLong();
                }
            } else {
                seq = 0;
            }
            lastMs = now;
            return ((now - EPOCH) << (WORKER_BITS + SEQ_BITS)) | (worker << SEQ_BITS) | seq;
        }
    }

    static String describe(long id) {
        long ts = (id >>> (WORKER_BITS + SEQ_BITS)) + EPOCH;
        long worker = (id >>> SEQ_BITS) & ((1L << WORKER_BITS) - 1);
        long seq = id & MAX_SEQ;
        return "ts=" + ts + " worker=" + worker + " seq=" + seq;
    }

    /** Segment allocator: a database row hands out ranges; the app serves ids from memory. */
    static class SegmentStore {
        final AtomicLong maxId = new AtomicLong(0); int roundTrips = 0;
        synchronized long[] allocate(int step) {
            roundTrips++;
            long start = maxId.getAndAdd(step) + 1;
            return new long[] {start, start + step - 1};
        }
    }

    static class SegmentGenerator {
        final SegmentStore store; final int step;
        long cur, end; long[] standby;
```

sreram

IdGen.java (continued) JAVA

```
        SegmentGenerator(SegmentStore store, int step) {
            this.store = store; this.step = step;
            long[] s = store.allocate(step); cur = s[0]; end = s[1];
        }
        synchronized long next() {
            if (standby == null && cur > end - step * 0.9) {   // 10% consumed: prefetch
                standby = store.allocate(step);
            }
            if (cur > end) {
                if (standby == null) standby = store.allocate(step);
                cur = standby[0]; end = standby[1]; standby = null;
            }
            return cur++;
        }
    }

    public static void main(String[] args) {
        // 1) bit layout
        long[] fake = {EPOCH + 1000};
        Snowflake sf = new Snowflake(37, () -> fake[0]);
        long a = sf.next(), b = sf.next(); fake[0] += 1; long c = sf.next();
        System.out.println(a + "  " + describe(a));
        System.out.println(b + "  " + describe(b));
        System.out.println(c + "  " + describe(c));
        System.out.println("max ids per worker per ms = " + (MAX_SEQ + 1)
            + ", per worker per second = " + (MAX_SEQ + 1) * 1000);
        System.out.println("lifetime of 41 bits = "
            + (1L << 41) / (1000.0 * 3600 * 24 * 365.25) + " years");

        // 2) sequence overflow: the clock advances 1 ms per 5000 reads, faster than 4096 ids per ms
        long[] tick = {0};
        Snowflake busy = new Snowflake(1, () -> EPOCH + (tick[0]++ / 5000));
        long prev = -1; boolean sorted = true;
        for (int i = 0; i < 10_000; i++) { long id = busy.next(); sorted &= id > prev; prev = id; }
        System.out.println("10000 ids, strictly increasing=" + sorted
            + ", overflow waits=" + busy.waits);

        // 3) clock rollback: 3 ms back is absorbed, 40 ms back is refused
        long[] t = {EPOCH + 100};
        Snowflake sf2 = new Snowflake(2, () -> t[0]);
        long x = sf2.next();
        t[0] -= 3; long y = sf2.next();
        System.out.println("after 3 ms rollback: id still increasing = " + (y > x));
        t[0] = EPOCH + 100 - 40;
        try { sf2.next(); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }

        // 4) segment allocator
        SegmentStore store = new SegmentStore();
        SegmentGenerator g = new SegmentGenerator(store, 1000);
        long last = 0; boolean mono = true;
        for (int i = 0; i < 10_500; i++) { long id = g.next(); mono &= id > last; last = id; }
        System.out.println("segment: 10500 ids, last=" + last + ", monotonic=" + mono
            + ", database round trips=" + store.roundTrips);
        // crash: the in-memory remainder of the segment is lost, ids skip but never repeat
        SegmentGenerator g2 = new SegmentGenerator(store, 1000);
        System.out.println("after restart the first id is " + g2.next() + " (gap, no reuse)");
    }
}
```

Run IdGen (JDK 21) TERMINAL

```
$ java IdGen.java
4194455552  ts=1700000001000 worker=37 seq=0
4194455553  ts=1700000001000 worker=37 seq=1
4198649856  ts=1700000001001 worker=37 seq=0
max ids per worker per ms = 4096, per worker per second = 4096000
lifetime of 41 bits = 69.6828420270236 years
10000 ids, strictly increasing=true, overflow waits=2
after 3 ms rollback: id still increasing = true
clock moved back 40 ms, refusing to generate
segment: 10500 ids, last=10500, monotonic=true, database round trips=12
after restart the first id is 12001 (gap, no reuse)
```

What to notice. The decoded id shows the three fields. Ten thousand ids were generated when the clock advanced slower than 4,096 ids per millisecond; the generator waited twice for the next millisecond (overflow waits) and the ids stayed strictly increasing. A rollback of 3 ms was absorbed by staying on the last timestamp, a rollback of 40 ms raised an error. The segment allocator served 10,500 ids with 12 database round trips (including one prefetched segment that was not yet used), and after a restart it started at 12,001: the unused part of the old segment is a gap, but no id is reused.

2.4.7 Bottlenecks, failures and real-world problems

Duplicate ids from two nodes with the same worker id

Scenario. A deployment template copies the worker id 17 to a second instance during a scale-out. **Symptoms.** Unique-key violations on inserts, appearing only under load and only for rows created in the same millisecond by different nodes.

Diagnosis. Decode the colliding ids: identical timestamp and worker fields, different callers. **Root cause.** Worker ids were assigned by configuration instead of an exclusive lease.

Fix. Acquire worker ids from the registry with a lease, and refuse to start without one. **Prevention.** Log the worker id on every node at start and alert on two nodes with the same value; test cloned images in staging.

Clock steps backwards after a time synchronisation

Scenario. An NTP daemon steps the clock back 40 ms after a long drift. **Symptoms.** Ids go out of order; if the generator has no guard, ids are duplicated when the clock passes the old range again.

Diagnosis. Compare the generation log with the system log of the time daemon; find the step event at the same instant. **Root cause.** Wall-clock time is not monotonic.

Fix. Persist the last timestamp, wait for small rollbacks and refuse for large ones, and have the node report unhealthy so that the load balancer removes it. **Prevention.** Configure slewing instead of stepping, monitor clock offset, and alert above a threshold of a few milliseconds.

Sequence overflow under a burst

Scenario. A batch job asks one node for 50,000 ids inside a single millisecond. **Symptoms.** Latency of the batch increases in steps of one millisecond.

Diagnosis. The overflow-wait counter rises. **Root cause.** The sequence is 12 bits, so at most 4,096 ids per millisecond per node are available.

Fix. Wait for the next millisecond when the sequence overflows (the demo shows two waits for 10,000 ids when the clock is slow), spread large batches over several nodes, or give callers a block of ids to use locally. **Prevention.** Size the worker-bit and sequence-bit split for the peak burst, and measure the overflow counter.

Large ids corrupted in a browser

Scenario. A web page displays order id 4198649857 correctly but id 1802389301384290305 differently from the server value. **Symptoms.** Clicking a link opens the wrong order or a 404; the last digits of the id differ.

Diagnosis. Compare the JSON text with the value in the JavaScript console. **Root cause.** The JSON number was parsed as an IEEE 754 double, which is exact only up to 2^53.

Fix. Serialise ids as strings in APIs consumed by JavaScript. **Prevention.** Contract tests with ids above 2^53, and a lint rule for `long` fields in public schemas.

2.4.8 Trade-offs and extensions

- **Time-ordered versus unguessable.** Time-ordered ids leak creation time and, through the sequence, approximate volume; where this matters, expose a separate public token and keep the Snowflake id internal.
- **Bit layout.** The 10 + 12 split can be changed (for example 8 worker bits and 14 sequence bits) to match the number of nodes and the burst size; once ids exist the layout is fixed.
- **Epoch.** The custom epoch should be a recent date, because 41 bits last 69.7 years from it; moving the epoch later would only be possible with a new id format.
- **Embedding.** The generator can run in each application process with worker ids from the registry, which removes the network hop and the service, at the price of a larger registry and worker-id space.

sreram

2.4.9 Questions to expect

Why not use UUIDs?

A random UUID is 128 bits and its randomness spreads inserts over the whole index, which causes page splits and poor cache locality in a B-tree. A time-ordered 64-bit id appends at the right edge of the index and halves the key size. UUID v7 repairs the ordering but keeps 128 bits.

How does the generator guarantee uniqueness?

Three fields make an id unique: the timestamp, the exclusive worker id and the per-millisecond sequence. Uniqueness holds as long as worker ids are exclusive and the timestamp never decreases for a worker.

What happens if the clock moves backwards?

The generator compares the clock with its last timestamp; it waits for small differences, refuses for larger ones and reports unhealthy, so that duplicates are impossible. Operations avoid clock steps through slewing.

How many ids can one node issue per second?

4,096 per millisecond, which is 4,096,000 per second with 12 sequence bits. Overflow waits for the next millisecond.

What is the difference between this and segment allocation?

Segment allocation hands ranges of numbers from a database to servers, which count locally. It needs no clock and gives dense numbers, but ids are not time-ordered across servers and a crash leaves gaps. Snowflake needs a clock and worker ids and is naturally k-sortable.

How are ids made unique across regions?

Worker ids are drawn from one registry that is global, or the worker bits are split into region and node parts (for example 3 plus 7 bits), so that each region assigns its own range without coordination.

Can ids be strictly ordered?

Not without a single sequencer or consensus. Snowflake ids are ordered per worker and approximately across workers within the clock error. Systems needing a strict total order use a consensus log, at the price of throughput and latency.

What breaks after 69.7 years?

The 41-bit timestamp overflows; by choosing a recent custom epoch the lifetime starts at that date. A migration to a new format would be needed long before that.

KEY POINTS

- 64 bits split into 41 time, 10 worker, 12 sequence give 4,096 ids per millisecond per node and 69.7 years of time.
- Uniqueness depends on exclusive worker ids (use leases, never copied configuration) and on a timestamp that never decreases.
- Wait for small clock rollbacks, refuse large ones, and configure time daemons to slew instead of step.
- Return 64-bit ids as strings to JavaScript clients; numbers above 2^53 are corrupted.
- Segment allocation trades ordering for simplicity: no clock, dense numbers, gaps after a crash.

2.5 Design: Distributed Key-Value Store

A key-value store maps keys to values and offers `put`, `get` and `delete`. Designing one that spans many machines, survives failures of nodes and data centres, and stays writable during partitions is the subject of the Dynamo paper (Amazon, SOSP 2007), which is the model for this chapter; several later systems follow the same family of ideas (consistent hashing, tunable quorums, vector clocks, hinted handoff, anti-entropy repair). The chapter designs a Dynamo-style store, labelled as such: where a design choice is taken from the paper, the text says so, and the numbers are assumptions for this exercise. The store trades strong consistency for availability and latency, and the interesting part is how it repairs the inconsistency that this trade creates.

sreeram

2.5.1 Requirements and scope

Functional requirements

- `put(key, value)`, `get(key)` and `delete(key)` for values up to about 1 MB, with typical sizes of 2 KB (assumed).
- Per-request consistency choice: the client selects how many replicas must acknowledge a write (`W`) and answer a read (`R`).
- Incremental scaling: nodes are added and removed while the system serves traffic.

Non-functional requirements

- Always writable: a write is accepted as long as `W` nodes (or their stand-ins) are reachable.
- Latency at p99 below 10 ms inside one data centre (assumed).
- No single point of failure: no master, every node can coordinate any request.
- Durability: three replicas of every item, placed on different failure domains.
- Eventual consistency with explicit conflict detection.

Out of scope are multi-key transactions, secondary indexes, range scans across keys, and strong consistency (a consensus-based store is the alternative when those are required). **Assumptions**: 5 billion items, 2 KB each, 100,000 operations per second at peak with 80% reads, one region with three availability zones.

2.5.2 Capacity estimation

Quantity	Arithmetic	Result
Logical data	5.0e9 items x 2 KB	10 TB
Replicated data (N = 3)	10 TB x 3	30 TB
Usable disk per node	4 TB NVMe at 50% fill, leaving room for compaction	2 TB
Nodes for storage	30 TB / 2 TB	15 nodes
Client writes	100,000 x 20%	20,000 per second
Client reads	100,000 x 80%	80,000 per second
Replica operations	writes 20,000 x 3 + reads 80,000 x 2 (R = 2)	220,000 per second
Load per node	220,000 / 15	about 14,700 operations per second
Node capacity	assumed, must be benchmarked	25,000 operations per second (59% used)
Bloom filter memory	1.0e9 replicas per node x 10 bits	about 1.25 GB per node
Replication traffic	writes 20,000 x 2 KB x 2 extra copies + reads 80,000 x 2 KB x 1 extra read	about 80 MB/s + 160 MB/s inside the cluster
Data moved when adding the 16th node	30 TB x 1/16	about 1.9 TB, 6.25% of the data

Capacity estimate for the key-value store (inputs assumed).

Storage, not operation rate, sets the node count here (15 nodes); the operation rate leaves a 41% reserve, which is needed because a failed node's load moves to its neighbours.

2.5.3 API design

API

```
PUT /v1/kv/{key}?w=2
  headers: X-Context: <opaque vector clock from the last get; omit for a new key>
  body: value bytes (up to 1 MB)
  200 OK  { "context": "..." }          503 if fewer than w replicas (or stand-ins) acknowledged

GET /v1/kv/{key}?r=2
  200 { "versions": [ { "value": "...", "context": "..." } ] }   one entry, or several siblings
  404 not found

DELETE /v1/kv/{key}?w=2          writes a tombstone (a version that marks deletion)
```

The `get` may return several versions (siblings) when concurrent writes could not be ordered; the client merges them and writes the result back with the merged context. A client that does not want to merge selects the `lww` option for the key space, where the version with the highest timestamp wins and the others are discarded (last write wins), which is simple but can lose a write when clocks disagree.

2.5.4 Data model

Partitioning. The key is hashed with a 128-bit hash and the result is a position on a ring (the paper uses MD5). Each physical node owns many virtual tokens (assumed 128), and the owner of a key is the node whose token is the first one clockwise from the key's position. The `N = 3` replicas are the first three distinct physical nodes found while walking clockwise, with the constraint that they lie in different availability zones; this list is the preference list of the key.

On-node layout. Each node uses a log-structured engine: writes are appended to a commit log and applied to an in-memory sorted table (memtable); full memtables are flushed to immutable sorted files (SSTables) that are merged by background compaction; each file has a Bloom filter to skip files that cannot contain a key. The engine details belong to the databases material (LSM trees); the store only requires `put`, `get` by key and delete-by-tombstone.

item.json — JSON

```
{
  "key": "cart:42",
  "versions": [
    { "value": "<bytes>", "clock": { "E": 3 },       "ts": 1760866800123, "tombstone": false },
    { "value": "<bytes>", "clock": { "B": 1, "E": 2 }, "ts": 1760866800456, "tombstone": false }
  ]
}
```

The record holds the set of current versions, each with a vector clock (one counter per coordinating node) and a wall-clock timestamp used only for last-write-wins mode and for tombstone expiry. A tombstone is retained for a fixed period (assumed 10 days) so that a node that missed the delete can learn about it during repair.

2.5.5 High-level architecture

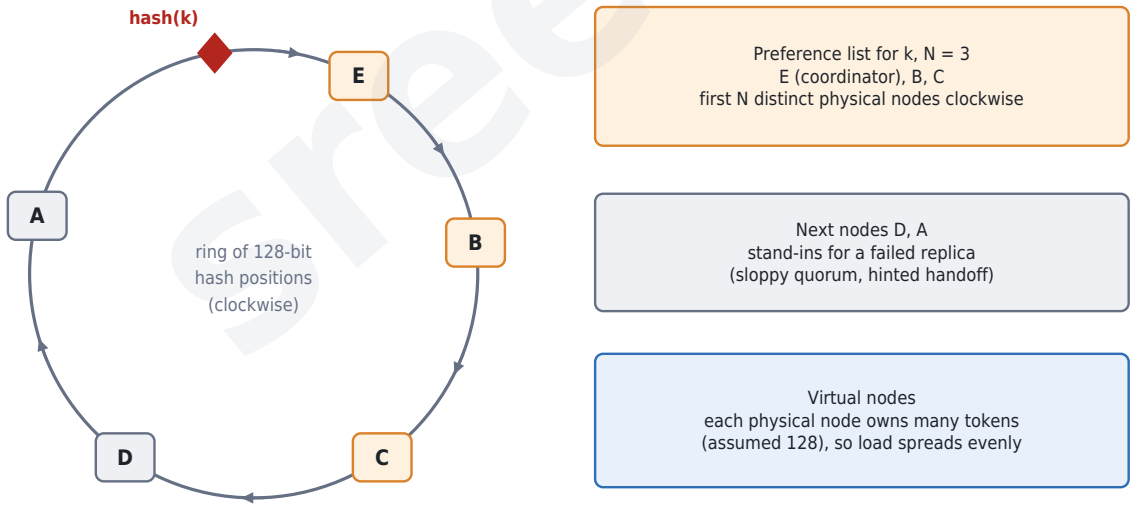


Figure 2.14. A key is placed on a ring by its hash; the first three distinct nodes clockwise form its preference list, and the next nodes serve as stand-ins when a replica is unreachable.

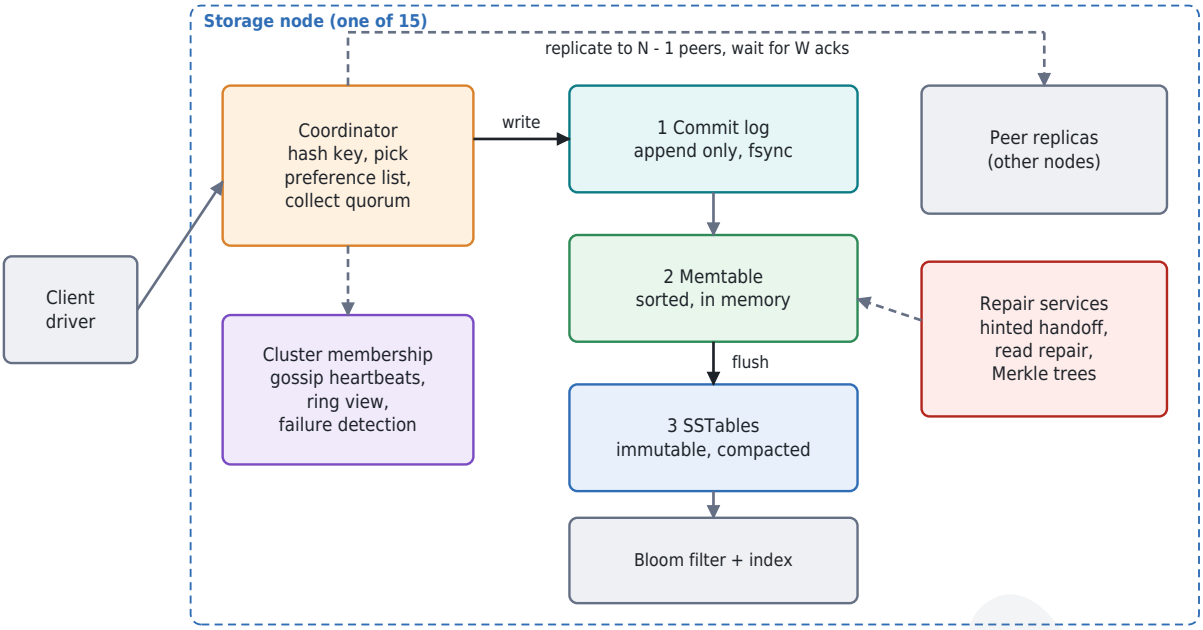


Figure 2.15. Every node contains a coordinator, a membership component that learns the ring through gossip, a storage engine, and background repair services; there is no master.

Write flow. (1) The client sends `put` to any node (or to a node in the preference list, if its driver knows the ring), which becomes the coordinator. (2) The coordinator hashes the key, finds the preference list, increments its own counter in the vector clock supplied by the client, and writes the new version locally. (3) It sends the version to the other replicas in parallel. (4) When `W` replicas have acknowledged it returns success; the remaining replicas finish in the background. (5) If a replica is unreachable, the coordinator writes the version, tagged with a hint naming the intended owner, to the next healthy node on the ring (a stand-in), which counts towards `W`; this is a sloppy quorum.

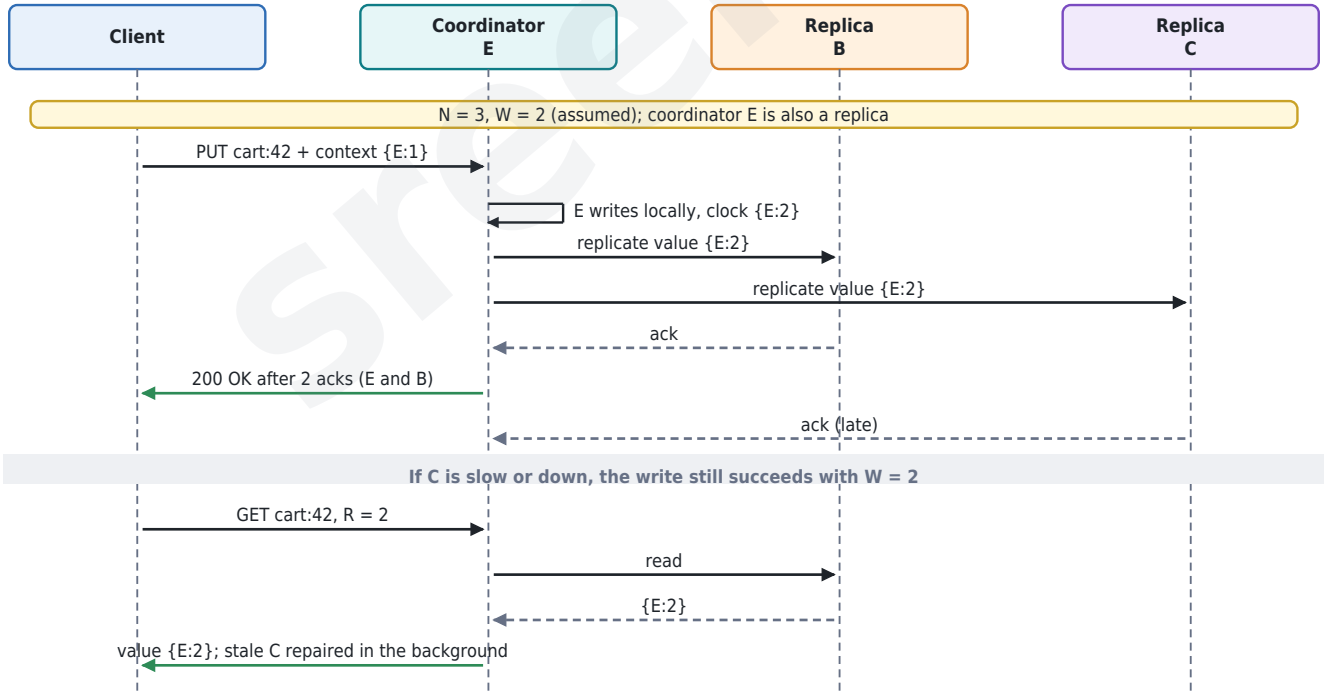


Figure 2.16. A write succeeds after W = 2 acknowledgements and a read after R = 2 answers; the slow third replica is repaired in the background.

Read flow. The coordinator asks the replicas, waits for `R` answers, discards versions whose vector clock is dominated by another, and returns the rest. If the answers differ, it sends the newest versions to the stale replicas (read repair).

Membership and failure detection. Nodes exchange membership and heartbeat information by gossip, so every node learns the ring and the liveness of the others within seconds without a central service. A node that is unreachable for longer than a timeout is considered down; permanent removal is an explicit administrative action.

2.5.6 Deep dive

Problem 1: tunable consistency with quorums

With `N` replicas, a write acknowledged by `W` replicas and a read that consults `R` replicas, the read set and the write set intersect whenever `R + W > N`, so the read sees at least one replica that holds the latest acknowledged write. The settings trade latency and availability:

R	W	R+W > N	Behaviour	Typical use
2	2	yes	Balanced; tolerates one slow or down replica on both paths	Default
1	3	yes	Fast reads; a write fails if any replica is down	Read-heavy data that changes rarely
3	1	yes	Fast, always-available writes; reads need all replicas	Write-heavy ingestion
1	1	no	Lowest latency; stale reads are possible and conflicts frequent	Caches, counters that tolerate staleness

Quorum settings for N = 3.

The Dynamo paper reports `(N, R, W) = (3, 2, 2)` as the common production setting; this table follows it as a default. The simulation below makes the intersection argument visible together with hinted handoff and siblings.

KvSim.java JAVA

```java
import java.util.*;

public class KvSim {
    record Version(String value, Map<String, Integer> clock) {
        public String toString() { return value + clock; }
    }

    static boolean descends(Map<String, Integer> a, Map<String, Integer> b) {   // a >= b
        for (var e : b.entrySet()) if (a.getOrDefault(e.getKey(), 0) < e.getValue()) return false;
        return true;
    }

    /** Drop versions that are dominated by another version. */
    static List<Version> reconcile(Collection<Version> all) {
        List<Version> out = new ArrayList<>();
        for (Version v : all) {
            boolean dominated = false;
            for (Version o : all) {
                if (o != v && descends(o.clock(), v.clock()) && !o.clock().equals(v.clock())) {
                    dominated = true;
                }
            }
            if (!dominated && out.stream().noneMatch(x -> x.clock().equals(v.clock()))) out.add(v);
        }
        return out;
    }

    static class Node {
        final String name; boolean up = true;
        final Map<String, List<Version>> data = new HashMap<>();
        final List<Object[]> hints = new ArrayList<>();         // {targetName, key, version}
        Node(String name) { this.name = name; }
        void store(String key, Version v) {
            List<Version> cur = new ArrayList<>(data.getOrDefault(key, List.of()));
            cur.add(v);
            data.put(key, reconcile(cur));
        }
    }

    final int n = 3;
    final Map<String, Node> nodes = new LinkedHashMap<>();
    final TreeMap<Integer, String> ring = new TreeMap<>();

    KvSim(String... names) {
        for (String s : names) {
            nodes.put(s, new Node(s));
            for (int v = 0; v < 8; v++) ring.put(hash(s + "#" + v), s);
```

KvSim.java (continued) JAVA

```
        }
    }

    static int hash(String s) {
        int h = 0x811C9DC5;
        for (char c : s.toCharArray()) { h ^= c; h *= 0x01000193; }
        h ^= h >>> 15; h *= 0x2C1B3C6D; h ^= h >>> 12;
        return h;
    }

    /** Distinct physical nodes clockwise from the key position; first n are the preference list. */
    List<String> walk(String key) {
        List<String> out = new ArrayList<>();
        var tail = ring.tailMap(hash(key), true).values();
        for (String s : tail) if (!out.contains(s)) out.add(s);
        for (String s : ring.values()) if (!out.contains(s)) out.add(s);
        return out;
    }

    /** Sloppy-quorum write. Returns acks. Coordinator = first live node of the preference list. */
    int put(String key, String value, Map<String, Integer> ctx, int w, Set<String> unreachable) {
        List<String> walk = walk(key);
        List<String> pref = walk.subList(0, n);
        String coord = pref.stream().filter(s -> nodes.get(s).up).findFirst().orElseThrow();
        Map<String, Integer> clock = new TreeMap<>(ctx);
        clock.merge(coord, 1, Integer::sum);
        Version v = new Version(value, clock);
        int acks = 0, spare = n;
        for (String target : pref) {
            if (nodes.get(target).up && !unreachable.contains(target)) {
                nodes.get(target).store(key, v); acks++;
            } else {                                    // stand-in node holds a hint
                while (spare < walk.size() && (!nodes.get(walk.get(spare)).up
                        || unreachable.contains(walk.get(spare)))) spare++;
                if (spare < walk.size()) {
                    Node s = nodes.get(walk.get(spare++));
                    s.hints.add(new Object[] {target, key, v});
                    acks++;
                    System.out.println("    hint for " + target + " stored on " + s.name);
                }
            }
        }
        System.out.println("  put " + key + "=" + value + " coordinator=" + coord + " acks=" + acks
            + (acks >= w ? " (W=" + w + " met)" : " (W=" + w + " NOT met)"));
        return acks;
    }

    List<Version> get(String key, int r, Set<String> unreachable) {
        List<Version> seen = new ArrayList<>();
        int asked = 0;
        for (String s : walk(key).subList(0, n)) {
            Node nd = nodes.get(s);
            if (!nd.up || unreachable.contains(s) || asked == r) continue;
            asked++;
            seen.addAll(nd.data.getOrDefault(key, List.of()));
        }
        return reconcile(seen);
    }

    void handoff() {
        for (Node s : nodes.values()) {
            if (!s.up) continue;
            Iterator<Object[]> it = s.hints.iterator();
            while (it.hasNext()) {
                Object[] h = it.next();
                Node target = nodes.get((String) h[0]);
                if (target.up) {
                    target.store((String) h[1], (Version) h[2]);
                    it.remove();
                    System.out.println("    handoff of " + h[1] + " from " + s.name
                        + " to " + target.name);
                }
            }
        }
    }
```

Sreeram

KvSim.java (continued) JAVA

```
    public static void main(String[] args) {
        KvSim kv = new KvSim("A", "B", "C", "D", "E");
        String key = "cart:42";
        System.out.println("preference list for " + key + " = " + kv.walk(key).subList(0, 3)
            + ", stand-in nodes = " + kv.walk(key).subList(3, 5));
        Set<String> none = Set.of();

        System.out.println("1) normal write and read (N=3, W=2, R=2)");
        kv.put(key, "[book]", Map.of(), 2, none);
        System.out.println("  get -> " + kv.get(key, 2, none));

        String down = kv.walk(key).get(1);
        System.out.println("2) replica " + down + " is down: sloppy quorum and hinted handoff");
        kv.nodes.get(down).up = false;
        List<Version> cur = kv.get(key, 2, none);
        kv.put(key, "[book, pen]", cur.get(0).clock(), 2, none);
        kv.nodes.get(down).up = true;
        System.out.println("  " + down + " is back, stale copy = "
            + kv.nodes.get(down).data.get(key));
        kv.handoff();
        System.out.println("  " + down + " after handoff = " + kv.nodes.get(down).data.get(key));

        System.out.println("3) concurrent writers on both sides of a partition");
        List<String> pref = kv.walk(key).subList(0, 3);
        Map<String, Integer> base = kv.get(key, 2, none).get(0).clock();
        Set<String> sideA = Set.of(pref.get(1), pref.get(2));     // writer 1 sees only pref[0]
        kv.put(key, "[book, pen, lamp]", base, 1, sideA);
        Set<String> sideB = Set.of(pref.get(0));                  // writer 2: first node down
        kv.nodes.get(pref.get(0)).up = false;
        kv.put(key, "[book, pen, mug]", base, 1, sideB);
        kv.nodes.get(pref.get(0)).up = true;
        List<Version> siblings = kv.get(key, 3, none);
        System.out.println("  read after heal returns " + siblings.size()
            + " siblings: " + siblings);
        Map<String, Integer> merged = new TreeMap<>();
        for (Version v : siblings) v.clock().forEach((k, c) -> merged.merge(k, c, Math::max));
        kv.put(key, "[book, pen, lamp, mug]", merged, 2, none);
        System.out.println("  after client merge -> " + kv.get(key, 3, none));

        System.out.println("4) why R + W > N matters: W=1 write reaching only one replica");
        String k2 = "profile:7";
        List<String> p2 = kv.walk(k2).subList(0, 3);
        kv.put(k2, "v1", Map.of(), 3, none);
        Node n1 = kv.nodes.get(p2.get(0));
        Version v2 = new Version("v2", new TreeMap<>(Map.of(p2.get(0), 2)));
        n1.store(k2, v2);
        System.out.println("  only " + p2.get(0) + " holds v2; replicas "
            + p2.subList(1, 3) + " hold v1");
        System.out.println("  R=1 reading from " + p2.get(2) + " only -> "
            + kv.get(k2, 1, Set.of(p2.get(0), p2.get(1))));
        System.out.println("  R=2 over " + p2.get(0) + "," + p2.get(1) + " -> "
            + kv.get(k2, 2, Set.of(p2.get(2))));
        System.out.println("  R=3 -> " + kv.get(k2, 3, none));
    }
}
```

Run KvSim (JDK 21; a simulation on one machine) TERMINAL

```
$ java KvSim.java
preference list for cart:42 = [E, B, C], stand-in nodes = [D, A]
1) normal write and read (N=3, W=2, R=2)
  put cart:42=[book] coordinator=E acks=3 (W=2 met)
  get -> [[book]{E=1}]
2) replica B is down: sloppy quorum and hinted handoff
    hint for B stored on D
  put cart:42=[book, pen] coordinator=E acks=3 (W=2 met)
  B is back, stale copy = [[book]{E=1}]
    handoff of cart:42 from D to B
  B after handoff = [[book, pen]{E=2}]
3) concurrent writers on both sides of a partition
    hint for B stored on D
    hint for C stored on A
  put cart:42=[book, pen, lamp] coordinator=E acks=3 (W=1 met)
```

Run KvSim (JDK 21; a simulation on one machine) (continued) TERMINAL

```
    hint for E stored on D
  put cart:42=[book, pen, mug] coordinator=B acks=3 (W=1 met)
  read after heal returns 2 siblings: [[book, pen, lamp]{E=3}, [book, pen, mug]{B=1, E=2}]
  put cart:42=[book, pen, lamp, mug] coordinator=E acks=3 (W=2 met)
  after client merge -> [[book, pen, lamp, mug]{B=1, E=4}]
4) why R + W > N matters: W=1 write reaching only one replica
  put profile:7=v1 coordinator=D acks=3 (W=3 met)
  only D holds v2; replicas [E, B] hold v1
  R=1 reading from B only -> [v1{D=1}]
  R=2 over D,E -> [v2{D=2}]
  R=3 -> [v2{D=2}]
```

What to notice. In step 2 the replica B is down, the coordinator still reaches `W = 2` and places a hint on the stand-in D; after B returns it has the stale value until the handoff delivers the new one. In step 3 two writers working from the same base version on both sides of a failure produce the siblings `{E:3}` and `{B:1, E:2}`, neither of which descends from the other, so a read returns both; the client merges them and the next write's clock `{B:1, E:4}` dominates both. In step 4, a value that reached only one replica is invisible to a read with `R = 1` from another replica but visible to a read with `R = 2` that includes the up-to-date replica, which is exactly the intersection property. Because the numbers come from a single-process simulation, they illustrate the logic and say nothing about performance.

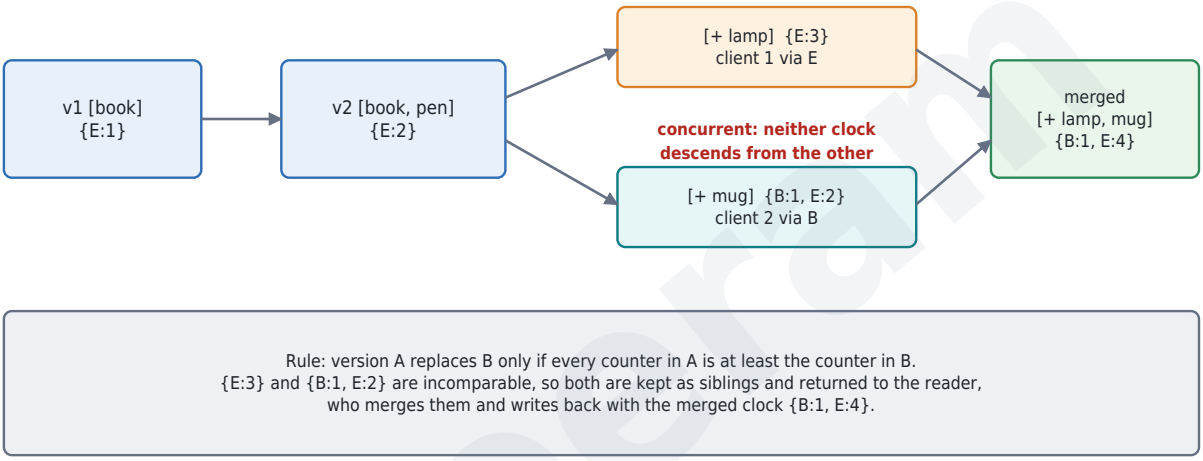


Figure 2.17. Concurrent updates produce two versions whose clocks are incomparable; both are returned as siblings and a client merge produces a version that dominates both.

Problem 2: conflicts

Strategy	How it works	Weakness	Verdict
Last write wins	Highest timestamp wins	Loses concurrent writes; depends on clock quality	Idempotent or overwrite-only data
Vector clocks with client merge	Detect concurrency, return siblings, application merges	Complexity for the client; clocks grow with the number of coordinators	Chosen for data with a natural merge (carts)
CRDTs (conflict-free replicated data types)	Data structures whose merge is defined (counters, sets)	Restricted data types	Counters, sets, flags

Conflict resolution alternatives.

Vector clocks grow with every coordinator that handled a write for the key. The paper bounds the size by dropping the oldest entries beyond a threshold (it mentions 10), accepting that a truncated clock can make a causally ordered update look concurrent and so produce an extra sibling, but never lose data.

Problem 3: failure handling and repair

Three mechanisms cover different time scales. **Hinted handoff** covers short outages: the stand-in keeps the hint and hands the data over when the owner returns; hints are kept for a bounded window (assumed 3 hours). **Read repair** fixes replicas that were stale when a read touched them. **Anti-entropy with Merkle trees** covers everything else: two replicas compare hash trees over key ranges, descend only into ranges whose hashes differ, and exchange the differing keys, so the cost of synchronising is proportional to the amount of difference and not to the data size. The three together are why replicas converge without a coordinator.

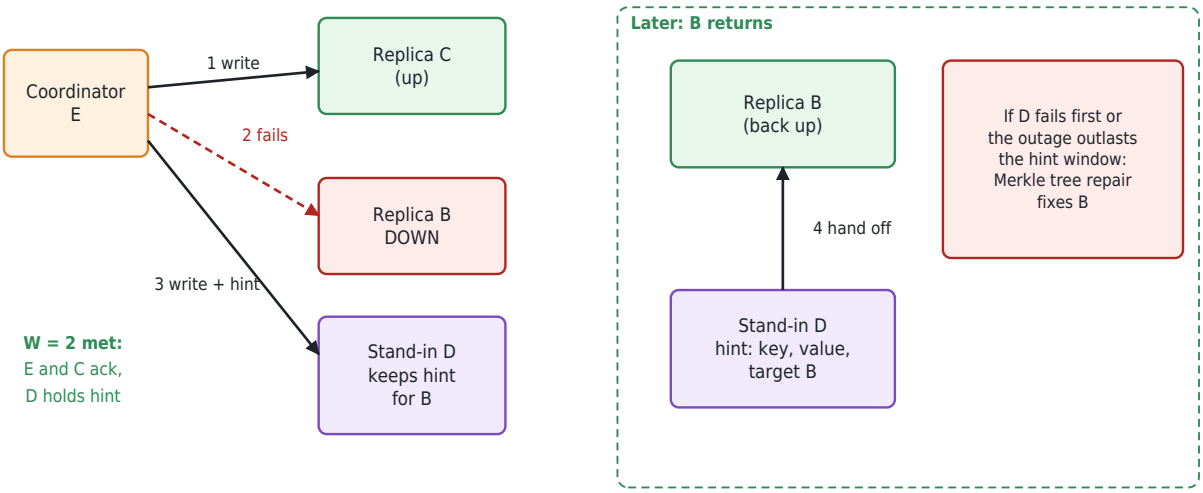


Figure 2.18. With replica B down, the write still reaches W = 2 and a stand-in keeps a hint; when B returns the hint is handed off, and if that fails Merkle-tree repair closes the gap.

2.5.7 Bottlenecks, failures and real-world problems

Stale read although R + W > N

Scenario. A client writes with `W = 2` during a short failure of replica B; the write is acknowledged by C and by the stand-in D (hint for B). A second client reads with `R = 2` from B and C a second later. **Symptoms.** The read does not return the latest value although `R + W > N` holds on paper.

Diagnosis. The acknowledgement of the write came from the stand-in, which is not in the preference list and was not asked during the read. **Root cause.** A sloppy quorum counts nodes outside the preference list towards `W`, so the intersection argument no longer holds until the hint is delivered.

Fix. For keys that need read-your-writes, use a strict quorum (the stand-in does not count), or read from all replicas when a hint may be pending, or route the session to the coordinator that handled its last write. **Prevention.** State in the API contract that sloppy quorums provide availability over strict intersection, and monitor the number of pending hints.

Deleted data returns

Scenario. A key is deleted with a tombstone; node X is down for 12 days while the tombstone is purged after 10 days; X returns with the old value. **Symptoms.** A key reappears; users see deleted items.

Diagnosis. The key is absent on two replicas and present on one; the tombstone is absent everywhere. **Root cause.** The tombstone retention period was shorter than the longest outage, so the node's old copy looked like the newest version during repair.

Fix. Rebuild a node that was down longer than the tombstone retention from scratch (clear its data and re-stream) instead of rejoining it, and run repair within the retention period. **Prevention.** Set the tombstone retention above the maximum tolerated outage plus the repair cycle length, and refuse to start a node whose last-seen time exceeds the retention.

Sibling explosion on a hot key

Scenario. Many clients update the same shopping cart concurrently from different coordinators. **Symptoms.** Reads return dozens of siblings; object size and read latency grow.

Diagnosis. The number of versions per key and the vector clock length appear in the per-key metrics. **Root cause.** Every blind write (a write without the context of a prior read) creates a new sibling, and the clients never merge.

Fix. Require the context on updates, merge on read and write the merged value back, bound the number of siblings, and use a CRDT set for carts. **Prevention.** Document that writes must carry the context of a previous read, and alert on the number of siblings.

One slow replica inflates tail latency

Scenario. A node with a failing disk answers in 400 ms instead of 3 ms. **Symptoms.** p99 latency for all reads whose preference list contains that node rises, though the node is "up".

Diagnosis. Per-replica latency histograms; the coordinator logs show the slow node last to respond. **Root cause.** With `R = 2` of 3, the coordinator waits for the second answer, and when the slow node is one of the two chosen the read waits for it.

Fix. Send the read to all three replicas and use the first two answers (a speculative read), and route away from nodes whose latency exceeds a threshold. **Prevention.** Probe replicas continuously, eject slow nodes from the coordinator's choice, and replace failing hardware.

2.5.8 Trade-offs and extensions

- **Availability against consistency.** The design follows the "AP" choice: it accepts writes during partitions and repairs later. Where a stale read is unacceptable (balances, inventory), a consensus-based store with a leader per partition is the alternative, at the price of lower availability during partitions.
- **Coordinator selection.** A client driver that knows the ring avoids one hop; a plain load balancer is simpler and adds a forwarding step.
- **Multi-data-centre.** Place replicas across zones in one region; across regions use asynchronous replication with local quorums, which gives local latency and eventual cross-region convergence.
- **Compaction and repair load.** Repair and compaction compete with foreground traffic; rate-limit both and schedule them off peak.

2.5.9 Questions to expect

Why consistent hashing with virtual nodes?

Adding or removing a node moves only the keys of the ranges it owns (6.25% of the data when the 16th node joins), and many tokens per node spread the load evenly and let a failed node's ranges be shared by many neighbours instead of one.

What does R + W > N guarantee, and what does it not?

It guarantees that the read set and write set intersect, so a read sees the latest acknowledged write under strict quorums. It does not provide linearizability: concurrent writes, sloppy quorums and partial failures can still produce stale or conflicting reads.

How does the store resolve conflicting versions?

Vector clocks detect that two versions are concurrent. The store returns both to the client, which merges them; alternatively last write wins discards the older timestamp. CRDTs make the merge automatic for suitable types.

What is hinted handoff?

When a replica is down, another node accepts the write with a hint naming the owner and delivers it when the owner returns. It preserves write availability and speeds up convergence, but it is bounded in time and size.

How do replicas that missed writes catch up?

Read repair fixes the keys that reads touch; anti-entropy compares Merkle trees between replicas and transfers only the differing ranges.

How is a node failure detected without a master?

Nodes gossip heartbeats; a node that is not heard from for a timeout is marked down by its peers locally. Permanent removal is a separate, explicit step.

Why is a delete a write?

A replica that missed a delete would otherwise resurrect the value during repair. A tombstone is a version that marks deletion, replicated and repaired like any other, and purged only after a retention period.

sreeram

How would the design change to give strong consistency?

Replace the leaderless replication with a consensus group per partition (Raft or Paxos), so that a majority commits each write in a total order. Reads go through the leader or use a read lease. The cost is that a partition without a majority cannot accept writes.

KEY POINTS

- Hash keys onto a ring with many virtual nodes; the preference list is the first N distinct nodes clockwise, spread across zones.
- Tune R and W per request: R + W > N gives intersection with strict quorums, and (3, 2, 2) is the balanced default.
- Vector clocks detect concurrent writes; the client merges siblings, or last write wins where losing a write is acceptable.
- Hinted handoff, read repair and Merkle-tree anti-entropy repair short, medium and long inconsistencies.
- Sloppy quorums keep writes available but weaken the intersection guarantee; tombstones must outlive the longest outage.

2.6 Design: Distributed Cache

A single cache server runs out of memory and of request capacity long before a large system does. A distributed cache spreads keys over many nodes, keeps copies to survive failures, and lets nodes be added without flushing everything. This chapter designs a Redis-like cluster in the style of Redis Cluster, whose documented design is used as a reference (hash slots, a cluster bus, asynchronous replication, redirections); the cache patterns that applications use (cache-aside, write-through, invalidation, stampede control) are covered in the chapter on caching and are not repeated. The focus is the infrastructure: partitioning, routing, replication, failover and online resharding.

2.6.1 Requirements and scope

Functional requirements

- `GET`, `SET` with time to live, `DEL`, and atomic operations on a single key (increment, list and hash updates); multi-key operations only for keys that share a hash tag.
- Eviction when memory is full, with a policy chosen per cluster (least recently used, least frequently used, or time-to-live based).
- Add and remove nodes online, with automatic rebalancing of keys.

Non-functional requirements

- Latency below 1 ms at p99 for a single-key operation inside the data centre (assumed).
- Throughput of 1,000,000 operations per second at peak (assumed).
- Availability: the loss of one node must not interrupt service for longer than a few seconds.
- Cache semantics: losing some data on failure is acceptable, because the source of truth is elsewhere.

Out of scope: durable storage guarantees, multi-key transactions across shards, global secondary indexes, cross-region replication. **Assumptions**: 500 million keys of 1 KB average value, 90% reads, one region with three availability zones.

sheeran

2.6.2 Capacity estimation

Quantity	Arithmetic	Result
Per-key memory	value 1,000 bytes + about 70 bytes of overhead (assumed)	about 1.07 KB
Total data	500,000,000 x 1.07 KB	535 GB
Memory limit per node	64 GB machine, `maxmemory` 48 GB (headroom for fragmentation and fork)	48 GB
Masters for memory	535 GB / 48 GB = 11.1; choose 16 for CPU headroom and smaller failover units	16 masters, 33.4 GB each (70%)
Total nodes	16 masters + 16 replicas	32
Slots per master	16,384 / 16	1,024
Operations per master	1,000,000 / 16	62,500 per second
Capacity per master	assumed, a single command thread, to be benchmarked	100,000 operations per second (62.5% used)
Replication traffic	10% writes x 1,000,000 x 1 KB	about 100 MB/s across the cluster
Client bandwidth	1,000,000 x 1 KB	1 GB/s, about 62 MB/s per master

Capacity estimate for the distributed cache (inputs assumed).

The command thread of each node is the limit that determines throughput: a node executes commands one at a time, which gives atomicity per key and bounds one node at the rate it can execute commands. Scaling therefore adds nodes, not threads.

2.6.3 API design

The cache speaks a simple request/response protocol (the Redis serialisation protocol), and the client library hides the cluster.

Client-visible commands and cluster redirections

```
GET key                      value or nil
SET key value EX 3600        set with a time to live of one hour
DEL key  /  UNLINK key       delete (UNLINK reclaims memory in a background thread)
MGET {user:42}.name {user:42}.email     allowed: both keys hash to the same slot
MGET a b                     rejected with -CROSSSLOT when a and b are in different slots

-MOVED 3999 10.0.1.7:6379    permanent: update the slot map, retry on the new owner
-ASK   3999 10.0.1.9:6379    temporary during migration: send ASKING, then retry there once
```

The client library fetches the slot map once (the `CLUSTER SLOTS` or `CLUSTER SHARDS` command), computes the slot of each key locally and sends the command directly to the owner, so the normal request needs no proxy hop. Redirections occur only after topology changes.

2.6.4 Data model

A key belongs to the slot `CRC16(key) mod 16384`; if the key contains `{...}` with a non-empty content, only that content is hashed, which lets applications co-locate related keys. Each slot is owned by exactly one master. Each node keeps its keys in an in-memory hash table, a second table of expiry times, and, per replica link, a replication backlog. Values are strings or collection types; the cache design uses the same keyspace model for all of them.

The program below implements the slot function (CRC16 XMODEM, the variant used by Redis Cluster), shows hash tags and measures how many of one million keys change owner when a fourth node joins.

ClusterSlots.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class ClusterSlots {
    /** CRC16 XMODEM (polynomial 0x1021, initial value 0), the variant used by Redis Cluster. */
    static int crc16(byte[] data) {
        int crc = 0;
        for (byte b : data) {
            crc ^= (b & 0xFF) << 8;
```

ClusterSlots.java (continued) JAVA

```
            for (int i = 0; i < 8; i++) {
                crc = ((crc & 0x8000) != 0) ? ((crc << 1) ^ 0x1021) : (crc << 1);
                crc &= 0xFFFF;
            }
        }
        return crc;
    }

    /** Slot of a key; only the text inside the first non-empty {...} is hashed. */
    static int slot(String key) {
        int open = key.indexOf('{');
        if (open >= 0) {
            int close = key.indexOf('}', open + 1);
            if (close > open + 1) key = key.substring(open + 1, close);
        }
        return crc16(key.getBytes(java.nio.charset.StandardCharsets.UTF_8)) % 16384;
    }

    /** Contiguous slot ranges, one per node. */
    static int[] ownerBySlot(int nodes) {
        int[] owner = new int[16384];
        for (int s = 0; s < 16384; s++) owner[s] = (int) ((long) s * nodes / 16384);
        return owner;
    }

    public static void main(String[] args) {
        System.out.printf("crc16(\"123456789\") = 0x%04X (XMODEM check value is 0x31C3)%n",
            crc16("123456789".getBytes()));
        for (String k : new String[] {"foo", "bar", "user:1000", "{user:1000}.cart",
                                      "{user:1000}.orders", "foo{}{bar}"}) {
            System.out.printf("slot(%-20s) = %d%n", "\"" + k + "\"", slot(k));
        }
        int[] three = ownerBySlot(3);
        System.out.println("3 masters: node0 owns slots 0.."
            + lastSlot(three, 0) + ", node1 owns " + firstSlot(three, 1) + ".." + lastSlot(three, 1)
            + ", node2 owns " + firstSlot(three, 2) + "..16383");

        // growth from 3 to 4 masters with 1,000,000 keys
        int keys = 1_000_000;
        int[] four = ownerBySlot(4);
        long movedSlotsOnly = 0;
        long movedModulo = 0;
        for (int i = 0; i < keys; i++) {
            int s = slot("item:" + i);
            if (three[s] != four[s]) movedSlotsOnly++;
        }
        for (int i = 0; i < keys; i++) {
            int h = ("item:" + i).hashCode() & 0x7FFFFFFF;
            if (h % 3 != h % 4) movedModulo++;
        }
        System.out.printf("3 -> 4 nodes, equal contiguous re-split: %d moved (%.1f%%)%n",
            movedSlotsOnly, 100.0 * movedSlotsOnly / keys);
        // minimal migration: each old master hands over only what it holds above 4096 slots
        int[] minimal = three.clone();
        int[] count = new int[4];
        for (int o : three) count[o]++;
        int[] surplus = {count[0] - 4096, count[1] - 4096, count[2] - 4096};
        for (int s = 0; s < 16384; s++) {
            int o = three[s];
            if (surplus[o] > 0) { surplus[o]--; minimal[s] = 3; count[o]--; count[3]++; }
        }
        long movedMin = 0;
        for (int i = 0; i < keys; i++) {
            int s = slot("item:" + i);
            if (three[s] != minimal[s]) movedMin++;
        }
        System.out.printf("3 -> 4 nodes, only surplus slots: %d moved (%.1f%%), per node %s%n",
            movedMin, 100.0 * movedMin / keys, java.util.Arrays.toString(count));
        System.out.printf("3 -> 4 nodes with hash(key) mod N: %d moved (%.1f%%)%n",
            movedModulo, 100.0 * movedModulo / keys);
    }

    static int firstSlot(int[] o, int n) {
        for (int s = 0; s < o.length; s++) if (o[s] == n) return s;
        return -1;
```

ClusterSlots.java (continued) JAVA

```
    }

    static int lastSlot(int[] o, int n) {
        for (int s = o.length - 1; s >= 0; s--) if (o[s] == n) return s;
        return -1;
    }
}
```

Run ClusterSlots (JDK 21) TERMINAL

```
$ java ClusterSlots.java
crc16("123456789") = 0x31C3 (XMODEM check value is 0x31C3)
slot("foo"               ) = 12182
slot("bar"               ) = 5061
slot("user:1000"         ) = 1649
slot("{user:1000}.cart"  ) = 1649
slot("{user:1000}.orders") = 1649
slot("foo{}{bar}"        ) = 8363
3 masters: node0 owns slots 0..5461, node1 owns 5462..10922, node2 owns 10923..16383
3 -> 4 nodes, equal contiguous re-split: 500090 moved (50.0%)
3 -> 4 nodes, only surplus slots: 249975 moved (25.0%), per node [4096, 4096, 4096, 4096]
3 -> 4 nodes with hash(key) mod N: 750010 moved (75.0%)
```

What to notice. The CRC16 of the string `123456789` is the XMODEM check value `0x31C3`, which confirms the implementation, and the keys `foo` and `bar` map to slots 12182 and 5061. Two keys with the same hash tag share a slot, and a key with empty braces is hashed as a whole. Re-splitting the slots into four equal contiguous ranges moves 50% of the keys, because ranges shift; moving only the surplus slots (1,366 from each of A, B and C) moves 25%, which is the theoretical minimum for going from 3 to 4 nodes; hashing with `mod N` moves 75%. The conclusion is that a layer of slots between keys and nodes makes rebalancing a matter of choosing which slots move, independent of the hash function.

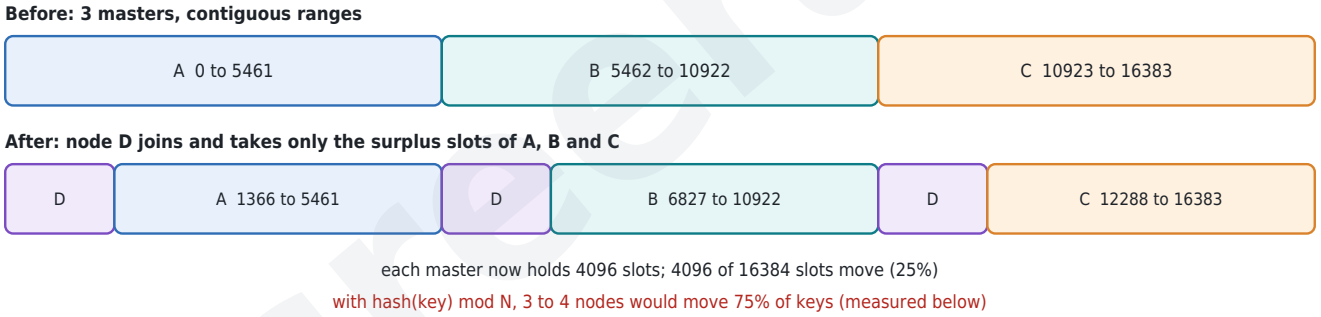


Figure 2.19. Moving only the surplus slots of each master to the new node transfers one quarter of the data; a naive modulo scheme would transfer three quarters.

2.6.5 High-level architecture

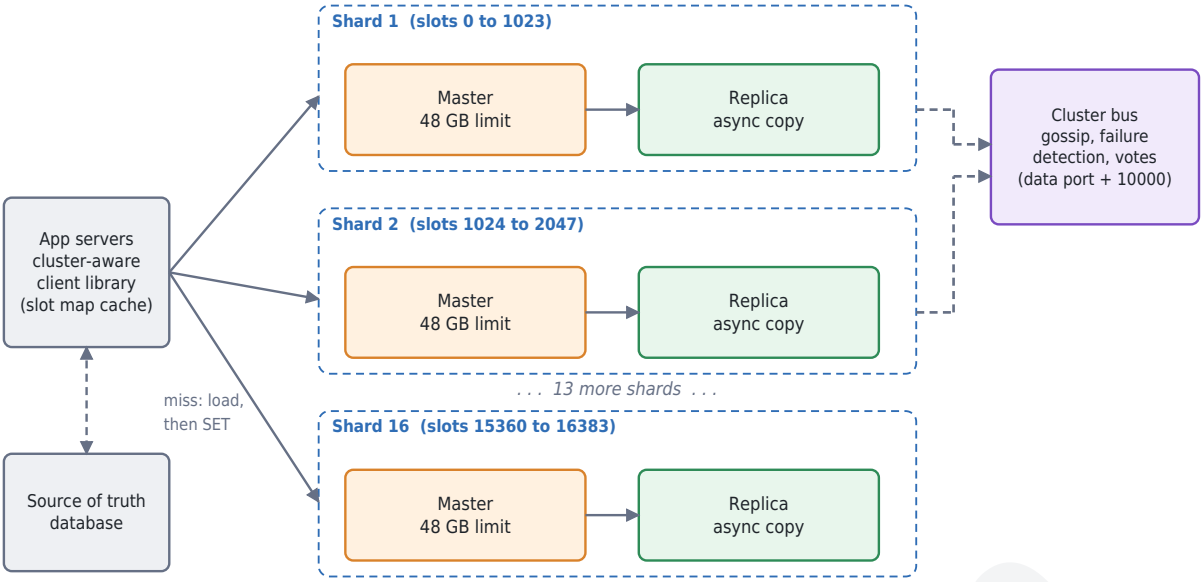


Figure 2.20. Application servers route each command directly to the master that owns the slot; each master has a replica, and all nodes are connected by a gossip bus that carries failure detection and votes.

Request flow. (1) The client library computes the slot and finds the owner in its slot map. (2) It sends the command to the master. (3) The master executes it on its single command thread and replies; for writes it also streams the command to its replica asynchronously. (4) A read may be sent to a replica if the client declares that stale reads are acceptable, which spreads read load. (5) On a miss the application loads the value from the source of truth and sets it, as in the cache-aside pattern.

Membership. Every node keeps a TCP connection to every other node on the cluster bus (the data port plus 10000) and exchanges gossip messages carrying its view of slot ownership and node health. A configuration epoch, a counter that increases on every ownership change, resolves conflicts: the higher epoch wins.

Replication. A master streams its write commands to its replicas asynchronously. A write is acknowledged to the client before the replicas have it, which keeps latency low and means that a master failure can lose the most recent writes.

2.6.6 Deep dive

Problem 1: partitioning scheme

Scheme	How it works	Problems	Verdict
Hash mod N	node = hash(key) mod N	Adding a node moves 75% of keys (3 to 4 nodes, measured above)	Only for fixed-size clusters
Consistent hashing ring on the client	Keys and nodes hashed onto a ring	Every client library must agree on the ring; no authority; uneven load without virtual nodes	Acceptable for small caches
Proxy tier	Clients talk to a proxy that routes	Extra hop and extra tier to operate	When clients cannot be cluster-aware
Fixed hash slots with an owner table	16,384 slots mapped to masters; the table is gossiped	Fixed upper bound on shard count (16,384)	Chosen

Partitioning alternatives.

Fixed slots separate two concerns: the hash function decides which slot a key belongs to and never changes, and the ownership table decides where the slot lives and changes freely. The table is small (16,384 entries), so every node and client can hold all of it.

Problem 2: failure detection and failover

A node that does not answer a ping within the node timeout is marked as possibly failing (PFAIL) by the node that noticed. When a majority of masters report the same node as failing within a validity window, the state becomes FAIL and is broadcast. A replica of the failed master then starts an election: it waits a delay of 500 ms plus a random 0 to 500 ms plus 1,000 ms times its rank (replicas with more replicated data have a better rank), asks the masters for votes, and promotes itself when a majority of masters has voted, taking over the slots of the failed master.

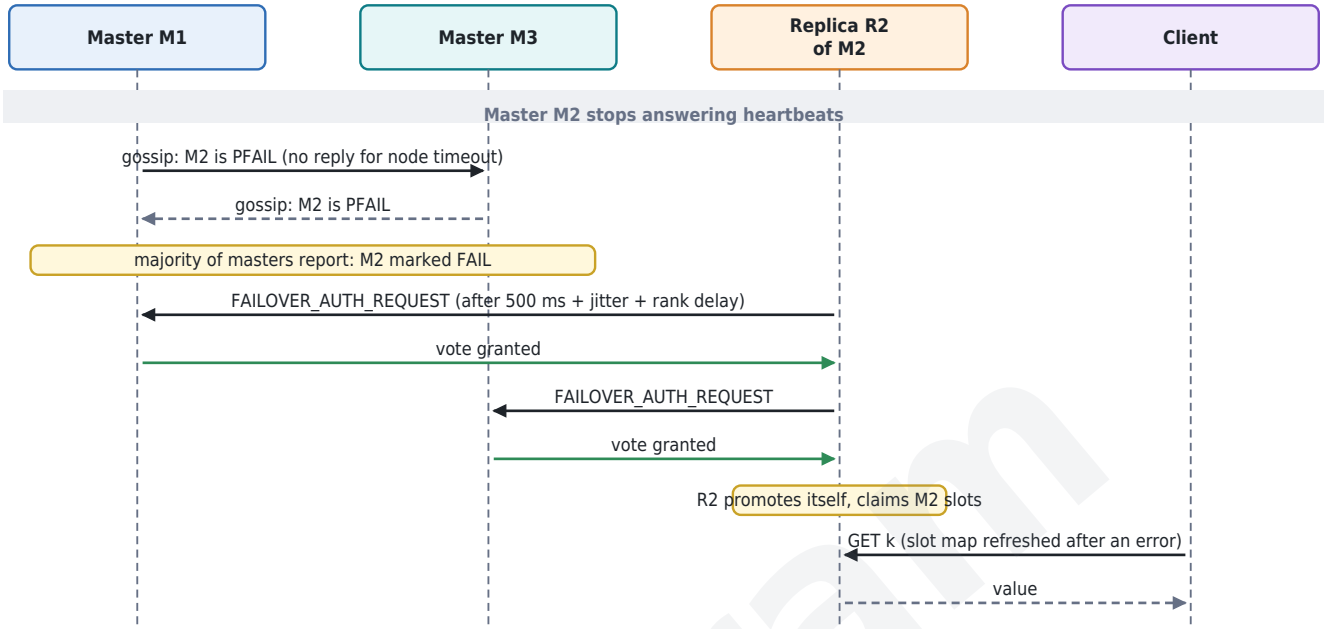


Figure 2.21. Failure is declared by a majority of masters, the replica is elected by a majority of masters, and clients learn the new owner by refreshing their slot map.

Because replication is asynchronous, writes acknowledged by the old master and not yet received by the replica are lost; this is inherent in the design and is acceptable for a cache. The cluster stops serving a slot when no node owns it, if the default option requiring full coverage is enabled; teams that prefer partial availability disable it.

Problem 3: online resharding

To move slot 5100 from node A to node B, A marks the slot as migrating and B marks it as importing. Keys are moved one at a time with an atomic migrate command. While the slot is in transition, a request to A for a key that has already moved receives `-ASK`, and the client sends `ASKING` followed by the command to B; the client does not update its map because the move is not final. When the last key has moved, ownership changes and later requests to A receive `-MOVED`, which does update the client map.

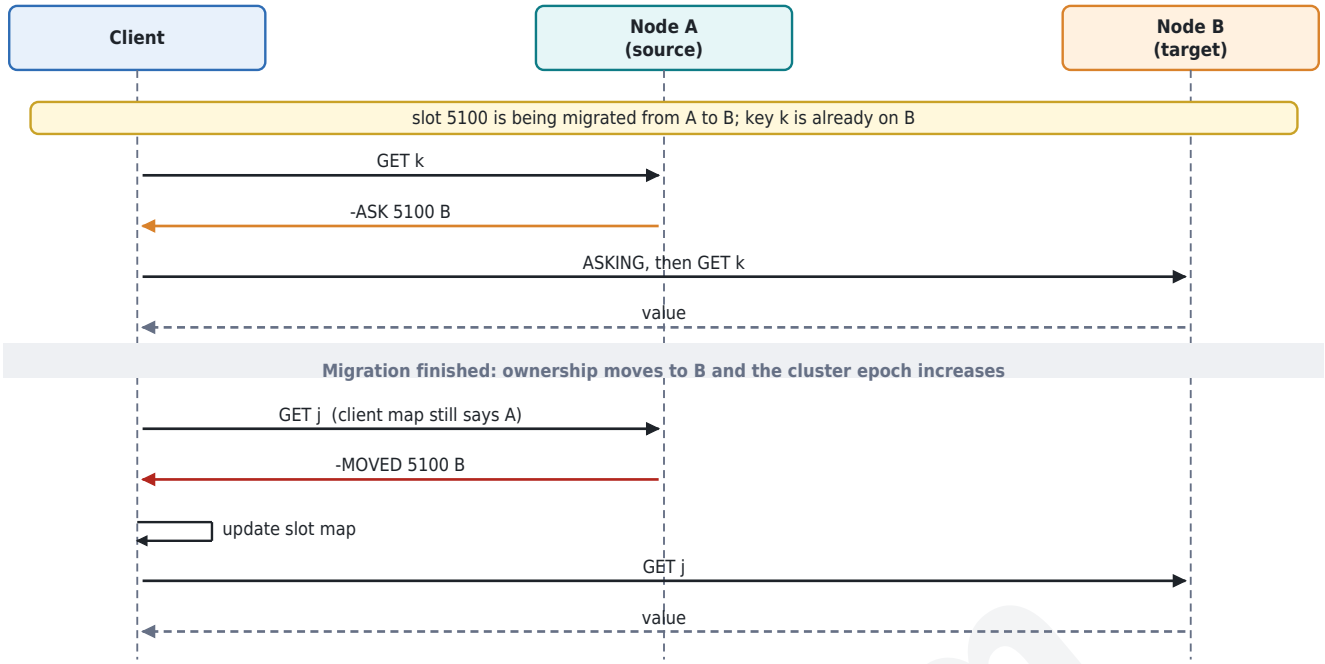


Figure 2.22. During migration the source redirects individual requests with ASK without changing the client's map; after the move completes it answers with MOVED, which updates the map permanently.

2.6.7 Bottlenecks, failures and real-world problems

Hot key saturates a single node

Scenario. A product page goes viral and one key receives 300,000 reads per second. **Symptoms.** One master reaches 100% CPU on its command thread; latency of all keys in its 1,024 slots rises, while the other 15 masters are idle.

Diagnosis. Sample the command stream for the most frequent keys or use the server's hot-key statistics (when LFU eviction is enabled); compare per-node request rates. **Root cause.** A key lives in one slot on one master, so one key cannot use more than one node's capacity (100,000 operations per second assumed), however many nodes exist.

Fix. Add a local in-process cache with a short time to live in the application servers; read the key from replicas; for extreme cases store the value under several keys (`product:7:{0}` to `product:7:{7}`) and read a random copy. **Prevention.** Load-test with skewed key popularity, and monitor the share of traffic held by the top key per node.

A big key blocks the node

Scenario. A leaderboard hash grows to 2 GB, and a job deletes it with `DEL`. **Symptoms.** The node stops answering for several seconds; clients time out; the replica is marked failing and a failover starts.

Diagnosis. The slow log shows one `DEL` taking seconds; memory statistics show a single key dominating. **Root cause.** The command thread frees the memory synchronously, so one deletion of a large value blocks every other command on that node.

Fix. Use `UNLINK` (asynchronous free), split the structure into many small keys, and cap value sizes in the client library. **Prevention.** Scan for big keys regularly, set alerts on key size, and forbid unbounded collections.

Writes lost or divergent after a partition

Scenario. A network partition isolates a master with a minority of the cluster; its replica is promoted on the majority side; clients on the minority side keep writing to the old master for 15 seconds. **Symptoms.** After the partition heals the old master becomes a replica and discards its data; writes accepted on the minority side are gone.

Diagnosis. Compare the write acknowledgement logs with the data after the heal; the cluster logs show the epoch change and the demotion. **Root cause.** Asynchronous replication with a "last failover wins" merge: acknowledged writes are not guaranteed to survive, and the minority-side master keeps accepting writes until its node timeout expires.

Fix. Use a short node timeout and configure masters to stop accepting writes when they have too few healthy replicas (`min-replicas-to-write`), and use the `WAIT` command for the few writes that need replica acknowledgement. Treat the cache as lossy. **Prevention.** Keep the source of truth outside the cache, and design reads to repopulate from it.

Memory spike during background save

Scenario. A node with 40 GB of data starts a background snapshot while heavy writes continue. **Symptoms.** The operating system's memory usage climbs towards the physical limit and the out-of-memory killer terminates the process.

Diagnosis. Resident memory of the parent and child during the snapshot; the copy-on-write page count. **Root cause.** The snapshot child shares pages with the parent; every page the parent modifies is copied, so heavy writes can approach double memory.

Fix. Set `maxmemory` well below physical memory (48 GB of 64 GB in the estimate), take snapshots on replicas, and disable persistence on cache nodes where it is not needed. **Prevention.** Alert on the ratio of resident memory to `maxmemory` and test snapshots under write load.

2.6.8 Trade-offs and extensions

- **Eviction policy.** An approximated least-recently-used policy samples a few keys per eviction and costs less memory than an exact list; a least-frequently-used policy protects frequently read keys from one-time scans. The default policy of the server refuses writes when memory is full, which is rarely what a cache needs, so a cache cluster sets an eviction policy explicitly.
- **Replicas for reads.** Reading from replicas raises read capacity and accepts stale reads; the replication lag is usually milliseconds but unbounded during incidents.
- **Persistence.** A cache that must survive restarts without a cold start can write snapshots on replicas; the cost is memory headroom and disk bandwidth.
- **Proxy versus smart client.** A smart client avoids a hop and is a dependency in every language used; a proxy centralises routing and connection pooling.
- **Multi-region.** A cache per region, filled from the local database, avoids cross-region latency; invalidation across regions is handled by events, not by cache replication.

2.6.9 Questions to expect

Why 16,384 slots and not more?

The gossip messages carry the slots owned by a node as a bitmap, and a bitmap of 16,384 slots occupies 2 KB (16,384 bits), which keeps the periodic messages small; the cluster specification also suggests a practical maximum of about 1,000 nodes, far below the slot count. The number is a fixed design constant, so the number of masters is bounded by it. The rationale given here is an inference from these two facts and not a statement of the original designers.

How does a client find the right node?

It keeps a slot map from the cluster, computes `CRC16(key) mod 16384` itself and connects to the owner. When the topology changes, a node answers `-MOVED` and the client refreshes the map.

What happens to keys when a node is added?

Selected slots migrate to the new node key by key; clients get `-ASK` for keys already moved while the slot is migrating, and `-MOVED` once ownership changes. The surplus-slot approach moves one quarter of the keys going from three to four nodes.

Is the cache consistent?

Per key, commands are serialised on one node, so single-key operations are atomic. Replication is asynchronous, so replicas can be stale and a failover can lose recent writes. Cross-key consistency is not provided across slots.

How is a hot key handled?

Add a local cache in the application, read from replicas, or split the key into several copies. Detect hot keys by sampling and treat them as a capacity problem of one node, not of the cluster.

When is a distributed cache the wrong answer?

When the data must not be lost, when operations need multi-key transactions across the key space, or when the working set fits on one node with replicas; a single replicated node is simpler and avoids slot and redirection logic.

sreeram

How are cache stampedes handled at this layer?

The infrastructure helps with request coalescing in the client library and with jittered time to live values; the pattern-level defences are covered in the chapter on caching.

How many nodes survive a failure?

A shard with one replica survives the loss of either node. Losing both a master and its replica makes their slots unavailable (or, with partial coverage allowed, unserved), so replicas are placed in different availability zones from their masters.

KEY POINTS

- Fixed hash slots (16,384) separate the hash function from node placement; rebalancing means moving slots, one quarter of the data when going from 3 to 4 nodes.
- Clients route directly using a cached slot map; MOVED is permanent, ASK is temporary during migration.
- Replication is asynchronous: failover is fast and may lose the latest writes, which a cache accepts.
- One key lives on one node, so hot keys and big keys are the characteristic failures; use local caches, key splitting and UNLINK.
- Keep memory headroom (maxmemory at 75% of RAM here) for fork copy-on-write and fragmentation.

2.7 Design: Notification System

A notification system delivers messages to users through push notifications, SMS and email on behalf of many producing services: one-time passwords, order updates, security alerts and marketing campaigns. The problem is not sending a message; it is sending the right message exactly once from the user's point of view, through unreliable third-party providers, with the urgent messages never waiting behind the bulk ones, and with respect for the user's preferences. The design is built on a message queue (as covered in the part on asynchronous processing) and uses idempotency, retries with backoff, priority isolation and per-provider rate limiting as its core mechanisms.

2.7.1 Requirements and scope

Functional requirements

- Accept a notification request from internal services (API or event) and deliver it on one or more channels: mobile push, SMS, email.
- Priorities: critical (one-time passwords, security alerts), transactional (order and payment updates), bulk (marketing, digests).
- Templates with localisation; user preferences per channel and type (opt-out, quiet hours).
- Scheduled and campaign notifications (one message to a segment of 20 million users).
- Delivery status tracking: queued, sent, delivered, failed, with provider callbacks.
- No duplicates in normal operation, and retries for transient provider errors.

Non-functional requirements

- Critical messages are delivered within 5 seconds at p99 (assumed), independent of bulk load.
- At-least-once delivery inside the system, effectively once for the user through deduplication.
- High availability of intake: a producer must be able to hand over a message even when providers are down.
- Provider rate limits and cost (SMS is the most expensive channel) are respected.

Out of scope: in-app inbox UI, the content of marketing segmentation, A/B experimentation, and the internals of the providers. **Assumptions**: 100 million notifications per day; third-party providers for each channel; a message broker with partitions.

2.7.2 Capacity estimation

Quantity	Arithmetic	Result
Volume	assumed	100 million per day
Average rate	100,000,000 / 86,400	about 1,160 per second
Campaign burst	20,000,000 messages sent within 1 hour: 20,000,000 / 3,600	about 5,600 per second, on top of the average
Design peak	average 1,160 + campaign 5,600 + headroom, rounded	12,000 per second
Channel split	push 50%, email 40%, SMS 10% (assumed)	peak push 6,000; email 4,800; SMS 1,200 per second
Notification log	100,000,000 x 500 bytes	50 GB per day
Retention 90 days	50 GB x 90	4.5 TB
Queue backlog	a campaign of 20,000,000 x 1 KB enqueued at once	20 GB
Drain time of the backlog	20,000,000 / 5,000 per second (assumed push+email capacity)	about 67 minutes

Capacity estimate for the notification system (inputs assumed).

Two conclusions drive the architecture. The volume is moderate, so the problem is not throughput but isolation: a campaign that takes an hour to drain must not delay a one-time password. And the log is large enough (4.5 TB) to be a partitioned store, not a single database.

2.7.3 API design

REST API

```
POST /v1/notifications
  headers: Idempotency-Key: order-9981-shipped        (required, 24 h dedup window)
  body: { "userId": 42, "type": "order_shipped", "priority": "transactional",
          "channels": ["push", "email"],              (optional, default from preferences)
          "templateId": "order_shipped_v3", "params": { "orderId": 9981 },
          "sendAfter": "2026-10-20T08:00:00Z" }       (optional)
  202 Accepted { "id": "4198649856" }                 accepted, not yet delivered
  200 OK       { "id": "4198649856", "duplicate": true }   same key seen before
  429 Too Many Requests                               producer quota exceeded

GET  /v1/notifications/{id}              status per channel and attempt
POST /v1/campaigns                       { "segmentId", "templateId", "schedule", "ratePerSecond" }
PUT  /v1/users/{id}/preferences          channels, types, quiet hours
POST /v1/callbacks/{provider}            delivery receipts from providers (authenticated)
```

The response `202 Accepted` states honestly that delivery is asynchronous. The idempotency key makes retries by producers harmless: a repeated request returns the original id and does not create a second notification.

2.7.4 Data model

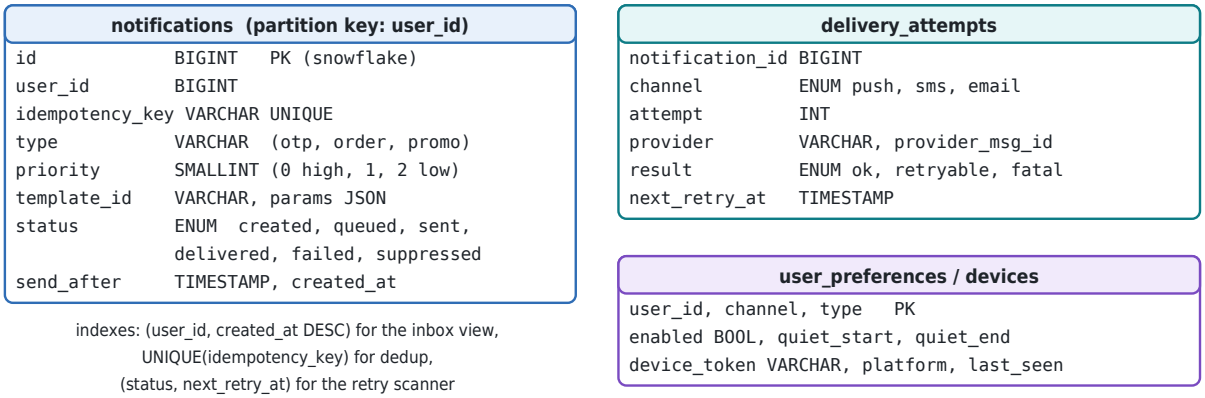


Figure 2.23. The notification log is partitioned by user, attempts are stored per channel, and preferences and device tokens are small tables that the router reads on every message.

The notification log is written once per message and updated as status changes; reads are by id and by user (inbox and support views). A wide-column store or a sharded relational store partitioned by `user_id` fits; the status field is updated by workers and callbacks, so updates must be idempotent and ordered by status rank (a late `sent` never overwrites a `delivered`). The `idempotency_key` has a unique index (or a dedup key in a cache with a 24-hour time to live in front of it) to reject duplicates quickly.

2.7.5 High-level architecture

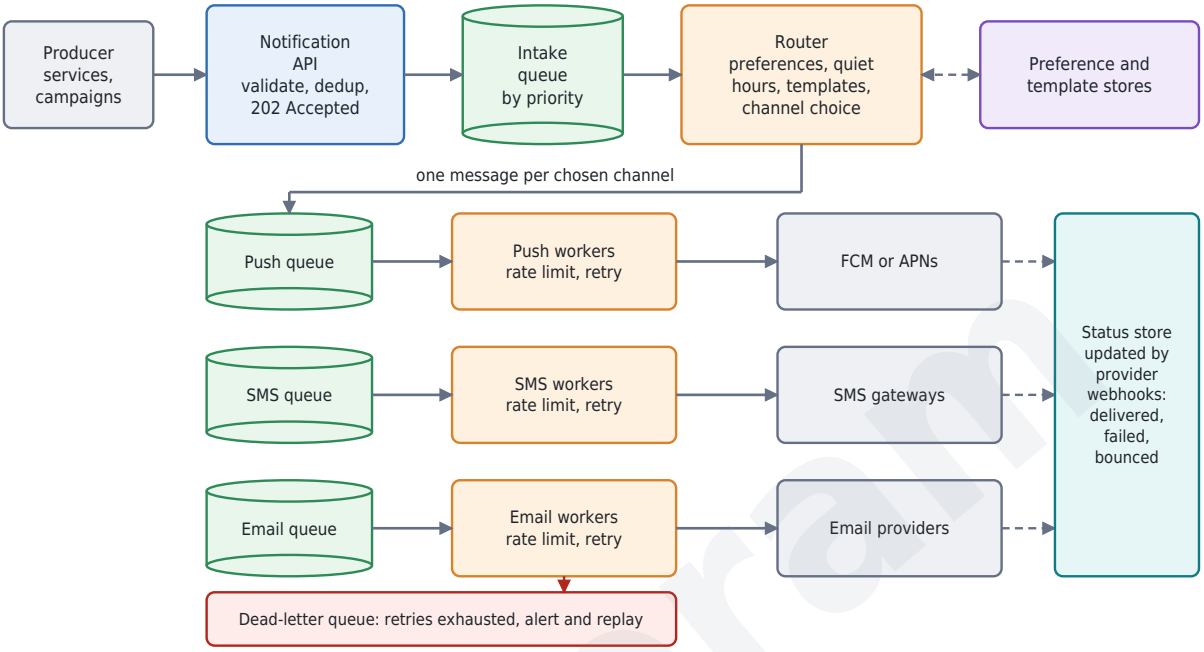


Figure 2.24. Intake is separated from delivery by queues; a router applies preferences and templates and fans out to per-channel queues, where workers enforce provider limits, retry, and send failures to a dead-letter queue.

Intake flow. (1) The API authenticates the producer, validates the request and checks the idempotency key (`SET key NX` with a 24-hour time to live; if the key exists, the existing id is returned). (2) It writes the notification record and publishes it to the intake topic for its priority; only then does it answer `202`. (3) Events from other systems (order created) enter the same way through a consumer that calls the same code path.

Routing. The router consumes the intake topics, loads user preferences and device tokens, suppresses messages the user opted out of, defers bulk messages during quiet hours, renders the template with the parameters in the user's language, and publishes one message per chosen channel to that channel's queue. Preferences are read at send time and not at intake, so an opt-out takes effect even for messages that have waited in a queue.

Delivery. Channel workers consume their queue, take a token from the provider-level rate limiter, call the provider with a request id that the provider can use for its own deduplication, record the attempt, and acknowledge the queue message. Transient errors schedule a retry; fatal errors (invalid address, unregistered device token) mark the message failed without retry and update the device or address record.

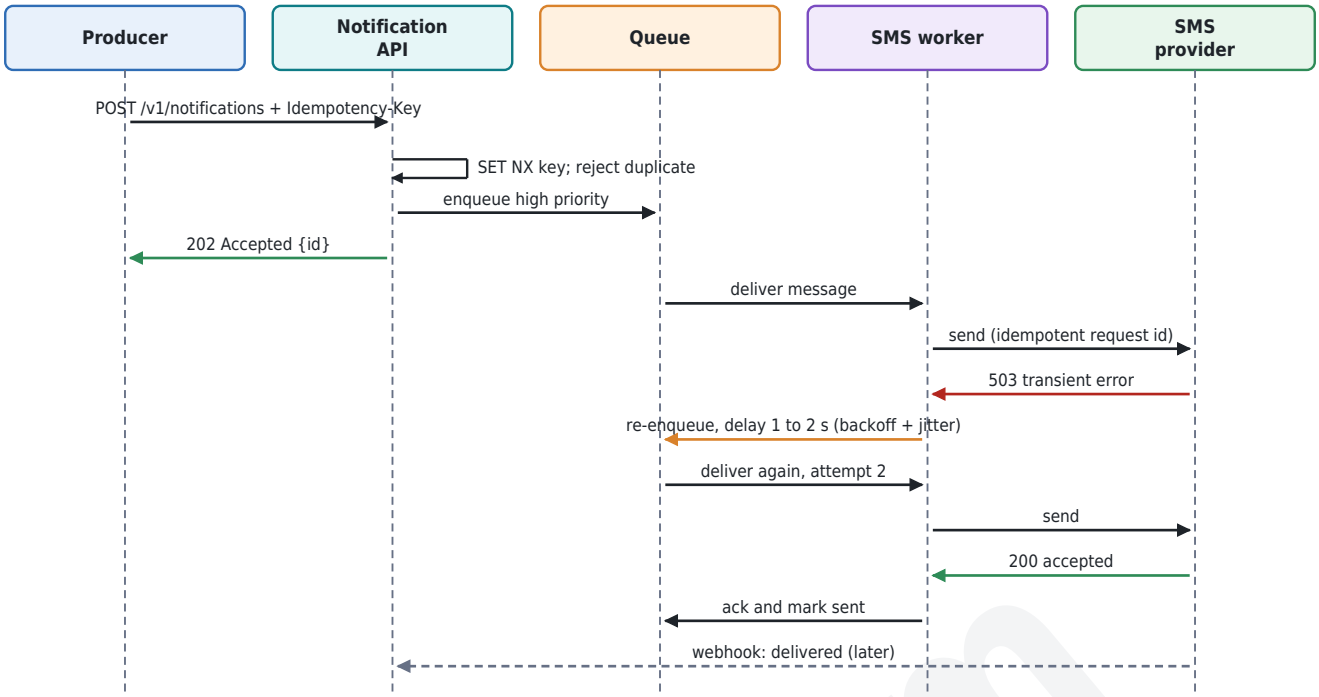


Figure 2.25. A transient provider error is handled by re-enqueueing with a delay; the delivery receipt arrives later through a webhook and updates the status.

2.7.6 Deep dive

Problem 1: priorities without starvation

A single queue is first-in first-out: 20 million campaign messages enqueued at 08:00 delay a password reset requested at 08:01 by more than an hour. Brokers such as Kafka do not provide message priorities within a partition, so priority is implemented by separating the traffic.

Approach	How it works	Problems	Verdict
One queue, FIFO	All messages share a queue	Head-of-line blocking by bulk messages	Not acceptable
Priority queue inside the broker	Broker orders by priority field	Not available in every broker; hard to partition	Only where supported
Separate queues per priority, workers drain high first	Dedicated consumers or a strict-priority poll loop	Low priority can starve under sustained high load	Chosen with a share for low
Separate worker pools per priority	Capacity reserved per class	Idle reserved capacity	When isolation outweighs efficiency

Priority alternatives.

The simulation below sends 2,000 promotional messages at time 0 and 20 one-time passwords during the first 10 seconds, with a provider that accepts 50 messages per second and fails transiently 20% of the time. It also submits 30 duplicate promotional requests (same dedup key) and retries failures with exponential backoff and full jitter up to 5 attempts.

NotifySim.java JAVA

```java
import java.util.*;

public class NotifySim {
    enum Priority { HIGH, NORMAL, LOW }

    static final class Msg {
        final String id, dedupKey; final Priority prio; final int createdAt;
        int readyAt, attempts;
        Msg(String id, String dedupKey, Priority prio, int createdAt) {
            this.id = id; this.dedupKey = dedupKey; this.prio = prio;
            this.createdAt = createdAt; this.readyAt = createdAt;
```

NotifySim.java (continued) JAVA

```
        }
    }

    static final int CAPACITY = 50;             // provider accepts 50 messages per second
    static final double FAIL = 0.20;            // transient provider error rate
    static final int MAX_ATTEMPTS = 5;

    static void run(String mode, boolean priorityAware) {
        Random rnd = new Random(42);
        Comparator<Msg> order = priorityAware
            ? Comparator.<Msg>comparingInt(m -> m.prio.ordinal()).thenComparingInt(m -> m.readyAt)
            : Comparator.<Msg>comparingInt(m -> m.readyAt).thenComparing(m -> m.id);
        PriorityQueue<Msg> ready = new PriorityQueue<>(order);
        List<Msg> waiting = new ArrayList<>();          // retry timers
        Set<String> seen = new HashSet<>();
        List<Integer> highLatency = new ArrayList<>();
        int suppressed = 0, delivered = 0, dead = 0, retries = 0;

        List<Msg> incoming = new ArrayList<>();
        for (int i = 0; i < 2000; i++) {
            incoming.add(new Msg("promo-" + i, "promo:" + i, Priority.LOW, 0));
        }
        for (int s = 0; s < 10; s++)
            for (int i = 0; i < 2; i++)
                incoming.add(new Msg("otp-" + s + "-" + i, "otp:" + s + ":" + i, Priority.HIGH, s));
        for (int i = 0; i < 30; i++)                    // duplicate submissions of existing promos
            incoming.add(new Msg("promo-dup-" + i, "promo:" + i, Priority.LOW, 1));

        int t = 0;
        while (t < 200 && (t < 10 || !ready.isEmpty() || !waiting.isEmpty())) {
            for (Msg m : incoming) {
                if (m.createdAt == t) {
                    if (seen.add(m.dedupKey)) ready.add(m); else suppressed++;
                }
            }
            Iterator<Msg> it = waiting.iterator();
            while (it.hasNext()) {
                Msg m = it.next();
                if (m.readyAt <= t) { it.remove(); ready.add(m); }
            }
            for (int slot = 0; slot < CAPACITY && !ready.isEmpty(); slot++) {
                Msg m = ready.poll();
                m.attempts++;
                if (rnd.nextDouble() < FAIL) {
                    if (m.attempts >= MAX_ATTEMPTS) { dead++; continue; }
                    // exponential backoff with full jitter: uniform in (0, 2^attempts] seconds
                    int backoff = (int) Math.ceil(rnd.nextDouble() * (1 << m.attempts));
                    m.readyAt = t + backoff;
                    waiting.add(m);
                    retries++;
                } else {
                    delivered++;
                    if (m.prio == Priority.HIGH) highLatency.add(t - m.createdAt);
                }
            }
            t++;
        }
        Collections.sort(highLatency);
        System.out.printf("%-26s delivered=%d retries=%d dead=%d duplicates suppressed=%d%n",
            mode, delivered, retries, dead, suppressed);
        System.out.printf("%-26s OTP latency (s): p50=%d max=%d; queue empty at t=%d s%n", "",
            highLatency.get(highLatency.size() / 2), highLatency.get(highLatency.size() - 1), t);
    }

    public static void main(String[] args) {
        System.out.println("2000 promotional + 20 OTP messages + 30 duplicate submissions;");
        System.out.println("provider limit 50/s, 20% transient failures");
        run("single FIFO queue", false);
        run("priority queues (HIGH first)", true);
        double p = Math.pow(FAIL, MAX_ATTEMPTS);
        System.out.printf("probability that one message fails all %d attempts = %.5f%n",
            MAX_ATTEMPTS, p);
    }
}
```

Sheran

Run NotifySim (JDK 21; a simulation with a fixed seed) TERMINAL

```
$ java NotifySim.java
2000 promotional + 20 OTP messages + 30 duplicate submissions;
provider limit 50/s, 20% transient failures
single FIFO queue         delivered=2019 retries=455 dead=1 duplicates suppressed=30
                          OTP latency (s): p50=36 max=48; queue empty at t=68 s
priority queues (HIGH first) delivered=2019 retries=454 dead=1 duplicates suppressed=30
                          OTP latency (s): p50=0 max=1; queue empty at t=69 s
probability that one message fails all 5 attempts = 0.00032
```

What to notice. With a single FIFO queue the one-time passwords wait a median of 36 seconds and a maximum of 48 seconds behind the promotional backlog; with priority ordering the median is 0 and the maximum 1 second, and the total time to drain everything is nearly the same (68 and 69 seconds), so priority changes who waits without costing throughput. The 30 duplicate submissions were suppressed. About 455 retries happened for 2,020 messages, in line with a 20% failure rate, and one message exhausted its 5 attempts and was dead-lettered; the probability of that for a message is 0.2 to the power 5, which is 0.032%, and 2,020 messages at that rate give about 0.65 expected dead letters, so one is a plausible outcome. Strict priority can starve the low class if high-priority traffic is continuous; a weighted share (for example one in ten polls serves a lower class) prevents it.

Problem 2: effectively-once delivery

The system is at-least-once: a worker can crash after calling the provider and before acknowledging the queue message, and the message will be delivered again. Four layers reduce duplicates for the user. The idempotency key rejects duplicate requests at intake. The router keys per-channel messages by `(notification id, channel)` and the channel worker checks the attempt table before sending. The request to the provider carries the same request id on every retry, which providers that support idempotent sends use to discard repeats. And the status update is conditional (`sent` only from `queued`). None of these is perfect: after a crash between the provider call and the attempt record, the worker cannot know whether the provider accepted the message, and one duplicate can occur; for critical messages a rare duplicate is preferable to a loss.

Problem 3: provider failures and rate limits

Providers throttle (HTTP 429 with a retry hint), time out or fail for minutes. The worker therefore uses a token bucket per provider and per credential, whose rate is the provider's contractual limit; exponential backoff with full jitter for retries (the delay is uniform between 0 and 2 to the power of the attempt in seconds, so that retries of many messages do not synchronise); a circuit breaker per provider that stops calls after consecutive failures; and a secondary provider for the channel, used when the breaker of the primary is open. Messages that cannot be sent wait in a delay queue and drain at a limited rate after recovery.

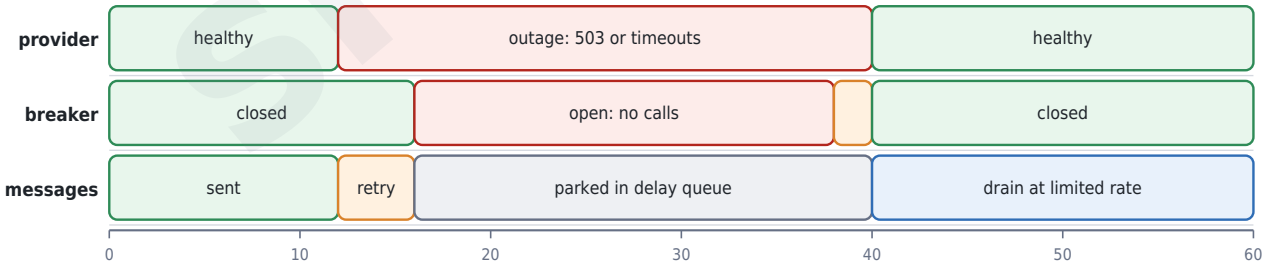


Figure 2.26. During an outage the breaker opens after a few failures, messages are parked instead of hammering the provider, and after a probe succeeds the backlog drains at a limited rate.

Problem 4: campaigns

A campaign to 20 million users is not 20 million API calls. The campaign service reads the segment in pages, creates messages in batches, and enqueues them at the configured rate with the bulk priority, so that the queue does not hold the full set at once. Sending is spread over the user's local time (for example 09:00 in each time zone) when the content allows it, which also lowers the peak. Campaign messages carry a campaign id for per-campaign status counts and a kill switch that stops further enqueueing.

2.7.7 Bottlenecks, failures and real-world problems

Bulk campaign delays critical messages

Scenario. A campaign of 20 million messages starts at 08:00; at 08:05 users request password reset codes. **Symptoms.** Codes arrive after 30 to 60 minutes; support tickets rise; the login failure rate rises.

Diagnosis. Queue lag per topic shows all traffic in one queue; the oldest message age of the critical class equals the campaign backlog. **Root cause.** Critical and bulk messages shared a queue and a worker pool.

Fix. Separate queues and worker capacity per priority, as in the simulation (median critical wait 36 s against 0 s), and cap the campaign's enqueue rate. **Prevention.** Alert on the oldest-message age per priority, with a hard target for critical messages, and load-test with a simultaneous campaign.

Duplicate notifications after a worker crash or producer retry

Scenario. A worker is killed after the provider accepted an SMS and before the queue acknowledgement; separately, a producer retries a request after a timeout. **Symptoms.** Users receive the same message twice; the duplicate rate follows deployments and timeouts.

Diagnosis. Group the attempt table by `(notification id, channel)`; two successful attempts reveal the redelivery, and two notification ids with the same business event reveal the producer retry. **Root cause.** At-least-once delivery without deduplication at each hop.

Fix. Require an idempotency key from producers, deduplicate at intake and at the channel worker, pass a stable request id to providers, and make status updates conditional. **Prevention.** Alert on the duplicate rate, and make graceful shutdown finish in-flight sends before acknowledgement.

Provider outage causes a retry storm

Scenario. The SMS provider returns errors for 25 minutes; every worker retries each message immediately, three times. **Symptoms.** Request rate to the provider triples, the provider returns 429, the account is throttled for longer, and the queue grows.

Diagnosis. Outbound request rate per provider rises while success rate falls; the retry count per message is high. **Root cause.** Retries without backoff, jitter or a circuit breaker amplify a partial failure into a full one.

Fix. Backoff with full jitter, a breaker that opens after consecutive failures (as in the outage diagram), a secondary provider, and a bounded retry budget per message (5 attempts, then dead-letter). **Prevention.** Track the ratio of retries to first attempts and alert when it exceeds a threshold such as 20%.

Invalid device tokens and poison messages

Scenario. After a large number of app uninstalls, 8% of push sends fail with "unregistered"; separately one template with a missing parameter throws an exception during rendering. **Symptoms.** The error rate and cost rise; a consumer partition stops advancing because the same message fails repeatedly.

Diagnosis. Error codes by provider response; a single message id appears again and again in the consumer logs. **Root cause.** Fatal errors were retried like transient ones, and a rendering failure was not isolated from the queue.

Fix. Classify results: remove tokens that the provider reports as invalid, mark fatal failures without retry, and move a message that fails rendering to the dead-letter queue after the retry limit so that the partition continues. **Prevention.** Validate templates against a schema at publish time, and review the dead-letter queue as an operational task.

2.7.8 Trade-offs and extensions

- **Push, SMS or email.** SMS is the most expensive and the most reliable for reaching a phone without an app; push is cheap and fast but depends on the app and the device token; email is cheap and slow. A fallback chain (push first, SMS if undelivered after 60 seconds for critical messages) trades cost for reach.
- **Exactly once.** True exactly-once delivery to a third-party provider is impossible without provider support; the system offers deduplication and accepts rare duplicates for critical messages.
- **Synchronous versus asynchronous API.** A synchronous send gives the producer immediate status but couples its latency to the provider; the asynchronous API with a status endpoint and callbacks is preferred.
- **Ordering.** Messages to one user are not ordered across channels; where order matters (a status update sequence), include a sequence number and let the client display the newest.
- **Compliance.** Opt-outs and regulatory limits on marketing messages are enforced in the router, using the preference value at send time, with an audit record of each decision.

2.7.9 Questions to expect

How do critical messages avoid waiting behind a campaign?

By separating them: distinct queues and worker capacity per priority, workers that poll the high queue first, and a rate cap on bulk enqueueing. The simulation shows a median wait of 0 s instead of 36 s.

How are duplicates prevented?

An idempotency key at intake, a unique `(notification id, channel)` at the channel worker, a stable request id passed to the provider, and conditional status updates. Rare duplicates remain possible after a crash between the provider call and the acknowledgement.

What happens when a provider is down?

Workers retry with exponential backoff and jitter, a circuit breaker stops calls after consecutive failures, a secondary provider takes over for the channel, and parked messages drain at a limited rate after recovery. Messages that exhaust their retries go to the dead-letter queue.

Why exponential backoff with jitter?

Backoff reduces the load on a failing provider, and jitter spreads the retries of many messages over time so that they do not arrive together at the same instants.

How are user preferences respected for queued messages?

Preferences are evaluated by the router at send time, not at intake, and checked again for bulk messages when they leave the delay queue.

How is a campaign to 20 million users sent without overloading anything?

The campaign service pages through the segment and enqueues at a configured rate with bulk priority; the channel workers' rate limiters enforce the provider limit; sending is spread across time zones; a kill switch stops the campaign.

How do delivery receipts work?

Providers call a webhook with their own message id and status; the callback handler authenticates the call, finds the attempt by provider message id and updates the status monotonically. Receipts can be late, duplicated or missing, so status is best effort and the system also times out `sent` messages.

How is the system scaled?

Partition topics by user id so that one user's messages are processed in order by one consumer, scale workers per channel independently, shard the log by user, and add providers per channel for capacity and redundancy.

KEY POINTS

- Separate intake from delivery with queues, and separate priorities with distinct queues and worker capacity so that bulk messages never delay critical ones.
- Delivery is at-least-once; idempotency keys, per-channel dedup, stable provider request ids and conditional status updates give effectively-once behaviour.
- Retry with exponential backoff and full jitter, bound the attempts, use circuit breakers and a secondary provider, and dead-letter what cannot be delivered.
- Evaluate preferences and quiet hours at send time, and classify provider errors into transient and fatal.
- Campaigns are paced, paged and killable; the numbers (5,600 per second for 20 million in an hour) set the capacity.

sreeram

2.8 Design: Web Crawler

A web crawler downloads pages, extracts the links they contain and follows those links to discover more pages. Search engines use crawlers to build their index, and the same machinery serves price monitoring, archiving and data collection. The system looks simple, a queue of URLs and a loop that fetches them, and the difficulty lies in the constraints that a naive loop violates: the crawler must not overload any single web site (politeness), must not fetch the same page twice (deduplication), must not get lost in infinite generated URL spaces (traps), and must keep a very large set of known URLs under control. The design below is a typical one, in the style of the published Mercator-type crawler architecture; the numbers are assumptions.

2.8.1 Requirements and scope

Functional requirements

- Start from a set of seed URLs, fetch pages over HTTP and HTTPS, extract links and add new URLs to the frontier.
- Respect `robots.txt` (disallow rules and crawl delay) and a configurable politeness delay per host.
- Avoid duplicate work: the same URL is fetched once per crawl cycle, and pages with identical or near-identical content are stored once.
- Keep content fresh: recrawl pages in proportion to how often they change and how important they are.
- Store raw pages and emit records (URL, fetch time, status, content hash, links) for downstream indexing.

Non-functional requirements

- Scalability to billions of pages by adding machines.
- Politeness: never more than one open request per host and no more than one request per 2 seconds per host (assumed default).
- Robustness: slow servers, malformed HTML, redirects loops, huge responses and traps must not stall the system.
- Extensibility: new content types (PDF, images) can be added as parsing modules.

Out of scope: ranking and indexing, rendering JavaScript-heavy pages (a headless browser pool is an extension), pages behind a login, and legal review of what may be collected. **Assumptions**: 1 billion pages are fetched per month, 5 billion URLs are known, HTML pages average 100 KB.

2.8.2 Capacity estimation

Quantity	Arithmetic	Result
Fetch rate (average)	1,000,000,000 / (30 x 86,400)	about 386 pages per second
Fetch rate (peak)	386 x 2	about 770 pages per second
Concurrent fetches	Little's law: 386 pages/s x 0.5 s average latency (assumed)	about 193 (about 390 at peak)
Hosts active at once	at 0.5 requests per second per host (2 s delay): 386 / 0.5	at least 772 hosts in rotation
Download bandwidth	386 x 100 KB	about 38.6 MB/s, 309 Mbit/s
Raw storage per month	1,000,000,000 x 100 KB	100 TB (about 33 TB at 3:1 compression)
URL metadata	5,000,000,000 x 100 bytes (hash, timestamps, ETag, change rate)	500 GB
Seen-URL set, exact	5,000,000,000 x 8-byte hash	40 GB
Seen-URL set, Bloom filter	5,000,000,000 x 10 bits	6.25 GB
DNS lookups	386 per second before caching	far fewer with a cache

Capacity estimate for the web crawler (inputs assumed).

The most informative figure is the third: because each host may be contacted only every 2 seconds, sustaining 386 pages per second requires hundreds of different hosts to be ready at all times. The crawler's throughput is bounded by the number of distinct hosts in the frontier, not by network bandwidth, and a frontier dominated by a few large sites runs slowly however many fetchers exist.

streeram

2.8.3 API design

The crawler is an internal pipeline. Its interfaces are small.

Internal interfaces

```
POST /v1/seeds              { "urls": ["https://example.org/"], "priority": "high" }
GET  /v1/crawl/stats        pages per second, queue depth, errors per host

Frontier (RPC):   NextBatch(workerId, max) -> [ {url, hostId} ]      only hosts that are ready
                  Report(url, status, links[], nextFetchHint)
Output record (topic "pages"):
  { "url": "...", "fetchedAt": "...", "status": 200, "contentHash": "ab12...",
    "blobRef": "s3://pages/ab/12/...", "etag": "...", "links": 40 }
```

The frontier hands out URLs in batches belonging to hosts whose politeness delay has elapsed, so fetchers never decide politeness themselves.

2.8.4 Data model

crawler_tables.sql SQL

```sql
CREATE TABLE urls (
    url_hash      BIGINT       PRIMARY KEY,      -- 64-bit hash of the normalised URL
    url           VARCHAR(2048) NOT NULL,
    host_id       INT          NOT NULL,         -- partition key: hash(host)
    depth         SMALLINT     NOT NULL,
    priority      SMALLINT     NOT NULL,
    state         SMALLINT     NOT NULL,         -- discovered, queued, fetched, failed
    last_fetch    TIMESTAMP,
    next_fetch    TIMESTAMP,                     -- recrawl schedule
    etag          VARCHAR(128),
    last_modified VARCHAR(40),
    content_hash  BYTEA,
    change_rate   REAL                           -- estimated changes per day
);
CREATE TABLE hosts (
    host_id        INT PRIMARY KEY,
    hostname       VARCHAR(255) NOT NULL,
    robots_txt     TEXT,
    robots_fetched TIMESTAMP,
    crawl_delay_ms INT       NOT NULL DEFAULT 2000,
    next_allowed   TIMESTAMP,
    error_count    INT       NOT NULL DEFAULT 0
);
```

Partitioning is by host, not by URL: all URLs of one host live in one frontier shard, so the politeness queue of that host is local to one machine and no distributed lock is needed. The cost is that a very large host is limited to one shard's throughput, which matches the politeness limit anyway. Page bodies go to object storage under a key derived from the content hash, which gives exact deduplication of bodies for free.

2.8.5 High-level architecture

preram

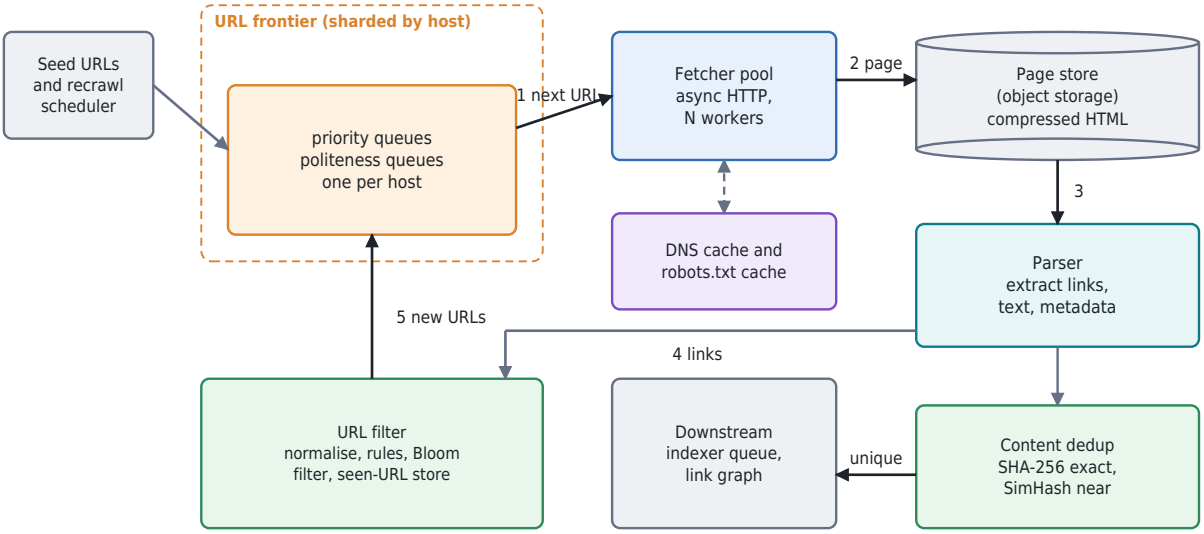


Figure 2.27. URLs flow in a cycle: the frontier releases ready URLs to fetchers, pages go to storage and the parser, new links pass the URL filter and return to the frontier, and unique pages go downstream.

Crawl flow. (1) The frontier releases the next batch of URLs, one per ready host. (2) A fetcher resolves the host through the DNS cache, checks the cached `robots.txt` of the host (fetching it first if it is older than 24 hours, assumed), and downloads the page with a timeout and a size cap. (3) The page body is stored and handed to the parser. (4) The parser extracts links and metadata, and the URL filter normalises each link, applies rules (allowed schemes, blocked patterns, depth limit) and tests the Bloom filter and the seen-URL store. (5) Unseen URLs are added to the frontier with a priority. (6) The content fingerprint is compared with known pages; unique pages go to the downstream queue.

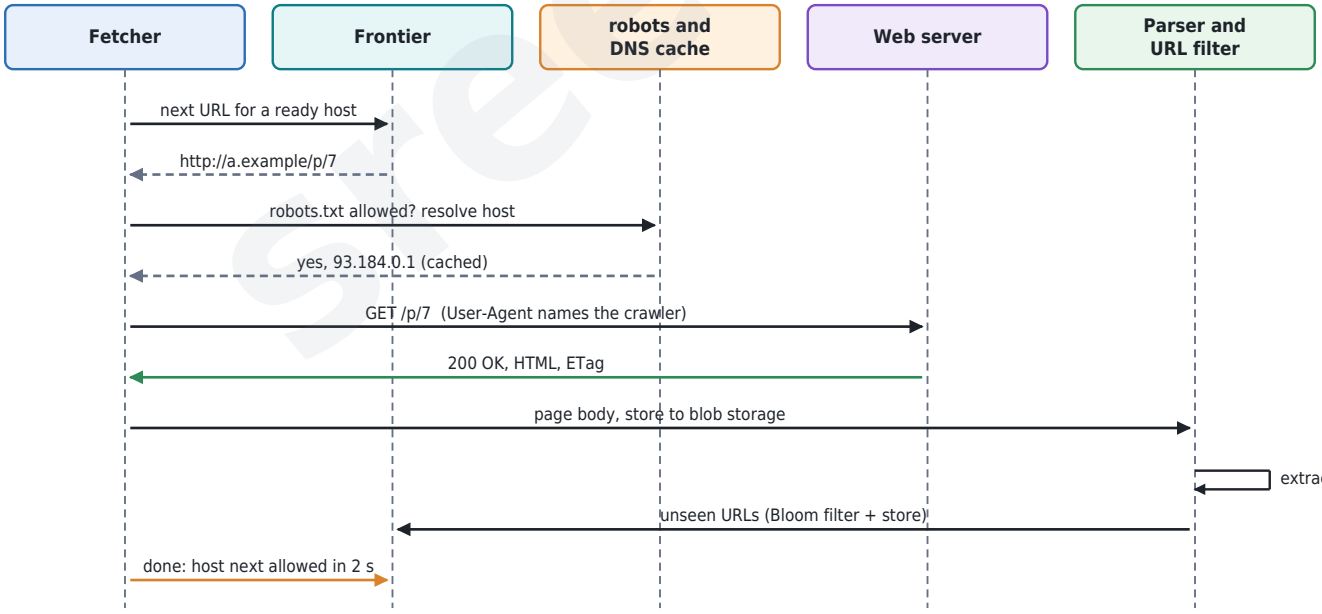


Figure 2.28. The fetcher asks the frontier only for ready hosts, consults the caches before touching the network, and reports completion so that the frontier can schedule the host's next allowed time.

2.8.6 Deep dive

Problem 1: the URL frontier and politeness

The frontier must answer two questions at once: which URL is most valuable (priority), and which host may be contacted now (politeness). The Mercator-style structure separates them. Front queues hold URLs by priority; a router moves them into back queues, one queue per host; a heap holds the hosts ordered by the time at which each may next be contacted; a fetcher takes the host at the top of the heap if its time has come, fetches one URL from that host's queue, and re-inserts the host with `now + delay`.

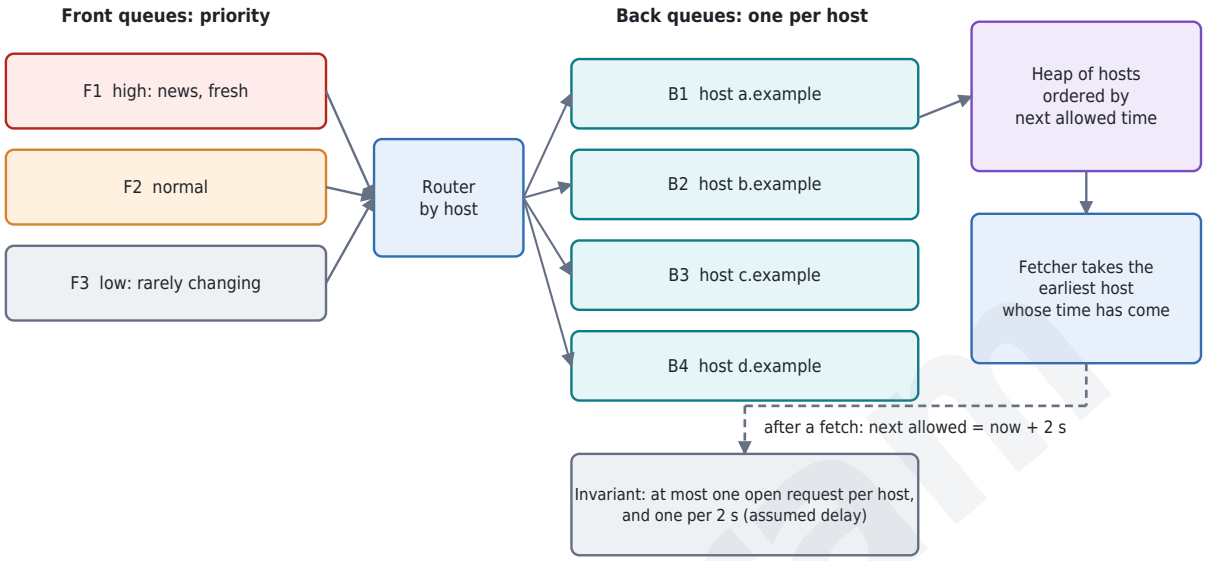


Figure 2.29. Priority is decided before the router, politeness after it: each host has its own queue, and a heap of next-allowed times releases at most one URL per host per delay interval.

The simulation below uses 40 hosts with a skewed number of pages (the largest 1,403 pages, the smallest 36, a Zipf-like distribution), 16 workers, a fetch time of 0.5 s and a politeness delay of 2 s. It compares a global first-in first-out queue without politeness, the same queue with politeness checks, and the per-host structure. The second part measures Bloom filter false positive rates for URL deduplication. It is a simulation of the scheduling logic with synthetic data; no network requests are made.

crawl_sim.py — PYTHON

```
import collections
import hashlib
import heapq
import random

random.seed(11)
HOSTS, DELAY, WORKERS, FETCH, LOOKAHEAD = 40, 2.0, 16, 0.5, 400
# synthetic web: host sizes follow a Zipf-like law, about 6000 pages in total
norm = sum(1 / (j + 1) for j in range(HOSTS))
sizes = [int(1 + 6000 * (1 / (i + 1)) / norm) for i in range(HOSTS)]
pages = [(h, p) for h in range(HOSTS) for p in range(sizes[h])]
print("hosts:", HOSTS, "pages:", len(pages), "largest host:", max(sizes), "smallest:", min(sizes))


def worst_burst(hits):
    worst = 0
    for ts in hits.values():
        ts.sort()
        j = 0
        for i in range(len(ts)):
            while j < len(ts) and ts[j] < ts[i] + 1.0:
                j += 1
            worst = max(worst, j - i)
    return worst


def global_fifo(order, polite):
    """One global queue; a free worker scans at most LOOKAHEAD URLs for an allowed host."""
    queue = collections.deque(order)
```

crawl_sim.py (continued) PYTHON

```
    next_ok = collections.defaultdict(float)
    free = [(0.0, w) for w in range(WORKERS)]
    heapq.heapify(free)
    hits = collections.defaultdict(list)
    now = 0.0
    while queue:
        t, w = heapq.heappop(free)
        now = max(now, t)
        skipped, pick = [], None
        while queue and len(skipped) < LOOKAHEAD:
            item = queue.popleft()
            if not polite or next_ok[item[0]] <= now:
                pick = item
                break
            skipped.append(item)
        queue.extendleft(reversed(skipped))
        if pick is None:
            wake = min(next_ok[h] for h, _ in skipped)
            heapq.heappush(free, (max(now + 0.01, wake), w))
            continue
        next_ok[pick[0]] = now + DELAY
        hits[pick[0]].append(now)
        heapq.heappush(free, (now + FETCH, w))
    return max(t for t, _ in free), worst_burst(hits)


def host_queues():
    """One FIFO per host plus a heap of (next allowed time, host): O(log hosts) per fetch."""
    queues = collections.defaultdict(collections.deque)
    for h, p in pages:
        queues[h].append(p)
    ready = [(0.0, h) for h in queues]
    heapq.heapify(ready)
    free = [(0.0, w) for w in range(WORKERS)]
    heapq.heapify(free)
    hits = collections.defaultdict(list)
    while ready:
        t, w = heapq.heappop(free)
        allowed, h = heapq.heappop(ready)
        start = max(t, allowed)
        queues[h].popleft()
        hits[h].append(start)
        if queues[h]:
            heapq.heappush(ready, (start + DELAY, h))
        heapq.heappush(free, (start + FETCH, w))
    return max(t for t, _ in free), worst_burst(hits)


rows = [("global FIFO, no politeness", global_fifo(sorted(pages), False)),
        ("global FIFO, 2 s politeness, grouped", global_fifo(sorted(pages), True)),
        ("per-host queues, 2 s politeness", host_queues())]
for name, (end, burst) in rows:
    print("%-38s time=%6.0f s  max requests to one host in 1 s: %d" % (name, end, burst))
print("lower bound, largest host alone: (%d - 1) pages x %.0f s = %.0f s"
      % (max(sizes), DELAY, (max(sizes) - 1) * DELAY))


class Bloom:
    def __init__(self, m, k):
        self.m, self.k, self.bits = m, k, bytearray(m // 8 + 1)

    def _idx(self, s):
        d = hashlib.sha256(s.encode()).digest()
        a = int.from_bytes(d[:8], "big")
        b = int.from_bytes(d[8:16], "big") | 1
        return [(a + i * b) % self.m for i in range(self.k)]

    def add(self, s):
        for i in self._idx(s):
            self.bits[i >> 3] |= 1 << (i & 7)

    def __contains__(self, s):
        return all(self.bits[i >> 3] & (1 << (i & 7)) for i in self._idx(s))
```

crawl_sim.py (continued) PYTHON

```python
def url(i):
    return "http://site%d.example/p/%d" % (i % 997, i)


n = 200_000
for bits_per in (8, 10, 16):
    k = round(0.693 * bits_per)
    bf = Bloom(n * bits_per, k)
    for i in range(n):
        bf.add(url(i))
    fp = sum(1 for i in range(n, n + 100_000) if url(i) in bf)
    print("bloom %2d bits/url, k=%2d: %3d KB for %d urls, false positive rate %.3f%%"
          % (bits_per, k, n * bits_per // 8 // 1024, n, fp / 1000.0))
```

Run crawl_sim.py (Python 3; fixed seed, deterministic scheduling) TERMINAL

```
$ python3 crawl_sim.py
hosts: 40 pages: 6018 largest host: 1403 smallest: 36
global FIFO, no politeness            time=   188 s  max requests to one host in 1 s: 32
global FIFO, 2 s politeness, grouped  time=  4368 s  max requests to one host in 1 s: 1
per-host queues, 2 s politeness       time=  2804 s  max requests to one host in 1 s: 1
lower bound, largest host alone: (1403 - 1) pages x 2 s = 2804 s
bloom  8 bits/url, k= 6: 195 KB for 200000 urls, false positive rate 2.179%
bloom 10 bits/url, k= 7: 244 KB for 200000 urls, false positive rate 0.850%
bloom 16 bits/url, k=11: 390 KB for 200000 urls, false positive rate 0.052%
```

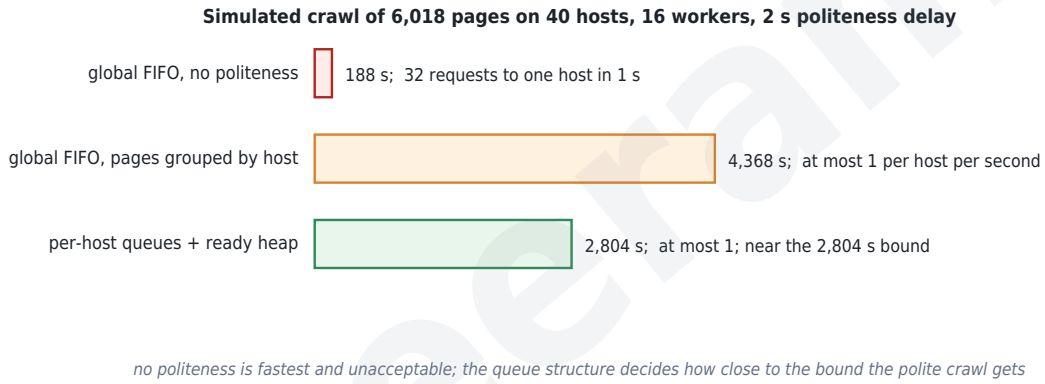


Figure 2.30. Without politeness the crawl is fastest and hits one host 32 times in a second; the per-host structure finishes within a fraction of a second of the theoretical bound set by the largest host.

What to notice. The first row finishes in 188 seconds by sending up to 32 requests to one host within a second, which is the behaviour that gets a crawler blocked. The second row obeys the delay but takes 4,368 seconds, because pages of the same host are adjacent in the global queue, workers scan only a limited window (400 URLs) and often find nothing ready: this is head-of-line blocking. The third row takes 2,804 seconds, the lower bound set by the largest host (1,403 pages at 2 seconds each, 1,402 intervals) plus the final fetch. Once the structure is right, the remaining limit is the largest host, so a real crawler also limits how many pages it takes from one host per cycle (a per-host budget) instead of draining it in order.

Problem 2: avoiding duplicate work

Two kinds of duplication exist. **URL duplication**: the same page is reachable through many spellings (`HTTP://Example.com:80/a?b=1&a=2#top`). Normalisation lowercases the scheme and host, removes default ports and fragments, resolves relative paths and sorts query parameters; known tracking parameters are dropped. After normalisation the URL is hashed to 64 bits and looked up. **Content duplication**: different URLs serve the same or almost the same body (mirrors, session ids in links, print views). An exact digest (SHA-256) catches identical bodies; a locality-sensitive fingerprint such as SimHash (64 bits, with pages treated as near duplicates when the Hamming distance is small, 3 bits being a commonly cited threshold) catches near duplicates.

For 5 billion URLs an exact in-memory set of 8-byte hashes takes 40 GB, which is large but feasible when sharded by host (the frontier shard that owns the host also owns its seen set). A Bloom filter reduces it to 6.25 GB at 10 bits per URL. The measured false positive rates in the run above (0.85% at 10 bits, 2.18% at 8 bits, 0.052% at 16 bits per URL) mean that about one URL in a hundred that is new would be wrongly treated as already seen and skipped. For a crawler that visits the same site through many links this is usually acceptable, since the page may be reached by another link, and the filter is paired with the exact store to confirm positives when precision matters.

Problem 3: freshness and recrawl

A page that changes hourly and a page that never changes should not be recrawled at the same rate. The scheduler estimates a change rate from the history of content hashes, and sets the next fetch time from it, constrained by importance (for example the number of inbound links) and by the budget of fetches per cycle. Conditional requests (`If-None-Match` with the stored `ETag`, `If-Modified-Since`) let the server answer `304 Not Modified` without a body, which saves bandwidth for unchanged pages.

Policy	How it works	Weakness	Verdict
Uniform	Every page every N days	Wastes capacity on static pages; stale news	Baseline only
Change-rate based	Interval follows estimated change rate	Needs history; new pages have none	Chosen with defaults for new pages
Importance based	Higher priority for pages with many inbound links	Ignores change	Combined with the change rate
Event or sitemap driven	Use sitemaps and update pings	Depends on site cooperation	Additional signal

Recrawl policy alternatives.

2.8.7 Bottlenecks, failures and real-world problems

Spider trap

Scenario. A calendar page links to the next month, indefinitely; a site generates session-id URLs; a relative link loop produces paths such as `/a/b/a/b/a/b`. **Symptoms.** The frontier fills with URLs from one host, depth grows without bound, storage fills with near-identical pages, and no new content is discovered.

Diagnosis. Per-host statistics show a large number of URLs with a repeating pattern, rising depth and a low ratio of new content hashes to fetches. **Root cause.** The crawler treats each distinct URL as new content, and a URL space can be infinite.

Fix. Limit depth and URL length, set a page budget per host per cycle, detect repeating path segments, and stop expanding hosts whose recent fetches produce no new content hashes. **Prevention.** Monitor new-content ratio per host and alert, and keep a per-host kill switch.

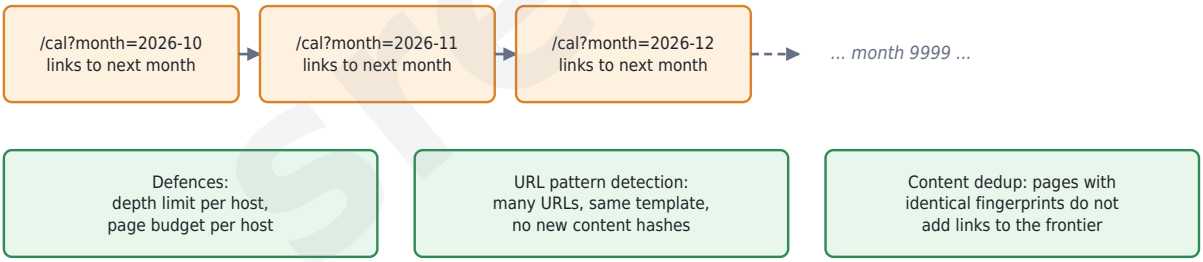


Figure 2.31. A generated URL space has no end; depth limits, per-host budgets and content-hash checks stop the expansion.

Slow servers and DNS stall the fetchers

Scenario. A few hosts respond after 30 seconds or never; DNS resolution for some domains takes seconds. **Symptoms.** Throughput drops although CPU and bandwidth are idle; all workers are busy waiting.

Diagnosis. The distribution of fetch time shows a long tail; the number of workers blocked on DNS or on connect is high. **Root cause.** Fetchers used blocking calls with long or no timeouts, so a slow host consumed a worker for the whole timeout (Little's law: concurrency equals rate times latency, so a 30 s latency needs 60 times more workers than 0.5 s).

Fix. Use asynchronous I/O with a high concurrency, connect and total timeouts of a few seconds, a response size cap, a local caching DNS resolver, and an error counter per host that backs off the host exponentially. **Prevention.** Track fetch latency percentiles per host and exclude hosts with repeated failures from the active rotation for a period.

One huge host dominates

Scenario. A site with 50 million pages is added; its URLs fill the frontier. **Symptoms.** Only that host's queue grows; throughput per host is capped at one request per 2 seconds; the crawl of other hosts is unaffected but the large site would need 100 million seconds.

Diagnosis. Per-host queue depth and the estimated drain time (pages times delay). **Root cause.** Politeness bounds the rate per host, so the time to crawl a host is its page count times the delay, regardless of the number of fetchers.

Fix. Apply a per-cycle budget per host (for example 50,000 pages), prioritise by importance within the host, and negotiate a smaller crawl delay with the site owner where a legitimate need exists. **Prevention.** Show the estimated drain time per host at scheduling time, and have the planner choose priorities before queues grow.

Blocked or rate-limited by sites

Scenario. A site returns 403 or 429 to the crawler, or adds the crawler's addresses to a deny list. **Symptoms.** Error rates for the host rise to 100%; other hosts on the same network may follow.

Diagnosis. The status code distribution per host; `Retry-After` headers; complaints from site owners. **Root cause.** The crawler exceeded the site's tolerance, ignored `robots.txt`, or did not identify itself.

Fix. Honour `robots.txt` and `Retry-After`, send a descriptive `User-Agent` with a contact URL, back off exponentially on 429 and 5xx, and honour the host's crawl delay. **Prevention.** Keep a conservative default delay, publish a page that describes the crawler, and automatically pause hosts that show rising error rates.

2.8.8 Trade-offs and extensions

- **Breadth-first versus priority order.** Breadth-first finds popular pages early but ignores value; a priority frontier follows importance and freshness at the cost of complexity.
- **Centralised versus distributed frontier.** Partitioning the frontier by host hash keeps politeness local; links found by one machine for a host owned by another are forwarded to the owner.
- **JavaScript rendering.** A headless browser pool fetches pages that build content in the browser; it is 10 to 100 times more expensive per page (an assumed order of magnitude) and is used selectively.
- **Storage.** Keep the latest version in object storage and older versions only for selected pages; deduplicate bodies by hash.
- **Legal and ethical limits.** Respect `robots.txt`, copyright and terms of use; the crawler's behaviour is a policy decision as well as a technical one.

2.8.9 Questions to expect

How does the crawler stay polite?

One queue per host, a heap of next-allowed times, at most one open request per host and one request per delay interval, `robots.txt` rules and crawl delay honoured, and exponential back-off after errors.

How are duplicate URLs detected at 5 billion URLs?

Normalise, hash to 64 bits, and test a Bloom filter backed by an exact store sharded by host. The Bloom filter needs 6.25 GB at 10 bits per URL, against 40 GB for an exact set of 8-byte hashes, at a false positive rate of about 1%.

How are near-duplicate pages found?

By a locality-sensitive fingerprint such as SimHash: similar documents produce fingerprints with a small Hamming distance. Exact duplicates are found by a cryptographic digest of the body.

How is a spider trap avoided?

Depth and URL length limits, a per-host page budget, detection of repeated path patterns, and stopping hosts that produce no new content hashes.

What limits the crawl rate?

Bandwidth is rarely the limit (309 Mbit/s here). The number of distinct hosts ready at any time is: at one request per 2 seconds per host, 386 pages per second need at least 772 hosts, and the largest host sets the minimum duration of the cycle.

How is freshness maintained?

Estimate each page's change rate from its content hash history, schedule recrawls accordingly, combine with importance, and use conditional requests so that unchanged pages cost no body transfer.

How does the design scale out?

Shard the frontier and the seen-URL set by host hash, add fetchers per shard, and forward discovered links to the owning shard. Page storage scales independently as object storage.

What happens when a frontier machine fails?

Its hosts are reassigned to other shards using the persisted frontier state (the queues are checkpointed or rebuilt from the URL table); fetches in progress are repeated, which is safe because fetching is idempotent and storage is keyed by content hash.

KEY POINTS

- Politeness makes the host, not the URL, the unit of scheduling: per-host queues plus a heap of next-allowed times, sharded by host.
- Throughput needs many distinct hosts in rotation (772 for 386 pages per second at a 2 second delay); the largest host sets the minimum cycle time.
- Normalise URLs, deduplicate with a Bloom filter backed by an exact store (about 1% false positives at 10 bits per URL), and fingerprint content for near duplicates.
- Defend against traps with depth limits, per-host budgets and a new-content ratio; defend against slow servers with async I/O, timeouts and a DNS cache.
- Recrawl by estimated change rate and importance, using conditional requests to save bandwidth.

2.9 Design: Distributed Task Scheduler and Job Queue

A task scheduler runs work at a requested time or on a recurring schedule, across many worker machines, and keeps doing so when workers crash. Typical jobs are sending a reminder at 09:00, expiring a reservation after 15 minutes, generating a nightly report, retrying a failed payment callback or resizing an uploaded image. Two problems are combined in one system: a time problem (find the jobs that are due, among tens of millions, within a second) and a reliability problem (never lose a job, and avoid running it twice when a worker disappears mid-way). The design below uses a partitioned job store, leases with fencing tokens and idempotent handlers; the lease mechanics are demonstrated in simulations and the claim query is run against a real PostgreSQL 16 instance.

2.9.1 Requirements and scope

Functional requirements

- Submit a job with a handler name, a payload, a due time (immediately, delayed, or at an absolute time), a priority and a retry policy.
- Recurring schedules in cron syntax with a time zone; the scheduler creates the runs.
- Workers execute jobs and report success or failure; failed jobs are retried with backoff up to a limit, then marked dead.
- Cancel a job, query its status and history, and list a tenant's jobs.

Non-functional requirements

- A due job starts within 1 second of its due time at p99 under normal load (assumed).
- At-least-once execution: a job is never lost, and a job whose worker crashes is run again; handlers are idempotent so that repeated execution has the effect of one.
- Horizontal scalability of submission, dispatch and execution; no single point of failure.
- Fairness between tenants and priorities.

Out of scope: workflow engines with dependency graphs between jobs, distributed batch computation, and long-running jobs that last hours without checkpoints. **Assumptions**: 50 million jobs per day, an average execution time of 2 seconds, a sharded relational database as the job store, and a queue between dispatch and workers.

2.9.2 Capacity estimation

Quantity	Arithmetic	Result
Jobs per day	assumed	50 million
Average rate	50,000,000 / 86,400	about 580 per second
Peak rate	580 x 5 (assumed peak factor)	about 2,900 per second
Concurrent executions (average)	Little's law: 580 x 2 s	about 1,160
Concurrent executions (peak)	2,900 x 2 s	about 5,800
Worker hosts at peak	5,800 / 50 slots per host (assumed)	about 116
Claims per second	peak 2,900 / batch of 500	about 6 claim queries per second
Row size	payload, status, lease fields, indexes	about 1 KB
Future scheduled jobs	100 million x 1 KB	100 GB
Finished jobs kept 7 days	50 million x 7 x 1 KB	350 GB
Recovery delay after a crash	lease 30 s + poll interval 1 s	at most about 31 s
Cron spike	200,000 schedules with `0 * * * *` (assumed) all due at second 0; drain at 2,900 per second	about 69 s

Capacity estimate for the task scheduler (inputs assumed).

The last row shows why recurring schedules need care: the average load is moderate, and the problem is the alignment of due times on round clock values.

2.9.3 API design

REST API

```
POST /v1/jobs
  headers: Idempotency-Key: reminder-order-77-v1
  body: { "type": "send_reminder", "payload": { "orderId": 77 },
          "runAt": "2026-10-20T09:00:00Z",          (omit for immediate)
          "priority": 1, "maxAttempts": 5, "timeoutSeconds": 60 }
  202 Accepted { "id": "4198649856", "status": "SCHEDULED" }
GET    /v1/jobs/{id}                  status, attempts, last error
DELETE /v1/jobs/{id}                  cancel if not yet leased (409 if running)

POST /v1/schedules
  body: { "cron": "0 */15 * * * *", "timezone": "Asia/Kolkata", "type": "sync_inventory",
          "misfire": "skip" }                    skip | run_once | catch_up
Worker protocol:  Lease(workerId, max) -> jobs with fence tokens
                  Heartbeat(jobId, fence)   extends the lease
                  Complete(jobId, fence, result) / Fail(jobId, fence, error, retryable)
```

Every worker call carries the job's **fence token**, a number that increases with each lease of that job. The store accepts `Complete` and `Fail` only for the current token, which is what protects the job from workers that were paused or partitioned and believe they still own it.

2.9.4 Data model

jobs (partition key: id)

```
id              BIGINT PK
tenant_id       BIGINT
type            VARCHAR  handler name
payload         JSON
run_at          TIMESTAMP  when due
priority        SMALLINT
status          SCHEDULED | LEASED | DONE | DEAD
attempts, max_attempts   INT
lease_owner     VARCHAR, lease_until TIMESTAMP
fence           BIGINT   (grows with every lease)
idempotency_key VARCHAR UNIQUE per tenant
```

index (priority, run_at) WHERE status IN (SCHEDULED, LEASED)

schedules (recurring)

```
id           BIGINT PK
cron_expr    VARCHAR  '0 */15 * * * *'
timezone     VARCHAR
next_fire    TIMESTAMP
misfire      ENUM skip | run_once | catch_up
```

job_runs (history)

```
job_id, attempt   PK
worker, started_at, ended_at
result  ENUM ok | retry | fatal, error
```

Figure 2.32. Jobs carry their lease state and fence token; schedules produce job rows; run history is kept separately for diagnosis.

The core index is `(priority, run_at)` restricted to jobs that are not finished, so that the question "which jobs are due now?" is answered by reading the front of an index and not by scanning the table. The following script creates the table and fills it with 20,000 due jobs and 20,000 jobs due in an hour; it was run on PostgreSQL 16.

jobs_setup.sql — SQL

```sql
DROP TABLE IF EXISTS jobs;
CREATE TABLE jobs (
    id          bigserial   PRIMARY KEY,
    priority    smallint    NOT NULL DEFAULT 1,
    run_at      timestamptz NOT NULL,
    status      text        NOT NULL DEFAULT 'SCHEDULED',   -- SCHEDULED, LEASED, DONE
    lease_owner text,
    lease_until timestamptz,
    attempts    int         NOT NULL DEFAULT 0
);
CREATE INDEX jobs_due ON jobs (priority, run_at) WHERE status IN ('SCHEDULED', 'LEASED');
-- 20,000 jobs that are due and 20,000 that run in an hour
INSERT INTO jobs (run_at, priority)
SELECT now() - random() * interval '10 minutes', (random() * 2)::int
FROM generate_series(1, 20000);
INSERT INTO jobs (run_at) SELECT now() + interval '1 hour' FROM generate_series(1, 20000);
ANALYZE jobs;
```

2.9.5 High-level architecture

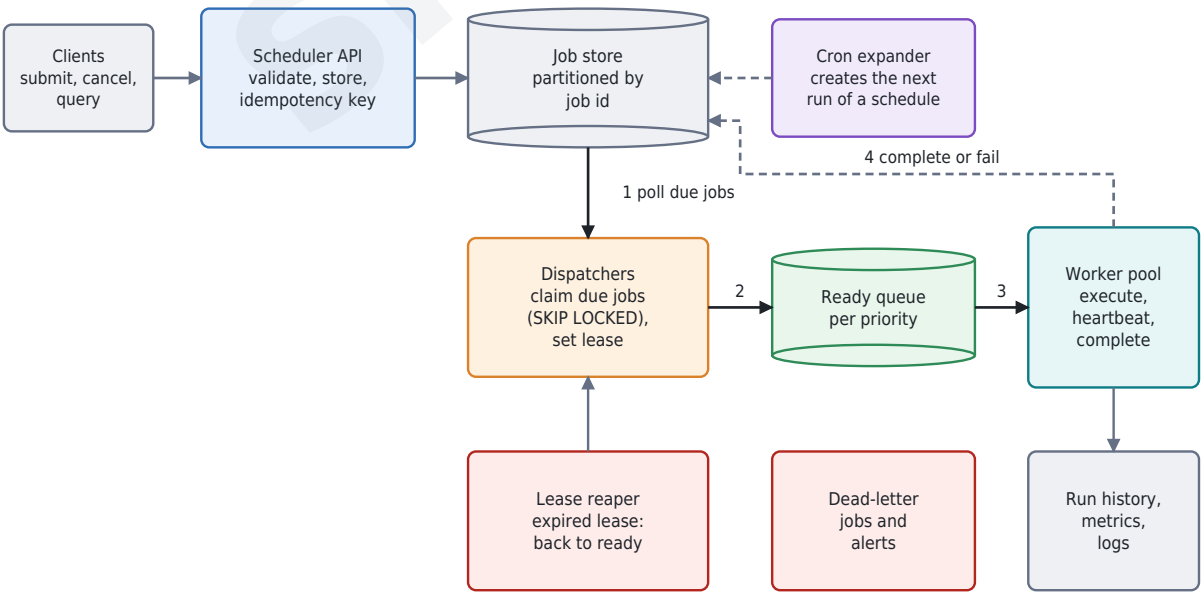


Figure 2.33. The API stores jobs; dispatchers claim due jobs and hand them to workers; workers complete or fail them with their fence token; a reaper returns jobs with expired leases; a cron expander creates the next run of each schedule.

Sreeram

Submit flow. (1) The API validates the request, rejects a repeated `Idempotency-Key` by returning the stored job, and inserts the row in status `SCHEDULED`. (2) A job with `run_at` in the past or `null` is immediately due.

Dispatch flow. (1) A dispatcher polls its partitions each 200 milliseconds (assumed) with a claim query: it selects a batch of due jobs, locks them with `FOR UPDATE SKIP LOCKED` so that concurrent dispatchers skip rows already locked by others, and updates them to `LEASED` with an owner, a lease deadline and a new fence token, in one statement. (2) The claimed jobs are pushed to the ready queue for the worker pool, or handed to workers directly. (3) The worker executes the handler, extends the lease with heartbeats, and finishes with `Complete` or `Fail`.

Failure flow. If a worker stops heartbeating, the lease deadline passes; the next claim query treats a `LEASED` job whose deadline has passed as due, increments the attempt and issues a new fence token.

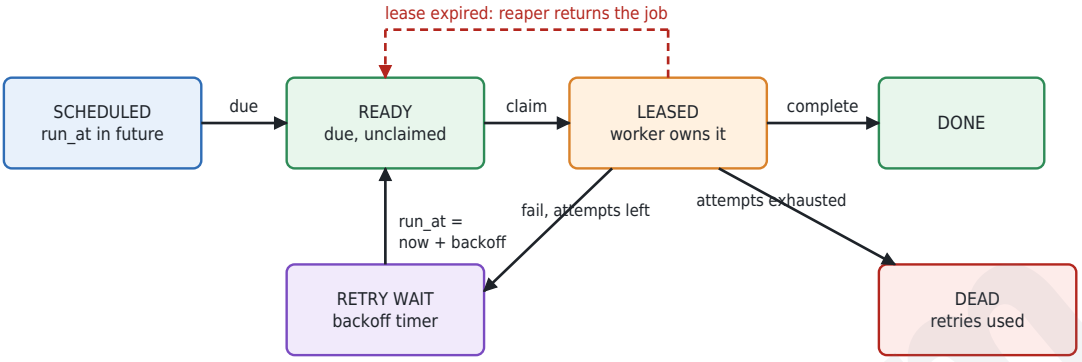


Figure 2.34. A job moves from SCHEDULED to READY when it is due, to LEASED when claimed, and ends in DONE, in a backoff wait, or in DEAD; an expired lease returns it to READY.

The claim statement is the heart of the dispatcher. Four sessions claimed 500 jobs each at the same time, holding their row locks for one second to force the overlap:

claim.sql SQL

```sql
BEGIN;
WITH due AS (
    SELECT id FROM jobs
    WHERE status = 'SCHEDULED' AND run_at <= now()
    ORDER BY priority, run_at
    LIMIT 500
    FOR UPDATE SKIP LOCKED
)
UPDATE jobs j
SET status = 'LEASED', lease_owner = :'owner',
    lease_until = now() + interval '30 seconds', attempts = j.attempts + 1
FROM due WHERE j.id = due.id;
SELECT pg_sleep(1);   -- keep the row locks for a second so that the sessions overlap
COMMIT;
```

Run four concurrent claims on PostgreSQL 16 (own instance on this machine) TERMINAL

```
$ for w in w1 w2 w3 w4; do psql -d jobsdb -v owner=$w -f claim.sql & done; wait
$ psql -d jobsdb -c "SELECT lease_owner, count(*) AS claimed, min(id), max(id) \
    FROM jobs WHERE status='LEASED' GROUP BY 1 ORDER BY 1"
 lease_owner | claimed | min |  max
-------------+---------+-----+-------
 w1          |     500 |  52 | 19982
 w2          |     500 |   4 | 19984
 w3          |     500 |  80 | 19980
 w4          |     500 |  74 | 19989
(4 rows)

$ psql -d jobsdb -c "SELECT count(*) AS total_leased, count(DISTINCT id) AS distinct_jobs, \
    max(attempts) AS max_attempts FROM jobs WHERE status='LEASED'"
 total_leased | distinct_jobs | max_attempts
--------------+---------------+--------------
         2000 |          2000 |            1
(1 row)

$ psql -d jobsdb -c "EXPLAIN (COSTS OFF) SELECT id FROM jobs WHERE status = 'SCHEDULED' \
    AND run_at <= now() ORDER BY priority, run_at LIMIT 500 FOR UPDATE SKIP LOCKED"
                     QUERY PLAN
-----------------------------------------------------
```

Run four concurrent claims on PostgreSQL 16 (own instance on this machine) (continued) TERMINAL

```
 Limit
   ->  LockRows
         ->  Index Scan using jobs_due on jobs
               Index Cond: (run_at <= now())
               Filter: (status = 'SCHEDULED'::text)
(5 rows)
```

What to notice. The four sessions together leased 2,000 distinct jobs, each exactly once (`max_attempts` is 1), and the ids interleave, so each session took rows that the others had locked and skipped: no session waited and none was handed the same row. The plan reads the front of the partial index and locks rows as it goes, so the cost of a claim depends on the batch size and not on the 40,000 rows in the table. The commands are shortened (connection options omitted); the output was produced by the script above on the build machine, and the specific ids vary from run to run.

2.9.6 Deep dive

Problem 1: waiting for the due time

Mechanism	How it works	Problems	Verdict
Polling an indexed table	Dispatchers query the due range every few hundred milliseconds	Query load; resolution equals the poll interval	Chosen: durable and simple
In-memory timers or timing wheel	Dispatcher holds upcoming jobs in a hierarchical timing wheel	State lost on crash; must reload from the store	As an optimisation on top of the table
Sorted set in a cache (score = due time)	Pop members whose score is at most now	Durability depends on the cache; two-step claim needs a script	Smaller systems
Broker delay feature	Message visible after a delay	SQS delays are limited to 15 minutes; many brokers have no delay	Short delays only

Alternatives for finding due jobs.

The table is the source of truth, and an in-memory structure may be added for sub-second precision: each dispatcher loads jobs due within the next minute into a timing wheel and fires them at the right millisecond, still claiming them with the same conditional update so that a crash only costs a reload.

Problem 2: executing at least once, effectively once

An exactly-once execution cannot be guaranteed when workers can crash after performing an effect and before reporting it; the guarantee is at least once, and the handler must be idempotent: sending an email checks a sent-marker keyed by the job id; charging a payment passes the job id as the idempotency key to the payment service; updating a row sets a value instead of incrementing it. Leases bound the delay after a crash, and fence tokens prevent a stale worker's completion from overwriting the state set by its successor. The simulation below uses a fake clock: worker `w2` crashes while running `job-2`, worker `w3` is slow on `job-4` and finishes after its lease expired, and the idempotency key suppresses the repeated effect.

SchedSim.java JAVA

```java
import java.util.*;

public class SchedSim {
    enum State { READY, LEASED, DONE }

    static final class Job {
        final String id; final int runAt; State state = State.READY;
        int leaseUntil, attempts; long fence;            // fence: increases on every lease
        Job(String id, int runAt) { this.id = id; this.runAt = runAt; }
    }

    /** Stands in for a table polled with FOR UPDATE SKIP LOCKED; synchronized = atomic. */
    static final class JobStore {
        final Map<String, Job> jobs = new LinkedHashMap<>();
        long fenceCounter = 0; final int leaseSeconds;
        JobStore(int leaseSeconds) { this.leaseSeconds = leaseSeconds; }

        synchronized Job lease(int now) {
            for (Job j : jobs.values()) {
                boolean due = j.state == State.READY && j.runAt <= now;
                boolean expired = j.state == State.LEASED && j.leaseUntil <= now;
```

SchedSim.java (continued) JAVA

```
                if (due || expired) {
                    j.state = State.LEASED; j.leaseUntil = now + leaseSeconds;
                    j.attempts++; j.fence = ++fenceCounter;
                    return j;
                }
            }
            return null;
        }

        synchronized boolean complete(String id, long fence) {   // conditional update
            Job j = jobs.get(id);
            if (j.state != State.LEASED || j.fence != fence) return false;
            j.state = State.DONE;
            return true;
        }
    }

    record Running(Job job, long fence, int finishAt) {}

    public static void main(String[] args) {
        JobStore store = new JobStore(10);
        for (int i = 1; i <= 4; i++) store.jobs.put("job-" + i, new Job("job-" + i, i * 2));
        Set<String> sideEffects = new LinkedHashSet<>();     // idempotent effect keyed by job id
        int duplicateEffects = 0;
        // worker behaviour: w1 healthy (3 s per job); w2 crashes at t=7; w3 is slow (14 s) on job-4
        Map<String, Running> busy = new HashMap<>();
        boolean w2Dead = false;
        for (int t = 0; t <= 40; t++) {
            if (t == 7 && busy.containsKey("w2")) {
                System.out.printf("t=%2d  w2 crashes while running %s%n", t,
                    busy.remove("w2").job().id);
                w2Dead = true;
            }
            for (String w : List.of("w1", "w2", "w3")) {
                Running r = busy.get(w);
                if (r != null && r.finishAt() <= t) {
                    busy.remove(w);
                    boolean first = sideEffects.add(r.job().id);
                    if (!first) duplicateEffects++;
                    boolean ok = store.complete(r.job().id, r.fence());
                    System.out.printf("t=%2d  %s finished %s (fence %d): %s%s%n", t, w,
                        r.job().id, r.fence(), ok ? "accepted" : "REJECTED (stale fence)",
                        first ? "" : "; effect already applied, skipped");
                }
                if (busy.containsKey(w) || (w.equals("w2") && w2Dead)) continue;
                Job j = store.lease(t);
                if (j == null) continue;
                int duration = w.equals("w3") ? 14 : 3;
                busy.put(w, new Running(j, j.fence, t + duration));
                System.out.printf("t=%2d  %s leased %s attempt=%d fence=%d lease until t=%d%n",
                    t, w, j.id, j.attempts, j.fence, j.leaseUntil);
            }
        }
        System.out.println("final states:");
        for (Job j : store.jobs.values()) {
            System.out.println("  " + j.id + " " + j.state + " attempts=" + j.attempts);
        }
        System.out.println("distinct side effects=" + sideEffects.size()
            + ", executions that were repeats=" + duplicateEffects);

        // recurring job: next fire time on a fixed interval, skipping missed runs after downtime
        int interval = 900, last = 3600, now = 3600 + 4 * 900 + 120;
        int next = last + ((now - last) / interval + 1) * interval;
        System.out.println("interval 900 s, last fired at 3600, restart at " + now
            + " -> next fire at " + next
            + " (skipped " + ((next - last) / interval - 1) + ")");
    }
}
```

Run SchedSim (JDK 21; a simulation with a fake clock) TERMINAL

```
$ java SchedSim.java
t= 2  w1 leased job-1 attempt=1 fence=1 lease until t=12
t= 4  w2 leased job-2 attempt=1 fence=2 lease until t=14
t= 5  w1 finished job-1 (fence 1): accepted
t= 6  w1 leased job-3 attempt=1 fence=3 lease until t=16
t= 7  w2 crashes while running job-2
t= 8  w3 leased job-4 attempt=1 fence=4 lease until t=18
t= 9  w1 finished job-3 (fence 3): accepted
t=14  w1 leased job-2 attempt=2 fence=5 lease until t=24
t=17  w1 finished job-2 (fence 5): accepted
t=18  w1 leased job-4 attempt=2 fence=6 lease until t=28
t=21  w1 finished job-4 (fence 6): accepted
t=22  w3 finished job-4 (fence 4): REJECTED (stale fence); effect already applied, skipped
final states:
  job-1 DONE attempts=1
  job-2 DONE attempts=2
  job-3 DONE attempts=1
  job-4 DONE attempts=2
distinct side effects=4, executions that were repeats=1
interval 900 s, last fired at 3600, restart at 7320 -> next fire at 8100 (skipped 4)
```

What to notice. `job-2` was leased by `w2` with fence 2, `w2` crashed at t=7, and at t=14, when the lease expired, `w1` leased it again with fence 5 and completed it (attempt 2). `job-4` was leased by the slow `w3` with fence 4; at t=18 the lease expired and `w1` ran it again with fence 6; when `w3` finally finished at t=22 its completion was rejected because the fence was stale, and the idempotency key showed that the effect had already been applied once (one repeated execution, four distinct effects). The last line shows the schedule rule: with a 900-second interval, last fire at 3,600 and a restart at 7,320, the four runs due at 4,500, 5,400, 6,300 and 7,200 were skipped and the next fire is 8,100.

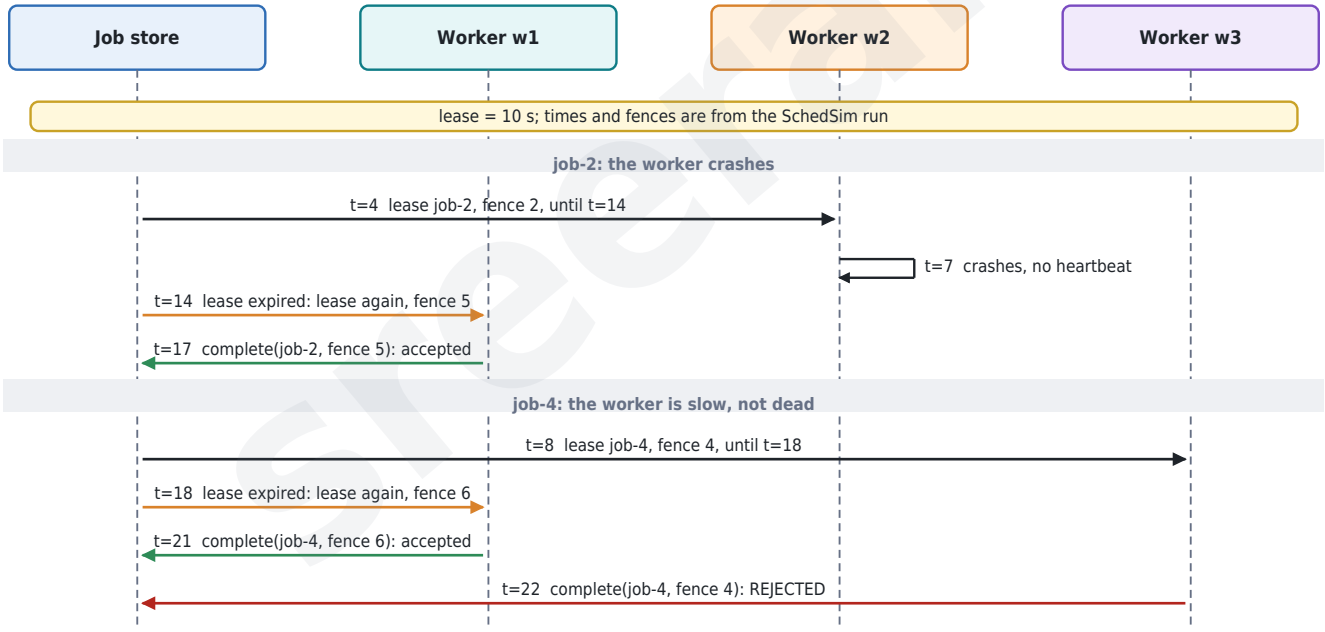


Figure 2.35. A crashed worker is recovered when its lease expires; a slow worker that completes after expiry is rejected by its stale fence token, so the newer attempt is the one that counts.

Problem 3: recurring schedules

The next fire time is computed from the schedule and not from the completion of the previous run, so a slow run does not shift the schedule. The cron expander creates the next job when the current one is created (one job ahead) or on a periodic scan of schedules whose `next_fire` is near. After downtime a **misfire policy** decides what happens to the runs that were missed: `skip` ignores them (for refreshes where only the latest matters), `run_once` runs one catch-up job, `catch_up` runs all of them (for accounting where every period must be processed). Time zones matter for cron in local time: a job at 02:30 does not exist on the day clocks move forward and exists twice on the day they move back, so the policy for those days must be explicit; many schedulers evaluate in UTC to avoid the problem.

Problem 4: scaling the dispatcher

A single dispatcher and one table become the limit at some size: the poll rate and the index size. The job store is partitioned by job id, each partition polled by one dispatcher that owns it through a lease in a coordination service; when a dispatcher fails, its partitions move after the lease period. The conditional claim update, not the ownership lease, guarantees that a job is never leased twice, so a brief overlap of two owners costs efficiency and not correctness.

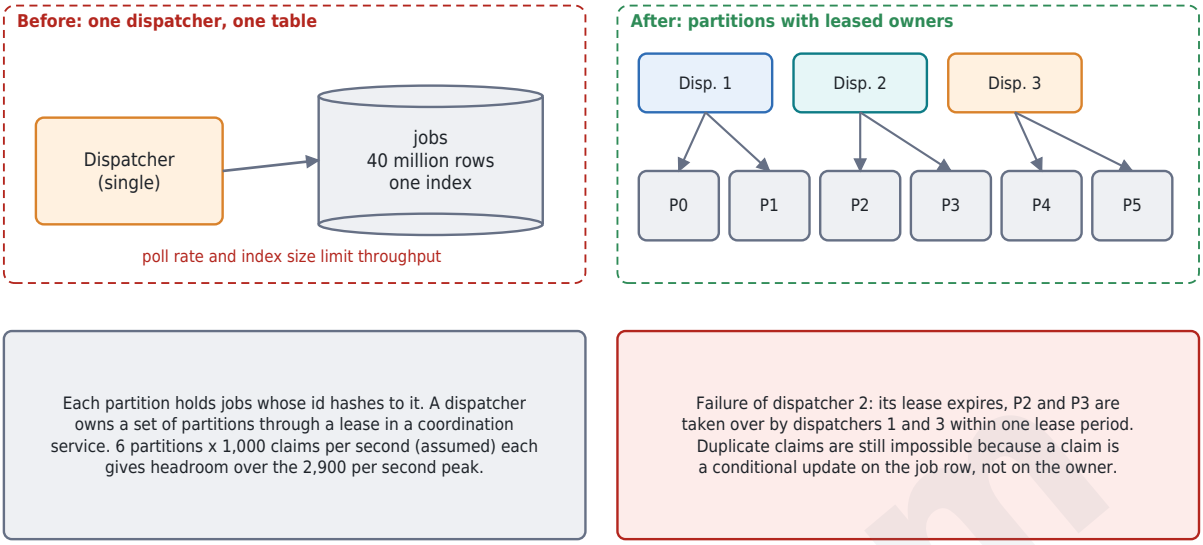


Figure 2.36. Partitioning the job store by id and giving each dispatcher a lease on several partitions scales dispatch; a dispatcher failure moves its partitions to the others.

2.9.7 Bottlenecks, failures and real-world problems

Worker crashes in the middle of a job

Scenario. A worker host is terminated during a deployment while running 40 jobs. **Symptoms.** Those 40 jobs stay in `LEASED`; their results are missing; users see delayed reminders.

Diagnosis. The query for jobs with `lease_until` in the past and status `LEASED` returns them; the worker's heartbeat stopped. **Root cause.** A crash cannot report failure, so the system must infer it from missing heartbeats.

Fix. Leases with heartbeats: a job whose lease expires becomes claimable again with a new fence token, at most lease-length (30 s) plus poll interval later. Workers drain gracefully on shutdown by finishing or releasing their jobs. **Prevention.** Keep handlers idempotent, alert on the number of expired leases per minute, and set the lease length from the typical job duration.

A slow worker completes after its lease expired

Scenario. A garbage-collection pause of 40 seconds makes a worker miss its heartbeats; another worker runs the job; the first worker resumes and reports completion. **Symptoms.** The job's effect is applied twice, or its final status flips between results.

Diagnosis. Two successful attempts for one job in the run history, with overlapping time ranges. **Root cause.** The first worker did not know that it had lost the lease; time-based ownership cannot prevent a stale actor from continuing.

Fix. Fence tokens: every completion is a conditional update on the current token (as in the simulation, the stale completion is rejected), and downstream effects use the job id as an idempotency key. **Prevention.** Pass the fence token to every side-effecting call that can validate it, and test with injected pauses.

Thundering herd of cron jobs at the top of the hour

Scenario. 200,000 schedules are configured as `0 * * * *`; at 10:00:00 all become due. **Symptoms.** Queue depth jumps by 200,000 at once; at 2,900 dispatches per second the last job starts about 69 seconds late; other tenants' jobs queue behind them.

Diagnosis. A histogram of `run_at` seconds shows a spike on round minutes; start-delay percentiles rise at :00 each hour. **Root cause.** Humans choose round times, and nothing spreads the load.

Fix. Apply a deterministic jitter to each schedule (a hash of the schedule id modulo a window such as 5 minutes) unless the owner asks for exact time, use per-tenant fairness limits in the claim query, and use a lower priority class for non-urgent schedules. **Prevention.** Track start delay per priority and tenant, and publish guidance that recommends offsets.

Poison job retried forever

Scenario. A job with a malformed payload throws an exception on every attempt; the retry policy has no limit. **Symptoms.** The same job id appears in the logs thousands of times, consumes worker capacity, and its error floods the alerting channel.

Diagnosis. The attempts column of the job is high; the error text is identical each time. **Root cause.** Retries without a limit or a distinction between transient and permanent failures.

Fix. A maximum number of attempts (for example 5) with exponential backoff and jitter, classification of errors (a validation error is fatal and not retried), and a `DEAD` state with an alert and a replay tool. **Prevention.** Validate payloads at submission against the handler's schema, and review the dead jobs regularly.

2.9.8 Trade-offs and extensions

- **Database queue versus a broker.** A table with `SKIP LOCKED` gives transactional submission (insert the job in the same transaction as the business change), delays, priorities and visibility; a broker gives higher raw throughput and simpler consumption but weaker delay and inspection features. The outbox pattern bridges the two.
- **Lease length.** A short lease recovers quickly and increases false takeovers for slow jobs; a long lease does the opposite. Heartbeats let the lease stay short while jobs run long.
- **Fairness.** Per-tenant concurrency limits (claim at most k running jobs per tenant) protect small tenants from a flood by a large one.
- **Workflows.** Dependencies between jobs are modelled by a parent job that submits children and waits for their completion records; a full workflow engine adds state machines and history.
- **Ordering.** Jobs have no ordering by default; per-key ordering (all jobs for one account in sequence) is achieved by routing the key to one partition and claiming one job per key at a time.

2.9.9 Questions to expect

How does the scheduler find due jobs among 100 million?

A partial index on the due time and priority restricted to unfinished jobs, so each poll reads the front of the index: the cost depends on the batch size and not on the table size. Partitioning by job id splits the polling.

How are two dispatchers prevented from taking the same job?

The claim is one conditional update using `FOR UPDATE SKIP LOCKED`: rows locked by one dispatcher are skipped by the others. In the PostgreSQL run, four concurrent claimers took 2,000 distinct jobs with no duplicates.

What happens when a worker dies?

Its lease expires because no heartbeat extends it; the job becomes claimable again, with an increased attempt count and a new fence token, within lease length plus poll interval, 31 seconds in the estimate.

Why fence tokens when there are leases?

Leases are time-based, and a paused worker may resume after its lease expired. The fence token makes the store reject its completion, because the token is no longer current. Leases alone cannot stop a stale actor.

Can a job run twice?

Yes, at least once is the guarantee. Handlers must be idempotent: use the job id as an idempotency key for external effects, and write results as set operations.

How are recurring jobs kept on schedule after downtime?

The next fire time is computed from the schedule, and a misfire policy (skip, run once, catch up) decides about the missed runs, as in the demo where four missed runs were skipped.

How would a 15-minute reminder for 10 million reservations be built?

Insert a job with `run_at` equal to creation plus 15 minutes in the same transaction as the reservation. If the reservation is confirmed before, the handler checks the state and does nothing; cancellation sets the job to cancelled or the handler tolerates it.

How is fairness between tenants provided?

The claim query limits the number of leased jobs per tenant, or uses a per-tenant queue with round-robin selection, so that one tenant's burst does not occupy all workers.

KEY POINTS

- Store jobs in a partitioned table with an index on due time; claim them with one conditional update using SKIP LOCKED so that concurrent dispatchers never take the same job.
- A lease with heartbeats recovers jobs of crashed workers within lease length plus poll interval; a fence token rejects completions from stale workers.
- Execution is at least once: make handlers idempotent with the job id as the idempotency key.
- Compute the next fire of a schedule from the schedule, define a misfire policy, and jitter round-time schedules to avoid herds.
- Bound retries, classify permanent errors, and dead-letter poison jobs.

2.10 Design: News Feed

A news feed presents each user with a ranked or chronological list of posts written by the accounts that user follows. The design problem is a read-heavy workload with a highly skewed write amplification: a single post by an account with many followers must become visible in many individual feeds within seconds, while a typical feed read must complete in tens of milliseconds. The central decision is where the work of assembling a feed is performed: at write time (fan-out on write), at read time (fan-out on read), or by a hybrid of both.

2.10.1 Requirements and scope

Functional requirements

- A user publishes a post consisting of text and up to four media items.
- A user follows and unfollows other users.
- A user retrieves a home feed: the posts of followed accounts, newest first (ranked ordering is treated as an extension).
- The feed supports cursor-based pagination and an infinite scroll interaction.

Non-functional requirements

- Feed read latency: p99 below 200 ms for the first page, measured at the service boundary.
- Freshness: a post from a typical account appears in followers' feeds within 5 seconds; from a celebrity account within 30 seconds.
- Availability of the read path above 99.95 percent; the write path may degrade to queued acceptance.
- Eventual consistency of the feed is acceptable. A user must always see their own post immediately (read-your-writes for the author).

Out of scope

Ranking models, ad insertion, search, direct messages, content moderation pipelines, notifications, and analytics. These attach to the design through the event stream described below.

Assumptions

All volumes below are assumed for the purpose of sizing and are not measurements of any real service: 200 million daily active users (DAU), 10 feed opens per user per day, 0.5 posts per user per day, an average of 200 followers per account, and a heavy tail of accounts with more than 1 million followers.

2.10.2 Capacity estimation

Quantity	Arithmetic	Result
Feed read QPS (average)	200e6 users x 10 opens / 86,400 s	23,148 per s
Feed read QPS (peak)	23,148 x 3	about 70,000 per s
Post write QPS (average)	200e6 x 0.5 / 86,400	1,157 per s
Post write QPS (peak)	1,157 x 3	about 3,500 per s
Timeline inserts (push)	100e6 posts x 200 followers / 86,400	231,481 per s
Timeline inserts (peak)	231,481 x 3	about 700,000 per s
Timeline cache size	16 B x 800 ids x 200e6 users	2.56 TB (5.1 TB with 2 replicas)
Cache nodes	5.1 TB / 64 GB usable per node	about 80 nodes
Post text storage	100e6 posts x 1 KB	100 GB per day, 36.5 TB per year
Media storage	20e6 posts with media x 1 MB	20 TB per day
Feed response bandwidth	70,000 per s x 20 posts x 1 KB	1.4 GB per s (11 Gbit per s), text only

Capacity estimate (all inputs assumed; peak factor 3 over the daily average).

The estimate exposes the shape of the problem. Reads outnumber post writes by a factor of 20 (23,148 against 1,157 per second), but each write under a push model produces 200 cache inserts on average, so the cache absorbs about 10 times more insert operations than the number of feed reads it serves. The write amplification, not the read rate, is the scaling constraint; it also motivates keeping only identifiers (16 bytes per entry) in the cache rather than post bodies.

2.10.3 API design

Endpoint	Request / response	Notes
POST /v1/posts	Header Idempotency-Key. Body: text, media_ids[]. Response 201: post_id, created_at	Retried requests with the same key return the original post_id
GET /v1/feed	Query: cursor, limit (default 20, max 50). Response: items[], next_cursor	Cursor is the last post_id returned
PUT /v1/users/{id}/follow	Empty body. Response 204	Idempotent by construction
DELETE /v1/users/{id}/follow	Response 204	Triggers timeline cleanup lazily

REST interface (all endpoints authenticated with a bearer token).

Pagination uses a keyset cursor rather than an offset. Post identifiers are time-ordered 64-bit identifiers (snowflake-style), so "posts older than the cursor" is a stable predicate: new posts arriving at the head of the feed do not shift the pages already delivered, which an offset-based page number would do. The idempotency key on post creation protects against duplicate posts when a mobile client retries after a network timeout.

2.10.4 Data model

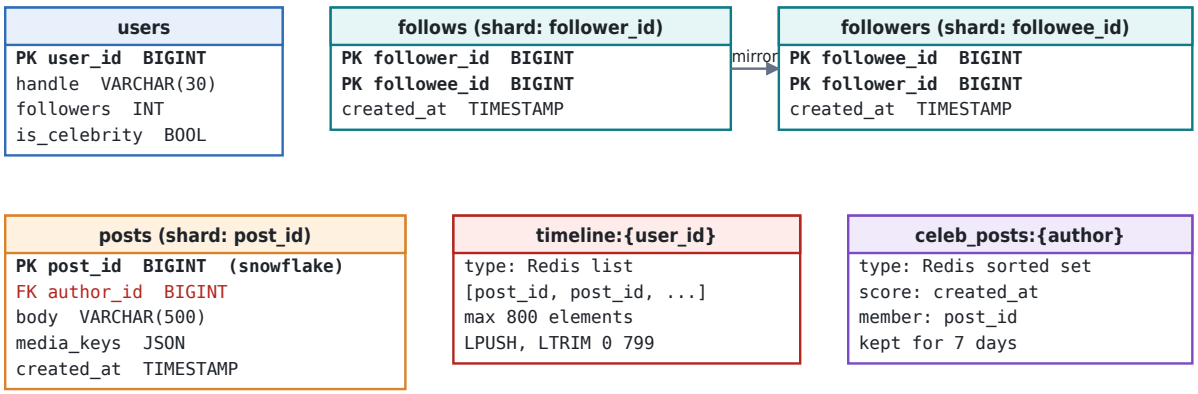


Figure 2.37. Relational tables are sharded by the key shown; the follower relation is stored twice, once per query direction, and the two Redis structures hold only identifiers.

The follow relation is queried in two directions: "whom does this user follow" (pull path, ranking features) and "who follows this author" (fan-out). Each direction is stored as its own table with its own shard key, written together by the follow service. Posts are sharded by post_id, which is a snowflake identifier, so that writes spread evenly and range scans by time remain possible within a shard. A sharded, replicated relational or wide-column store is appropriate here, as covered in the databases material; the design does not depend on the choice. The timeline is a Redis list per user holding at most 800 post identifiers, trimmed on every push. An entry is 16 bytes in the estimate (8 bytes post id, 4 bytes author id, 4 bytes flags). Per the 2013 description of Twitter's architecture on High Scalability, that system kept timelines in Redis with a maximum of 800 entries per user and retained only recently active users (a login within 30 days) in memory; the numbers here follow the same pattern but are otherwise assumptions.

2.10.5 High-level architecture

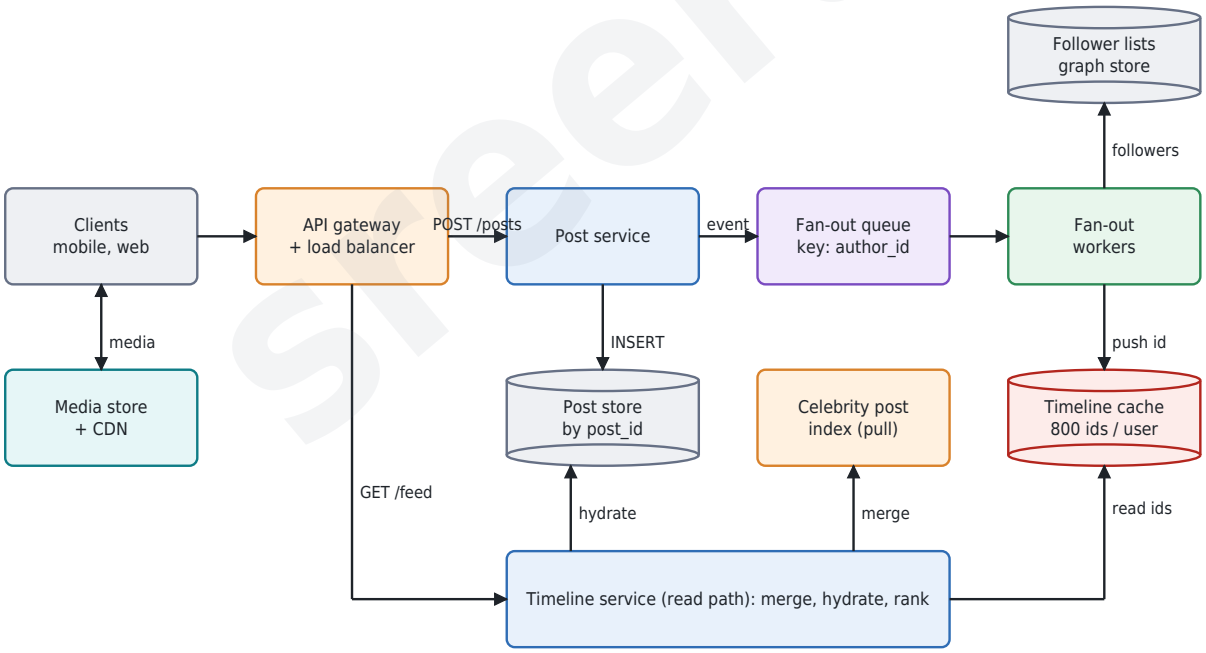


Figure 2.38. The write path (top row) ends in the timeline cache through a queue and fan-out workers; the read path (bottom row) merges the cache, the celebrity post index and the post store.

Write path

1. The client calls `POST /v1/posts`. The post service validates the request, assigns a snowflake identifier and inserts the row into the post store.
2. The post service publishes a `PostCreated` event to the fan-out queue, partitioned by author_id so that the events of one author remain ordered.

3. A fan-out worker reads the author's follower list in pages of 1,000 and pushes the post identifier onto each follower's timeline (`LPUSH`, then `LTRIM` to 800). Accounts above the celebrity threshold are not fanned out; the event only updates the celebrity post index.
4. The service returns 201 as soon as step 1 and the queue publication succeed. Fan-out is asynchronous.

Read path

1. The client calls `GET /v1/feed`. The timeline service reads the user's list from the cache.
2. For every celebrity the user follows (a short list stored with the user), the service reads that author's recent posts from the celebrity index.
3. The identifiers are merged in descending order, the page boundary is applied using the cursor, and the post bodies, author profiles and counters are fetched in batch from the post store and its cache.
4. A cache miss (an inactive user) triggers a rebuild: the last 7 days of posts of followed accounts are read, merged and written back.

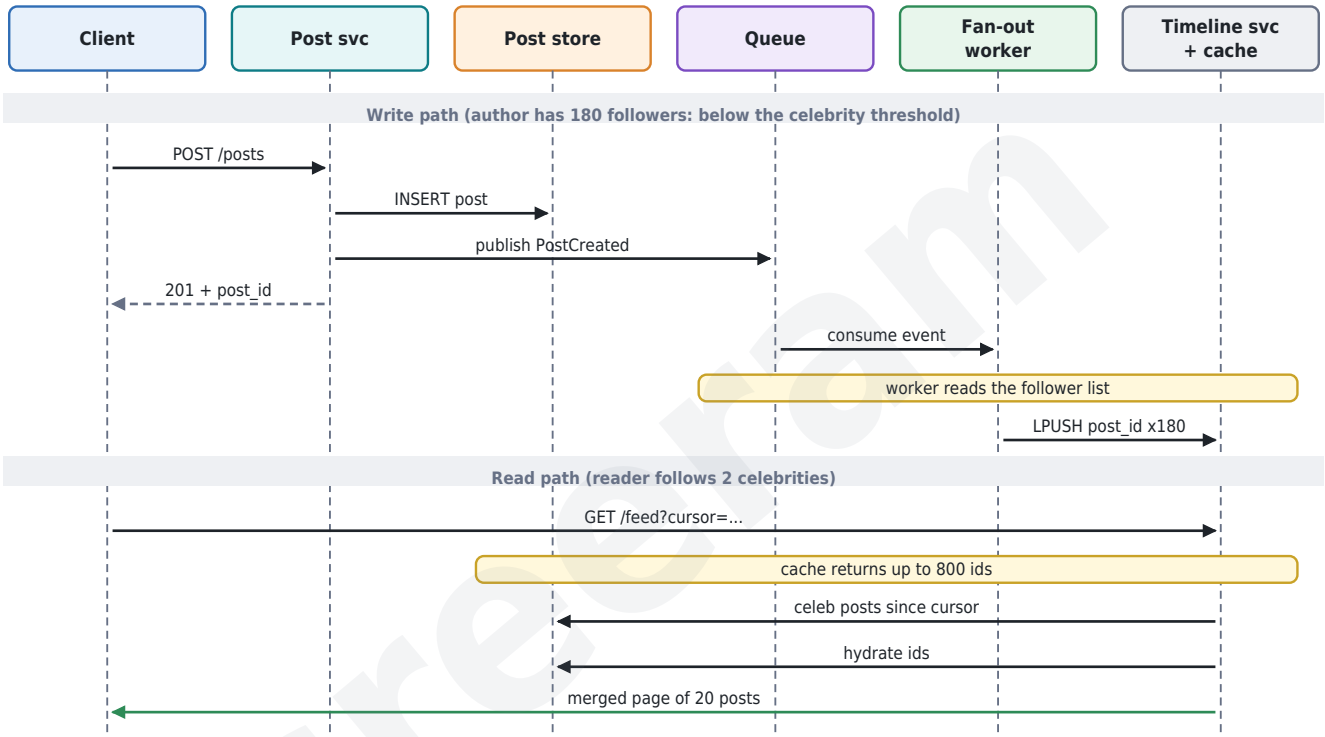


Figure 2.39. The same two paths as messages: the author's post is acknowledged before fan-out begins, and the reader's merge happens at request time only for celebrity authors.

2.10.6 Deep dive

Fan-out on write, fan-out on read, or hybrid

Property	Fan-out on write (push)	Fan-out on read (pull)	Hybrid
Cost of one post	O(followers) inserts	One insert	O(followers) for non-celebrities only
Cost of one feed read	One cache read	O(followees) queries and merge	Cache read plus O(celebrity followees)
Read latency	Lowest, predictable	Highest, grows with follows	Low, slightly variable
Wasted work	Fan-out to inactive users	None	Reduced
Failure mode	Celebrity post storm, lag	Slow reads, query fan-in	Two code paths to operate

Comparison of feed assembly strategies.

The decision is driven by the read to write ratio and by the follower distribution. Pure push is simplest to reason about and is the right choice when no account has more than a few thousand followers. Pure pull is acceptable for small systems. When the follower distribution has a heavy tail, the hybrid is the standard answer: push for the many, pull for the few. The threshold is a tunable, chosen by comparing the push cost saved against the read cost added.

sreeram

Measuring the trade-off with a simulation

The following program builds a synthetic follow graph of 100,000 users, each following 50 accounts chosen with a power-law popularity (weight proportional to rank to the power of -0.8), and counts the timeline inserts per day (one post per user) and the post-store lookups per feed read for each strategy. The graph is synthetic and the counts are simulation results, not production measurements.

FanoutSim.java JAVA

```java
import java.util.*;

public class FanoutSim {
    public static void main(String[] args) {
        final int users = 100_000, follows = 50;
        Random rnd = new Random(42);
        // popularity weight ~ power law: weight of rank r is r^-0.8
        double[] cum = new double[users];
        double sum = 0;
        for (int r = 0; r < users; r++) { sum += Math.pow(r + 1, -0.8); cum[r] = sum; }
        int[] followers = new int[users];
        int[][] followees = new int[users][];
        for (int u = 0; u < users; u++) {
            Set<Integer> set = new HashSet<>();
            while (set.size() < follows) {
                int v = lowerBound(cum, rnd.nextDouble() * sum);
                if (v != u) set.add(v);
            }
            followees[u] = set.stream().mapToInt(Integer::intValue).toArray();
            for (int v : followees[u]) followers[v]++;
        }
        int max = Arrays.stream(followers).max().getAsInt();
        long edges = (long) users * follows;
        System.out.printf("users=%d follows/user=%d edges=%d max followers=%d%n",
            users, follows, edges, max);
        System.out.println("strategy             inserts/day (1 post each)  lookups per read");
        System.out.printf("fan-out on write      %,12d            %d%n", edges, 1);
        System.out.printf("fan-out on read       %,12d            %d%n", (long) users, follows);
        for (int threshold : new int[] {1000, 5000, 20000}) {
            long writes = 0;
            for (int v = 0; v < users; v++) writes += followers[v] <= threshold ? followers[v] : 0;
            long celebLookups = 0; int celebs = 0;
            for (int v = 0; v < users; v++) if (followers[v] > threshold) celebs++;
            for (int u = 0; u < users; u++)
                for (int v : followees[u]) if (followers[v] > threshold) celebLookups++;
            double perRead = 1 + (double) celebLookups / users;
            System.out.printf("hybrid T=%-6d       %,12d             %.2f   (celebrities=%d)%n",
                threshold, writes, perRead, celebs);
        }
    }

    static int lowerBound(double[] a, double x) {
        int lo = 0, hi = a.length - 1;
        while (lo < hi) { int m = (lo + hi) >>> 1; if (a[m] < x) lo = m + 1; else hi = m; }
        return lo;
    }
}
```

Run (JDK 21, seed fixed, output deterministic) TERMINAL

```
$ java FanoutSim.java
users=100000 follows/user=50 edges=5000000 max followers=67337
strategy             inserts/day (1 post each)  lookups per read
fan-out on write         5,000,000            1
fan-out on read            100,000            50
hybrid T=1000            3,787,498            13.13   (celebrities=363)
hybrid T=5000            4,390,792            7.09   (celebrities=47)
hybrid T=20000           4,745,358            3.55   (celebrities=7)
```

What to notice: the most popular account has 67,337 followers, so one post under pure push costs 67,337 inserts. Raising the celebrity threshold from 1,000 to 20,000 followers reduces pull lookups per read from 13.13 to 3.55 but increases daily inserts from 3.79 million to 4.75 million. With the 20,000 threshold, 7 accounts are handled by pull, which removes the only accounts whose fan-out exceeds 20,000 inserts. The threshold is therefore a cost curve, and the operational choice is to select the point where the worst-case fan-out latency stays within the freshness target (see the failure scenarios below).

Timeline cache structure and rebuild

A Redis list supports `LPUSH` and `LTRIM` in constant time per element and returns a page with `LRANGE`. Its weakness is that a duplicate push produces a duplicate entry. A sorted set keyed by post_id with the post_id as score makes the insertion idempotent (`ZADD` of an existing member changes nothing) and allows range queries by cursor (`ZREVRANGEBYSCORE` below the cursor) at the cost of roughly twice the memory per entry. The design chooses the sorted set when the fan-out queue delivers at least once, which is the normal case; the list is a valid optimisation when duplicates are removed on read. Timelines of inactive users are evicted. When such a user returns, the cache miss path rebuilds the timeline from the follow list and the post store, bounded to the last 7 days and 800 entries. The rebuild is rate limited per shard so that a morning peak of returning users does not overload the post store.

Pagination and ranking

Chronological ordering needs no ranking service. When ranked ordering is added, the architecture becomes two-stage: the timeline cache supplies up to 800 candidates, a ranking service scores them with features read from a feature store, and the top page is returned. The cursor then refers to a ranking session identifier stored server-side for a short time, because the score order is not reproducible from a post_id alone.

2.10.7 Bottlenecks, failures and real-world problems



Figure 2.40. Fan-out time grows linearly with follower count; a single account with tens of millions of followers occupies a worker pool for minutes.

Celebrity fan-out storm

Scenario. An account with 50 million followers publishes a post. The fan-out workers begin 50 million inserts. **Symptoms.** Fan-out queue lag rises for all authors, because the partition holding the celebrity event is blocked; ordinary users report that their posts take minutes to appear. **Diagnosis.** The consumer lag metric per partition shows one partition far behind; the worker for that partition reports a single event in progress for several minutes. **Root cause.** Fan-out cost proportional to followers is applied uniformly, and one event occupies a partition worker serially. **Fix.** Exclude accounts above the threshold from push and serve them at read time through the celebrity index; for accounts near the threshold, split the follower list into chunks processed by a pool of workers in parallel. **Prevention.** Alert on the maximum fan-out duration, not the average; classify accounts automatically by follower count with hysteresis (promote at 100,000, demote at 80,000) to avoid flapping.

Stale entries after delete or unfollow

Scenario. A user deletes a post or a viewer unfollows an author. **Symptoms.** The deleted post still appears in followers' feeds; an unfollowed author continues to appear. **Diagnosis.** The identifier is present in the timeline cache but absent (or flagged deleted) in the post store. **Root cause.** The cache holds identifiers written in the past; removing them from every follower timeline is as expensive as the original fan-out. **Fix.** Treat the cache as a hint. Hydration reads the post store, and entries whose post is deleted, or whose author is no longer followed, are dropped from the page. Entries are physically removed lazily by the trim operation and the periodic rebuild. **Prevention.** Keep the hydration step mandatory and apply a block list and deletion flags at read time.

Cold cache after a cache cluster failure

Scenario. A cache shard group is lost and restarted empty. **Symptoms.** The post store CPU rises sharply, feed latency exceeds the objective, and error rates grow as rebuild requests queue. **Diagnosis.** The cache hit ratio drops to near zero for the affected shard range while rebuild queries dominate the post store. **Root cause.** Every returning user triggers an expensive rebuild simultaneously (a thundering herd on a cold cache). **Fix.** Rate limit rebuilds per shard, serve a degraded feed (the last 50 posts from the pull path) while rebuilding, and coalesce concurrent rebuilds of the same user. **Prevention.** Replicate the cache across availability zones, keep a rebuild budget as configuration, and test the cold start in a game day.

Preeram

Duplicate fan-out

Scenario. A worker crashes after pushing to half of the followers and before committing its queue offset. **Symptoms.** After the restart, some followers see the same post twice. **Diagnosis.** Duplicate identifiers in a single user's timeline. **Root cause.** At-least-once delivery combined with a non-idempotent insert. **Fix.** Use an idempotent insert (sorted set keyed by post_id) or de-duplicate on read. **Prevention.** Design every consumer in the pipeline as idempotent; commit offsets only after the work is complete.

2.10.8 Trade-offs and extensions

- Push optimises read latency at the price of storage and write amplification; pull optimises write cost at the price of read latency. The hybrid adds an operational burden (threshold management and two code paths).
- Storing identifiers only keeps memory small but adds a hydration round trip on every read; caching hydrated post bodies in a separate cache reduces it.
- Ranked feeds require a feature store and an online ranker, and change the cursor semantics.
- Extension: following a hashtag or a topic is another graph edge type; the fan-out worker consults a generic subscription index instead of the follower table.
- Extension: multi-region deployment places a timeline cache per region, replicating post events asynchronously; a user reads from the nearest region and may observe cross-region lag of a few seconds.

2.10.9 Questions to expect

Why not compute the feed at read time for everyone?

The read cost is O(followees) store queries plus a merge per request. At 70,000 peak reads per second and 200 followees, that is 14 million store queries per second. Push converts that into one cache read.

How is the celebrity threshold chosen?

By the fan-out latency target. If the pool performs 100,000 inserts per second and the freshness target is 30 seconds, the largest account that may be pushed has about 3 million followers; a threshold far below that (for example 100,000) leaves a safety margin and reduces wasted writes to inactive followers.

How does an author see their own post immediately?

The client inserts the post locally, and the timeline service additionally merges the author's own recent posts from the post store into the author's feed at read time, so read-your-writes holds without waiting for fan-out.

What happens to a follower who is offline for a week?

The timeline cache entry is evicted after the inactivity window. On return, the rebuild path reads recent posts of followed accounts; older posts are reachable by the pull path with pagination.

How is the ordering of posts guaranteed in the timeline?

Order follows the snowflake identifier, which encodes time. Events of one author remain ordered by partitioning the queue by author_id; ordering across authors is by identifier and is approximate across clock skew of a few milliseconds.

How would the design change for 10 times the users?

The cache cluster grows linearly and is resharded with consistent hashing; the celebrity threshold is lowered; fan-out workers scale with queue partitions; post store shards are added. The design has no single global component except the identifier scheme, which is distributed.

KEY POINTS

Key points

- Feed design is dominated by write amplification (followers per post), not by the raw read rate.
- Store identifiers in the timeline cache and hydrate from the post store; the cache is a hint, never the source of truth.
- Use push for ordinary accounts and pull for accounts above a follower threshold; derive the threshold from the fan-out latency target.
- Make every stage of the pipeline idempotent because the queue delivers at least once.
- Use keyset (cursor) pagination so that new posts do not shift delivered pages.

sre.gram

2.11 Design: Chat System

A chat system delivers short messages between users in near real time, keeps a durable conversation history, and tolerates devices that disconnect and reconnect at arbitrary times. The design problem differs from a feed in three ways: connections are long-lived and stateful, per-conversation ordering is visible to users, and the delivery state of every message (sent, delivered, read) is part of the product. The central decisions concern the connection layer, the source of message order, and the representation of delivery state.

2.11.1 Requirements and scope

Functional requirements

- One-to-one conversations and group conversations (up to 1,024 members, assumed limit).
- Text messages and media attachments (media travels through object storage, not through the chat path).
- Delivery receipts: sent, delivered, read.
- Offline delivery: a message sent to an offline user is delivered after reconnection and a push notification is raised.
- Message history with pagination, available on every device of the user.

Non-functional requirements

- End-to-end latency below 200 ms at p99 between two online users in the same region.
- No message loss once the sender has received the server acknowledgement; no duplicates visible to the user.
- Messages of one conversation appear in the same order to all participants.
- Availability above 99.99 percent for message acceptance; delivery may be delayed during partial failures.

Out of scope

End-to-end encryption protocols (the server treats the payload as opaque bytes), voice and video calls, message search, bots, and spam detection.

Assumptions

Assumed volumes: 500 million DAU, 40 messages sent per user per day, 30 percent of DAU connected at the daily peak, an average of 1.3 recipients per message (a mixture of one-to-one and group traffic), 200 bytes per stored message including metadata.

2.11.2 Capacity estimation

Quantity	Arithmetic	Result
Messages sent per day	500e6 x 40	`20 billion`
Send QPS (average / peak)	20e9 / 86,400 ; x 3	`231,481 / about 694,000 per s`
Deliveries per second (peak)	694,000 x 1.3	`about 900,000 per s`
Concurrent connections (peak)	500e6 x 30 percent	`150 million`
Gateway nodes	150e6 / 100,000 per node	`1,500 nodes`
Heartbeats	150e6 connections / one ping per 30 s	`5,000,000 per s`
Message storage	20e9 x 200 B	`4 TB per day (12 TB with 3 replicas)`
Storage per year	4 TB x 365	`1.46 PB (4.4 PB replicated)`
Message bandwidth (peak)	694,000 x 200 B	`139 MB per s`

Capacity estimate (all inputs assumed; peak factor 3).

Two results shape the design. First, the number of heartbeats (5 million per second) exceeds the message rate by a factor of seven, so the connection layer must be optimised for idle connections: the gateway does no application work for a heartbeat and the heartbeat interval is a significant cost parameter. Second, the message bandwidth is small. The difficulty of chat is the number of connections and the state attached to them, not the data volume. The figure of 100,000 connections per node is an assumption that is achievable on tuned hardware with an event-driven server and about 10 KB of memory per connection (1 GB per node).

Sreeram

2.11.3 API design

The primary channel is a WebSocket per device, carrying binary or JSON frames. Control operations that are not latency critical use REST.

Frame / endpoint	Fields	Direction
SEND	conv_id, client_msg_id (UUID), body, media_key	client to server
ACK	client_msg_id, seq, server_ts	server to client
MSG	conv_id, seq, sender_id, body, server_ts	server to client
RECEIPT	conv_id, kind (delivered or read), upto_seq	both
SYNC	conv_id, after_seq, limit	client to server
POST /v1/media	Response: pre-signed upload URL and media_key	REST
GET /v1/conversations	Query: cursor. Response: conversations ordered by last activity with unread counts	REST
POST /v1/conversations	Body: member ids, type. Response: conv_id	REST

WebSocket frames and REST endpoints.

The client_msg_id is an idempotency key generated on the device. If the connection drops before the ACK arrives, the client resends the same SEND frame, and the server returns the sequence number assigned the first time instead of storing a second copy.

2.11.4 Data model

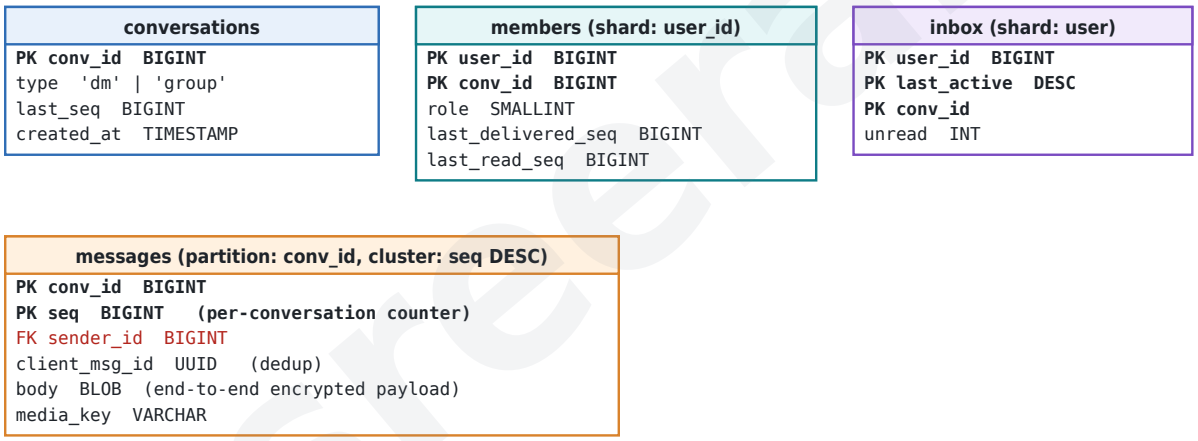


Figure 2.41. Messages are partitioned by conversation and clustered by a per-conversation sequence number; delivery state is stored as two cursors per member rather than as a row per message and recipient.

The messages table is the conversation log. Its partition key is conv_id, so that one conversation resides on one replica set and a page of history is a single sequential read; its clustering key is seq in descending order. This layout suits a wide-column store with tunable consistency, as covered in the databases material, but a sharded relational store with the same keys is equivalent. The members table is partitioned by user_id because the dominant query is "the conversations of this user". It carries the receipt state as two integers: `last_delivered_seq` and `last_read_seq`. A receipt for 5,000 unread messages is a single update of one integer instead of 5,000 row updates. A very active group could make one partition large. The usual remedy is a bucketed partition key (conv_id plus a time bucket of, for example, one week), which bounds partition size while keeping a page of recent history within one or two partitions.

2.11.5 High-level architecture

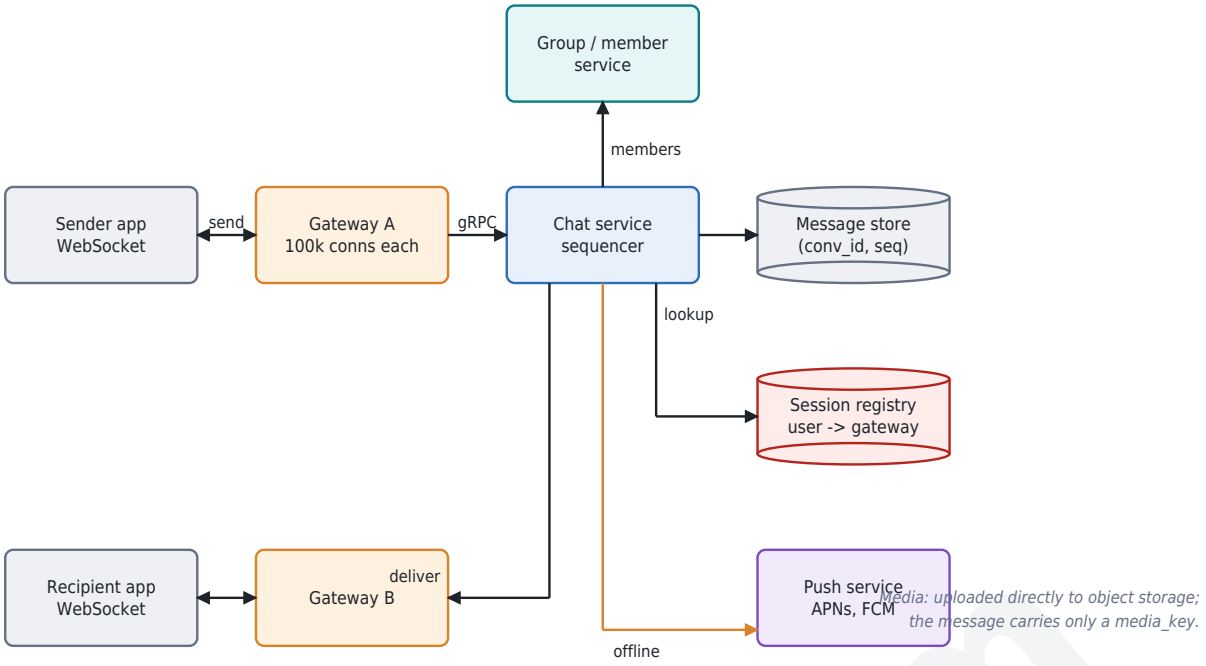


Figure 2.42. Clients hold WebSocket connections to gateways; the chat service sequences and stores each message, finds the recipient gateway through the session registry, and falls back to push notification when no session exists.

Send and deliver flow

1. The sender app transmits a SEND frame over its WebSocket. The gateway forwards it to the chat service by gRPC; the gateway holds no conversation state.
2. The chat service obtains the conversation's next sequence number from its owning shard, appends the message to the message store with a quorum write, and only then returns the ACK carrying the sequence number. The sender displays one tick (sent).
3. The chat service reads the member list (cached), queries the session registry for the gateway of each online member, and sends one push RPC per gateway carrying the message and the list of local recipients.
4. The gateway writes the frame to each recipient connection. The recipient device acknowledges, and the chat service advances `last_delivered_seq`. A delivered receipt flows back to the sender.
5. When the recipient opens the conversation, the device sends RECEIPT(read, upto_seq). The service updates `last_read_seq` and notifies the sender.

sreram

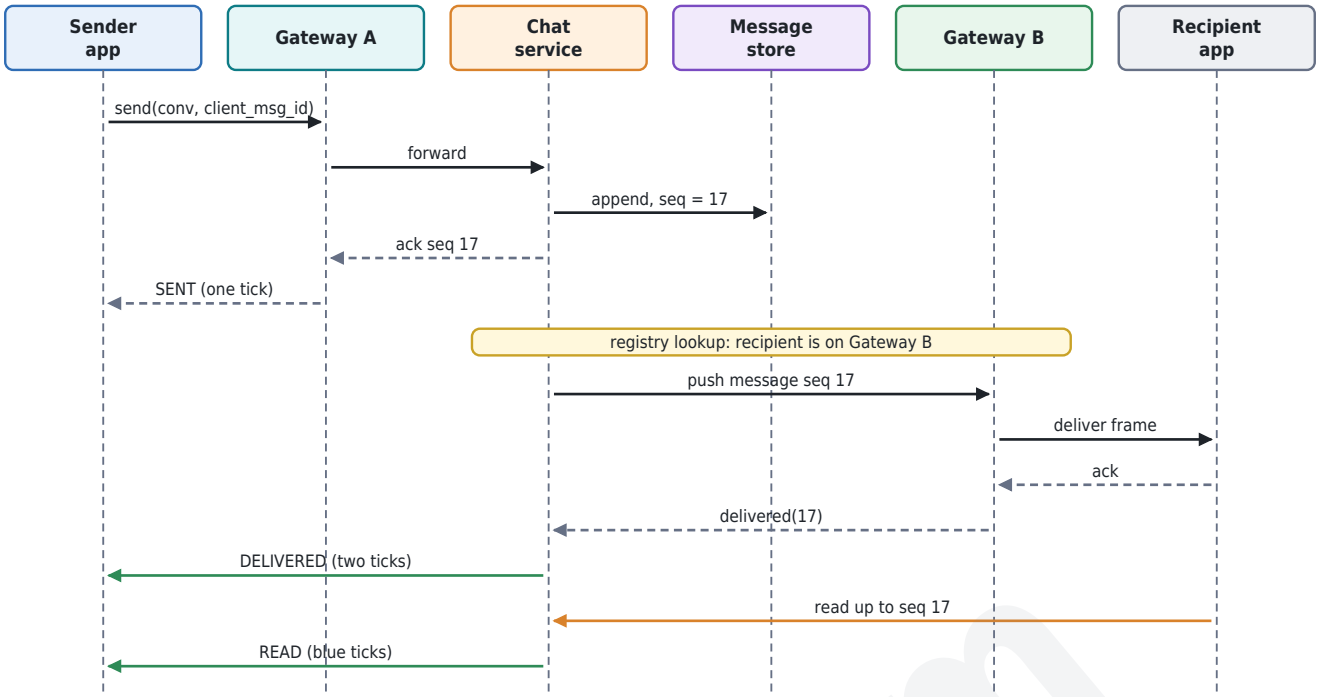


Figure 2.43. The sender is acknowledged after the durable write, not after delivery; delivery and read state flow back as cursor updates.

Offline path

If the registry has no entry for a member, the message is already durable in the log. The chat service enqueues a push notification (APNs or FCM) containing only a wake-up hint. On reconnection the device sends SYNC for each conversation with the highest sequence it holds, or one inbox-level sync request listing its cursors. The server returns the missing range from the log. Because the log is the source of truth, no per-recipient offline queue is required.

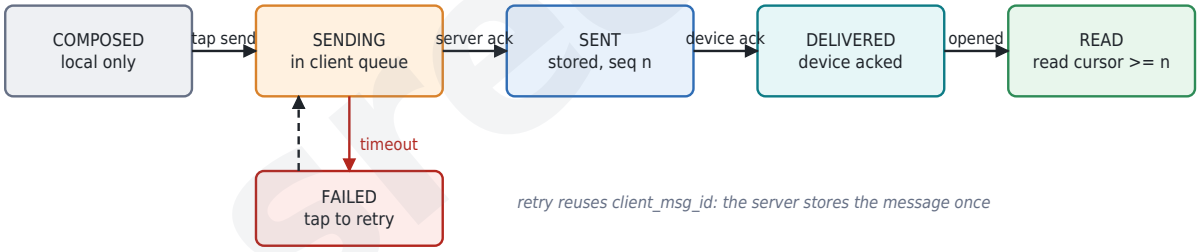


Figure 2.44. The message life cycle on the sender device: the failed state returns to sending with the same client_msg_id, so a retry never creates a second server copy.

2.11.6 Deep dive

Message ordering

Approach	Mechanism	Strength	Weakness
Client timestamps	Sort by device clock	No coordination	Clock skew reorders messages; trivially forged
Server timestamps	Sort by arrival time at the receiving node	Simple	Skew between nodes; ties; concurrent senders reach different nodes
Per-conversation sequence	One owner assigns seq = previous + 1	Total order, gap detection	The owner is a serialisation point per conversation
Logical clocks	Lamport or vector clocks	No single owner	Partial order; complex client merge

Alternatives for ordering messages within a conversation.

Decision: a per-conversation sequence number assigned by the shard that owns the conversation. The serialisation point is acceptable because one conversation produces a small message rate (a very active group may produce tens of messages per second, far below the capacity of one shard). The owner is chosen by consistent hashing of conv_id and replicated; a failover must not reuse sequence numbers, so the counter is persisted with the log or derived from the highest stored seq under the new owner. Sequence numbers make gap detection trivial: a client holding seq 10 that receives seq 12 knows that seq 11 is missing.

Verifying the ordering and deduplication logic

The following program implements the two halves of the protocol: a sequencer that deduplicates by client_msg_id, and a client receiver that buffers out-of-order frames, discards duplicates and detects gaps. The network is simulated by a fixed permutation of arrivals; this is a single-process simulation, not a network test.

ChatSim.java JAVA

```
import java.util.*;

public class ChatSim {
    record Msg(String conv, long seq, String clientMsgId, String body) {}

    /** Server side: one sequencer per conversation; deduplicates by client message id. */
    static class Sequencer {
        private final Map<String, Long> next = new HashMap<>();
        private final Map<String, Msg> byClientId = new HashMap<>();
        Msg accept(String conv, String clientMsgId, String body) {
            Msg dup = byClientId.get(conv + "/" + clientMsgId);
            if (dup != null) return dup;                    // retry of an already stored message
            long seq = next.merge(conv, 1L, Long::sum);
            Msg m = new Msg(conv, seq, clientMsgId, body);
            byClientId.put(conv + "/" + clientMsgId, m);
            return m;
        }
    }

    /** Client side: delivers strictly in sequence order, buffers gaps, ignores duplicates. */
    static class Receiver {
        long lastShown = 0;
        final TreeMap<Long, Msg> pending = new TreeMap<>();
        final List<String> shown = new ArrayList<>();
        void onMessage(Msg m) {
            if (m.seq() <= lastShown || pending.containsKey(m.seq())) {
                shown.add("  (drop duplicate seq " + m.seq() + ")");
                return;
            }
            pending.put(m.seq(), m);
            while (!pending.isEmpty() && pending.firstKey() == lastShown + 1) {
                Msg n = pending.pollFirstEntry().getValue();
                lastShown = n.seq();
                shown.add("show seq " + n.seq() + ": " + n.body());
            }
        }
        long gapFrom() { return pending.isEmpty() ? -1 : lastShown + 1; }
    }

    public static void main(String[] args) {
        Sequencer server = new Sequencer();
        String conv = "c42";
        Msg a = server.accept(conv, "u1-001", "hello");
        Msg b = server.accept(conv, "u2-001", "hi, are you there?");
        Msg c = server.accept(conv, "u1-002", "meeting at 10");
        Msg retry = server.accept(conv, "u1-002", "meeting at 10");   // client retry
        System.out.println("retry returns same seq: " + (retry.seq() == c.seq())
            + " (seq " + retry.seq() + ")");

        Receiver r = new Receiver();
        // network reorders and duplicates: c, a, a, b
        for (Msg m : List.of(c, a, a, b)) {
            r.onMessage(m);
            System.out.println("arrive seq " + m.seq() + " -> pending=" + r.pending.keySet()
                + " lastShown=" + r.lastShown);
        }
        r.shown.forEach(System.out::println);

        Receiver lossy = new Receiver();
```

sreeram

ChatSim.java (continued) JAVA

```
        lossy.onMessage(a);
        lossy.onMessage(c);                                          // seq 2 lost
        System.out.println("gap detected, client requests sync from seq " + lossy.gapFrom());
        lossy.onMessage(b);                                          // resync delivers seq 2
        System.out.println("after resync lastShown=" + lossy.lastShown
            + " pending=" + lossy.pending.keySet());
    }
}
```

Run (JDK 21, deterministic) TERMINAL

```
$ java ChatSim.java
retry returns same seq: true (seq 3)
arrive seq 3 -> pending=[3] lastShown=0
arrive seq 1 -> pending=[3] lastShown=1
arrive seq 1 -> pending=[3] lastShown=1
arrive seq 2 -> pending=[] lastShown=3
show seq 1: hello
  (drop duplicate seq 1)
show seq 2: hi, are you there?
show seq 3: meeting at 10
gap detected, client requests sync from seq 2
after resync lastShown=3 pending=[]
```

What to notice: the retried SEND returns the existing sequence number 3 rather than allocating 4; the client shows nothing for sequence 3 until sequences 1 and 2 have arrived; the duplicate of sequence 1 is dropped; and a lost frame is visible as a gap, which triggers a SYNC from the first missing sequence.

Receipts and group fan-out

Two storage strategies exist for delivering a group message.

Aspect	One log per conversation (read-time fan-in)	One inbox copy per member (write-time fan-out)
Writes per message	1	Group size (up to 1,024)
Read of history	One partition scan	One partition scan per user
Receipts	Two cursors per member	Per-copy state
Storage	1 x	Group size x
Suited to	Chat (history is shared)	Email-like systems with per-user mutation

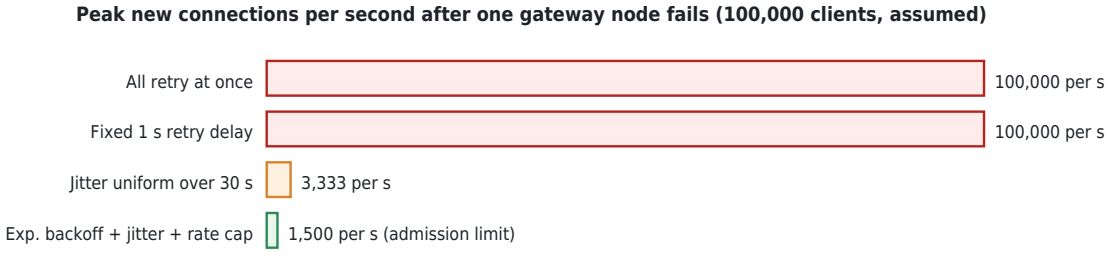
Group message storage strategies.

Chat history is shared and immutable apart from deletion flags, so the conversation log is stored once. Delivery fans out only over connections, never over storage: the chat service groups the online members of a group by gateway and sends one RPC per gateway. A group of 1,024 members spread over 300 gateways costs 300 RPCs, not 1,024. Read receipts in large groups are expensive in the UI as well as on the server; a common simplification is to disable per-member read receipts above a size (for example 100 members) and show only a count.

Connection management

Heartbeats detect dead connections: an interval of 30 seconds with a 3-interval timeout bounds the time during which the registry entry of a silently dead client is stale to 90 seconds. The session registry entry carries a time-to-live slightly above the timeout and is refreshed by heartbeats; an entry pointing to a failed gateway therefore expires on its own. A delivery RPC that fails because the gateway is unreachable removes the entry and takes the offline path.

2.11.7 Bottlenecks, failures and real-world problems



Clients reconnect to the nearest healthy gateway; admission control bounds the handshake rate.

Figure 2.45. Without jitter, the failure of one gateway turns 100,000 simultaneous reconnections into a handshake spike; spreading them over 30 seconds reduces the peak by a factor of 30.

Reconnection storm

Scenario. A gateway node holding 100,000 connections is restarted during a deployment. **Symptoms.** Neighbouring gateways show a spike in new connections and TLS handshakes, CPU saturates, and the authentication service returns errors; reconnection attempts fail and repeat. **Diagnosis.** The new-connection rate per gateway rises from a few hundred per second to tens of thousands within one second and correlates with the restart time. **Root cause.** Clients reconnect immediately and in lockstep, and each reconnection performs authentication and a full sync. **Fix.** Clients apply exponential backoff with random jitter (for example uniform over 0 to 30 seconds for the first attempt). Gateways enforce an admission rate and answer excess connections with a retry-after value. Deployments drain connections gradually (a few percent per second) and send a reconnect hint before closing. **Prevention.** Load test node loss with realistic client behaviour; keep authentication tokens verifiable locally so that reconnection does not depend on a central call.

Stale session registry entry

Scenario. A gateway fails without cleanup; the registry still lists its users as connected. **Symptoms.** Messages to those users are pushed to a dead node, no delivered receipt arrives, and users report delayed messages although the log contains them. **Diagnosis.** Push RPCs time out for one gateway address; the registry holds entries with that address. **Root cause.** The registry is an optimistic cache of connection location and can be wrong. **Fix.** Time-to-live on entries; a delivery timeout that deletes the entry and falls back to the offline path; the push notification and the sync on reconnect remain the safety net. **Prevention.** Treat the registry as a hint, never as proof of delivery; delivery is confirmed only by a device acknowledgement.

Duplicate and out-of-order delivery

Scenario. A client retries a SEND after a timeout; a delivery RPC is retried after a gateway timeout. **Symptoms.** The same message appears twice, or messages appear in a different order on two devices. **Diagnosis.** Two stored rows with different seq and one client_msg_id, or frames with decreasing seq on a client. **Root cause.** At-least-once delivery without deduplication, and ordering based on arrival time. **Fix.** Deduplicate by (conv_id, client_msg_id) at the sequencer and by seq at the client; order only by seq. **Prevention.** Property-test the protocol with injected duplication, reordering and loss, as the simulation above does in miniature.

Hot conversation partition

Scenario. A large public group receives thousands of messages per second after an event. **Symptoms.** The shard owning the conversation shows high CPU and write latency; other conversations on the same shard slow down. **Diagnosis.** Per-partition write rate far above the median; one conv_id dominates. **Root cause.** All writes of a conversation serialise on one owner and one partition. **Fix.** Time-bucketed partitions, batching of sequence allocation (reserve ranges of 1,000 sequence numbers), and a read-only broadcast mode for very large channels where followers receive a feed-like stream. **Prevention.** Limit group size and message rate, and classify very large channels as a different product with a feed-style design.

2.11.8 Trade-offs and extensions

- A per-conversation sequencer gives total order and simple gap detection but couples the availability of a conversation to its owner; replication and fast failover limit the effect.
- Cursor-based receipts reduce write cost by orders of magnitude but cannot express per-message receipts out of order.
- Multi-device support requires a per-device cursor and a per-device delivery state; the log is shared, so only cursors multiply.
- Presence and typing indicators are best-effort, ephemeral, and sent over the same connection with a short time-to-live and no durability; they must be throttled, since presence changes of large contact lists generate fan-out comparable to messages.
- End-to-end encryption moves the group fan-out of keys to the clients and prevents server-side search and spam inspection of content.

2.11.9 Questions to expect

Why WebSocket and not HTTP polling?

Polling at a 5 second interval from 150 million clients is 30 million requests per second, nearly all empty. A WebSocket keeps one connection per client, with a heartbeat of 5 million per second at a 30 second interval and with no application work, and allows server push with latency of one network hop.

How does the server know which gateway holds a user?

Through a session registry (an in-memory key-value store) written by the gateway on connect and refreshed by heartbeats, with a time-to-live. It is a hint; a failed delivery falls back to the offline path.

How is message loss prevented?

The ACK is sent only after a quorum write to the log. The client retains unacknowledged messages and retries with the same client_msg_id. The recipient syncs from its last cursor, so any missed push is recovered from the log.

How are read receipts stored?

As a per-member cursor (`last_read_seq`). Per-message state is derived by comparing the message seq with the cursor.

How does the design scale a conversation beyond one shard?

A single conversation is not split; the throughput of one owner suffices for realistic groups. For very large channels the design changes to a broadcast model with fan-out on read.

What happens on a network partition between gateway and chat service?

The gateway cannot forward SEND, so the client does not receive an ACK and retries after reconnecting to another gateway; the idempotency key prevents duplicates. Deliveries to users of the isolated gateway time out and take the offline path.

How is the order across devices of the same user kept consistent?

All devices read the same log ordered by seq; each device keeps its own cursor and local send queue. A message sent from device A appears on device B through the normal delivery path with its sequence number.

KEY POINTS

Key points

- Chat load is dominated by connection count and heartbeats, not by bytes; size the gateway tier by connections per node.
- Order messages with a per-conversation sequence assigned by the owning shard; never by client or arrival time.
- Acknowledge the sender after the durable write; deduplicate with a client-generated message id; confirm delivery with a device acknowledgement.
- Store each message once per conversation and represent receipts as cursors per member.
- Treat the session registry as a hint, and protect gateways from reconnection storms with jitter and admission control.

Sheeram

2.12 Design: Video Streaming Platform

A video streaming platform accepts uploaded videos, converts them into formats that any device and network can play, and delivers them to a very large audience. The workload has two independent halves with different constraints: an ingest half that is compute-bound (transcoding) and tolerates minutes of latency, and a delivery half that is bandwidth-bound and must start playback within about two seconds. The central decisions are how transcoding is parallelised, how the player adapts to changing bandwidth, and how a content delivery network (CDN) is arranged so that the origin never sees the audience.

2.12.1 Requirements and scope

Functional requirements

- Creators upload video files of up to 10 GB; uploads resume after a network failure.
- The platform transcodes each upload into a ladder of resolutions and bitrates and publishes it for playback.
- Viewers play a video with adaptive bitrate selection, seek to any position, and resume on another device.
- Videos have metadata (title, description, visibility) and a processing status visible to the creator.

Non-functional requirements

- Playback start time below 2 seconds at p95 for a cached title; rebuffering ratio below 1 percent of watch time.
- A 10-minute upload is playable within 10 minutes of upload completion (a low-resolution rendition earlier).
- Durability of stored originals and renditions of at least 99.999999999 percent (eleven nines, the figure offered by typical object stores).
- Availability of playback above 99.99 percent; uploads may degrade independently of playback.

Out of scope

Recommendation, comments, live streaming, digital rights management details, advertising, and content identification. Moderation is represented as one pipeline stage.

Assumptions

All figures are assumed: 100 million DAU, 5 videos watched per user per day with 8 minutes watched per video, an average delivered bitrate of 2.5 Mbit per second, 500,000 uploads per day averaging 10 minutes and 500 MB, and an edge cache hit ratio of 90 percent.

2.12.2 Capacity estimation

Quantity	Arithmetic	Result
Watch time per day	100e6 x 5 x 8 min	4 billion minutes
Egress per user per day	40 min x 60 s x 2.5 Mbit / 8	750 MB
Egress per day	750 MB x 100e6	75 PB
Egress bandwidth (average)	75e15 B x 8 / 86,400 s	6.9 Tbit per s
Egress bandwidth (peak)	6.9 x 2	about 14 Tbit per s
Origin egress at 90 percent hit	14 Tbit per s x 10 percent	1.4 Tbit per s
Concurrent viewers (peak)	4e9 min / 1,440 x 2	5.6 million
Segment requests (peak, 4 s segments)	5.6e6 / 4 s	1.4 million per s
Raw ingest	500,000 x 500 MB	250 TB per day (23 Gbit per s)
Ladder bitrate sum	0.4 + 0.8 + 1.4 + 2.8 + 5.0	10.4 Mbit per s
Rendition storage per video	10.4 Mbit per s x 600 s / 8	780 MB
Rendition storage per day	780 MB x 500,000	390 TB (142 PB per year)
Transcoding compute (average)	500,000 x 10 min x 3 core-min per video-min / 1,440	about 10,400 cores

Capacity estimate (inputs assumed; peak factor 2 for delivery, 3 for transcoding).

Sreeram

The estimate shows that delivery is three orders of magnitude larger than ingest in bytes (75 PB against 250 TB per day). No single data centre or origin can serve 14 Tbit per second, so the architecture is defined by the CDN. The 3 core-minutes of compute per video minute is an assumption for a five-rung ladder; real cost depends on codec, preset and hardware, and newer codecs cost considerably more. The storage figure grows by roughly the ladder size, which is why platforms store rarely watched renditions on cheaper tiers or transcode them on demand.

2.12.3 API design

Endpoint	Request / response	Notes
`POST /v1/videos`	Body: title, size_bytes, content_type. Response: video_id, upload_id, part_size, pre-signed part URLs	Creates a row in state UPLOADING
`PUT {part URL}`	Bytes of one part (for example 8 MB); response ETag	Goes directly to object storage, bypassing the API tier
`POST /v1/videos/{id}/complete`	Body: list of part numbers and ETags. Response 202	Triggers processing; idempotent
`GET /v1/videos/{id}`	Response: status, renditions available	Polled or pushed to the creator
`GET /v1/videos/{id}/playback`	Response: manifest_url (signed, expires in 6 hours), resume position	One call per playback start
`POST /v1/telemetry`	Batch of player events: startup time, stalls, bitrate switches	Feeds quality dashboards

Upload and playback API.

Upload bytes never pass through application servers. The API hands out pre-signed URLs for individual parts of a multipart upload, so the application tier handles a few thousand small requests per second while the object store absorbs 23 Gbit per second. A part that fails is retried alone; the `parts_done` bitmap allows a client that lost its state to ask which parts are present.

2.12.4 Data model

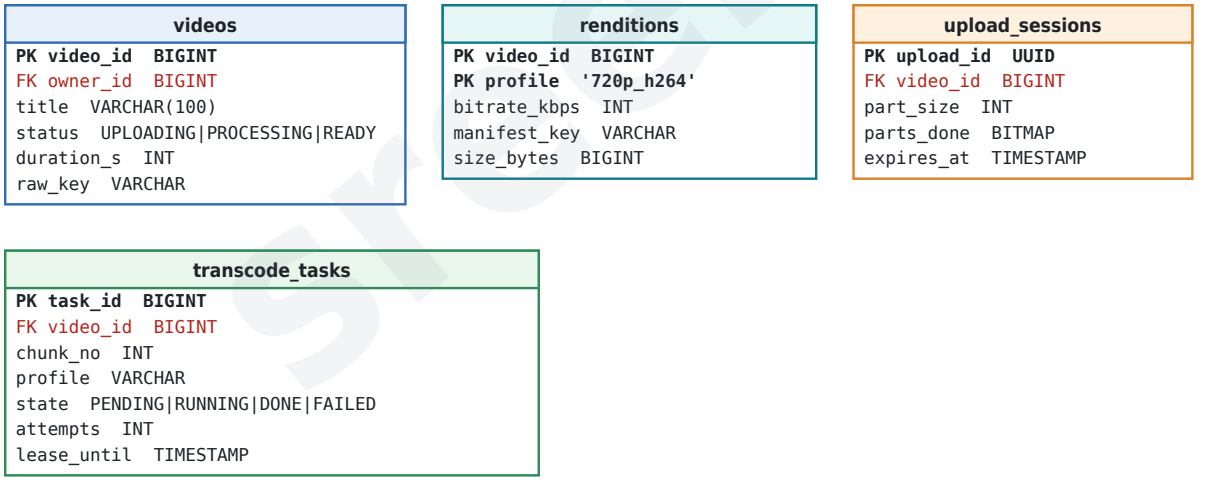


Figure 2.46. Video metadata and renditions are small relational records; the transcode task table is the work queue state, with a lease timestamp used to reclaim tasks from crashed workers.

Video metadata is a low-volume, relational workload (hundreds of millions of rows, modest write rate) and fits a sharded relational store keyed by video_id. The large objects reside in object storage under keys such as `raw/{video_id}` and `hls/{video_id}/{profile}/seg_00042.m4s`. The task table (or an equivalent durable queue) holds one row per chunk and rendition; its `state`, `attempts` and `lease_until` columns implement at-least-once execution with recovery, as discussed in the deep dive. View counts and watch history are high-volume counters and event streams that do not belong in this store; they are written to a log and aggregated asynchronously.

2.12.5 High-level architecture

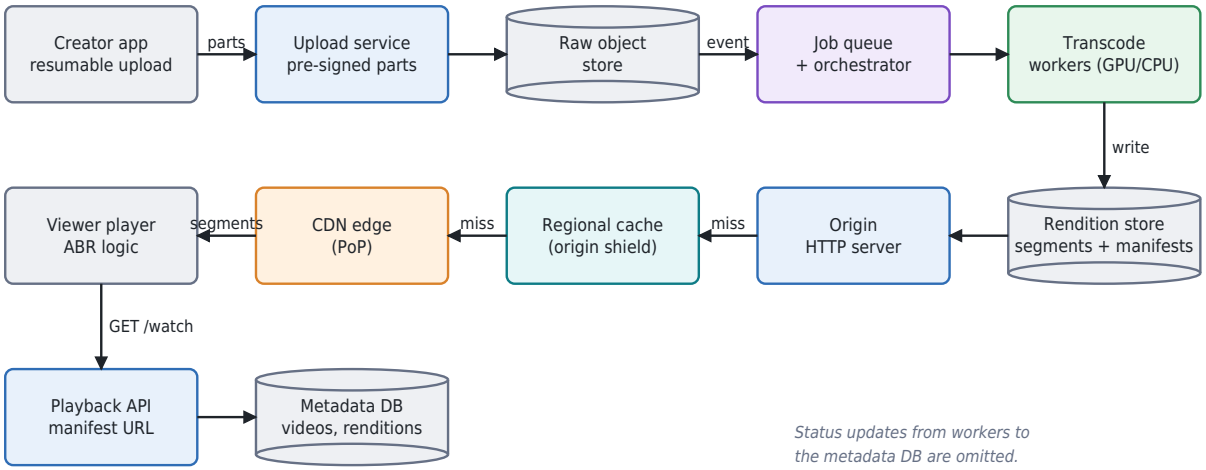


Figure 2.47. The upper row is the ingest path (upload, queue, transcode); the lower row is the delivery path in which the CDN edge, a regional cache and the origin form a three-level cache hierarchy.

Upload and processing flow

1. The creator app calls `POST /v1/videos` and uploads parts directly to the raw object store with the pre-signed URLs. After the last part, it calls `complete`.
2. The object store emits an event that is placed on the job queue. The orchestrator probes the file, rejects unsupported content, and splits the video at keyframe (GOP) boundaries into chunks of about 10 seconds.
3. Transcode tasks are created for every chunk and rendition and executed in parallel by a worker pool. Workers write segments to the rendition store under deterministic keys.
4. When all tasks of a rendition are done, the packager writes the manifests (HLS playlists and DASH manifests) and the orchestrator sets the status to READY. The 240p and 360p renditions finish first and allow early publication.

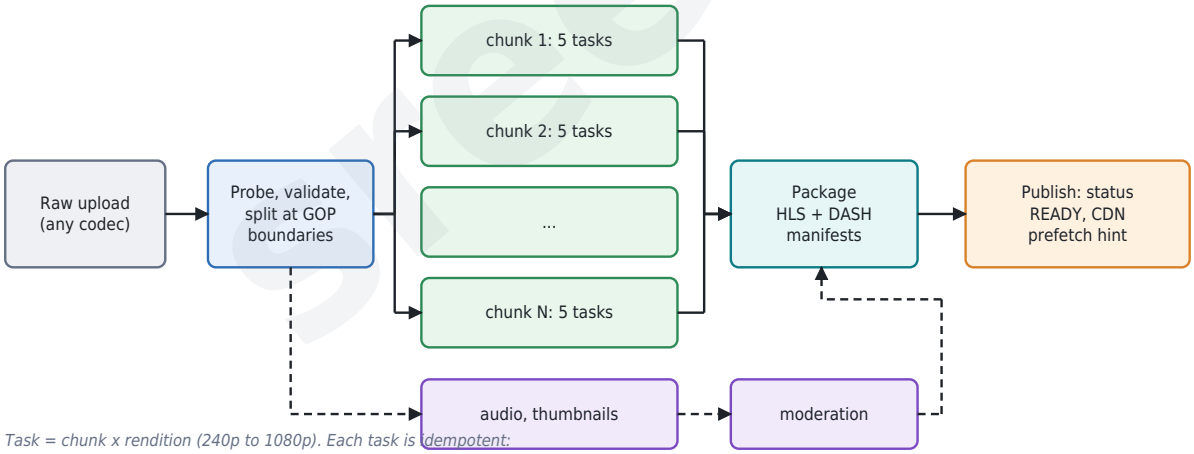


Figure 2.48. Transcoding is a directed acyclic graph: chunk-by-rendition tasks are independent and run in parallel; packaging waits for all of them, while audio, thumbnails and moderation run on a side branch.

Playback flow

1. The player calls the playback API, which verifies visibility and returns a signed manifest URL.
2. The player downloads the master manifest from the nearest CDN edge. It lists the renditions and their bitrates.
3. The player requests the first segment at a conservative rendition, measures throughput and buffer level, and selects each following segment's rendition by its adaptation policy.
4. A segment missing from the edge is fetched from the regional cache and, if absent there, from the origin; each level stores the object for later viewers.

2.12.6 Deep dive

Parallel transcoding

Strategy	Mechanism	Strength	Weakness
Whole file per worker	One worker produces every rendition sequentially	Simple	Latency proportional to video length; a failure restarts everything
Rendition per worker	One worker per rendition	Five-way parallelism	Latency set by the slowest (highest resolution) rendition
Chunk by rendition (DAG)	Tasks for each 10 s chunk and rendition	Latency nearly independent of length; failures cost one task	Chunk boundary artefacts if not keyframe aligned; orchestration complexity
Per-title ladder	Trial encodes choose the ladder per video	Fewer bits for equal quality	Extra compute per title

Transcoding strategies.

Decision: the chunk-by-rendition DAG. A 10-minute video has 60 chunks and 300 tasks, so with 300 workers available the wall-clock time approaches the time to encode one chunk. Tasks are idempotent: the output key is derived from (video, chunk, profile), so a repeated execution overwrites the same object. Each task row carries a lease; a worker claims a task by setting `lease_until` and renews it while encoding. A task whose lease has expired returns to PENDING with an incremented `attempts`, and after a limit (for example 5) the task moves to FAILED and the video to a manual review queue. Per-title encoding is an optimisation on top of the DAG. Per the Netflix Technology Blog post on per-title encode optimisation, a single fixed bitrate ladder (first deployed in late 2010) wastes bits on simple content and under-serves complex content, and choosing the ladder from trial encodes per title reduced bitrate for live-action content by about 20 percent in the published example (a 1080p top rung from 5,800 to 4,640 kbit per second at similar quality). The saving applies to storage and, more importantly, to delivery bandwidth, which dominates cost in the estimate above.

Adaptive bitrate streaming

The video is stored as short segments (2 to 10 seconds) in several renditions. A manifest lists them; the player is the control loop. It chooses the rendition of each next segment from observed throughput, from its buffer level, or from both. Segment duration is a trade-off:

Segment length	Requests per second at peak	Reaction to bandwidth change	Compression efficiency
2 s	2.8 million	Fast	Lower (more keyframes)
4 s	1.4 million	Moderate	Good
10 s	0.56 million	Slow; larger stall risk	Best

Segment duration trade-off (requests computed for 5.6 million concurrent viewers).

The following simulation compares three player policies on one synthetic throughput trace of 60 segments of 4 seconds with a ladder of 400, 800, 1,400, 2,800 and 5,000 kbit per second. The throughput rule picks the highest rendition not above 80 percent of an exponentially weighted throughput estimate; the buffer rule maps the buffer level between 6 and 16 seconds onto the ladder; the fixed policy always requests 5,000 kbit per second. The trace and policies are illustrative simplifications, not a model of any production player.

AbrSim.py PYTHON

```python
"""Adaptive bitrate simulation: three player policies on one throughput trace (segment = 4 s)."""
LADDER = [400, 800, 1400, 2800, 5000]          # kbit/s renditions
SEG = 4.0                                      # seconds of video per segment
MAXBUF = 20.0
# kbit/s available while each of the 60 segments is downloaded
TRACE = [6000] * 12 + [1200] * 10 + [3500] * 14 + [900] * 8 + [4500] * 16


def run(policy):
    buf, t_stall, stalls, ests, last_bitrate = 0.0, 0.0, 0, None, None
    chosen, switches, started = [], 0, False
    for i, bw in enumerate(TRACE):
        if policy == "fixed5000":
            rate = LADDER[-1]
        elif policy == "throughput":
            fit = [r for r in LADDER if ests is not None and r <= 0.8 * ests]
            rate = max(fit) if fit else LADDER[0]
        else:  # buffer based: map buffer level 6..16 s onto the ladder
```

AbrSim.py (continued) PYTHON

```
            idx = int((buf - 6.0) / 10.0 * len(LADDER))
            rate = LADDER[max(0, min(len(LADDER) - 1, idx))] if started else LADDER[0]
        dl_time = rate * SEG / bw                       # seconds to fetch the segment
        if started:
            if dl_time > buf:
                t_stall += dl_time - buf
                stalls += 1
                buf = 0.0
            else:
                buf -= dl_time
        buf += SEG
        started = True
        if buf > MAXBUF:
            buf = MAXBUF
        ests = bw if ests is None else 0.5 * ests + 0.5 * bw
        if last_bitrate is not None and rate != last_bitrate:
            switches += 1
        last_bitrate = rate
        chosen.append(rate)
    return chosen, stalls, t_stall, switches


print("policy       avg kbit/s  stalls  stall s  switches")
res = {}
for p in ("fixed5000", "throughput", "buffer"):
    c, s, ts, sw = run(p)
    res[p] = c
    print("%-12s %9.0f %7d %8.1f %9d" % (p, sum(c) / len(c), s, ts, sw))
```

Run (Python 3, deterministic) TERMINAL

```
$ python3 AbrSim.py
policy       avg kbit/s  stalls  stall s  switches
fixed5000          5000      48    296.2         0
throughput         1847       0      0.0         8
buffer             2960       0      0.0        32
```

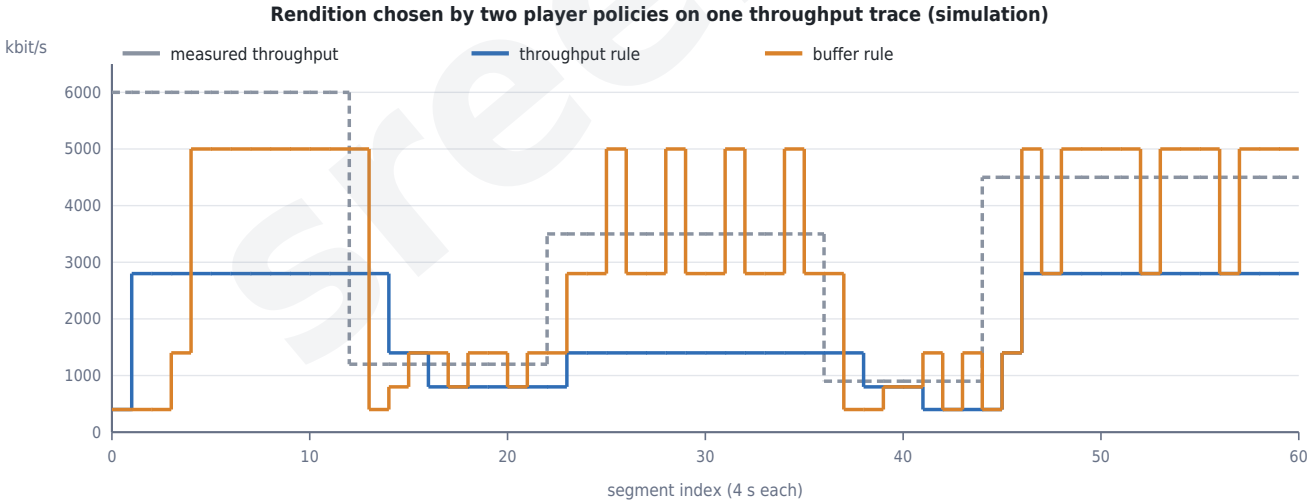


Figure 2.49. The throughput rule follows the available bandwidth conservatively and switches rarely; the buffer rule reaches higher bitrates but oscillates, because the buffer level fluctuates around a mapping boundary.

What to notice: the fixed 5,000 kbit per second policy stalls on 48 of 60 segments and accumulates 296 seconds of rebuffering, which shows why a manifest with a single rendition is unusable on variable networks. Neither adaptive policy stalled. The buffer rule delivered a higher average bitrate (2,960 against 1,847 kbit per second) at the price of 32 rendition switches against 8, and frequent switching is itself a quality penalty. Production players combine both signals and add a switching penalty.

CDN design and origin protection

Popularity of video is heavy tailed: a small fraction of titles accounts for most views. The head is served from the edge (the assumed 90 percent hit ratio); the long tail is served from regional caches or the origin. Three mechanisms protect the origin: a cache hierarchy (edge, regional, origin), request coalescing (concurrent misses for one object at one cache produce a single upstream request), and prefetching of the first segments of newly published popular titles. Some operators additionally place cache servers inside access networks, close to viewers; this trades capital cost for lower transit cost and latency.

2.12.7 Bottlenecks, failures and real-world problems

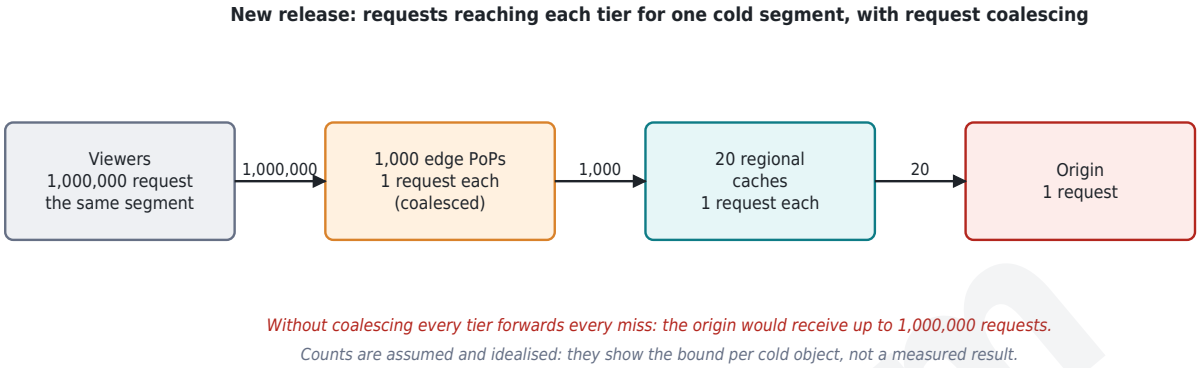


Figure 2.50. With coalescing, the origin receives one request per cold segment regardless of audience size; the numbers are an idealised upper bound under the stated topology.

Origin overload on a new release

Scenario. A highly anticipated video is published at a fixed time and a million viewers start playback within seconds. **Symptoms.** Origin egress and CPU saturate, edge caches log upstream timeouts, playback start times exceed 10 seconds, and unaffected titles also degrade because they share the origin. **Diagnosis.** The origin request rate for a small set of object keys is orders of magnitude above normal; cache hit ratio for those keys is near zero. **Root cause.** Every edge that missed forwarded its own request (no coalescing, or coalescing disabled for range requests), and the first segments were not warmed. **Fix.** Enable request coalescing at edge and regional tiers; prefetch the first segments of a scheduled release to the edge before publication; serve stale content while revalidating. **Prevention.** Schedule publication through a pipeline that warms caches, and load test with a synthetic flash crowd on a cold cache.

Poison video in the transcoding queue

Scenario. A malformed file crashes the encoder on one chunk. **Symptoms.** The same task fails and is retried indefinitely; workers are repeatedly consumed; the queue backlog grows. **Diagnosis.** The `attempts` column of one task is rising; worker logs show the same input and the same crash signature. **Root cause.** Unbounded retries of a deterministic failure. **Fix.** Cap attempts, route the task to a dead-letter state with the input and logs, mark the video FAILED with a reason shown to the creator, and sandbox decoders because untrusted media files are an attack surface. **Prevention.** Validate container and codec during the probe step, set time limits on tasks, and fuzz the decoder pipeline.

Abandoned multipart uploads

Scenario. Millions of uploads are started and never completed. **Symptoms.** Storage cost grows without matching published videos; the object store holds incomplete parts that do not appear in normal listings. **Diagnosis.** Compare bytes stored in the raw bucket with bytes of completed videos; list incomplete multipart uploads. **Root cause.** Parts uploaded and never assembled remain billable. **Fix.** A lifecycle rule that aborts incomplete multipart uploads after a fixed period (for example 7 days) and a job that deletes `upload_sessions` rows whose `expires_at` has passed. **Prevention.** Set expiry at session creation and monitor the ratio of started to completed uploads.

Quality collapse from an aggressive player policy

Scenario. A new player release raises the target bitrate. **Symptoms.** The rebuffering ratio increases after the release, without any infrastructure change. **Diagnosis.** Telemetry segmented by player version shows stall rates higher for the new version at equal network conditions. **Root cause.** The policy selects rungs above sustainable throughput, as the fixed policy does in the simulation (48 stalls). **Fix.** Roll back or correct the policy; add a safety factor and a buffer floor. **Prevention.** Release player changes gradually with quality metrics as the rollback criteria.

2.12.8 Trade-offs and extensions

- Transcoding all renditions eagerly maximises playback quality and minimises start time but spends compute and storage on videos that are never watched; lazy transcoding of unpopular titles reduces cost at the price of a slow first view.
- Larger segments lower request volume and improve compression but slow adaptation; low-latency streaming (live) needs partial segments and a different design.
- Newer codecs reduce bitrate for equal quality but increase encoding cost and require device support, so a second ladder is maintained for capable devices.
- Extension: live streaming adds an ingest protocol, a real-time packager and tighter latency budgets; the CDN design carries over.
- Extension: digital rights management encrypts segments and delivers keys through a license server; the CDN serves ciphertext.

2.12.9 Questions to expect

Why not stream the original file?

The original has one bitrate and codec, often unsupported by the viewer's device, and a size that cannot adapt to bandwidth. A ladder of renditions with short segments allows each viewer to receive what the connection sustains.

Why transcode in chunks?

Chunks make wall-clock latency independent of video length, limit the cost of a failure to one task, and allow elastic use of workers. Chunk boundaries must coincide with keyframes to avoid visible seams.

How does seeking work?

Seeking maps a time offset to a segment number through the manifest (segment duration is fixed or listed), and the player requests that segment directly. No server-side state is needed.

How is the CDN hit ratio improved?

By a cache hierarchy, request coalescing, consistent hashing of objects to cache servers within a site (to avoid duplicate copies), prefetching, and long time-to-live values for immutable segments, which never change once written.

How are views counted at this scale?

Players emit events to a log; a stream processor aggregates them into counters per video and time window, and the counters are written to the metadata store periodically. Exact real-time counts are not required, so eventual consistency is acceptable.

How are resumable uploads implemented?

By multipart upload: the client uploads numbered parts independently, the server tracks which parts exist, and a retry resends only the missing parts. The completion call assembles the object.

What determines the cost of the system?

Delivery bandwidth dominates (75 PB per day in the estimate), then storage of renditions (390 TB per day added), then transcoding compute. Per-title encoding, better codecs and a high hit ratio at the edge reduce the largest item.

KEY POINTS

Key points

- Delivery bytes exceed ingest bytes by about 300 times in the estimate; the CDN, not the origin, carries the audience.
- Upload directly to object storage with multipart pre-signed URLs; keep application servers out of the byte path.
- Parallelise transcoding as a DAG of idempotent chunk-by-rendition tasks with leases, retry limits and a dead-letter state.
- Adaptive bitrate is a client-side control loop over short segments; fixed bitrate fails on variable networks.
- Protect the origin with a cache hierarchy, request coalescing and prefetching of new releases.

sreram

2.13 Design: Ride Hailing and Proximity Service

A ride-hailing service connects a rider with a nearby available driver and tracks the trip. The distinguishing characteristics are a continuous stream of location updates from a very large number of moving devices, a spatial query ("which drivers are within 1.5 km of this point?") that must be answered in milliseconds, and a matching step in which two parties must agree and exactly one assignment may result. The same spatial machinery underlies other proximity services such as restaurant search or delivery. The central decisions are the spatial index, the way the index is sharded under a write-heavy load, and the protocol that prevents double assignment.

2.13.1 Requirements and scope

Functional requirements

- Drivers go online and report their location periodically; riders see nearby drivers on a map.
- A rider requests a ride with a pickup point and destination; the system selects a driver, offers the trip, and confirms on acceptance.
- Both parties see live position during the pickup and the trip; the trip ends with a fare calculation.
- Trips can be cancelled by either party, with defined state transitions.

Non-functional requirements

- Matching latency (request to offer sent) below 2 seconds at p95 in a normal market.
- Location freshness: a driver position in the index is at most 5 seconds old.
- A driver receives at most one active offer at a time, and a trip is assigned to at most one driver, even under concurrent requests.
- Availability above 99.99 percent for request and dispatch; the map view may degrade to stale data.

Out of scope

Pricing and surge computation (represented by an input to the ETA step), payments, driver onboarding, ratings, ride pooling, and map rendering. Routing is consumed as a service that returns a travel time.

Assumptions

All volumes are assumed: 10 million trips per day, 1 million drivers online at the daily peak, a location update every 4 seconds, 50 million map sessions per day of 30 seconds each with a refresh every 5 seconds, and a rush-hour peak factor of 5 for trip requests and 3 for map queries.

2.13.2 Capacity estimation

Quantity	Arithmetic	Result
Location updates (peak)	1,000,000 drivers / 4 s	250,000 per s
Update bandwidth	250,000 x 60 B	15 MB per s
Location index memory	1,000,000 x 100 B	100 MB
Trip requests (average / peak)	10e6 / 86,400 ; x 5	116 / 580 per s
Dispatch searches (peak, 3 attempts per trip)	580 x 3	about 1,740 per s
Map queries (average)	50e6 sessions x 6 queries / 86,400	3,472 per s
Map queries (peak)	3,472 x 3	about 10,400 per s
Trip trace storage	10e6 trips x 300 points x 40 B	120 GB per day
Trip rows	10e6 x 500 B	5 GB per day

Capacity estimate (inputs assumed).

The workload is write dominated on the location path (250,000 updates per second against about 12,000 spatial queries per second) and the index is small (100 MB). The data is ephemeral: a lost position is replaced within 4 seconds by the next update, so the index does not require durable storage or synchronous replication. The difficulty is throughput and spatial locality, not volume. One in-memory node may sustain on the order of 50,000 simple updates

per second (an assumption to be validated by load testing), so at least 5 shards are required for the update rate; the design uses more to leave headroom and to isolate regions.

2.13.3 API design

Endpoint	Request / response	Notes
`POST /v1/drivers/location`	Body: lat, lon, heading, speed, ts. Response 204	Sent every 4 s; carried over a persistent connection in practice
`GET /v1/nearby-drivers`	Query: lat, lon, limit (default 8). Response: drivers[] with position and heading	Map view; slightly stale data is acceptable
`POST /v1/trips`	Header Idempotency-Key. Body: pickup, destination, product. Response 201: trip_id, state	Retried requests return the same trip
`GET /v1/trips/{id}`	Response: state, driver, ETA, position	Also pushed over WebSocket
`POST /v1/trips/{id}/cancel`	Response 200	Compare-and-set on state
`POST /v1/offers/{id}/accept`	Driver only. Response 200 or 409	409 if the offer expired or the trip was cancelled

Rider, driver and internal APIs.

The idempotency key on trip creation prevents a rider who taps twice, or whose client retries, from creating two trips. The 409 response on accept is a normal outcome, not an error condition: it reports that another transition won a race.

2.13.4 Data model

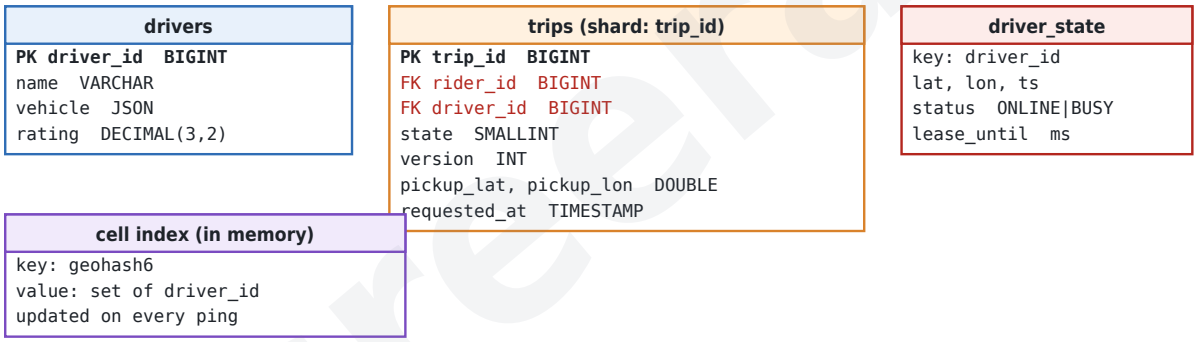


Figure 2.51. Trips are durable, relational records with a version column for compare-and-set; the location data is in memory and keyed by geohash cell.

The trips table is sharded by trip_id and carries the state and a version number; every state change executes `UPDATE trips SET state = ?, version = version + 1 WHERE trip_id = ? AND state = ? AND version = ?`. Driver state (position, availability, offer lease) lives in a key-value store keyed by driver_id and the cell index maps a geohash cell to the set of drivers within it. When a driver moves from one cell to another, the update removes the driver from the old cell set and adds it to the new one. Trip traces (the sequence of positions during a trip) are appended to a log and archived to object storage for disputes and analytics; they are not read on the hot path.

2.13.5 High-level architecture

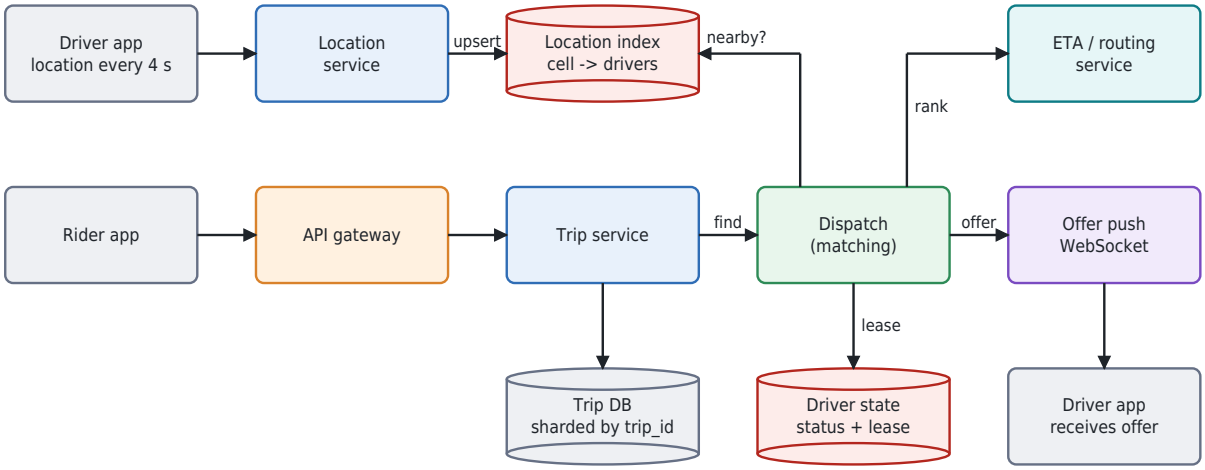


Figure 2.52. Location updates (top row) feed an in-memory index; the dispatch service (middle row) consults the index and the ETA service, leases a driver in the driver state store, and pushes the offer.

Location update flow

1. The driver app sends a position every 4 seconds over a persistent connection to the location service, which validates it and computes the geohash cell.
2. The service upserts the driver record in the shard that owns the cell prefix and moves the driver between cell sets when the cell changes. The same event is appended to a log for trip traces and analytics.

Ride request and dispatch flow

1. The rider app calls `POST /v1/trips`. The trip service inserts the row with state REQUESTED and calls dispatch.
2. Dispatch queries the index for drivers within a radius (1.5 km, widened stepwise if too few) whose status is ONLINE and who hold no lease. It ranks the candidates by estimated time of arrival obtained from the ETA service.
3. Dispatch leases the best candidate (an atomic set of `lease_until` if empty), sends the offer through the push channel with a time-to-live of 10 seconds, and waits.
4. If the driver accepts, the trip row moves from REQUESTED to ACCEPTED by compare-and-set; the rider receives the driver details. If the driver declines or the offer expires, the lease is released and the next candidate is tried.

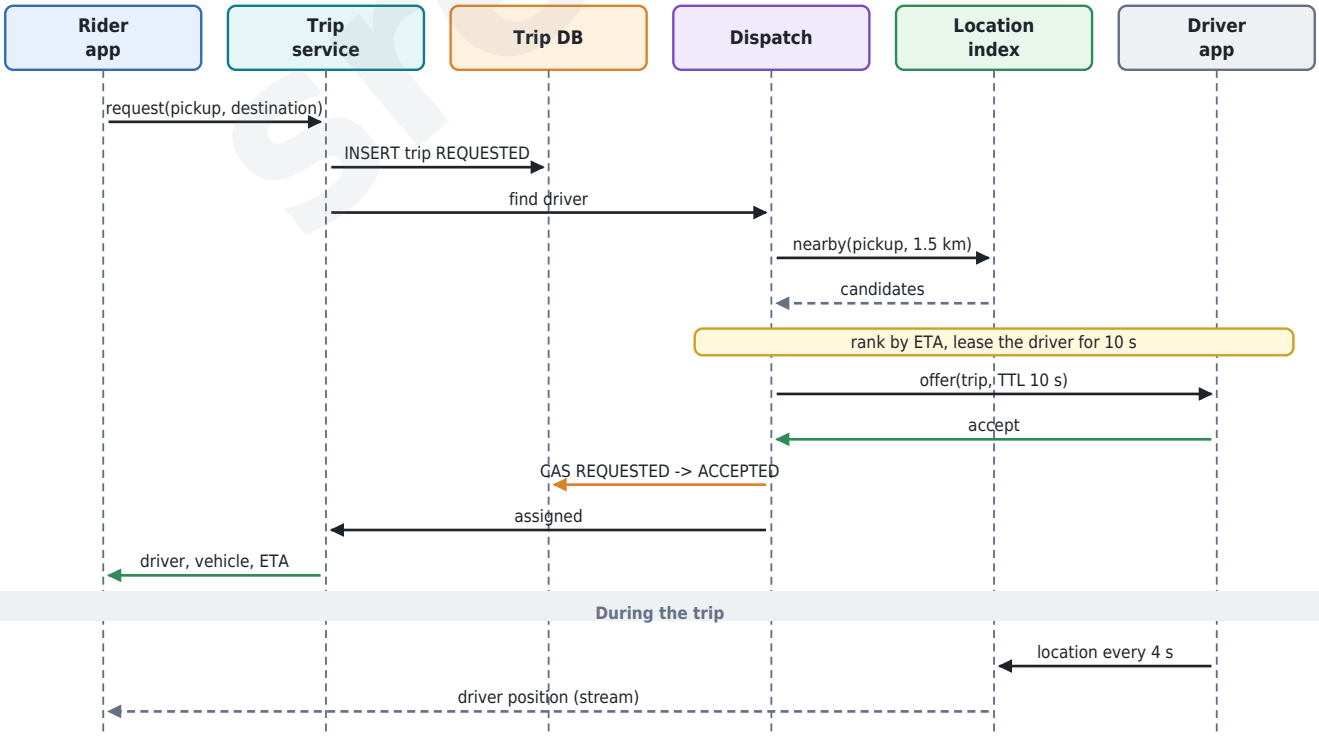


Figure 2.53. The accept step is a compare-and-set on the trip row, which makes a late accept after a cancellation fail cleanly; the position stream bypasses the trip service.

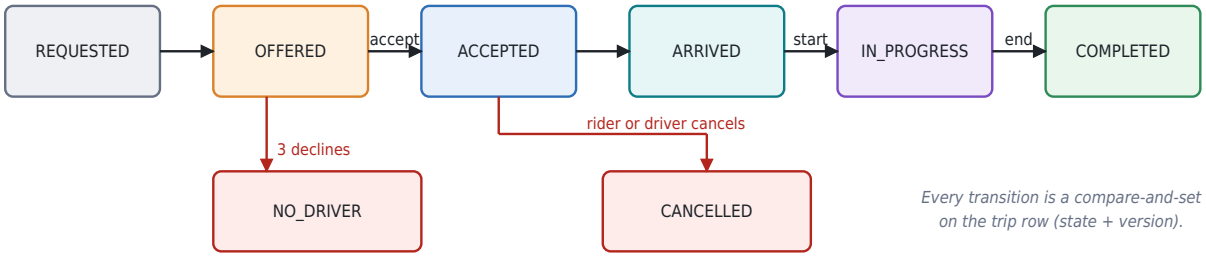


Figure 2.54. The trip life cycle; NO_DRIVER and CANCELLED are terminal states reached from OFFERED and ACCEPTED respectively.

2.13.6 Deep dive

Choosing the spatial index

Index	Mechanism	Strength	Weakness
Geohash grid	Base-32 string encoding of a rectangular cell; prefixes nest	Simple, string keys work in any store, prefix = parent cell	Cells have different widths with latitude; neighbours across prefix boundaries need extra computation
Quadtree	Recursive four-way split adaptive to density	Adapts to dense areas	Tree must be maintained under constant movement; harder to shard
H3 hexagonal grid	Hexagonal cells with a 64-bit index and 16 resolutions	Uniform neighbour distance, good for aggregation and pricing	Requires a library; cells do not nest exactly
S2 cells	Hilbert-curve ordered cells on a sphere	Range scans over cell ids; used for region covering	Library dependency; more complex
Database spatial index (R-tree)	Bounding boxes with a tree	Rich queries	Poor fit for 250,000 updates per second

Spatial indexing alternatives.

Per Uber's engineering blog post on H3, Uber uses H3 hexagons to bucket events and to compute pricing and dispatch signals by measuring supply and demand within cells; the post states that a hexagon has a single distance to all its neighbours, which simplifies analysis. Any of the grid indexes is adequate for the nearby-driver query. Decision: a fixed-precision grid with a hash map from cell to driver set, because the updates are frequent and a hash map update is O(1), where a tree rebalances. Geohash is used in the following demonstration because it is simple to implement completely; the design is unchanged with H3.

Verifying the grid search

The program below generates 100,000 drivers in a 40 km by 40 km area, indexes them by geohash cell (precision 6, a cell of about 1.19 km by 0.61 km at this latitude), and answers 1,000 radius queries both through the grid and by scanning all drivers. It checks that the two methods return identical results and compares the work done.

NearbyDrivers.java JAVA

```
import java.util.*;

public class NearbyDrivers {
    static final String BASE32 = "0123456789bcdefghjkmnpqrstuvwxyz";
    static final int PRECISION = 6;                       // 30 bits: 15 lon, 15 lat
    static final double LAT_STEP = 180.0 / (1 << 15);     // about 0.61 km
    static final double LON_STEP = 360.0 / (1 << 15);     // about 1.19 km at 13 degrees latitude

    static String geohash(double lat, double lon) {
        double latLo = -90, latHi = 90, lonLo = -180, lonHi = 180;
        StringBuilder sb = new StringBuilder();
        boolean evenBit = true;
        int bit = 0, ch = 0;
        while (sb.length() < PRECISION) {
            if (evenBit) {
                double mid = (lonLo + lonHi) / 2;
                if (lon >= mid) { ch = (ch << 1) | 1; lonLo = mid; } else { ch <<= 1; lonHi = mid; }
            } else {
                double mid = (latLo + latHi) / 2;
                if (lat >= mid) { ch = (ch << 1) | 1; latLo = mid; } else { ch <<= 1; latHi = mid; }
```

NearbyDrivers.java (continued) JAVA

```
            }
            evenBit = !evenBit;
            if (++bit == 5) { sb.append(BASE32.charAt(ch)); bit = 0; ch = 0; }
        }
        return sb.toString();
    }

    static double km(double lat1, double lon1, double lat2, double lon2) {   // haversine
        double dLat = Math.toRadians(lat2 - lat1), dLon = Math.toRadians(lon2 - lon1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) + Math.cos(Math.toRadians(lat1))
            * Math.cos(Math.toRadians(lat2)) * Math.sin(dLon / 2) * Math.sin(dLon / 2);
        return 6371.0 * 2 * Math.asin(Math.sqrt(a));
    }

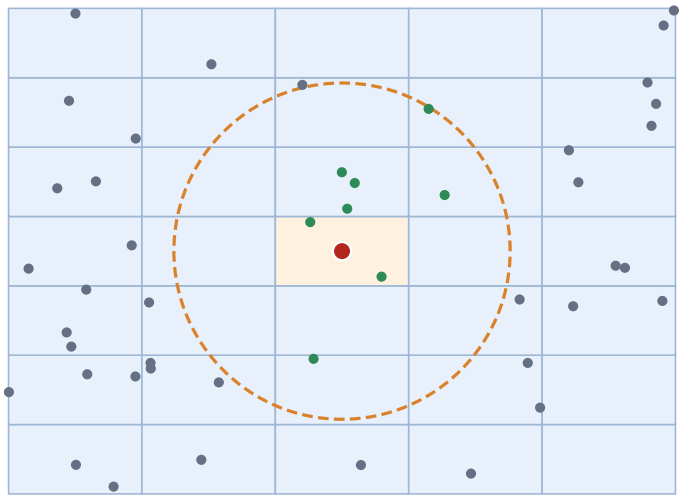
    record Driver(int id, double lat, double lon) {}

    public static void main(String[] args) {
        Random rnd = new Random(7);
        double lat0 = 12.97, lon0 = 77.59;                // city centre; box of about 40 km x 40 km
        List<Driver> all = new ArrayList<>();
        Map<String, List<Driver>> index = new HashMap<>();
        for (int i = 0; i < 100_000; i++) {
            double lat = lat0 + (rnd.nextDouble() - 0.5) * 0.36;
            double lon = lon0 + (rnd.nextDouble() - 0.5) * 0.36;
            Driver d = new Driver(i, lat, lon);
            all.add(d);
            index.computeIfAbsent(geohash(lat, lon), k -> new ArrayList<>()).add(d);
        }
        System.out.println("drivers=" + all.size() + " non-empty cells=" + index.size()
            + " avg per cell=" + String.format("%.1f", all.size() / (double) index.size()));
        System.out.println("rider cell " + geohash(lat0, lon0) + ", cell about 1.19 km x 0.61 km");

        double radiusKm = 1.5;
        int rows = (int) Math.ceil(radiusKm / 0.61), cols = (int) Math.ceil(radiusKm / 1.19);
        int queries = 1000;
        long examined = 0, mismatches = 0, found = 0;
        long tGrid = 0, tBrute = 0;
        for (int q = 0; q < queries; q++) {
            double lat = lat0 + (rnd.nextDouble() - 0.5) * 0.30;
            double lon = lon0 + (rnd.nextDouble() - 0.5) * 0.30;
            long t0 = System.nanoTime();
            Set<Integer> viaGrid = new TreeSet<>();
            for (int dr = -rows; dr <= rows; dr++) {
                for (int dc = -cols; dc <= cols; dc++) {
                    String key = geohash(lat + dr * LAT_STEP, lon + dc * LON_STEP);
                    List<Driver> cell = index.get(key);
                    if (cell == null) continue;
                    for (Driver d : cell) {
                        examined++;
                        if (km(lat, lon, d.lat(), d.lon()) <= radiusKm) viaGrid.add(d.id());
                    }
                }
            }
            long t1 = System.nanoTime();
            Set<Integer> viaScan = new TreeSet<>();
            for (Driver d : all) {
                if (km(lat, lon, d.lat(), d.lon()) <= radiusKm) viaScan.add(d.id());
            }
            long t2 = System.nanoTime();
            tGrid += t1 - t0; tBrute += t2 - t1;
            if (!viaGrid.equals(viaScan)) mismatches++;
            found += viaScan.size();
        }
        System.out.printf("queries=%d radius=%.1f km cells per query=%d%n", queries, radiusKm,
            (2 * rows + 1) * (2 * cols + 1));
        System.out.printf("avg drivers inside radius=%.1f%n", found / (double) queries);
        System.out.printf("grid examined per query=%.0f vs brute force=%d%n",
            examined / (double) queries, all.size());
        System.out.println("result mismatches vs brute force=" + mismatches);
        System.out.printf("avg time per query: grid=%.3f ms brute=%.3f ms (typical)%n",
            tGrid / 1e6 / queries, tBrute / 1e6 / queries);
    }
}
```

Run (JDK 21; the timing line is typical for the build machine and varies) TERMINAL

```
$ java NearbyDrivers.java
drivers=100000 non-empty cells=2178 avg per cell=45.9
rider cell tdr1v9, cell about 1.19 km x 0.61 km
queries=1000 radius=1.5 km cells per query=35
avg drivers inside radius=453.3
grid examined per query=1632 vs brute force=100000
result mismatches vs brute force=0
avg time per query: grid=0.265 ms brute=8.397 ms (typical)
```



5 columns x 7 rows = 35 cells scanned

Radius query with a geohash grid

- rider (red) at the centre cell, precision 6
- candidate inside 1.5 km radius
- scanned but outside the radius (rejected)

cell size: 1.19 km x 0.61 km
rows = ceil(1.5 / 0.61) = 3 each side
columns = ceil(1.5 / 1.19) = 2 each side
step 1: compute the cell keys
step 2: read the 35 cell lists
step 3: exact distance filter, sort by ETA

(illustrative: 8 inside, 38 outside)

Figure 2.55. A radius query reads the 35 cells that cover the circle and filters by exact distance; most scanned drivers are rejected, but the scan examines about 1.6 percent of the population.

What to notice: the grid examines 1,632 drivers per query on average against 100,000 for the scan, with zero mismatches in results. The ratio of scanned to accepted drivers (1,632 to 453) reflects that cells overcover the circle; a finer precision reduces this at the price of more cells per query. The cell neighbourhood is derived from the radius divided by the cell dimensions, which is why the row and column counts differ (3 and 2): cells are twice as wide as they are tall.

Sharding the index

Shard key	Update path	Query path
driver_id (hash)	Single shard, perfectly balanced	Every query is scattered to all shards and merged
Geohash prefix (precision 4, about 39 km x 20 km)	Single shard	One shard, or two to four near a boundary
City or region	Single shard	Single shard; large cities are hot

Sharding the location index.

Decision: shard by geohash prefix, mapping prefixes to shards by consistent hashing. A nearby query is spatially local, so reading one to four shards is cheaper than scattering to all of them. Updates are also local. The price is hot regions: a prefix covering a city centre receives a disproportionate share of updates. A hot prefix is split by moving to a longer prefix (precision 5) as the unit of assignment.

Matching and the offer protocol

Two dispatch strategies are common. Greedy dispatch offers the trip to the nearest driver at once; it minimises the rider's wait for that request but can leave later requests with poor matches. Batched dispatch collects the requests of a short window (for example 2 seconds) and solves an assignment problem between riders and drivers, minimising total pickup time; it improves aggregate efficiency at the cost of up to the window length in added latency. Greedy dispatch is the baseline; batching is an extension for dense markets. Correctness is carried by two mechanisms. First, the driver lease: setting `lease_until` is an atomic set-if-absent (or compare-and-set) in the driver state store, so two concurrent requests cannot both offer to one driver. The lease expires by itself after the offer time-to-live, which handles a driver app that crashed. Second, the trip transition: the move from REQUESTED to ACCEPTED is a compare-and-set on the trip row, so an accept that arrives after a cancellation or after another driver was assigned is rejected with 409.

2.13.7 Bottlenecks, failures and real-world problems

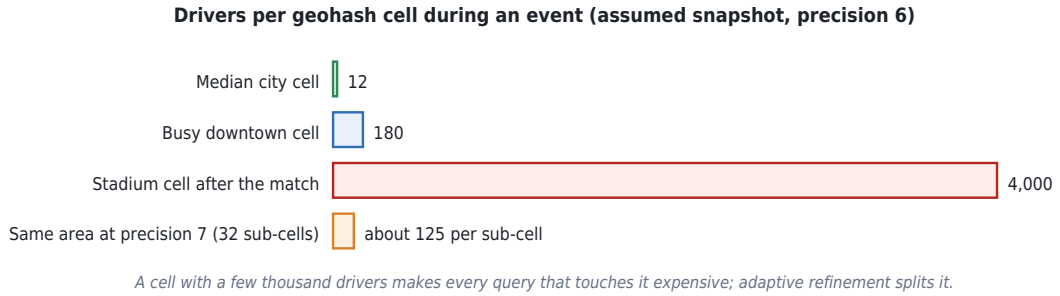


Figure 2.56. Cell population is heavy tailed; one event can place thousands of drivers in a single cell, which multiplies the cost of every query that touches it.

Hot cell at an event

Scenario. A stadium match ends; thousands of drivers and riders converge on the same few cells. **Symptoms.** Latency of nearby queries and of the owning shard rises; dispatch time exceeds the objective; the shard's CPU is saturated while others are idle. **Diagnosis.** Per-cell driver counts and per-shard update rates show a single prefix with a rate far above the median. **Root cause.** A fixed-resolution grid and a prefix-to-shard assignment cannot adapt to density. **Fix.** Adaptive refinement: cells above a population threshold (for example 500 drivers) are indexed at a finer precision, and the shard assignment unit becomes the longer prefix; dispatch limits the candidate list to the nearest N after a partial scan. **Prevention.** Monitor the maximum cell population and shard update rate, and rehearse load with a synthetic crowd.

Double assignment

Scenario. Two riders request at the same moment and both searches select the same nearby driver. **Symptoms.** The driver receives two offers, or two trips show the same driver. **Diagnosis.** Two trip rows reference one driver_id with overlapping active states. **Root cause.** The check "driver is available" and the later assignment were separate, unsynchronised steps. **Fix.** An atomic lease acquisition on the driver before the offer, and a compare-and-set on the trip row at acceptance; the losing request continues to the next candidate. **Prevention.** Make every transition a conditional write, and run a periodic invariant check ("at most one active trip per driver") that alerts on violation.

Stale or noisy positions

Scenario. A driver enters a tunnel or loses connectivity, or the GPS jumps. **Symptoms.** Riders are matched with drivers who are far away, or the map shows cars crossing water. **Diagnosis.** Last update timestamps in the index are older than the freshness bound; consecutive positions imply impossible speeds. **Root cause.** The index retains the last known position indefinitely, and raw GPS contains outliers. **Fix.** Expire entries older than 10 seconds from queries, reject updates implying speeds above a threshold, and smooth positions with a filter (for example, map matching to roads). **Prevention.** Include position age in the ranking function and in telemetry.

Retry storm on region failover

Scenario. A shard fails and its prefixes are reassigned. **Symptoms.** Drivers in the affected area vanish from the map for several seconds; dispatch returns no candidates and riders retry repeatedly. **Diagnosis.** A burst of trip creations with the same rider ids, no candidates in the affected prefixes, and an empty index on the new owner. **Root cause.** The index is rebuilt from incoming pings only, so it is empty until each driver reports once, while rider apps retry aggressively. **Fix.** The new owner accepts updates immediately (the index is complete after one 4-second cycle); dispatch widens its radius and queues requests briefly instead of failing; clients use backoff with jitter. **Prevention.** Keep a warm standby that consumes the same update stream, so failover does not start from an empty index.

2.13.8 Trade-offs and extensions

- Freshness against cost: a shorter update interval improves positions and increases load linearly; adaptive intervals (slower for stationary drivers) save traffic and battery.
- Durable against ephemeral location state: not persisting the index sacrifices nothing that the next ping does not restore, but trip traces, which are needed for disputes, are persisted separately.
- Greedy against batched matching: batching improves global efficiency and increases latency.
- Extension: ride pooling adds a route-insertion problem (insert a new pickup into an existing route under detour limits) to dispatch.

- Extension: the same index serves delivery couriers, scooters and restaurant search by changing the object type and filters.

2.13.9 Questions to expect

Why a grid and not a database spatial index?

The update rate (250,000 per second) is the constraint. A grid with a hash map updates in constant time and shards by prefix, while a tree index must be rebalanced and is harder to distribute.

How is the radius chosen and widened?

Start with a radius that normally yields enough candidates (for example 1.5 km), and widen in steps when fewer than N drivers are found, up to a maximum pickup distance that depends on the market.

How is double assignment prevented?

By a driver lease acquired atomically before sending the offer, and by a compare-and-set on the trip state when the offer is accepted.

What if the driver app crashes after accepting?

The accept is stored by the compare-and-set before the response is sent. Missing arrival heartbeats lead the system to reassign the trip after a timeout, and the driver state returns to OFFLINE after missed pings.

How does the rider see the driver moving?

The driver continues to send positions to the location service; the rider app subscribes to the trip's driver position over its connection, and the service pushes updates, interpolated by the client between 4-second updates.

Why is the location index not stored in a database?

The data changes every 4 seconds and is fully replaced by new updates, so durability adds cost without value. A warm standby is sufficient for failover.

How does the design handle the boundary between two shards?

A query whose covering cells include a different prefix reads that shard as well; the number of such boundary queries is small relative to the total because most of the radius lies within one prefix of about 39 km by 20 km.

KEY POINTS

Key points

- The location path is write dominated (250,000 updates per second) and ephemeral; keep the index in memory and shard it by geographic prefix.
- A grid with a hash map answers a radius query by reading the cells that cover the circle and filtering by exact distance.
- Prevent double assignment with an atomic driver lease and a compare-and-set on the trip state.
- Adapt cell resolution to density; fixed cells fail at events with thousands of drivers in one area.
- Treat positions as perishable: expire stale entries, reject outliers, and rebuild the index from the next update cycle after failover.

2.14 Design: Ticket Booking System

A ticket booking system sells a fixed, small inventory (the seats of a show) to a large number of concurrent buyers. Unlike a feed or a video platform, correctness takes precedence over throughput: a seat sold twice is a defect that costs money and trust, while a slow response costs little. The workload is also extremely bursty: sales for a popular event open at a known instant, and hundreds of thousands of users compete for the same few thousand rows. The central decisions are how a seat is reserved without overselling, how temporary holds expire, how payment is reconciled with the hold, and how the burst is admitted.

sreeram

2.14.1 Requirements and scope

Functional requirements

- Users browse events and shows, and view a seat map with the current status of each seat.
- A user selects seats; the system holds them for a limited time (10 minutes) while the user pays.
- On successful payment the booking is confirmed and a ticket is issued; on failure or expiry the seats are released.
- Users can cancel before a cut-off time, with a refund.

Non-functional requirements

- No seat is ever sold to two buyers (strict consistency for inventory).
- Seat map reads are fast (p99 below 100 ms) and may be a few seconds stale.
- The system survives a sale opening with a million users for a 60,000-seat event without collapse, and admits users fairly.
- Booking confirmation is durable: a charged payment always results in a ticket or a refund.

Out of scope

Event creation and venue layouts, pricing rules and dynamic pricing, ticket transfer and resale, and fraud scoring beyond rate limits.

Assumptions

All volumes are assumed: 50,000 shows per day with 200 seats on average, 1 million bookings per day of 2.5 seats each, 50 million seat-map views per day, a peak factor of 10, a conversion from hold to booking of 40 percent, and a flash sale of 1 million users for 60,000 seats.

2.14.2 Capacity estimation

Quantity	Arithmetic	Result
Bookings per second (average / peak)	1e6 / 86,400 ; x 10	`11.6 / 116 per s`
Seat-map reads (average / peak)	50e6 / 86,400 ; x 10	`579 / 5,790 per s`
Holds per second (peak)	116 / 40 percent conversion	`290 per s`
Seats held at any instant (peak, upper bound)	290 x 2.5 seats x 600 s	`435,000`
Seat rows (30 days ahead)	50,000 shows x 200 seats x 30 days	`300 million (30 GB at 100 B)`
Booking storage	1e6 x 300 B	`300 MB per day`
Flash sale, no admission control	1e6 users x 3 requests / 60 s	`50,000 per s`
Flash sale, waiting room at 500 users per s	500 x 3	`1,500 per s`
Time to admit all users	1e6 / 500 per s	`2,000 s (33 minutes)`

Capacity estimate (inputs assumed).

In normal operation the load is modest: 116 bookings and 290 holds per second are within the capacity of a single well-indexed relational primary, sharded by show for headroom. The design effort is therefore concentrated on two abnormal conditions: the flash sale (50,000 requests per second aimed at a handful of rows) and the correctness of concurrent updates to a single row. Inventory is small, so a relational database with transactions is the natural store; its row-level atomicity is the main tool.

sreeram

2.14.3 API design

Endpoint	Request / response	Notes
`GET /v1/shows/{id}/seats`	Response: seat status map, version	Served from cache; stale by at most 2 s
`POST /v1/shows/{id}/holds`	Header Idempotency-Key. Body: seat_ids[]. Response 201: hold_id, expires_at; 409 with the unavailable seats	All-or-nothing across the requested seats
`POST /v1/holds/{id}/checkout`	Body: payment method. Response 202: payment_id	Moves the hold to PAYMENT_PENDING and starts the charge
`GET /v1/bookings/{id}`	Response: status, ticket	Polled or pushed after the payment webhook
`POST /v1/bookings/{id}/cancel`	Response 200	Releases seats, starts a refund
`POST /v1/payments/webhook`	PSP event with signature	Idempotent by payment_id

Booking API.

A hold request is all-or-nothing: either every requested seat is held, or none is, and the response of the failure case lists which seats were taken so the client can refresh the map. The Idempotency-Key makes a retried request return the original hold instead of failing with 409 because the seats are now held by the same user.

2.14.4 Data model

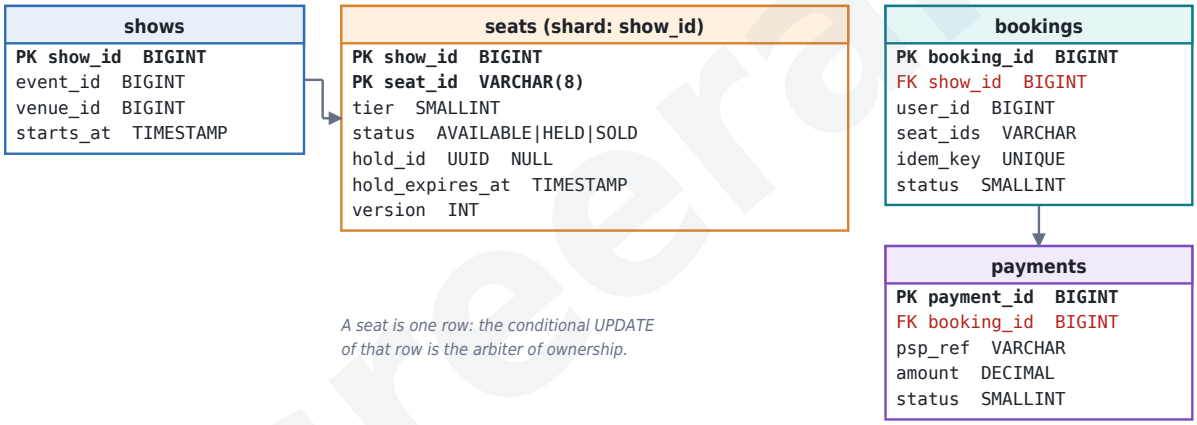


Figure 2.57. A seat is one row; ownership changes only through conditional updates of that row. Bookings carry an idempotency key, and payments reference bookings.

The seats table is the inventory, sharded by show_id so that all seats of a show reside on one shard and a multi-seat hold remains a single-shard transaction. A seat has a status (AVAILABLE, HELD, SOLD), a hold identifier and an expiry time; the expiry time is the authoritative record, so an expired hold is treated as available without any background job having run (lazy expiry). The `version` column supports optimistic concurrency where a client-side read-modify-write is unavoidable. The seat-map cache is a read model derived from the table and is never used to decide ownership.

2.14.5 High-level architecture

sreeram

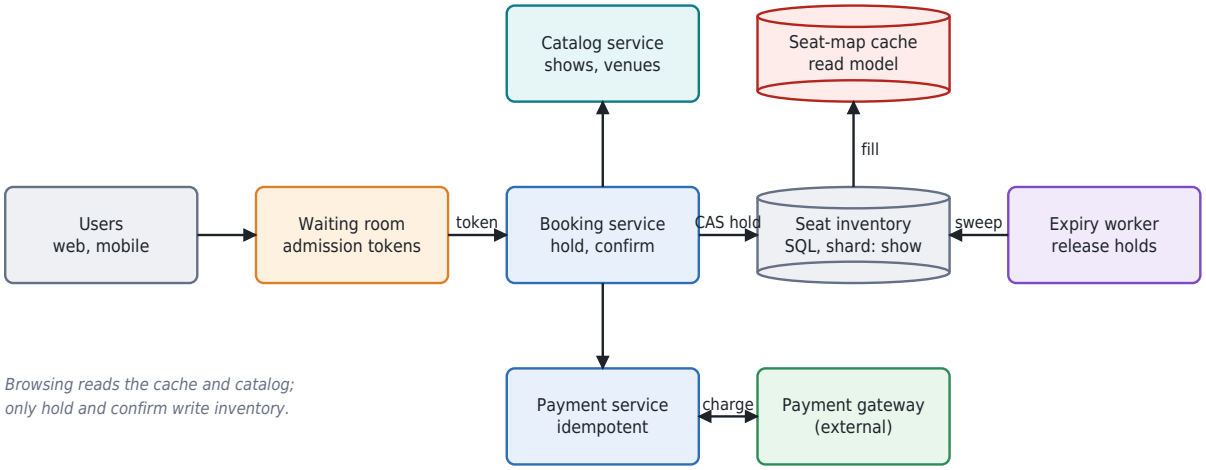


Figure 2.58. Reads go to the catalog and the seat-map cache; the booking service writes the inventory through conditional updates and coordinates with the payment service. A waiting room in front limits the arrival rate during a flash sale.

Booking flow

1. The user opens the seat map, served from the cache, and selects seats.
2. The booking service executes the hold as one transaction: a conditional update of all selected seats (status AVAILABLE, or HELD with an expired time) setting the hold identifier and an expiry 10 minutes ahead. If fewer rows than requested are updated, the transaction rolls back and the response is 409.
3. At checkout the service moves the hold to PAYMENT_PENDING and extends its expiry by a grace period (5 minutes), creates a payment with the hold identifier as idempotency key, and redirects to the payment gateway.
4. The gateway reports the result by webhook. On success the booking service converts the seats to SOLD and inserts the booking in one transaction. On failure the seats return to AVAILABLE.
5. A late success (the hold expired and the seat was resold) cannot be honoured; the payment service refunds automatically.

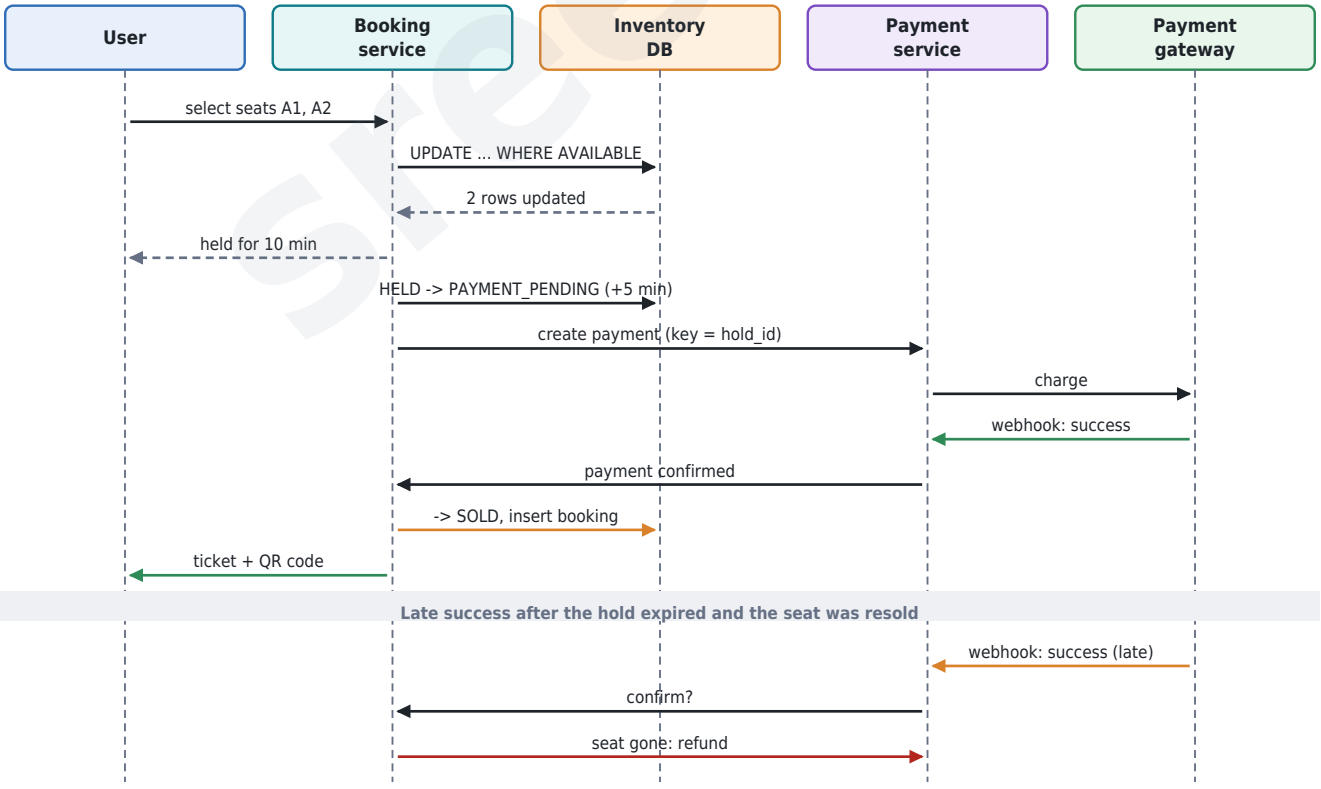


Figure 2.59. The seats are claimed before payment starts; the confirmation is a second conditional write, and a late payment is detected at that write and refunded.

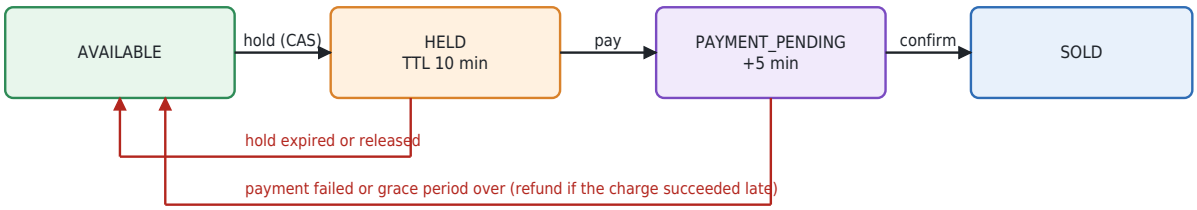


Figure 2.60. A seat moves forward only through conditional updates; every backward arrow is an expiry or a failure, and every forward arrow checks the current state.

2.14.6 Deep dive

Preventing overselling

Approach	Mechanism	Strength	Weakness
Check, then update	Read status, then write if available	Looks natural	Incorrect under concurrency: the check and the write are separate
Conditional UPDATE	`UPDATE ... WHERE status = 'AVAILABLE'`; success = one row changed	Atomic, no explicit lock, fast	Contention retries by the caller
Pessimistic row lock	`SELECT ... FOR UPDATE`, then check and update	Simple reasoning	Holds locks while the transaction runs; deadlocks possible for multiple seats
Unique constraint	Unique index on (show_id, seat_id) in bookings	Last line of defence enforced by the store	Reports violation after the work; not a hold mechanism
In-memory hold (Redis SET NX with expiry)	Fast hold layer in front of the database	Offloads the hot path	A second source of truth; must be reconciled with the database

Concurrency control alternatives for a seat.

The following experiment runs the first alternatives on a real PostgreSQL 16 instance with 20 concurrent psql clients running 200 booking attempts against 2 hot seats. In strategies A, C and D the transaction sleeps 0.3 seconds between the check and the write to widen the race window to a size that the experiment can observe; the logic otherwise corresponds to production code. The scripts are shown first.

naive.sql (strategy A: read committed, no lock) SQL

```sql
BEGIN;
SELECT (status = 'AVAILABLE') AS ok FROM seats WHERE seat_id = :seat \gset
\if :ok
SELECT pg_sleep(0.3) \g /dev/null
UPDATE seats SET status = 'SOLD', owner = :user WHERE seat_id = :seat;
INSERT INTO bookings(seat_id, user_id) VALUES (:seat, :user);
\endif
COMMIT;
```

conditional.sql (strategy B: single conditional statement) SQL

```sql
WITH u AS (
  UPDATE seats SET status = 'SOLD', owner = :user
  WHERE seat_id = :seat AND status = 'AVAILABLE' RETURNING seat_id, owner)
INSERT INTO bookings(seat_id, user_id) SELECT seat_id, owner FROM u;
```

seat_race.sh BASH

```bash
#!/bin/bash
# Seat booking race on PostgreSQL 16: 200 attempts, 2 hot seats, 20 concurrent clients per strategy.
PSQL="psql -X -q -h /tmp/osw_y4/pg -p 22400 -U postgres -d seats"
reset() {
  $PSQL -c "TRUNCATE seats, bookings" \
        -c "INSERT INTO seats SELECT g, 'AVAILABLE', NULL FROM generate_series(1,2) g"
}
report() {
  $PSQL -At -c "SELECT '  seats SOLD=' || (SELECT count(*) FROM seats WHERE status='SOLD') ||
    '  booking rows=' || (SELECT count(*) FROM bookings) ||
    '  oversold seats=' || (SELECT count(*) FROM (SELECT seat_id FROM bookings
```

seat_race.sh (continued) BASH

```
                        GROUP BY seat_id HAVING count(*)>1) s)"
}
run() {  # $1 = sql file
  seq 1 200 | xargs -P 20 -I{} sh -c \
    "$PSQL -v seat=\$(( ({}-1) % 2 + 1 )) -v user={} -f $1 2>/dev/null"
}
echo "A  check, then update (read committed, no lock)";   reset; run naive.sql;       report
echo "B  conditional UPDATE ... WHERE status='AVAILABLE'"; reset; run conditional.sql; report
echo "C  SELECT ... FOR UPDATE, then check";              reset; run forupdate.sql;   report
echo "D  naive logic + UNIQUE(seat_id) on bookings";      reset
$PSQL -c "CREATE UNIQUE INDEX bookings_seat_uq ON bookings(seat_id)"; run naive.sql; report
$PSQL -c "DROP INDEX bookings_seat_uq"
```

Run (PostgreSQL 16.15, local cluster; strategy A count varies between runs) TERMINAL

```
$ ./seat_race.sh
A  check, then update (read committed, no lock)
  seats SOLD=2  booking rows=21  oversold seats=2
B  conditional UPDATE ... WHERE status='AVAILABLE'
  seats SOLD=2  booking rows=2  oversold seats=0
C  SELECT ... FOR UPDATE, then check
  seats SOLD=2  booking rows=2  oversold seats=0
D  naive logic + UNIQUE(seat_id) on bookings
  seats SOLD=2  booking rows=2  oversold seats=0
```

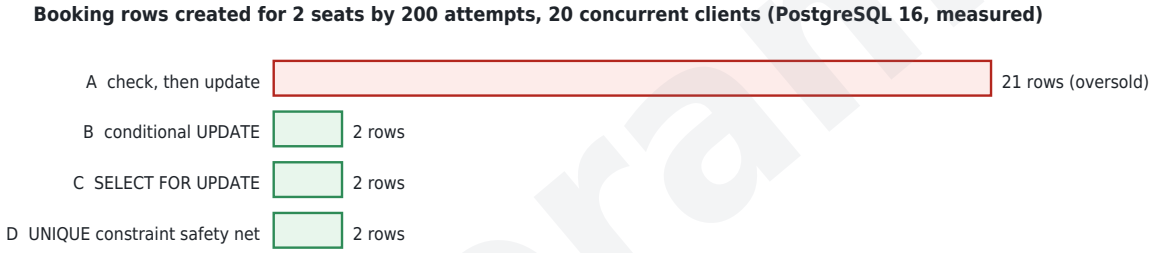


Figure 2.61. Without a guard between the check and the write, concurrent clients each booked seats that others were booking; the conditional update, the row lock and the unique constraint each yield exactly one winner per seat.

What to notice: in strategy A the clients of the first wave read AVAILABLE before any update committed, and each inserted a booking, so the two seats received 21 bookings in this run. The count varies between runs (20 in an earlier run) because it depends on timing and on the concurrency width of the experiment; the zero for strategies B, C and D does not vary. Strategy B needs no explicit lock: the database serialises the two updates on the same row, and the second one re-evaluates the predicate, finds status SOLD and updates zero rows. Strategy C serialises through the row lock at the price of holding it for the whole transaction; with multiple seats per hold, locks must be taken in a consistent order (sorted by seat_id) to avoid deadlock. Strategy D shows the value of a constraint as a final safeguard: the naive logic still ran, but the unique index rejected the duplicate inserts. Decision: the conditional update (strategy B) as the primary mechanism, with the primary key and a unique booking index as safeguards. A Redis hold layer is added only if measurements show the database cannot absorb the hold rate; it adds a second source of truth that needs reconciliation.

Hold expiry and the payment race

A hold must expire without relying on a background job being timely. Storing `hold_expires_at` and treating `HELD` with an elapsed expiry as available (lazy expiry) makes the system correct at any moment; the expiry worker only tidies the seat-map cache and counters. The remaining problem is the time between payment submission and the gateway answer, which can exceed the hold. The checkout step therefore extends the hold by a grace period, and the confirmation step re-checks ownership and expiry. The program below models these rules with a virtual clock.

SeatHolds.java JAVA

```
import java.util.*;

public class SeatHolds {
    enum State { AVAILABLE, HELD, PAYMENT_PENDING, SOLD }

    static class Seat {
        State state = State.AVAILABLE;
        String owner;
```

SeatHolds.java (continued) JAVA

```
        long expiresAt;                    // virtual clock, seconds
    }

    static final Map<String, Seat> seats = new TreeMap<>();
    static long now = 0;
    static final long HOLD_TTL = 600, PAYMENT_GRACE = 300;

    /** A seat is takeable if available or if its hold has expired (lazy expiry). */
    static synchronized boolean hold(String seatId, String user) {
        Seat s = seats.get(seatId);
        boolean free = s.state == State.AVAILABLE
            || ((s.state == State.HELD || s.state == State.PAYMENT_PENDING) && s.expiresAt <= now);
        if (!free) return false;
        s.state = State.HELD; s.owner = user; s.expiresAt = now + HOLD_TTL;
        return true;
    }

    /** Payment started: extend the hold so that the gateway has time to answer. */
    static synchronized boolean startPayment(String seatId, String user) {
        Seat s = seats.get(seatId);
        if (s.state != State.HELD || !user.equals(s.owner) || s.expiresAt <= now) return false;
        s.state = State.PAYMENT_PENDING; s.expiresAt = now + PAYMENT_GRACE;
        return true;
    }

    /** Payment succeeded: confirm only if this user still holds the seat. */
    static synchronized boolean confirm(String seatId, String user) {
        Seat s = seats.get(seatId);
        boolean ok = (s.state == State.PAYMENT_PENDING || s.state == State.HELD)
            && user.equals(s.owner) && s.expiresAt > now;
        if (ok) s.state = State.SOLD;
        return ok;
    }

    static void log(String msg) { System.out.printf("t=%4ds  %s%n", now, msg); }

    public static void main(String[] args) {
        seats.put("A1", new Seat());
        seats.put("B7", new Seat());
        log("alice holds A1: " + hold("A1", "alice"));
        log("bob tries A1: " + hold("A1", "bob") + " (held by alice)");
        now = 590;
        log("alice starts payment: " + startPayment("A1", "alice")
            + ", hold now ends at t=" + seats.get("A1").expiresAt);
        now = 700;
        log("bob tries A1 during payment: " + hold("A1", "bob"));
        now = 880;
        log("gateway answers, confirm(alice): " + confirm("A1", "alice"));
        now = 1000;
        log("carol holds B7: " + hold("B7", "carol"));
        now = 1700;
        log("carol's payment succeeds at t=1700, hold ended at t=1600: "
            + confirm("B7", "carol") + " -> refund required");
        log("dave holds B7 (expired hold is reusable): " + hold("B7", "dave"));
        seats.forEach((id, s) -> log(id + " -> " + s.state + " owner=" + s.owner));
    }
}
```

Run (JDK 21, deterministic: virtual clock) TERMINAL

```
$ java SeatHolds.java
t=   0s  alice holds A1: true
t=   0s  bob tries A1: false (held by alice)
t= 590s  alice starts payment: true, hold now ends at t=890
t= 700s  bob tries A1 during payment: false
t= 880s  gateway answers, confirm(alice): true
t=1000s  carol holds B7: true
t=1700s  carol's payment succeeds at t=1700, hold ended at t=1600: false -> refund required
t=1700s  dave holds B7 (expired hold is reusable): true
t=1700s  A1 -> SOLD owner=alice
t=1700s  B7 -> HELD owner=dave
```

What to notice: Bob cannot take A1 while Alice is paying, because checkout moved it to PAYMENT_PENDING with a later expiry; Alice's confirmation at t=880 succeeds inside the grace period. Carol's payment succeeds at t=1700 after her

hold ended at t=1600; the confirmation returns false and the seat has been given to Dave, so the payment service must refund Carol. This is not an error path to be avoided but one that the design must handle explicitly, because gateway latency is outside the system's control.

Admission control for a flash sale

Approach	Mechanism	Effect
Scale out only	Add instances behind the load balancer	The contention is on the same rows; more instances increase conflicts, not throughput
Rate limit per user and IP	Token bucket at the gateway	Limits abuse; does not shape the aggregate rate
Waiting room (virtual queue)	Users receive a random or ordered position and are admitted at a fixed rate with a signed token	Bounds the arrival rate at the inventory; fair
Allocation by the system	The system selects the best available seats for the party	Spreads conflicts; fewer clients compete for a given seat

Handling a sale-opening burst.

The decisive point is that the bottleneck is logical contention on a small set of rows, not CPU. A waiting room converts 50,000 requests per second into 1,500 (500 admitted users per second at 3 requests each), at the price of a wait of up to 33 minutes for the last user in the assumed event. Admission order is randomised among users who arrived in the first minutes, because strict first-come ordering rewards bots with the fastest connections. Seat selection by the system ("best available") is an effective complement because it avoids a crowd of users all clicking the same front-row seats.

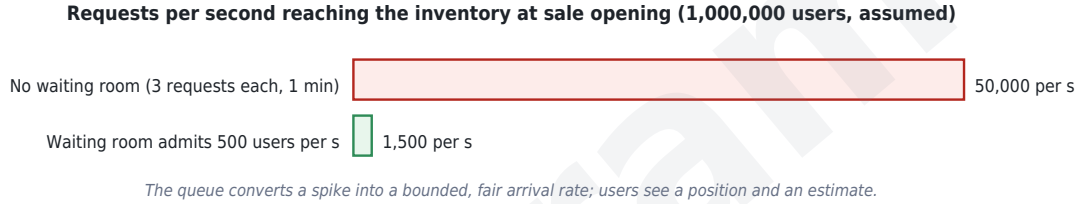


Figure 2.62. Admission control bounds the inventory request rate to a value the database sustains, independently of the number of waiting users.

2.14.7 Bottlenecks, failures and real-world problems

Overselling under concurrency

Scenario. Two users select the last seat in the same second. **Symptoms.** Two confirmation emails for one seat; the venue reports a duplicate at the door. **Diagnosis.** Two booking rows reference the same (show_id, seat_id); the application log shows both transactions reading AVAILABLE. **Root cause.** A read-then-write sequence without a conditional update, lock or constraint. **Fix.** Conditional update with row-count verification; a unique index on booked seats as a safeguard; refund and apology for the affected customer. **Prevention.** A concurrency test that fires parallel requests at the same seat (as in the experiment above) in the continuous integration pipeline.

Abandoned holds block inventory

Scenario. Users select seats and close the browser. **Symptoms.** A show appears sold out while many seats are held; seats reappear only after 10 minutes. **Diagnosis.** The count of HELD seats with future expiry is high and the conversion rate is low; the seat map and the table differ if the cache is stale. **Root cause.** A fixed hold time is applied equally to genuine buyers and to idle sessions. **Fix.** Lazy expiry in the update predicate, a shorter hold at the start of the checkout steps (for example 5 minutes, refreshed on activity), explicit release on page exit, and a cap on seats held per user. **Prevention.** Track hold conversion by show and adapt the hold duration; alert on the held fraction of inventory.

Payment succeeds after the hold expired

Scenario. The gateway answers 40 seconds late; the seat was resold. **Symptoms.** A charged customer without a ticket. **Diagnosis.** A payment with status SUCCESS and no booking; the confirmation attempt returned false. **Root cause.** Time between submission and confirmation exceeded the hold. **Fix.** Extend the hold at checkout; on a failed confirmation refund automatically and notify the user; run a reconciliation job comparing successful payments with bookings. **Prevention.** Use the hold identifier as the payment idempotency key so retries cannot double charge, and monitor the late-confirmation rate.

Hot shard on a popular show

Scenario. All seats of one blockbuster show reside on one shard, which receives most of the sale traffic. **Symptoms.** The shard's CPU and lock wait times rise; other shows on the shard are slow. **Diagnosis.** Lock waits concentrate on one show_id; per-shard QPS is skewed. **Root cause.** The partition key (show_id) concentrates the contention by design, to keep a hold transaction on one shard. **Fix.** Admission control; move the show to a dedicated shard; serve the seat map from a read model; take holds in batches in one statement. **Prevention.** Provision dedicated capacity for announced high-demand sales, and rehearse with synthetic load.

Deadlock on multi-seat holds

Scenario. User X holds seats A1 and A2 while user Y requests A2 and A1. **Symptoms.** Occasional deadlock errors and aborted transactions. **Diagnosis.** The database log reports a deadlock between two transactions locking the same two rows in opposite order. **Root cause.** Row locks acquired in request order. **Fix.** Update seats in a single statement or order the seat identifiers before locking; retry the aborted transaction. **Prevention.** A coding rule: any multi-row write acquires rows in a canonical order.

2.14.8 Trade-offs and extensions

- Strict consistency of inventory limits the write throughput per show to the capacity of one shard; this is acceptable because the inventory is small, and it is protected by admission control.
- A longer hold improves the payment success rate and increases the fraction of inventory that is unavailable to others; the right value is determined by measuring conversion.
- Showing "N seats left" requires either an exact counter (a contention point) or an approximate cached count; most products use the latter.
- Extension: general admission tickets (no seat identity) are a counter per section; the conditional update becomes `UPDATE ... SET remaining = remaining - n WHERE remaining >= n`.
- Extension: waitlists and resale subscribe to release events from the expiry worker.

2.14.9 Questions to expect

How is double booking prevented?

By making ownership transfer a single conditional write on the seat row and verifying that exactly one row changed, with a unique constraint as a safeguard. The check and the write must not be separate steps.

Why not a distributed lock?

A distributed lock adds a service and failure modes (lease expiry, clock issues) for a problem that a single-row conditional update in the inventory database already solves atomically.

How does the seat map stay current?

It is a cache refreshed on every inventory change (or every second or two) and pushed to open clients over server-sent events or polling. Its staleness never affects correctness because the hold step validates against the table.

What if the booking service crashes after the payment succeeded?

The payment webhook is retried by the gateway and is idempotent. A reconciliation job compares successful payments without a booking and either completes the booking (if the hold is valid) or refunds.

How is the flash sale handled?

By a waiting room that admits users at a rate the database sustains, randomised ordering within an arrival window, per-user limits, and seat allocation by the system.

How are bots and scalpers limited?

Per-account and per-device purchase limits, rate limiting, proof-of-work or challenge in the waiting room, and verified accounts. None is fully effective; the design combines several layers.

sreeram

Why is the booking separate from the payment?

The payment depends on an external system with unbounded latency. The hold decouples the scarce resource (seats) from that latency, and the idempotency key and reconciliation make the two-step process safe to retry.

KEY POINTS

Key points

- Seat ownership is decided by a conditional update of one row, verified by its row count; a check followed by a write is a race.
- Holds carry an expiry timestamp and are treated as available after it, so correctness does not depend on a background job.
- Extend the hold at checkout and refund automatically when a payment succeeds after the seat has been resold.
- A waiting room turns an unbounded burst into a bounded, fair arrival rate; scaling out does not remove row contention.
- Keep constraints as a last safeguard and test concurrency explicitly with parallel requests.

2.15 Design: E-Commerce Checkout and Inventory

An e-commerce platform has a read path that is large, cacheable and tolerant of staleness (catalog, search, product pages) and a write path that is small, transactional and intolerant of error (cart conversion, stock, order, payment). This chapter concentrates on the write path from cart to confirmed order. The distinguishing difficulty is that one business transaction spans several services with separate databases (orders, inventory, payments), so no single database transaction can protect it. The central decisions are how inventory is reserved, how the multi-service checkout is coordinated and made recoverable (the saga pattern), and how retries are made safe through idempotency.

2.15.1 Requirements and scope

Functional requirements

- Users browse and search products, maintain a cart (also as guests), and check out.
- Checkout validates the cart, reserves stock, authorises payment, and creates an order.
- Users view order status and may cancel before shipment; cancellation releases stock and refunds the payment.
- Inventory is tracked per SKU and warehouse, with reservations that expire.

Non-functional requirements

- Never sell stock that does not exist (no overselling), never charge a customer twice for one order.
- Catalog read p99 below 100 ms; checkout API p99 below 1 second excluding the external payment call.
- The system tolerates the failure of any single service without losing or duplicating orders; every order reaches a terminal state.
- Availability above 99.95 percent for browsing and cart; checkout may reject new orders during a payment outage.

Out of scope

Search ranking, recommendations, tax and shipping rate computation, warehouse operations, marketplace seller settlement, and returns logistics. Fulfilment is represented as a consumer of order events.

Assumptions

All volumes are assumed: 50 million DAU, 4 product views per user per day, 100 million cart operations per day, 2 million orders per day with 3 items each, 50 million SKUs, a sale-event peak factor of 10 for reads and 20 for orders.

sreera

2.15.2 Capacity estimation

Quantity	Arithmetic	Result
Product views (average / sale peak)	200e6 / 86,400 ; x 10	`2,315 / 23,150 per s`
Cart operations (average / peak)	100e6 / 86,400 ; x 5	`1,157 / 5,787 per s`
Orders (average / sale peak)	2e6 / 86,400 ; x 20	`23 / 463 per s`
Stock reservations (sale peak)	463 orders x 3 items	`about 1,390 per s`
Order events (average)	23 orders per s x 6 events per order	`140 per s`
Order storage	2e6 x 2 KB	`4 GB per day (1.46 TB per year)`
Catalog size	50e6 SKUs x 5 KB	`250 GB`
Stock rows	50e6 SKUs x 4 warehouses x 40 B	`8 GB`
Carts	100e6 x 1 KB	`100 GB`

Capacity estimate (inputs assumed).

The write path is small in absolute terms (463 orders per second at the peak), which a sharded relational store handles comfortably. Contention, not volume, is the problem: a promotion on one SKU concentrates the entire reservation load of the sale on one stock row. The read path, at 23,150 requests per second, is served almost entirely by caches and a CDN.

2.15.3 API design

Endpoint	Request / response	Notes
`PUT /v1/cart/items/{sku}`	Body: qty. Response: cart with prices and availability hints	Idempotent: sets the quantity
`POST /v1/checkout`	Header Idempotency-Key. Body: cart_id, cart_version, address, payment method. Response 202: order_id, state	Starts the saga; the client polls or subscribes
`GET /v1/orders/{id}`	Response: state, items, payment status, tracking	Read from the order store
`POST /v1/orders/{id}/cancel`	Response 202	Triggers compensation or refund
`POST /v1/payments/webhook`	Signed PSP event	Idempotent by event id

Cart, checkout and order API.

The checkout key is derived by the client for each purchase attempt (for example a UUID generated when the checkout screen opens). It is stored in a unique column of the orders table, so a retried POST returns the existing order. The `cart_version` field makes the request fail with 409 if the cart changed after the user reviewed the total, which prevents a price or quantity mismatch between what was shown and what is charged. Per the Stripe API documentation on idempotent requests, a payment provider accepts an `Idempotency-Key` header with a key of up to 255 characters, returns the saved response for a repeated key, and rejects the same key with different parameters; keys may be pruned after at least 24 hours. The payment service of this design forwards the order identifier as that key.

2.15.4 Data model

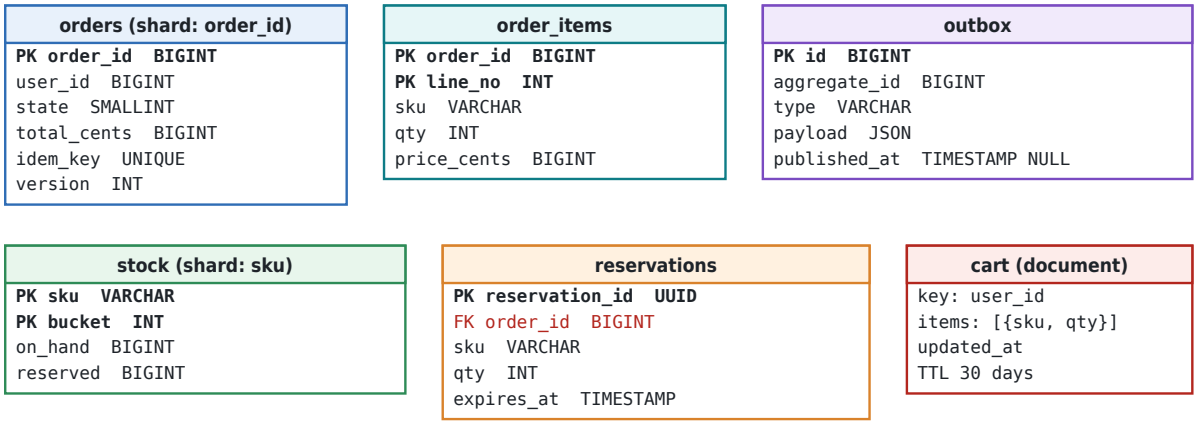


Figure 2.63. Orders are sharded by order_id; the outbox is written in the same transaction as the order change; stock is split into buckets per SKU, and a reservation links an order to the stock it holds.

The order and its outbox rows share a database, so that one local transaction updates the state and records the event to be published (the transactional outbox pattern). A relay publishes unpublished outbox rows to the event bus and marks them; consumers must therefore tolerate duplicates. The stock table holds `on_hand` and `reserved` per (SKU, bucket); the available quantity is `on_hand - reserved`. The cart is a document keyed by user (or by a guest token) with a time to live; it is a convenience record, not a source of truth, and prices in it are re-evaluated at checkout.

2.15.5 High-level architecture

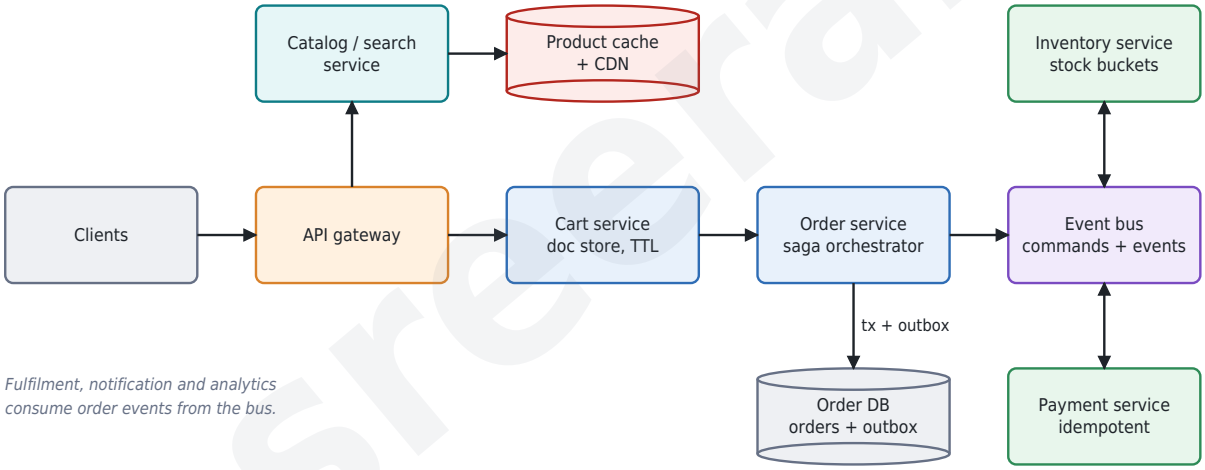


Figure 2.64. Browsing uses the catalog and its caches; checkout enters the order service, which orchestrates inventory and payment through the event bus and persists every state change with an outbox row.

Checkout flow

1. The cart service sends the cart to the order service. The order service validates prices against the catalog, creates the order in state PENDING with the idempotency key, and commits the order and the outbox row in one transaction.
2. The relay publishes a `ReserveStock` command. The inventory service executes a conditional update per item, records a reservation with an expiry, and replies `StockReserved` or `OutOfStock`.
3. On `StockReserved`, the order service issues `AuthorizePayment` with the order id as idempotency key. The payment service calls the gateway and replies `PaymentAuthorized` or `PaymentDeclined`.
4. On success the order moves to CONFIRMED and fulfilment is notified. On `OutOfStock` or `PaymentDeclined` the order service runs the compensations for steps already completed (release the reservation) and the order becomes CANCELLED.
5. Payment is captured when the shipment is created, not at authorisation, so a cancellation before shipment only voids an authorisation.

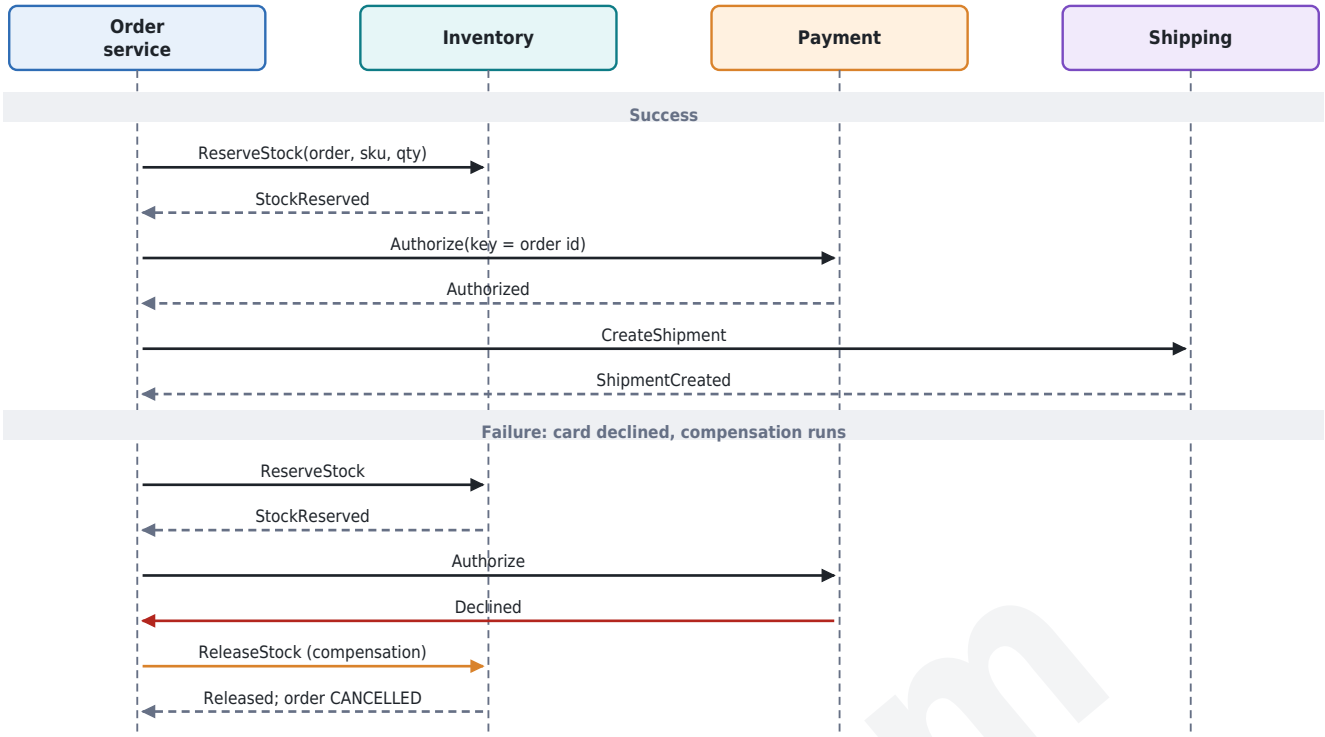


Figure 2.65. The orchestrator invokes one local transaction per service and, on failure, invokes a compensating action for every step that had succeeded.

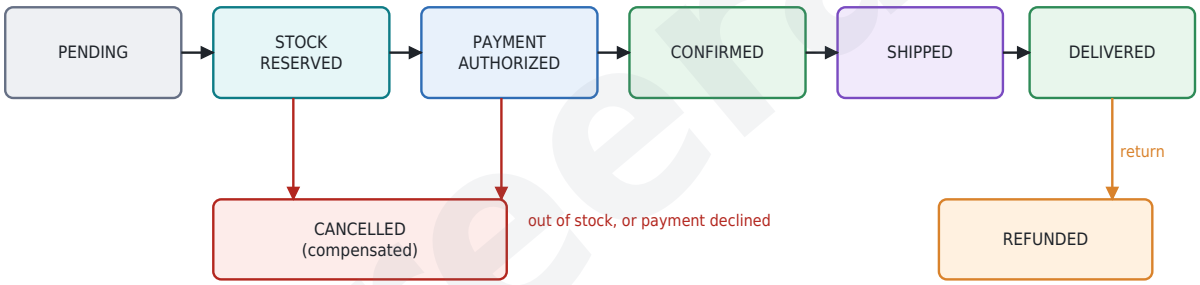


Figure 2.66. The order state machine: every cancellation path passes through compensation, and a refund is a distinct state after delivery.

2.15.6 Deep dive

Inventory accounting and hot SKUs

Stock is reserved at checkout, not when an item enters the cart. Reserving at add-to-cart holds stock for many users who never buy; reserving at payment completion allows a customer to pay for stock that is no longer there. The reservation is a single conditional statement:

Reserve stock SQL

```sql
UPDATE stock SET reserved = reserved + :qty
 WHERE sku = :sku AND bucket = :bucket AND on_hand - reserved >= :qty;
-- one row updated: reserved; zero rows: insufficient stock in this bucket
```

A reservation row with `expires_at` records the claim; a sweeper decrements `reserved` for expired reservations, and shipment converts the reservation into a decrement of `on_hand`. The row-level atomicity of the conditional update prevents overselling for the same reason as in the ticket booking design. When one SKU receives a large share of the traffic, the row lock held by each reservation transaction serialises them. Splitting the stock of a SKU into N bucket rows lets reservations proceed in parallel; a request selects a bucket at random and, on a zero-row result, tries another bucket before reporting out of stock. The experiment below uses pgbench with 20 clients against one hot SKU; every transaction holds its row lock for about 2 ms (modelling other work performed inside the transaction), with 1, 4 and 16 buckets.

hot_sku.sh BASH

```bash
#!/bin/bash
# Hot SKU: one stock row versus N buckets (PostgreSQL 16, pgbench, 20 clients, 5 s)
export PGOPTIONS="-c client_min_messages=warning"
export PGHOST=/tmp/osw_y4/pg PGPORT=22400 PGUSER=postgres
psql -X -q -d postgres -c "DROP DATABASE IF EXISTS shop" -c "CREATE DATABASE shop"
cat > /tmp/osw_y4/e/reserve.sql <<'SQL'
\set b random(1, :nb)
BEGIN;
UPDATE stock SET qty = qty - 1 WHERE sku = 1 AND bucket = :b AND qty >= 1;
SELECT pg_sleep(0.002);
END;
SQL
for nb in 1 4 16; do
  psql -X -q -d shop -c "DROP TABLE IF EXISTS stock" \
       -c "CREATE TABLE stock(sku int, bucket int, qty bigint, PRIMARY KEY (sku, bucket))" \
       -c "INSERT INTO stock SELECT 1, b, 1000000 / $nb FROM generate_series(1, $nb) b"
  echo "buckets=$nb"
  pgbench -n -c 20 -j 2 -T 5 -D nb=$nb -f /tmp/osw_y4/e/reserve.sql shop \
    | grep -E "tps|latency average"
  psql -X -At -d shop \
       -c "SELECT '  stock left=' || sum(qty) || ' min bucket=' || min(qty) FROM stock"
done
```

Run (PostgreSQL 16.15, 2 CPUs; throughput is typical for the build machine) TERMINAL

```
$ ./hot_sku.sh
buckets=1
latency average = 65.562 ms
tps = 305.054347 (without initial connection time)
  stock left=998458 min bucket=998458
buckets=4
latency average = 17.388 ms
tps = 1150.202977 (without initial connection time)
  stock left=994237 min bucket=248501
buckets=16
latency average = 6.298 ms
tps = 3175.367949 (without initial connection time)
  stock left=984130 min bucket=61459
```

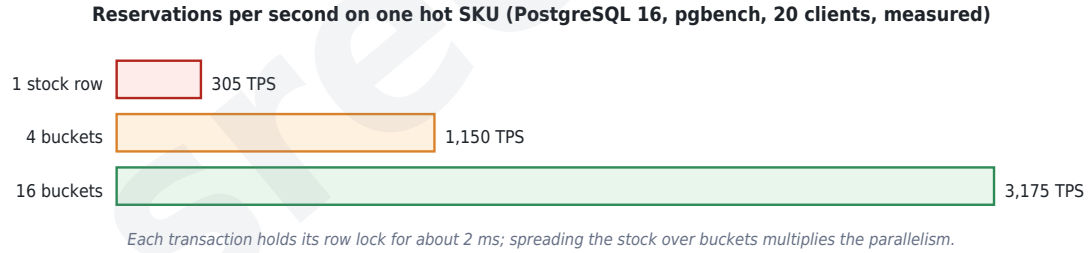


Figure 2.67. Throughput on the hot SKU grows nearly in proportion to the number of buckets until another resource (here two CPUs) limits it.

What to notice: with one row, throughput is bounded by the lock hold time (about 305 transactions per second at an average latency of 66 ms); with 16 buckets the same 20 clients achieve about 10 times the throughput (3,175 per second) at 6 ms; the figures vary by run. The remaining stock is consistent with the number of transactions executed and no bucket went negative. The cost of buckets is that the "available" figure is a sum over rows and that the last units of stock may be stranded in other buckets when one bucket is empty, which the retry across buckets mitigates.

Coordinating the checkout: saga

A distributed transaction across the order, inventory and payment databases (two-phase commit) would block resources during a slow payment call and is unavailable across most payment providers. The saga pattern replaces it with a sequence of local transactions, each with a compensating action.

Aspect	Orchestration	Choreography
Control flow	One service (order) sends commands and tracks the state	Services react to each other's events
Visibility	The state of every order is in one place	Flow is implicit in the event subscriptions
Coupling	Participants know the orchestrator's commands	Participants know each other's events
Failure handling	Timers and retries in one place	Spread across services; harder to reason about

Aspect	Orchestration	Choreography
Best for	Multi-step business processes with compensation	Simple two or three step reactions

Saga coordination styles.

Decision: orchestration for checkout, because the process has ordered steps, timeouts and compensations that must be visible and testable in one place. The orchestrator persists its state in the order row, so after a crash it resumes from the recorded state; a timer finds orders stuck in an intermediate state longer than a limit and re-drives or cancels them. Every step must be idempotent because messages are delivered at least once and the orchestrator retries. The program below implements the orchestration with in-memory services and injected failures: a normal order, a declined card (with compensation), a lost payment reply followed by a retry with the same idempotency key, and exhausted stock.

CheckoutSaga.java JAVA

```
import java.util.*;

public class CheckoutSaga {
    static class Inventory {
        final Map<String, Integer> available = new HashMap<>(Map.of("sku-1", 2));
        final Map<String, Integer> reservations = new HashMap<>();          // orderId -> quantity
        boolean reserve(String orderId, String sku, int qty) {
            if (reservations.containsKey(orderId)) return true;             // idempotent on orderId
            if (available.get(sku) < qty) return false;
            available.merge(sku, -qty, Integer::sum);
            reservations.put(orderId, qty);
            return true;
        }
        void release(String orderId, String sku) {
            Integer q = reservations.remove(orderId);
            if (q != null) available.merge(sku, q, Integer::sum);
        }
    }

    static class Payments {
        final Map<String, String> byIdempotencyKey = new HashMap<>();
        int authorizations = 0;
        int dropReplies = 0;                    // lost replies: work done, answer not delivered
        boolean declineNext = false;
        String authorize(String key, int cents) {
            String done = byIdempotencyKey.get(key);
            if (done != null) return done;      // same key: return the stored result
            String result;
            if (declineNext) result = "DECLINED"; else { authorizations++; result = "AUTHORIZED"; }
            byIdempotencyKey.put(key, result);
            if (dropReplies-- > 0) throw new RuntimeException("timeout");
            return result;
        }
        void voidAuth(String key) { byIdempotencyKey.put(key, "VOIDED"); }
    }

    static String checkout(String orderId, Inventory inv, Payments pay) {
        List<String> log = new ArrayList<>();
        log.add("order " + orderId + " PENDING");
        if (!inv.reserve(orderId, "sku-1", 1)) return "CANCELLED (out of stock)";
        log.add("inventory reserved");
        String result = null;
        for (int attempt = 1; attempt <= 3 && result == null; attempt++) {
            try {
                result = pay.authorize("pay-" + orderId, 4999);
            } catch (RuntimeException e) {
                log.add("payment try " + attempt + " failed: " + e.getMessage());
            }
        }
        if (!"AUTHORIZED".equals(result)) {
            inv.release(orderId, "sku-1");
            log.add("compensation: inventory released");
            System.out.println("  " + String.join(" -> ", log));
            return "CANCELLED (" + result + ")";
        }
        log.add("payment authorized");
        System.out.println("  " + String.join(" -> ", log));
        return "CONFIRMED";
    }
```

Sriram

CheckoutSaga.java (continued) JAVA

```
    public static void main(String[] args) {
        Inventory inv = new Inventory();
        Payments pay = new Payments();
        System.out.println("1. normal order");
        System.out.println("  result: " + checkout("o1", inv, pay) + ", stock=" + inv.available);
        System.out.println("2. card declined");
        pay.declineNext = true;
        System.out.println("  result: " + checkout("o2", inv, pay) + ", stock=" + inv.available);
        pay.declineNext = false;
        System.out.println("3. first payment reply lost, orchestrator retries with the same key");
        pay.dropReplies = 1;
        System.out.println("  result: " + checkout("o3", inv, pay) + ", stock=" + inv.available
            + ", total authorizations=" + pay.authorizations + " (o1 and o3)");
        System.out.println("4. stock exhausted");
        System.out.println("  result: " + checkout("o4", inv, pay) + ", stock=" + inv.available);
    }
}
```

Run (JDK 21, deterministic) TERMINAL

```
$ java CheckoutSaga.java
1. normal order
  order o1 PENDING -> inventory reserved -> payment authorized
  result: CONFIRMED, stock={sku-1=1}
2. card declined
  order o2 PENDING -> inventory reserved -> compensation: inventory released
  result: CANCELLED (DECLINED), stock={sku-1=1}
3. first payment reply lost, orchestrator retries with the same key
  order o3 PENDING -> inventory reserved -> payment try 1 failed: timeout -> payment authorized
  result: CONFIRMED, stock={sku-1=0}, total authorizations=2 (o1 and o3)
4. stock exhausted
  result: CANCELLED (out of stock), stock={sku-1=0}
```

What to notice: in scenario 2 the reservation made in the first step is released by the compensation, so the stock returns to 1; in scenario 3 the payment was performed on the first call and the reply was lost, yet the retry with the same key returned the stored result, so only one authorisation exists for the order (two in total, for o1 and o3). Without the idempotency key the retry would have charged the customer twice. A real orchestrator adds persistence of its state, backoff between attempts, and a cap on attempts after which the order is cancelled and an operator alert is raised.

Idempotency and consistency of prices

Idempotency exists at three levels: the checkout key (one order per purchase attempt), the reservation key (the order id: a repeated `ReserveStock` returns the existing reservation) and the payment key (the order id). Prices are snapshotted into `order_items` at checkout; the catalog cache may be a few seconds stale, so the checkout step re-reads authoritative prices and rejects the request with 409 if the total differs from the one the user confirmed.

2.15.7 Bottlenecks, failures and real-world problems

Overselling during a promotion

Scenario. A discounted item with 1,000 units receives 20,000 checkout attempts in one minute. **Symptoms.** Orders confirmed beyond stock, followed by cancellations and apologies; or the opposite, latency collapse on the stock row. **Diagnosis.** The sum of confirmed quantities exceeds the initial stock; lock wait time on one stock row dominates the database. **Root cause.** A read-check-write sequence for stock, or a single contended row for a hot SKU. **Fix.** The conditional update shown above and stock buckets for hot SKUs; reject early at the edge when a cached counter shows zero stock (a hint, never the decision). **Prevention.** Declare promotions in advance so that hot SKUs are pre-split into buckets, and load test with the expected concurrency.

Double charge on retry

Scenario. The payment gateway answers slowly; the order service times out and retries. **Symptoms.** The customer sees two charges; support receives complaints. **Diagnosis.** Two gateway transactions with different keys, or no key, for one order. **Root cause.** A retry that creates a new operation instead of repeating the same one. **Fix.** A deterministic idempotency key (order id) on every call; check the gateway for the transaction by key before retrying after an ambiguous failure; a reconciliation job that compares captures with orders daily. **Prevention.** Contract tests that inject a lost reply and assert a single charge.

Stuck saga

Scenario. The order service crashes between sending `AuthorizePayment` and receiving the reply. **Symptoms.** Orders remain in STOCK_RESERVED; stock is reserved without an order progressing. **Diagnosis.** Orders in an intermediate state older than the timeout; reservations nearing their expiry. **Root cause.** The orchestrator's progress lived only in memory or the reply was lost. **Fix.** Persisted state and a recovery timer that re-sends the command (safe because it is idempotent) or cancels the order after a limit, releasing the reservation. **Prevention.** Alert on the age of the oldest non-terminal order, and test crash points between every pair of steps.

Duplicate or lost events from the outbox

Scenario. The relay publishes an event and crashes before marking it published. **Symptoms.** A consumer receives the same event twice (for example, two fulfilment tasks for one order). **Diagnosis.** Duplicate message ids in the consumer log. **Root cause.** At-least-once publication; the alternative of publishing before committing could lose events. **Fix.** Consumers deduplicate by event id (a processed-events table or a unique key on the effect); the outbox guarantees that no event is lost. **Prevention.** Design every consumer as idempotent and test it with duplicated delivery.

Phantom reservations

Scenario. Many checkouts are abandoned after the reservation, for example when users leave the payment page. **Symptoms.** Products appear out of stock while units sit in reservations. **Diagnosis.** The reserved quantity is high relative to confirmed orders; reservation age distribution shows many near the expiry. **Root cause.** Reservations that are released only at expiry. **Fix.** A short reservation time (for example 10 minutes), release on explicit cancel or payment failure, and a sweeper that runs frequently. **Prevention.** Monitor the ratio of reservations to confirmed orders per SKU.

2.15.8 Trade-offs and extensions

- Reserving at checkout minimises phantom stock; some retailers instead oversell slightly and handle shortages by back-ordering, trading customer experience for conversion.
- Orchestration centralises the process and adds a component to keep available; choreography has no such component and is harder to understand as the number of steps grows.
- Authorise then capture reduces refunds on cancellation but requires capturing within the authorisation validity window.
- Extension: multi-warehouse allocation chooses the warehouse by distance and stock; reservations then name the warehouse.
- Extension: pre-orders and back-orders add a state between STOCK_RESERVED and CONFIRMED.

2.15.9 Questions to expect

Why not a distributed transaction (two-phase commit) across services?

It blocks resources for the duration of the slowest participant, the payment gateway does not take part in such protocols, and a coordinator failure leaves participants in doubt. A saga keeps each transaction local and uses compensation.

What is the transactional outbox and why is it needed?

It writes the event to a table in the same database transaction as the state change, and a relay publishes it afterwards. This avoids the dual-write problem: updating the database and the message broker cannot be made atomic, and one of them could succeed alone.

How is an idempotency key implemented?

A unique column (or table) stores the key with the result; the first request inserts the key and executes, a repeated request finds the key and returns the stored result, and a request with the same key and different parameters is rejected.

How is overselling prevented for a hot item?

By a conditional stock update, bucketed stock rows to avoid a single contended row, an edge hint to reject obviously sold-out requests, and, for extreme events, a waiting room as in the ticket booking design.

What happens if the payment succeeds but the order service fails to record it?

The payment service retains the authorisation keyed by the order id. On recovery the order service re-sends the command, receives the stored result, and continues; the reconciliation job detects authorisations without orders.

How is the cart kept when a guest logs in?

The guest cart (keyed by a token) is merged into the user's cart at login, summing quantities and keeping the most recent price evaluation; the guest document is then deleted.

How would cancellation after shipment work?

It becomes a return: a separate process creates a return authorisation, and the refund state follows delivery of the returned goods; stock is restored only after inspection.

KEY POINTS

Key points

- A checkout spans several databases; replace the distributed transaction with an orchestrated saga of local transactions and compensations.
- Reserve stock at checkout with a conditional update; split a hot SKU into buckets, which raised throughput from 305 to 3,175 transactions per second in the experiment.
- Make every step idempotent: checkout key, reservation keyed by order id, payment keyed by order id.
- Use a transactional outbox so that state changes and events cannot diverge, and make consumers tolerate duplicates.
- Persist the saga state, time out intermediate states, and reconcile payments with orders.

2.16 Design: Search Autocomplete

Search autocomplete (typeahead) suggests completions for the text a user has typed so far, updating after every keystroke. It is a read-dominated service with extremely strict latency (the suggestions must appear before the next keystroke, which bounds the server time to a few tens of milliseconds) and a data set that is derived from the query history of all users. The design therefore separates an offline pipeline that computes popular completions from an online service that only performs a lookup. The central decisions are the data structure that answers prefix queries in time independent of the data size, the caching strategy that absorbs most of the traffic, and the update pipeline that keeps suggestions fresh and safe.

2.16.1 Requirements and scope

Functional requirements

- Given a prefix, return the top 5 completions ranked by popularity.
- Suggestions reflect recent search behaviour: a newly trending query appears within minutes.
- Inappropriate or blocked terms never appear.
- Suggestions are specific to a locale (language and region).

Non-functional requirements

- Server latency p99 below 20 ms; end-to-end below 100 ms including the network.
- Peak load of several hundred thousand requests per second; availability above 99.99 percent (a missing suggestion degrades but does not break search).

- Suggestions may be slightly stale; they must never be offensive or illegal (a safety property, stricter than the freshness property).

Out of scope

Spelling correction and fuzzy matching, personalisation from the user's own history, semantic (embedding-based) suggestions, and search result ranking. Personalisation is discussed as an extension.

Assumptions

All volumes are assumed: 500 million DAU, 6 searches per user per day, 5 autocomplete requests per search (after client-side debouncing), peak factor 2, 100 million distinct queries retained after removing rare ones, an average query length of 25 characters.

2.16.2 Capacity estimation

Quantity	Arithmetic	Result
Searches per day	500e6 x 6	`3 billion`
Suggestion requests per day	3e9 x 5	`15 billion`
Suggestion QPS (average)	15e9 / 86,400	`173,611 per s`
Suggestion QPS (peak)	173,611 x 2	`about 350,000 per s`
Requests reaching the service (80 percent edge hit)	350,000 x 20 percent	`70,000 per s`
Service instances (10,000 per s each, assumed)	70,000 / 10,000, doubled for redundancy	`about 14`
Trie nodes	100e6 queries x about 3 nodes per query (shared prefixes)	`300 million`
Trie memory	300e6 nodes x 64 B (node plus 5 query ids)	`19.2 GB`
Response bandwidth (peak, before the edge)	350,000 x 0.5 KB	`175 MB per s (1.4 Gbit per s)`
Query log volume	3e9 x 100 B	`300 GB per day`

Capacity estimate (inputs assumed).

Two results shape the design. The trie of about 19 GB fits in the memory of a single server, so the data does not need to be sharded for capacity: every replica holds the whole structure and any replica can answer any request, which gives simple load balancing and fault tolerance. And the request rate is dominated by a small set of very short prefixes, which makes caching extremely effective, as the following section shows.

2.16.3 API design

Endpoint	Request / response	Notes
`GET /v1/suggest`	Query: q (prefix), limit (default 5), locale. Response: q, suggestions[], snapshot_version	Cache-Control: public, max-age=60
`POST /v1/events/query`	Body: query text, locale, ts. Response 202	Called by the search service when a search is executed, not by the typeahead
`PUT /v1/admin/blocklist/{term}`	Operator only. Response 204	Takes effect at serving time without a rebuild

Suggestion API.

The request is a GET with the prefix in the query string, so that CDN and browser caches can store the response keyed by URL. The client waits 150 ms after the last keystroke before sending (debouncing), cancels a request that is still in flight when the next key is pressed, and ignores a response whose prefix no longer matches the input box. These client rules reduce the request rate by a large factor and are part of the system's capacity plan.

2.16.4 Data model

query_counts (per window)

```
PK window_start  TIMESTAMP
PK query_id  BIGINT
count  BIGINT
locale  VARCHAR(8)
```

query_dictionary

```
PK query_id  BIGINT
text  VARCHAR(100)
locale  VARCHAR(8)
blocked  BOOL
```

blocklist

```
PK term  VARCHAR
kind  EXACT|PATTERN
added_at  TIMESTAMP
```

trie snapshot file (read-only, memory mapped)

```
header: version, locale, node_count, checksum
node[i]: char, first_child, child_count, topk[5] of query_id
string table: query_id -> text, count
```

Figure 2.68. The offline side keeps counts per time window and a query dictionary; the online side loads a read-only trie snapshot file built from them.

Counts are aggregated per window (hour or day) and locale from the query log. The query dictionary assigns a numeric identifier to each distinct query so that the trie stores five small identifiers per node instead of five strings. The snapshot is an immutable file with a header, a node table and a string table; because it is read-only and laid out contiguously, it can be loaded or memory mapped by many replicas, and publishing a new version is a file swap rather than an in-place update.

2.16.5 High-level architecture

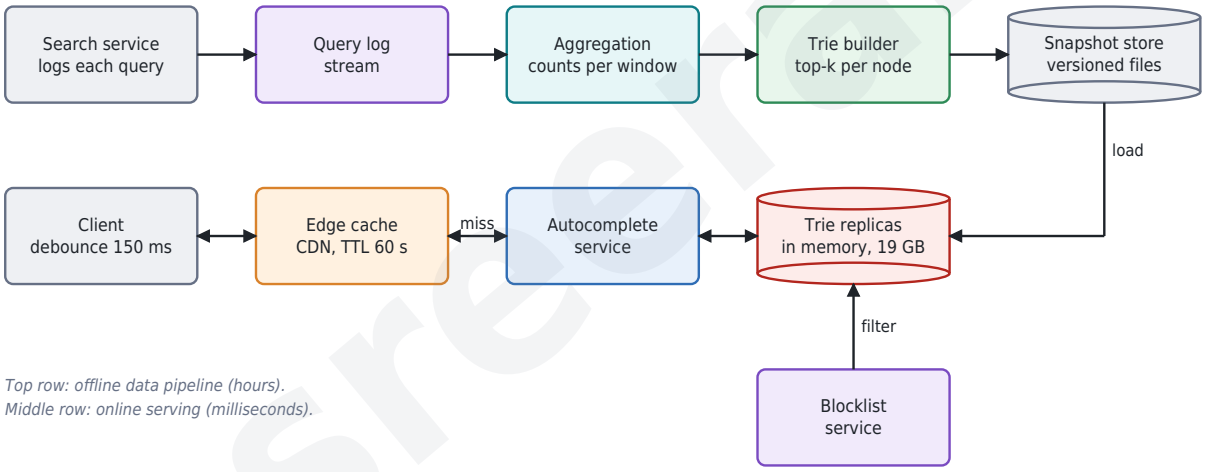


Figure 2.69. The online path (middle row) performs a cache lookup and an in-memory trie lookup; all computation of popularity happens in the offline pipeline (top row) that produces versioned snapshots.

Online request flow

1. The client sends `GET /v1/suggest?q=tre` to the edge cache. A hit returns immediately (about 80 percent of requests in the estimate).
2. On a miss the edge forwards to an autocomplete service replica, which walks the trie one character at a time to the node for the prefix and returns the stored top-5 list. The walk costs a number of steps equal to the prefix length.
3. The service applies the live blocklist to the five results and returns them with a cache header of 60 seconds.

sreeram

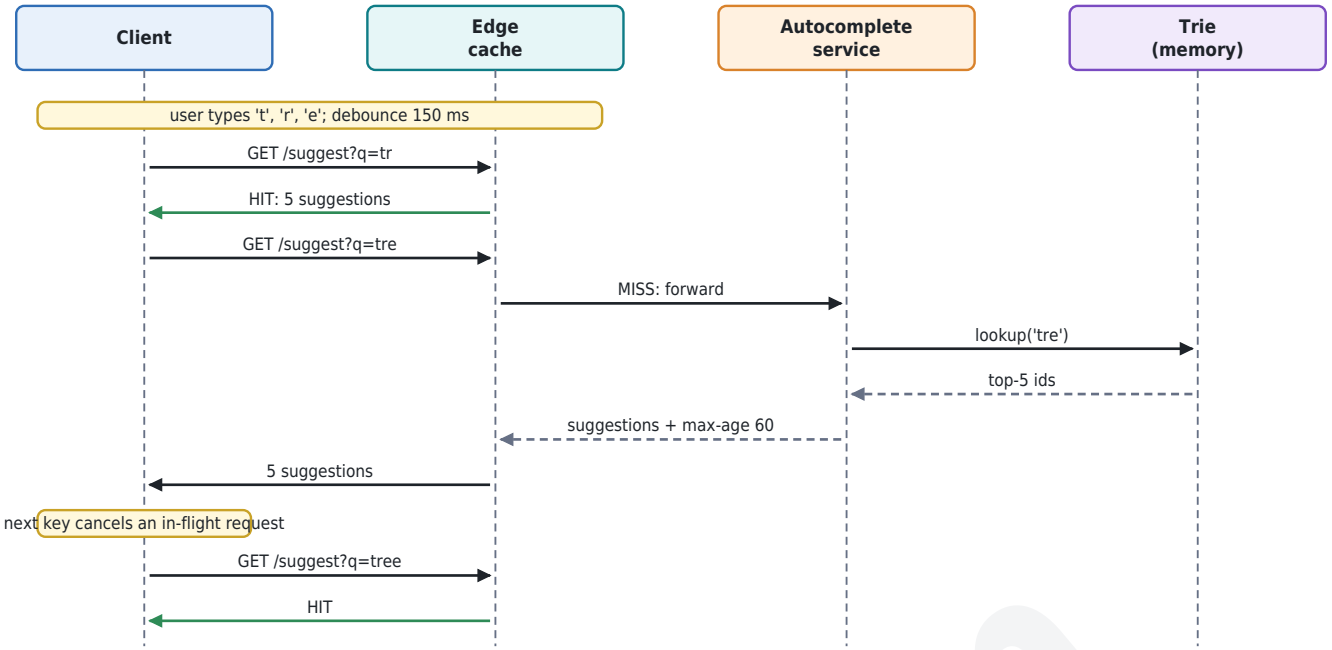


Figure 2.70. The first two keystrokes are served from the edge; the third misses and reaches the trie; the client cancels outdated requests and sends nothing while typing continues within the debounce interval.

Offline pipeline flow

1. The search service writes each executed query to a log stream.
2. The aggregation job counts queries per window and locale, applies decay to older windows and removes queries below a minimum count and those that match the blocklist.
3. The trie builder computes, for every node, the top five queries below it and writes a versioned snapshot to storage. Replicas download and load the new snapshot, then switch to it atomically.

2.16.6 Deep dive

Data structure for prefix lookup

Approach	Mechanism	Strength	Weakness
Scan with LIKE or startsWith	Compare the prefix with every query	Trivial	O(N) per request; unusable at 100 million queries
Trie, DFS at request time	Walk to the prefix node, traverse the subtree, rank	Small memory	Subtree of a short prefix contains millions of entries
Trie with top-k at each node	Precomputed list of the best k completions per node	O(L) lookup independent of N	Memory for lists; a count change updates L nodes
Hash map from every prefix to its list	One entry per distinct prefix	O(1) lookup	More entries than a trie; no sharing of structure
Search engine suggester (finite-state transducer)	Compact automaton with weights	Compact, rich features	Operational dependency; less control over ranking

Prefix lookup alternatives.

Decision: a trie with the top-k list stored at each node. The lookup is O(L) where L is the prefix length (typically below 10) and does not depend on the number of queries. The price is memory for the lists (5 identifiers per node, counted in the estimate) and a more expensive update: a change in the count of one query must refresh the lists of all L ancestors.

Verifying the trie

The program below builds such a trie for 89,637 synthetic queries with skewed popularity, answers 2,000 random prefixes both from the trie and by scanning all queries, checks that the results are identical, compares the lookup times, and shows how an update for a trending query propagates up the path. The vocabulary is synthetic; the structure and the measurements of the algorithm are real.

TrieAutocomplete.java JAVA

```java
import java.util.*;

public class TrieAutocomplete {
    static final int K = 5;

    record Entry(String query, long count) {}

    static class Node {
        final Map<Character, Node> next = new TreeMap<>();
        final List<Entry> top = new ArrayList<>();          // best K completions under this prefix
    }

    final Node root = new Node();

    /** Insert or update one query; every node on its path refreshes its top-K list: O(L * K). */
    void update(String q, long newCount) {
        Node n = root;
        offer(n, q, newCount);
        for (char c : q.toCharArray()) {
            n = n.next.computeIfAbsent(c, k -> new Node());
            offer(n, q, newCount);
        }
    }

    private static void offer(Node n, String q, long count) {
        n.top.removeIf(e -> e.query().equals(q));
        n.top.add(new Entry(q, count));
        n.top.sort((a, b) -> b.count() != a.count() ? Long.compare(b.count(), a.count())
                                                    : a.query().compareTo(b.query()));
        while (n.top.size() > K) n.top.remove(n.top.size() - 1);
    }

    List<Entry> suggest(String prefix) {
        Node n = root;
        for (char c : prefix.toCharArray()) {
            n = n.next.get(c);
            if (n == null) return List.of();
        }
        return n.top;
    }

    static List<Entry> bruteForce(Map<String, Long> counts, String prefix) {
        List<Entry> all = new ArrayList<>();
        for (var e : counts.entrySet()) {
            if (e.getKey().startsWith(prefix)) all.add(new Entry(e.getKey(), e.getValue()));
        }
        all.sort((a, b) -> b.count() != a.count() ? Long.compare(b.count(), a.count())
                                                  : a.query().compareTo(b.query()));
        return all.subList(0, Math.min(K, all.size()));
    }

    public static void main(String[] args) {
        // synthetic vocabulary: 2,000 pronounceable words, Zipf-like popularity
        Random rnd = new Random(1);
        String[] syl = {"ba", "ko", "ri", "tu", "men", "sa", "lo", "vi", "ne", "pra", "do",
            "chi", "an", "el", "or"};
        List<String> words = new ArrayList<>();
        while (words.size() < 2000) {
            StringBuilder sb = new StringBuilder();
            int n = 2 + rnd.nextInt(3);
            for (int i = 0; i < n; i++) sb.append(syl[rnd.nextInt(syl.length)]);
            if (!words.contains(sb.toString())) words.add(sb.toString());
        }
        Map<String, Long> counts = new HashMap<>();
        for (int i = 0; i < 300_000; i++) {
            int w1 = (int) Math.min(1999, Math.abs(rnd.nextGaussian()) * 350);   // skewed
            String q = words.get(w1) + (rnd.nextInt(3) == 0 ? " " + words.get((int) Math.min(1999,
                Math.abs(rnd.nextGaussian()) * 350)) : "");
            counts.merge(q, 1L + rnd.nextInt(20), Long::sum);
        }
        TrieAutocomplete t = new TrieAutocomplete();
        long t0 = System.nanoTime();
        counts.forEach(t::update);
        long build = System.nanoTime() - t0;
        System.out.printf("distinct queries=%d, build time=%.0f ms (typical)%n",
```

Sheram

TrieAutocomplete.java (continued) JAVA

```
            counts.size(), build / 1e6);

        String p = words.get(0).substring(0, 2);
        System.out.println("suggest(\"" + p + "\"):");
        for (Entry e : t.suggest(p)) System.out.println("  " + e.query() + "  " + e.count());

        int checked = 0, mismatches = 0;
        long tTrie = 0, tScan = 0;
        for (String q : counts.keySet()) {
            if (checked >= 2000) break;
            String prefix = q.substring(0, 1 + rnd.nextInt(Math.min(4, q.length())));
            long a = System.nanoTime();
            List<Entry> fast = t.suggest(prefix);
            long b = System.nanoTime();
            List<Entry> slow = bruteForce(counts, prefix);
            long c = System.nanoTime();
            tTrie += b - a; tScan += c - b;
            if (!fast.equals(slow)) mismatches++;
            checked++;
        }
        System.out.printf("prefixes checked=%d mismatches=%d%n", checked, mismatches);
        System.out.printf("avg lookup: trie=%.1f us, scan of all queries=%.0f us (typical)%n",
            tTrie / 1e3 / checked, tScan / 1e3 / checked);

        String hot = words.get(1500);
        String pre = hot.substring(0, 3);
        System.out.println("before: top for \"" + pre + "\" = " + t.suggest(pre).get(0));
        t.update(hot, 1_000_000);                          // trending query appears
        System.out.println("after:  top for \"" + pre + "\" = " + t.suggest(pre).get(0));
    }
}
```

Run (JDK 21; build and lookup times are typical for the build machine and vary between runs) TERMINAL

```
$ java TrieAutocomplete.java
distinct queries=89637, build time=423 ms (typical)
suggest("el"):
  elpravine  5488
  elvi  5164
  elkosa  5076
  elpralolo  4911
  elchi  4774
prefixes checked=2000 mismatches=0
avg lookup: trie=2.8 us, scan of all queries=2669 us (typical)
before: top for "nea" = Entry[query=neanturi, count=4490]
after:  top for "nea" = Entry[query=neanmenan, count=1000000]
```

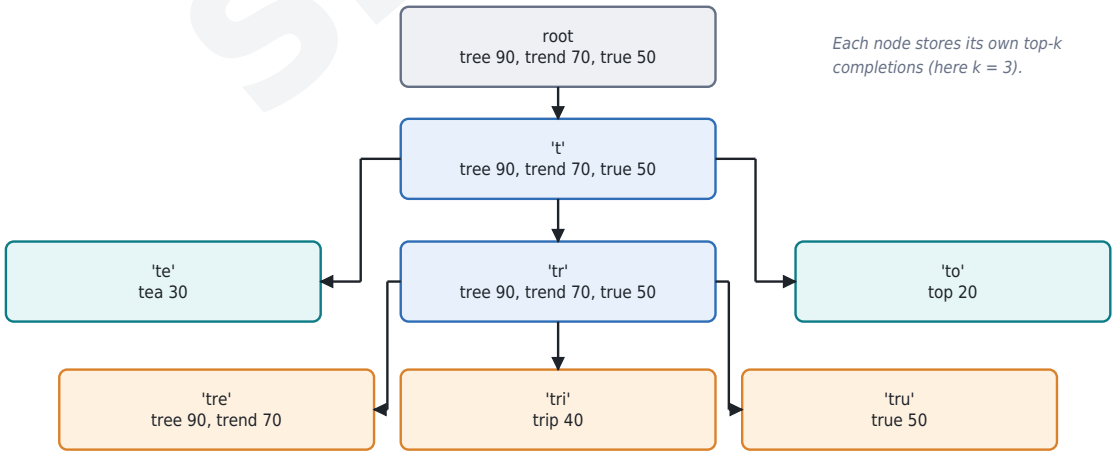


Figure 2.71. Every node stores its own top-k list, so the lookup for a prefix follows one edge per character and returns the stored list; the node for 'tr' already holds the answer for the whole subtree.

What to notice: zero mismatches in 2,000 checks against the brute-force ranking; a lookup in under 3 microseconds against about 2.7 milliseconds for the scan, a factor near 1,000 on this data set, growing with the data size because the scan is linear and the trie lookup is not. The update at the end shows the propagation: a query given a count of

1,000,000 becomes the first suggestion for its prefixes immediately, because the call refreshes the lists of every node on its path (O(L x k) work). The ranking breaks ties by query text, which makes results deterministic and cacheable.

Ranking, decay and freshness

Ranking by total historical count favours old queries. Production systems weight recent windows more strongly, for example with an exponential decay `score = sum(count_w x 0.5^(age_w / half_life))` over windows, which lets the score of a query that stopped being searched fall away. Freshness is obtained by several paths with different cost:



Figure 2.72. Delay between a query becoming popular and its appearance in suggestions, for three pipeline schedules; the faster paths cost more compute and more operational risk.

A common arrangement is a daily full rebuild for stability, an hourly incremental snapshot, and a small trending list computed from 5-minute windows that is merged at serving time into the results of matching prefixes. The merge keeps the large trie stable and limits the volume of data that changes quickly.

Caching

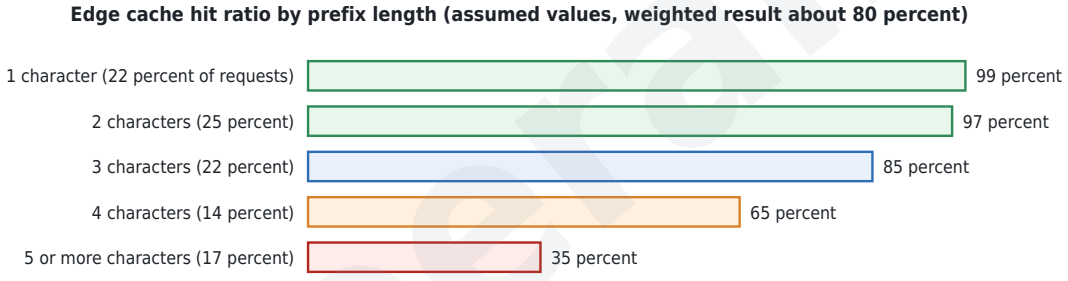


Figure 2.73. The hit ratio at the edge is highest for the shortest prefixes, which carry a large share of requests and repeat across millions of users.

Because a suggestion response depends only on the prefix, locale and snapshot, it is cacheable by URL at the CDN, in the browser (a short max-age), and in the service. The weighted hit ratio of 80 percent in the estimate follows from the assumed distribution of prefix lengths; it must be measured in production, and the long-prefix tail is the part that cannot be cached effectively.

Safe publication of snapshots

A defective snapshot reaches every user at once, so publication follows a gated rollout.

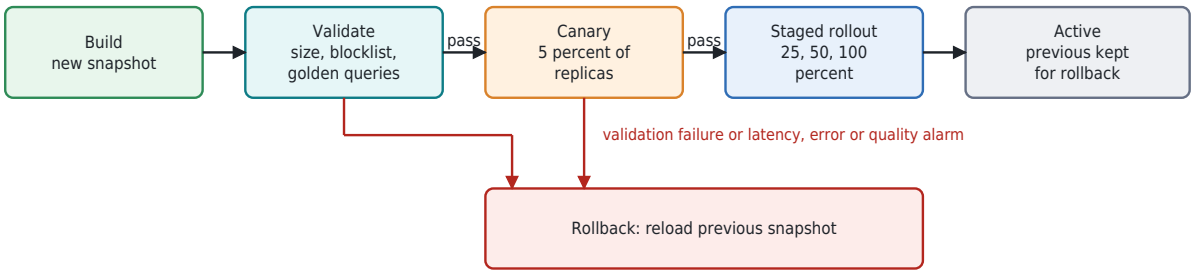


Figure 2.74. Validation compares the new snapshot with the previous one and with golden queries; a canary exposes 5 percent of replicas; any alarm reloads the previous snapshot, which every replica keeps.

Validation checks that the size is within a tolerance of the previous snapshot (for example plus or minus 20 percent), that no blocklisted term is present, and that a set of golden prefixes returns the expected leaders. A snapshot that fails any check is never loaded.

2.16.7 Bottlenecks, failures and real-world problems

Offensive or legally problematic suggestion

Scenario. A coordinated group searches for a defamatory phrase until it becomes a top suggestion for a person's name. **Symptoms.** User reports and press attention; the phrase appears for a specific prefix. **Diagnosis.** The phrase has a high count concentrated in a short period; it passed the pipeline because it was not on the blocklist. **Root cause.** Popularity is the only ranking signal and popularity can be manufactured. **Fix.** Add the phrase to the live blocklist, applied at serving time so that no rebuild is needed, and purge the cached responses for the affected prefixes at the CDN. **Prevention.** Detect bursts from few distinct users (count distinct users per query, not only total count), apply classifiers for abusive content in the pipeline, and keep a fast removal path with an owner on call.

Bad snapshot deployment

Scenario. A pipeline bug drops half of the queries; the new snapshot is published to all replicas. **Symptoms.** Suggestions for many prefixes are empty; click-through on suggestions falls. **Diagnosis.** The snapshot size and node count are far below the previous version; quality metrics fell at the time of the switch. **Root cause.** The publication had no validation or staged rollout. **Fix.** Roll back to the previous snapshot on the replicas, which retain it. **Prevention.** The validation gate and canary shown above, and an automatic rollback on a quality alarm.

Cache expiry stampede on a popular prefix

Scenario. The cached entry of a very popular prefix expires at one edge location at the same moment for thousands of clients. **Symptoms.** A burst of identical misses reaches the service; latency spikes periodically at intervals of the TTL. **Diagnosis.** Request logs show many identical URLs within a few milliseconds after each expiry. **Root cause.** Synchronised expiry with no request coalescing. **Fix.** Coalesce concurrent misses for one URL at the edge, serve stale content while revalidating, and add random jitter to the TTL. **Prevention.** Load test expiry at peak traffic; monitor the origin request rate per URL.

Keystroke storm from clients

Scenario. A client release removes the debounce. **Symptoms.** The request rate rises by a factor of two to four without any change in the number of users. **Diagnosis.** Requests per search in the logs jump after the release date; the requests carry prefixes of increasing length from the same device within milliseconds. **Root cause.** The capacity plan depends on client behaviour that is not under server control. **Fix.** Roll back the client; add a per-client rate limit at the edge as protection. **Prevention.** Record requests per search as a service level indicator and alert on it.

Memory pressure and pauses

Scenario. The trie is built from millions of small objects on the heap. **Symptoms.** Long garbage-collection pauses and a high p99 on the replicas after loading a new snapshot. **Diagnosis.** Garbage-collection logs show pauses proportional to the live heap size; latency spikes align with snapshot loads. **Root cause.** An object-per-node representation of a 19 GB structure. **Fix.** Represent the trie as flat arrays or a memory-mapped file and load a new snapshot into a separate region before switching a pointer. **Prevention.** Keep the serving structure off the managed heap or in primitive arrays, and measure pauses in the load test.

2.16.8 Trade-offs and extensions

- Storing top-k per node trades memory and update cost for lookup speed; a smaller k reduces memory, a larger k allows client-side filtering and personalisation re-ranking.
- Frequent snapshots improve freshness and increase operational risk and load; the trending side list is the compromise.
- Personalisation merges the global suggestions with the user's recent queries on the client or in a small per-user store, without changing the global trie.
- Extension: typo tolerance requires a fuzzy matching layer (edit distance automata or n-gram index) in front of or beside the trie.
- Extension: very large vocabularies or many locales are sharded by locale first, then by prefix range with weights, because the first-letter distribution is skewed.

2.16.9 Questions to expect

Why store the top-k at every node rather than computing it per request?

Computing it requires traversing the subtree of the prefix, which for a one-letter prefix holds a large fraction of all queries. Precomputing makes the lookup depend only on the prefix length.

How is the trie updated when counts change?

Not in place for the bulk of the data: a new snapshot is built from aggregated counts and swapped in. A small trending structure handles fast changes and is merged at serving time. In-place updates refresh the top-k lists on the path, as the demonstration program does.

How is the system kept fast for the first letter, which has the largest subtree?

Its answer is precomputed and cached; the edge hit ratio for one-character prefixes is the highest, so they rarely reach the service.

How are results filtered for safety?

At three points: the aggregation removes blocklisted terms, validation rejects a snapshot containing any, and the service applies the live blocklist at response time so that a removal takes effect in seconds.

How would the design handle many languages?

A separate trie per locale, because prefixes and popularity differ, and because a unicode-aware tokeniser or normalisation (case folding, accents) is needed per language. The locale is part of the cache key.

What is the failure mode when the service is down?

The search box works without suggestions. The edge continues to serve cached responses (stale if necessary), so the system degrades gracefully rather than failing.

How is the dictionary of queries bounded?

By a minimum count threshold, a maximum query length, removal of queries that look like personal data (long numbers, email-like strings), and decay so that old queries expire.

KEY POINTS

Key points

- Autocomplete is a lookup service: all ranking is computed offline and stored as the top-k list at each trie node, giving O(prefix length) lookups.
- Short prefixes dominate traffic and cache extremely well; the edge cache and client debouncing carry most of the load.
- Publish immutable, versioned snapshots through a validated, staged rollout with fast rollback.
- Combine a stable daily trie with a small trending list for freshness, and apply the blocklist at serving time for safety.
- Represent the serving trie in flat arrays or a mapped file to avoid garbage-collection pauses.

2.17 Design: File Storage and Sync (Dropbox, Google Drive)

A file storage and synchronisation service keeps a user's files identical on several devices and in the cloud, and allows folders to be shared. The workload mixes two very different kinds of data: a small, strongly consistent metadata set (the tree of names, versions and permissions) and a very large, immutable blob set (the bytes of files). The design problems are efficient transfer of large files that change slightly, detection and propagation of changes to every device, correct behaviour when two devices change the same file, and keeping the storage bill bounded. The central decisions are the chunking strategy, the commit protocol that separates data from metadata, and the conflict policy.

sreeram

2.17.1 Requirements and scope

Functional requirements

- Upload, download, rename, move and delete files and folders; files up to 50 GB.
- Automatic synchronisation of a folder tree to multiple devices, with offline edits merged on reconnection.
- File version history (30 days or the last 100 revisions) and restore.
- Sharing of folders with other users with read or edit permissions.

Non-functional requirements

- Durability of file content of at least 99.999999999 percent (eleven nines); a committed file is never lost.
- A change on one device is visible on other online devices within about 5 seconds for small files.
- Bandwidth efficiency: a small edit in a large file transfers a small amount of data.
- Metadata operations are strongly consistent per namespace (a user's folder tree); content transfer is eventually complete.

Out of scope

Real-time co-editing of documents (operational transformation or CRDTs), content search and preview generation, end-to-end encryption, and malware scanning. These attach to the journal of changes described below.

Assumptions

All volumes are assumed: 100 million daily active users, 500 petabytes of logical user data, 5 file changes per active user per day, an average changed file of 2 MB of which 40 percent of the bytes are uploaded after delta detection, an average of 200 files per user, 2 devices per user with 30 percent online at the peak, and a peak factor of 3.

2.17.2 Capacity estimation

Quantity	Arithmetic	Result
File changes (average / peak)	100e6 x 5 / 86,400 ; x 3	5,787 / 17,361 per s
Metadata commits (two per change)	5,787 x 2 ; x 3	11,574 / 34,700 per s
Upload volume per day	500e6 changes x 2 MB x 40 percent	400 TB
Upload bandwidth (average / peak)	400e12 x 8 / 86,400 ; x 3	37 / 111 Gbit per s
Chunk references at 4 MiB	500e15 B / 4.19e6 B	1.19e11
Chunk index at 4 MiB (40 B each)	1.19e11 x 40 B	4.8 TB
Chunk index at 64 KiB	7.6e12 x 40 B	305 TB
File rows	100e6 users x 200 files x 300 B	6 TB
Sync connections (peak)	100e6 x 2 devices x 30 percent	60 million
Notification nodes	60e6 / 100,000 per node	600 nodes

Capacity estimate (inputs assumed).

The chunk index is the item that depends most strongly on a design parameter: with chunks of 4 MiB the index is 4.8 TB and shards comfortably; with chunks of 64 KiB it is 305 TB, a large database in its own right. Smaller chunks give better deduplication and smaller deltas, larger chunks give a cheaper index and fewer requests; the choice of an average of a few MiB (4 MiB is assumed here) is a typical compromise. The demonstration later in this chapter uses 64 KiB chunks only because the files in the test are 8 MiB.

2.17.3 API design

Endpoint	Request / response	Notes
`POST /v1/commit`	Body: namespace, path, base_rev, size, chunk_hashes[]. Response: needs_upload[] or revision	Called twice per change: negotiation, then final
`PUT /v1/blocks/{sha256}`	Body: chunk bytes (up to 8 MiB). Response 200	The server verifies that the hash matches the content
`GET /v1/blocks/{sha256}`	Response: chunk bytes	Cacheable forever: content-addressed
`GET /v1/changes`	Query: namespace, cursor, limit. Response: entries[], new_cursor	Delta since a journal cursor
`GET /v1/changes/longpoll`	Query: namespace, cursor, timeout (30 s). Response: changed (bool)	Notification channel
`POST /v1/share`	Body: folder, user, role. Response 204	Adds an entry to the ACL

Metadata and block API.

The two-step commit is the core protocol. The client sends the list of chunk hashes of the new version; the server answers with those it does not already hold; the client uploads only those and repeats the commit, which succeeds if every chunk exists and `base_rev` equals the file's current revision (an atomic compare-and-set). Chunk upload is idempotent by construction (the key is the content hash), so interrupted uploads resume by re-sending the missing chunks.

2.17.4 Data model

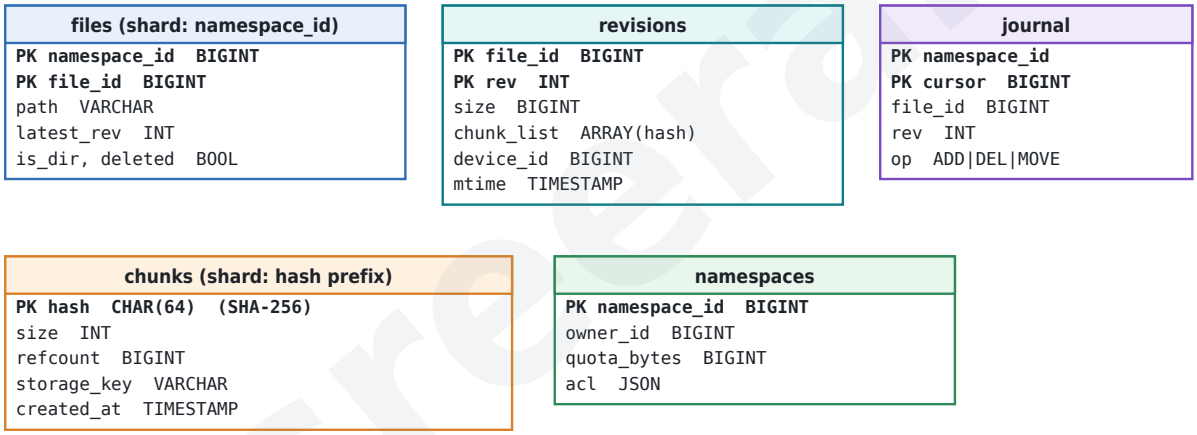


Figure 2.75. Metadata (files, revisions, journal) is sharded by namespace so that a user's folder tree is one consistency domain; the chunk table is sharded by hash prefix and independent of users.

A namespace is the unit of consistency: a user's root folder or a shared folder. All files, revisions and journal entries of a namespace live on one shard, so a commit, a rename or a directory move is a single-shard transaction. The journal is an append-only sequence with a monotonic cursor per namespace; every device records the last cursor it applied, and "what changed" is "all entries after my cursor". A revision stores an ordered list of chunk hashes; it does not contain bytes. The chunk table maps a hash to a storage location and a reference count. Because the hash identifies the content, the same chunk uploaded by different files is stored once.

2.17.5 High-level architecture

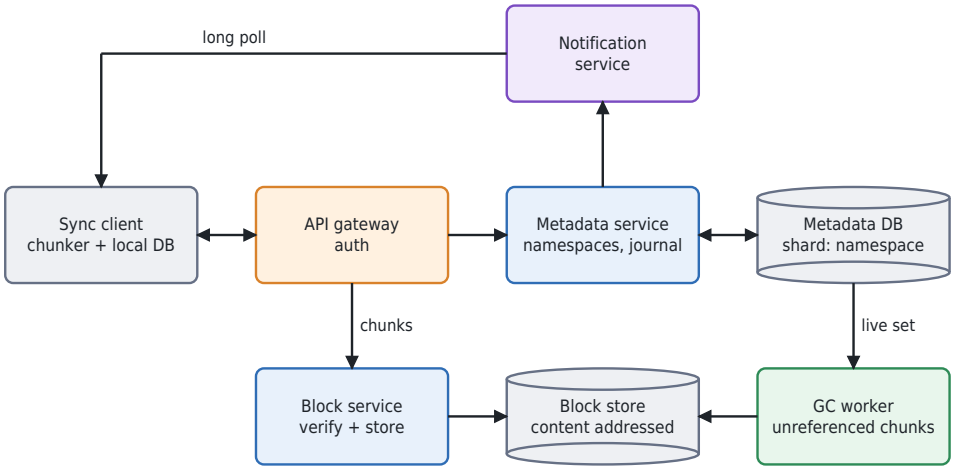


Figure 2.76. Chunk bytes flow through the block service into content-addressed storage; metadata changes flow through the metadata service, which appends to a journal and notifies connected devices.

Upload flow

1. The sync client detects a change (file system events), splits the file into chunks, computes the SHA-256 of each, and calls `commit` with the chunk list and the base revision.
2. The metadata service checks which chunk hashes exist in the chunk table and returns the missing ones.
3. The client uploads the missing chunks to the block service, which verifies each hash and writes to block storage.
4. The client repeats `commit`. The metadata service verifies the chunks, compares the base revision, creates the new revision, appends a journal entry and returns the revision number.

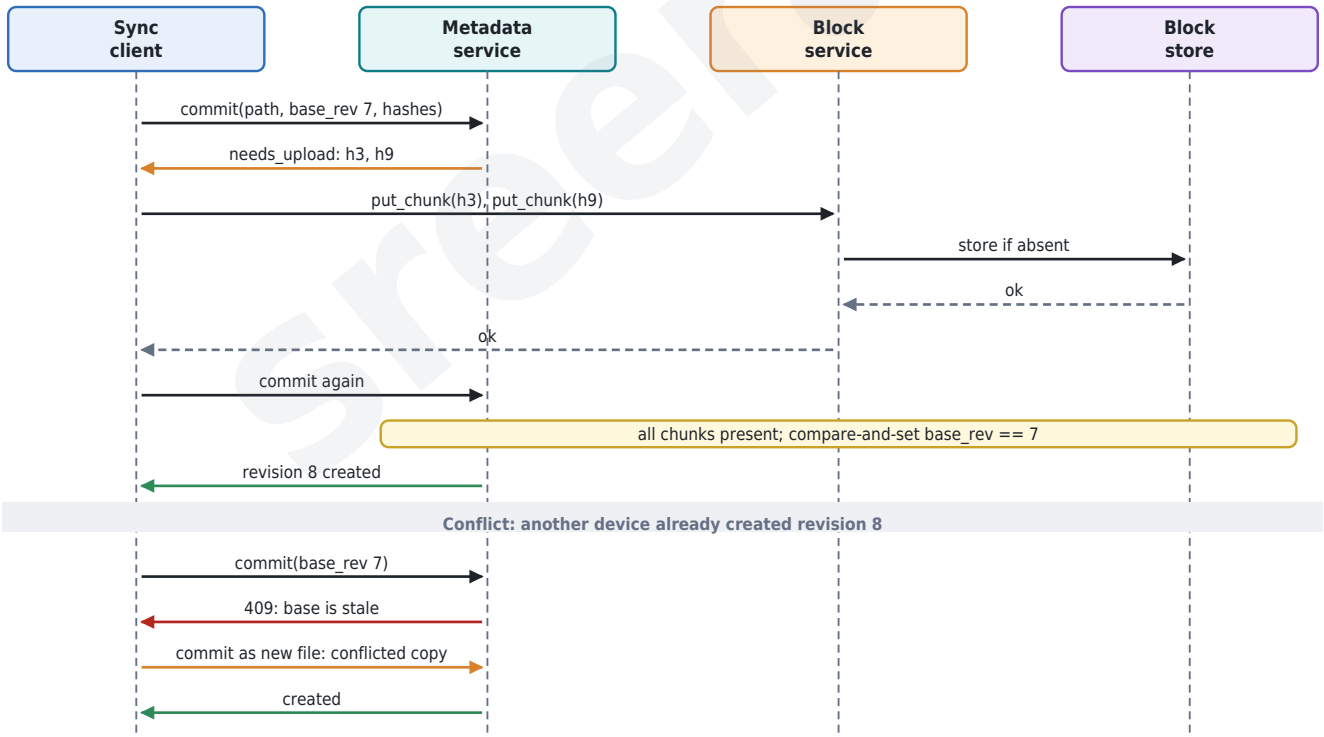


Figure 2.77. The first commit negotiates which chunks are needed, the second commits the revision; a stale base revision produces a conflict instead of an overwrite.

Download and synchronisation flow

1. Each device keeps a long poll open per namespace with its last cursor. When the journal advances, the notifier answers the poll.
2. The device calls `changes` with its cursor, receives the entries (file, revision, chunk hashes), downloads the chunks it does not hold locally, assembles the file, updates its local database and records the new cursor.

sreeram

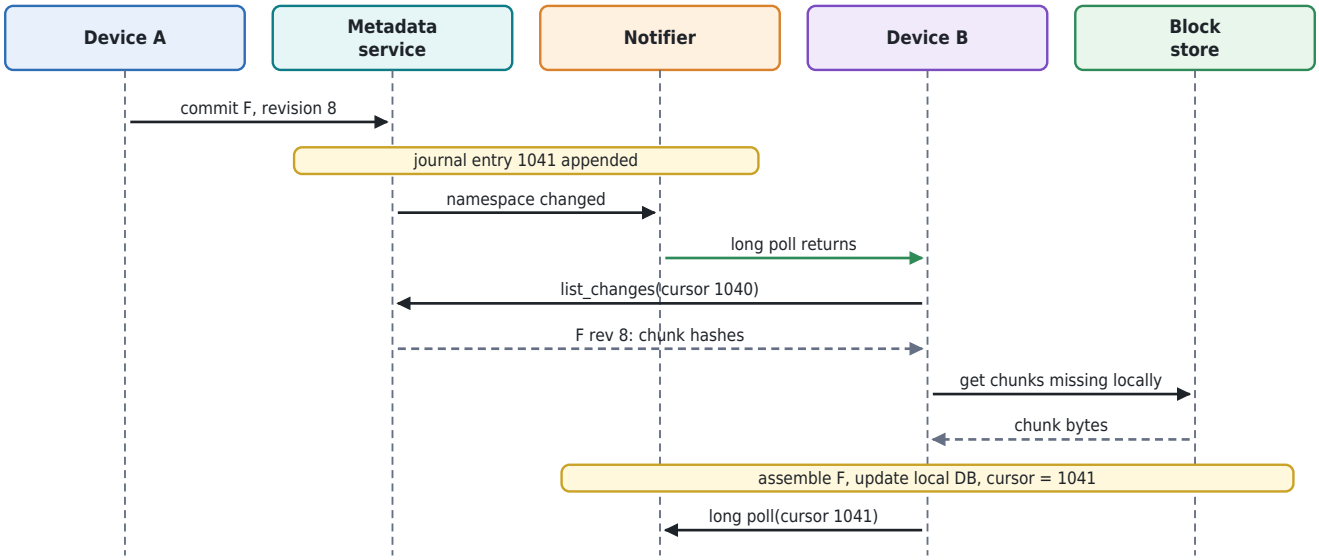


Figure 2.78. Notification only says that the namespace changed; the device then pulls everything after its cursor, so a lost notification delays synchronisation but cannot lose a change.

2.17.6 Deep dive

Chunking strategy

Strategy	Mechanism	Strength	Weakness
Whole file	One object per file version	Simplest	Any edit uploads the whole file; no deduplication inside files
Fixed-size chunks	Cut every N bytes	Simple, cheap	An insertion shifts every later boundary: all later chunks change
Content-defined chunks	Cut where a rolling hash of the last bytes matches a pattern	Boundaries survive insertions and deletions	CPU cost of the rolling hash; variable chunk sizes

Chunking alternatives.

The program below compares fixed 64 KiB chunks with content-defined chunking (a gear rolling hash with a boundary probability of 1 in 65,536 and chunk size limits of 16 KiB and 256 KiB) on an 8 MiB file under three edits. A chunk counts as "to upload" if its SHA-256 is not among the hashes of the previous version.

ChunkDemo.java JAVA

```java
import java.security.MessageDigest;
import java.util.*;

public class ChunkDemo {
    static final int FIXED = 64 * 1024;
    static final int MIN = 16 * 1024, MAX = 256 * 1024;
    static final long MASK = (1L << 16) - 1;                  // boundary probability 1/65536
    static final long[] GEAR = new long[256];
    static { Random r = new Random(99); for (int i = 0; i < 256; i++) GEAR[i] = r.nextLong(); }

    static List<byte[]> fixed(byte[] d) {
        List<byte[]> out = new ArrayList<>();
        for (int i = 0; i < d.length; i += FIXED) {
            out.add(Arrays.copyOfRange(d, i, Math.min(d.length, i + FIXED)));
        }
        return out;
    }

    /** Content-defined chunking (gear rolling hash): boundaries depend on content, not offsets. */
    static List<byte[]> cdc(byte[] d) {
        List<byte[]> out = new ArrayList<>();
        int start = 0;
        long h = 0;
        for (int i = 0; i < d.length; i++) {
            h = (h << 1) + GEAR[d[i] & 0xFF];
            int len = i - start + 1;
            if ((len >= MIN && (h & MASK) == 0) || len >= MAX) {
```

ChunkDemo.java (continued) JAVA

```
                out.add(Arrays.copyOfRange(d, start, i + 1));
                start = i + 1;
                h = 0;
            }
        }
        if (start < d.length) out.add(Arrays.copyOfRange(d, start, d.length));
        return out;
    }

    static String sha(byte[] c) throws Exception {
        return HexFormat.of().formatHex(MessageDigest.getInstance("SHA-256").digest(c));
    }

    static Set<String> hashes(List<byte[]> chunks) throws Exception {
        Set<String> s = new HashSet<>();
        for (byte[] c : chunks) s.add(sha(c));
        return s;
    }

    static void report(String edit, byte[] oldData, byte[] newData) throws Exception {
        for (String mode : new String[] {"fixed 64 KiB", "content-defined"}) {
            boolean fx = mode.startsWith("fixed");
            List<byte[]> oc = fx ? fixed(oldData) : cdc(oldData);
            List<byte[]> nc = fx ? fixed(newData) : cdc(newData);
            Set<String> known = hashes(oc);
            long upload = 0; int changed = 0;
            for (byte[] c : nc) {
                if (!known.contains(sha(c))) { changed++; upload += c.length; }
            }
            System.out.printf("%-28s %-16s chunks=%3d upload=%3d chunks %8d B (%.1f%%)%n",
                edit, mode, nc.size(), changed, upload, 100.0 * upload / newData.length);
        }
    }

    static byte[] random(int n, long seed) {
        byte[] b = new byte[n];
        new Random(seed).nextBytes(b);
        return b;
    }

    public static void main(String[] args) throws Exception {
        byte[] base = random(8 * 1024 * 1024, 5);
        byte[] patch = random(100, 6);
        byte[] ins = new byte[base.length + 100];              // 100 bytes inserted near the start
        System.arraycopy(base, 0, ins, 0, 1000);
        System.arraycopy(patch, 0, ins, 1000, 100);
        System.arraycopy(base, 1000, ins, 1100, base.length - 1000);
        byte[] over = base.clone();                            // 100 bytes overwritten
        System.arraycopy(patch, 0, over, 4_000_000, 100);
        byte[] app = Arrays.copyOf(base, base.length + 1024 * 1024);    // 1 MiB appended
        System.arraycopy(random(1024 * 1024, 7), 0, app, base.length, 1024 * 1024);
        System.out.println("file: 8 MiB; chunks: fixed 64 KiB vs content-defined (16..256 KiB)");
        report("insert 100 B at offset 1000", base, ins);
        report("overwrite 100 B at 4 MB", base, over);
        report("append 1 MiB", base, app);
    }
}
```

Run (JDK 21, deterministic) TERMINAL

```
$ java ChunkDemo.java
file: 8 MiB; chunks: fixed 64 KiB vs content-defined (16..256 KiB)
insert 100 B at offset 1000  fixed 64 KiB     chunks=129 upload=129 chunks  8388708 B (100.0%)
insert 100 B at offset 1000  content-defined  chunks=104 upload=  1 chunks    54484 B (0.6%)
overwrite 100 B at 4 MB      fixed 64 KiB     chunks=128 upload=  1 chunks    65536 B (0.8%)
overwrite 100 B at 4 MB      content-defined  chunks=104 upload=  1 chunks   245823 B (2.9%)
append 1 MiB                 fixed 64 KiB     chunks=144 upload= 16 chunks  1048576 B (11.1%)
append 1 MiB                 content-defined  chunks=116 upload= 13 chunks  1064131 B (11.3%)
```

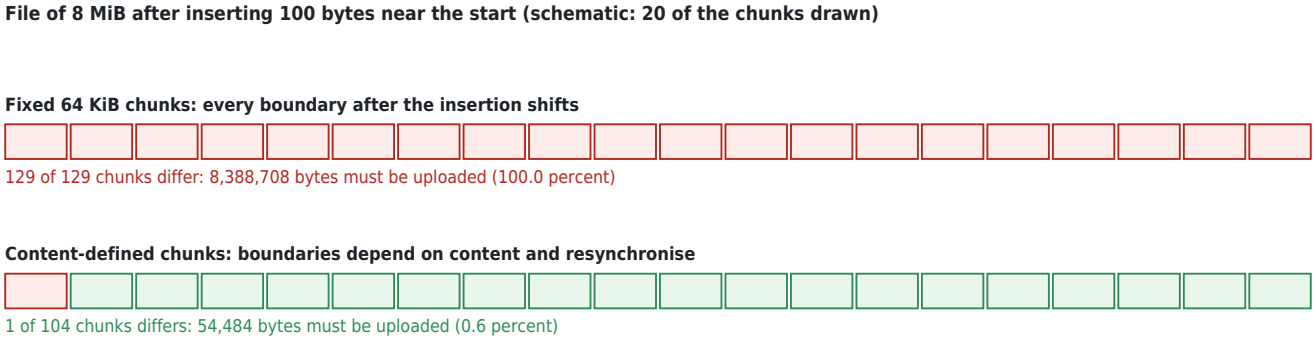


Figure 2.79. Inserting 100 bytes at the start moves every fixed boundary, so all 129 chunks differ; content-defined boundaries resynchronise after the insertion and only 1 of 104 chunks differs.

What to notice: for the insertion, fixed chunking uploads 100 percent of the file and content-defined chunking uploads 0.6 percent, a reduction by a factor of about 150. For an overwrite in place (no shift) fixed chunking is slightly better (one chunk of 64 KiB against one content-defined chunk of 240 KiB, because content-defined chunks are variable and may be large). For an append both strategies upload about the appended 1 MiB. The result explains why synchronisation clients that must handle editors that rewrite files (insertions, reordered sections, compressed archives) use content-defined chunking, while workloads with in-place updates of fixed-size records can use fixed chunks.

Deduplication and storage cost

Chunks are addressed by the SHA-256 of their content, so identical chunks (the same file in two folders, a copy shared between users, unchanged parts of a new revision) are stored once. Cross-user deduplication has a privacy side effect: if the server answers "I already have this chunk" a user can test whether another user stored a specific file. A common countermeasure is to require proof that the client possesses the content (for example by hashing a server-supplied random challenge together with the chunk) before the server treats the chunk as present, or to limit deduplication to a single account or team. Durability is obtained by erasure coding of the stored chunks rather than by three full replicas. The estimate below shows the combined effect on 500 PB of logical data under assumed ratios.

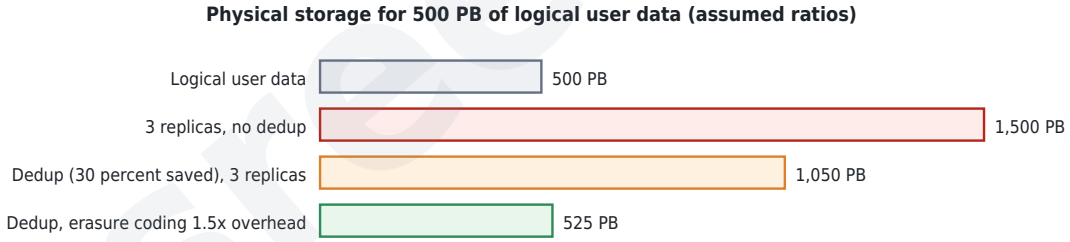


Figure 2.80. Deduplication (30 percent assumed) and erasure coding at 1.5 times overhead reduce physical storage from 1,500 PB (three replicas) to 525 PB.

Conflict resolution

Policy	Mechanism	Strength	Weakness
Last writer wins	Later commit overwrites	Simple	Silently destroys one user's edits
Conflicted copy	Reject the stale commit; client saves its version as a new file	No data loss; no merge logic	A person must reconcile copies
Merge (operational transformation or CRDT)	Combine edits automatically	Seamless for structured documents	Requires format-specific logic; not possible for opaque files

Conflict policies.

Decision: the conflicted-copy policy for generic files, enforced by the compare-and-set on `base_rev`. Editors with a document model can layer merge on top, using the journal as the transport.

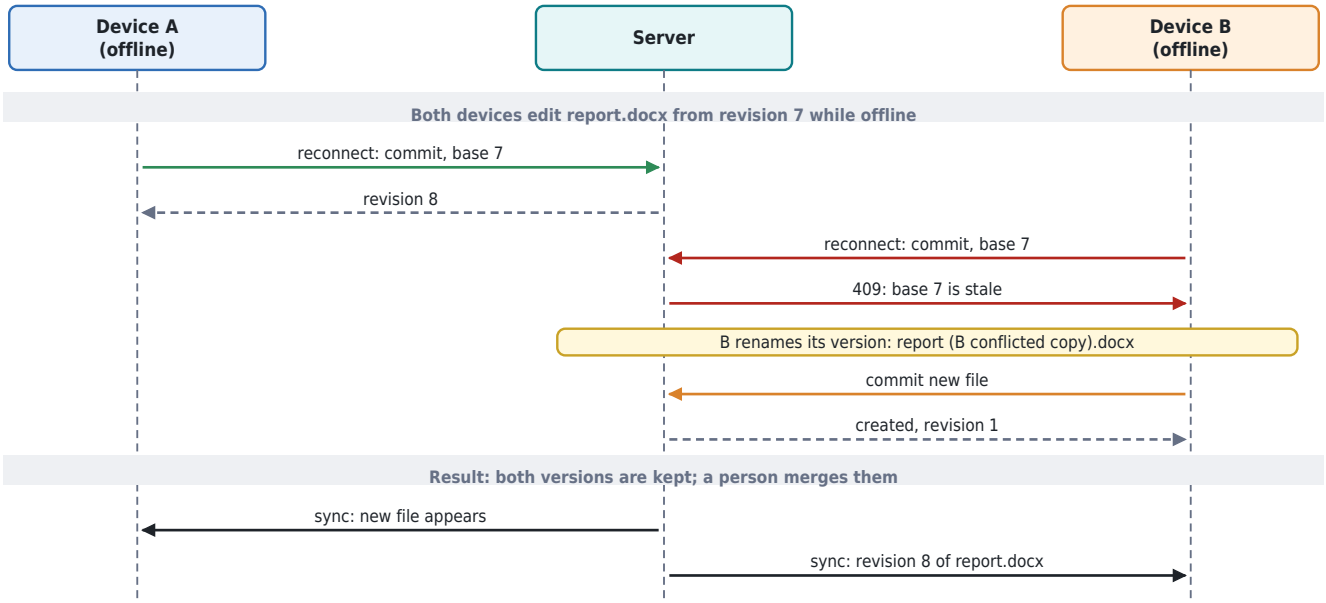


Figure 2.81. Two devices edit the same revision offline; the second commit is rejected as stale and its content is preserved as a separate file, so no edit is lost.

2.17.7 Bottlenecks, failures and real-world problems

Orphaned chunks from interrupted uploads

Scenario. A client uploads 200 chunks of a large file and its laptop is closed before the final commit. **Symptoms.** Storage grows faster than the sum of live file sizes; chunk rows exist with a reference count of zero. **Diagnosis.** Chunks older than a day with no referencing revision; the gap between bytes stored and bytes referenced widens. **Root cause.** Chunks are written before the revision that references them, by design. **Fix.** A garbage collector that deletes chunks with zero references after a grace period (for example 24 hours), which also lets a resumed upload reuse the chunks. **Prevention.** Record the upload time on each chunk, and monitor the unreferenced fraction.

Race between garbage collection and a new commit

Scenario. The collector selects an unreferenced chunk for deletion at the moment a client commits a revision that references it by deduplication. **Symptoms.** A revision whose download fails with a missing chunk; data loss for one file. **Diagnosis.** A revision lists a hash that has no storage object. **Root cause.** Deletion decided on stale reference information. **Fix.** Delete only chunks that have been unreferenced for longer than the grace period, re-check the reference count in the same transaction that marks the chunk deleted, and make the commit increment the count before it returns success. **Prevention.** Verify chunk existence at commit time (already part of the protocol) and run an integrity scanner that compares revisions with storage.

Synchronisation loop between devices

Scenario. Two clients keep rewriting a file because an application changes its modification time or content when the file is opened. **Symptoms.** Constant sync traffic for one file; a long revision history; battery drain. **Diagnosis.** A rising revision count for one path, with revisions whose chunk lists are identical or alternate between two values. **Root cause.** Change detection by modification time rather than by content hash. **Fix.** Compare content hashes before committing; ignore a commit whose chunk list equals the latest revision; rate limit revisions per file. **Prevention.** Exclude temporary and lock files by pattern and alert on revision counts per path.

Initial synchronisation storm

Scenario. A new device or a team of 10,000 users begins synchronising several terabytes at once. **Symptoms.** Metadata service latency rises; block service bandwidth saturates; other users' sync slows. **Diagnosis.** A single namespace or a single tenant dominates request counts and bytes. **Root cause.** No per-tenant limits on listing and chunk transfer. **Fix.** Per-tenant and per-device rate limits, large batched `changes` pages, and prioritisation of small recent changes over bulk transfer. **Prevention.** Capacity tests with an onboarding scenario, and a scheduler that throttles bulk synchronisation during peak hours.

Hot shared namespace

Scenario. A shared folder with 100,000 members receives frequent changes. **Symptoms.** Journal appends and long-poll wake-ups on one shard saturate it; notifications lag. **Diagnosis.** One namespace id dominates both write rate and notification fan-out. **Root cause.** A namespace is a single consistency domain, so one shard serves all its traffic. **Fix.** Coalesce notifications (a poll response says "changed" once per interval, not once per change), batch journal reads, and cache recent journal pages. **Prevention.** Limit the membership of editable namespaces and treat very large shares as read-only publication.

2.17.8 Trade-offs and extensions

- Chunk size trades index size and request count against deduplication and delta size; the choice of about 4 MiB targets the former, content-defined boundaries recover much of the latter.
- Strong consistency per namespace limits the write rate of one shared folder to the capacity of one shard; this is acceptable for the intended workload and enforced by limits.
- Client-side chunking and hashing costs CPU and battery on the device but moves the work off the servers and enables deduplication before upload.
- Extension: end-to-end encryption removes cross-user deduplication and server-side preview or search, because the server cannot see content.
- Extension: LAN synchronisation lets devices on the same network exchange chunks directly, using the chunk hash as the identifier.

2.17.9 Questions to expect

Why separate chunk upload from the metadata commit?

Chunks are large, idempotent and addressed by content, so they can be uploaded in parallel, resumed and deduplicated without coordination. The commit is small and needs strong consistency. Separating them keeps the consistent component light and makes interrupted uploads harmless.

How does a device know what changed?

It stores the cursor of the last journal entry it applied and asks for the entries after it. A notification only triggers the request; losing a notification does not lose a change, because the next request with the same cursor returns it.

How are two simultaneous edits handled?

The commit carries the base revision; the server accepts it only if the file is still at that revision. The loser receives a conflict and stores its version as a conflicted copy, so no data is lost.

How are large files uploaded reliably?

As a sequence of chunks, each verified by its hash. A failure repeats only the missing chunks; after a restart the client calls the negotiation step again and receives the list of chunks still needed.

How are deleted files and old versions removed?

A delete creates a journal entry and a tombstone revision. Chunks lose references as revisions expire from the version history, and the collector removes chunks that stay unreferenced beyond the grace period.

How would permissions be enforced for shared folders?

Each namespace carries an access control list. The gateway authorises every metadata request against it; chunk downloads are authorised through the revision that references them (or short-lived signed URLs), never by hash alone, since a hash must not act as a capability.

Why is the notification channel separate from the metadata API?

Millions of mostly idle connections have a different resource profile from request-response traffic, as in the chat design: they need many cheap connections, while the metadata service needs transactional capacity.

stream

KEY POINTS

Key points

- Split the system into immutable, content-addressed chunks and a small strongly consistent metadata store with a per-namespace journal.
- Commit in two steps (negotiate missing chunks, then compare-and-set on the base revision) so that interrupted uploads are harmless and conflicts are detected.
- Content-defined chunking made an insertion cost 0.6 percent of the file instead of 100 percent in the experiment; choose the average chunk size for index cost and delta size.
- Devices pull changes by cursor; notifications are hints and cannot lose changes.
- Keep both versions on conflict, delete chunks only after a grace period and a reference re-check, and use erasure coding and deduplication to bound storage.

2.18 Design: Payment System and Wallet

A payment system moves money on behalf of its users: it charges a customer through an external payment service provider (PSP), records the resulting obligations, and may hold balances that users transfer among themselves (a wallet). The requirements differ from other designs in this part: the cost of an error is direct financial loss, so correctness, auditability and recoverability take priority over latency and even over availability in some paths. The system must produce the effect of each payment exactly once although every network call can fail, time out or be repeated. The central decisions are the idempotency design, the double-entry ledger as the record of truth, the handling of outcomes that are unknown, and reconciliation with the external world.

2.18.1 Requirements and scope

Functional requirements

- A merchant creates a payment for an amount and currency; the customer pays with a card or another method held by the PSP (tokenised: this system never stores card numbers).
- Payments support authorisation, capture, void and (partial) refund.
- Users hold wallet balances, top them up, and transfer between wallets.
- Every movement of money is recorded in an auditable ledger; balances are queryable.
- A daily reconciliation compares internal records with the PSP's settlement report.

Non-functional requirements

- Exactly-once effect: no double charge, no lost payment, no negative wallet balance.
- Strong consistency of the ledger; durability with synchronous replication before a payment is acknowledged.
- Payment initiation availability above 99.99 percent; latency p99 below 2 seconds, dominated by the PSP.
- Full audit trail: every state change is attributable and no record is modified or deleted.

Out of scope

Card data handling and PCI DSS compliance (reduced by tokenisation and hosted payment fields, covered by reference in the security material), fraud scoring, foreign exchange, payouts to bank accounts, tax, and regulatory reporting.

Assumptions

All volumes are assumed: 10 million PSP payments per day, 20 million wallet transfers per day, a peak factor of 10 for payments and 5 for transfers, 4 ledger entries per payment and 2 per transfer, 200 bytes per ledger entry, idempotency keys retained for 7 days.

2.18.2 Capacity estimation

Quantity	Arithmetic	Result
Payments (average / peak)	10e6 / 86,400 ; x 10	`116 / 1,157 per s`
Wallet transfers (average / peak)	20e6 / 86,400 ; x 5	`231 / 1,157 per s`
Ledger entries per day	10e6 x 4 + 20e6 x 2	`80 million`
Ledger entries per second (average)	80e6 / 86,400	`926 per s`
Ledger storage	80e6 x 200 B	`16 GB per day (5.8 TB per year)`
Idempotency keys retained	30e6 requests x 7 days x 150 B	`31.5 GB`
Settlement file per day	10e6 rows x 200 B	`2 GB`
Balance reads	50e6 per day / 86,400	`579 per s`

Capacity estimate (inputs assumed).

The throughput is low by the standards of the other designs: a few thousand writes per second fit one sharded relational cluster with synchronous replication. The engineering effort therefore goes into correctness under failure rather than into scale. Storage is also modest and grows linearly, since ledger entries are never deleted.

2.18.3 API design

Endpoint	Request / response	Notes
`POST /v1/payments`	Header Idempotency-Key. Body: merchant_id, amount_minor, currency, payment_method_token. Response 201: payment_id, state	Amount is an integer in minor units
`POST /v1/payments/{id}/capture`	Header Idempotency-Key. Body: amount_minor. Response 200	Capture at most the authorised amount
`POST /v1/payments/{id}/refunds`	Header Idempotency-Key. Body: amount_minor. Response 201	Cumulative refunds cannot exceed the captured amount
`GET /v1/payments/{id}`	Response: state, amounts, history	Read from the primary for read-your-writes
`POST /v1/wallets/{id}/transfers`	Header Idempotency-Key. Body: to_wallet, amount_minor, currency. Response 201 or 422 (insufficient funds)	Atomic debit and credit
`GET /v1/wallets/{id}/balance`	Response: amount_minor, as_of	Derived from the ledger
`POST /v1/webhooks/psp`	Signed event: event_id, type, psp_ref, status	Signature verified; deduplicated by event_id

Payment and wallet API.

Money is represented as integers in the smallest currency unit with an explicit currency code. Floating-point numbers are never used, because values such as 0.1 have no exact binary representation and rounding errors accumulate; fee splits that do not divide evenly are allocated by a defined rule (the remainder goes to a named account) so that the entries still sum to the total. Per the Stripe API documentation on idempotent requests, a key may be up to 255 characters, the first result for a key is stored and returned on retries (including error results), a retry with different parameters is rejected, and keys may be removed after at least 24 hours. This design adopts the same contract for its own API and forwards a derived key to the PSP.

2.18.4 Data model

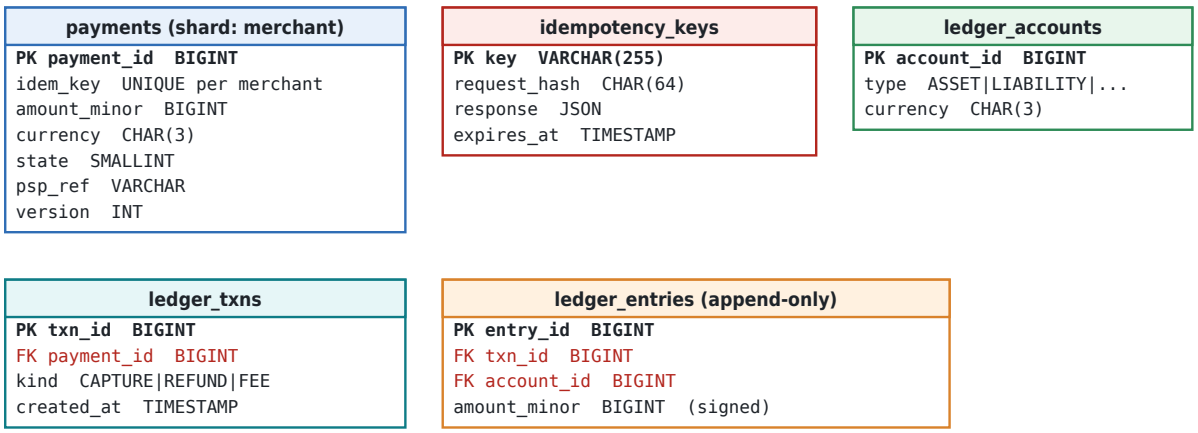


Figure 2.82. Payments carry a unique idempotency key and a version for compare-and-set; the ledger consists of transactions and append-only entries; stored idempotency responses make retries return the original result.

The ledger is the source of truth for money inside the system. A ledger transaction groups two or more entries whose signed amounts sum to zero (debits negative, credits positive in this representation). Entries are only ever inserted. The balance of an account is the sum of its entries; a cached balance column may be kept for fast reads and for the overdraft check, but it is maintained in the same database transaction as the entries and can always be recomputed and verified from them. The payment row holds the business state; the ledger holds the financial consequences.

Transaction 1: capture of 100.00 (minor units: 10000)

account	debit	credit
customer_funds	10000	
merchant_payable		9800
platform_fee_revenue		200
total	**10000**	**10000**

Transaction 2: refund of 40.00

account	debit	credit
merchant_payable	4000	
customer_funds		4000
total	**4000**	**4000**

Rules: every transaction has total debits equal to total credits; entries are never updated or deleted; a correction is a new transaction that reverses an earlier one; balances are sums of entries.

Balance of merchant_payable now: 9800 - 4000 = 5800

Amounts are integers in minor units with a currency code; floating point is never used for money.

Figure 2.83. A capture creates one balanced transaction; a refund creates a second balanced transaction that reverses part of the first, and nothing is edited or deleted.

2.18.5 High-level architecture

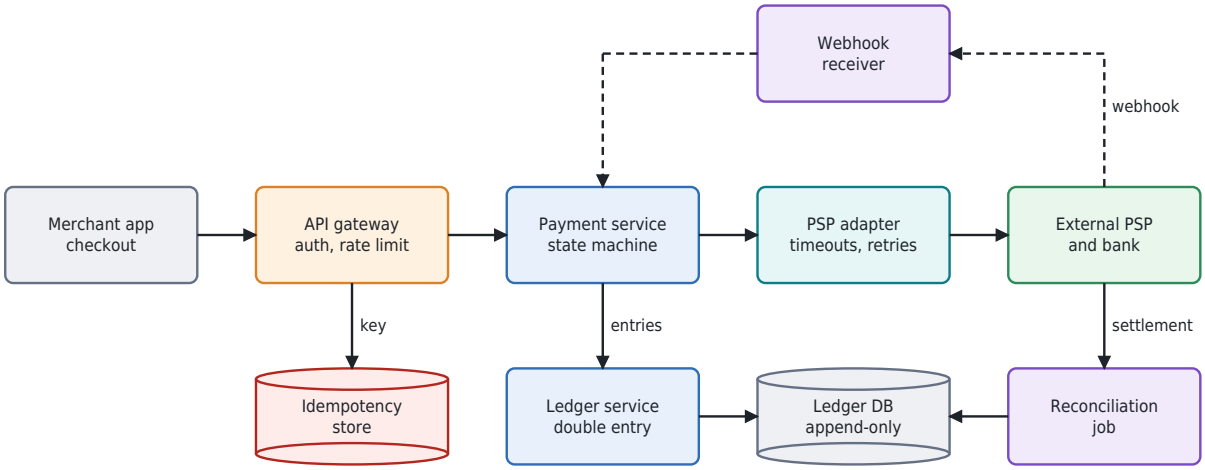


Figure 2.84. The payment service coordinates the idempotency store, the PSP adapter and the ledger; the PSP reports outcomes asynchronously through webhooks and daily through the settlement file used by reconciliation.

Payment flow

1. The merchant calls `POST /v1/payments` with an Idempotency-Key. The gateway authenticates the caller; the payment service inserts the key into the idempotency store (a unique insert). If the key exists, the stored response is returned and processing ends.
2. The payment row is created in state CREATED and moved to PROCESSING in the same transaction that records the intent to call the PSP.
3. The PSP adapter calls the PSP with a key derived from the payment id, with a timeout and bounded retries.
4. On an answer, the payment service updates the state (AUTHORIZED or FAILED) with a compare-and-set on the version, writes the ledger transaction when money is committed (capture), stores the response under the idempotency key, and replies to the merchant.
5. If there is no answer, the state becomes UNKNOWN. A background resolver queries the PSP by key and, in parallel, the webhook receiver may deliver the outcome; whichever arrives first resolves the payment, and the second is recognised as a duplicate.

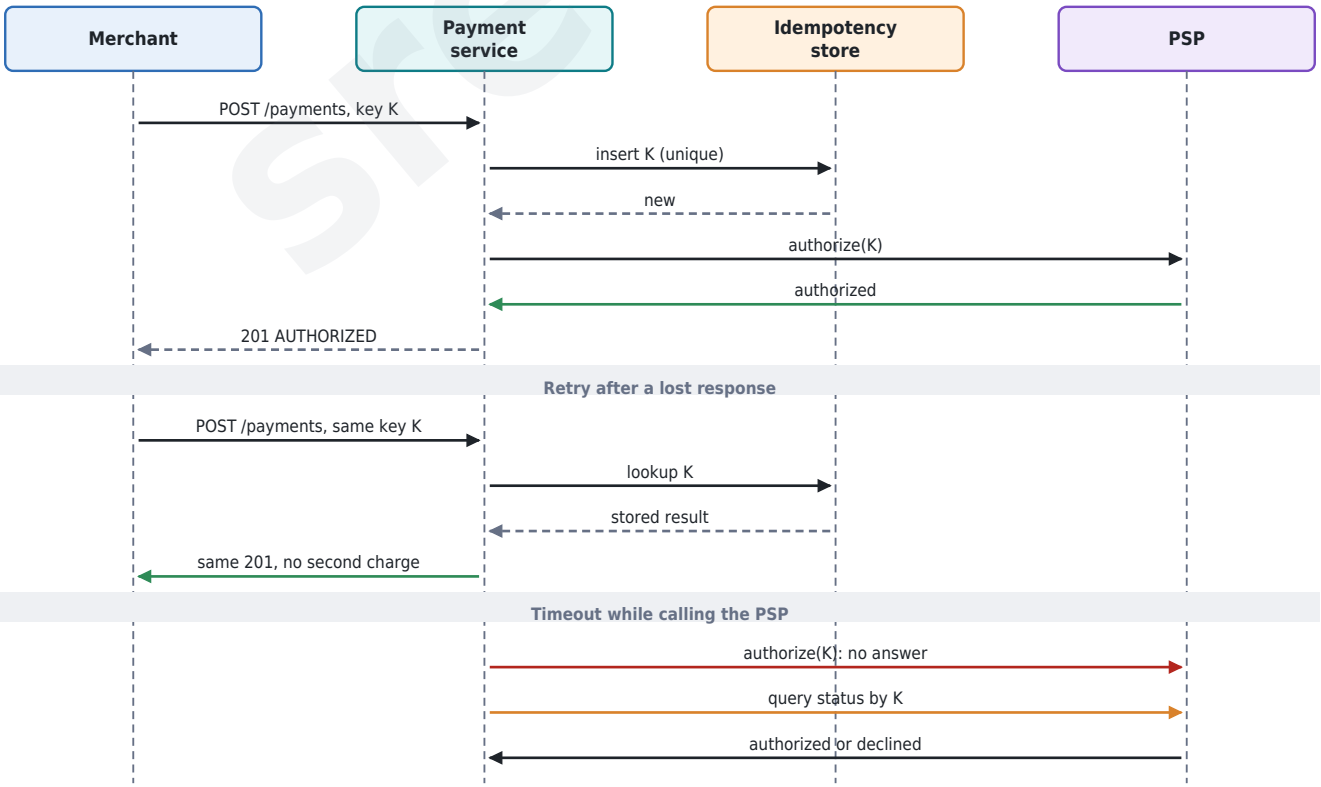
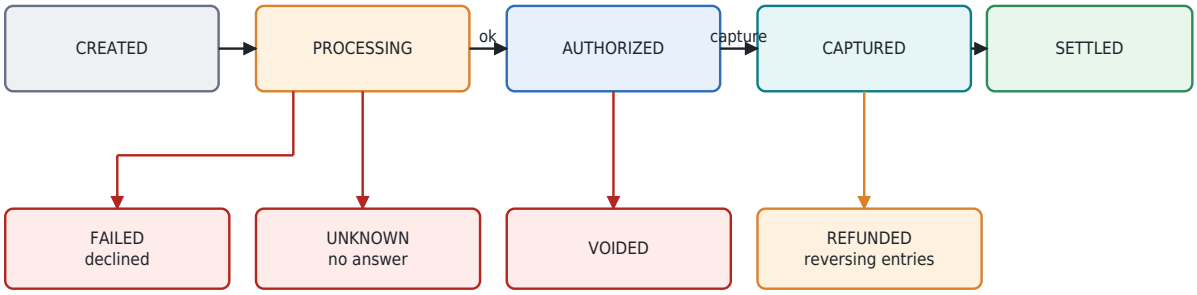


Figure 2.85. A retry with the same key returns the stored result and never reaches the PSP a second time; a timeout leaves the outcome unknown until the PSP is asked.



UNKNOWN (timeout or lost reply) is resolved by querying the PSP with the same idempotency key.

Figure 2.86. Every transition is a compare-and-set on the payment version; UNKNOWN is a first-class state that is resolved by querying the PSP.

2.18.6 Deep dive

Idempotency

Retries come from every layer: mobile clients, load balancers, the service itself and the PSP. The effect of a request must occur once, whatever the number of deliveries. The protocol is:

1. The client generates a unique key per logical operation (a UUID created when the user confirms the payment) and sends it on every retry.
2. The server inserts (key, request hash) with a unique constraint before doing any work. The first insert wins and proceeds.
3. A later request with the same key and the same request hash returns the stored response, or 409 if the first request is still in progress. A later request with the same key and a different request hash is rejected with 422, because it indicates a client bug.
4. The result of the operation, including a failure that is final, is stored under the key in the same transaction as the state change, so that the stored result and the effect cannot diverge.

The key must also be carried to every external side effect. The PSP call uses a key derived from the payment id and attempt purpose, so that a retry after a timeout cannot create a second charge at the provider.

Double-entry ledger and concurrent transfers

Double-entry bookkeeping guarantees a global invariant: the sum of all ledger entries is zero, so money is neither created nor destroyed by a bug in a single code path. A wallet transfer debits one account and credits another in one database transaction. Two concurrency hazards follow: an overdraft (two concurrent transfers each see enough balance) and a deadlock (two transfers lock the same two accounts in opposite orders). The experiment below creates 4 accounts with 1,000 units each and runs 1,000 transfer attempts from 20 concurrent clients over 200 distinct idempotency keys (so most attempts are retries of an earlier key, and parameters are determined by the key). The `transfer_naive` function locks the source account, waits 2 ms (modelling other work), then locks the destination; `transfer_ordered` locks both rows in one statement ordered by account id.

ledger_setup.sql SQL

```sql
DROP TABLE IF EXISTS ledger_entries, transfers, accounts CASCADE;
DROP FUNCTION IF EXISTS transfer_ordered, transfer_naive;
CREATE TABLE accounts (
  account_id int PRIMARY KEY,
  balance    bigint NOT NULL CHECK (balance >= 0)     -- cached; the ledger is the truth
);
CREATE TABLE transfers (
  idem_key   int PRIMARY KEY,                         -- idempotency key from the client
  src int NOT NULL, dst int NOT NULL, amount bigint NOT NULL,
  status     text NOT NULL
);
CREATE TABLE ledger_entries (
  entry_id   bigserial PRIMARY KEY,
  idem_key   int NOT NULL REFERENCES transfers,
  account_id int NOT NULL REFERENCES accounts,
  amount     bigint NOT NULL                          -- signed: debit < 0, credit > 0
);
INSERT INTO accounts SELECT g, 1000 FROM generate_series(1, 4) g;

CREATE FUNCTION transfer_ordered(k int, a int, b int, amt bigint) RETURNS text AS $$
```

ledger_setup.sql (continued) SQL

```
DECLARE bal bigint;
BEGIN
  INSERT INTO transfers VALUES (k, a, b, amt, 'PENDING') ON CONFLICT DO NOTHING;
  IF NOT FOUND THEN RETURN 'duplicate'; END IF;   -- a retry: the stored result stands
  IF a = b THEN
    UPDATE transfers SET status = 'REJECTED' WHERE idem_key = k;
    RETURN 'rejected';
  END IF;
  PERFORM 1 FROM accounts WHERE account_id IN (a, b)
   ORDER BY account_id FOR UPDATE;                -- canonical lock order
  SELECT balance INTO bal FROM accounts WHERE account_id = a;
  IF bal < amt THEN
    UPDATE transfers SET status = 'REJECTED' WHERE idem_key = k;
    RETURN 'rejected';
  END IF;
  INSERT INTO ledger_entries(idem_key, account_id, amount) VALUES (k, a, -amt), (k, b, amt);
  UPDATE accounts SET balance = balance - amt WHERE account_id = a;
  UPDATE accounts SET balance = balance + amt WHERE account_id = b;
  UPDATE transfers SET status = 'OK' WHERE idem_key = k;
  RETURN 'ok';
END $$ LANGUAGE plpgsql;

CREATE FUNCTION transfer_naive(k int, a int, b int, amt bigint) RETURNS text AS $$
DECLARE bal bigint;
BEGIN
  INSERT INTO transfers VALUES (k, a, b, amt, 'PENDING') ON CONFLICT DO NOTHING;
  IF NOT FOUND THEN RETURN 'duplicate'; END IF;
  IF a = b THEN
    UPDATE transfers SET status = 'REJECTED' WHERE idem_key = k;
    RETURN 'rejected';
  END IF;
  SELECT balance INTO bal FROM accounts WHERE account_id = a FOR UPDATE;   -- source first
  PERFORM pg_sleep(0.002);
  PERFORM 1 FROM accounts WHERE account_id = b FOR UPDATE;                 -- then destination
  IF bal < amt THEN
    UPDATE transfers SET status = 'REJECTED' WHERE idem_key = k;
    RETURN 'rejected';
  END IF;
  INSERT INTO ledger_entries(idem_key, account_id, amount) VALUES (k, a, -amt), (k, b, amt);
  UPDATE accounts SET balance = balance - amt WHERE account_id = a;
  UPDATE accounts SET balance = balance + amt WHERE account_id = b;
  UPDATE transfers SET status = 'OK' WHERE idem_key = k;
  RETURN 'ok';
END $$ LANGUAGE plpgsql;
```

run_ledger.sh BASH

```
#!/bin/bash
# Double-entry transfers on PostgreSQL 16: 20 clients, 1,000 attempts, 200 idempotency keys
export PGOPTIONS="-c client_min_messages=warning -c deadlock_timeout=50ms"
export PGHOST=/tmp/osw_y4/pg PGPORT=22400 PGUSER=postgres
psql -X -q -d postgres -c "DROP DATABASE IF EXISTS pay" -c "CREATE DATABASE pay"
for fn in transfer_naive transfer_ordered; do
  psql -X -q -d pay -f ledger_setup.sql
  cat > /tmp/osw_y4/p/t.sql <<SQL
\set k random(1, 200)
\set a (:k % 4) + 1
\set b ((:k / 4) % 4) + 1
\set amt (:k % 7) * 40 + 40
SELECT $fn(:k, :a, :b, :amt);
SQL
  echo "== $fn"
  pgbench -n -c 20 -j 2 -t 50 --failures-detailed -f /tmp/osw_y4/p/t.sql pay \
    | grep -E "failed transactions|deadlock"
  psql -X -At -d pay \
    -c "SELECT 'transfers: ' || count(*) || ' (ok ' || count(*) FILTER (WHERE status='OK')
        || ', rejected ' || count(*) FILTER (WHERE status='REJECTED') || ')' FROM transfers" \
    -c "SELECT 'ledger entries: ' || count(*) || '  sum of amounts: ' || coalesce(sum(amount),0)
        FROM ledger_entries" \
    -c "SELECT 'accounts total: ' || sum(balance) || '  min balance: ' || min(balance)
        FROM accounts" \
```

run_ledger.sh (continued) BASH

```
    -c "SELECT 'accounts disagreeing with ledger: ' || count(*) FROM accounts a
        WHERE a.balance <> 1000 + coalesce((SELECT sum(amount) FROM ledger_entries l
                                            WHERE l.account_id = a.account_id), 0)"
done
```

Run (PostgreSQL 16.15, deadlock_timeout lowered to 50 ms; the failure count varies between runs) TERMINAL

```
$ ./run_ledger.sh
== transfer_naive
number of failed transactions: 486 (48.600%)
number of deadlock failures: 486 (48.600%)
transfers: 132 (ok 80, rejected 52)
ledger entries: 160  sum of amounts: 0
accounts total: 4000  min balance: 160
accounts disagreeing with ledger: 0
== transfer_ordered
number of failed transactions: 0 (0.000%)
number of deadlock failures: 0 (0.000%)
transfers: 199 (ok 144, rejected 55)
ledger entries: 288  sum of amounts: 0
accounts total: 4000  min balance: 520
accounts disagreeing with ledger: 0
```



Figure 2.87. With the unordered lock sequence, nearly half of the attempts were chosen as deadlock victims and rolled back; with a canonical lock order none failed.

What to notice: the invariants held in both runs, which is the point of using database transactions for the ledger: the sum of entries is zero, the account total is still 4,000, no balance went negative (the `CHECK` constraint and the explicit balance test), and the cached balances agree with the entries. The unordered variant paid for correctness with availability: 486 of 1,000 attempts were aborted as deadlock victims and had to be retried by the caller, which is safe only because of the idempotency key. Duplicate keys were absorbed by the unique insert into `transfers` (199 distinct keys in the ordered run, each executed once). The ordered variant yields identical financial results without aborts; the rule is to acquire row locks in a canonical order, here ascending account id.

Unknown outcomes and webhooks

A call to the PSP can end in three ways: success, a definitive failure, or no answer. The third must not be treated as failure (the customer may have been charged) or as success. The payment enters UNKNOWN; the resolver queries the PSP by the idempotency key until it obtains a definitive answer, and an unresolved payment after a limit is escalated to manual review. Webhooks provide the same information asynchronously. They can arrive late, twice or out of order, so the handler verifies the signature, deduplicates by event id, and applies a transition only if it is valid from the current state (a late AUTHORIZED event cannot move a CAPTURED payment backwards).

Reconciliation

The PSP's settlement report is the independent record of what money actually moved. A daily job matches each report row with a captured payment on provider reference, amount and currency and sorts the differences into classes.

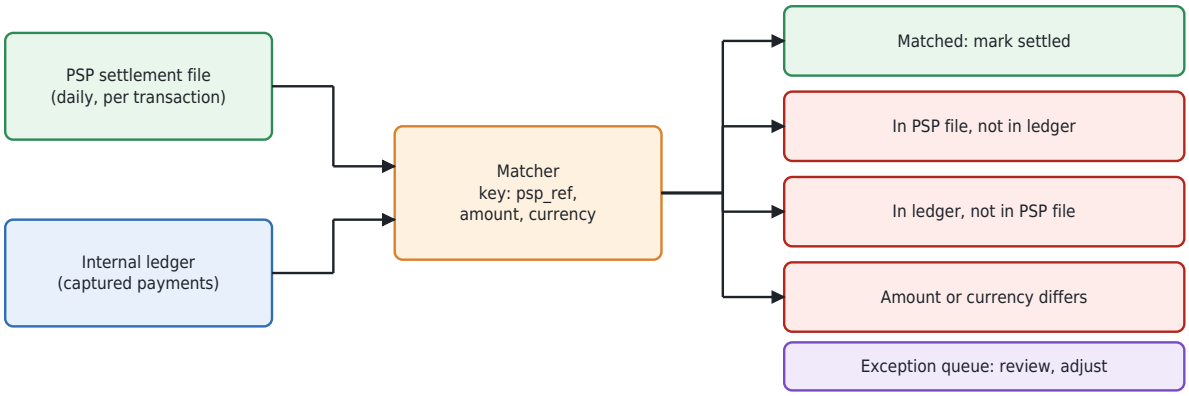


Figure 2.88. Matched rows are marked settled; three classes of difference go to an exception queue, where they are corrected by new compensating ledger transactions and never by editing history.

Typical causes are a webhook lost before capture was recorded (in the report, not in the ledger), a capture recorded but never settled by the provider (in the ledger, not in the report), and amount differences from fees or currency conversion. Corrections are new ledger transactions that reference the original, which preserves the audit trail.

Hot accounts

Every payment credits the platform fee account. At 1,157 payments per second a single account row receives all those updates, serialising them on one row lock, exactly as a hot SKU does in the e-commerce design. The remedy is the same: split the account into N sub-accounts (the entry names one chosen by hashing the payment id) and compute the fee balance as their sum when needed; per-entry inserts do not conflict, and only the cached balance of each sub-account is updated.

2.18.7 Bottlenecks, failures and real-world problems

Double charge after a lost response

Scenario. The PSP charges the card; the response is lost; the payment service retries. **Symptoms.** The customer is charged twice; a support ticket follows. **Diagnosis.** Two PSP transactions for one payment id, with different request keys. **Root cause.** The retry generated a new operation, or the PSP call carried no idempotency key. **Fix.** Refund the duplicate. Derive the PSP key deterministically from the payment id and purpose, and check the PSP by key before any retry after an ambiguous failure. **Prevention.** Fault-injection tests that drop a response after the provider has acted, with an assertion of exactly one charge.

Charged but not recorded

Scenario. The service crashes after the PSP confirms the charge and before the payment row is updated. **Symptoms.** The customer was charged and sees a pending or failed order; the merchant has no payment. **Diagnosis.** A payment in PROCESSING for longer than the timeout; the PSP shows a successful transaction under its key. **Root cause.** The side effect at the provider and the local state change are not atomic. **Fix.** The resolver queries the PSP for payments stuck in PROCESSING or UNKNOWN and completes them; the daily reconciliation catches anything missed. **Prevention.** Persist the intent before calling the PSP, resolve unknown outcomes automatically, and alert on the age of the oldest unresolved payment.

Overdraft through a race

Scenario. A wallet holds 100; two concurrent transfers of 80 are submitted. **Symptoms.** The balance becomes negative, or one transfer is lost. **Diagnosis.** Both transactions read the balance 100 before either wrote. **Root cause.** A balance check followed by a separate write without a lock or conditional update. **Fix.** Lock the account row before the check (as in `transfer_ordered`) or use a conditional update `SET balance = balance - 80 WHERE balance >= 80`; keep a CHECK constraint as a final guard. **Prevention.** Concurrency tests on every money-moving path and a continuous invariant monitor on balances.

Duplicate or forged webhook

Scenario. The PSP redelivers an event, or an attacker posts a fabricated success event. **Symptoms.** A payment is credited twice, or goods are released without payment. **Diagnosis.** Repeated event ids in the log; events with invalid or missing signatures. **Root cause.** The handler trusted the request without verification and without deduplication. **Fix.** Verify the signature on every event, deduplicate by event id, and on any doubt confirm the status by calling the PSP. **Prevention.** Treat webhooks as hints that trigger a status query for critical transitions.

Ledger imbalance from a partial write

Scenario. A bug writes one entry of a transaction without its counterpart. **Symptoms.** The invariant check (sum of entries is not zero) fails; balances drift from expectations. **Diagnosis.** A transaction id with a non-zero sum of entries. **Root cause.** Entries inserted in separate transactions or with a code path that skips the second leg. **Fix.** Insert all entries of a ledger transaction in one database transaction; a deferred constraint trigger or a periodic job verifies that every transaction sums to zero; repair by a compensating transaction. **Prevention.** Build ledger postings through one library function that accepts a balanced set of entries and rejects an unbalanced set.

2.18.8 Trade-offs and extensions

- Synchronous replication of the ledger raises write latency by a network round trip and removes the loss of acknowledged payments on failover; for money this cost is accepted.
- A cached balance accelerates reads and overdraft checks but is a second representation that must be verified against the entries; some systems compute balances only from entries and snapshot periodically.
- Strong consistency limits cross-region active-active operation; a common design keeps one write region per account and fails over deliberately.
- Extension: multi-currency wallets keep one ledger account per currency and record the exchange as a transaction between currency accounts with an explicit rate.
- Extension: payouts add an asynchronous workflow with its own states and reconciliation against bank files.

2.18.9 Questions to expect

How is a double charge prevented?

By an idempotency key on every request, stored with its result before side effects occur, and by passing a deterministic key to the PSP. After an ambiguous failure the system queries the PSP before retrying.

Why double-entry bookkeeping?

It makes every movement a balanced transaction, so the sum of all entries is zero, and a partial or erroneous posting is detectable by a simple check. It also yields an audit trail and balances derived from immutable entries.

Why are amounts stored as integers?

Floating-point types cannot represent most decimal fractions exactly and accumulate rounding errors. Integers in minor units with a currency code are exact; rounding is explicit and allocated by rule.

What happens if the PSP times out?

The payment moves to UNKNOWN, not FAILED. The system queries the provider by key, accepts a webhook if it arrives first, and reconciles daily. A final answer is never guessed.

How is the wallet protected from overdraft under concurrency?

By locking the account rows in a canonical order before the balance check, or by a conditional update that fails when the balance is insufficient, with a CHECK constraint as a safeguard.

How is the ledger kept correct over years?

Entries are immutable; corrections are new reversing transactions; a job verifies that every transaction balances and that cached balances equal the sum of entries; reconciliation with external records runs daily.

sieran

Why not use a distributed transaction between the payment service and the PSP?

The provider does not participate in such protocols. The design uses idempotent calls, a durable record of intent, resolution of unknown outcomes and reconciliation, which together reach the same guarantee of an exactly-once effect.

KEY POINTS

Key points

- Idempotency keys on every request and on every external call make retries safe; store the result with the effect in one transaction.
- Record money in an append-only double-entry ledger; every transaction balances, and balances derive from entries.
- Treat no answer as UNKNOWN, resolve it by querying the provider, and use reconciliation as the final safety net.
- Acquire row locks in a canonical order: in the experiment the unordered sequence aborted 486 of 1,000 attempts with deadlocks and the ordered sequence aborted none.
- Represent money as integer minor units with a currency; correct errors with compensating transactions, never by editing history.

sreeram

PART 3

Low-Level Design in Java

Class-level design: principles, patterns, an interview method, and twelve machine-coding problems with code that was compiled and run.

In this part

3.1 Object-Oriented Design Principles
3.2 Design Patterns for LLD
3.3 The LLD Interview Process
3.4 LLD: Parking Lot
3.5 LLD: LRU and LFU Cache
3.6 LLD: Elevator System
3.7 LLD: Splitwise, Expense Sharing
3.8 LLD: Vending Machine
3.9 LLD: Rate Limiter
3.10 LLD: Logger Framework
3.11 LLD: Movie Ticket Booking, Seat Hold
3.12 LLD: Tic-Tac-Toe
3.13 LLD: Pub-Sub Event Bus
3.14 LLD: Task Scheduler
3.15 LLD: Connection Pool

3.1 Object-Oriented Design Principles

High-level design decides which services and data stores exist. Low-level design (LLD) decides how the code inside one service is shaped: which classes exist, what each one is responsible for, how they depend on one another, and where the program is expected to change. A design that passes every test today but forces edits in five classes for each new requirement is a poor design, and LLD interviews and code reviews measure exactly that property.

This chapter states the principles used throughout the rest of the part. Each principle is shown with a violation and a repair, written in Java 21 and executed. The complete listing for the chapter is a single file, `Principles.java`; the output of the whole program is shown at the end of the chapter.

3.1.1 Concept and mechanism

Encapsulation and abstraction

Encapsulation keeps the state of an object private and exposes behaviour that preserves the invariants of that state. **Abstraction** exposes what an object does and hides how. The two are related but distinct: encapsulation protects data, abstraction hides detail.

Violation: public mutable state JAVA

```java
static class LooseAccount {
    public long balance;                // any caller can write any value
}
```

Repair: private state, behaviour enforces the invariant JAVA

```java
static final class Account {
    private long balance;               // minor units, never negative

    Account(long opening) {
        if (opening < 0) throw new IllegalArgumentException("opening < 0");
        this.balance = opening;
    }
    void withdraw(long amount) {
        if (amount <= 0 || amount > balance) throw new IllegalArgumentException("bad amount");
        balance -= amount;
    }
    long balance() { return balance; }
}
```

The repaired `Account` cannot reach a negative balance through any public method. The invariant is stated and enforced in one place, so a reader needs to inspect only that class to trust it.

Inheritance versus composition

Inheritance models an is-a relationship and couples the subclass to every public and protected member of its parent, including members added in later versions. Composition models has-a: the class holds a collaborator and forwards only the calls it chooses to expose. The default choice is composition; inheritance is appropriate when the subclass is genuinely substitutable for the parent in every context (see the Liskov principle below) and the parent was designed for extension.

Violation: BadStack inherits the whole ArrayList API JAVA

```
static class BadStack<T> extends ArrayList<T> {      // inherits add(int, T), remove(int) ...
    void push(T t) { add(t); }
    T pop() { return remove(size() - 1); }
}
```

Repair: Stack2 wraps a Deque and exposes three methods JAVA

```
static class Stack2<T> {                              // composition: only push/pop are exposed
    private final Deque<T> items = new ArrayDeque<>();
    void push(T t) { items.addLast(t); }
    T pop() { return items.removeLast(); }
    int size() { return items.size(); }
}
```

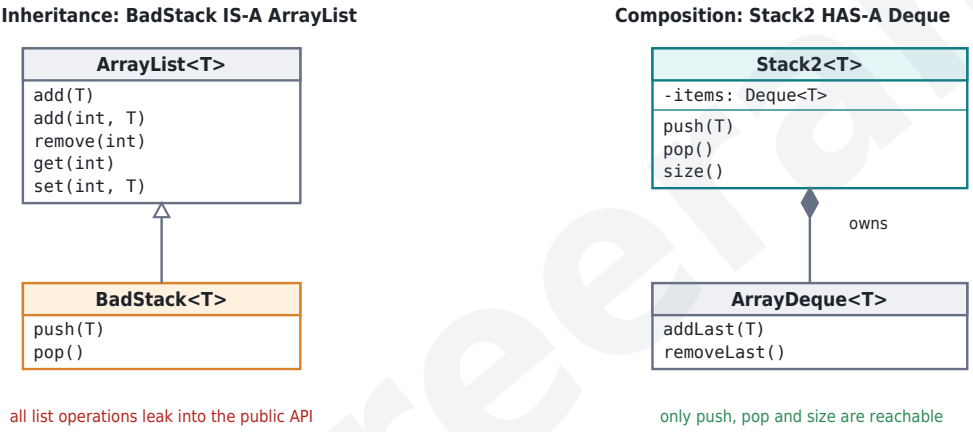


Figure 3.1. BadStack inherits `add(int, T)` and `remove(int)` and therefore allows non-LIFO access; Stack2 holds the deque privately and exposes only `push`, `pop` and `size`.

Polymorphism

Polymorphism lets a caller depend on an interface while the concrete behaviour is selected at run time by the receiver. It is the mechanism behind every pattern in the next chapter: the caller invokes `strategy.price(base)` and the object decides what that means. In Java, dynamic dispatch on interface and virtual methods provides it; the cost is one indirect call, which the JIT compiler usually removes for monomorphic call sites.

3.1.2 SOLID with a violation and a repair for each principle

S: single responsibility

A class should have one reason to change. `InvoiceBad` below changes when the tax rate changes, when the output format changes and when the storage changes: three reasons. The repair separates calculation (`Invoice`), presentation (`InvoiceRenderer`) and persistence (`InvoiceStore`).

Violation: calculation, formatting and storage in one class JAVA

```
static class InvoiceBad {                 // three reasons to change: maths, format, storage
    double total(List<Double> lines) { return lines.stream().mapToDouble(d -> d).sum() * 1.18; }
    String render(double total) { return "TOTAL " + total; }
    void save(String text) { System.out.println("INSERT INTO invoice ... " + text); }
}
```

sreeram

Repair: one responsibility per type JAVA

```java
record Invoice(List<Double> lines) {
    double total() { return lines.stream().mapToDouble(d -> d).sum() * 1.18; }
}
interface InvoiceRenderer { String render(Invoice inv); }
interface InvoiceStore { void save(Invoice inv, String text); }
static class TextRenderer implements InvoiceRenderer {
    public String render(Invoice inv) { return String.format("TOTAL %.2f", inv.total()); }
}
```

O: open for extension, closed for modification

New behaviour should be added by writing new code, not by editing tested code. The `switch` in `discountBad` must be edited for every new customer category, and each edit risks the existing branches. With a `DiscountPolicy` interface, a new policy is a new class.

Violation: a switch that grows with every variant JAVA

```java
static double discountBad(String type, double amount) {
    switch (type) {                      // every new customer type edits this method
        case "REGULAR": return amount * 0.95;
        case "PREMIUM": return amount * 0.90;
        default: return amount;
    }
}
```

Repair: a new discount is a new implementation JAVA

```java
interface DiscountPolicy { double apply(double amount); }
record Percent(double rate) implements DiscountPolicy {
    public double apply(double amount) { return amount * (1 - rate); }
}
record FlatOff(double off) implements DiscountPolicy {
    public double apply(double amount) { return Math.max(0, amount - off); }
}
```

A `switch` is not wrong in itself. Over a closed set that never grows, for example the four suits of a card deck, a `switch` over an enum or a sealed type is clearer than polymorphism. The principle applies where variants are expected to arrive.

L: Liskov substitution

Code written against a base type must keep working when a subtype is supplied. A `Square` that overrides `setWidth` to also change the height breaks the contract of `Rect`, because a caller that sets width 5 and height 4 expects an area of 20.

Violation: Square narrows the postcondition of Rect JAVA

```java
static class Rect {
    protected int w, h;
    void setWidth(int w) { this.w = w; }
    void setHeight(int h) { this.h = h; }
    int area() { return w * h; }
}
static class Square extends Rect {         // forces w == h, breaking the Rect contract
    @Override void setWidth(int w) { this.w = w; this.h = w; }
    @Override void setHeight(int h) { this.w = h; this.h = h; }
}
static int stretch(Rect r) {               // valid for any Rect: 5 x 4 = 20
    r.setWidth(5);
    r.setHeight(4);
    return r.area();
}
```

Repair: immutable siblings under a sealed interface JAVA

```java
sealed interface Shape permits Rectangle, Sq {
    int area();
}
record Rectangle(int w, int h) implements Shape { public int area() { return w * h; } }
record Sq(int side) implements Shape { public int area() { return side * side; } }
```

sreeram

The repair removes the false is-a relationship. Mutability was what made the relationship unsound; as immutable values, `Rectangle` and `Sq` share only the operation `area()`.

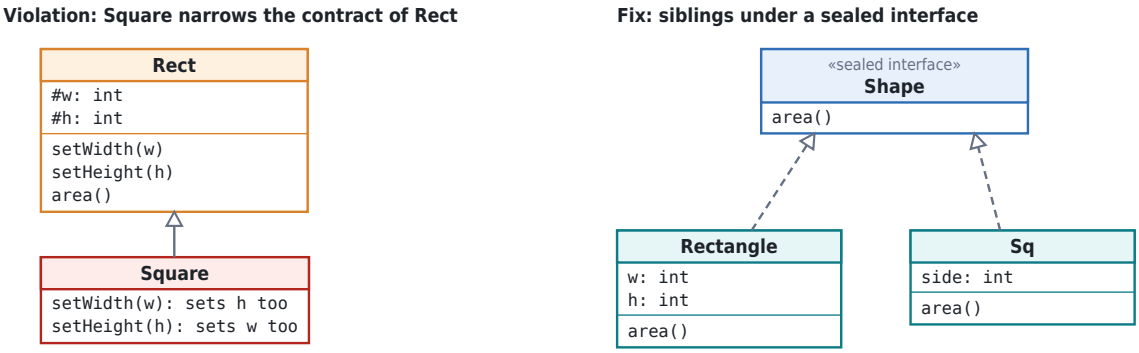


Figure 3.2. The violation (left) uses inheritance to forbid a combination that the parent allows; the repair (right) makes both shapes siblings that implement the one operation they share.

I: interface segregation

Clients should not be forced to depend on methods they do not use. A `MultiFunctionDevice` interface forces a basic printer to implement `scan` and `fax` with empty or exception-throwing bodies. Small role interfaces (`Printer`, `Scanner2`) let each class implement exactly what it supports.

Violation: one fat interface JAVA

```java
interface MultiFunctionDevice {            // a basic printer must stub scan() and fax()
    void print(String doc);
    void scan(String doc);
    void fax(String doc);
}
```

Repair: role interfaces JAVA

```java
interface Printer { void print(String doc); }
interface Scanner2 { String scan(); }
static class BasicPrinter implements Printer {
    public void print(String doc) { System.out.println("print " + doc); }
}
```

D: dependency inversion

High-level policy should depend on abstractions, and low-level details should depend on the same abstractions. `OrderServiceBad` constructs `MySqlOrderRepo` itself, so the service cannot be tested without a database and cannot change its store without being edited. Passing an `OrderRepository` through the constructor inverts the dependency; this is exactly what constructor injection in Spring provides.

Violation: the service creates its own concrete dependency JAVA

```java
static class MySqlOrderRepo {
    void save(String order) { System.out.println("mysql save " + order); }
}
static class OrderServiceBad {
    private final MySqlOrderRepo repo = new MySqlOrderRepo();   // concrete, hidden dependency
    void place(String order) { repo.save(order); }
}
```

Repair: depend on an interface, inject the implementation JAVA

```java
interface OrderRepository { void save(String order); }
static class OrderService {
    private final OrderRepository repo;                          // abstraction, injected
    OrderService(OrderRepository repo) { this.repo = repo; }
    void place(String order) { repo.save(order); }
}
static class InMemoryOrderRepo implements OrderRepository {
    final List<String> rows = new ArrayList<>();
    public void save(String order) { rows.add(order); }
}
```

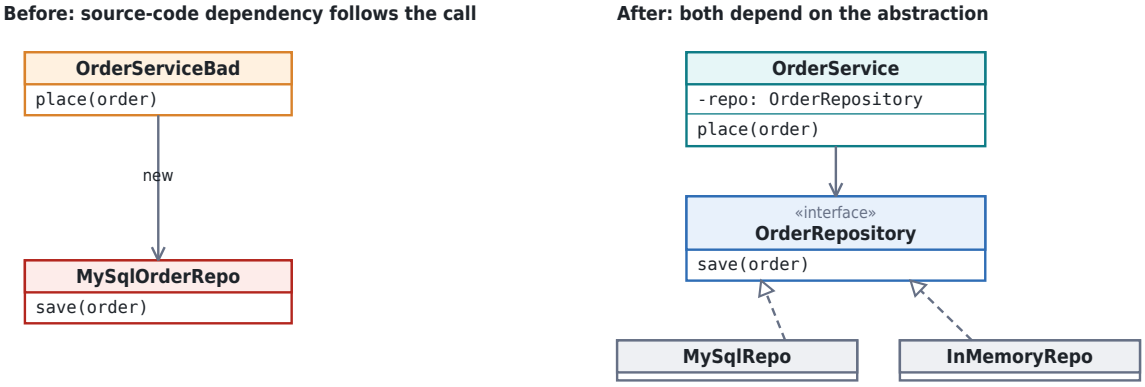


Figure 3.3. In the repaired design both the service and the two repositories point at `OrderRepository`; the source-code dependency no longer follows the direction of the call.

3.1.3 Other working rules

Rule	Statement	Typical violation
DRY	Every piece of knowledge has one authoritative representation.	The same validation copied into three controllers; a rule fixed in two of them.
KISS	Choose the simplest structure that satisfies the requirements.	An abstract factory for a class that has one implementation.
YAGNI	Do not build for requirements that do not exist yet.	A plug-in framework before a second plug-in is requested.
Law of Demeter	A method calls only its own members, its parameters and objects it creates.	`order.getCustomer().getAddress().getCity()` in a service.
High cohesion	The members of a class serve one purpose.	`Utils` classes with unrelated static methods.
Low coupling	A class knows as few other classes as possible, and only through abstractions.	A domain class importing a web framework type.

Working rules for everyday design decisions.

DRY and YAGNI pull in opposite directions when applied carelessly. Duplicating code twice is acceptable; the third occurrence is the signal that a shared abstraction has a stable shape. Abstracting at the first occurrence usually produces the wrong abstraction.

Law of Demeter

Callers talk to Order; the chain of getters is hidden JAVA

```
record Address(String city) {}
record Customer(Address address) { String city() { return address.city(); } }
record Order(Customer customer) {
    String shippingCity() { return customer.city(); }       // callers talk to Order only
}
```

The repair is sometimes called "tell, do not ask": `Order` offers `shippingCity()`, and the navigation through `Customer` and `Address` becomes a private matter of the order model. Fluent builders and stream pipelines are not violations: each call returns the same kind of object.

3.1.4 Hands-on

The program below combines every snippet above. It was compiled with `javac --release 21` and run on JDK 21; its output is deterministic.

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out Principles.java
$ java -cp out Principles
loose balance = -500
account balance = 700
rejected: bad amount
BadStack pop = 2, but contents = [99, 1]
Stack2 pop = 2
TOTAL 177.00
discountBad = 180.0
Percent[rate=0.1] -> 180.0
FlatOff[off=25.0] -> 175.0
Rect area = 20
Square area = 16 (expected 20)
sealed Shape area = 20
print report.pdf
mysql save order-1
in-memory rows = [order-2]
city = Vijayawada
```

Three lines of the output carry the lesson. `loose balance = -500` shows an invariant broken from outside. `BadStack pop = 2, but contents = [99, 1]` shows an element inserted underneath the stack top through an inherited method. `Square area = 16 (expected 20)` shows a subtype that breaks a caller written against its parent.

3.1.5 Real-world problems and solutions

Anemic domain model with logic scattered in services

Scenario. A billing service keeps `Invoice` as a bag of getters and setters, and five service classes each recompute totals.

Symptoms. A tax change requires edits in five places; two of them are missed and the totals differ between the web screen and the exported PDF.

Diagnosis. Search for the formula shows duplicates; `Invoice` has no behaviour.

Root cause. The rule has no single owner (violation of DRY and of encapsulation).

Fix. Move the calculation into `Invoice.total()` and make the fields private.

Prevention. Review rule: logic that reads only one object's fields belongs on that object.

Inheritance used for code reuse

Scenario. `ReportExporter extends ArrayList<Row>` to reuse list operations.

Symptoms. A library upgrade adds a default method on `List`; callers start invoking it on exporters with undefined results. Serialisation of the subclass also leaks the list internals.

Diagnosis. `ReportExporter` is not a list in any domain sense; the dependency was introduced only for convenience.

Root cause. Inheritance chosen over composition.

Fix. Hold a `List<Row>` field and expose `add(Row)` and `export()`.

Prevention. Ask whether the subclass can be used wherever the parent is used. If not, use composition.

Tests that need a database

Scenario. Unit tests of `OrderService` take four minutes because the service creates a real repository.

Symptoms. Tests fail when the shared test database is unavailable; developers skip them.

Diagnosis. The constructor calls `new MySqlOrderRepo()`.

Root cause. Violation of dependency inversion.

Fix. Introduce `OrderRepository`, inject it, and use `InMemoryOrderRepo` in unit tests.

Prevention. Allow `new` of collaborators only in composition roots (the application configuration or `main`).

sreeram

3.1.6 Questions to expect

- **Q. Why is composition preferred over inheritance?** A. Inheritance binds a subclass to the full parent API and to its implementation details at compile time; composition binds only to the methods called, allows the collaborator to be replaced at run time, and avoids fragile base-class changes.
- **Q. Give a Liskov violation that is not Rectangle and Square.** A. A `ReadOnlyList` subclass of `List` whose `add` throws `UnsupportedOperationException`: callers written against `List` fail. Java's `List.of` has this shape, which is why APIs should accept `Collection` or `Iterable` when they only read.
- **Q. How do open/closed and a `switch` relate?** A. A `switch` on a type code is a signal that each new type edits the method. Polymorphism or a sealed hierarchy with pattern matching moves the variation to new types. A closed set of variants may keep the `switch`.
- **Q. What is the difference between dependency inversion and dependency injection?** A. Inversion is the design rule: depend on abstractions. Injection is a technique for supplying the implementation from outside. Spring provides injection; the rule is still the developer's responsibility.
- **Q. When does applying SOLID hurt?** A. When it creates interfaces with one implementation and no foreseeable second one. The extra indirection costs readability without buying flexibility; apply a principle where change is expected.
- **Q. What is cohesion and how is it recognised?** A. Cohesion measures how strongly the members of a class relate. A class whose methods use disjoint subsets of its fields is a candidate for splitting.

KEY POINTS

Encapsulate invariants: private state, behaviour that rejects invalid transitions.

- Prefer composition; inherit only when substitutability holds in every context.
- SRP counts reasons to change, OCP removes edits from tested code, LSP keeps subtypes honest, ISP shrinks interfaces, DIP points dependencies at abstractions.
- DRY, KISS and YAGNI are balancing forces; abstract on the third occurrence, not the first.
- Run the checks mentally: can the class be tested without infrastructure, and does a new requirement add a class or edit five?

3.2 Design Patterns for LLD

A design pattern is a named, reusable solution to a recurring design problem, stated at the level of classes and their collaboration. The value of a pattern in an interview is a shared vocabulary: saying "the elevator uses the State pattern and dispatch is a Strategy" conveys a structure in one sentence. The value in production is the same, plus the knowledge of where the pattern costs more than it returns.

Thirteen patterns that recur in machine-coding problems are covered here. Each entry gives the intent, the structure, Java code that was run, and a place where the pattern already exists in the JDK or in Spring. Two programs hold all the code: `Patterns1.java` (creational patterns, Strategy and Observer) and `Patterns2.java` (structural patterns, Command, State, Chain of Responsibility and Template Method).

3.2.1 Concept and mechanism

Pattern	Intent	Existing example
`Strategy`	Make an algorithm replaceable at run time behind an interface.	`Comparator` passed to `sort`
`Factory / Abstract Factory`	Centralise object creation; create families of related objects.	`List.of`, `DocumentBuilderFactory`
`Builder`	Construct an object with many optional parts in a readable, validated way.	`StringBuilder`, `HttpRequest.newBuilder()`
`Singleton`	Exactly one instance with a global access point.	Spring singleton-scoped beans (one per container)
`Observer`	Notify dependents when state changes without coupling to them.	Spring `ApplicationEvent` and `@EventListener`
`Decorator`	Add behaviour by wrapping an object that has the same interface.	`new BufferedInputStream(new FileInputStream(f))`
`Adapter`	Convert one interface into the one the client expects.	`InputStreamReader` (bytes to chars), `Arrays.asList`
`Command`	Represent a request as an object: queue, log or undo it.	`Runnable` submitted to an `ExecutorService`
`State`	Let an object change behaviour when its internal state changes.	Workflow and order-lifecycle engines
`Chain of Responsibility`	Pass a request along handlers until one handles it.	Servlet `FilterChain`, Spring Security filter chain
`Template Method`	Fix the skeleton of an algorithm, defer steps to subclasses.	`JdbcTemplate`, `HttpServlet.doGet`
`Proxy`	Control access to an object through a stand-in with the same interface.	Spring AOP (`@Transactional`) via JDK dynamic proxies or CGLIB
`Facade`	Offer one simple interface over a set of subsystems.	A Spring `@Service` coordinating several repositories

The thirteen patterns, grouped by the problem they solve.

3.2.2 Creational patterns

Strategy

Intent: select an algorithm at run time. The context (`Checkout`) holds a reference to the interface and delegates. This removes `if`/`switch` chains on a type code and satisfies the open/closed principle.

Strategy: pricing rules behind one interface JAVA

```java
interface PricingStrategy { double price(double base); }
static class Regular implements PricingStrategy {
    public double price(double b) { return b; }
}
static class Festival implements PricingStrategy {
    public double price(double b) { return b * 0.8; }
}

static class Checkout {
    private PricingStrategy strategy;
    Checkout(PricingStrategy s) { this.strategy = s; }
    void use(PricingStrategy s) { this.strategy = s; }      // swappable at run time
    double total(double base) { return strategy.price(base); }
}
```

Factory method and Abstract Factory

A factory hides which concrete class is instantiated. The sealed `Notifier` hierarchy makes the set of products explicit, and `NotifierFactory.create` is the single place that changes when a channel is added. An **Abstract Factory** produces a family whose members must match, for example a dark theme button with a dark theme checkbox.

Factory: one place knows the concrete classes JAVA

```java
sealed interface Notifier permits EmailNotifier, SmsNotifier {
    String send(String to, String msg);
}
record EmailNotifier() implements Notifier {
    public String send(String to, String msg) { return "EMAIL to " + to + ": " + msg; }
}
record SmsNotifier() implements Notifier {
    public String send(String to, String msg) { return "SMS to " + to + ": " + msg; }
}
static class NotifierFactory {
    static Notifier create(String channel) {            // the only place that knows the classes
        return switch (channel) {
            case "email" -> new EmailNotifier();
            case "sms" -> new SmsNotifier();
            default -> throw new IllegalArgumentException("unknown channel " + channel);
        };
    }
}
```

Abstract Factory: a consistent family of products JAVA

```java
interface Button { String draw(); }
interface Checkbox { String draw(); }
interface UiFactory { Button button(); Checkbox checkbox(); }   // a family of related products

static class DarkUi implements UiFactory {
    public Button button() { return () -> "[dark button]"; }
    public Checkbox checkbox() { return () -> "[dark checkbox]"; }
}
static class LightUi implements UiFactory {
    public Button button() { return () -> "[light button]"; }
    public Checkbox checkbox() { return () -> "[light checkbox]"; }
}
```

Builder

Use a builder when a constructor would have many parameters, several of them optional, or when the object must be validated as a whole before it exists. The product is immutable and its constructor is private, so the only route is through `build()`, where validation lives.

Builder: optional parts, validation in build() JAVA

```java
static final class HttpRequest2 {
    private final String method, url;
    private final Map<String, String> headers;
    private final int timeoutMs;

    private HttpRequest2(Builder b) {
        this.method = b.method; this.url = b.url;
        this.headers = Map.copyOf(b.headers); this.timeoutMs = b.timeoutMs;
    }
    public String toString() {
        return method + " " + url + " " + headers + " timeout=" + timeoutMs;
    }
    static Builder builder(String url) { return new Builder(url); }

    static final class Builder {
        private final String url;
        private String method = "GET";
        private final Map<String, String> headers = new HashMap<>();
        private int timeoutMs = 3000;
        private Builder(String url) { this.url = url; }
        Builder method(String m) { this.method = m; return this; }
        Builder header(String k, String v) { headers.put(k, v); return this; }
        Builder timeoutMs(int t) { this.timeoutMs = t; return this; }
        HttpRequest2 build() {
            if (timeoutMs <= 0) throw new IllegalStateException("timeout must be > 0");
            return new HttpRequest2(this);
        }
    }
}
```

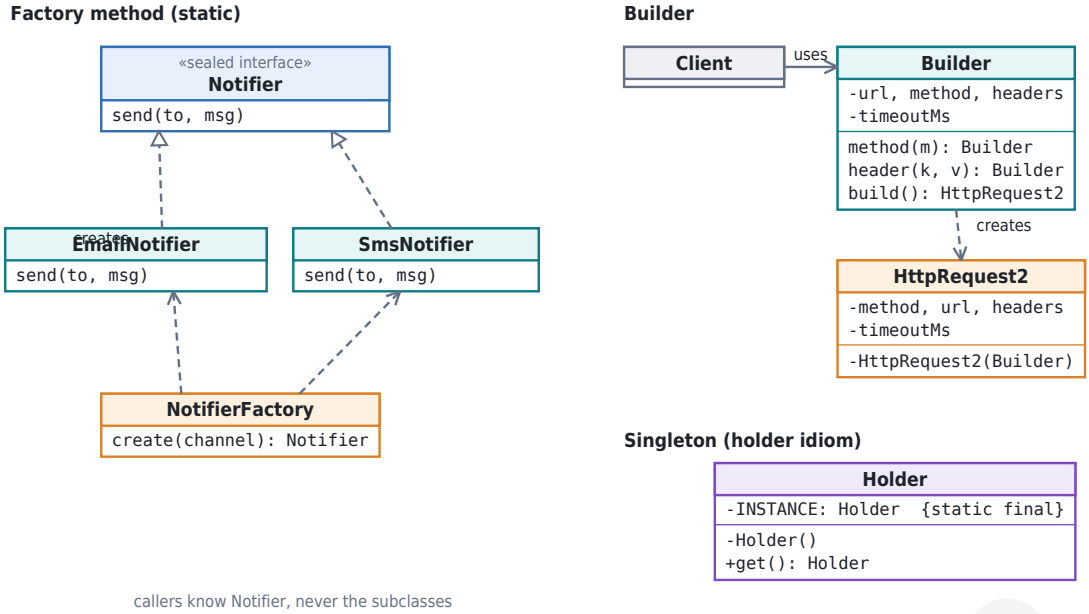


Figure 3.4. The factory is the only class that references the concrete notifiers; the builder is the only route to an `HttpRequest2`; the holder class provides the singleton instance.

Singleton

A singleton restricts a class to one instance. Three thread-safe forms are shown. The **enum** form is the shortest and is safe against serialisation and reflection attacks. The **initialisation-on-demand holder** is lazy because the nested class loads on first use, and class initialisation is guaranteed thread-safe by the JVM. **Double-checked locking** works only when the field is `volatile`; without it another thread may observe a partially constructed object.

Singleton: enum, holder and double-checked locking JAVA

```
enum ConfigEnum {                                   // serialisation-safe
    INSTANCE;
    final String env = "prod";
}
static final class Holder {                         // lazy, thread-safe, no locking
    private Holder() {}
    private static class H { static final Holder I = new Holder(); }
    static Holder get() { return H.I; }
}
static final class DoubleChecked {
    private static volatile DoubleChecked instance;   // volatile is mandatory
    private DoubleChecked() {}
    static DoubleChecked get() {
        DoubleChecked r = instance;
        if (r == null) {
            synchronized (DoubleChecked.class) {
                r = instance;
                if (r == null) instance = r = new DoubleChecked();
            }
        }
        return r;
    }
}
```

GOTCHA

A Spring singleton bean is one instance per application context, not one per JVM, and it is created by the container, not by a static method. Hand-written singletons hide dependencies, are hard to replace in tests and are shared mutable state across the whole process. Prefer a container-managed singleton and constructor injection; keep the hand-written form for genuinely process-wide resources such as a logger registry.

3.2.3 Behavioural patterns, first group

Observer

A subject keeps a list of listeners and calls each when an event occurs. The publisher knows only the `OrderListener` interface. Notice `List.copyOf(listeners)`: iterating a snapshot lets a listener subscribe or unsubscribe during notification without a `ConcurrentModificationException`. Spring offers the same model with `ApplicationEventPublisher.publishEvent` and `@EventListener`; the JDK has `PropertyChangeListener` and the `java.util.concurrent.Flow` interfaces.

Observer: publisher and listener interface JAVA

```java
interface OrderListener { void onOrderPlaced(String orderId); }

static class OrderPublisher {
    private final List<OrderListener> listeners = new ArrayList<>();
    void subscribe(OrderListener l) { listeners.add(l); }
    void place(String id) {
        for (OrderListener l : List.copyOf(listeners)) l.onOrderPlaced(id);   // snapshot
    }
}
```

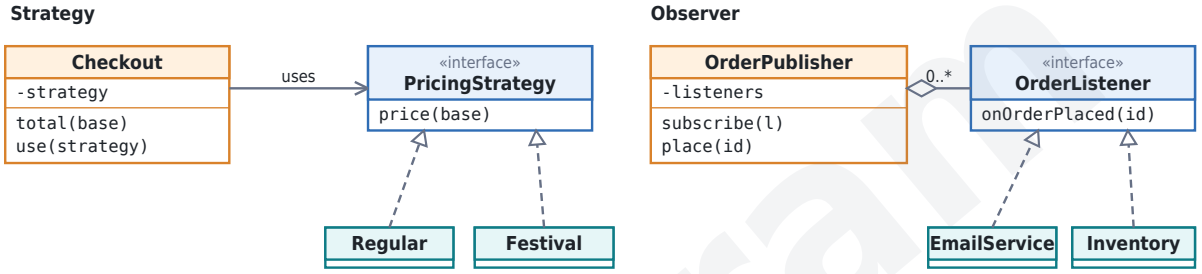


Figure 3.5. Strategy delegates to an interface; Observer holds a collection of listener interfaces (aggregation with multiplicity 0..).*

Run of the first program

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out Patterns1.java
$ java -cp out Patterns1
regular  = 1000.0
festival = 800.0
EMAIL to a@x.com: hello
SMS to +91-99: otp 123
error: unknown channel fax
[dark button] [dark checkbox]
[light button] [light checkbox]
POST https://api.example.com/orders {Accept=application/json} timeout=1500
true
true
distinct DoubleChecked instances = 1
email service: confirmation for ORD-1001
inventory service: reserve stock for ORD-1001
```

3.2.4 Structural patterns

Decorator

A decorator implements the interface it wraps and forwards calls, adding behaviour before or after. Decorators compose: `Caramel(Milk(Espresso))` costs 80 + 15 + 25. It is the alternative to a subclass explosion (`EspressoWithMilkAndCaramel`). `java.io` is the canonical case.

Decorator: stackable add-ons with the same interface JAVA

```java
interface Coffee { double cost(); String describe(); }
static class Espresso implements Coffee {
    public double cost() { return 80; }
    public String describe() { return "espresso"; }
}
static abstract class CoffeeDecorator implements Coffee {
    protected final Coffee inner;
    CoffeeDecorator(Coffee inner) { this.inner = inner; }
}
static class Milk extends CoffeeDecorator {
    Milk(Coffee c) { super(c); }
    public double cost() { return inner.cost() + 15; }
    public String describe() { return inner.describe() + " + milk"; }
}
static class Caramel extends CoffeeDecorator {
    Caramel(Coffee c) { super(c); }
    public double cost() { return inner.cost() + 25; }
    public String describe() { return inner.describe() + " + caramel"; }
}
```

Adapter

An adapter translates between an existing interface (the legacy bank takes paise as a `long`) and the one the client requires (`pay(int rupees)`). The client stays unaware of the legacy API, and the unit conversion is confined to one class.

Adapter: legacy API behind the expected interface JAVA

```java
interface PaymentGateway { String pay(int rupees); }              // what our code expects
static class LegacyBank {                                         // what we are given
    String makePaymentInPaise(long paise) { return "legacy paid " + paise + " paise"; }
}
static class LegacyBankAdapter implements PaymentGateway {
    private final LegacyBank bank = new LegacyBank();
    public String pay(int rupees) { return bank.makePaymentInPaise(rupees * 100L); }
}
```

Proxy

A proxy has the same interface as the real object and controls access: lazy loading, access control, caching or timing. The JDK dynamic proxy creates the stand-in at run time for interfaces; Spring AOP uses it (or CGLIB for classes) to implement `@Transactional`. A consequence worth knowing: a call from one method to another inside the same object bypasses the proxy, so the advice does not run.

Proxy: a timing proxy with java.lang.reflect.Proxy JAVA

```java
interface Repo { String find(int id); }
static class RealRepo implements Repo {
    public String find(int id) { return "row-" + id; }
}
static Repo timed(Repo target) {          // JDK dynamic proxy (Spring AOP, interfaces)
    InvocationHandler h = (proxy, method, a) -> {
        long t0 = System.nanoTime();
        try { return method.invoke(target, a); }
        finally { System.out.println("[proxy] " + method.getName() + " took "
            + (System.nanoTime() - t0) / 1000 + " us (typical)"); }
    };
    return (Repo) Proxy.newProxyInstance(Repo.class.getClassLoader(),
        new Class<?>[] {Repo.class}, h);
}
```

Facade

A facade offers a narrow entry point over several collaborating subsystems. `OrderFacade.placeOrder` hides the order of reservation, payment and shipping. It does not prevent direct access to the subsystems; it only provides the convenient path.

Facade: one call over three subsystems JAVA

```
static class Inventory {
    boolean reserve(String sku) { System.out.println("reserve " + sku); return true; }
}
static class Payment {
    boolean charge(int amt) { System.out.println("charge " + amt); return true; }
}
static class Shipping {
    void schedule(String sku) { System.out.println("ship " + sku); }
}

static class OrderFacade {                      // one simple entry point over three subsystems
    private final Inventory inv = new Inventory();
    private final Payment pay = new Payment();
    private final Shipping ship = new Shipping();
    boolean placeOrder(String sku, int amount) {
        if (!inv.reserve(sku) || !pay.charge(amount)) return false;
        ship.schedule(sku);
        return true;
    }
}
```

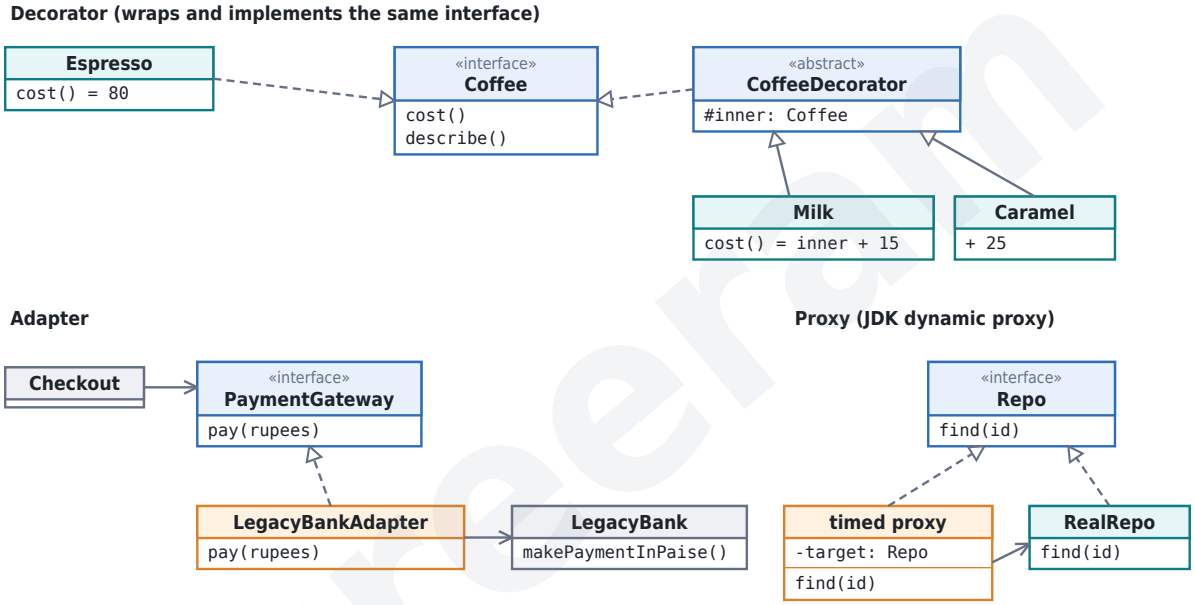


Figure 3.6. Decorator and Proxy both wrap an interface; the decorator adds behaviour to the call, the proxy controls whether and how the call reaches the real object. The adapter changes the interface itself.

3.2.5 Behavioural patterns, second group

Command

A command object bundles an action and the data needed to run it. Because it is an object it can be stored, queued, logged and reversed. `History` keeps executed commands on a stack, so `undo()` pops and reverses the last one. In the JDK, `Runnable` and `Callable` handed to an executor are commands.

Command: execute and undo JAVA

```
interface Command { void execute(); void undo(); }
static class TextEditor {
    final StringBuilder text = new StringBuilder();
}
static class AppendCommand implements Command {
    private final TextEditor ed; private final String s;
    AppendCommand(TextEditor ed, String s) { this.ed = ed; this.s = s; }
    public void execute() { ed.text.append(s); }
    public void undo() { ed.text.setLength(ed.text.length() - s.length()); }
}
static class History {
```

Command: execute and undo (continued) JAVA

```
    private final Deque<Command> done = new ArrayDeque<>();
    void run(Command c) { c.execute(); done.push(c); }
    void undo() { if (!done.isEmpty()) done.pop().undo(); }
}
```

State

The state pattern replaces a conditional on a status field with polymorphism: each state decides which event leads where. The door example uses an enum with an abstract method; larger machines (elevator, vending machine, order lifecycle) use one class per state so that each can hold its own logic. Unlisted transitions stay in the same state, or throw, and that policy should be explicit.

State: transitions owned by the state JAVA

```
enum DoorState {
    CLOSED { DoorState next(String ev) { return ev.equals("open") ? OPEN : this; } },
    OPEN   { DoorState next(String ev) { return ev.equals("close") ? CLOSED : this; } };
    abstract DoorState next(String event);
}
```

Chain of Responsibility

A request passes through a chain of handlers, each of which handles it or forwards it. The sender knows only the first handler, and the order of handlers can be changed by configuration. Servlet filters and the Spring Security filter chain follow this structure; in those chains every filter normally forwards, which is the variant used by the logger problem later in this part.

Chain of Responsibility: approval limits JAVA

```
interface Handler {
    Handler next(Handler h);
    String handle(int amount);
}
static abstract class Approver implements Handler {
    private Handler next;
    public Handler next(Handler h) { this.next = h; return h; }
    public String handle(int amount) {
        if (canApprove(amount)) return name() + " approved " + amount;
        if (next == null) return "rejected " + amount;
        return next.handle(amount);
    }
    abstract boolean canApprove(int amount);
    abstract String name();
}
static class Manager extends Approver {
    boolean canApprove(int a) { return a <= 10_000; }
    String name() { return "manager"; }
}
static class Director extends Approver {
    boolean canApprove(int a) { return a <= 100_000; }
    String name() { return "director"; }
}
```

Template Method

An abstract class fixes the algorithm in a `final` method and leaves steps abstract (mandatory) or overridable (hooks). The framework controls the flow; the subclass supplies the steps. `JdbcTemplate` applies the same idea through callbacks instead of inheritance, which is the usual preference today because it composes better.

Template Method: fixed skeleton, variable steps JAVA

```
static abstract class DataImport {
    final void run() {                          // the invariant skeleton
        List<String> rows = read();
        for (String r : rows) if (validate(r)) store(r);
    }
    abstract List<String> read();
    boolean validate(String row) { return !row.isBlank(); }   // hook with a default
    abstract void store(String row);
}
```

Template Method: fixed skeleton, variable steps (continued) JAVA

```java
static class CsvImport extends DataImport {
    List<String> read() { return List.of("a,1", "", "b,2"); }
    void store(String row) { System.out.println("stored " + row); }
}
```

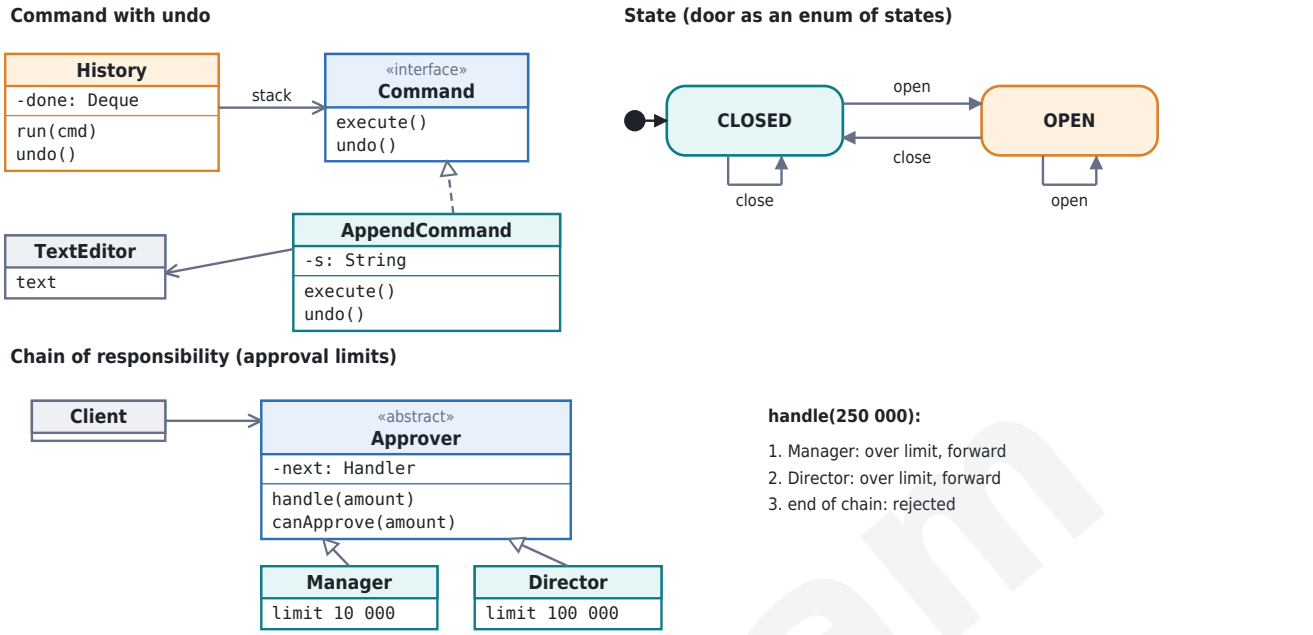


Figure 3.7. Command keeps executed commands for undo; the state enum chooses the next state per event; the chain forwards an over-limit request until a handler accepts it or the chain ends.

Run of the second program

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out Patterns2.java
$ java -cp out Patterns2
espresso + milk + caramel = 120.0
legacy paid 25000 paise
Hello, world
Hello
open -> OPEN
open -> OPEN
close -> CLOSED
manager approved 5000
director approved 50000
rejected 500000
stored a,1
stored b,2
[proxy] find took 10623 us (typical)
row-7
is proxy class: true
reserve SKU-9
charge 499
ship SKU-9
true
```

The proxy timing line varies from run to run (the first reflective call includes class-initialisation cost); every other line is deterministic.

sreeram

3.2.6 Patterns overused and when not to apply them

Patterns solve variation. Where no variation exists, a pattern adds classes without adding flexibility.

Misuse	Why it hurts
Factory with one product	One `new` call is replaced by two classes and a method.
Strategy with one implementation and no planned second	An interface that documents nothing and complicates navigation.
Hand-written Singleton for application services	Global state, hidden dependencies, tests that interfere with one another.
Deep Decorator stacks	Behaviour depends on wrapping order; stack traces become hard to read.
Observer without ownership rules	Listeners that are never removed leak memory (the lapsed listener problem); notification order becomes an accidental contract.
Facade that grows business logic	The facade becomes a god class that every caller depends on.
Builder for a three-field object	A record with a compact constructor is shorter.

Common misuses.

INTERVIEW ANGLE

Which pattern would model an elevator, and why? State for the car itself, because behaviour (accept a call, move, open the door) depends on the state and each transition belongs to a state object; Strategy for dispatch, because the policy for choosing a car is independent of the car and should be replaceable.

3.2.7 Real-world problems and solutions

Memory leak through a registered listener

Scenario. A dashboard screen registers a listener with a long-lived event publisher each time it opens and never unregisters.

Symptoms. Heap grows by one screen object graph per open; old screens still react to events.

Diagnosis. A heap dump shows many screen instances retained by the publisher's `listeners` list.

Root cause. Observer registration without a removal path; the publisher holds a strong reference.

Fix. Unsubscribe in the screen's close method, or hold listeners through weak references, or scope the subscription to a lifetime object that removes it automatically.

Prevention. Every `subscribe` has a matching `unsubscribe`; return a handle from `subscribe` and use try-with-resources.

Double-checked locking without volatile

Scenario. A lazily created configuration object is accessed by request threads.

Symptoms. Rare `NullPointerException` on a field of the supposedly initialised instance, seen only under load and never in unit tests.

Diagnosis. The `instance` field is non-volatile.

Root cause. Without a happens-before edge, another thread may see the reference before the constructor writes are visible.

Fix. Declare the field `volatile`, or use the holder idiom or an enum.

Prevention. Default to the holder idiom or container-managed singletons; avoid double-checked locking.

Self-invocation bypasses the proxy

Scenario. A service method `save()` calls another method of the same class annotated `@Transactional`.

Symptoms. No transaction is opened for the inner call; a failure leaves partial data.

Diagnosis. Logs show no proxy frame between the two methods.

Root cause. Proxy-based AOP intercepts only calls that arrive through the proxy; `this.inner()` is a direct call.

Fix. Move the inner method to another bean, or restructure so that the annotated method is the entry point.

sreeram

Prevention. Know that the pattern is the mechanism: annotations on internal calls of a proxied bean have no effect.

3.2.8 Questions to expect

- **Q. Strategy versus State: both delegate to an interface.** A. In Strategy the client chooses the algorithm and it does not change by itself. In State the object changes its own delegate as events occur, and the states know each other.
- **Q. Decorator versus Proxy?** A. Same structure, different intent: a decorator adds responsibilities, a proxy controls access to the same responsibilities.
- **Q. Factory method versus Abstract Factory?** A. A factory method creates one product type; an abstract factory creates a family of related products that must be used together.
- **Q. Why is the enum singleton preferred?** A. The JVM guarantees one instance per constant, serialisation and reflection cannot create a second one, and no locking code is needed. Its limits are that it cannot extend a class and is always eagerly initialised on first access to the enum.
- **Q. Where is Template Method used in Spring?** A. `JdbcTemplate`, `RestTemplate` and `TransactionTemplate` fix the resource handling and error translation and leave the variable part to a callback.
- **Q. How is Chain of Responsibility different from a loop over handlers?** A. It is the same behaviour with the iteration encoded in the objects. The loop form is simpler when the list is static; the chain form lets a handler stop propagation or change what is passed on.

KEY POINTS

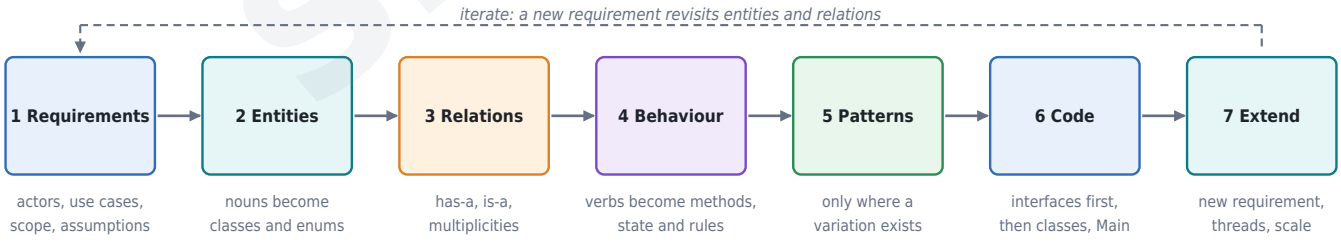
Patterns name solutions to variation: Strategy and Factory vary algorithm and creation, Observer and Command vary reaction and request, State varies behaviour by status.

- Decorator, Proxy and Adapter all wrap; ask whether the wrapper adds, controls or converts.
- Prefer enum or holder singletons, and container-managed singletons in Spring.
- Add a pattern only where a second variant is plausible; otherwise the pattern is cost.

3.3 The LLD Interview Process

A machine-coding or LLD round gives a short statement, for example "design a parking lot", and expects a compilable, extensible design in 45 to 90 minutes. The statement is deliberately incomplete. Candidates are judged on the method as much as the result: whether requirements were clarified, whether the classes model the domain, whether extension points are placed where change is likely, and whether concurrency was considered.

3.3.1 Concept and mechanism



Typical time split in a 45-minute round: 5 + 5 + 5 + 10 + 5 + 10 + 5 minutes (assumed)

Figure 3.8. The seven steps of the method; the loop at the top reflects that each new requirement revisits entities and relations.

Step	Activity	Output
Requirements	Ask about actors, use cases, scale of the in-memory model, concurrency, persistence, and what is out of scope.	A short written list of use cases and assumptions.
Entities	Underline the nouns in the requirements; keep those that hold state or rules.	A list of classes, enums and records.
Relationships	Decide is-a, has-a and uses; fix multiplicities.	A class diagram.
Behaviour	Assign each verb to the class that owns the data it needs.	Method signatures; a sequence diagram for the main flow.

Step	Activity	Output
Patterns	Identify the variations (algorithms, states, notifications) and apply a pattern only to those.	Interfaces at the variation points.
Code	Write interfaces and value types first, then services, then a `Main` that demonstrates the use cases.	Compilable code with a demo.
Extensions	Walk through two or three likely changes and one concurrency scenario.	A list of what changes and what does not.

The seven steps and their output.

3.3.2 Notation

Class diagrams show the static structure, sequence diagrams show one flow over time, and state diagrams show lifecycles. The diagrams in this part use the following notation.

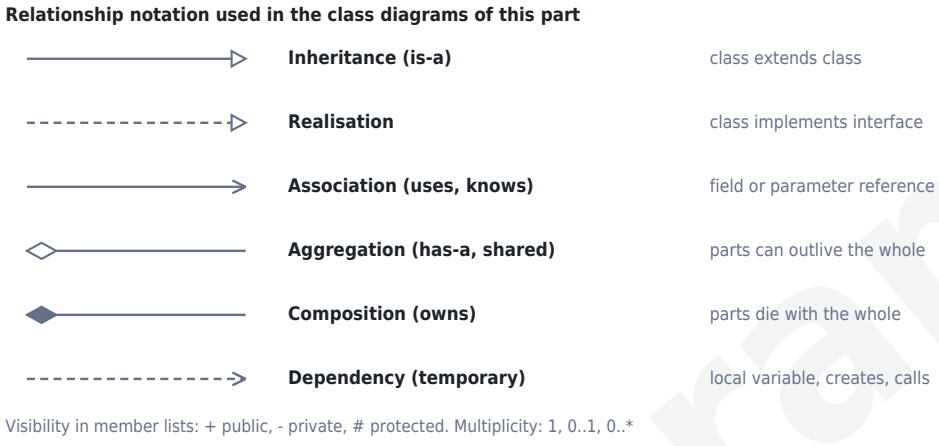


Figure 3.9. Six relationship types. In Java, realisation is `implements`, inheritance is `extends`, composition is typically a `final` field created by the owner, and dependency is a local variable or parameter.

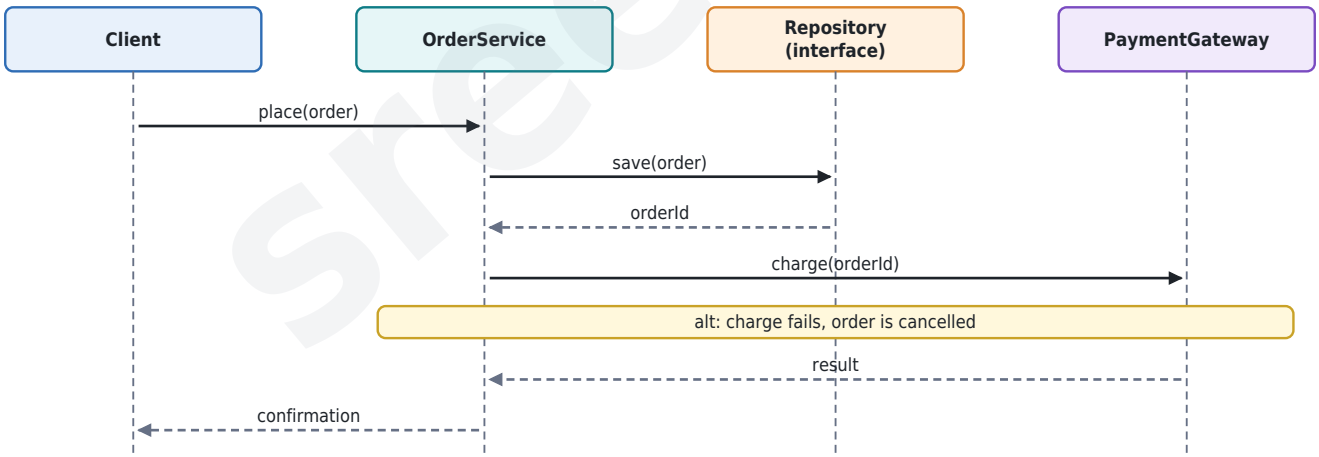


Figure 3.10. A sequence diagram: lifelines are dashed, solid arrows are calls, dashed arrows are returns, and the note spans the participants involved in an alternative path.

3.3.3 Worked example of the method

For "design a parking lot" the requirements step produces: vehicles are motorcycles, cars and trucks; spots have sizes; a vehicle takes the smallest free spot that fits; the lot issues a ticket at entry and charges at exit by hourly rate; several gates operate at the same time; payment and persistence are out of scope. The nouns are `Vehicle`, `ParkingSpot`, `Ticket`, `ParkingLot`; the verbs `park` and `exit` belong to `ParkingLot`; the two variations are spot selection and the fee rule, so those become strategies. Concurrency is the last step: two gates must never assign the same spot. The full solution appears as the first problem later in this part.

sreeram

3.3.4 Concurrency checklist

Concurrency is the step most often skipped and the one interviewers press on. The checklist below applies to every problem in this part.

1. Identify the shared mutable state: the maps, counters and status fields that two threads can touch.
2. For each, choose a protection: confinement to one thread, immutability, `synchronized` or `ReentrantLock`, an atomic variable, or a concurrent collection.
3. Locate every check-then-act sequence (check availability, then take it); each must be atomic as a whole. A `ConcurrentHashMap` does not make two separate calls atomic.
4. Decide lock granularity: one lock for the object is simple; per-resource locks scale better but require a global acquisition order to avoid deadlock.
5. Keep slow work (I/O, calls to unknown code) outside locks.
6. Inject the clock and the random source so that time-dependent behaviour can be tested.
7. State what the design assumes: single JVM, in-memory state, and what would change if the state moved to a database.

3.3.5 Hands-on: scoring your own design

Before moving to code, review the design against five questions. Each "no" is a defect to fix on paper, where it is cheaper than in code.

Question	Where it is answered
Does each requirement map to a method on exactly one class?	Behaviour step
Can a new variant (vehicle type, fee rule, notification channel) be added without editing existing classes?	Patterns step
Can every class be tested without threads, files or the network?	Code step; injected dependencies
Is each invariant enforced in one place?	Entities step
Which two operations may run concurrently, and which lock orders them?	Concurrency checklist

Review questions.

3.3.6 Real-world problems and solutions

Over-designed solution that is not finished

Scenario. A candidate spends 40 minutes on abstract factories and plug-in interfaces for a parking lot and has no working `park` method.

Symptoms. The code does not compile at the end; the demo is missing.

Diagnosis. Time was spent on extension points before the base flow existed.

Root cause. YAGNI was ignored and the sequence of the method was inverted.

Fix. Implement the simplest end-to-end flow first, then extract the interface at each variation point named in the requirements.

Prevention. Time-box: 5 minutes requirements, 10 minutes entities and relations, 25 minutes code, 5 minutes extensions (assumed budget for a 45-minute round).

Requirements assumed instead of asked

Scenario. The candidate models a library with books only; the interviewer then asks about members with borrowing limits and fines.

Symptoms. The class model must be restructured late.

Diagnosis. No explicit list of use cases was agreed at the start.

Root cause. The first step was skipped.

Fix. Write the use cases on the board and have the interviewer confirm them.

Prevention. Ask about actors, scale, concurrency and persistence in every problem.

3.3.7 Questions to expect

- **Q. How are entities distinguished from attributes?** A. A noun that has its own behaviour or lifecycle becomes a class; a noun that only describes another becomes a field or a value type (record).
- **Q. When should a method be a static utility?** A. Rarely: when it is a pure function of its arguments with no state and no variation, for example a parsing helper. Behaviour that varies belongs behind an interface.
- **Q. How much concurrency is expected in a machine-coding round?** A. At least the identification of the shared state and a correct choice for the critical check-then-act operation, stated aloud, even when the solution is single-threaded.
- **Q. What is the difference between a class diagram and a sequence diagram?** A. A class diagram describes what exists and how the types relate; a sequence diagram describes the order of calls in one scenario.
- **Q. Records, enums and sealed interfaces: when?** A. Records for immutable data carriers, enums for closed sets of constants with behaviour, sealed interfaces for closed variants that pattern matching can process exhaustively.

KEY POINTS

Follow the seven steps in order: requirements, entities, relationships, behaviour, patterns, code, extensions.

- Apply patterns only at variation points that the requirements name.
- Walk the concurrency checklist: shared state, protection, check-then-act, lock granularity, slow work, injected clock.
- Finish a working end-to-end flow first, then add extension points.

3.4 LLD: Parking Lot

3.4.1 Requirements and use cases

The lot has several floors; each floor has spots of three sizes (small, medium, large). Vehicles are motorcycles, cars and trucks. A vehicle needs a spot at least as large as its own size class. On entry the system assigns a spot and issues a ticket; on exit the ticket is presented and a fee is computed from the stay.

Item	Statement
Use cases	Park a vehicle, release a spot by ticket, report free spots, reject when full.
Fee rule	Hourly rate by vehicle type, rounded up, minimum one hour (assumed rates: Rs 10, 30, 60).
Allocation rule	Lowest floor first, then the tightest fitting spot (assumed).
Concurrency	Several gates call `park` simultaneously within one JVM.
Out of scope	Payment, persistence, reservations, license-plate recognition.

Use cases and assumptions.

3.4.2 Entities and relationships

`ParkingLot` owns the spots and the active tickets. `Ticket` and `Vehicle` are immutable records. Two variations are isolated behind interfaces: `SpotAllocationStrategy` and `FeeStrategy`.

sreeram

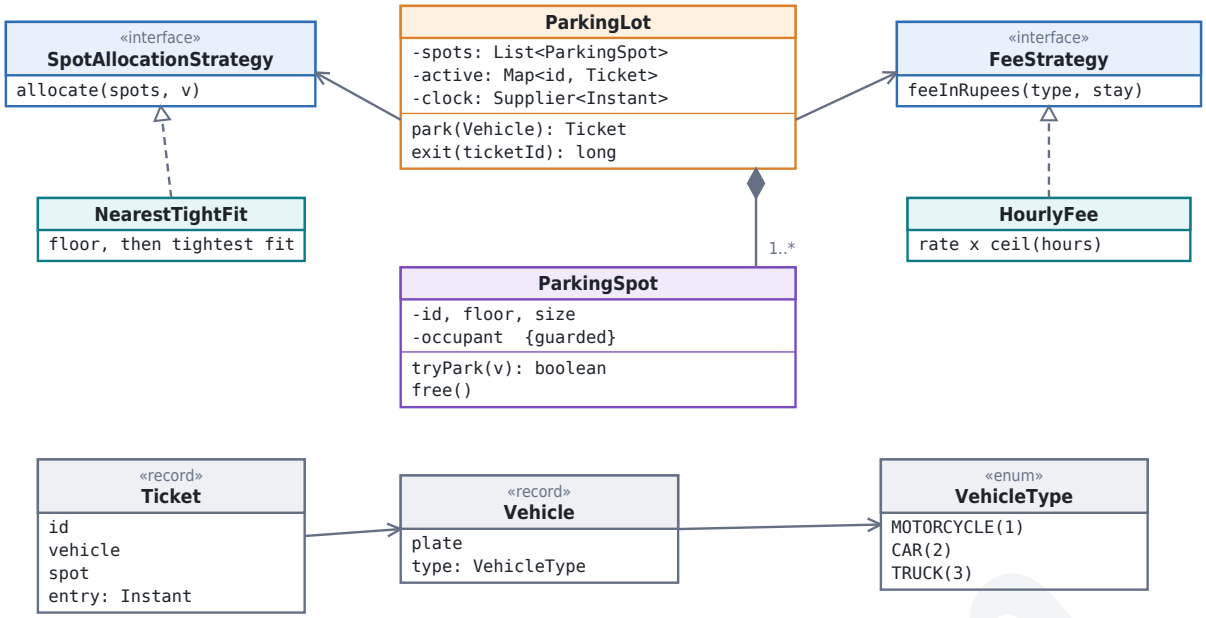


Figure 3.11. `ParkingLot` depends on two strategy interfaces and composes its spots; records and the enum carry data only.

3.4.3 Design decisions and patterns

Decision	Choice	Reason
Spot selection	Strategy	Nearest-first today, "reserve large spots for trucks" or "closest to the lift" tomorrow, with no edit to `ParkingLot` (open/closed).
Fee rule	Strategy	Rates change per site and per time of day.
Time	Injected `Supplier<Instant>`	The demo and tests control the clock; no sleeping.
Atomic claim	`ParkingSpot.tryPark` is `synchronized`	The check (is it free and large enough) and the act (set occupant) must be one step.
Double exit	`ConcurrentHashMap.remove` returns the ticket once	A second exit with the same ticket finds nothing and fails.

Design decisions.

SOLID mapping: single responsibility (the lot coordinates, the spot guards its own occupancy, the strategy decides), open/closed and dependency inversion (strategies injected through the constructor).

sreeram

3.4.4 Implementation

ParkingLotDemo.java JAVA

```java
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;

enum VehicleType {
    MOTORCYCLE(1), CAR(2), TRUCK(3);
    final int size;                                  // smallest spot size that fits
    VehicleType(int size) { this.size = size; }
}

record Vehicle(String plate, VehicleType type) {}

record Ticket(String id, Vehicle vehicle, ParkingSpot spot, Instant entry) {}

class ParkingSpot {
    private final String id;
    private final int floor;
    private final int size;                          // 1 small, 2 medium, 3 large
    private Vehicle occupant;                        // guarded by this

    ParkingSpot(String id, int floor, int size) {
        this.id = id; this.floor = floor; this.size = size;
    }
    String id() { return id; }
    int floor() { return floor; }
    int size() { return size; }

    synchronized boolean tryPark(Vehicle v) {        // atomic check-and-set
        if (occupant != null || v.type().size > size) return false;
        occupant = v;
        return true;
    }
    synchronized void free() { occupant = null; }
    synchronized boolean isFree() { return occupant == null; }
}
interface SpotAllocationStrategy {
    Optional<ParkingSpot> allocate(List<ParkingSpot> spots, Vehicle v);
}

/** Lowest floor first, then the tightest fit, so large spots stay available for trucks. */
class NearestTightFit implements SpotAllocationStrategy {
    public Optional<ParkingSpot> allocate(List<ParkingSpot> spots, Vehicle v) {
        for (ParkingSpot s : spots.stream()
                .sorted((a, b) -> a.floor() != b.floor() ? a.floor() - b.floor()
                                                         : a.size() - b.size())
                .toList()) {
            if (s.tryPark(v)) return Optional.of(s);   // tryPark is the atomic claim
        }
        return Optional.empty();
    }
}

interface FeeStrategy { long feeInRupees(VehicleType type, Duration stay); }

class HourlyFee implements FeeStrategy {
    private static final Map<VehicleType, Long> RATE =
        Map.of(VehicleType.MOTORCYCLE, 10L, VehicleType.CAR, 30L, VehicleType.TRUCK, 60L);
    public long feeInRupees(VehicleType type, Duration stay) {
        long hours = Math.max(1, (stay.toMinutes() + 59) / 60);      // round up, minimum one hour
        return hours * RATE.get(type);
    }
}
class ParkingFullException extends RuntimeException {
```

Sreeram

ParkingLotDemo.java (continued) JAVA

```
    ParkingFullException(String m) { super(m); }
}

class ParkingLot {
    private final List<ParkingSpot> spots;
    private final SpotAllocationStrategy allocation;
    private final FeeStrategy fees;
    private final Supplier<Instant> clock;                          // injected for testing
    private final Map<String, Ticket> active = new ConcurrentHashMap<>();

    ParkingLot(List<ParkingSpot> spots, SpotAllocationStrategy a, FeeStrategy f,
               Supplier<Instant> clock) {
        this.spots = List.copyOf(spots); this.allocation = a; this.fees = f; this.clock = clock;
    }

    Ticket park(Vehicle v) {
        ParkingSpot spot = allocation.allocate(spots, v)
            .orElseThrow(() -> new ParkingFullException("no spot for " + v));
        Ticket t = new Ticket(UUID.randomUUID().toString().substring(0, 8), v, spot, clock.get());
        active.put(t.id(), t);
        return t;
    }

    long exit(String ticketId) {
        Ticket t = active.remove(ticketId);                 // atomic: no double exit
        if (t == null) throw new IllegalArgumentException("unknown or used ticket " + ticketId);
        t.spot().free();
        return fees.feeInRupees(t.vehicle().type(), Duration.between(t.entry(), clock.get()));
    }

    long freeSpots() { return spots.stream().filter(ParkingSpot::isFree).count(); }
}
public class ParkingLotDemo {
    public static void main(String[] args) throws Exception {
        List<ParkingSpot> spots = new ArrayList<>();
        for (int floor = 0; floor < 2; floor++) {
            spots.add(new ParkingSpot("F" + floor + "-S1", floor, 1));
            spots.add(new ParkingSpot("F" + floor + "-M1", floor, 2));
            spots.add(new ParkingSpot("F" + floor + "-M2", floor, 2));
            spots.add(new ParkingSpot("F" + floor + "-L1", floor, 3));
        }
        Instant[] now = {Instant.parse("2026-01-01T09:00:00Z")};     // controllable clock
        ParkingLot lot = new ParkingLot(spots, new NearestTightFit(), new HourlyFee(),
                                        () -> now[0]);

        Ticket bike = lot.park(new Vehicle("AP39-1111", VehicleType.MOTORCYCLE));
        Ticket car = lot.park(new Vehicle("AP39-2222", VehicleType.CAR));
        Ticket truck = lot.park(new Vehicle("AP39-3333", VehicleType.TRUCK));
        System.out.println("bike -> " + bike.spot().id() + ", car -> " + car.spot().id()
            + ", truck -> " + truck.spot().id() + ", free = " + lot.freeSpots());

        now[0] = now[0].plus(Duration.ofMinutes(150));
        System.out.println("car fee after 150 min = Rs " + lot.exit(car.id()));
        try { lot.exit(car.id()); } catch (IllegalArgumentException e) {
            System.out.println("second exit rejected: " + e.getMessage().substring(0, 23));
        }

        // concurrency check: 40 cars compete for the remaining medium-or-larger spots
        ExecutorService pool = Executors.newFixedThreadPool(8);
        AtomicInteger ok = new AtomicInteger(), full = new AtomicInteger();
        for (int i = 0; i < 40; i++) {
            final int n = i;
            pool.submit(() -> {
                try { lot.park(new Vehicle("CAR-" + n, VehicleType.CAR)); ok.incrementAndGet(); }
                catch (ParkingFullException e) { full.incrementAndGet(); }
            });
        }
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("parked = " + ok + ", rejected = " + full
            + ", free = " + lot.freeSpots());
    }
}
```

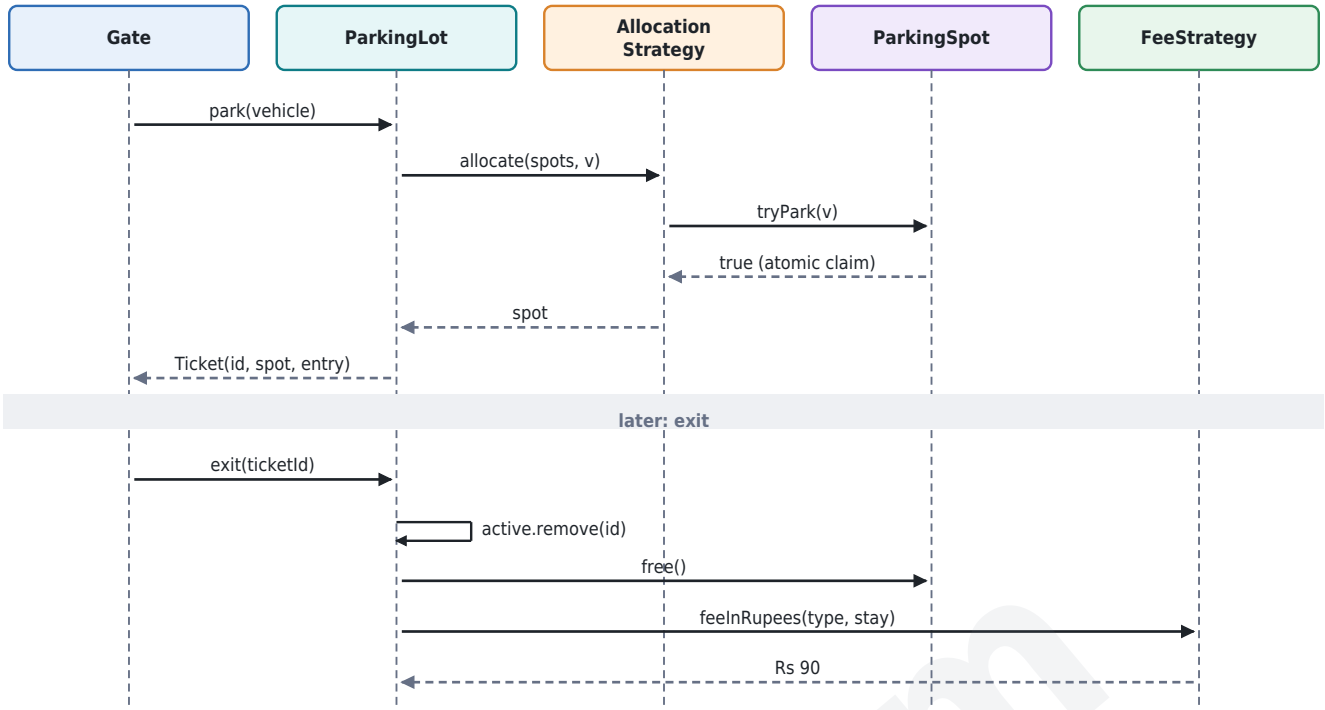


Figure 3.12. Entry asks the strategy for a spot and the spot performs the atomic claim; exit removes the ticket first, then frees the spot and computes the fee.

3.4.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out ParkingLotDemo.java
$ java -cp out ParkingLotDemo
bike -> F0-S1, car -> F0-M1, truck -> F0-L1, free = 5
car fee after 150 min = Rs 90
second exit rejected: unknown or used ticket
parked = 5, rejected = 35, free = 1
```

Reading the output: the bike took the smallest spot on floor 0, the car the tightest medium spot, the truck the large one. After 150 minutes the car pays 3 hours at Rs 30. In the concurrency check 40 threads competed for the five spots that can hold a car; exactly five succeeded, so no spot was assigned twice.

3.4.6 Concurrency and extensions

Where the race is. The allocation loop reads the spot list and then claims a spot. Between those steps another gate may take it, which is why the claim is `tryPark` returning a boolean and the loop continues to the next candidate. The list itself is immutable after construction, so iteration needs no lock.

Lock granularity. One lock per spot allows gates to claim different spots in parallel. A single lot-wide lock would be simpler and would suffice for tens of requests per second; the finer lock matters only when allocation is a hot path.

Cost. `NearestTightFit` sorts all spots on every call, O(n log n). For thousands of spots keep free spots in a `TreeSet` ordered by (floor, size) per size class, or one `ConcurrentSkipListSet`, and poll from it.

Extensions. Add a new vehicle type: add an enum constant with a size. Electric charging spots: a spot attribute and a strategy that filters on it. Multiple lots: a `ParkingService` holding a map of lots. Persistence: replace the in-memory ticket map with a repository interface; the atomic claim then becomes a database update with a condition (`UPDATE spot SET vehicle = ? WHERE id = ? AND vehicle IS NULL`), as in the databases material.

3.4.7 Real-world problems and solutions

Two vehicles assigned the same spot

Scenario. An earlier version checked `spot.isFree()` and then called `spot.assign(v)` in two statements.

Symptoms. Occasionally two tickets reference one spot; free-spot count becomes negative.

Diagnosis. A test with several threads and a small lot reproduces it; ticket spot ids contain duplicates.

Root cause. Check-then-act without atomicity.

Fix. A single `synchronized tryPark` that checks and sets under the same monitor.

Prevention. Name atomic operations as one method; run a multi-threaded test that asserts the count of successes equals the capacity.

Ticket reused at the exit

Scenario. A driver photographs a ticket and two exits process it.

Symptoms. Revenue is correct but the free-spot count increases twice and a second vehicle is assigned an occupied spot.

Diagnosis. Active tickets were read with `get` and removed later.

Root cause. Non-atomic read-then-remove.

Fix. `remove(ticketId)` as the first step; a null result means the ticket is unknown or already used.

Prevention. The operation that consumes a token must also invalidate it atomically.

3.4.8 Questions to expect

- **Q. Why a Strategy for allocation instead of an `if` on vehicle type?** A. Allocation policy varies independently of the vehicle (site rules, pricing tiers). A strategy lets it change without editing `ParkingLot`.
- **Q. How would the lot support reservations?** A. Add a `RESERVED` spot state with an expiry, similar to the seat hold in the booking problem, and let the allocation strategy skip reserved spots.
- **Q. How is the free-spot count kept O(1)?** A. Maintain an `AtomicInteger` per size class updated in `tryPark` and `free`, or keep free spots in per-size queues; counting by scanning is O(n).
- **Q. How would this run across several servers?** A. The spot becomes a row; the claim becomes a conditional update or a distributed lock with a fencing token; the lot service is stateless.
- **Q. Why is time injected?** A. A test advances the clock to verify fees without sleeping, and the production clock can be replaced by a monotonic or synchronised source.

KEY POINTS

Model the variations (allocation, fee) as strategies and the data (ticket, vehicle) as records.

- The claim of a spot must be one atomic operation; `tryPark` returns a boolean.
- Consuming a ticket and invalidating it must be one atomic step.
- Inject the clock to test time-based behaviour.

3.5 LLD: LRU and LFU Cache

3.5.1 Requirements and use cases

A fixed-capacity in-memory cache with `get(key)` and `put(key, value)`, both expected in O(1). When the cache is full a `put` of a new key evicts one entry chosen by policy: least recently used (LRU) or least frequently used (LFU). The cache is shared by several threads. Out of scope: expiry by time, loading on a miss, statistics beyond a hit ratio, and distribution (see the caching material for cache-aside and distributed caches).

3.5.2 Entities and relationships

An LRU cache needs two structures: a hash map for lookup and a doubly linked list that orders entries by recency. Sentinel head and tail nodes remove special cases at the ends. An LFU cache needs a map from key to frequency and a map from frequency to the keys with that frequency, ordered by insertion, plus the current minimum frequency.

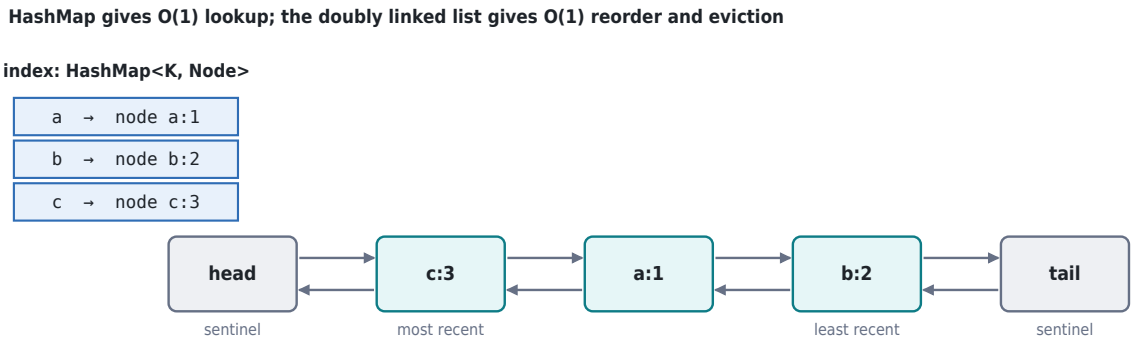


Figure 3.13. Lookups use the map; recency changes are pointer updates on the list; the eviction victim is always `tail.prev`.

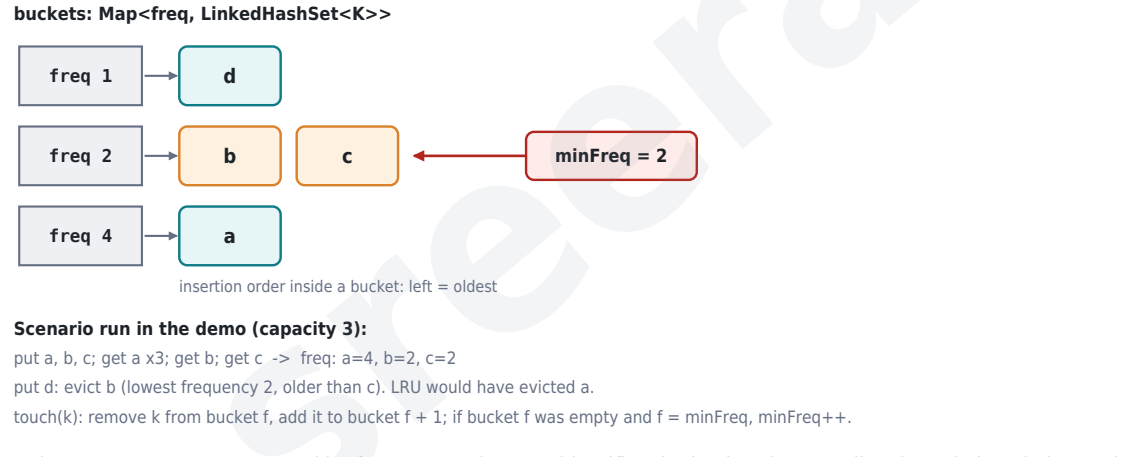


Figure 3.14. Keys are grouped by frequency; `minFreq` identifies the bucket that supplies the eviction victim, and `LinkedHashSet` keeps insertion order for tie-breaking.

3.5.3 Design decisions and patterns

Decision	Choice	Reason
Interface	`Cache<K, V>` with `get` returning `Optional`	Callers may use LRU or LFU without change (Strategy by interface); `Optional` distinguishes a miss from a null value.
LRU structure	Own `HashMap` and linked list	Shows the mechanism that interviewers ask for; `LinkedHashMap` with access order and `removeEldestEntry` is the production shortcut.
LRU locking	`ReentrantLock`	`get` mutates the recency list, so reads also write; a read-write lock would not help.
LFU locking	`synchronized` methods	Same reasoning; simple and correct.
Eviction tie in LFU	Oldest key in the lowest-frequency bucket	`LinkedHashSet` iteration order is insertion order.

Design decisions.

sreeram

3.5.4 Implementation

CacheDemo.java JAVA

```java
import java.util.HashMap;
import java.util.Iterator;
import java.util.LinkedHashSet;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.LongAdder;
import java.util.concurrent.locks.ReentrantLock;

interface Cache<K, V> {
    Optional<V> get(K key);
    void put(K key, V value);
    int size();
}
class LruCache<K, V> implements Cache<K, V> {
    private final class Node {
        final K key; V value; Node prev, next;
        Node(K key, V value) { this.key = key; this.value = value; }
    }
    private final int capacity;
    private final Map<K, Node> index = new HashMap<>();
    private final Node head = new Node(null, null);       // sentinel: most recently used side
    private final Node tail = new Node(null, null);       // sentinel: least recently used side
    private final ReentrantLock lock = new ReentrantLock();

    LruCache(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity");
        this.capacity = capacity;
        head.next = tail; tail.prev = head;
    }
    public Optional<V> get(K key) {
        lock.lock();                          // get mutates the recency list, so it needs the lock
        try {
            Node n = index.get(key);
            if (n == null) return Optional.empty();
            unlink(n); addFirst(n);
            return Optional.of(n.value);
        } finally { lock.unlock(); }
    }
    public void put(K key, V value) {
        lock.lock();
        try {
            Node n = index.get(key);
            if (n != null) { n.value = value; unlink(n); addFirst(n); return; }
            if (index.size() == capacity) {
                Node lru = tail.prev;
                unlink(lru); index.remove(lru.key);
            }
            n = new Node(key, value);
            index.put(key, n); addFirst(n);
        } finally { lock.unlock(); }
    }
    public int size() {
        lock.lock();
        try { return index.size(); } finally { lock.unlock(); }
    }
    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void addFirst(Node n) {
        n.next = head.next; n.prev = head; head.next.prev = n; head.next = n;
    }
}
class LfuCache<K, V> implements Cache<K, V> {
    private final int capacity;
    private final Map<K, V> values = new HashMap<>();
    private final Map<K, Integer> freq = new HashMap<>();
    private final Map<Integer, LinkedHashSet<K>> buckets = new HashMap<>();   // freq -> keys
    private int minFreq = 0;

    LfuCache(int capacity) { this.capacity = capacity; }
```

CacheDemo.java (continued) JAVA

```
    public synchronized Optional<V> get(K key) {
        if (!values.containsKey(key)) return Optional.empty();
        touch(key);
        return Optional.of(values.get(key));
    }
    public synchronized void put(K key, V value) {
        if (values.containsKey(key)) { values.put(key, value); touch(key); return; }
        if (values.size() == capacity) {                    // least frequent, oldest
            Iterator<K> it = buckets.get(minFreq).iterator();
            K victim = it.next();
            it.remove();
            values.remove(victim); freq.remove(victim);
        }
        values.put(key, value); freq.put(key, 1);
        buckets.computeIfAbsent(1, f -> new LinkedHashSet<>()).add(key);
        minFreq = 1;
    }
    public synchronized int size() { return values.size(); }

    private void touch(K key) {
        int f = freq.get(key);
        LinkedHashSet<K> set = buckets.get(f);
        set.remove(key);
        if (set.isEmpty() && minFreq == f) minFreq = f + 1;
        freq.put(key, f + 1);
        buckets.computeIfAbsent(f + 1, x -> new LinkedHashSet<>()).add(key);
    }
}
public class CacheDemo {
    static void scenario(String name, Cache<String, Integer> c) {
        c.put("a", 1); c.put("b", 2); c.put("c", 3);
        c.get("a"); c.get("a"); c.get("a"); c.get("b"); c.get("c");   // a: 4 uses, oldest
        c.put("d", 4);                                  // capacity 3: someone is evicted
        StringBuilder sb = new StringBuilder(name + " after put(d): ");
        for (String k : new String[] {"a", "b", "c", "d"})
            sb.append(k).append(c.get(k).isPresent() ? "=hit " : "=miss ");
        System.out.println(sb);
    }

    static void stress(String name, Cache<Integer, Integer> c) throws Exception {
        LongAdder hits = new LongAdder(), ops = new LongAdder();
        ExecutorService pool = Executors.newFixedThreadPool(4);
        for (int t = 0; t < 4; t++) {
            pool.submit(() -> {
                ThreadLocalRandom r = ThreadLocalRandom.current();
                for (int i = 0; i < 100_000; i++) {
                    int key = r.nextInt(10) < 8 ? r.nextInt(100) : r.nextInt(1000);   // 80/20 skew
                    if (c.get(key).isPresent()) hits.increment(); else c.put(key, key);
                    ops.increment();
                }
            });
        }
        pool.shutdown();
        pool.awaitTermination(30, TimeUnit.SECONDS);
        System.out.printf("%s: size=%d (capacity 200), hit ratio=%.2f%n", name, c.size(),
            hits.doubleValue() / ops.doubleValue());
    }

    public static void main(String[] args) throws Exception {
        scenario("LRU", new LruCache<>(3));
        scenario("LFU", new LfuCache<>(3));
        stress("LRU", new LruCache<>(200));
        stress("LFU", new LfuCache<>(200));
    }
}
```

Stream

3.5.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out CacheDemo.java
$ java -cp out CacheDemo
LRU after put(d): a=miss b=hit c=hit d=hit
LFU after put(d): a=hit b=miss c=hit d=hit
LRU: size=200 (capacity 200), hit ratio=0.83
LFU: size=200 (capacity 200), hit ratio=0.84
```

The first two lines use the same sequence on both caches (a is used four times, b and c twice, and c most recently) and show the policies disagreeing: LRU evicts `a`, the least recently touched, while LFU evicts `b`, the oldest among the least frequently used. The last two lines come from four threads performing 400 000 operations on a capacity-200 cache with an 80/20 skewed key distribution; they show that the size never exceeds the capacity under contention. The hit ratios are typical values and vary slightly between runs.

3.5.6 Concurrency and extensions

Why reads take the lock. A `get` moves a node to the front. Any design that records recency on reads has writes on the read path. Production caches such as Caffeine avoid a lock on reads by recording accesses in a buffer and applying them in batches; that is an optimisation to mention, not to implement in an interview.

Contention. One lock serialises all operations. Striping the cache into N independent segments by `hash(key) % N` reduces contention at the cost of an approximate global policy (each segment evicts independently).

Extensions. Time-based expiry: store an expiry instant per node and check it on `get`. Loading: a `computeIfAbsent`-style method that holds a per-key lock to prevent a stampede. Eviction listener: call a callback with the victim. Memory bound: replace `capacity` with a weight function.

> **GOTCHA**
> LFU has a known weakness: an entry that was popular long ago keeps a high count and may never be evicted, so the cache fills with stale hot entries. Real implementations age the counts (decay) or combine recency and frequency, as TinyLFU does.

3.5.7 Real-world problems and solutions

Unbounded growth from a plain HashMap used as cache

Scenario. A service caches product details in a `HashMap` with no limit.

Symptoms. Heap usage grows over days; garbage collection pauses lengthen; eventually `OutOfMemoryError`.

Diagnosis. A heap histogram shows the map and its values dominating the heap.

Root cause. No eviction policy and no maximum size.

Fix. Use a bounded cache (this chapter's LRU or a library cache) with a capacity derived from the memory budget.

Prevention. Every cache has a bound and an eviction policy; expose size and hit ratio as metrics.

Corrupted list under concurrent access

Scenario. An LRU built on `LinkedHashMap(accessOrder=true)` is shared without synchronisation, on the assumption that reads are safe.

Symptoms. `ConcurrentModificationException`, infinite loops, or entries missing.

Diagnosis. A multi-threaded test fails; a thread dump shows threads spinning inside `LinkedHashMap`.

Root cause. With access order enabled `get` is a structural modification.

Fix. Guard every operation with one lock, or use a library cache.

Prevention. Treat reads as writes for any structure that tracks access.

sheeran

3.5.8 Questions to expect

- **Q. Why a doubly linked list and not a singly linked one?** A. Removing a node from the middle in O(1) requires its predecessor; the previous pointer provides it without a search.
- **Q. Why sentinels?** A. They remove the null checks for the first and last node, so `unlink` and `addFirst` are uniform.
- **Q. What is the complexity of the LFU described?** A. O(1) for `get` and `put`: map lookups, a `LinkedHashSet` removal and insertion, and an integer update for `minFreq`.
- **Q. How is it made faster under high concurrency?** A. Stripe into segments, or buffer recency updates and apply them asynchronously, accepting that recency is slightly stale.
- **Q. LRU or LFU for a product catalogue?** A. A stable popular set favours LFU; a workload with shifting popularity (news) favours LRU. Measure the hit ratio on a recorded trace; there is no universal answer.
- **Q. What happens when `put` is called for an existing key?** A. The value is replaced and the entry becomes most recent (LRU) or its frequency increases (LFU); no eviction occurs.

KEY POINTS

LRU is a hash map plus a doubly linked list: O(1) lookup, O(1) reorder, eviction at `tail.prev`.

- LFU is a key-to-frequency map plus frequency buckets and `minFreq`; ties use insertion order.
- A cache that tracks access turns reads into writes, so reads need the lock.
- Bound every cache, and measure the hit ratio before choosing the policy.

3.6 LLD: Elevator System

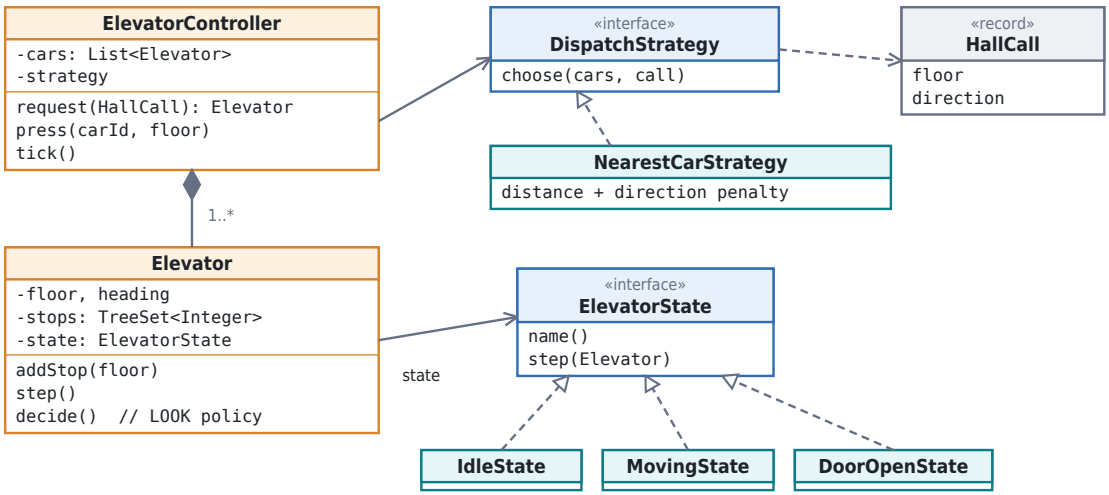
3.6.1 Requirements and use cases

A building with several floors and several elevator cars. A passenger presses a hall call (floor and direction) outside, and a destination button inside. The system assigns a car to each hall call and moves cars so that stops along the direction of travel are served first. Assumed: the simulation advances in discrete ticks; one tick moves a car one floor or runs one door cycle. Out of scope: weight limits, emergency modes, destination dispatch, and real timing.

3.6.2 Entities and relationships

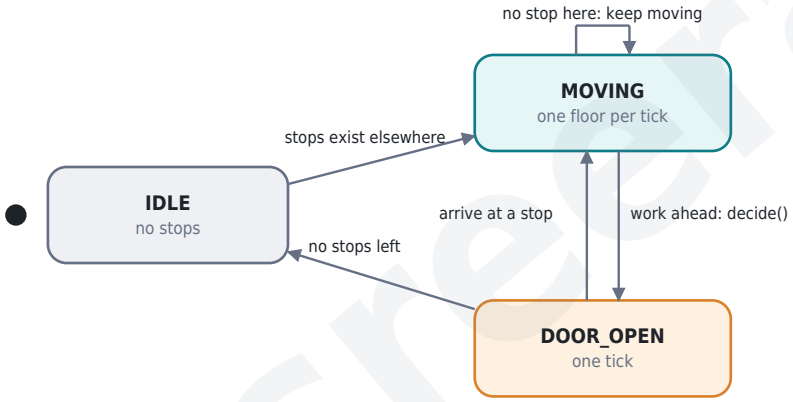
An `Elevator` stores its floor, heading and sorted set of stops. Its behaviour depends on its state: idle, moving, or door open. The `ElevatorController` receives requests and uses a `DispatchStrategy` to choose a car.

sreeram



Each state object decides the next transition in step(); Elevator only stores data.
Dispatch is a Strategy: a SCAN-aware or load-balancing policy can replace NearestCarStrategy.

Figure 3.15. The `Elevator` owns a `TreeSet` of stops and a state object; each state class implements `step`. Dispatch is a separate interface.



Heading is kept while stops exist in that direction (LOOK); otherwise decide() reverses or goes idle.

Figure 3.16. Three states. The self-transition on MOVING advances one floor per tick; DOOR_OPEN lasts one tick and then `decide()` chooses the next move.

3.6.3 Design decisions and patterns

Decision	Choice	Reason
Car lifecycle	State pattern	Each state decides its transition; no `switch` on a status field.
Car selection	Strategy (`NearestCarStrategy`)	Policies (nearest, least loaded, zoned) replace one another.
Movement policy	LOOK: continue in the current direction while stops exist ahead, then reverse	Avoids starvation of floors behind the car and wasted travel to the ends.
Stops	`TreeSet<Integer>`	`higher(floor)` and `lower(floor)` answer "anything ahead" in O(log n).
Thread model	Cars stepped by one controller tick; `Elevator` methods `synchronized`	Requests may arrive on other threads while the tick runs.

Design decisions.

sreeram

3.6.4 Implementation

ElevatorDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.TreeSet;

enum Direction { UP, DOWN, IDLE }

record HallCall(int floor, Direction direction) {}

interface ElevatorState {
    String name();
    void step(Elevator e);                    // advance by one tick
}

class Elevator {
    final int id;
    private int floor;
    private Direction heading = Direction.IDLE;
    private ElevatorState state = new IdleState();
    private final TreeSet<Integer> stops = new TreeSet<>();

    Elevator(int id, int startFloor) { this.id = id; this.floor = startFloor; }

    int floor() { return floor; }
    Direction heading() { return heading; }
    String stateName() { return state.name(); }
    boolean hasStops() { return !stops.isEmpty(); }
    boolean hasStopHere() { return stops.contains(floor); }
    boolean hasStopsAbove() { return stops.higher(floor) != null; }
    boolean hasStopsBelow() { return stops.lower(floor) != null; }

    synchronized void addStop(int f) { stops.add(f); }
    synchronized void step() { state.step(this); }

    void setState(ElevatorState s) { this.state = s; }
    void setHeading(Direction d) { this.heading = d; }
    void moveOne() { floor += (heading == Direction.UP) ? 1 : -1; }
    void arrive() { stops.remove(floor); }

    /** LOOK policy: keep going while there is work ahead, otherwise reverse or stop. */
    void decide() {
        boolean up = hasStopsAbove(), down = hasStopsBelow();
        if (heading == Direction.UP && up) setState(new MovingState());
        else if (heading == Direction.DOWN && down) setState(new MovingState());
        else if (up) { heading = Direction.UP; setState(new MovingState()); }
        else if (down) { heading = Direction.DOWN; setState(new MovingState()); }
        else { heading = Direction.IDLE; setState(new IdleState()); }
    }
}
class IdleState implements ElevatorState {
    public String name() { return "IDLE"; }
    public void step(Elevator e) {
        if (e.hasStopHere()) { e.arrive(); e.setState(new DoorOpenState()); }
        else if (e.hasStops()) e.decide();
    }
}

class MovingState implements ElevatorState {
    public String name() { return "MOVING"; }
    public void step(Elevator e) {
        e.moveOne();
        if (e.hasStopHere()) { e.arrive(); e.setState(new DoorOpenState()); }
    }
}

class DoorOpenState implements ElevatorState {
    public String name() { return "DOOR_OPEN"; }
    public void step(Elevator e) { e.decide(); }             // doors close, then choose next move
}
interface DispatchStrategy { Elevator choose(List<Elevator> cars, HallCall call); }

class NearestCarStrategy implements DispatchStrategy {
```

ElevatorDemo.java (continued) JAVA

```
    public Elevator choose(List<Elevator> cars, HallCall call) {
        Elevator best = null;
        int bestCost = Integer.MAX_VALUE;
        for (Elevator e : cars) {
            int cost = Math.abs(e.floor() - call.floor());
            boolean idle = e.heading() == Direction.IDLE;
            boolean onTheWay = (e.heading() == Direction.UP && e.floor() <= call.floor()
                                && call.direction() == Direction.UP)
                            || (e.heading() == Direction.DOWN && e.floor() >= call.floor()
                                && call.direction() == Direction.DOWN);
            if (!idle && !onTheWay) cost += 100;              // moving away: heavy penalty
            if (cost < bestCost) { bestCost = cost; best = e; }
        }
        return best;
    }
}

class ElevatorController {
    private final List<Elevator> cars = new ArrayList<>();
    private final DispatchStrategy strategy;
    ElevatorController(int count, DispatchStrategy s) {
        for (int i = 0; i < count; i++) cars.add(new Elevator(i, 0));
        this.strategy = s;
    }
    List<Elevator> cars() { return cars; }
    Elevator request(HallCall call) {
        Elevator e = strategy.choose(cars, call);
        e.addStop(call.floor());
        return e;
    }
    void press(int carId, int floor) { cars.get(carId).addStop(floor); }
    void tick() { cars.forEach(Elevator::step); }
}
public class ElevatorDemo {
    public static void main(String[] args) {
        ElevatorController ctl = new ElevatorController(2, new NearestCarStrategy());
        ctl.cars().get(1).addStop(8);                 // car 1 is sent to floor 8 first
        Map<Integer, Runnable> script = new TreeMap<>();
        script.put(1, () -> System.out.println("   call UP at 3 -> car "
            + ctl.request(new HallCall(3, Direction.UP)).id));
        script.put(2, () -> System.out.println("   call DOWN at 6 -> car "
            + ctl.request(new HallCall(6, Direction.DOWN)).id));
        script.put(5, () -> {
            ctl.press(0, 9);
            System.out.println("   passenger in car 0 presses 9");
        });
        for (int t = 0; t <= 14; t++) {
            Runnable r = script.get(t);
            if (r != null) r.run();
            ctl.tick();
            StringBuilder sb = new StringBuilder(String.format("t=%2d", t));
            for (Elevator e : ctl.cars())
                sb.append(String.format("  car%d floor %d %-9s", e.id, e.floor(), e.stateName()));
            System.out.println(sb);
        }
    }
}
```

Preram

3.6.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out ElevatorDemo.java
$ java -cp out ElevatorDemo
t= 0  car0 floor 0 IDLE       car1 floor 0 MOVING
   call UP at 3 -> car 0
t= 1  car0 floor 0 MOVING     car1 floor 1 MOVING
   call DOWN at 6 -> car 1
t= 2  car0 floor 1 MOVING     car1 floor 2 MOVING
t= 3  car0 floor 2 MOVING     car1 floor 3 MOVING
t= 4  car0 floor 3 DOOR_OPEN  car1 floor 4 MOVING
   passenger in car 0 presses 9
t= 5  car0 floor 3 MOVING     car1 floor 5 MOVING
t= 6  car0 floor 4 MOVING     car1 floor 6 DOOR_OPEN
t= 7  car0 floor 5 MOVING     car1 floor 6 MOVING
t= 8  car0 floor 6 MOVING     car1 floor 7 MOVING
t= 9  car0 floor 7 MOVING     car1 floor 8 DOOR_OPEN
t=10  car0 floor 8 MOVING     car1 floor 8 IDLE
t=11  car0 floor 9 DOOR_OPEN  car1 floor 8 IDLE
t=12  car0 floor 9 IDLE       car1 floor 8 IDLE
t=13  car0 floor 9 IDLE       car1 floor 8 IDLE
t=14  car0 floor 9 IDLE       car1 floor 8 IDLE
```

Walk through the trace. Car 1 starts with a stop at floor 8. At tick 1 the call at floor 3 goes to car 0, which is idle and closer. At tick 2 the call at floor 6 (direction DOWN) goes to car 1; both cars are heading up, so both receive a penalty and car 1 wins on distance. Car 1 stops at 6 on its way to 8. This exposes a limitation of the simple version: the stop at 6 serves a DOWN request while the car is travelling up, so the passenger is picked up and carried in the wrong direction. Handling this properly requires stops to carry a direction (separate up and down sets), listed under extensions.

3.6.6 Concurrency and extensions

Thread safety. Hall-call threads call `addStop` and the tick thread calls `step`; both are `synchronized` on the elevator, so the stop set is never modified during a decision. The controller picks a car by reading its floor and heading without the lock; a slightly stale read can choose a sub-optimal car but cannot corrupt state. A production system would serialise all requests through one event loop (a single-threaded executor), which removes most locking.

Extensions.

- Directional stops: keep `upStops` and `downStops`; a hall call joins the set of its direction, and the car serves the set that matches its heading.
- Capacity and overload: a `load` field and a state that refuses further stops when full.
- Maintenance mode: a new state that clears stops and rejects requests, added without editing the existing states.
- Dispatch by estimated time of arrival: a new strategy that sums remaining stops ahead of the call.

3.6.7 Real-world problems and solutions

Starvation of a floor

Scenario. A policy that always serves the nearest stop is used.

Symptoms. A passenger on floor 1 waits a long time while floors near the car keep generating calls.

Diagnosis. Wait-time metrics show a long tail for floors far from the busiest zone.

Root cause. Nearest-first has no fairness bound.

Fix. Use a sweep policy (LOOK or SCAN): the car serves every stop in its direction, then reverses, so any stop is served within one sweep.

Prevention. Test with a trace that contains a persistent cluster of nearby requests and assert a maximum wait.

Car assigned by stale state

Scenario. Two hall calls arrive within the same tick and both choose the same idle car, leaving another car idle.

Symptoms. One car carries all calls; average wait increases.

Diagnosis. Dispatch logs show identical car ids for simultaneous calls.

Root cause. The strategy reads `heading` while the previous assignment has not yet changed it; the new stop is added but the car is still IDLE.

Fix. Include the number of pending stops in the cost, or make assignment and cost update one atomic step in the controller.

Prevention. The cost function must see all assignments already made.

3.6.8 Questions to expect

- **Q. Why State instead of an enum with a switch?** A. Each state has its own transition logic and may acquire its own data (door timer, maintenance reason). New states are new classes; the elevator is not edited.
- **Q. SCAN versus LOOK?** A. SCAN travels to the end of the shaft before reversing; LOOK reverses at the last requested stop and wastes less travel.
- **Q. How would several cars share load?** A. A dispatch strategy that estimates the time to serve a call for each car and picks the minimum, with penalties for cars heading away.
- **Q. How is a hall call distinguished from a destination button?** A. A hall call carries a direction and is assigned by the controller; a destination is a stop added to a known car.
- **Q. Where would an emergency stop fit?** A. As a state reachable from any state, with `step` doing nothing; the controller stops dispatching to the car.

KEY POINTS

Car behaviour is the State pattern; car selection is a Strategy.

- LOOK serves stops in the heading direction first, bounding the wait of every floor.
- A `TreeSet` makes "is there a stop above or below" a logarithmic query.
- Direction-aware stops and atomic cost updates are the main known extensions.

3.7 LLD: Splitwise, Expense Sharing

3.7.1 Requirements and use cases

Users record shared expenses: one person pays, and the cost is divided among participants in one of three ways (equally, by exact amounts, by percentages). The system maintains who owes whom and can produce a simplified list of payments that settles all debts with few transfers.

Item	Statement
Money	Stored as `long` paise; no floating point.
Rounding	Equal split gives the remainder paise to the first participants; percent split gives the residue to the last participant.
Validation	Exact amounts must sum to the total; percentages must sum to 100.
Scope	In-memory, thread-safe `add`; no groups, currencies, persistence or notifications.

Assumptions.

3.7.2 Entities and relationships

`Split` is a sealed interface with three implementations. `Expense` is a record. `ExpenseManager` keeps two views: the net balance per user (paid minus owed), from which simplification is computed, and the pairwise debts, which preserve who owes whom.

Sriram

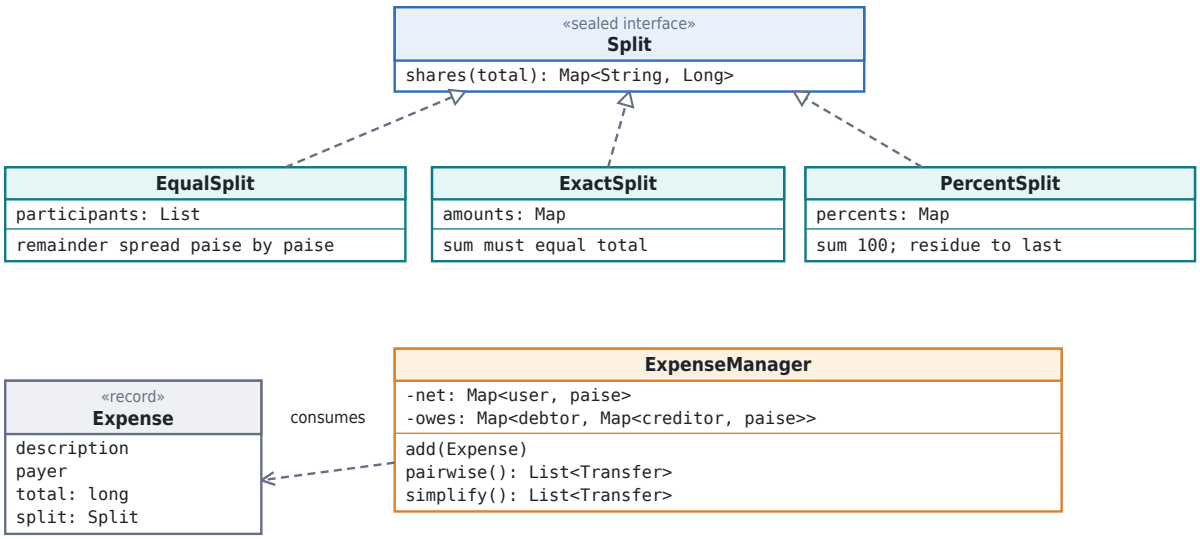


Figure 3.17. The split variants share one operation, `shares(total)`; the manager consumes expenses and maintains net balances and pairwise debts.

3.7.3 Design decisions and patterns

Decision	Choice	Reason
Split kinds	Sealed interface with records (Strategy by polymorphism)	A new split type is a new record; validation lives next to the arithmetic.
Money	`long` paise	Binary floating point cannot represent 0.1; sums drift.
Simplification	Greedy matching of the largest creditor with the largest debtor using two priority queues	At most n-1 transfers for n users with non-zero balance; simple and deterministic.
Two views	Net map and pairwise map	The pairwise map answers "how much does A owe B"; the net map drives settlement.

Design decisions.

sreeram

3.7.4 Implementation

SplitwiseDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.TreeMap;

sealed interface Split permits EqualSplit, ExactSplit, PercentSplit {
    /** Returns each participant's share in paise; the shares always add up to the total. */
    Map<String, Long> shares(long total);
}

record EqualSplit(List<String> participants) implements Split {
    public Map<String, Long> shares(long total) {
        Map<String, Long> out = new LinkedHashMap<>();
        long base = total / participants.size();
        long remainder = total % participants.size();          // spread leftover paise one by one
        for (int i = 0; i < participants.size(); i++)
            out.put(participants.get(i), base + (i < remainder ? 1 : 0));
        return out;
    }
}

record ExactSplit(Map<String, Long> amounts) implements Split {
    public Map<String, Long> shares(long total) {
        long sum = amounts.values().stream().mapToLong(Long::longValue).sum();
        if (sum != total)
            throw new IllegalArgumentException("exact amounts sum " + sum + " != " + total);
        return amounts;
    }
}

record PercentSplit(Map<String, Integer> percents) implements Split {
    public Map<String, Long> shares(long total) {
        int sum = percents.values().stream().mapToInt(Integer::intValue).sum();
        if (sum != 100) throw new IllegalArgumentException("percentages sum " + sum + " != 100");
        Map<String, Long> out = new LinkedHashMap<>();
        long assigned = 0;
        String last = null;
        for (var e : percents.entrySet()) {
            long s = total * e.getValue() / 100;
            out.put(e.getKey(), s); assigned += s; last = e.getKey();
        }
        out.merge(last, total - assigned, Long::sum);          // residue to the last person
        return out;
    }
}
record Expense(String description, String payer, long total, Split split) {}
record Transfer(String from, String to, long amount) {}

class ExpenseManager {
    private final Map<String, Long> net = new TreeMap<>();                  // paid minus owed
    private final Map<String, Map<String, Long>> owes = new TreeMap<>();    // debtor, creditor

    synchronized void add(Expense e) {
        Map<String, Long> shares = e.split().shares(e.total());
        net.merge(e.payer(), e.total(), Long::sum);
        for (var s : shares.entrySet()) {
            net.merge(s.getKey(), -s.getValue(), Long::sum);
            if (!s.getKey().equals(e.payer()))
                owes.computeIfAbsent(s.getKey(), k -> new TreeMap<>())
                    .merge(e.payer(), s.getValue(), Long::sum);
        }
    }

    synchronized List<Transfer> pairwise() {
        List<Transfer> out = new ArrayList<>();
        owes.forEach((d, m) -> m.forEach((c, a) -> out.add(new Transfer(d, c, a))));
        return out;
    }

    /** Greedy minimum cash flow: the largest debtor pays the largest creditor. */
```

sreram

SplitwiseDemo.java (continued) JAVA

```
    synchronized List<Transfer> simplify() {
        PriorityQueue<Map.Entry<String, Long>> creditors = new PriorityQueue<>(
            Map.Entry.<String, Long>comparingByValue().reversed());
        PriorityQueue<Map.Entry<String, Long>> debtors = new PriorityQueue<>(
            Map.Entry.<String, Long>comparingByValue().reversed());
        net.forEach((u, v) -> {
            if (v > 0) creditors.add(new java.util.AbstractMap.SimpleEntry<>(u, v));
            if (v < 0) debtors.add(new java.util.AbstractMap.SimpleEntry<>(u, -v));
        });
        List<Transfer> out = new ArrayList<>();
        while (!creditors.isEmpty() && !debtors.isEmpty()) {
            var c = creditors.poll();
            var d = debtors.poll();
            long pay = Math.min(c.getValue(), d.getValue());
            out.add(new Transfer(d.getKey(), c.getKey(), pay));
            if (c.getValue() > pay) { c.setValue(c.getValue() - pay); creditors.add(c); }
            if (d.getValue() > pay) { d.setValue(d.getValue() - pay); debtors.add(d); }
        }
        return out;
    }
    synchronized Map<String, Long> net() { return new TreeMap<>(net); }
}
public class SplitwiseDemo {
    static String rs(long paise) { return String.format("Rs %d.%02d", paise / 100, paise % 100); }

    public static void main(String[] args) {
        ExpenseManager m = new ExpenseManager();
        List<String> all = List.of("Asha", "Bala", "Chitra", "Dev");
        m.add(new Expense("Dinner", "Asha", 100_000, new EqualSplit(all)));
        m.add(new Expense("Cab", "Bala", 40_001,
            new EqualSplit(List.of("Asha", "Bala", "Chitra"))));
        m.add(new Expense("Rent share", "Chitra", 90_000,
            new PercentSplit(new LinkedHashMap<>(Map.of("Chitra", 40, "Dev", 60)))));
        m.add(new Expense("Tickets", "Dev", 30_000,
            new ExactSplit(Map.of("Asha", 5_000L, "Bala", 10_000L, "Dev", 15_000L))));

        System.out.println("pairwise debts (debtor -> creditor):");
        for (Transfer t : m.pairwise())
            System.out.println("  " + t.from() + " -> " + t.to() + "  " + rs(t.amount()));
        System.out.println("net balances: " + m.net());
        System.out.println("simplified settlement:");
        for (Transfer t : m.simplify())
            System.out.println("  " + t.from() + " pays " + t.to() + "  " + rs(t.amount()));
        try {
            m.add(new Expense("Bad", "Asha", 1000,
                new PercentSplit(Map.of("Asha", 50, "Bala", 40))));
        } catch (IllegalArgumentException e) {
            System.out.println("rejected: " + e.getMessage());
        }
    }
}
```

3.7.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out SplitwiseDemo.java
$ java -cp out SplitwiseDemo
pairwise debts (debtor -> creditor):
  Asha -> Bala  Rs 133.34
  Asha -> Dev  Rs 50.00
  Bala -> Asha  Rs 250.00
  Bala -> Dev  Rs 100.00
  Chitra -> Asha  Rs 250.00
  Chitra -> Bala  Rs 133.33
  Dev -> Asha  Rs 250.00
  Dev -> Chitra  Rs 540.00
net balances: {Asha=56666, Bala=-8333, Chitra=15667, Dev=-64000}
simplified settlement:
  Dev pays Asha  Rs 566.66
  Bala pays Chitra  Rs 83.33
  Dev pays Chitra  Rs 73.34
rejected: percentages sum 90 != 100
```

Sriram

The taxi expense of Rs 400.01 split three ways shows the remainder rule: shares are 133.34, 133.34 and 133.33. The net balances add to zero, which is the invariant of the whole system. Eight pairwise debts reduce to three transfers.

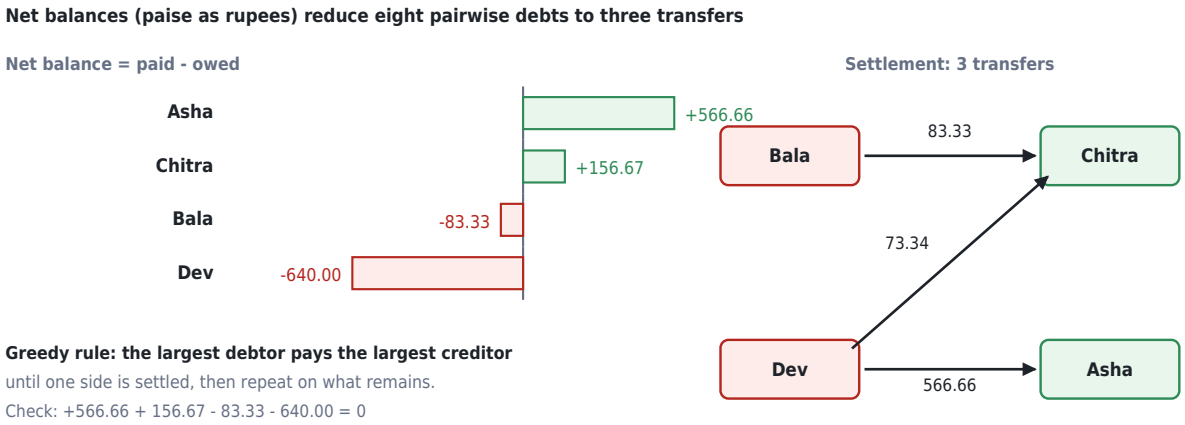


Figure 3.18. The left panel shows net balances; the greedy rule matches the largest debtor (Dev) with the largest creditor (Asha), then continues with the remainder.

3.7.6 Concurrency and extensions

Thread safety. All mutating and reading methods of `ExpenseManager` are `synchronized` on the manager. An expense touches several balances, so the update must be atomic as a whole; per-user locks would need an ordering to avoid deadlock for no real gain at this scale.

Greedy is not optimal in general. Finding the minimum number of transfers is NP-hard (it is equivalent to partitioning the balances into the maximum number of zero-sum subsets). The greedy method yields at most n-1 transfers and is correct in the sense that it settles every balance, which is the usual expectation. State this explicitly if asked about optimality.

Extensions. Groups: a `Group` entity with its own manager. Currencies: store `Money(amount, currency)` and convert at settlement. Audit: an append-only list of expenses from which balances can be recomputed (event sourcing). Persistence: expenses in a table, balances as a derived view, as in the databases material.

3.7.7 Real-world problems and solutions

A cent that disappears

Scenario. An equal split of Rs 100.00 among three users is computed with `double` and division.

Symptoms. Balances sum to 0.01 or -0.01 instead of zero; reconciliation reports small differences that accumulate.

Diagnosis. Summing all shares and comparing with the total shows a mismatch.

Root cause. Rounding each share independently loses or creates smallest units.

Fix. Integer minor units and an explicit remainder rule so that shares always add up to the total.

Prevention. Assert that the shares of every expense sum to its total; assert that all net balances sum to zero.

Partial update on failure

Scenario. An expense with an invalid percentage split throws after the payer's balance was already credited.

Symptoms. Balances no longer sum to zero.

Diagnosis. The invariant test fails after a rejected expense.

Root cause. State was changed before validation completed.

Fix. Compute all shares first (validation happens in `shares`), then apply updates; the code does this.

Prevention. Validate, then mutate; make the mutation step unable to fail.

sreeram

3.7.8 Questions to expect

- **Q. Why not `double` for money?** A. Binary fractions cannot represent most decimals; errors accumulate. Use integer minor units or `BigDecimal` with an explicit rounding mode.
- **Q. How does simplification work?** A. Compute net balance per user, put creditors and debtors in two max-heaps, repeatedly match the largest of each and transfer the smaller of the two amounts.
- **Q. Is the result minimal?** A. Not guaranteed; the exact minimum is NP-hard. The greedy result has at most n-1 transfers.
- **Q. How would a new split type, for example by shares, be added?** A. A new record implementing `Split` with its own validation and `shares`; no other class changes.
- **Q. How is idempotency handled if the client retries an expense?** A. Each expense carries a client-generated id; the manager ignores an id it has already applied.

KEY POINTS

Money is integer paise; every split must add up to the total, and every set of net balances must add up to zero.

- Split variants are a sealed interface, so new variants do not change the manager.
- Net balances drive settlement; the greedy matching yields at most n-1 transfers, not necessarily the minimum.
- Validate first, then mutate, under one lock.

3.8 LLD: Vending Machine

3.8.1 Requirements and use cases

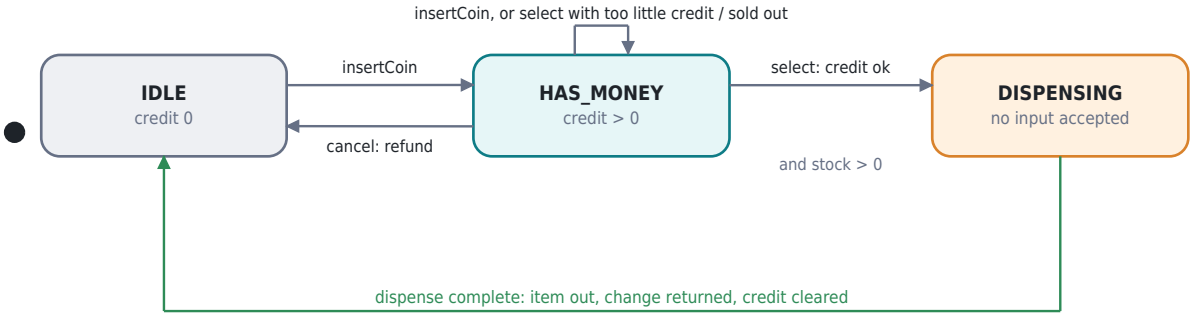
A machine sells products from coded slots. The user inserts coins, selects a product, and receives the product and change. The user may cancel before selecting and receive a refund. The machine must reject operations that make no sense in its current state, for example a selection before any money is inserted.

Item	Statement
Money	Whole-rupee coins of 10, 5, 2 and 1; change by greedy largest-first (optimal for this denomination set).
Inventory	Quantity per product code; a sold-out product is rejected without consuming credit.
Concurrency	One user at a time; the public methods are `synchronized` to keep state changes atomic if a service layer calls from several threads.
Out of scope	Card payment, limited coin reserves in the machine, maintenance mode, and hardware faults.

Assumptions.

3.8.2 Entities and relationships

`VendingMachine` holds the inventory, the current credit and the current state. The state objects implement the user operations; the machine delegates every call to the state.



IDLE and DISPENSING reject operations they do not support with IllegalStateException (default methods).

Figure 3.19. Three states. A selection with too little credit or an unavailable product is handled inside HAS_MONEY without a state change.

3.8.3 Design decisions and patterns

Decision	Choice	Reason
Behaviour by status	State pattern	The same call (`select`) means different things in IDLE and HAS_MONEY; no `if (state == ...)` chains.
Illegal operations	A default method in the interface throws `IllegalStateException` naming the operation and the state	Each state overrides only what it supports; forgotten cases fail loudly.
Sale step	`DISPENSING` state entered before the item is released	Makes the sale a distinct phase that accepts no input.
Change	Separate `ChangeMaker`	Isolated and testable; replaceable if coin reserves are added.

Design decisions.

SOLID mapping: each state has one responsibility (open/closed: a new state, for example `OutOfService`, requires no edit to the others); the interface is small (interface segregation) because defaults supply the unsupported operations.

sreeram

3.8.4 Implementation

VendingDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

record Product(String code, String name, int price) {}

class Inventory {
    private final Map<String, Product> products = new HashMap<>();
    private final Map<String, Integer> stock = new HashMap<>();

    void load(Product p, int qty) {
        products.put(p.code(), p);
        stock.merge(p.code(), qty, Integer::sum);
    }
    Product find(String code) { return products.get(code); }
    boolean available(String code) { return stock.getOrDefault(code, 0) > 0; }
    void take(String code) {
        if (!available(code)) throw new IllegalStateException("sold out " + code);
        stock.merge(code, -1, Integer::sum);
    }
}

class ChangeMaker {
    private static final int[] DENOMS = {10, 5, 2, 1};            // rupee coins, largest first
    static List<Integer> change(int amount) {
        List<Integer> out = new ArrayList<>();
        for (int d : DENOMS) while (amount >= d) { out.add(d); amount -= d; }
        return out;
    }
}
interface VendingState {
    default void insertCoin(VendingMachine m, int coin) { deny("insertCoin"); }
    default void select(VendingMachine m, String code) { deny("select"); }
    default void cancel(VendingMachine m) { deny("cancel"); }
    String name();
    private void deny(String op) {
        throw new IllegalStateException(op + " not allowed in state " + name());
    }
}

class IdleState implements VendingState {
    public String name() { return "IDLE"; }
    public void insertCoin(VendingMachine m, int coin) {
        m.addCredit(coin);
        m.setState(new HasMoneyState());
    }
}

class HasMoneyState implements VendingState {
    public String name() { return "HAS_MONEY"; }
    public void insertCoin(VendingMachine m, int coin) { m.addCredit(coin); }
    public void select(VendingMachine m, String code) {
        Product p = m.inventory().find(code);
        if (p == null || !m.inventory().available(code)) {
            System.out.println("  '" + code + "' unavailable, credit kept: " + m.credit());
            return;
        }
        if (m.credit() < p.price()) {
            System.out.println("  need Rs " + (p.price() - m.credit()) + " more for " + p.name());
            return;
        }
        m.setState(new DispensingState());
        m.dispense(p);                                  // Dispensing state completes the sale
    }
    public void cancel(VendingMachine m) {
        System.out.println("  refund " + ChangeMaker.change(m.credit()));
        m.clearCredit();
        m.setState(new IdleState());
    }
}

class DispensingState implements VendingState {
```

Sriram

VendingDemo.java (continued) JAVA

```
    public String name() { return "DISPENSING"; }        // all user actions are denied by default
}
class VendingMachine {
    private final Inventory inventory = new Inventory();
    private VendingState state = new IdleState();
    private int credit;

    Inventory inventory() { return inventory; }
    int credit() { return credit; }
    void addCredit(int c) { credit += c; }
    void clearCredit() { credit = 0; }
    void setState(VendingState s) { state = s; }
    String stateName() { return state.name(); }

    synchronized void insertCoin(int coin) { state.insertCoin(this, coin); }
    synchronized void select(String code) { state.select(this, code); }
    synchronized void cancel() { state.cancel(this); }

    void dispense(Product p) {                           // called while in DISPENSING
        inventory.take(p.code());
        int change = credit - p.price();
        System.out.println("  dispensed " + p.name() + ", change " + ChangeMaker.change(change));
        clearCredit();
        setState(new IdleState());
    }
}
public class VendingDemo {
    static void step(VendingMachine m, String what, Runnable r) {
        try {
            r.run();
        } catch (IllegalStateException e) {
            System.out.println("  ! " + e.getMessage());
        }
        System.out.println(what + " -> state " + m.stateName() + ", credit " + m.credit());
    }

    public static void main(String[] args) {
        VendingMachine m = new VendingMachine();
        m.inventory().load(new Product("A1", "Water", 20), 1);
        m.inventory().load(new Product("B2", "Chips", 35), 5);

        step(m, "select before paying", () -> m.select("A1"));
        step(m, "insert 10", () -> m.insertCoin(10));
        step(m, "select A1 (needs 20)", () -> m.select("A1"));
        step(m, "insert 10 + 5 + 2 + 1", () -> {
            m.insertCoin(10); m.insertCoin(5); m.insertCoin(2); m.insertCoin(1);
        });
        step(m, "select A1", () -> m.select("A1"));
        step(m, "insert 10", () -> m.insertCoin(10));
        step(m, "select A1 again (sold out)", () -> m.select("A1"));
        step(m, "cancel", m::cancel);
    }
}
```

3.8.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out VendingDemo.java
$ java -cp out VendingDemo
  ! select not allowed in state IDLE
select before paying -> state IDLE, credit 0
insert 10 -> state HAS_MONEY, credit 10
  need Rs 10 more for Water
select A1 (needs 20) -> state HAS_MONEY, credit 10
insert 10 + 5 + 2 + 1 -> state HAS_MONEY, credit 28
  dispensed Water, change [5, 2, 1]
select A1 -> state IDLE, credit 0
insert 10 -> state HAS_MONEY, credit 10
  'A1' unavailable, credit kept: 10
select A1 again (sold out) -> state HAS_MONEY, credit 10
  refund [10]
cancel -> state IDLE, credit 0
```

The sequence covers each path: an operation rejected by the state, a selection with insufficient credit, a successful sale with change `[5, 2, 1]` for credit 28 and a price of 20, a sold-out rejection that keeps the credit, and a refund on cancel.

3.8.6 Concurrency and extensions

Thread safety. The three public methods are `synchronized`, so a transition and the credit update are atomic. Since the sale happens within `select`, no other thread can observe the machine in `DISPENSING` through the public API; the state exists for clarity and for future asynchronous hardware calls.

Extensions.

- Limited change: `ChangeMaker` consults a coin inventory and may fail; the machine then refunds in `HasMoneyState`.
- Out of service: a new state returned when a sensor fails; every operation throws or refunds.
- Different payment methods: a `PaymentMethod` strategy replaces `insertCoin`.
- Restocking: an admin interface separate from the user interface (interface segregation).

> **NOTE**
> Greedy change-making is optimal only for canonical coin systems such as 10, 5, 2, 1. For arbitrary denominations a dynamic programming solution is required, which is the classic coin change problem from the DSA material.

3.8.7 Real-world problems and solutions

Product dispensed but money not recorded

Scenario. The sale decrements inventory, then an exception occurs while computing change.

Symptoms. Stock decreases and credit remains; the next user receives the leftover credit.

Diagnosis. Inventory changed before all fallible steps completed.

Root cause. Mutations ordered before validation of the whole operation.

Fix. Compute change first, then take stock, then clear credit; failures before the take leave state unchanged.

Prevention. Order each operation: validate, compute, then mutate; test failure injection at each step.

Machine stuck in DISPENSING

Scenario. The dispense step throws and the state was already set to `DispensingState`.

Symptoms. Every later operation is rejected with "not allowed in state DISPENSING".

Diagnosis. The state name in the error shows the machine never returned to IDLE.

Root cause. The transition back is on the success path only.

Fix. Wrap the dispense in try/finally that restores a safe state and refunds on failure.

Prevention. A state that is entered must have a guaranteed exit.

3.8.8 Questions to expect

- **Q. Why State instead of boolean flags?** A. Flags such as `hasMoney` and `dispensing` multiply into invalid combinations; states make legal combinations explicit and give each its own code.
- **Q. How are illegal operations handled?** A. A default interface method throws; alternatives are to ignore the call or return a result object. Throwing is appropriate for programmer errors, a result for user mistakes.
- **Q. Where is the state transition stored?** A. In the state (it calls `m.setState`), which keeps transition knowledge local; a table-driven machine is the alternative for large machines.
- **Q. How would a coin reserve be added?** A. Give `ChangeMaker` a coin inventory, return `Optional<List<Integer>>`, and refund when it is empty.
- **Q. State versus Strategy?** A. State changes by itself as events occur; a strategy is chosen by the client and stays.

sreeram

KEY POINTS

State pattern: one class per status, with default methods rejecting unsupported operations.

- Order every operation as validate, compute, mutate; guarantee a way out of every state.
- Greedy change is correct for canonical denominations only.
- Separate the user interface from the administrator interface.

3.9 LLD: Rate Limiter

3.9.1 Requirements and use cases

A library component that decides whether a request identified by a key (user id, API key, IP address) may proceed. The caller invokes `tryAcquire(key)` and receives a boolean. Different algorithms must be replaceable. The distributed design of a rate limiter service (shared counters in a store, consistency between nodes) is covered in the high-level design material; this chapter covers the class design of the in-process component.

Item	Statement
Algorithm 1	Token bucket: capacity `C`, refill rate `r` tokens per second; allows bursts up to `C`.
Algorithm 2	Sliding window log: at most `N` requests in any window of length `W`; exact but stores timestamps.
Keys	Created on first use; removal of idle keys is an extension.
Concurrency	Many threads call `tryAcquire` for the same and for different keys.

Assumptions.

3.9.2 Entities and relationships

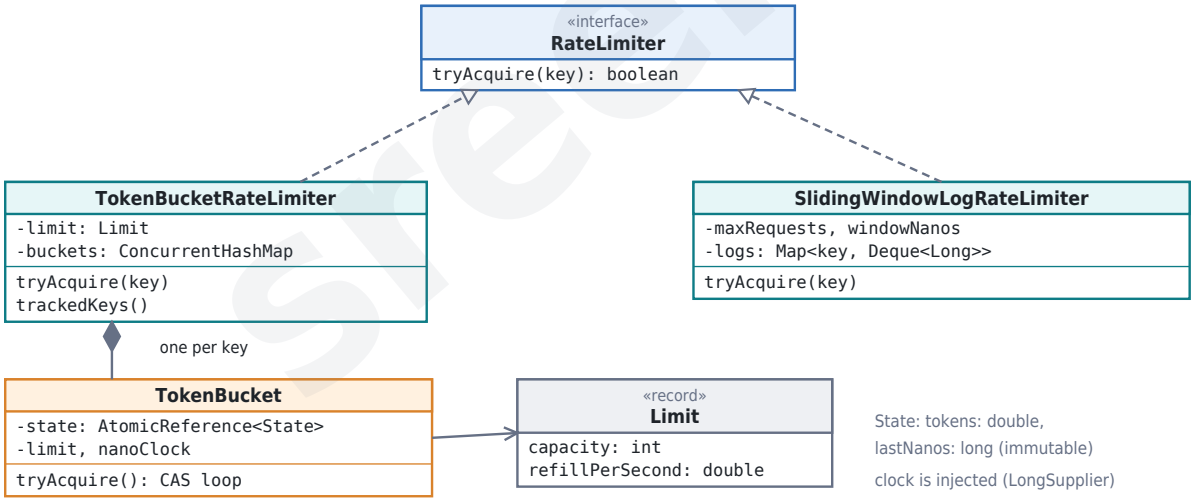


Figure 3.20. `RateLimiter` is the interface; two implementations realise it. The token bucket keeps its state in one immutable record updated by compare-and-set.

sreram

3.9.3 Design decisions and patterns

Decision	Choice	Reason
Algorithm choice	`RateLimiter` interface, two implementations	Strategy: the caller does not change when the algorithm does.
Refill	Lazy, on each call, from elapsed time	No background thread; cost is proportional to calls.
Bucket state	Immutable `State(tokens, lastNanos)` in an `AtomicReference` with a compare-and-set loop	No lock; a thread that loses the race recomputes from the new state.
Clock	Injected `LongSupplier`	Deterministic tests; production passes `System::nanoTime`, which is monotonic.
Fractional tokens	`double` tokens	A rate of 2 tokens per second yields 0.5 tokens after 250 ms; truncating would lose time.

Design decisions.

sreeram

3.9.4 Implementation

RateLimiterDemo.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;
import java.util.function.LongSupplier;

interface RateLimiter {
    boolean tryAcquire(String key);
}

record Limit(int capacity, double refillPerSecond) {
    Limit {
        if (capacity <= 0 || refillPerSecond < 0) throw new IllegalArgumentException("limit");
    }
}
final class TokenBucket {
    private record State(double tokens, long lastNanos) {}

    private final Limit limit;
    private final LongSupplier nanoClock;
    private final AtomicReference<State> state;

    TokenBucket(Limit limit, LongSupplier nanoClock) {
        this.limit = limit;
        this.nanoClock = nanoClock;
        this.state = new AtomicReference<>(new State(limit.capacity(), nanoClock.getAsLong()));
    }

    boolean tryAcquire() {
        while (true) {                                  // lock-free compare-and-set loop
            State s = state.get();
            long now = nanoClock.getAsLong();
            double refilled = Math.min(limit.capacity(),
                s.tokens() + (now - s.lastNanos()) / 1e9 * limit.refillPerSecond());
            if (refilled < 1.0) {
                // persist the fractional refill so that elapsed time is not lost
                if (state.compareAndSet(s, new State(refilled, now))) return false;
            } else if (state.compareAndSet(s, new State(refilled - 1.0, now))) {
                return true;
            }
        }
    }
}
class TokenBucketRateLimiter implements RateLimiter {
    private final Limit limit;
    private final LongSupplier clock;
    private final Map<String, TokenBucket> buckets = new ConcurrentHashMap<>();

    TokenBucketRateLimiter(Limit limit, LongSupplier clock) {
        this.limit = limit;
        this.clock = clock;
    }

    public boolean tryAcquire(String key) {
        return buckets.computeIfAbsent(key, k -> new TokenBucket(limit, clock)).tryAcquire();
    }
    int trackedKeys() { return buckets.size(); }
}

/** Alternative policy behind the same interface: exact, but O(limit) memory per key. */
class SlidingWindowLogRateLimiter implements RateLimiter {
    private final int maxRequests;
    private final long windowNanos;
    private final LongSupplier clock;
    private final Map<String, Deque<Long>> logs = new ConcurrentHashMap<>();

    SlidingWindowLogRateLimiter(int maxRequests, long windowMillis, LongSupplier clock) {
        this.maxRequests = maxRequests;
```

RateLimiterDemo.java (continued) JAVA

```
        this.windowNanos = windowMillis * 1_000_000L;
        this.clock = clock;
    }
    public boolean tryAcquire(String key) {
        Deque<Long> log = logs.computeIfAbsent(key, k -> new ArrayDeque<>());
        synchronized (log) {
            long now = clock.getAsLong();
            while (!log.isEmpty() && now - log.peekFirst() >= windowNanos) log.pollFirst();
            if (log.size() >= maxRequests) return false;
            log.addLast(now);
            return true;
        }
    }
}
public class RateLimiterDemo {
    static long[] now = {0L};                                   // fake clock in nanoseconds
    static void advanceMillis(long ms) { now[0] += ms * 1_000_000L; }

    static String burst(RateLimiter rl, String key, int n) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) sb.append(rl.tryAcquire(key) ? 'A' : 'x');
        return sb.toString();
    }

    public static void main(String[] args) throws Exception {
        RateLimiter tb = new TokenBucketRateLimiter(new Limit(5, 2.0), () -> now[0]);
        System.out.println("token bucket (capacity 5, 2 tokens/s), A = allowed, x = rejected");
        System.out.println("  t=0.0s burst of 7 for alice : " + burst(tb, "alice", 7));
        System.out.println("  t=0.0s burst of 3 for bob   : " + burst(tb, "bob", 3));
        advanceMillis(1000);
        System.out.println("  t=1.0s burst of 4 for alice : " + burst(tb, "alice", 4));
        advanceMillis(10_000);
        System.out.println("  t=11s  burst of 7 for alice : " + burst(tb, "alice", 7));

        now[0] = 0;
        RateLimiter sw = new SlidingWindowLogRateLimiter(3, 1000, () -> now[0]);
        System.out.println("sliding window log (3 per 1000 ms)");
        System.out.println("  t=0     : " + burst(sw, "carol", 4));
        advanceMillis(600);
        System.out.println("  t=0.6s  : " + burst(sw, "carol", 1));
        advanceMillis(500);
        System.out.println("  t=1.1s  : " + burst(sw, "carol", 4));

        TokenBucketRateLimiter shared = new TokenBucketRateLimiter(new Limit(100, 0.0), () -> 0L);
        AtomicInteger allowed = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int t = 0; t < 8; t++)
            pool.submit(() -> {
                for (int i = 0; i < 1000; i++)
                    if (shared.tryAcquire("hot")) allowed.incrementAndGet();
            });
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("8 threads x 1000 calls, capacity 100, no refill: allowed = " + allowed);
    }
}
```

3.9.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out RateLimiterDemo.java
$ java -cp out RateLimiterDemo
token bucket (capacity 5, 2 tokens/s), A = allowed, x = rejected
  t=0.0s burst of 7 for alice : AAAAAxx
  t=0.0s burst of 3 for bob   : AAA
  t=1.0s burst of 4 for alice : AAxx
  t=11s  burst of 7 for alice : AAAAAxx
sliding window log (3 per 1000 ms)
  t=0     : AAAx
  t=0.6s  : x
  t=1.1s  : AAAx
8 threads x 1000 calls, capacity 100, no refill: allowed = 100
```

Token level of one bucket (capacity 5, refill 2 tokens per second), values from the demo

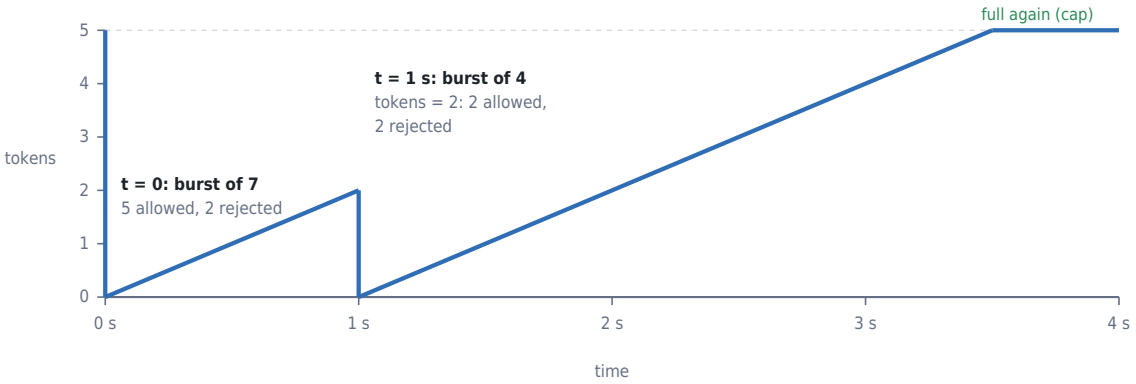


Figure 3.21. The first burst drains the bucket; at 1 s two tokens have accumulated; with no further requests the bucket refills to its capacity of five.

Reading the output: with capacity 5 and a rate of 2 per second, a burst of 7 admits 5. One second later exactly 2 tokens exist, so 2 of 4 requests pass. After ten idle seconds the bucket is full again (capped at 5, not 20). The sliding window log admits 3 requests, rejects the fourth, rejects a request at 0.6 s because three entries are still inside the window, and at 1.1 s the old entries have expired. In the last line eight threads made 8 000 calls against a capacity of 100 with no refill, and exactly 100 were admitted.

3.9.6 Concurrency and extensions

Why compare-and-set works. Each attempt reads the immutable state, computes the successor, and publishes it only if nobody changed the state in between. A losing thread loops and recomputes. The loop is lock-free: some thread always makes progress, and no thread holds a lock while descheduled.

Memory. The map of buckets grows with the number of distinct keys. For public traffic, evict idle buckets: store the last access time and sweep periodically, or use a cache with expiry after access.

Comparison of the two algorithms.

Property	Token bucket	Sliding window log
Memory per key	One state record	Up to `N` timestamps
Burst behaviour	Allows bursts up to `C`	Strict limit in every window
Accuracy	Rate limit is approximate over short intervals	Exact
Cost per call	O(1)	Amortised O(1), worst O(N) for cleanup

Token bucket against sliding window log.

Extensions. Weighted requests (`tryAcquire(key, permits)`), a blocking `acquire` that waits for tokens, per-tier limits looked up from configuration, metrics for rejected calls. Distributed use: keep the state in a shared store and apply the update atomically (a script or compare-and-set), as covered in the Redis material.

3.9.7 Real-world problems and solutions

Burst at the window boundary

Scenario. A fixed-window counter allows 100 requests per minute.

Symptoms. A client sends 100 requests at 12:00:59 and 100 at 12:01:00; the backend sees 200 requests in two seconds.

Diagnosis. Request timestamps cluster around the window boundary.

Root cause. The counter resets at the boundary; the limit is not enforced over an arbitrary window.

Fix. Use a sliding window or a token bucket whose capacity bounds the burst.

Prevention. Choose the algorithm from the burst requirement, not from implementation convenience.

Memory growth from one-time keys

Scenario. Rate limiting by IP address on a public endpoint.

Symptoms. Heap grows steadily; the bucket map holds millions of entries that were used once.

Diagnosis. Heap histogram shows `TokenBucket` instances dominating.

Root cause. Buckets are never removed.

Fix. Evict buckets idle for longer than the time to refill (a full bucket is equivalent to a new one).

Prevention. Every per-key map has an eviction rule.

Wall-clock time used for refill

Scenario. The limiter uses `System.currentTimeMillis`.

Symptoms. After an NTP adjustment the clock moves backwards; elapsed time is negative and the bucket is over- or under-filled.

Diagnosis. Negative elapsed values in a debug log.

Root cause. Wall-clock time is not monotonic.

Fix. Use `System.nanoTime` for intervals; guard with `Math.max(0, elapsed)`.

Prevention. Wall clock is for timestamps, monotonic time for durations.

3.9.8 Questions to expect

- **Q. Token bucket versus leaky bucket?** A. A token bucket permits bursts up to its capacity and then limits the average rate; a leaky bucket releases requests at a constant rate and smooths bursts into a queue or rejects them.
- **Q. Why store immutable state in an `AtomicReference`?** A. Tokens and timestamp must change together; two separate atomics could be seen inconsistent. One immutable object swapped by compare-and-set is atomic as a unit.
- **Q. How is the limiter made distributed?** A. Keep the state in a shared store and update it with an atomic operation; accept added latency or use local buckets with a share of the global limit and periodic synchronisation, trading accuracy for speed.
- **Q. What does `tryAcquire` return when the key is new?** A. A new bucket starts full, so the first requests pass; start empty if the policy requires warm-up.
- **Q. How are tests made deterministic?** A. The clock is injected as a function; tests advance a variable instead of sleeping.

KEY POINTS

The limiter is an interface; token bucket and sliding window log are interchangeable strategies.

- Refill lazily from elapsed monotonic time; keep tokens and timestamp in one immutable state swapped by compare-and-set.
- Evict idle keys; inject the clock.
- Token bucket bounds bursts by capacity; the log is exact but costs memory.

3.10 LLD: Logger Framework

3.10.1 Requirements and use cases

A logging library with levels (DEBUG, INFO, WARN, ERROR), several destinations (console, file), a configurable minimum level, formatting, and safe use from many threads. Writing to slow destinations must not block application threads unless configured to.

Item	Statement
Routing	Each handler has a minimum level; an event goes to every handler whose level it meets.
Destinations	`Appender` implementations: console, in-memory (stand-in for a file), asynchronous wrapper.
Configuration	Replaced at run time atomically.

Item	Statement
Out of scope	Rolling files, MDC, structured JSON output, and logger hierarchies by name (SLF4J and Logback provide them).

Assumptions.

3.10.2 Entities and relationships

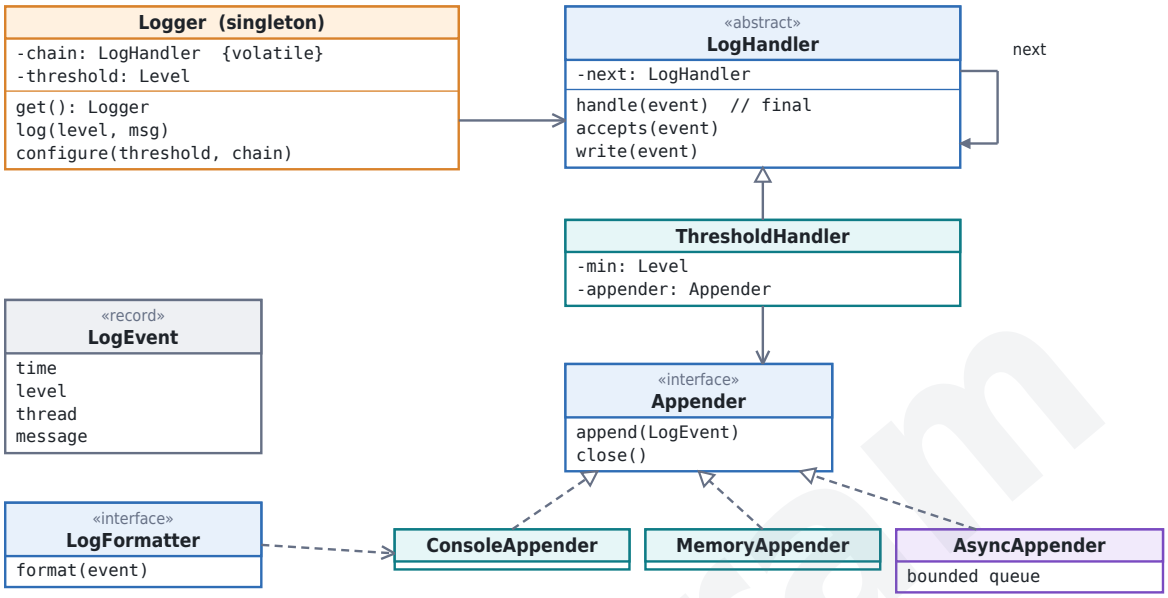


Figure 3.22. The singleton `Logger` holds the head of a handler chain; each `ThresholdHandler` owns an appender; appenders are decorated by `AsyncAppender`.

3.10.3 Design decisions and patterns

Pattern	Where	Reason
Singleton (holder idiom)	`Logger.get()`	A process-wide entry point; lazy and thread-safe without locks.
Chain of Responsibility	`LogHandler.handle`	Handlers are linked and each decides independently; here every handler forwards, so one event may reach several destinations.
Strategy	`LogFormatter`, `Appender`	Format and destination vary independently.
Decorator	`AsyncAppender` wraps any appender	Adds buffering without changing the wrapped class.
Producer-consumer	`AsyncAppender` queue and worker thread	Callers enqueue and return; the worker performs the slow write.

Patterns used.

> **NOTE**
> A pure chain of responsibility stops at the first handler that handles the request. A logger needs the other variant: every handler decides and forwards. The code expresses this in the `final handle` method (a template method), which keeps the forwarding rule in one place.

Sreeram

3.10.4 Implementation

LoggerDemo.java JAVA

```java
import java.time.Instant;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

enum Level { DEBUG, INFO, WARN, ERROR }

record LogEvent(Instant time, Level level, String thread, String message) {}

interface LogFormatter { String format(LogEvent e); }

class PlainFormatter implements LogFormatter {
    public String format(LogEvent e) {
        return String.format("%-5s [%s] %s", e.level(), e.thread(), e.message());
    }
}
interface Appender extends AutoCloseable {
    void append(LogEvent e);
    @Override default void close() {}
}

class ConsoleAppender implements Appender {
    private final LogFormatter formatter;
    ConsoleAppender(LogFormatter f) { this.formatter = f; }
    public void append(LogEvent e) { System.out.println(formatter.format(e)); }
}

class MemoryAppender implements Appender {            // stands in for a file appender in the demo
    final List<String> lines = new CopyOnWriteArrayList<>();
    private final LogFormatter formatter;
    MemoryAppender(LogFormatter f) { this.formatter = f; }
    public void append(LogEvent e) { lines.add(formatter.format(e)); }
}

/** Decouples callers from slow I/O: events go to a bounded queue drained by one worker thread. */
class AsyncAppender implements Appender {
    private static final LogEvent POISON = new LogEvent(Instant.EPOCH, Level.DEBUG, "", "");
    private final BlockingQueue<LogEvent> queue;
    private final Appender delegate;
    private final Thread worker;
    final AtomicLong dropped = new AtomicLong();

    AsyncAppender(Appender delegate, int capacity) {
        this.delegate = delegate;
        this.queue = new ArrayBlockingQueue<>(capacity);
        this.worker = new Thread(this::drain, "log-writer");
        worker.start();
    }
    public void append(LogEvent e) {
        if (!queue.offer(e)) dropped.incrementAndGet();     // drop, never block the caller
    }
    private void drain() {
        try {
            for (LogEvent e = queue.take(); e != POISON; e = queue.take()) delegate.append(e);
        } catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
    }
    @Override public void close() {
        try { queue.put(POISON); worker.join(); } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
        }
        delegate.close();
    }
}
/** Chain of responsibility: every handler decides for itself, then forwards to the next one. */
abstract class LogHandler {
    private LogHandler next;
    LogHandler linkTo(LogHandler n) { this.next = n; return n; }

    final void handle(LogEvent e) {
        if (accepts(e)) write(e);
        if (next != null) next.handle(e);
```

LoggerDemo.java (continued) JAVA

```
    }
    protected abstract boolean accepts(LogEvent e);
    protected abstract void write(LogEvent e);
}

class ThresholdHandler extends LogHandler {
    private final Level min;
    private final Appender appender;
    ThresholdHandler(Level min, Appender appender) { this.min = min; this.appender = appender; }
    protected boolean accepts(LogEvent e) { return e.level().compareTo(min) >= 0; }
    protected void write(LogEvent e) { appender.append(e); }
}
final class Logger {                                        // singleton via initialisation-on-demand
    private static class Holder { static final Logger INSTANCE = new Logger(); }
    static Logger get() { return Holder.INSTANCE; }

    private volatile LogHandler chain;                      // swapped atomically on reconfiguration
    private volatile Level threshold = Level.INFO;
    private Logger() {}

    void configure(Level threshold, LogHandler chain) {
        this.threshold = threshold;
        this.chain = chain;
    }

    void log(Level level, String msg) {
        LogHandler c = chain;
        if (c == null || level.compareTo(threshold) < 0) return;      // cheap early exit
        c.handle(new LogEvent(Instant.now(), level, Thread.currentThread().getName(), msg));
    }
    void debug(String m) { log(Level.DEBUG, m); }
    void info(String m) { log(Level.INFO, m); }
    void warn(String m) { log(Level.WARN, m); }
    void error(String m) { log(Level.ERROR, m); }
}
public class LoggerDemo {
    public static void main(String[] args) throws Exception {
        PlainFormatter fmt = new PlainFormatter();
        MemoryAppender errorFile = new MemoryAppender(fmt);
        AsyncAppender asyncErrors = new AsyncAppender(errorFile, 1024);
        LogHandler console = new ThresholdHandler(Level.INFO, new ConsoleAppender(fmt));
        console.linkTo(new ThresholdHandler(Level.ERROR, asyncErrors));

        Logger log = Logger.get();
        log.configure(Level.INFO, console);
        log.debug("cache miss for key 42");                  // below threshold: dropped early
        log.info("order 1001 accepted");
        log.warn("payment provider slow: 1800 ms");
        log.error("order 1002 failed: timeout");
        System.out.println("same instance: " + (Logger.get() == log));

        asyncErrors.close();                                  // flush the queue and stop the worker
        System.out.println("error appender received: " + errorFile.lines);
        System.out.println("dropped: " + asyncErrors.dropped.get());
    }
}
```

3.10.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out LoggerDemo.java
$ java -cp out LoggerDemo
INFO  [main] order 1001 accepted
WARN  [main] payment provider slow: 1800 ms
ERROR [main] order 1002 failed: timeout
same instance: true
error appender received: [ERROR [main] order 1002 failed: timeout]
dropped: 0
```

The DEBUG message is discarded by the logger threshold before an event object is created. The console handler (minimum INFO) prints three lines; the error handler (minimum ERROR) receives one, through the asynchronous appender, and `close()` flushes the queue before the final lines are printed.

3.10.6 Concurrency and extensions

Configuration swap. `chain` and `threshold` are `volatile`, so a thread that calls `configure` publishes a complete new chain; readers copy the reference to a local variable once (`LogHandler c = chain`) and use it for the whole call, so they never see a half-changed chain.

Backpressure policy. `AsyncAppender` uses a bounded queue and drops events when it is full, counting the drops. Alternatives: block the caller (safe for audit logs, dangerous for latency), or drop the oldest. The right choice depends on whether losing a log line or slowing a request is worse; state it as an assumption.

Shutdown. A poison-pill object tells the worker to stop after all earlier events; `close()` waits for the worker. Without it, events queued at shutdown are lost.

Extensions. Named loggers with level inheritance; message formatting with lazy evaluation (`Supplier<String>`) so that debug strings are not built when disabled; correlation id per request stored in a `ThreadLocal`; rolling files by size or date.

GOTCHA
A hand-written singleton logger cannot be replaced in tests. Expose the configuration (`configure`) so tests can install an in-memory appender, or pass a logger through constructors in code that matters.

3.10.7 Real-world problems and solutions

Request latency spikes caused by synchronous logging

Scenario. Every request logs to a network file system through a synchronised appender.

Symptoms. p99 latency rises together with log volume; thread dumps show many threads blocked on the appender lock.

Diagnosis. Stack traces point to `append` while the writer is blocked on I/O.

Root cause. Request threads perform and serialise slow I/O.

Fix. Asynchronous appender with a bounded queue and an explicit overflow policy.

Prevention. Measure logging cost; keep I/O off request threads.

Lost final log lines at shutdown

Scenario. The process exits right after logging a fatal error.

Symptoms. The error is missing from the file although the console shows it.

Diagnosis. The async queue still held the event when the JVM exited.

Root cause. No flush on shutdown.

Fix. Close the appenders in a shutdown hook and drain the queue.

Prevention. Test shutdown with events in flight.

Unbounded queue

Scenario. The async appender uses an unbounded queue and the disk is slow.

Symptoms. Heap grows until `OutOfMemoryError`.

Diagnosis. Heap dump shows millions of `LogEvent` instances in the queue.

Root cause. Producers outpace the consumer and nothing limits the backlog.

Fix. Bound the queue and choose drop or block.

Prevention. Every queue between a fast producer and a slow consumer has a bound.

sreram

3.10.8 Questions to expect

- **Q. Why is the singleton implemented with a holder class?** A. Class initialisation is lazy and thread-safe by JVM guarantee, so no locking or `volatile` is needed.
- **Q. Chain of Responsibility versus a list of handlers?** A. Equivalent in behaviour for fixed lists; the chain lets a handler stop propagation and changes the order by relinking.
- **Q. How is logging made cheap when disabled?** A. Check the level before creating the event and accept a `Supplier<String>` so the message is not built.
- **Q. Why drop instead of block?** A. Dropping bounds latency of the application at the cost of lost lines; blocking preserves lines at the cost of latency. Audit data needs blocking or a durable queue.
- **Q. What is a poison pill?** A. A sentinel placed on the queue to tell the consumer to stop after processing everything before it.

KEY POINTS

Logger: singleton entry point, chain of handlers, strategies for format and destination, a decorator for asynchronous writing.

- The chain here forwards to every handler; stopping at the first match is a different variant.
- Bound the async queue, choose an overflow policy, and flush on shutdown.
- Keep the configuration replaceable so tests can observe output.

3.11 LLD: Movie Ticket Booking, Seat Hold

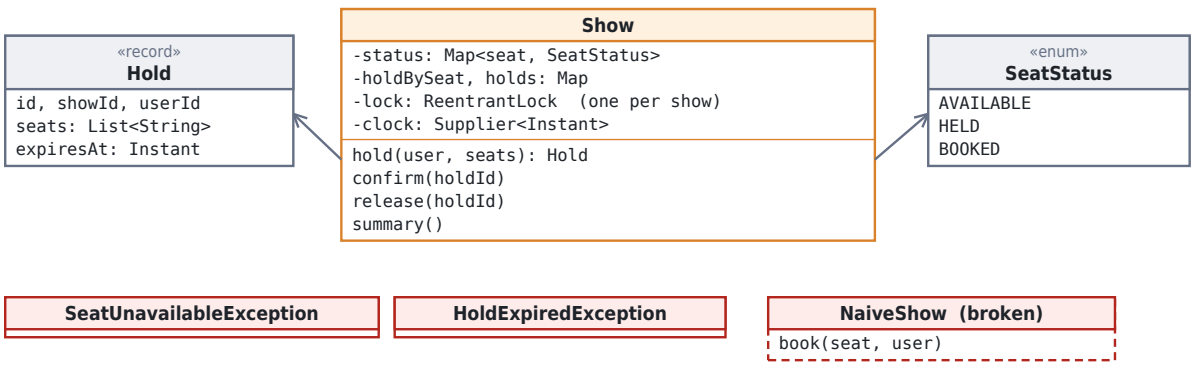
3.11.1 Requirements and use cases

A show has a set of seats. A user selects seats and holds them for five minutes while paying. If payment succeeds the seats become booked; if it fails or the time expires the seats become available again. No seat may ever be sold twice. The distributed version of this problem (payments, database transactions, queues for flash sales) appears in the high-level design of a ticket booking system; this chapter designs the in-process core.

Item	Statement
Hold duration	5 minutes (assumed).
All-or-nothing	A hold of several seats succeeds for all or none.
Expiry	Lazy: expired holds are released when the show is next accessed; a scheduler may also sweep (extension).
Concurrency	Many users contend for the same seats in one JVM.

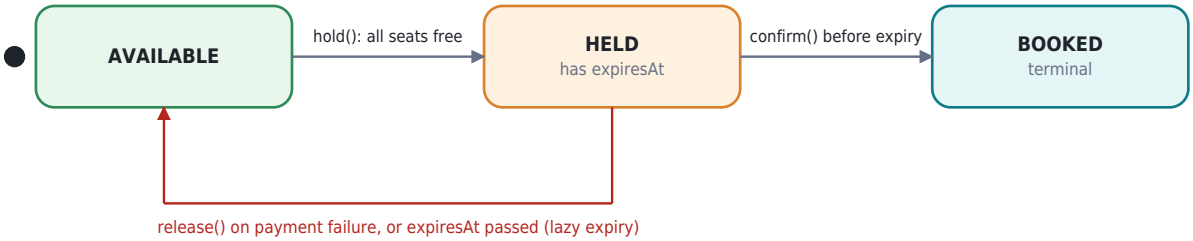
Assumptions.

3.11.2 Entities and relationships



Invariant: a seat is in exactly one of AVAILABLE, HELD (by one Hold) or BOOKED; every transition holds the show lock.
NaiveShow shows the check-then-act race that the lock removes; it is kept for the experiment only.

Figure 3.23. `Show` owns the seat map and the holds behind one lock; `NaiveShow` is included only to demonstrate the race that the lock prevents.



A second hold() on a HELD or BOOKED seat is rejected with SeatUnavailableException; nothing is changed.

Figure 3.24. Each seat is in exactly one of three states; expiry and release return a held seat to AVAILABLE.

3.11.3 Design decisions and patterns

Alternative	Behaviour	Verdict
One lock per show	Simple; hold, confirm and expiry are mutually exclusive	Chosen: correct and sufficient for one show's traffic; different shows do not contend.
One lock per seat	Higher parallelism; multi-seat holds need an ordering to avoid deadlock	Worth it only when one show has extreme traffic; acquire locks in sorted seat order.
Check then put on a concurrent map	No lock, but not atomic	Rejected: this is the bug demonstrated by `NaiveShow`.
Database row lock or conditional update	Works across servers	The production answer for several instances; see the high-level design chapter.

Alternatives considered for protecting the seats.

Other decisions: the clock is injected so that expiry is testable; `hold` validates all seats before changing any (all-or-nothing); a `Hold` is an immutable record carrying its expiry; expiry is applied lazily at the start of each operation, so no background thread is needed for correctness.

sreeram

3.11.4 Implementation

BookingDemo.java JAVA

```java
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeSet;
import java.util.UUID;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;
import java.util.function.Supplier;

enum SeatStatus { AVAILABLE, HELD, BOOKED }

record Hold(String id, String showId, String userId, List<String> seats, Instant expiresAt) {}

class SeatUnavailableException extends RuntimeException {
    SeatUnavailableException(String m) { super(m); }
}

class HoldExpiredException extends RuntimeException {
    HoldExpiredException(String m) { super(m); }
}
class Show {
    private static final Duration HOLD_TTL = Duration.ofMinutes(5);

    private final String id;
    private final Supplier<Instant> clock;
    private final ReentrantLock lock = new ReentrantLock();          // one lock per show
    private final Map<String, SeatStatus> status = new HashMap<>();
    private final Map<String, Hold> holdBySeat = new HashMap<>();
    private final Map<String, Hold> holds = new HashMap<>();

    Show(String id, List<String> seatIds, Supplier<Instant> clock) {
        this.id = id; this.clock = clock;
        seatIds.forEach(s -> status.put(s, SeatStatus.AVAILABLE));
    }

    /** All-or-nothing: either every requested seat is held, or none is. */
    Hold hold(String userId, List<String> seats) {
        lock.lock();
        try {
            expireStale();                                           // lazy expiry on every call
            for (String s : new TreeSet<>(seats))
                if (status.get(s) != SeatStatus.AVAILABLE)
                    throw new SeatUnavailableException("seat " + s + " is " + status.get(s));
            Hold h = new Hold(UUID.randomUUID().toString().substring(0, 6), id, userId,
                              List.copyOf(seats), clock.get().plus(HOLD_TTL));
            for (String s : seats) { status.put(s, SeatStatus.HELD); holdBySeat.put(s, h); }
            holds.put(h.id(), h);
            return h;
        } finally { lock.unlock(); }
    }

    void confirm(String holdId) {                                    // called after payment succeeds
        lock.lock();
        try {
            expireStale();
            Hold h = holds.remove(holdId);
            if (h == null) throw new HoldExpiredException("hold " + holdId + " expired or unknown");
            for (String s : h.seats()) { status.put(s, SeatStatus.BOOKED); holdBySeat.remove(s); }
        } finally { lock.unlock(); }
    }

    void release(String holdId) {                                    // payment failed or user left
        lock.lock();
        try {
            Hold h = holds.remove(holdId);
            if (h == null) return;
```

BookingDemo.java (continued) JAVA

```
            for (String s : h.seats()) {
                status.put(s, SeatStatus.AVAILABLE);
                holdBySeat.remove(s);
            }
        } finally { lock.unlock(); }
    }

    private void expireStale() {                                    // caller holds the lock
        Instant now = clock.get();
        List<String> dead = new ArrayList<>();
        for (Hold h : holds.values()) if (!h.expiresAt().isAfter(now)) dead.add(h.id());
        for (String hid : dead) release(hid);                // reentrant: lock already held
    }

    Map<SeatStatus, Long> summary() {
        lock.lock();
        try {
            expireStale();
            Map<SeatStatus, Long> m = new java.util.EnumMap<>(SeatStatus.class);
            status.values().forEach(v -> m.merge(v, 1L, Long::sum));
            return m;
        } finally { lock.unlock(); }
    }
}
class NaiveShow {                                           // BROKEN on purpose: check-then-act race
    private final Map<String, String> owner = new java.util.concurrent.ConcurrentHashMap<>();

    boolean book(String seat, String user) {
        if (owner.containsKey(seat)) return false;          // check
        try {
            Thread.sleep(1);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        owner.put(seat, user);                              // act: others may have passed the check
        return true;
    }
}
public class BookingDemo {
    public static void main(String[] args) throws Exception {
        Instant[] now = {Instant.parse("2026-03-01T18:00:00Z")};
        Show show = new Show("S1", List.of("A1", "A2", "A3", "A4"), () -> now[0]);

        Hold alice = show.hold("alice", List.of("A1", "A2"));
        System.out.println("alice holds " + alice.seats() + " until " + alice.expiresAt());
        try { show.hold("bob", List.of("A2", "A3")); } catch (SeatUnavailableException e) {
            System.out.println("bob rejected: " + e.getMessage() + " (A3 stays free)");
        }
        show.confirm(alice.id());
        System.out.println("after alice confirms: " + show.summary());

        Hold bob = show.hold("bob", List.of("A3"));
        now[0] = now[0].plus(Duration.ofMinutes(6));                // bob never pays
        System.out.println("after 6 minutes: " + show.summary());
        try { show.confirm(bob.id()); } catch (HoldExpiredException e) {
            System.out.println("late payment rejected: expired hold");
        }

        // race 1: naive check-then-act
        NaiveShow naive = new NaiveShow();
        AtomicInteger naiveWins = new AtomicInteger();
        race(20, i -> { if (naive.book("A1", "u" + i)) naiveWins.incrementAndGet(); });
        System.out.println("naive:  20 users race for one seat, winners = " + naiveWins);

        // race 2: locked hold
        Show safe = new Show("S2", List.of("A1"), Instant::now);
        AtomicInteger safeWins = new AtomicInteger();
        race(20, i -> {
            try { safe.hold("u" + i, List.of("A1")); safeWins.incrementAndGet(); }
            catch (SeatUnavailableException ignored) { }
        });
        System.out.println("locked: 20 users race for one seat, winners = " + safeWins);
    }

    static void race(int threads, java.util.function.IntConsumer task) throws Exception {
```

BookingDemo.java (continued) JAVA

```
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        CountDownLatch go = new CountDownLatch(1);
        for (int i = 0; i < threads; i++) {
            final int n = i;
            pool.submit(() -> {
                try { go.await(); } catch (InterruptedException e) { return; }
                task.accept(n);
            });
        }
        go.countDown();
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
    }
}
```

3.11.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out BookingDemo.java
$ java -cp out BookingDemo
alice holds [A1, A2] until 2026-03-01T18:05:00Z
bob rejected: seat A2 is HELD (A3 stays free)
after alice confirms: {AVAILABLE=2, BOOKED=2}
after 6 minutes: {AVAILABLE=2, BOOKED=2}
late payment rejected: expired hold
naive:  20 users race for one seat, winners = 19
locked: 20 users race for one seat, winners = 1
```

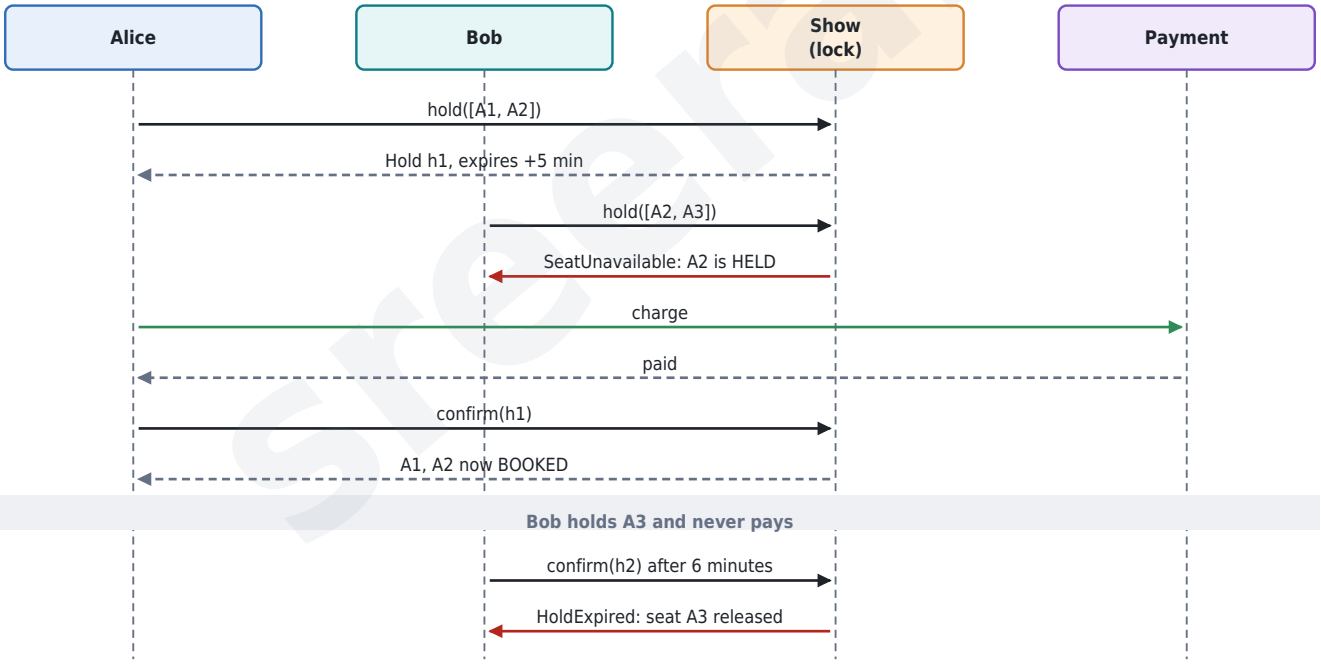


Figure 3.25. Bob's overlapping request is rejected without side effects; Alice's confirmation books her seats; Bob's late confirmation fails because the hold expired and the seat was released.

The two race lines are the key result. Twenty threads released together by a latch tried to take one seat. The naive check-then-act version admitted several winners (6 in one run, 19 in another; the number varies with scheduling, which is typical of a race), while the locked version admitted exactly one.

3.11.6 Concurrency and extensions

Why the naive version fails. `containsKey` and `put` are each atomic, but another thread can run between them; all threads pass the check before any performs the put. The injected one-millisecond sleep widens the window to make the failure reproducible; without it the window is still there and appears under load.

Lazy expiry under the lock. `expireStale` runs while the lock is held and calls `release`, which locks again; `ReentrantLock` allows this. The cost is a scan of all holds per call; keep holds in a priority queue ordered by expiry for large shows.

Extensions.

- Scheduled sweep: a `ScheduledExecutorService` calls `summary()` or a dedicated `expire()` periodically, so that seat maps shown to users are fresh even without traffic.
- Payment integration: `confirm` is called by the payment callback; it must be idempotent (a repeated callback for the same hold must not fail the booking).
- Seat categories and pricing: a `Seat` entity with a price strategy.
- Multi-instance: move the state to a database with a conditional update; the hold expiry becomes a column.

3.11.7 Real-world problems and solutions

Double booking under load

Scenario. Seat availability is checked in the controller and the booking is saved in the service.

Symptoms. At the start of a popular sale two customers hold tickets for the same seat; support receives complaints.

Diagnosis. Booking records show two rows for one seat id within milliseconds.

Root cause. Check-then-act across two steps with no atomicity.

Fix. One atomic operation: lock in memory, or a conditional update in the database (`UPDATE ... WHERE status = 'AVAILABLE'` with a check of the affected row count).

Prevention. A concurrency test that fires many parallel requests at one seat and asserts one winner; a unique constraint on (show, seat) as a safety net.

Seats stuck in HELD

Scenario. Users abandon the payment page.

Symptoms. The show appears sold out while many seats are held; revenue drops.

Diagnosis. Count of HELD seats with `expiresAt` in the past is large.

Root cause. Release depends on the user completing or cancelling; there was no expiry.

Fix. Hold with expiry, enforced lazily and by a periodic sweep.

Prevention. Every temporary reservation has a deadline.

Late payment confirmation for an expired hold

Scenario. The payment succeeds at minute 5:10 and the confirmation arrives after the hold expired and another user took the seat.

Symptoms. The customer is charged without a ticket.

Diagnosis. `HoldExpiredException` on confirm, with a successful payment record.

Root cause. Payment time is not bounded by the hold time.

Fix. Refund automatically on `HoldExpiredException`; extend the hold when the payment starts; give the payment a timeout shorter than the hold.

Prevention. Treat confirm failure after payment as a compensating action in the saga.

sreeram

3.11.8 Questions to expect

- **Q. How is overselling prevented across several servers?** A. The atomic claim moves to the shared store: a conditional update or a transaction with a unique constraint; an in-memory lock does not span processes.
- **Q. Why all-or-nothing for multi-seat holds?** A. A partial hold would leave the user with some seats and a failed request; validating all seats before changing any gives a clean rejection.
- **Q. Why is deadlock impossible here?** A. There is one lock per show and no operation holds two show locks. With per-seat locks, deadlock is avoided by acquiring in sorted order.
- **Q. How are expired holds released if nobody calls the show?** A. A scheduled task sweeps periodically; lazy expiry guarantees correctness, the sweep guarantees freshness.
- **Q. What if the confirm callback is delivered twice?** A. The second call finds no hold; return success if the seats are already booked for that hold id, so the callback is idempotent.

KEY POINTS

The core is one atomic transition per operation under a lock per show: hold validates all seats, then marks them.

- Never check and act in separate steps; demonstrate the race with a test.
- Holds have a deadline, enforced lazily and by a sweep; confirm after expiry fails and triggers a refund.
- Across servers the atomic step becomes a conditional database update.

3.12 LLD: Tic-Tac-Toe

3.12.1 Requirements and use cases

Two players alternate marking cells of an N by N board; the first to fill a complete row, column or diagonal with their mark wins; a full board without a winner is a draw. Players may be scripted (tests) or automatic. Invalid moves (outside the board or on an occupied cell) are rejected and the same player moves again.

Item	Statement
Board size	Configurable N (the demo plays 3 by 3 and 4 by 4); N in a row wins.
Win detection	O(1) per move, using counters, not a scan of the board.
Players	Interface-based; a human console player or a minimax bot can be added without changing `Game`.
Out of scope	A user interface, network play, and game history.

Assumptions.

3.12.2 Entities and relationships

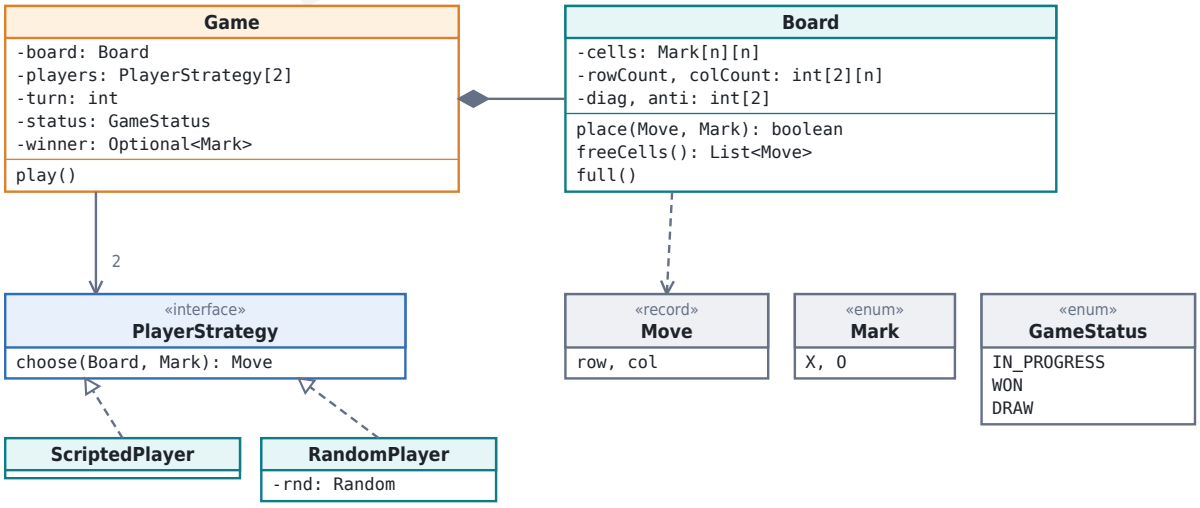


Figure 3.26. `Game` coordinates; `Board` enforces placement rules and detects wins; `PlayerStrategy` supplies moves.

3.12.3 Design decisions and patterns

Decision	Choice	Reason
Win check	Counters per player for each row, column and the two diagonals	A move increments at most four counters; the player wins when one reaches N. A scan would cost O(N) per move.
Move source	`PlayerStrategy`	Scripted, random and minimax players are interchangeable.
Data carriers	Records and enums (`Move`, `Mark`, `GameStatus`)	Immutable, comparable by value.
Errors	`InvalidMoveException` handled in `Game.play`	The game loop decides the policy (re-prompt); `Board` only reports the violation.
Responsibility split	`Board` knows the rules of the grid; `Game` knows turns and status	Single responsibility; each can be tested alone.

Design decisions.

sreeram

3.12.4 Implementation

TicTacToeDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.Random;

enum Mark { X, O }

record Move(int row, int col) {}

enum GameStatus { IN_PROGRESS, WON, DRAW }

class InvalidMoveException extends RuntimeException {
    InvalidMoveException(String m) { super(m); }
}

/** Board with O(1) win detection: per-player counters for each row, column and both diagonals. */
class Board {
    private final int n;
    private final Mark[][] cells;
    private final int[][] rowCount, colCount;              // [player][index]
    private final int[] diag, anti;
    private int moves;

    Board(int n) {
        this.n = n;
        cells = new Mark[n][n];
        rowCount = new int[2][n]; colCount = new int[2][n];
        diag = new int[2]; anti = new int[2];
    }
    int size() { return n; }
    Mark at(int r, int c) { return cells[r][c]; }
    boolean full() { return moves == n * n; }

    /** Places the mark and reports whether this move completed a line. */
    boolean place(Move m, Mark p) {
        if (m.row() < 0 || m.row() >= n || m.col() < 0 || m.col() >= n)
            throw new InvalidMoveException("outside board: " + m);
        if (cells[m.row()][m.col()] != null) throw new InvalidMoveException("occupied: " + m);
        cells[m.row()][m.col()] = p;
        moves++;
        int k = p.ordinal();
        // non-short-circuit | on purpose: every counter must be incremented
        boolean win = ++rowCount[k][m.row()] == n | ++colCount[k][m.col()] == n;
        if (m.row() == m.col()) win |= ++diag[k] == n;
        if (m.row() + m.col() == n - 1) win |= ++anti[k] == n;
        return win;
    }

    List<Move> freeCells() {
        List<Move> out = new ArrayList<>();
        for (int r = 0; r < n; r++)
            for (int c = 0; c < n; c++)
                if (cells[r][c] == null) out.add(new Move(r, c));
        return out;
    }
    String render() {
        StringBuilder sb = new StringBuilder();
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++)
                sb.append(cells[r][c] == null ? '.' : cells[r][c].name().charAt(0));
            sb.append('\n');
        }
        return sb.toString();
    }
}
interface PlayerStrategy { Move choose(Board board, Mark me); }

class ScriptedPlayer implements PlayerStrategy {
    private final List<Move> script;
    private int i;
    ScriptedPlayer(Move... moves) { this.script = List.of(moves); }
    public Move choose(Board b, Mark me) { return script.get(i++); }
}
```

sreeram

TicTacToeDemo.java (continued) JAVA

```java
class RandomPlayer implements PlayerStrategy {
    private final Random rnd;
    RandomPlayer(long seed) { rnd = new Random(seed); }
    public Move choose(Board b, Mark me) {
        List<Move> free = b.freeCells();
        return free.get(rnd.nextInt(free.size()));
    }
}
class Game {
    private final Board board;
    private final PlayerStrategy[] players;
    private int turn;                                   // 0 -> X, 1 -> O
    private GameStatus status = GameStatus.IN_PROGRESS;
    private Optional<Mark> winner = Optional.empty();

    Game(int n, PlayerStrategy x, PlayerStrategy o) {
        board = new Board(n);
        players = new PlayerStrategy[] {x, o};
    }
    Board board() { return board; }
    GameStatus status() { return status; }
    Optional<Mark> winner() { return winner; }

    void play() {
        while (status == GameStatus.IN_PROGRESS) {
            Mark me = Mark.values()[turn];
            Move m = players[turn].choose(board, me);
            try {
                if (board.place(m, me)) { status = GameStatus.WON; winner = Optional.of(me); }
                else if (board.full()) status = GameStatus.DRAW;
                else turn = 1 - turn;
            } catch (InvalidMoveException e) {
                System.out.println("  " + me + " tried an invalid move: " + e.getMessage());
                // same player moves again; a real UI would re-prompt
            }
        }
    }
}
public class TicTacToeDemo {
    public static void main(String[] args) {
        Game g1 = new Game(3,
            new ScriptedPlayer(new Move(0, 0), new Move(1, 1), new Move(2, 2)),
            new ScriptedPlayer(new Move(0, 0), new Move(0, 1), new Move(0, 2)));
        g1.play();
        System.out.println("game 1: " + g1.status() + " winner=" + g1.winner().orElse(null));
        System.out.print(g1.board().render());

        for (long seed : new long[] {7, 11}) {
            Game g = new Game(4, new RandomPlayer(seed), new RandomPlayer(seed + 100));
            g.play();
            System.out.println("4x4 random game seed " + seed + ": " + g.status()
                + " winner=" + g.winner().orElse(null));
            System.out.print(g.board().render());
        }
    }
}
```

3.12.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out TicTacToeDemo.java
$ java -cp out TicTacToeDemo
  O tried an invalid move: occupied: Move[row=0, col=0]
game 1: WON winner=X
XOO
.X.
..X
4x4 random game seed 7: WON winner=X
.OXX
OOXO
OOXX
OXXX
4x4 random game seed 11: WON winner=X
.OOO
X..O
XXXX
.XO.
```

The scripted game shows rejection of an invalid move: player O tries cell (0, 0), which X already holds, receives the error, and plays again; X then completes the diagonal. The two 4 by 4 games use seeded random players, so they are reproducible.

3.12.6 Concurrency and extensions

Concurrency. A single game is turn-based and needs no locking if one thread drives it. If moves arrive from two network threads, serialise them through one lock in `Game` (or one single-threaded executor per game) and reject a move that arrives out of turn.

Extensions.

- A minimax or alpha-beta player for 3 by 3: a new `PlayerStrategy` that copies the board and evaluates moves.
- Undo: keep a stack of moves and a Command object per move (see the Command pattern); `Board` needs a `clear(Move)` method that also decrements the counters.
- Chess: the same split (board, pieces, rules per piece, game loop) with a `Piece` hierarchy, each piece implementing `possibleMoves(Board)`; check detection becomes a query over opponent moves.

> **GOTCHA**
> The expression `++a == n | ++b == n` uses the non-short-circuit `|` deliberately: both counters must be incremented on every move. With `||` the second increment would be skipped whenever the first condition is true. Comment such lines, or split them into two statements.

3.12.7 Real-world problems and solutions

Win check scanning the whole board

Scenario. A solver evaluates millions of positions with an O(N) scan per move.

Symptoms. CPU profile shows the win check as the hot spot.

Diagnosis. Profiler attributes most time to nested loops in `checkWin`.

Root cause. Each move re-reads cells that the move did not change.

Fix. Maintain row, column and diagonal counters as moves are placed; check only the lines through the new cell.

Prevention. Use incremental updates when a derived property changes locally.

Counters not updated for the anti-diagonal

Scenario. A game on a 4 by 4 board never detects wins along the anti-diagonal.

Symptoms. Players fill the line and the game continues until the board is full.

Diagnosis. A unit test with moves (0,3), (1,2), (2,1), (3,0) fails.

Root cause. The condition for the anti-diagonal was written as `row == col` instead of `row + col == n - 1`.

Fix. Use the correct condition, with a test for each of the four line types on two board sizes.

sreeram

Prevention. Test every winning line type, not only rows.

3.12.8 Questions to expect

- **Q. How is the winner detected in O(1)?** A. Counters per player for each row, column and the two diagonals; after a move, increment the counters of the lines through the cell and compare with N.
- **Q. How would a bot be added?** A. Implement `PlayerStrategy`; for 3 by 3 a minimax search over at most 9! positions is feasible, with alpha-beta pruning.
- **Q. Where are the rules validated?** A. Placement rules (bounds, occupancy) in `Board`; turn order in `Game`.
- **Q. How is the design adapted for chess?** A. Add `Piece` with subclasses overriding the move generation, `Move` with from and to squares, and rules for check and checkmate in a separate component.
- **Q. Why are `Move` and `Mark` not plain ints?** A. Types prevent mixing a row with a column and make the code readable; records give equality and `toString` for free.

KEY POINTS

Split the board (rules of the grid) from the game (turns and status), and make players a strategy.

- Per-line counters give O(1) win detection.
- Invalid moves are reported by the board and handled by the game loop.
- Test every line type, including the anti-diagonal.

3.13 LLD: Pub-Sub Event Bus

3.13.1 Requirements and use cases

Components publish events to named topics, and any number of subscribers receive them. Publishing must not wait for subscribers. Events on one topic reach each subscriber in publication order. A failing subscriber must not affect the others, and events that cannot be processed after retries are kept for inspection.

Item	Statement
Scope	In-process bus (the same idea as Spring application events); a broker such as Kafka covers the distributed case in the messaging material.
Delivery	At most one handler thread per subscriber; ordered per subscriber and topic.
Failure	Two attempts per event, then a dead-letter entry.
Out of scope	Persistence, replay, wildcard topics and back-pressure on publishers.

Assumptions.

3.13.2 Entities and relationships

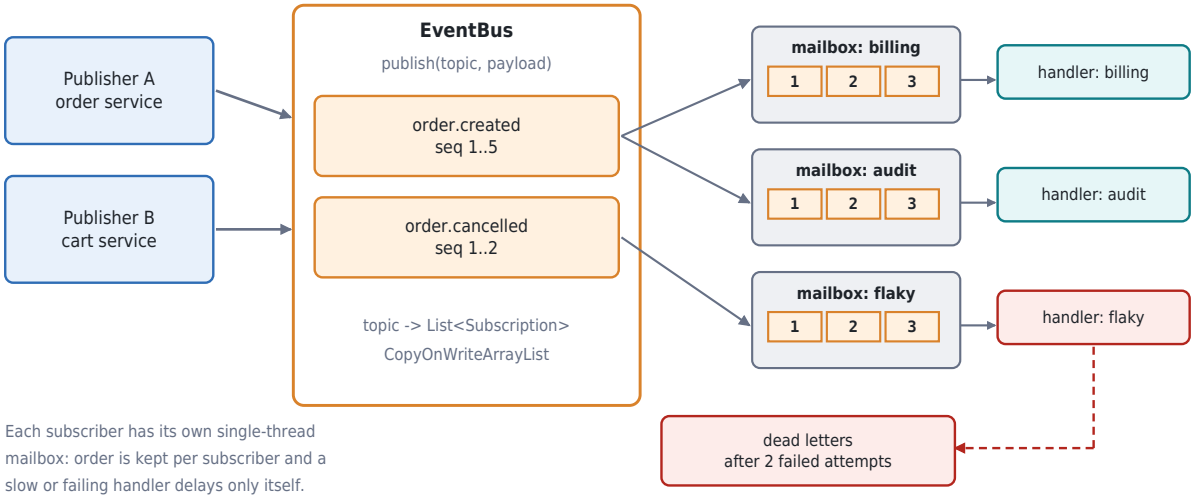


Figure 3.27. Each subscription owns a single-threaded mailbox; the bus maps topics to subscriptions and assigns per-topic sequence numbers.

3.13.3 Design decisions and patterns

Decision	Choice	Reason
Pattern	Observer, with an asynchronous mailbox per observer	The publisher knows only the bus; the bus knows only subscriptions.
Ordering	One single-thread executor per subscriber	Tasks of one executor run in submission order; a shared pool would reorder.
Isolation	Exceptions caught inside the mailbox task	A failing handler cannot reach the publisher or other subscribers.
Retry	Two attempts, then dead letter	Bounded effort; failures remain visible.
Subscriber list	`CopyOnWriteArrayList`	Many reads (every publish) and few writes (subscribe, unsubscribe).

Design decisions.

sreeram

3.13.4 Implementation

PubSubDemo.java JAVA

```java
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.function.Consumer;

record Event(String topic, long sequence, String payload) {}

/** One subscriber = one single-threaded mailbox: ordered delivery and failure isolation. */
final class Subscription {
    private final String name;
    private final Consumer<Event> handler;
    private final ExecutorService mailbox;
    private final EventBus bus;

    Subscription(String name, Consumer<Event> handler, EventBus bus) {
        this.name = name; this.handler = handler; this.bus = bus;
        this.mailbox = Executors.newSingleThreadExecutor(r -> new Thread(r, "sub-" + name));
    }
    void deliver(Event e) {
        mailbox.submit(() -> {
            for (int attempt = 1; attempt <= 2; attempt++) {
                try { handler.accept(e); return; }
                catch (RuntimeException ex) {
                    if (attempt == 2) bus.deadLetter(name, e, ex);      // give up after two tries
                }
            }
        });
    }
    void stop() throws InterruptedException {
        mailbox.shutdown();
        mailbox.awaitTermination(5, TimeUnit.SECONDS);
    }
}

final class EventBus {
    private final Map<String, List<Subscription>> topics = new ConcurrentHashMap<>();
    private final Map<String, Long> sequences = new ConcurrentHashMap<>();
    final List<String> deadLetters = new CopyOnWriteArrayList<>();

    Subscription subscribe(String topic, String name, Consumer<Event> handler) {
        Subscription s = new Subscription(name, handler, this);
        topics.computeIfAbsent(topic, t -> new CopyOnWriteArrayList<>()).add(s);
        return s;
    }
    void unsubscribe(String topic, Subscription s) {
        List<Subscription> l = topics.get(topic);
        if (l != null) l.remove(s);
    }
    void publish(String topic, String payload) {
        long seq = sequences.merge(topic, 1L, Long::sum);              // per-topic sequence number
        Event e = new Event(topic, seq, payload);
        for (Subscription s : topics.getOrDefault(topic, List.of())) s.deliver(e);
    }
    void deadLetter(String subscriber, Event e, Exception ex) {
        deadLetters.add(subscriber + " <- " + e.topic() + "#" + e.sequence()
            + " (" + ex.getMessage() + ")");
    }
    void shutdown() throws InterruptedException {
        for (List<Subscription> l : topics.values()) for (Subscription s : l) s.stop();
    }
}
public class PubSubDemo {
    public static void main(String[] args) throws Exception {
        EventBus bus = new EventBus();
        List<String> billing = new CopyOnWriteArrayList<>();
        List<String> audit = new CopyOnWriteArrayList<>();

        bus.subscribe("order.created", "billing",
            e -> billing.add(e.sequence() + ":" + e.payload()));
```

PubSubDemo.java (continued) JAVA

```
        bus.subscribe("order.created", "audit", e -> audit.add("audit " + e.payload()));
        bus.subscribe("order.created", "flaky", e -> {
            if (e.sequence() == 3) throw new IllegalStateException("mail server down");
        });
        Subscription temp = bus.subscribe("order.cancelled", "temp", e -> audit.add("temp " + e));

        for (int i = 1; i <= 5; i++) bus.publish("order.created", "order-" + i);
        bus.publish("order.cancelled", "order-2");
        bus.unsubscribe("order.cancelled", temp);
        bus.publish("order.cancelled", "order-4");                    // nobody listens any more
        bus.shutdown();
        temp.stop();

        System.out.println("billing (ordered): " + billing);
        System.out.println("audit count: "
            + audit.stream().filter(a -> a.startsWith("audit")).count());
        System.out.println("temp saw cancelled events: "
            + audit.stream().filter(a -> a.startsWith("temp")).count());
        System.out.println("dead letters: " + bus.deadLetters);
    }
}
```

3.13.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out PubSubDemo.java
$ java -cp out PubSubDemo
billing (ordered): [1:order-1, 2:order-2, 3:order-3, 4:order-4, 5:order-5]
audit count: 5
temp saw cancelled events: 1
dead letters: [flaky <- order.created#3 (mail server down)]
```

Billing saw sequences 1 to 5 in order. The `flaky` handler failed on event 3 twice and the bus recorded a dead letter, while the audit subscriber still received all five events. The `temp` subscriber received the cancellation published before it unsubscribed and nothing afterwards.

3.13.6 Concurrency and extensions

Ordering limit. `publish` increments the sequence and then delivers. If two threads publish to the same topic concurrently, sequence 2 may be delivered before 1 to a subscriber, because the increment and the hand-over to mailboxes are not one atomic step. If a global order per topic matters, make `publish` synchronised per topic or have a single dispatcher thread; the cost is publisher contention.

Resource cost. One thread per subscriber does not scale to thousands of subscribers. Use virtual threads (JDK 21) for mailboxes, or a shared pool with a per-subscriber queue and a flag that ensures one task at a time.

Backpressure. The executor queues are unbounded. Production designs bound them and choose to drop, block or fail the publisher, as for the logger.

Extensions. Typed events (`subscribe(Class<T>, Consumer<T>)`), wildcard topics, a replay buffer, retry with exponential backoff, and delivery metrics.

3.13.7 Real-world problems and solutions

One slow subscriber stalls all others

Scenario. The bus calls handlers sequentially on the publisher thread.

Symptoms. Order placement latency equals the sum of all handler times; a handler that waits on a network call blocks checkout.

Diagnosis. Thread dump of request threads shows frames inside handler code.

Root cause. Synchronous fan-out.

Fix. Asynchronous delivery with a mailbox per subscriber.

Prevention. Publishers must not execute subscriber code.

Events processed out of order

Scenario. Handlers run on a shared pool of eight threads.

Symptoms. An "order cancelled" update is applied before "order created"; the final state is wrong.

Diagnosis. Sequence numbers in the log are not monotonic per subscriber.

Root cause. Tasks for one subscriber ran concurrently on different threads.

Fix. Serialise per subscriber (single-thread executor or a per-subscriber queue).

Prevention. Ordering guarantees are stated per key and implemented by confinement.

Poison event retried forever

Scenario. A malformed event makes the handler throw on every attempt.

Symptoms. The subscriber's queue fills with retries; healthy events are delayed.

Diagnosis. The same event id appears in repeated error logs.

Root cause. Unbounded retry.

Fix. Bounded attempts and a dead-letter store, as implemented.

Prevention. Every retry loop has a maximum; failures are observable.

3.13.8 Questions to expect

- **Q. Why does each subscriber have its own executor?** A. It gives ordered delivery per subscriber and isolates slow or failing handlers.
- **Q. How is at-least-once delivery achieved in a distributed bus?** A. The broker redelivers until the consumer acknowledges; consumers must be idempotent. This in-process bus offers at-most-once effort with two attempts.
- **Q. Observer versus pub-sub?** A. In Observer the subject knows its observers directly; in pub-sub a bus with topics decouples publishers from subscribers.
- **Q. How is unsubscribe made safe during publish?** A. The copy-on-write list gives iteration over a snapshot, so changes do not disturb running iterations.
- **Q. What changes when moving to Kafka?** A. Topics become partitioned logs, ordering holds per partition, consumers track offsets, and delivery becomes at-least-once with idempotent handlers.

KEY POINTS

The bus decouples publishers and subscribers; delivery is asynchronous through one ordered mailbox per subscriber.

- Exceptions stay inside the mailbox; retries are bounded and end in a dead-letter record.
- Sequence assignment and hand-over must be atomic if strict per-topic order is required.
- Bound the queues in any production design.

3.14 LLD: Task Scheduler

3.14.1 Requirements and use cases

A scheduler runs tasks after a delay or at a fixed rate on a small pool of worker threads. Tasks may be cancelled. A task that throws must not kill the worker. The class is a simplified counterpart of `ScheduledThreadPoolExecutor`, built to expose the mechanism: a time-ordered queue and threads that sleep until the earliest task is due.

Item	Statement
Time source	`System.nanoTime` (monotonic).
Periodic tasks	Fixed rate: the next run is the previous scheduled time plus the period, so delays do not accumulate.
Failure	A throwing task is reported and, if periodic, is rescheduled (a policy; `ScheduledThreadPoolExecutor` stops a periodic task after an exception).
Out of scope	Priorities, persistence and cron expressions.

Assumptions.

3.14.2 Entities and relationships

`ScheduledTask` holds the body, the next run time, the period and a cancel flag, and is ordered by (run time, id). `SimpleScheduler` owns a priority queue, one lock, one condition and the workers.

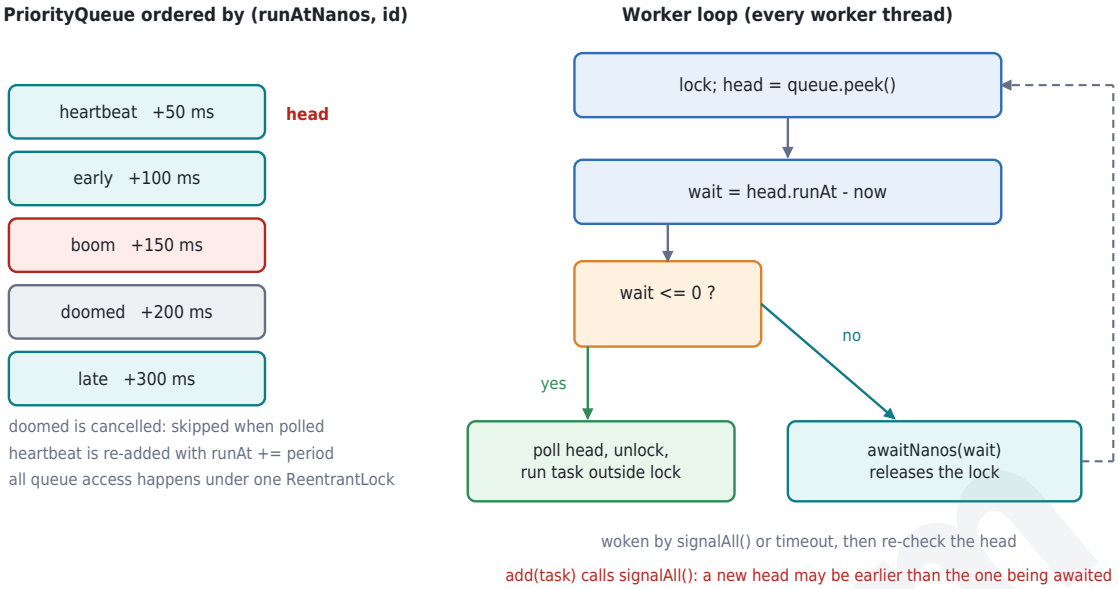


Figure 3.28. Workers peek at the head, compute the wait, and either run the task or sleep with `awaitNanos`; adding a task signals the condition because it may become the new head.

3.14.3 Design decisions and patterns

Decision	Choice	Reason
Queue	`PriorityQueue` under a `ReentrantLock`	Ordered by due time; `peek` and `poll` are cheap; the lock protects the heap.
Waiting	`Condition.awaitNanos`	Sleeps exactly until the head is due and is woken by `signalAll` when an earlier task arrives.
Tie-break	Task id from an `AtomicLong`	FIFO among tasks due at the same instant; a deterministic order.
Task execution	Outside the lock	A slow task must not block `schedule` or other workers.
Cancellation	A `volatile` flag checked before running	Cheap; the cancelled task is discarded when it reaches the head.
Pattern	Command (each task is a runnable object) and producer-consumer	The queue decouples submitters from workers.

Design decisions.

sreeram

3.14.4 Implementation

SchedulerDemo.java JAVA

```
import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

final class ScheduledTask implements Comparable<ScheduledTask> {
    private static final AtomicLong SEQ = new AtomicLong();

    final long id = SEQ.incrementAndGet();                // tie-breaker: FIFO among equal times
    final String name;
    final Runnable body;
    final long periodNanos;                               // 0 = one-shot
    volatile long runAtNanos;
    volatile boolean cancelled;

    ScheduledTask(String name, Runnable body, long runAtNanos, long periodNanos) {
        this.name = name;
        this.body = body;
        this.runAtNanos = runAtNanos;
        this.periodNanos = periodNanos;
    }
    void cancel() { cancelled = true; }
    public int compareTo(ScheduledTask o) {
        int c = Long.compare(runAtNanos, o.runAtNanos);
        return c != 0 ? c : Long.compare(id, o.id);
    }
}
final class SimpleScheduler {
    private final PriorityQueue<ScheduledTask> queue = new PriorityQueue<>();
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition changed = lock.newCondition();    // head may have changed
    private final List<Thread> workers = new ArrayList<>();
    private boolean shutdown;                                 // guarded by lock

    SimpleScheduler(int threads) {
        for (int i = 0; i < threads; i++) {
            Thread t = new Thread(this::workerLoop, "sched-" + i);
            workers.add(t);
            t.start();
        }
    }

    ScheduledTask schedule(String name, Runnable r, long delay, TimeUnit unit) {
        return add(new ScheduledTask(name, r, System.nanoTime() + unit.toNanos(delay), 0));
    }
    ScheduledTask scheduleAtFixedRate(String name, Runnable r, long initial, long period,
                                      TimeUnit u) {
        long first = System.nanoTime() + u.toNanos(initial);
        return add(new ScheduledTask(name, r, first, u.toNanos(period)));
    }

    private ScheduledTask add(ScheduledTask t) {
        lock.lock();
        try {
            if (shutdown) throw new IllegalStateException("scheduler is shut down");
            queue.add(t);
            changed.signalAll();                              // sleepers must re-check the head
            return t;
        } finally { lock.unlock(); }
    }

    private void workerLoop() {
        while (true) {
            ScheduledTask task;
            lock.lock();
            try {
                while (true) {
                    if (shutdown) return;
                    ScheduledTask head = queue.peek();
                    if (head == null) { changed.awaitUninterruptibly(); continue; }
```

Sreeram

SchedulerDemo.java (continued) JAVA

```
                    long wait = head.runAtNanos - System.nanoTime();
                    if (wait <= 0) { task = queue.poll(); break; }
                    try { changed.awaitNanos(wait); } catch (InterruptedException e) { return; }
                }
            } finally { lock.unlock(); }

            if (task.cancelled) continue;
            try { task.body.run(); }                          // run outside the lock
            catch (RuntimeException e) {
                System.out.println("task " + task.name + " failed: " + e);
            }
            if (task.periodNanos > 0 && !task.cancelled) {
                task.runAtNanos += task.periodNanos;          // fixed rate: no drift accumulation
                lock.lock();
                try { if (!shutdown) { queue.add(task); changed.signalAll(); } }
                finally { lock.unlock(); }
            }
        }
    }

    void shutdown() throws InterruptedException {
        lock.lock();
        try { shutdown = true; changed.signalAll(); } finally { lock.unlock(); }
        for (Thread t : workers) t.join();
    }
}
public class SchedulerDemo {
    public static void main(String[] args) throws Exception {
        long start = System.nanoTime();
        SimpleScheduler s = new SimpleScheduler(2);
        java.util.function.Function<String, Runnable> say = msg -> () ->
            System.out.printf("  +%3d ms  %s%n",
                ((System.nanoTime() - start) / 1_000_000 + 25) / 50 * 50, msg);

        s.schedule("late", say.apply("one-shot task due at +300 ms"), 300, TimeUnit.MILLISECONDS);
        s.schedule("early", say.apply("one-shot task due at +100 ms"), 100, TimeUnit.MILLISECONDS);
        ScheduledTask hb = s.scheduleAtFixedRate("heartbeat", say.apply("heartbeat (every 100 ms)"),
            50, 100, TimeUnit.MILLISECONDS);
        ScheduledTask doomed =
            s.schedule("doomed", say.apply("never printed"), 200, TimeUnit.MILLISECONDS);
        s.schedule("boom", () -> { throw new IllegalStateException("bad job"); },
            150, TimeUnit.MILLISECONDS);
        doomed.cancel();

        Thread.sleep(450);
        hb.cancel();
        s.shutdown();
        System.out.println("scheduler stopped");
    }
}
```

3.14.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out SchedulerDemo.java
$ java -cp out SchedulerDemo
  + 50 ms  heartbeat (every 100 ms)
  +100 ms  one-shot task due at +100 ms
  +150 ms  heartbeat (every 100 ms)
task boom failed: java.lang.IllegalStateException: bad job
  +250 ms  heartbeat (every 100 ms)
  +300 ms  one-shot task due at +300 ms
  +350 ms  heartbeat (every 100 ms)
  +450 ms  heartbeat (every 100 ms)
scheduler stopped
```

The times are rounded to 50 ms and vary slightly between runs (typical output). The heartbeat first runs at 50 ms and then every 100 ms until cancelled; the one-shot tasks run at 100 and 300 ms; the cancelled task never runs; the failing task is reported and does not stop the workers.

sreeram

3.14.6 Concurrency and extensions

Lost wake-up. If `add` did not signal after inserting, a worker sleeping until a distant head would miss a new earlier task. `signalAll` after every insertion prevents this; workers always re-check the head after waking, which also tolerates spurious wake-ups.

Why the task runs outside the lock. Holding the lock during `body.run()` would serialise all tasks and block `schedule` calls for the task's duration.

Fixed rate and fixed delay. The code adds the period to the previous scheduled time (fixed rate). Fixed delay would add the period to the completion time; it avoids bursts of catch-up runs after a long task but drifts. Both are standard options.

Extensions. A `Future` result for one-shot tasks; a bounded queue; a `shutdownNow` that interrupts workers; persistence of tasks for restart; clustering with a lease so that only one node runs a given task.

> **WATCH OUT**
> A periodic task that takes longer than its period causes the scheduled time to fall behind; with fixed rate the next run is immediately due, and runs may bunch up. Production code needs a policy: skip missed runs, or alert.

3.14.7 Real-world problems and solutions

Busy loop instead of waiting

Scenario. A first version polls `queue.peek()` in a loop with `Thread.sleep(1)`.

Symptoms. CPU stays at several percent with no tasks; tasks start up to one millisecond late.

Diagnosis. A profiler shows time in the polling loop.

Root cause. Polling instead of waiting on a condition.

Fix. `awaitNanos` until the head is due, with `signalAll` on insertion.

Prevention. Wait on conditions; never poll with sleep for coordination.

Task exception kills the worker

Scenario. A task throws a `RuntimeException`.

Symptoms. Worker threads disappear one by one; after as many failures as workers, nothing runs.

Diagnosis. Thread dump shows fewer scheduler threads than configured.

Root cause. The exception propagated out of the worker loop.

Fix. Catch exceptions around `body.run()` and report them.

Prevention. Worker loops catch everything a task can throw; test with a failing task.

Duplicate runs after a restart of a clustered scheduler

Scenario. Two nodes load the same schedule from a database.

Symptoms. A nightly job runs twice.

Diagnosis. Two start records for the same time slot.

Root cause. No mutual exclusion between nodes; an in-process queue cannot provide it.

Fix. Claim each due task with a conditional update or a lease row before running, with a fencing token.

Prevention. Cluster-wide singletons need a shared coordination point.

3.14.8 Questions to expect

- **Q. Why `awaitNanos` and not `Thread.sleep`?** A. The wait must be interruptible by a new earlier task and must release the lock while waiting; `sleep` does neither.
- **Q. What is a spurious wake-up?** A. A wait may return without a signal or timeout; a loop that re-checks the condition handles it.
- **Q. Fixed rate versus fixed delay?** A. Fixed rate keeps the average frequency (runs may bunch up after a delay); fixed delay keeps a gap between the end of one run and the start of the next.

sreeram

- **Q. How is cancellation handled?** A. A flag checked at execution; for large queues a lazy discard is cheaper than removing from the middle of a heap, O(n).
- **Q. How is it done in production Java?** A. `ScheduledThreadPoolExecutor` (via `Executors.newScheduledThreadPool`) or Spring's `@Scheduled` with a `TaskScheduler`; use Quartz or a database-backed scheduler for persistence and clustering.

KEY POINTS

A scheduler is a time-ordered queue plus workers that wait on a condition until the head is due.

- Signal on every insertion; re-check the head after every wake-up.
- Run tasks outside the lock and catch every task exception.
- Fixed rate and fixed delay are different policies; choose and document one.

3.15 LLD: Connection Pool

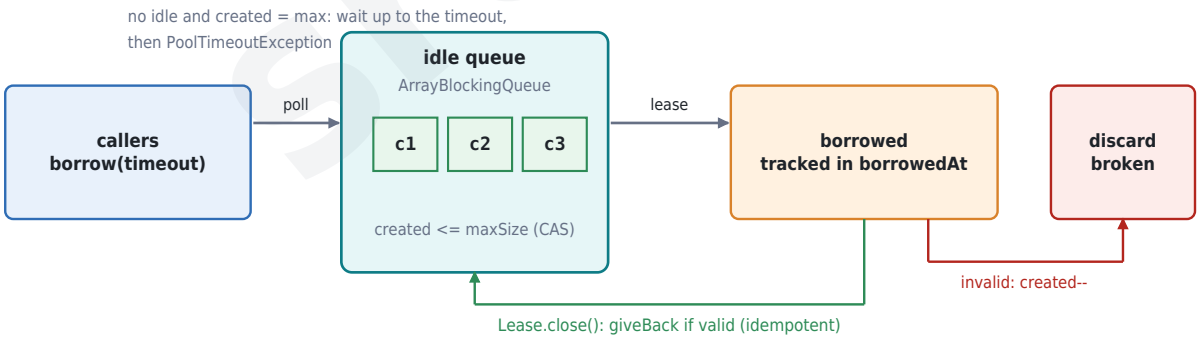
3.15.1 Requirements and use cases

A fixed-size pool of expensive objects (database connections) shared by many threads. A caller borrows a connection, uses it, and returns it. When the pool is exhausted the caller waits up to a timeout and then fails. Broken connections are discarded. Connections borrowed for too long are reported as probable leaks. The mature implementations are HikariCP and similar libraries (the pool behind Spring Boot's default data source); this chapter shows the structure.

Item	Statement
Size	Maximum `maxSize`; connections are created lazily.
Timeout	A borrower waits at most the given duration.
Validation	`isValid()` checked when borrowing and returning.
Out of scope	Idle eviction, maximum lifetime, metrics and a real driver.

Assumptions.

3.15.2 Entities and relationships



6 callers, pool of 3, 100 ms work, 60 ms patience: three callers time out (demo output).

Figure 3.29. A connection moves between the idle queue and the borrowed set; a broken connection leaves the cycle and the created count decreases so that the pool can create a replacement.

3.15.3 Design decisions and patterns

Decision	Choice	Reason
Idle storage	`ArrayBlockingQueue`	Thread-safe, bounded, and provides a timed `poll` for waiting.
Growth	CAS loop on an `AtomicInteger` (`tryReserve`)	The pool never creates more than `maxSize`, even when many threads see a free slot at once.
Return path	`Lease implements AutoCloseable`	try-with-resources returns the connection on every exit path; `close` is idempotent.
Leak detection	`borrowedAt` map and `suspectedLeaks(threshold)`	Finds callers that never returned a connection.
Pattern	Object pool; the lease is a Proxy-like handle	Callers hold the lease, not the pooled object.

Design decisions.

GOTCHA

The first version of this class used `created.incrementAndGet() <= maxSize` inside the condition. When the increment overshot the limit the counter was never decremented and the pool permanently believed it was full. A compare-and-set loop that increments only when below the limit avoids that error; the test for it is many threads borrowing at once.

sreeram

3.15.4 Implementation

PoolDemo.java JAVA

```java
import java.time.Duration;
import java.util.Map;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

interface Connection extends AutoCloseable {
    String query(String sql);
    boolean isValid();
    @Override void close();
}

/** Stand-in for a real driver connection. */
class FakeConnection implements Connection {
    private static final AtomicInteger IDS = new AtomicInteger();
    final int id = IDS.incrementAndGet();
    volatile boolean broken;
    public String query(String sql) { return "conn#" + id + " ran: " + sql; }
    public boolean isValid() { return !broken; }
    public void close() { }
}
class PoolTimeoutException extends RuntimeException {
    PoolTimeoutException(String m) { super(m); }
}

class ConnectionPool implements AutoCloseable {
    /** A lease returns the connection to the pool on close(), so try-with-resources cannot leak. */
    final class Lease implements AutoCloseable {
        private final FakeConnection conn;
        private boolean returned;
        Lease(FakeConnection c) { this.conn = c; }
        Connection connection() { return conn; }
        @Override public synchronized void close() {
            if (returned) return;                                  // idempotent
            returned = true;
            borrowedAt.remove(conn);
            giveBack(conn);
        }
    }

    private final int maxSize;
    private final BlockingQueue<FakeConnection> idle;
    private final AtomicInteger created = new AtomicInteger();      // idle + borrowed
    private final Map<FakeConnection, Long> borrowedAt = new ConcurrentHashMap<>();
    private volatile boolean closed;

    ConnectionPool(int maxSize) {
        this.maxSize = maxSize;
        this.idle = new ArrayBlockingQueue<>(maxSize);
    }

    Lease borrow(Duration timeout) throws InterruptedException {
        long deadline = System.nanoTime() + timeout.toNanos();
        while (!closed) {
            FakeConnection c = idle.poll();
            if (c == null && tryReserve()) {
                c = new FakeConnection();                          // grow lazily up to maxSize
            } else if (c == null) {
                long left = deadline - System.nanoTime();
                if (left <= 0) throw new PoolTimeoutException("no connection within " + timeout);
                c = idle.poll(left, TimeUnit.NANOSECONDS);         // wait for a return
                if (c == null) throw new PoolTimeoutException("no connection within " + timeout);
            }
            if (!c.isValid()) { created.decrementAndGet(); continue; }   // evict broken, retry
            borrowedAt.put(c, System.nanoTime());
            return new Lease(c);
        }
        throw new IllegalStateException("pool closed");
    }
```

PoolDemo.java (continued) JAVA

```
    private boolean tryReserve() {                                  // CAS: never exceeds maxSize
        int n;
        do {
            n = created.get();
            if (n >= maxSize) return false;
        } while (!created.compareAndSet(n, n + 1));
        return true;
    }

    private void giveBack(FakeConnection c) {
        if (closed || !c.isValid()) { created.decrementAndGet(); return; }
        idle.offer(c);
    }

    /** Reports connections borrowed for longer than the threshold: candidates for leaks. */
    int suspectedLeaks(Duration threshold) {
        long now = System.nanoTime();
        return (int) borrowedAt.values().stream()
            .filter(t -> now - t > threshold.toNanos()).count();
    }
    int idleCount() { return idle.size(); }
    int createdCount() { return created.get(); }
    @Override public void close() { closed = true; idle.clear(); }
}
public class PoolDemo {
    public static void main(String[] args) throws Exception {
        ConnectionPool pool = new ConnectionPool(3);
        CountDownLatch done = new CountDownLatch(6);
        AtomicInteger timeouts = new AtomicInteger();
        for (int i = 0; i < 6; i++) {
            final int n = i;
            new Thread(() -> {
                try (ConnectionPool.Lease l = pool.borrow(Duration.ofMillis(60))) {
                    l.connection().query("SELECT " + n);
                    Thread.sleep(100);                                  // simulated query time
                } catch (PoolTimeoutException e) {
                    timeouts.incrementAndGet();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                } finally { done.countDown(); }
            }).start();
        }
        done.await();
        System.out.println("6 callers, pool of 3, 100 ms each, 60 ms patience -> timeouts = "
            + timeouts + ", created = " + pool.createdCount() + ", idle = " + pool.idleCount());

        ConnectionPool tiny = new ConnectionPool(1);
        ConnectionPool.Lease leaked = tiny.borrow(Duration.ofSeconds(1));   // never closed
        try { tiny.borrow(Duration.ofMillis(100)); } catch (PoolTimeoutException e) {
            System.out.println("second caller: " + e.getMessage());
        }
        Thread.sleep(150);
        System.out.println("suspected leaks (borrowed > 100 ms): "
            + tiny.suspectedLeaks(Duration.ofMillis(100)));
        ((FakeConnection) leaked.connection()).broken = true;
        leaked.close();                                         // broken one is discarded
        try (ConnectionPool.Lease fresh = tiny.borrow(Duration.ofSeconds(1))) {
            System.out.println("after discarding the broken connection: "
                + fresh.connection().query("SELECT 1"));
        }
    }
}
```

3.15.5 Run

Compile and run (JDK 21) TERMINAL

```
$ javac --release 21 -d out PoolDemo.java
$ java -cp out PoolDemo
6 callers, pool of 3, 100 ms each, 60 ms patience -> timeouts = 3, created = 3, idle = 3
second caller: no connection within PT0.1S
suspected leaks (borrowed > 100 ms): 1
after discarding the broken connection: conn#5 ran: SELECT 1
```

With six callers, three connections, 100 ms of work and 60 ms of patience, the first three callers proceed and the other three time out; the pool created exactly three connections and all three are idle at the end. The second part shows a leak: the pool of size one is exhausted by a lease that is never closed, the second caller times out, and `suspectedLeaks` reports one connection borrowed longer than 100 ms. Closing the lease of a broken connection discards it, and the next borrow creates a replacement.

3.15.6 Concurrency and extensions

Invariants. `created` equals the number of live connections (idle plus borrowed) and never exceeds `maxSize`. Every borrow either returns a lease or throws; every lease close either returns the connection to the idle queue or decrements `created`.

Fairness. `ArrayBlockingQueue` is unfair by default; a waiting borrower may be overtaken. Construct it with `fair = true` where starvation matters, at some cost in throughput.

Sizing. A common rule of thumb is that the pool should be small: the number of connections that the database can execute at once is bounded by its CPU and I/O, so a large pool adds queueing inside the database. Start with a small multiple of the database core count and measure; this is an assumption to validate, not a rule.

Extensions. Maximum connection lifetime and idle eviction (a scheduled sweep), a `ConnectionFactory` strategy, wrapping the connection in a proxy that intercepts `close()` so that callers use the normal API, metrics for wait time and usage.

3.15.7 Real-world problems and solutions

Pool exhaustion from leaked connections

Scenario. An error path in a repository method returns without closing the connection.

Symptoms. Request threads block waiting for connections; timeouts; the database shows few active queries.

Diagnosis. Pool metrics show all connections borrowed and long borrow times; leak detection names the stack of the borrower.

Root cause. The lease was not closed on one path.

Fix. try-with-resources everywhere; enable leak detection with a threshold.

Prevention. Static analysis for unclosed `AutoCloseable` values; an integration test that asserts the pool is full after each request.

Thread pool larger than connection pool

Scenario. 200 web threads share 10 connections.

Symptoms. Latency rises with load; most threads wait for a connection; the database is not busy.

Diagnosis. Wait time in the pool dominates request time.

Root cause. Concurrency in the application exceeds the resource that it needs per request.

Fix. Reduce time holding a connection (do not call remote services inside a transaction), size the pool from measurements, and limit concurrency on the database path.

Prevention. Alert on pool wait time, not only on database load.

Pool never recovers after a database restart

Scenario. The database restarts and all pooled connections are stale.

Symptoms. Every query fails until the application restarts.

Diagnosis. Errors reference a closed connection; the pool hands out the same dead objects.

Root cause. No validation when borrowing.

Fix. Validate on borrow (or use a driver-level validity check) and discard failures, as `borrow` does.

Prevention. Test by killing connections from the server side.

Sreeram

3.15.8 Questions to expect

- **Q. Why a try-with-resources lease?** A. It guarantees return on every exit path and makes a double close harmless.
- **Q. How is the maximum size enforced under concurrency?** A. An atomic counter updated by compare-and-set only when below the maximum; a plain check followed by an increment would allow overshoot.
- **Q. What is a leak and how is it found?** A. A borrowed connection that is never returned. Record the borrow time (and optionally the stack) and report borrows older than a threshold.
- **Q. Why is a smaller pool often faster?** A. The database has limited parallelism; extra connections add contention inside it, while waiting in the pool is cheap.
- **Q. What happens when the timeout expires?** A. The caller receives `PoolTimeoutException`; it should fail the request quickly rather than retry immediately, to avoid amplifying overload.

KEY POINTS

A pool is a bounded idle queue, a CAS-guarded creation counter and a lease that returns the object on close.

- Validate on borrow, discard broken objects, and report long borrows as leaks.
- Bound the wait; a timeout is a signal of overload, not a reason to retry at once.
- Size by measurement; a small pool is usually enough.

sreeram

PART 4

Operating at Scale and Interview Preparation

How designs are run, secured, structured and defended: observability, security, architecture styles, failure studies, interview preparation and a complete product.

In this part

- **4.1** Observability and Operations at Scale
- **4.2** Security in System Design
- **4.3** Architecture Styles and Evolution
- **4.4** Failure Case Studies
- **4.5** Interview Preparation
- **4.6** Cheat Sheets and Study Plan
- **4.7** Putting It Together: Designing a Complete Product End to End

4.1 Observability and Operations at Scale

A design is incomplete until it states how the running system is observed, released, scaled and recovered. Interviewers frequently ask "how would this be monitored" or "how would this be deployed without downtime", and production incidents are mostly diagnosed and resolved with the mechanisms described in this chapter rather than with the architecture diagram. The chapter covers the three observability signals, alerting derived from service level objectives, capacity planning, autoscaling, deployment strategies, feature flags, multi-region operation, disaster recovery, cost and chaos engineering. Terms such as SLO, SLI, availability and graceful degradation are defined in the availability chapter of the fundamentals part; the resilience patterns (timeouts, retries, circuit breakers) are covered in the Spring Boot microservices material and are used here by reference.

4.1.1 Concept and mechanism

The three signals

Metrics are numeric time series aggregated at the source: counters (requests, errors), gauges (queue depth, open connections) and histograms (latency distribution). They are cheap to store and fast to query, which makes them the basis of dashboards and alerts. Two checklists decide what to measure. The **RED** method applies to request-driven services: Rate, Errors, Duration. The **USE** method applies to resources such as CPU, disks and connection pools: Utilisation, Saturation, Errors. Latency must be recorded as a histogram, because an average hides the tail: a service with a 40 ms average and a 2 s p99 appears healthy on an average-based dashboard.

Logs are timestamped event records. Structured logs (one JSON object per line with fields such as `trace_id`, `user_id`, `order_id`, `level`) can be filtered and aggregated; free-text logs cannot. Logs are expensive at scale, so retention is tiered: a short hot period in a search index and a long cold period in object storage.

Traces record the path of one request across services as a tree of **spans**, each with a start time, a duration and the identifier of its parent. A trace identifier is generated at the edge and propagated in a header (the W3C `traceparent` header is the standard format). A trace answers the question that metrics cannot: which hop in this particular slow request consumed the time. Tracing every request is usually unaffordable, so sampling is applied: head sampling decides at the start (for example 1 to 10 percent), tail sampling decides after the trace completes and keeps all traces that contain errors or exceed a latency threshold.

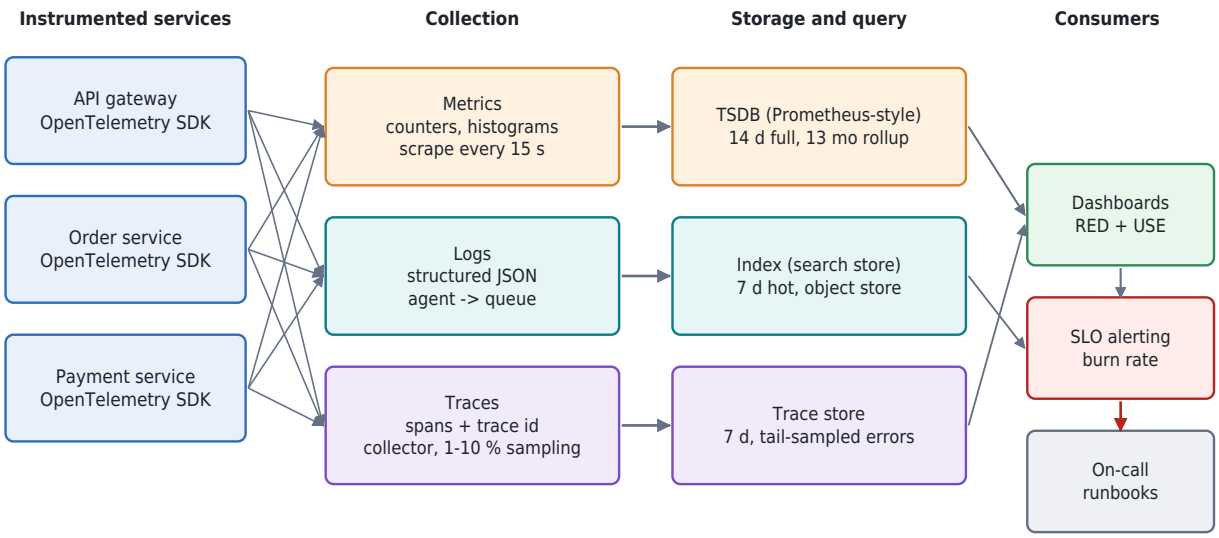


Figure 4.1. Each service emits all three signals through one instrumentation library; each signal has its own collection path, store and retention, and the alerting component consumes metrics rather than logs.

The trace in the next figure belongs to one slow checkout request. The gateway span lasts 420 ms, yet the payment span explains most of it: the first attempt to the external provider timed out and the second succeeded after 110 ms. No metric dashboard would identify this; the waterfall does so immediately.

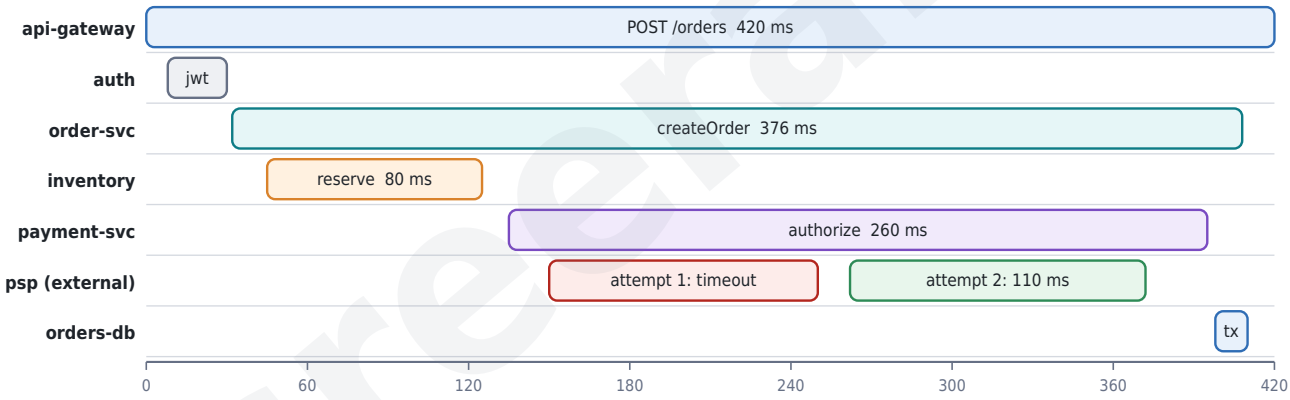


Figure 4.2. A trace waterfall for one request: child spans nest inside their parents, and the failed first attempt to the external provider accounts for roughly 100 ms of avoidable latency.

SLO-based alerting and burn rate

Alerting on causes (CPU above 80 percent) produces pages that require no action. Alerting on **symptoms experienced by users**, expressed against an SLO, produces pages that matter. An SLO of 99.9 percent success over 30 days defines an **error budget** of 0.1 percent of requests. The **burn rate** is the observed error ratio divided by the budget ratio: a burn rate of 1 consumes exactly the whole budget in 30 days, a burn rate of 14.4 consumes 2 percent of the budget in one hour.

A single threshold either fires late (long window) or fires on every blip (short window). The multiwindow, multi-burn-rate method combines a long window, which proves that the budget is being consumed significantly, with a short window, which proves that the problem is still occurring. Per the Google SRE Workbook chapter "Alerting on SLOs", the recommended parameters for a 99.9 percent SLO are:

Burn rate	Long window	Short window	Budget consumed	Severity
14.4	1 hour	5 minutes	2 percent	Page
6	6 hours	30 minutes	5 percent	Page
1	3 days	6 hours	10 percent	Ticket

Recommended multiwindow burn-rate alerts for a 99.9 percent SLO (per the Google SRE Workbook).



Figure 4.3. A 5 percent error ratio for 30 minutes (assumed incident) pushes the 5-minute burn rate to 50 immediately, but the page fires only at minute 617, when the 1-hour burn rate crosses 14.4; both windows must exceed the threshold.

Capacity planning and load testing

Capacity planning converts the estimate of peak traffic into a number of instances, with explicit headroom. The inputs are the peak request rate, the sustainable rate of one instance measured by a **load test**, a target utilisation (commonly 50 to 70 percent, leaving room for bursts and for garbage collection or retry spikes) and the failure domain that must be survivable (loss of one availability zone). A load test must use a realistic request mix, realistic data volume (a cache that fits in memory with test data may not with production data) and must increase load in steps until latency or error rate degrades, recording the **knee** of the curve rather than only the maximum. Little's law, $L = \lambda \times W$, relates concurrency to rate and latency and sizes connection pools and thread pools: 7,000 requests per second at 120 ms mean latency implies 840 requests in flight.

Autoscaling

Autoscaling adjusts the number of instances to demand. **Target tracking** keeps a metric near a target (Kubernetes Horizontal Pod Autoscaler computes `desiredReplicas = ceil(currentReplicas × currentMetric / targetMetric)`). **Step scaling** applies larger adjustments at larger breaches. **Scheduled scaling** pre-provisions for known peaks such as a lunch or a sale. **Predictive scaling** forecasts load from history. Three properties limit reactive autoscaling. First, the delay between the metric breach and a ready instance (metric aggregation, provisioning, image pull, warm-up) is typically minutes. Second, CPU utilisation saturates at 100 percent, so the autoscaler cannot observe how far demand exceeds capacity and scales in small increments. Third, scale-in must be slower than scale-out (stabilisation windows) or the fleet oscillates.

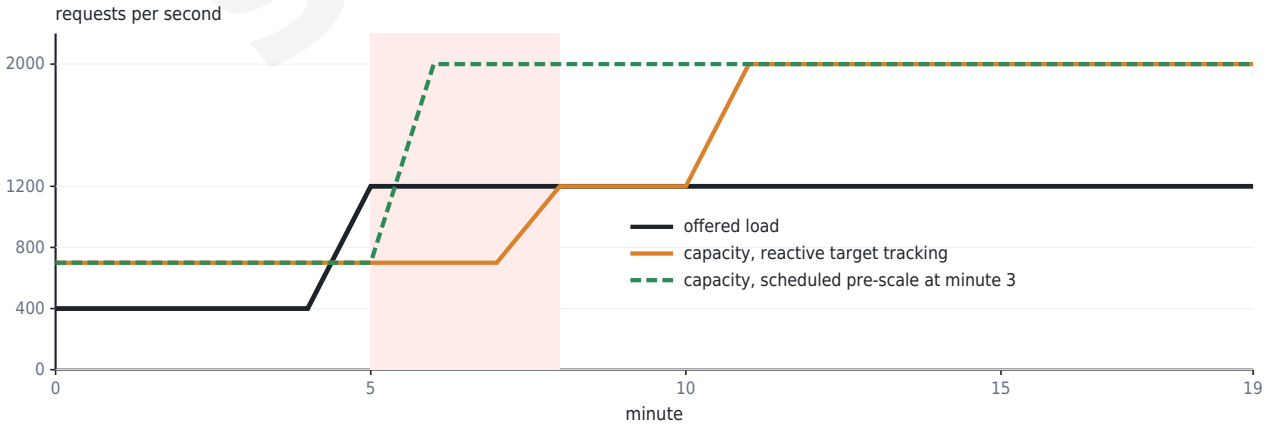


Figure 4.4. Simulated spike from 400 to 1,200 requests per second: reactive target tracking restores capacity only at minute 11 because saturated CPU hides the size of the overload, while a scheduled pre-scale limits the shortfall to one minute.

Deployment strategies and feature flags

Strategy	Mechanism	Advantage	Cost or risk
Rolling	Replace instances in batches behind the load balancer	No extra capacity; simple	Two versions serve traffic together; slow rollback
Blue-green	Build a full second environment, switch the router	Instant switch and rollback	Double capacity during release; database must serve both versions
Canary	Send 1 to 5 percent of traffic to the new version, compare metrics, then widen	Limits blast radius; real traffic validates the change	Requires automated metric comparison and enough traffic in the canary
Recreate	Stop old, start new	Simplest	Downtime

Deployment strategies compared.

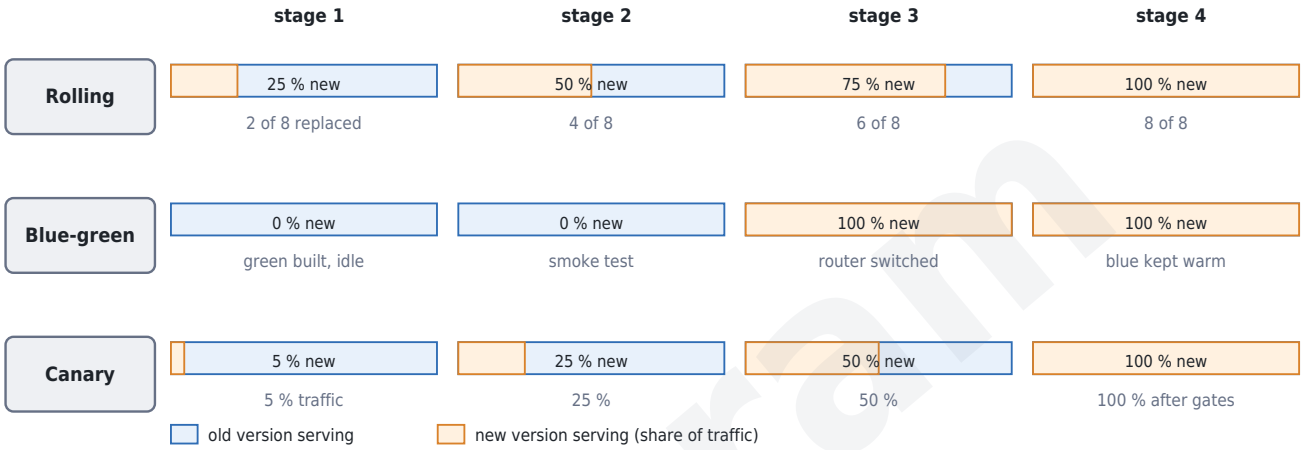


Figure 4.5. The three zero-downtime strategies differ in how the share of traffic served by the new version grows; blue-green changes from 0 to 100 percent in one step, canary grows it only after each gate passes.

Every strategy that runs two versions concurrently requires that the database schema is compatible with both. The standard method is **expand and contract**: first add the new column or table (expand) while the old code still works, then deploy code that writes both, then migrate data, then deploy code that reads only the new structure, and finally remove the old structure (contract). A rename done in one step breaks the old version during the rollout.

A **feature flag** separates deployment from release. Code ships disabled, and a flag service enables it for a percentage of users, a region or a cohort, and disables it within seconds without a deployment. Stable percentage rollout requires deterministic bucketing: the flag name and user identifier are hashed to a number from 0 to 99, and the user is enabled when the number is below the rollout percentage. Raising the percentage then only adds users and never flips an enabled user off. Flags carry operational cost: each is a branch to test, and flags that outlive their rollout become permanent hidden configuration, so each flag needs an owner and a removal date.

Multi-region design, data residency and disaster recovery

Multi-region operation serves two goals: latency (users reach a nearby region) and survival of a regional failure. It also introduces the hardest problem in distributed data: the same record cannot be written in two regions without conflict handling. Typical designs are **active-passive** (one region writes, the other is a standby), **active-active by partition** (each user or tenant has a home region that accepts its writes, as in the figure) and **active-active with conflict resolution** (multi-leader or CRDT-based, covered in the replication chapter).

Data residency rules (for example requirements that personal data of residents of a jurisdiction is stored in that jurisdiction) turn the home-region assignment from an optimisation into a constraint: the region of a user is chosen at registration, the user record and derived data are written only there, and cross-region calls carry identifiers rather than personal data.

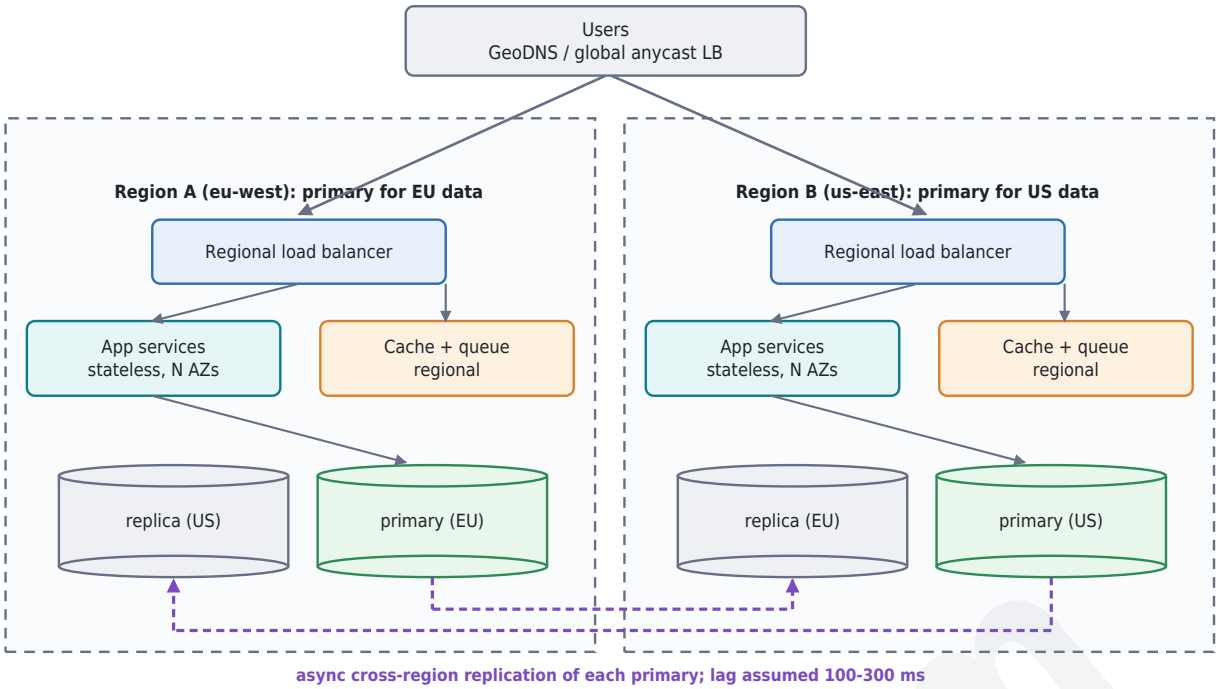


Figure 4.6. Each region is the primary for its own users and a replica for the other region; asynchronous replication means that a regional failure loses at most the replication lag of writes (RPO), and failover must promote the replica.

Disaster recovery is defined by two numbers. The **RPO** (recovery point objective) is the maximum acceptable data loss, measured in time. The **RTO** (recovery time objective) is the maximum acceptable time to restore service. Lower values require more standing infrastructure and therefore more cost, which gives the familiar ladder of strategies. A strategy is credible only when it is exercised: a restore that has never been performed has an unknown RTO. The GitHub incident in the failure case studies shows a recovery that took hours because multi-terabyte restores had never been rehearsed end to end.

Strategy	What runs in the standby region	RPO / RTO (typical)	Relative cost
Backup and restore	nothing; backups copied to a second region	hours / hours	
Pilot light	data replicated; app stack stopped	minutes / tens of min	
Warm standby	scaled-down full stack, live	seconds-min / minutes	
Active-active	full stack serves live traffic	near 0 / near 0	

Values are typical orders of magnitude for planning, not guarantees; each design must be tested.

Figure 4.7. The four disaster recovery strategies trade standing cost for lower RPO and RTO; the figures are typical orders of magnitude and each organisation must measure its own.

Cost awareness and chaos engineering

Cost is a design constraint. The largest items are usually compute (right-size, use autoscaling and committed-use discounts for the stable baseline), storage (tiering, retention limits, compression), cross-zone and cross-region data transfer (often billed per gigabyte and easily the dominant cost of a chatty microservice design or of synchronous multi-region replication), and observability itself (log volume and metric cardinality). A design statement such as "each order produces 12 KB of logs and 5 million orders per day gives 60 GB per day" is the kind of estimate expected of a candidate.

Chaos engineering is the practice of injecting failures deliberately, under controlled conditions, to verify that the system behaves as designed: terminate instances, add latency to a dependency, drop an availability zone, expire a

certificate. A chaos experiment states a hypothesis ("order success rate stays above 99.5 percent when one zone is removed"), limits the blast radius, defines an abort condition and is run first in staging and then in production. Game days, in which a team rehearses a failure and its runbook, serve the same purpose for people.

4.1.2 Worked example

The worked example sizes a stateless order API. Assumptions (all assumed): peak load of 7,000 requests per second, one instance sustains 300 requests per second at saturation in a load test, the service must run at no more than 60 percent utilisation at peak in normal operation, and the loss of one of three availability zones must keep the remaining instances at or below 80 percent utilisation.

Constraint	Arithmetic	Instances
Normal operation, 60 percent target	7,000 / (300 × 0.60) = 38.9	39
Survive loss of one of three zones, 80 percent limit	7,000 / (300 × 0.80) = 29.2 on two zones, so 15 per zone	45 in three zones
Chosen	maximum of the two constraints, 15 per zone	45
Utilisation, all zones healthy	7,000 / (45 × 300)	51.9 percent
Utilisation after losing a zone	7,000 / (30 × 300)	77.8 percent
Requests in flight (Little's law)	7,000 × 0.120 s	840

Capacity calculation for 7,000 requests per second at peak.

The zone-loss constraint, not the normal target, determines the size, and the result costs 15 percent more than the 39 instances that the normal target alone suggests. That premium is the price of surviving a zone failure without an incident. A reader who states only "load divided by instance capacity" has omitted the failure domain, which is the most common omission in capacity answers.

The error budget arithmetic for the same service at an assumed steady 2,000 requests per second and a 99.9 percent SLO is: 2,000 × 86,400 × 30 = 5,184,000,000 requests per 30 days, so the budget is 5,184,000 failed requests, which equals 43.2 minutes of total outage.

4.1.3 Hands-on

The first program computes the budget, replays a simulated day of per-minute error ratios and evaluates the two page conditions from the table above. The incident is assumed: 5 percent errors from minute 600 to minute 630 on top of a background of 0.03 percent.

SloBurn.java JAVA

```
public class SloBurn {
    static final double SLO = 0.999;
    static final double BUDGET = 1 - SLO;          // 0.1 percent of requests may fail
    static final int RPS = 2000;                   // assumed constant request rate
    static final int WINDOW_DAYS = 30;

    // error ratio per minute for a 24-hour day (assumed incident from minute 600 to 630)
    static double errorRatio(int minute) {
        if (minute >= 600 && minute < 630) return 0.05;   // 5 percent errors
        return 0.0003;                                    // background 0.03 percent
    }

    static double burn(double[] r, int endMinute, int windowMin) {
        double sum = 0;
        int from = Math.max(0, endMinute - windowMin + 1);
        for (int m = from; m <= endMinute; m++) sum += r[m];
        return (sum / (endMinute - from + 1)) / BUDGET;
    }

    public static void main(String[] args) {
        long perMonth = (long) RPS * 86400L * WINDOW_DAYS;
        System.out.printf("requests per 30 days   : %,d%n", perMonth);
        System.out.printf("error budget (requests): %,.0f%n", perMonth * BUDGET);
        System.out.printf("error budget (minutes) : %.1f%n", WINDOW_DAYS * 24 * 60 * BUDGET);
        double[] r = new double[1440];
        for (int m = 0; m < 1440; m++) r[m] = errorRatio(m);
        boolean fast = false, slow = false;
        double consumed = 0;
```

SloBurn.java (continued) JAVA

```java
        for (int m = 0; m < 1440; m++) {
            consumed += r[m] * RPS * 60;
            double b1h = burn(r, m, 60), b5m = burn(r, m, 5);
            double b6h = burn(r, m, 360), b30m = burn(r, m, 30);
            if (!fast && b1h >= 14.4 && b5m >= 14.4) {
                fast = true;
                System.out.printf("PAGE 14.4x at minute %d: burn(1h)=%.1f burn(5m)=%.1f%n",
                    m, b1h, b5m);
            }
            if (!slow && b6h >= 6 && b30m >= 6) {
                slow = true;
                System.out.printf("PAGE 6x    at minute %d: burn(6h)=%.1f burn(30m)=%.1f%n",
                    m, b6h, b30m);
            }
        }
        System.out.printf("budget consumed that day: %.1f percent of the 30-day budget%n",
            100 * consumed / (perMonth * BUDGET));
        double h = 0.05 / BUDGET;
        System.out.printf("burn rate during incident: %.0f -> budget gone in %.1f hours%n",
            h, WINDOW_DAYS * 24 / h);
    }
}
```

Run of SloBurn TERMINAL

```
$ java SloBurn.java
requests per 30 days   : 5,184,000,000
error budget (requests): 5,184,000
error budget (minutes) : 43.2
PAGE 14.4x at minute 617: burn(1h)=15.2 burn(5m)=50.0
budget consumed that day: 4.5 percent of the 30-day budget
burn rate during incident: 50 -> budget gone in 14.4 hours
```

What to notice: the page fires at minute 617, 17 minutes after the incident began, because the 1-hour window needs that long to accumulate a burn rate of 14.4 while the 5-minute window confirms the problem is ongoing. The slow-burn (6x over 6 hours) condition never fires, since the 30 minutes of incident average 4.2 over six hours. The incident consumed 4.5 percent of the monthly budget, and at that rate the entire budget would have been gone in 14.4 hours. A tighter alert would page earlier at the price of more false pages; that trade-off is the reason for the two windows.

The second program is a simulation of the autoscaler in the figure. One instance serves 100 requests per second, the target utilisation is 60 percent, a new instance needs 3 minutes to become ready and the observed utilisation is capped at 100 percent, as CPU is.

autoscale.py PYTHON

```python
import math
CAP=100; TARGET=0.6; WARM=3   # rps per instance, target utilisation, warm-up minutes
load=[400]*5+[1200]*15       # per-minute rps (assumed spike)
def run(name, scheduled=False):
    ready=[7]*5  # instances serving (400/100/0.6 ~ 7)
    pending=[]   # (ready_at, n)
    cur=7; served=0; dropped=0; rows=[]
    for t,l in enumerate(load):
        cur+=sum(n for (at,n) in pending if at==t); pending=[p for p in pending if p[0]!=t]
        inflight=sum(n for _,n in pending)
        util=min(l/(cur*CAP),1.0)  # CPU cannot exceed 100 percent
        s=min(l,cur*CAP); served+=s; dropped+=l-s
        want=math.ceil(cur*util/TARGET) if util>TARGET else cur+inflight
        if scheduled and t==3: want=math.ceil(1200/CAP/TARGET)
        if want>cur+inflight: pending.append((t+WARM,want-cur-inflight))
        rows.append((t,l,cur,util,l-s))
    print(name, "dropped rps-minutes:",int(dropped),"of",sum(load))
    for r in rows[3:14]: print("  min %2d load %4d ready %2d util %.2f dropped %4d"%r)
run("reactive target tracking (target 60%, warm-up 3 min)")
run("scheduled pre-scale at minute 3", True)
```

Run of autoscale.py (simulation) TERMINAL

```
$ python3 autoscale.py
reactive target tracking (target 60%, warm-up 3 min) dropped rps-minutes: 1500 of 20000
  min  3 load  400 ready  7 util 0.57 dropped    0
  min  4 load  400 ready  7 util 0.57 dropped    0
  min  5 load 1200 ready  7 util 1.00 dropped  500
```

Run of autoscale.py (simulation) (continued) TERMINAL

```
  min  6 load 1200 ready  7 util 1.00 dropped  500
  min  7 load 1200 ready  7 util 1.00 dropped  500
  min  8 load 1200 ready 12 util 1.00 dropped    0
  min  9 load 1200 ready 12 util 1.00 dropped    0
  min 10 load 1200 ready 12 util 1.00 dropped    0
  min 11 load 1200 ready 20 util 0.60 dropped    0
  min 12 load 1200 ready 20 util 0.60 dropped    0
  min 13 load 1200 ready 20 util 0.60 dropped    0
scheduled pre-scale at minute 3 dropped rps-minutes: 500 of 20000
  min  3 load  400 ready  7 util 0.57 dropped    0
  min  4 load  400 ready  7 util 0.57 dropped    0
  min  5 load 1200 ready  7 util 1.00 dropped  500
  min  6 load 1200 ready 20 util 0.60 dropped    0
  min  7 load 1200 ready 20 util 0.60 dropped    0
  min  8 load 1200 ready 20 util 0.60 dropped    0
  min  9 load 1200 ready 20 util 0.60 dropped    0
  min 10 load 1200 ready 20 util 0.60 dropped    0
  min 11 load 1200 ready 20 util 0.60 dropped    0
  min 12 load 1200 ready 20 util 0.60 dropped    0
  min 13 load 1200 ready 20 util 0.60 dropped    0
```

The reactive policy drops 1,500 of the 20,000 offered units (requests per second, summed over the minutes), and the scheduled pre-scale drops 500. The reactive run also shows the saturation effect: at minute 5 it computes 7 × 1.0 / 0.6, so it requests only 5 more instances although 13 more are needed, and the second correction arrives three minutes later. Real autoscalers add stabilisation windows and rate limits that change the numbers; the shape of the problem is the same.

The third program implements deterministic percentage rollout with a SHA-256 hash and verifies the two properties that matter: the enabled share matches the target, and the 5 percent cohort remains enabled at 25 percent.

Flags.java JAVA

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

public class Flags {
    static int bucket(String flag, String user) throws Exception {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] h = md.digest((flag + ":" + user).getBytes(StandardCharsets.UTF_8));
        int v = ((h[0] & 0xff) << 24) | ((h[1] & 0xff) << 16)
              | ((h[2] & 0xff) << 8) | (h[3] & 0xff);
        return (int) ((v & 0xffffffffL) % 100);
    }
    public static void main(String[] a) throws Exception {
        int users = 100_000;
        for (int pct : new int[] {1, 5, 25, 50}) {
            int on = 0;
            for (int u = 0; u < users; u++) if (bucket("new-checkout", "user-" + u) < pct) on++;
            System.out.printf("rollout %2d%% -> %6d of %d users enabled (%.2f%%)%n",
                pct, on, users, 100.0 * on / users);
        }
        int stay = 0, was = 0;
        for (int u = 0; u < users; u++) {
            boolean at5 = bucket("new-checkout", "user-" + u) < 5;
            boolean at25 = bucket("new-checkout", "user-" + u) < 25;
            if (at5) { was++; if (at25) stay++; }
        }
        System.out.printf("users enabled at 5%%: %d, still enabled at 25%%: %d%n", was, stay);
        System.out.println("user-42 buckets: new-checkout=" + bucket("new-checkout", "user-42")
            + " dark-mode=" + bucket("dark-mode", "user-42"));
    }
}
```

Run of Flags TERMINAL

```
$ java Flags.java
rollout  1% ->    982 of 100000 users enabled (0.98%)
rollout  5% ->   4928 of 100000 users enabled (4.93%)
rollout 25% ->  25025 of 100000 users enabled (25.03%)
rollout 50% ->  49928 of 100000 users enabled (49.93%)
users enabled at 5%: 4928, still enabled at 25%: 4928
user-42 buckets: new-checkout=0 dark-mode=55
```

4.1.4 Real-world problems and solutions

Alert fatigue from cause-based alerts

Scenario: a team pages on CPU above 80 percent, disk above 70 percent and any 5xx response, and receives 40 pages per week.

Symptoms: on-call engineers acknowledge pages without investigating; a real outage page is delayed because it is indistinguishable from the others.

Diagnosis: classify the last 100 pages by whether a human action was required; most require none.

Root cause: the alerts measure internal causes and not user-visible symptoms, and the thresholds are static.

Fix: replace them with SLO burn-rate alerts on the user journeys (checkout success, p99 latency), keep cause metrics on dashboards for diagnosis, and route non-urgent conditions to tickets.

Prevention: every page must have a runbook and an owner, a page that required no action is reviewed within a week, and the quarterly review counts pages per on-call shift.

Autoscaler too slow for a flash spike

Scenario: a push notification brings 3 times normal traffic within a minute.

Symptoms: latency and errors rise for about six minutes, then recover on their own; the instance count graph climbs in steps.

Diagnosis: compare the timestamp of the spike with the timestamps of instance readiness; the gap equals aggregation delay plus provisioning plus warm-up, and CPU shows a flat 100 percent.

Root cause: reactive scaling on a saturating metric with a multi-minute provisioning delay.

Fix: pre-scale before known events (scheduled scaling driven by the marketing calendar), keep a warm pool of pre-initialised instances, scale on a leading indicator such as request rate or queue depth, and shed low-priority load while capacity is added.

Prevention: load-test the autoscaling policy itself with a step increase, and record the time from breach to ready capacity as a service metric.

Rolling deployment breaks on a schema change

Scenario: version 2 renames the column `phone` to `mobile` and the migration runs before the rolling deployment.

Symptoms: during the rollout, instances still running version 1 return errors on every query that touches the column; rolling back does not help because the column is already renamed.

Diagnosis: errors correlate with the version label of the failing instances; the migration timestamp precedes the first error.

Root cause: a destructive change was made while two application versions were live.

Fix: restore compatibility by adding the old column back (or a view), then use expand and contract in separate releases.

Prevention: a migration review rule that allows only additive changes in the same release as code that needs them, and a staging test that runs the previous release against the new schema.

Metric cardinality explosion

Scenario: a developer adds the label `user_id` to a request counter.

Symptoms: the metrics store memory grows, queries time out, the monitoring bill increases several times, and finally ingestion is rejected.

Diagnosis: query the number of active series per metric name and sort descending; the new metric holds millions of series.

Root cause: each distinct label combination is a separate time series, so an unbounded label multiplies the series count by the number of users.

Fix: remove the label from metrics; record user-level detail in logs or traces, where it is searchable per event.

Prevention: enforce a series limit per metric in the collector, and review labels in code review: only bounded sets (status class, endpoint template, region) are acceptable.

sheeran

4.1.5 Questions to expect

What is the difference between monitoring and observability?

Monitoring checks known failure conditions with predefined metrics and alerts. Observability is the property that arbitrary new questions about internal state can be answered from the emitted signals (high-cardinality logs and traces) without deploying new code. A system needs monitoring for the known problems and observability for the unknown ones.

Explain SLI, SLO, SLA and error budget.

An SLI is a measured indicator (proportion of requests served successfully under 300 ms). An SLO is the target for it (99.9 percent over 30 days). An SLA is a contract with consequences, usually looser than the SLO. The error budget is one minus the SLO; when it is consumed, feature releases pause in favour of reliability work.

Why use two windows in a burn-rate alert?

The long window shows that significant budget has been consumed, which removes alerts caused by short blips. The short window shows that the error rate is still elevated, which makes the alert reset quickly after recovery. Requiring both keeps the page precise and timely.

How is the number of instances determined for a service?

Peak rate divided by the load-tested capacity of one instance at a target utilisation, then increased so that the remaining instances stay within a limit after losing the largest failure domain (one zone). Little's law sizes pools. The result is validated by a load test and adjusted with autoscaling for the variable part.

Compare blue-green and canary releases.

Blue-green switches all traffic at once between two full environments, which gives instant rollback but costs double capacity and exposes all users to a defect simultaneously. Canary exposes a small share first and widens on passing metric gates, which limits the blast radius but needs automated comparison and enough traffic in the canary to be statistically meaningful.

How is a database migration made safe with rolling deployments?

By expand and contract: additive schema changes first, dual-write code, backfill, switch reads, then remove the old structure in a later release. Each step is compatible with the previous and next application version, so a rollback is always possible.

What are RPO and RTO, and how do they drive the design?

RPO is tolerable data loss in time and determines the replication method (synchronous replication or frequent log shipping for near-zero RPO, nightly backups for hours). RTO is tolerable downtime and determines the standby model (restore from backup, pilot light, warm standby, active-active). Both are justified by business impact and verified by drills.

What would be done to find the cause of a latency increase?

Check the golden signals on the affected endpoint, narrow by dimension (region, version, instance), open traces of slow requests to find the slow span, then check the resource metrics (USE) of that dependency, and compare with recent deployments and flag changes, because changes cause most incidents.

KEY POINTS

Observability needs metrics for alerting, logs for detail and traces for the path of one request; a shared trace identifier links them.

- Alert on SLO burn rate with a long and a short window, not on resource causes.
- Capacity equals peak load over tested instance capacity at a target utilisation, enlarged to survive the loss of one failure domain.
- Reactive autoscaling lags and saturated metrics hide overload; pre-scale for known peaks.
- Release with canary or blue-green, keep schema changes expand-and-contract, and separate deploy from release with flags.
- RPO and RTO are measured, not declared; an untested recovery plan has an unknown RTO.

4.2 Security in System Design

Security appears in design interviews as a set of follow-up questions: how are users authenticated, how is access to another user's data prevented, how are abusive clients stopped, where are secrets kept. A design that answers them with named mechanisms and correct placement (which component enforces what) is considered complete. This chapter gives the placement and the reasoning; the Spring Security mechanics (filter chain, method security, OAuth2 resource server configuration) are covered in the Spring Boot guides and the networking and cryptography fundamentals in the Computer Networks material, and are used here by reference.

4.2.1 Concept and mechanism

Authentication and authorisation

Authentication establishes who the caller is; **authorisation** decides what that caller may do. The two are enforced at different places. Authentication is centralised at the edge (API gateway) so that no service parses credentials differently. Authorisation is enforced in the service that owns the data, because only that service knows whether an order belongs to the caller. A gateway that performs coarse checks (valid token, required scope) and services that perform fine-grained checks (object ownership, tenant, role) form the standard split.

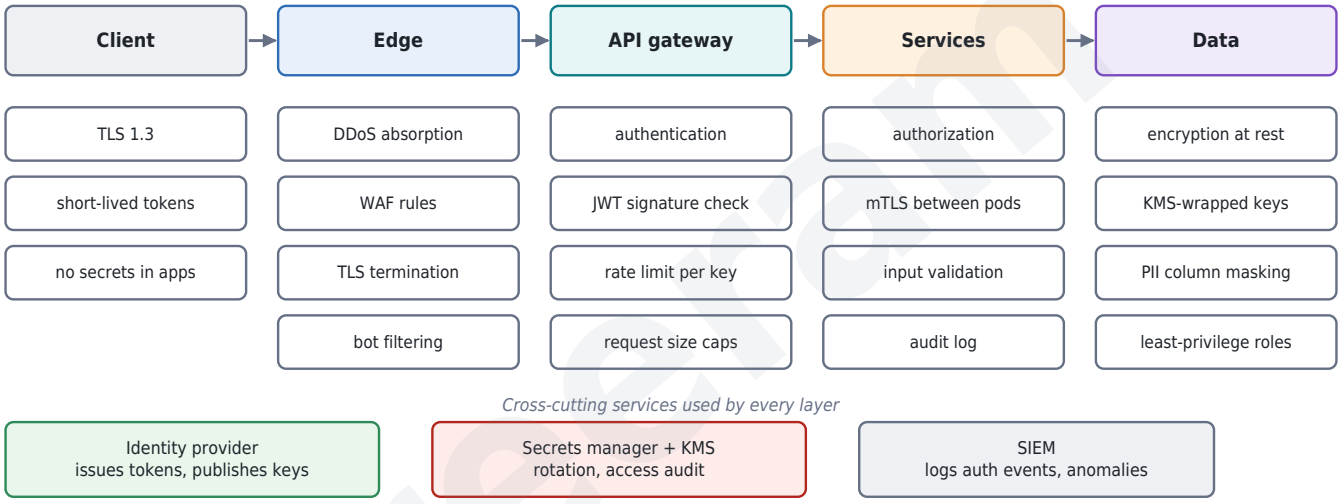


Figure 4.8. Security controls are layered: each layer assumes that the one before it can fail, and cross-cutting services (identity, secrets, audit) support every layer.

Token design

An **opaque token** is a random identifier that the resource server must look up at the authorisation server (token introspection) on each request or on a cached basis. A **JWT** (JSON Web Token) is a signed, self-contained token; the signature is verified locally with the issuer public key, which services obtain from a JWKS (JSON Web Key Set) endpoint and cache. JWTs remove a network call from every request, but a self-contained token cannot be revoked before it expires. The design consequences are the following:

- Access tokens are short-lived (5 to 15 minutes) so that revocation delay is bounded by the lifetime.
- A longer-lived **refresh token**, stored server-side with its own revocation record and rotated on each use, obtains new access tokens; reuse of an already-rotated refresh token signals theft and revokes the whole chain.
- The validator checks the algorithm against an allow-list (never trusting the `alg` field supplied by the token, which has enabled the "none" algorithm attack), the signature, `iss`, `aud`, `exp` and `nbf`.
- Claims are minimal (subject, scopes, tenant). Personal data does not belong in a token that every service and log can read; the payload is encoded, not encrypted.
- Where immediate revocation is required (a compromised account), a small deny-list of token identifiers or a user-level "not before" timestamp is checked at the gateway.

For third-party and mobile clients the standard flow is OAuth 2.0 authorization code with PKCE, with OpenID Connect adding an identity token. PKCE binds the authorisation code to the client that requested it through a one-time `code_verifier`, which prevents interception of the code on mobile and single-page clients.

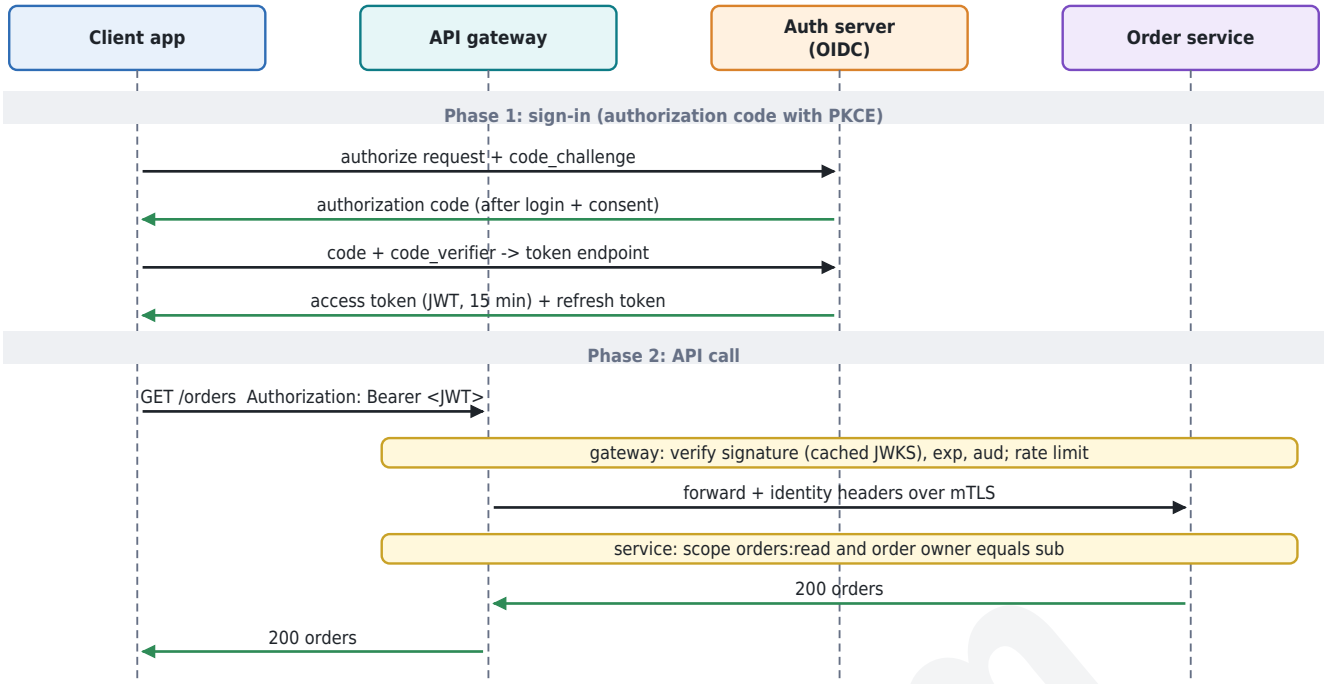


Figure 4.9. Authentication happens once against the authorisation server; every later API call carries only the short-lived token, which the gateway verifies locally and the service then authorises against the resource.

Abuse prevention, secrets, encryption and personal data

Rate limiting and abuse prevention. Limits are applied at several keys: per IP address (coarse, defeated by shared NAT and botnets), per API key or user (accurate), and per endpoint (login and password reset have far lower limits than reads). Login protection combines per-account throttling, progressive delay, detection of credential stuffing (many accounts from one source, or one password across many accounts), CAPTCHA escalation and breached-password checks. The algorithms and the distributed implementation are covered in the rate limiting chapters.

Secrets (database passwords, API keys, signing keys) are never stored in source control, container images or plain environment files. They live in a secrets manager, are injected at start or fetched at runtime with a workload identity, are rotated on a schedule and on suspicion, and every access is audited. Short-lived, automatically issued credentials (for example per-session database credentials) reduce the value of a leak.

Encryption in transit uses TLS 1.2 or later at the edge and between services; mutual TLS (mTLS) authenticates service to service and is commonly provided by a service mesh. **Encryption at rest** uses envelope encryption: a data key encrypts the data, and a key-management service (KMS) master key encrypts the data key, so that rotating the master key re-wraps small keys and does not re-encrypt terabytes, and key use is logged.

Personal data (PII). Collect the minimum, classify fields, mask them in logs and non-production copies, tokenise payment data so that card numbers stay inside a certified provider, restrict access by role with an audit trail, set retention limits and support deletion requests, which requires that the design knows every place a user identifier is stored (backups, search indexes, analytics copies and caches included).

The most common authorisation defect in APIs is **broken object-level authorisation** (listed first in the OWASP API Security Top 10): the service checks that the caller is authenticated but not that the requested object belongs to the caller, so changing an identifier in the URL returns another user's data.

4.2.2 Worked example

Securing `GET /orders/{id}` in the food-delivery design requires six checks, in this order and at these places:

Step	Check	Mechanism	Enforced at
1	Transport is encrypted	TLS 1.3 with HSTS	Edge
2	Caller is not flooding	60 requests per minute per user, 1,000 per key	Gateway
3	Token is genuine and current	Signature against cached JWKS, `exp`, `aud`, `iss`	Gateway
4	Caller may call this operation	Scope `orders:read` present	Gateway

Step	Check	Mechanism	Enforced at
5	Caller may see this object	`order.user_id == token.sub`, otherwise respond 404	Order service
6	Result is safe to log and return	Mask phone and address in logs, return only needed fields	Order service

Checks for one authenticated read.

Step 5 returns 404 and not 403 for an object owned by another user, so that identifiers cannot be enumerated to learn which exist. Steps 3 and 5 are the two that implementations most often omit: signature validation is skipped in favour of decoding, and ownership is assumed from the authentication.

4.2.3 Hands-on

The program issues an HS256 token, verifies it, and then attempts three attacks: using it after expiry, changing the scope in the payload without re-signing, and presenting an unsigned token with `alg` set to `none`. The secret in the listing is a demonstration value. Production services should prefer an asymmetric algorithm (RS256 or ES256) so that verifiers hold only the public key.

Jwt.java JAVA

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.Base64;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

public class Jwt {
    static final Base64.Encoder ENC = Base64.getUrlEncoder().withoutPadding();
    static final byte[] KEY =
        "demo-secret-key-do-not-use-in-production!".getBytes(StandardCharsets.UTF_8);

    static String b64(String s) { return ENC.encodeToString(s.getBytes(StandardCharsets.UTF_8)); }
    static String sign(String data) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(KEY, "HmacSHA256"));
        return ENC.encodeToString(mac.doFinal(data.getBytes(StandardCharsets.UTF_8)));
    }
    static String issue(String sub, String scope, long exp) throws Exception {
        String head = b64("{\"alg\":\"HS256\",\"typ\":\"JWT\"}");
        String body = b64("{\"sub\":\"" + sub + "\",\"scope\":\"" + scope
            + "\",\"exp\":" + exp + "}");
        return head + "." + body + "." + sign(head + "." + body);
    }
    static String verify(String token, long now) throws Exception {
        String[] p = token.split("\\.");
        if (p.length != 3) return "REJECT malformed";
        String head = new String(Base64.getUrlDecoder().decode(p[0]), StandardCharsets.UTF_8);
        if (!head.contains("\"alg\":\"HS256\"")) return "REJECT algorithm not allowed";
        boolean ok = MessageDigest.isEqual(sign(p[0] + "." + p[1]).getBytes(StandardCharsets.UTF_8),
                                           p[2].getBytes(StandardCharsets.UTF_8));
        if (!ok) return "REJECT bad signature";
        String body = new String(Base64.getUrlDecoder().decode(p[1]), StandardCharsets.UTF_8);
        long exp = Long.parseLong(body.replaceAll(".*\"exp\":(\\d+).*", "$1"));
        if (now >= exp) return "REJECT expired";
        return "ACCEPT " + body;
    }
    public static void main(String[] args) throws Exception {
        long now = 1_700_000_000L;
        String t = issue("user-42", "orders:read", now + 900);
        System.out.println("token parts: " + t.split("\\.").length + ", length " + t.length());
        System.out.println("valid          : " + verify(t, now));
        System.out.println("after expiry   : " + verify(t, now + 901));
        String[] p = t.split("\\.");
        String forged = p[0] + "." + b64("{\"sub\":\"user-42\",\"scope\":\"admin\",\"exp\":"
            + (now + 900) + "}") + "." + p[2];
        System.out.println("tampered scope : " + verify(forged, now));
        String none = b64("{\"alg\":\"none\",\"typ\":\"JWT\"}") + "." + p[1] + ".";
        System.out.println("alg none       : " + verify(none + "x", now));
    }
}
```

Run of Jwt TERMINAL

```
$ java Jwt.java
token parts: 3, length 156
valid          : ACCEPT {"sub":"user-42","scope":"orders:read","exp":1700000900}
after expiry   : REJECT expired
tampered scope : REJECT bad signature
alg none       : REJECT algorithm not allowed
```

What to notice: the tampered token is rejected because the signature no longer matches the signed bytes; the unsigned token is rejected because the algorithm is checked against an allow-list before any other processing; the comparison of signatures uses `MessageDigest.isEqual`, which takes constant time and avoids a timing side channel. The listing parses the payload with a regular expression for brevity; a production service uses a JSON library and a maintained JWT library and never writes the validation itself.

4.2.4 Real-world problems and solutions

Leaked long-lived token

Scenario: a mobile client stores a token that is valid for 30 days; a user reports a lost phone.

Symptoms: the token continues to work from a different device and country.

Diagnosis: audit logs show requests with the token from an unexpected location after the report.

Root cause: a self-contained long-lived token with no revocation path.

Fix: add a per-user "tokens valid after" timestamp checked at the gateway, and revoke the refresh-token chain.

Prevention: 15-minute access tokens, refresh-token rotation with reuse detection, device binding where available, and a revocation check that does not need a database read on the hot path (a cached deny-list).

Broken object-level authorisation

Scenario: `GET /orders/1001` returns an order that belongs to a different customer when the caller requests it with a valid token.

Symptoms: no errors; the defect is found by a penetration test or a customer report.

Diagnosis: replay a request with the token of user A against identifiers created by user B.

Root cause: the service loads the order by identifier only.

Fix: load by identifier and owner (`findByIdAndUserId`), or check ownership before returning, and respond 404.

Prevention: a shared authorisation helper used by every handler, automated tests with two users for each endpoint, and unguessable identifiers as a second line (never as the only one).

Secret committed to a repository

Scenario: a cloud access key is committed in a configuration file and the repository is public for 20 minutes.

Symptoms: unexpected resource creation and a billing alert, or a notice from the cloud provider.

Diagnosis: the access log of the key shows calls from unknown addresses; the repository history contains the key.

Root cause: secrets handled as files in the repository, and no scanning.

Fix: revoke and rotate the key immediately (removing it from history does not unleak it), review all activity of the key, and remove the created resources.

Prevention: a pre-commit and CI secret scanner, a secrets manager with runtime injection, and short-lived credentials from workload identity so that no long-lived key exists.

Credential stuffing against the login endpoint

Scenario: logins per second rise tenfold, with a low success rate spread over hundreds of thousands of accounts.

Symptoms: the authentication database saturates, genuine users time out, and some accounts are taken over.

Diagnosis: the failure rate per account is low (one or two attempts), so per-account lockout does not trigger; the source addresses are numerous and distributed.

Root cause: attackers replay credentials leaked from other sites, and the limits are keyed only on account.

Sheeran

Fix: add limits by source reputation and device fingerprint, introduce challenges for suspicious traffic, require a second factor for risky logins, and check passwords against known-breached lists.

Prevention: multi-factor authentication, anomaly detection on login failure ratio, a separate small pool for authentication so that an attack cannot take the rest of the system down.

4.2.5 Questions to expect

Where should authentication and authorisation be enforced?

Authentication at the gateway so that it is uniform; coarse authorisation (scope) at the gateway; fine-grained authorisation (ownership, tenant, role) in the service that owns the data. Services still verify the identity propagated to them, because a defence that exists only at the edge fails when one internal service is compromised.

JWT or session cookie with a server-side store?

A server-side session is simple to revoke and keeps data on the server, at the cost of a lookup per request. JWTs avoid the lookup and suit multi-service and mobile use, but cannot be revoked early, so they need short lifetimes and refresh rotation. A browser application with one backend is often better served by a secure, HTTP-only cookie session.

How are tokens revoked if they are self-contained?

By keeping lifetimes short, rotating refresh tokens, and checking a small deny-list or a per-user "valid after" timestamp at the gateway. The deny-list holds only identifiers that have not yet expired, so it stays small.

What is the purpose of PKCE?

It prevents an attacker who intercepts the authorisation code from exchanging it for tokens, because the exchange also requires the `code_verifier` known only to the client that started the flow. It is required for public clients (mobile, single-page) and recommended for all.

What is envelope encryption?

Data is encrypted with a data key; the data key is encrypted with a master key held in a KMS. Only small keys are re-wrapped on master key rotation, the master key never leaves the KMS, and each unwrap is an auditable event.

How would an API be protected against abuse?

Authentication, per-key and per-endpoint rate limits with quotas, request size and timeout limits, schema validation, bot detection at the edge, idempotency keys for writes, and monitoring of anomalies per key. Heavy endpoints (search, export) get separate lower limits.

What is mTLS and why is it used between services?

Both client and server present certificates, so each side authenticates the other and traffic is encrypted. It prevents a process on the network from impersonating a service, and it is typically automated by a service mesh that issues and rotates short-lived certificates.

KEY POINTS

Authenticate once at the edge, authorise at the service that owns the data, and treat object-level checks as mandatory.

- JWTs avoid lookups but cannot be revoked early: use short lifetimes, rotated refresh tokens and a small deny-list.
- Validate algorithm, signature, issuer, audience and expiry; never trust the header to choose the algorithm.
- Keep secrets in a secrets manager with rotation and audit; prefer short-lived credentials.
- Use TLS everywhere, mTLS between services and envelope encryption at rest; minimise, mask and expire personal data.

sreram

4.3 Architecture Styles and Evolution

The choice between a monolith and microservices is one of the most frequent topics in design interviews, and it is frequently answered with a preference and not with criteria. The correct answer states the forces (team size, rate of change, scaling profile, operational maturity, domain clarity) and the consequences of each style. This chapter compares the styles, describes the migration pattern that avoids a rewrite, and covers the Backend for Frontend pattern and the service mesh by reference.

4.3.1 Concept and mechanism

The styles

A **monolith** is one deployable unit with one database. It offers simple local development, in-process calls, ACID transactions across the whole domain and one thing to deploy and observe. Its problems emerge with size: long build and test cycles, a release that must include everything, a defect in one area that affects all, and coupling that grows because nothing prevents any class from calling any other.

A **modular monolith** keeps one deployable unit but enforces boundaries between modules: each module exposes a small public interface, hides its internals and owns its tables (separate schemas), and cross-module access goes through the interface or through events. It captures most of the organisational benefit of services at a fraction of the operational cost, and it prepares a later extraction because the seams already exist. In Java, module boundaries can be enforced with package-private visibility, separate build modules, the Java module system or architecture tests.

Microservices are independently deployable services, each owning its data, organised around business capabilities. They allow independent scaling, release cadence and technology, and they match team ownership. They convert in-process calls into network calls, with the failure modes of the earlier parts of this book: partial failure, latency, retries, inconsistent data across services (sagas instead of transactions) and the need for service discovery, tracing and contract testing. A system of microservices that must be deployed together, or that share a database, is a **distributed monolith**: it has the costs of both styles and the benefits of neither.

Event-driven architecture connects components through published events (a log or a broker) rather than direct calls. It decouples producers from consumers in time and knowledge and supports many consumers of one fact, but it makes the control flow implicit and harder to trace, introduces eventual consistency and requires care with ordering, duplicates and schema evolution. **Serverless** (functions and managed services billed per use) removes server management and scales to zero, which suits spiky, event-triggered workloads; its limits are cold-start latency, execution time limits, difficult local testing, vendor-specific services and cost that exceeds provisioned capacity at steady high volume.

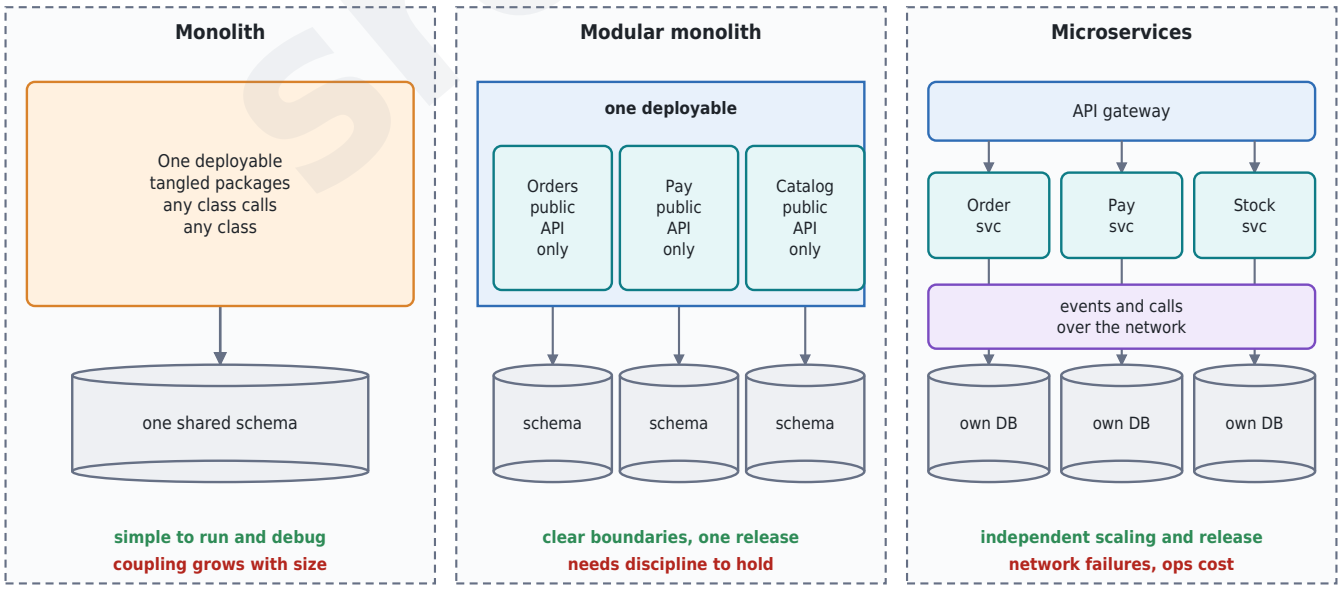


Figure 4.10. The three styles differ in where the boundaries are enforced: nowhere, at module level inside one deployment, or at the network between independently deployed services with their own data.

Force	Monolith	Modular monolith	Microservices	Serverless
Team size	up to about 10	10 to 50	many autonomous teams	any, per function

Force	Monolith	Modular monolith	Microservices	Serverless
Operational cost	lowest	low	high	low to medium
Independent scaling	none	none	per service	automatic
Consistency	ACID everywhere	ACID, per-module ownership	eventual, sagas	eventual
Failure modes	all or nothing	all or nothing	partial failure	cold starts, limits
Best fit	new product, unclear domain	growing product, clear modules	large organisation, stable boundaries	spiky event workloads

Choosing an architecture style by force.

The numbers in the team size row are rules of thumb and not measurements; the underlying criterion is whether teams block each other on releases.

Strangler migration

Rewriting a working system is high-risk: the rewrite must match years of behaviour and the business cannot wait. The **strangler fig** pattern replaces the system incrementally. A facade (reverse proxy or gateway) is placed in front of the legacy system without changing behaviour. One capability at a time is rebuilt as a new service, and the facade routes that capability to it, first for a small percentage of traffic, with shadow comparison of the responses, then fully. Data is synchronised with change data capture or dual writes during the transition, and when the last capability has moved, the legacy system is removed. Each step is reversible by changing a route, which is the property that makes the approach safe.

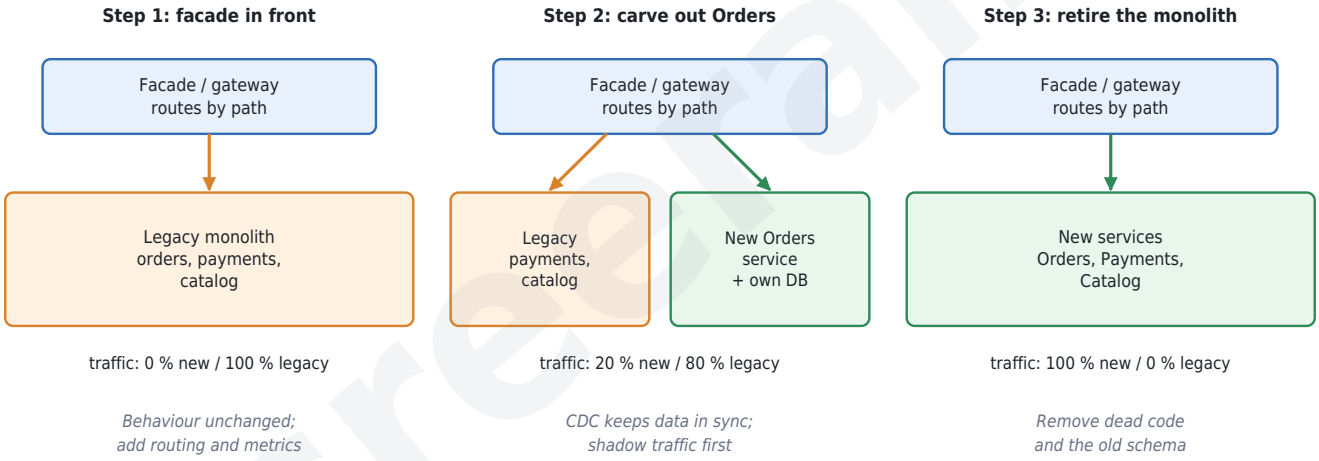


Figure 4.11. The facade makes migration a sequence of routing changes: behaviour is unchanged in step 1, one capability moves in step 2, and the legacy system disappears only when no route points at it.

Order of extraction matters: capabilities with clear boundaries, high change rate or distinct scaling needs go first; the capabilities in the tangled core, with many incoming dependencies, go last or remain in the monolith permanently if the cost exceeds the benefit.

Backend for Frontend and service mesh

A **Backend for Frontend (BFF)** is a server-side adapter dedicated to one client type. A mobile application needs small payloads and few round trips, a web application needs different aggregates, and a partner API needs stable versioned contracts. A generic API serves none of them well; one BFF per client, owned by the client team, composes calls to the domain services, shapes the responses and caches. The risk is duplicated logic: business rules stay in the domain services, and the BFF contains presentation concerns only.

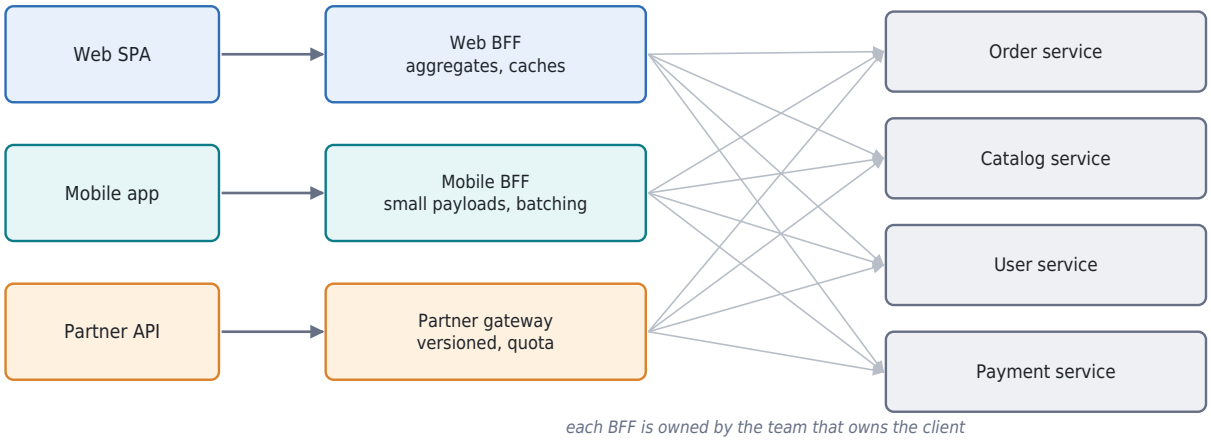


Figure 4.12. One BFF per client type shapes responses for that client, while the domain services stay unaware of client differences.

A **service mesh** moves cross-cutting network concerns out of application code into a proxy deployed beside each service (sidecar) or a node-level component: mutual TLS, retries and timeouts, traffic splitting for canaries, and uniform metrics and traces. The benefit is consistency across languages and teams; the costs are additional latency per hop, resource overhead, operational complexity and one more system that can fail. A mesh is justified when many services in several languages need uniform policy; a handful of services use a client library. The configuration details belong to the specific mesh product and are outside this book.

4.3.2 Worked example

The reason to be careful with service count is arithmetic. A request that synchronously calls a chain of services is available only when every service in the chain is available, so availability multiplies. A request that fans out to many services in parallel waits for the slowest, so the probability of meeting the tail increases quickly. The following program prints both effects; the values are computed, and the 99.9 percent per service and 1 percent tail are assumptions.

chain.py PYTHON

```python
# Availability of a call chain and tail amplification of a fan-out (arithmetic)
A = 0.999
print("services in chain | availability | downtime per 30 days")
for n in (1, 3, 5, 10, 20):
    a = A ** n
    print("%17d | %11.4f%% | %6.1f minutes" % (n, a * 100, (1 - a) * 30 * 24 * 60))
print()
print("parallel calls | P(at least one call slower than its own p99)")
for n in (1, 10, 50, 100):
    print("%14d | %5.1f%%" % (n, (1 - 0.99 ** n) * 100))
```

Run of chain.py (arithmetic) TERMINAL

```
$ python3 chain.py
services in chain | availability | downtime per 30 days
                1 |     99.9000% |   43.2 minutes
                3 |     99.7003% |  129.5 minutes
                5 |     99.5010% |  215.6 minutes
               10 |     99.0045% |  430.1 minutes
               20 |     98.0189% |  855.8 minutes

parallel calls | P(at least one call slower than its own p99)
             1 |   1.0%
            10 |   9.6%
            50 |  39.5%
           100 |  63.4%
```

Ten services at 99.9 percent each give 99.0 percent for a request that needs all of them, which is 430 minutes of downtime per 30 days against 43 minutes for one service. A page that calls 100 services in parallel hits at least one p99-slow call in 63 percent of loads. The consequences are design rules: keep synchronous chains short, make non-critical calls optional with a fallback (graceful degradation), cache, use asynchronous events for what need not be in the request path, and merge services that always change and fail together.

4.3.3 Real-world problems and solutions

Distributed monolith

Scenario: a company splits its application into 40 services over 18 months.

Symptoms: releases are coordinated across five teams, a change to a field breaks three other services, and a staging environment of all 40 services is needed to test anything.

Diagnosis: draw the call and dependency graph; count services that are always released together and services sharing tables.

Root cause: services were cut along technical layers or by entity and not along business capabilities, and shared libraries and a shared database couple them.

Fix: merge the services that always change together, give each remaining service its own data and replace synchronous chains with events or a single owner of the data.

Prevention: define boundaries by bounded contexts and team ownership, introduce consumer-driven contract tests, and measure the share of releases that needed a second service to change.

Premature microservices

Scenario: a team of six engineers builds a new product as eight services before the domain is understood.

Symptoms: most effort goes to infrastructure, deployment pipelines and cross-service debugging; a business change requires editing four repositories.

Diagnosis: compare time spent on feature work against time spent on platform and coordination.

Root cause: the service boundaries were guessed before the domain was known, and the operational cost exceeded the organisation maturity.

Fix: consolidate into a modular monolith with enforced module boundaries and extract a module into a service only when a concrete force (independent scaling, a separate team, a different release cadence) appears.

Prevention: start with a modular monolith and an explicit extraction criterion.

Strangler migration stalls

Scenario: a migration moved the easy 60 percent of the capabilities and stopped.

Symptoms: two systems must be maintained indefinitely, data is synchronised both ways and incidents involve both.

Diagnosis: the remaining capabilities are the tangled core, and no one owns the remaining work.

Root cause: the plan optimised for early visible progress and set no end state or deadline for the legacy system.

Fix: put the legacy decommission on the roadmap with an owner, tackle the core by first untangling it (anti-corruption layer, events), and track the share of traffic on the legacy path.

Prevention: every migration step must remove something, and the legacy cost is reported each quarter.

4.3.4 Questions to expect

When should a monolith be preferred over microservices?

When the team is small, the domain is still being discovered, and the organisation lacks the platform maturity (deployment automation, observability, on-call) that distributed systems need. A modular monolith defers the decision at low cost.

What are the disadvantages of microservices?

Network failures and latency replace function calls, transactions become sagas, data consistency is eventual, debugging requires distributed tracing, there is a fixed operational cost per service, and poorly chosen boundaries produce a distributed monolith.

How is a monolith migrated to microservices without a rewrite?

With the strangler pattern: a facade in front, one capability extracted at a time with its data, gradual traffic shift with shadow comparison, and removal of the legacy code at the end. Each step is reversible by changing routing.

sreeram

How should data be handled when a service is extracted?

The new service owns its tables. During transition, change data capture or dual writes keep the legacy data in step, reads move first, then writes, and finally the legacy tables are removed. Cross-service queries use API composition or a read model built from events.

What is a BFF and when is it needed?

A server-side backend dedicated to one client type that aggregates and shapes domain-service responses. It is needed when clients differ significantly in payload and interaction needs; it is not needed for a single client.

What does a service mesh provide and what does it cost?

Uniform mTLS, retries, timeouts, traffic splitting and telemetry without application changes. It costs extra latency and resources per hop and adds an operational system, so it fits organisations with many services in several languages.

How does event-driven architecture change testing and debugging?

Flows are implicit across producers and consumers, so tracing must propagate a correlation identifier through messages, schemas need compatibility rules and contract tests, and replaying events becomes a debugging and recovery tool.

KEY POINTS

Architecture style is chosen by forces (team size, change rate, scaling needs, operational maturity), not by fashion.

- A modular monolith gives enforced boundaries without the network cost and is the usual starting point.
- Microservices add independent deployment and scaling but cost failure modes, eventual consistency and operations.
- Synchronous chains multiply unavailability: ten services at 99.9 percent give 99.0 percent.
- The strangler pattern migrates through a facade in reversible steps; a BFF serves one client type; a mesh centralises network policy.

4.4 Failure Case Studies

Large systems fail in a small number of recurring ways. Studying the classes, and a set of public incident reports, gives a candidate a vocabulary for the "what could go wrong" part of a design and gives an engineer a list of failure modes to check in review. This chapter contains thirteen write-ups in a fixed form: Scenario, Symptoms, Diagnosis, Root cause, Fix, Prevention. Seven are **generic patterns**: a composite scenario that illustrates a failure class, with illustrative numbers that are labelled as assumed and are not taken from a specific company. Six are **documented public incidents**, whose facts (dates, durations, mechanisms, corrective actions) were checked against the operator's own published report for this book and are attributed with "per" and the source. The reports can be revised by their authors; the original posts are the authority.

Failure class	Kind	Documented source
Cascading failure	generic	related effects in the AWS October 2025 report
Retry storm and retry amplification	generic	no single incident
Metastable failure	generic	concept per Bronson et al., HotOS 2021
Cache avalanche after a cold start	generic	no single incident
Thundering herd	generic	herd effect in the Google Cloud June 2025 report
Clock skew	generic	no single incident
Hot partition	generic	no single incident
Split brain	documented	GitHub, 21 October 2018
Configuration push outage	documented	Google Cloud, 12 June 2025
Expensive input exhausting CPU	documented	Cloudflare, 2 July 2019
Operator error and shared dependency	documented	Amazon S3, 28 February 2017
Latent race in automation	documented	AWS us-east-1, 19 and 20 October 2025
Management plane lost with the network	documented	Meta, 4 October 2021

Index of the failure classes in this chapter.

4.4.1 Generic failure patterns

Cascading failure (generic pattern)

Scenario: a service in a chain of four loses one of its six instances to a failed deployment. The remaining five receive the traffic of six.

Symptoms: latency rises on the five survivors, their health checks start to fail intermittently, the load balancer removes them one by one, and within minutes the entire service is down although only one instance was originally faulty.

Diagnosis: the timeline of instance removals shows each removal increases the load on the remainder; thread pools and connection pools of the callers are exhausted by calls to the slow service, so upstream services fail next.

Root cause: a positive feedback loop. Capacity loss increases load per instance, which increases latency and failures, which reduces capacity further. Missing limits allow slow calls to hold callers' threads.

Fix: shed load at the overloaded service (reject excess requests quickly with an explicit status), add capacity, stop the callers' retries, and restore the failed component last, because restoring it into a flooded system fails again.

Prevention: headroom for the loss of a failure domain, timeouts and bulkheads (isolated pools per dependency), circuit breakers, load shedding by priority, and health checks that do not remove a working but busy instance. The AWS October 2025 report, covered below, documents cascades from a database name-resolution failure into instance launches and load balancer health checks.

Retry storm and retry amplification (generic pattern)

Scenario: a payment dependency becomes slow for 30 seconds. The API gateway retries three times, the order service retries three times, and the payment client retries three times.

Symptoms: the dependency receives up to 27 times its normal traffic, recovers more slowly or not at all, and callers' latency exceeds their own timeouts so that more retries are issued.

Diagnosis: compare the request count at the dependency with the request count at the edge; a ratio far above 1 identifies amplification. Traces show the same operation attempted at several layers.

Root cause: retries at every layer multiply (3 × 3 × 3), and synchronised retries arrive in waves because all clients use the same delay.

Fix: reduce retries to one layer, add exponential backoff with full jitter, cap retries with a budget (for example no more than 10 percent of requests in a window), propagate a deadline so that work whose caller has given up is not started, and open a circuit breaker.

Prevention: retry only idempotent or idempotency-keyed operations, set the retry policy at the layer nearest the dependency, and test with injected dependency slowdown.

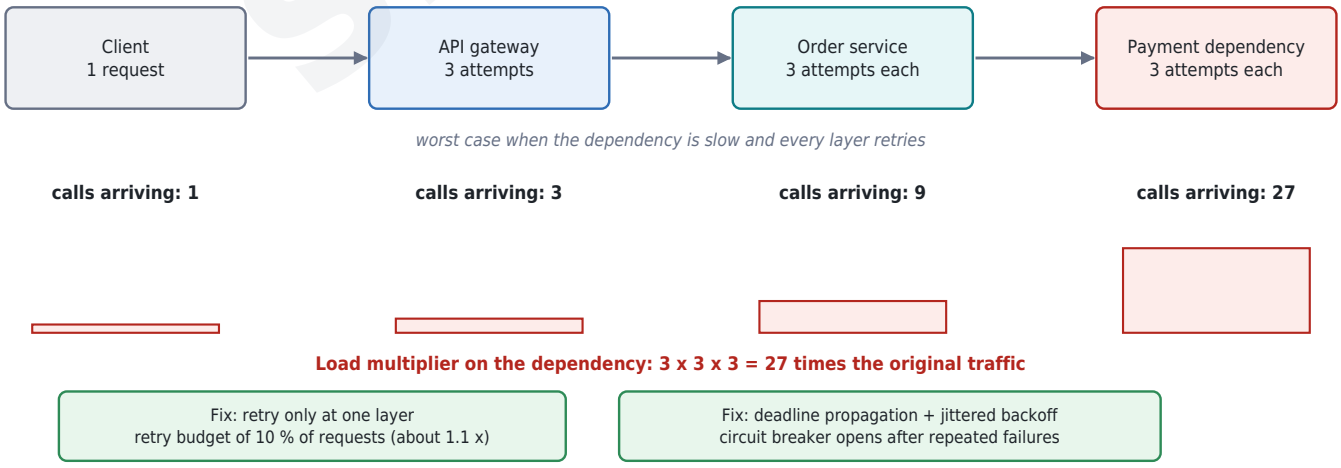


Figure 4.13. Three layers that each retry three times send up to 27 calls to the dependency for one client request; a budget of 10 percent keeps the amplification near 1.1.

sreram

Metastable failure (generic pattern, per Bronson et al., HotOS 2021)

Scenario: a service runs at 80 percent of its capacity (800 of 1,000 requests per second). A brief event halves its capacity for 10 seconds, after which capacity returns to normal. Clients give up after 2 seconds and retry up to three times.

Symptoms: after the trigger is gone, goodput stays at zero. The server is fully busy and nothing succeeds. The condition persists until traffic is reduced by an operator.

Diagnosis: the queue contains mostly requests whose clients have already given up; the server spends its entire capacity on work nobody waits for, while the retries of those clients add new load. Utilisation is high and goodput is zero.

Root cause: a **sustaining effect** (retries plus first-in-first-out processing of abandoned work) keeps the system in a stable overloaded state after the **trigger** (the capacity dip) has ended. The concept is described in "Metastable Failures in Distributed Systems" (Bronson, Aghayev, Charapko and Zhu, HotOS 2021). Systems operating with little headroom are most exposed, because a small trigger is enough.

Fix: break the feedback loop, not the trigger: drop incoming traffic or disable retries until the queue drains, then restore traffic gradually.

Prevention: discard requests whose deadline has expired before processing them, use bounded queues, retry budgets and client backoff, keep headroom, and load-test the recovery from a temporary dip in addition to steady-state capacity.



Figure 4.14. Simulated service with 800 requests per second of load and 1,000 of capacity: the same 10-second dip is recovered within seconds when expired work is skipped, but never when retries are unbounded and the queue is processed in arrival order.

Cache avalanche after a cold start (generic pattern)

Scenario: a cache cluster serving 20,000 reads per second at a 95 percent hit ratio is restarted after a maintenance error. The database behind it handles 3,000 reads per second (assumed).

Symptoms: immediately after the restart the database receives close to 20,000 reads per second, queries time out, the application fails, and the cache cannot refill because the loads that would populate it also time out.

Diagnosis: cache hit ratio drops from 95 to near 0 percent and database CPU and connection counts saturate in the same minute.

Root cause: the system depends on the cache to protect the database, and an empty cache removes that protection. Entries created at the same time also expire at the same time when they were given an identical TTL, which creates the same effect without a restart.

Fix: admit only as much traffic as the database can serve while the cache warms (admission control), coalesce concurrent misses for the same key into one database read, serve stale data where acceptable, and warm the cache from a snapshot or replica before returning to full traffic.

Prevention: random jitter added to TTLs, replicated or persistent caches, request coalescing, and a capacity plan that states what the database can sustain with a cold cache.

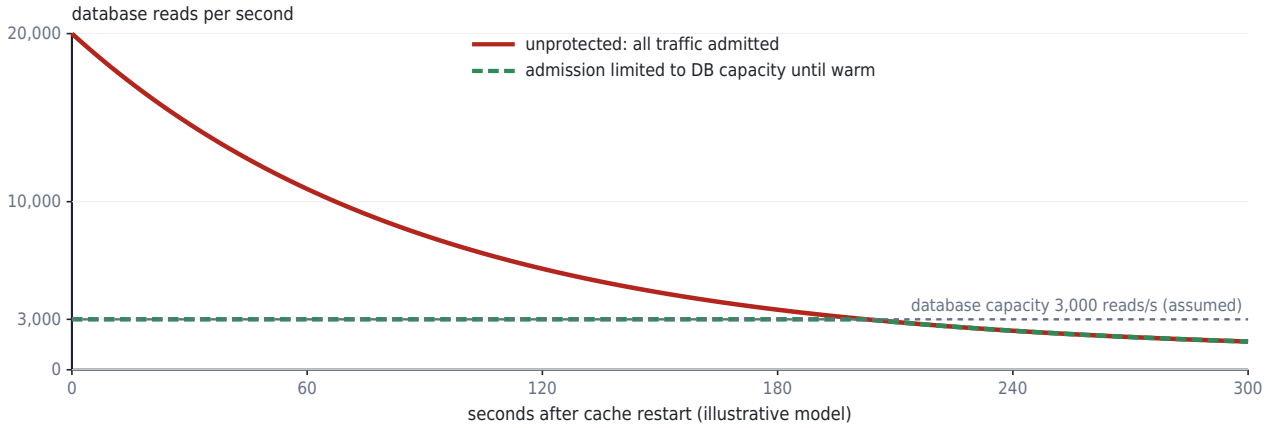


Figure 4.15. Illustrative model with a warm-up time constant of 90 seconds: without protection the database is overloaded for about 200 seconds, while admission control holds its load at its capacity until the hit ratio recovers.

Thundering herd (generic pattern)

Scenario: a popular cache key expires and 5,000 concurrent requests miss it in the same second. Alternatively, 100,000 clients reconnect to a recovered server at once.

Symptoms: a spike of identical expensive backend queries or of connection attempts, followed by timeouts and a second herd when the first wave fails.

Diagnosis: request logs show many identical queries within milliseconds, or connection attempts that line up on retry intervals.

Root cause: many independent actors react to one event at one instant: expiry, recovery or a restart.

Fix: per-key locking or request coalescing so that one caller recomputes while others wait or use stale data, early probabilistic refresh before expiry, and randomised delays on the client.

Prevention: exponential backoff with full jitter in every client library, and gradual admission after recovery. The simulation below compares three client retry policies.

Clock skew (generic pattern)

Scenario: two application servers have clocks that differ by 4 seconds. Orders are ordered by the timestamp assigned by the server that received them, and a lease on a job is considered expired by comparing local time with a stored expiry time.

Symptoms: events appear out of order in the feed, a job runs on two workers at once, a token is rejected as "not yet valid", and last-write-wins resolution keeps the older write.

Diagnosis: compare wall-clock timestamps of causally related events across hosts, and check the offset reported by the time synchronisation service.

Root cause: wall clocks of different machines are not synchronised exactly, and they can jump backwards when corrected. Ordering and expiry decisions built on them are unsound.

Fix: order events by a logical clock or a sequence assigned by one authority (a log partition, a database sequence), use monotonic clocks for measuring durations on one machine, and make lease holders stop work using fencing tokens that the storage layer checks.

Prevention: avoid comparing timestamps from different hosts for correctness, monitor clock offset, and use hybrid logical clocks where a time-like ordering is required. The coordination chapter covers logical clocks and fencing tokens.

Hot partition (generic pattern)

Scenario: a table is partitioned by user identifier. One account (a celebrity or a bot) receives 40 percent of the writes; in another system the partition key is the timestamp, so all current writes land in the newest partition.

Symptoms: one shard shows saturated CPU or throttling while the others are idle; adding shards does not help; latency of the affected keys increases.

Diagnosis: per-partition request metrics show a heavy skew; the key distribution identifies the dominant key.

Root cause: the access pattern is skewed or monotonic while the partitioning assumes a uniform spread.

Fix: split the hot key (append a random suffix or bucket number, and aggregate at read time), cache reads of the hot key, replicate the hot partition for reads, and for monotonic keys switch to a hash of the key or add a prefix.

Prevention: choose partition keys by access pattern, monitor per-partition load, and design a specific path for known hot entities. The partitioning chapter covers the techniques.

4.4.2 Hands-on: retry storms and metastable failure

The first program simulates the metastable scenario above. Each second 800 new requests arrive; the server processes 1,000 per second in first-in-first-out order, except that between seconds 10 and 20 it processes only 500; a client abandons a request after 2 seconds and sends a new attempt (at most three attempts). Four variants are compared. The model is deliberately simple: it is a discrete-time simulation on one machine and not a measurement of a real system.

metastable.py — PYTHON

```python
from collections import deque
CAP=1000; BASE=800; T=2; MAXTRY=3   # capacity and base load per second; timeout s; attempts
def run(name, retries=True, budget=None, shed=False):
    q=deque()                 # buckets [arrival, attempt, count, expired]
    good={}; out=[]
    for t in range(0,121):
        cap=500 if 10<=t<20 else CAP     # trigger: capacity halves for 10 s
        new=BASE
        # a client gives up on a request older than T seconds (the server does not know)
        # and sends a new attempt; the abandoned request stays in the server queue
        adds=[]
        for b in q:
            if not b[3] and t-b[0]>=T:
                b[3]=True
                if retries and b[1]<MAXTRY:
                    r=b[2] if budget is None else min(b[2],int(budget*new))
                    if r>0: adds.append([t,b[1]+1,r,False])
        q.extend(adds)
        q.append([t,1,new,False])
        left=cap; g=0
        while left>0 and q:
            b=q[0]
            if shed and b[3]: q.popleft(); continue     # deadline-aware: skip expired work
            take=min(left,b[2]); left-=take; b[2]-=take
            if not b[3]: g+=take
            if b[2]==0: q.popleft()
        good[t]=g; out.append(sum(x[2] for x in q))
    print("%-36s"%name, " ".join("%4d"%good[t] for t in (5,15,25,40,60,90,120)),
          "%8d"%out[120])
print("goodput = requests per second answered before the client gave up")
print("(base load 800 per second, capacity 1000 per second)")
print("%-36s"%"variant", " ".join("t=%-3d"%t for t in (5,15,25,40,60,90,120)), "queue@120")
run("no retries", retries=False)
run("3 attempts, FIFO, no budget")
run("3 attempts, retry budget 10 percent", budget=0.10)
run("3 attempts, skip expired (shedding)", shed=True)
```

Run of metastable.py (simulation) — TERMINAL

```
$ python3 metastable.py
goodput = requests per second answered before the client gave up
(base load 800 per second, capacity 1000 per second)
variant                              t=5   t=15  t=25  t=40  t=60  t=90  t=120 queue@120
no retries                            800    0    0  800  800  800  800        0
3 attempts, FIFO, no budget           800    0    0    0    0    0    0   151700
3 attempts, retry budget 10 percent   800    0    0    0    0  800  800        0
3 attempts, skip expired (shedding)   800  500 1000  800  800  800  800        0
```

What to notice: with three retries and no budget the goodput remains 0 at every later sample and the queue holds 151,700 requests at second 120, so the system has not recovered although capacity has been normal since second 20. Without retries the queue drains and service returns at second 27. A retry budget of 10 percent of the base load returns service at second 69: it recovers, but only after the backlog of abandoned work has been processed. Skipping requests whose client has already given up recovers fastest, at second 21, and serves 1,000 per second while it clears the

legitimate backlog. The conclusion is the central rule for overload: work that nobody is waiting for must not consume capacity.

The second program shows why backoff needs jitter. One thousand clients fail together; the server accepts 100 requests per 100 ms slot and rejects the rest, which retry.

jitter.py PYTHON

```python
import random
random.seed(7)
N=1000; SLOT=0.1; ACCEPT=100          # 1000 clients, server accepts 100 requests per 100 ms slot
def run(name, delay):
    nxt=[0.0]*N; done=[False]*N; attempts=0; peak=0; load={}; finish=0
    for rnd in range(40):
        pending=[i for i in range(N) if not done[i]]
        if not pending: break
        slots={}
        for i in pending: slots.setdefault(int(nxt[i]/SLOT),[]).append(i)
        for s,ids in sorted(slots.items()):
            peak=max(peak,len(ids)) if rnd>0 else peak; attempts+=len(ids)
            for k,i in enumerate(ids):
                if k<ACCEPT: done[i]=True; finish=max(finish,(s+1)*SLOT)
                else: nxt[i]=s*SLOT+delay(i,rnd)
    print("%-26s attempts=%5d  peak retry/slot=%4d  done after %.1f s"%(name,attempts,peak,finish))
base=0.5
run("fixed 500 ms",           lambda i,r: base)
run("exponential, no jitter", lambda i,r: min(30,base*2**r))
run("exponential, full jitter",lambda i,r: random.uniform(0,min(30,base*2**r)))
```

Run of jitter.py (simulation) TERMINAL

```
$ python3 jitter.py
fixed 500 ms               attempts= 5500  peak retry/slot= 900  done after 4.6 s
exponential, no jitter     attempts= 5500  peak retry/slot= 900  done after 121.6 s
exponential, full jitter   attempts= 2300  peak retry/slot= 209  done after 1.4 s
```

With a fixed 500 ms delay the retries arrive in waves of up to 900 requests per slot and the herd needs 5,500 attempts in total. Exponential backoff without jitter keeps the waves synchronised and completes only after 121.6 simulated seconds, because the waves become rarer as the delays grow. Full jitter (a uniform random delay up to the exponential bound) spreads the retries, lowers the peak retry load to 209 per slot, needs 2,300 attempts and finishes in 1.4 seconds.

4.4.3 Documented public incidents

The following write-ups summarise the operators' own published reports. They are examples of the classes above and of one further lesson: the largest incidents usually combine a small trigger, a missing safeguard and a recovery path that depends on the failed system.

Split brain: GitHub, 21 and 22 October 2018 (documented)

Source: GitHub engineering post-incident analysis of 21 October 2018.

Scenario: per the report, at 22:52 UTC on 21 October 2018 routine maintenance replacing failing 100G optical equipment cut connectivity between the US East Coast network hub and the primary US East Coast data center. Connectivity was restored after 43 seconds.

Symptoms: monitoring alerts began at 22:54. Engineers found database clusters in an unexpected topology at 23:02. Webhook delivery and GitHub Pages builds were paused at 23:19 to protect data. The site returned to normal only at 23:03 UTC on 22 October, after 24 hours and 11 minutes of degraded service.

Diagnosis: the topology manager (Orchestrator, using Raft consensus) had deselected the leader in the East Coast data center during the partition. Nodes in the West Coast data center and in the East Coast public cloud formed a quorum and promoted West Coast databases to primary. When connectivity returned, the application tier sent writes to the new West Coast primaries.

Root cause: both sites held writes that the other lacked. The East Coast data center had a few seconds of writes that had not replicated; the West Coast had accepted about 40 minutes of writes. Failing back was therefore not safe without reconciliation. The report states that one of the busiest clusters had 954 writes in the affected window and that no user data was lost, while reconciliation of some affected writes needed manual work. Cross-country latency made the promoted topology also slow for the application.

Fix: per the report, the team restored data from backups (taken every four hours and stored remotely), synchronised, and returned to a stable topology; dozens of replicas were hours behind and were supplemented by new read replicas. A backlog of more than five million hook events and 80 thousand Pages builds was processed, and about 200,000 webhook payloads that had exceeded an internal time-to-live were dropped.

Prevention: per the report, configure the failover manager to prevent primary promotion across regional boundaries, move toward active operation across several data centers, show component-level status, and invest in fault injection. The restore had never been performed for a complete cluster before and took hours, which illustrates why recovery drills matter.

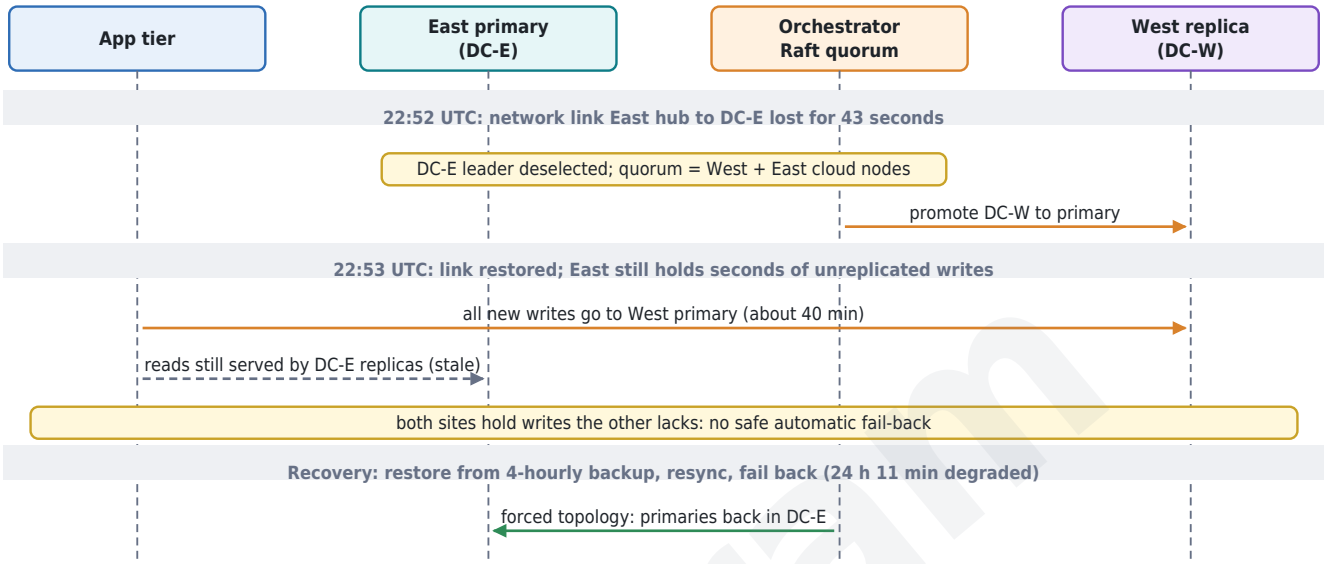


Figure 4.16. After a 43-second partition the quorum promoted the West primary, so two sites accepted divergent writes; reconciliation, not a switch, was required to recover.

Configuration push outage: Google Cloud, 12 June 2025 (documented)

Source: Google Cloud incident report for 12 June 2025.

Scenario: per the report, a new feature had been added on 29 May 2025 to Service Control, the regional system that performs API authorisation and quota checks. The new code path was not exercised during the region-by-region rollout and lacked error handling. On 12 June at about 10:45 PDT a policy change containing unintended blank fields was written to regional Spanner tables and replicated globally within seconds.

Symptoms: Service Control binaries crashed with a null pointer in every region and entered a crash loop. The incident ran from 10:51 to 18:18 PDT according to the report header. Recovery in us-central1 took up to about two hours and forty minutes. The first public status post came about an hour after the crashes because the status infrastructure was also affected.

Diagnosis: per the report, the on-call team began triage within two minutes and found the cause within ten. A "red button" that disabled the faulty policy-serving path was deployed in about forty minutes.

Root cause: a globally replicated configuration or data change reached all regions at once, the code that consumed it was not protected by a feature flag and did not handle the invalid value. In us-central1 the restarting tasks overloaded the Spanner table they depended on (a thundering herd), because the service lacked randomised exponential backoff.

Fix: per the report, task creation was throttled and traffic was routed to multi-regional databases in us-central1, and changes to the service were frozen after recovery.

Prevention: per the report, make failing checks fail open where appropriate, propagate globally replicated data incrementally with time to validate, require feature-flag protection for critical binaries (disabled by default), audit systems for randomised exponential backoff, and keep monitoring and communication independent of the systems being monitored.

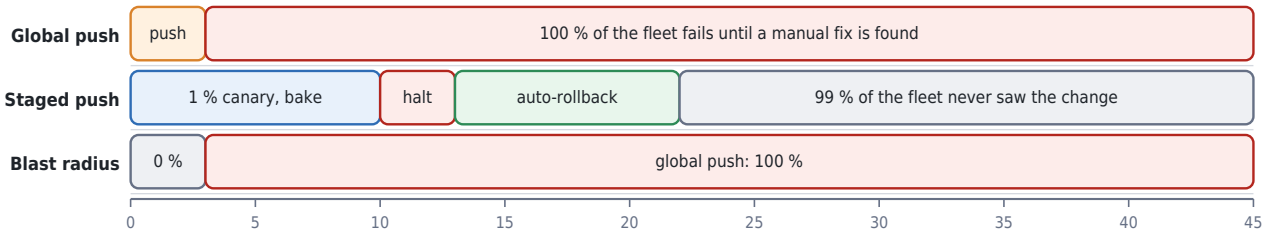


Figure 4.17. A global push exposes the entire fleet at once; a staged push exposes a small canary first, and a failed gate stops the rollout while most of the fleet is untouched.

Expensive input exhausting CPU: Cloudflare, 2 July 2019 (documented)

Source: Cloudflare post "Details of the Cloudflare outage on July 2, 2019".

Scenario: per the post, a change to the Web Application Firewall managed rules was deployed globally at about 13:42 UTC. It contained a regular expression that backtracked enormously.

Symptoms: CPU on every core that handles HTTP and HTTPS traffic became exhausted worldwide and customers received 502 errors. The first alert came about three minutes after the deployment. Internal tools were also hard to reach, and customers could not use the dashboard or API, since these pass through the same edge. A global "kill switch" disabled the WAF at 14:07; the WAF was re-enabled worldwide at 14:52.

Diagnosis: per the post, the WAF was identified as the cause and disabled globally with the kill switch; engineers first had to work around access problems because the access service was down too.

Root cause: per the post, several factors combined: the regular expression itself, a CPU-protection safeguard that had been removed in an earlier refactoring, a regular expression engine without complexity guarantees, a test suite that did not detect excessive CPU use, and a procedure that allowed non-emergency rules to be deployed globally without a staged rollout. The post gives an example of the growth: a pattern matched a three-character string in 23 steps, and 555 steps with 20 trailing characters.

Fix: global termination of the WAF.

Prevention: per the post, restore the CPU safeguard, add performance profiling of rules to the test suite, move to an engine with runtime guarantees (RE2 or the Rust regex engine), use staged rollouts for rule changes, and provide a way to move the dashboard and API off the edge.

Operator error and shared dependency: Amazon S3, 28 February 2017 (documented)

Source: AWS post-event summary for the Amazon S3 event in the Northern Virginia region.

Scenario: per the summary, at 9:37 AM PST an authorised team member debugging a slow billing process ran a command intended to remove a few servers from one subsystem. A mistyped input removed a much larger set, because the tool allowed too much capacity to be removed too quickly.

Symptoms: the removed servers supported the index subsystem (metadata and location of all objects) and the placement subsystem (allocation for new objects). Both needed a full restart and S3 could not serve requests meanwhile. Services that depend on S3 in the region, among them the S3 console, new EC2 instance launches, EBS volumes needing snapshot data and AWS Lambda, were affected. The AWS status dashboard could not update until 11:37 AM because its administration console depended on S3.

Diagnosis: the team identified the removed capacity and the subsystems affected; per the summary, the restart took longer than expected because of the growth of the service and the safety checks needed to validate metadata integrity.

Root cause: an administrative tool without a minimum-capacity safeguard and a large blast radius per subsystem, plus a dependency of the status mechanism on the service that failed.

Fix: per the summary, the index subsystem could serve reads at 12:26 PM, was fully recovered at 1:18 PM, and the placement subsystem completed recovery at 1:54 PM, about 4 hours and 17 minutes after the trigger.

Prevention: per the summary, remove capacity more slowly, block removal below a minimum capacity, audit other tools for similar safeguards, partition the index subsystem into smaller cells to reduce blast radius and recovery time, and run the status dashboard across multiple regions.

Latent race in automation: AWS us-east-1, 19 and 20 October 2025 (documented)

Source: AWS post-event summary for the DynamoDB service disruption in the Northern Virginia region.

Scenario: per the summary, the event began at 11:48 PM PDT on 19 October 2025 with rising DynamoDB API error rates. The cause was a latent race condition in the system that manages DNS for the DynamoDB regional endpoint. A planner produces versioned plans, and redundant enactors in three availability zones apply them. One enactor was delayed while retrying; another applied a newer plan and then deleted older plans. The delayed enactor then applied its stale plan, since its check that the plan was newer had been made earlier, and the cleanup that followed removed all IP addresses of the endpoint, leaving an empty record that automation could not repair.

Symptoms: customers and AWS services could not connect to DynamoDB. DynamoDB recovered for customers at 2:40 AM. A broader cascade followed: a component of EC2 that manages instance leases entered congestive collapse when re-establishing leases after DynamoDB returned, new instance launches failed until about 10:36 AM, and network load balancer health checks alternated between passing and failing because new instances lacked network state, which removed capacity; automatic health-check failover was disabled at 9:36 AM and restored at 2:09 PM. The report gives 2:20 PM as the time when the last affected services were restored.

Diagnosis: engineers identified the DNS state as the source by 12:38 AM, and restored DNS manually by 2:25 AM.

Root cause: a time-of-check to time-of-use race between redundant automation components, with a cleanup step that could delete the only valid state; and recovery that put sudden load on dependent control systems.

Fix: manual repair of the DNS record, throttling and restarts of the lease manager, and temporary disabling of health-check failover.

Prevention: per the summary, AWS disabled the planner and enactor automation worldwide pending a fix of the race and added safeguards. Generic lessons are to validate preconditions at the moment of the write (compare-and-set on a version), to never let cleanup delete the last good state, and to make recovery paths rate-limited so that a restored dependency is not flooded.

Management plane lost with the network: Meta, 4 October 2021 (documented)

Source: Meta engineering post "More details about the October 4 outage".

Scenario: per the post, a command issued during routine maintenance, intended to assess the availability of global backbone capacity, instead disconnected all backbone connections and so isolated the data centers. An audit tool that should have stopped the command had a bug.

Symptoms: the DNS name servers, which advertise their addresses to the internet through BGP, withdrew those advertisements when they could not reach the data centers, as designed to signal an unhealthy connection. The services became unreachable from the internet. The loss of DNS also broke many internal tools.

Diagnosis: normal remote access was unavailable, so engineers had to go to the data centers; physical and system security measures, designed to be hard to bypass, slowed the access.

Root cause: an unsafe command that passed a faulty audit, a design in which a health-based withdrawal of routes turned a backbone failure into a global DNS failure, and a recovery path that depended on the network that had failed.

Fix: after backbone connectivity returned, services were restored gradually; the report notes that power usage in data centers had dropped by tens of megawatts and a sudden rebound could strain systems, and that rehearsed "storm" drills enabled a controlled restart.

Prevention: per the post, improve audit tooling, extend the drills to simulate a global backbone outage, and accept that strong physical security slows recovery from internal errors.

4.4.4 Cross-cutting lessons

Class	Detect	Contain or recover
Cascading failure, retry storm	request ratio between layers, queue depth, saturation	timeouts, bulkheads, circuit breakers, load shedding, retry budgets
Metastable failure	goodput far below throughput	drop deadline-expired work, shed load until the queue drains
Cache avalanche, thundering herd	hit ratio, identical concurrent queries	TTL jitter, coalescing, admission control, stale serving
Split brain	two leaders, divergent write logs	quorum with fencing, no cross-region promotion, manual reconcile
Config and rule pushes	error rate per rollout wave	staged rollout, feature flags, automatic halt and rollback
Operator error	tool audit trail	minimum-capacity guards, small blast radius (cells), confirmation for bulk actions
Automation race	invariants checked by monitors	compare-and-set on versions, never delete the last good state
Dependency on the failed system	status page and tools reachable out of band	independent management plane, offline runbooks

Failure class and the controls that address it.

Four themes recur. First, rollouts of data and configuration need the same staging as code. Second, every protective mechanism (retry, failover, cache, auto-remediation) is itself a possible source of amplification and must have limits. Third, recovery must not depend on the system that has failed: the status page, the access control to repair tools and the deployment system need an independent path. Fourth, rehearsed recovery is faster than improvised recovery: several of the reports above state that a drill had never been run for the scenario that occurred.

4.4.5 Questions to expect

What is a cascading failure and how is it stopped?

A failure that spreads because the loss of capacity increases the load on the remaining components, or because slow calls exhaust callers' resources. It is stopped by shedding load at the overloaded component, by timeouts and bulkheads that isolate dependencies, by circuit breakers and by restoring capacity before reconnecting traffic.

What is a metastable failure?

A failure state that persists after the triggering event has ended, because a feedback loop such as retries sustains the overload. The recovery action targets the loop (reduce traffic, drop expired work), not the original trigger. Prevention includes retry budgets, bounded queues and headroom.

Why must retries use exponential backoff with jitter?

Backoff reduces the retry rate as failures persist; jitter spreads retries in time so that clients do not synchronise into waves. Without jitter a herd of clients retries together and re-creates the overload.

How can a cache stampede be prevented?

By coalescing concurrent misses into one load (single-flight or per-key lock), refreshing popular keys before expiry, adding random jitter to TTLs, serving stale data during recomputation, and limiting database concurrency.

How does a system avoid split brain?

By requiring a majority quorum to elect a leader, by fencing tokens that make stale leaders harmless at the storage layer, and by designing failover policy so that promotion happens only where latency and data-freshness conditions are acceptable. The GitHub incident shows that automation that promotes across regions can create divergent histories.

How are configuration changes made safe?

By treating them as deployments: validation of schema and values, a canary wave, metrics gates with automatic halt, feature flags for the code that consumes them, and a fast, tested rollback or kill switch. Globally replicated changes should propagate incrementally.

What should be done first during a major incident?

Mitigate before diagnosing fully: roll back the latest change, disable the feature flag, shed load or fail over, and communicate status. Root-cause analysis follows once users are protected. A blameless review then records the timeline, the contributing factors and concrete corrective actions.

Why is it dangerous when recovery tools depend on the production system?

If the repair path (access control, DNS, status page, deployment) runs on the failed system, repair is delayed or impossible. The S3, Cloudflare, Google Cloud and Meta reports all describe delays of this kind, which is why an independent out-of-band path is a design requirement.

KEY POINTS

Failures recur in classes: cascades, retry storms, metastable states, cold caches, herds, split brain, skew, hot keys and unsafe changes.

- Work that nobody is waiting for must not consume capacity; deadlines, bounded queues and retry budgets enforce this.
- Backoff without jitter does not remove synchronisation; use full jitter.
- Roll out configuration like code: stages, gates, flags, rollback.
- Do not promote or fail over across regions automatically without regard to data divergence.
- Keep the recovery path independent of the system being recovered, and rehearse it.

4.5 Interview Preparation

This chapter consolidates preparation for backend and full-stack interviews that include a system design round and a low-level design (machine-coding) round. It describes what interviewers evaluate, gives a time plan and a script for a 45-minute design interview, lists the mistakes that most often fail otherwise strong candidates, and provides a bank of about one hundred questions with model answers and twenty design prompts with an approach for each. The answers are outlines: each should be expanded with the numbers and trade-offs of the specific problem.

4.5.1 What interviewers evaluate

Dimension	Evidence of a strong answer	Evidence of a weak answer
Requirements	asks about scale, users, read/write ratio, consistency and latency needs; states explicit scope and non-goals	starts drawing boxes immediately
Estimation	derives QPS, storage and bandwidth with shown arithmetic and uses them to choose components	quotes numbers without derivation, or never uses them
Design structure	clear API, data model, high-level flow, then depth	an unordered list of technologies
Depth	explains the mechanism of the hard problem (fan-out, seat hold, dedup) with alternatives	names a product (Kafka, Redis) without explaining why or how
Trade-offs	states what is given up by each choice and the assumption behind the decision	presents one design as the only possibility
Failure thinking	identifies single points of failure, hot keys, retries, duplicates and degradation	only the happy path
Communication	narrates reasoning, checks assumptions, responds to hints, manages time	long silences, defensiveness, over-detail on one box
Judgement	proportionate design for the stated scale; simplest design that meets the requirements	maximal architecture for a small problem

Evaluation rubric for a system design round.

For a low-level design round the dimensions change: requirement clarification, identification of entities and relationships, appropriate use of patterns (not decoration), correctness and compilability of the code, handling of concurrency, extensibility (what changes when a requirement is added) and testability.

Communication practice is part of preparation. State the plan at the start ("requirements, estimation, API, data model, design, then two deep dives"), summarise after each phase, label assumptions as assumptions, give numbers with

units, and when stuck say what the options are and which would be tried first. Hints from an interviewer are information about the direction expected and are to be used rather than defended against.

4.5.2 The 45-minute walkthrough

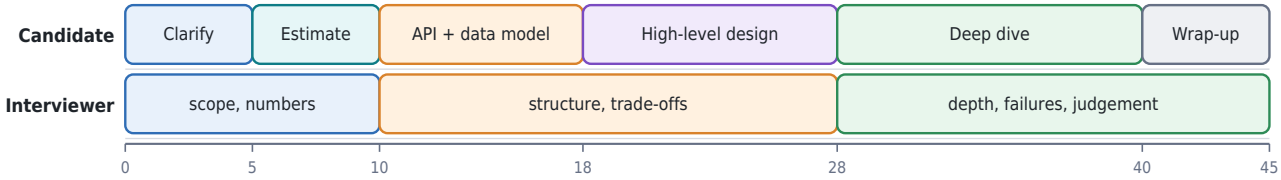


Figure 4.18. A 45-minute round divided into six phases; the interviewer assesses scoping in the first ten minutes, structure in the middle, and depth and judgement in the last third.

Phase	Minutes	What to say and produce
Clarify	0 to 5	Restate the problem. Ask: who are the users, which features are in scope, expected scale (users, requests per second), read-to-write ratio, latency and availability targets, consistency needs. Write the functional list and the non-functional list, and name two things that are out of scope.
Estimate	5 to 10	Daily active users to QPS (divide by 86,400, multiply by a peak factor of 2 to 3), storage per day and for the retention period, bandwidth, cache size by the 80/20 rule. Conclude with a statement of what the numbers imply (for example "read heavy, so cache; 5 TB per year, so one database is insufficient").
API and data model	10 to 18	Three to five endpoints with identifiers, pagination and idempotency for writes. Main tables or collections, keys, indexes and the partitioning key with the reason.
High-level design	18 to 28	Draw clients, load balancer, services, cache, database, queue, workers. Walk through the main read flow and write flow step by step with the data touched in each step.
Deep dive	28 to 40	Choose the hardest one or two problems (or those the interviewer selects). Give alternatives, compare them, decide, and state the assumption behind the decision. Cover bottlenecks and failures: hot key, duplicate delivery, failover, degradation.
Wrap-up	40 to 45	Summarise the design in three sentences, list known weaknesses and the next improvements (monitoring, multi-region, cost), and invite questions.

Script for each phase (minutes are guidance and are adjusted to the interviewer).

4.5.3 Common mistakes

- Skipping requirements and estimation, then choosing technology without a reason tied to a number.
- Naming products without the mechanism: "use Kafka" without stating the partition key, ordering needs, delivery semantics and consumer behaviour.
- Designing for planetary scale when the stated load is a few hundred requests per second; unnecessary complexity is a defect.
- Ignoring consistency: stating "eventual consistency" without saying which user-visible anomaly is acceptable and which is not (for example double charging is not acceptable, a stale like count is).
- Forgetting idempotency and duplicates in any flow that has retries or at-least-once delivery.
- A single database, single queue or single load balancer that is never discussed as a single point of failure.
- Treating the cache as free: no invalidation strategy, no stampede protection, no sizing.
- In machine coding: writing code before listing entities and use cases, using inheritance where composition is appropriate, patterns without a requirement that needs them, and no thread-safety statement for shared state.
- Silence for long periods, or arguing with a hint instead of exploring it.

4.5.4 Question bank

The bank is grouped by area. Each answer is the outline that an interviewer expects in the first minute; a complete answer adds figures and the failure case.

Fundamentals

Q1. What is the difference between latency and throughput? Latency is the time for one operation; throughput is the number of operations per unit time. They interact: past a utilisation of roughly 70 to 80 percent, queueing makes latency rise sharply while throughput gains flatten.

Q2. Vertical versus horizontal scaling? Vertical adds resources to one machine: simple, but bounded and a single point of failure. Horizontal adds machines: nearly unbounded and fault tolerant, but requires stateless services or partitioned state and a load balancer.

Q3. Why are stateless services preferred? Any instance can serve any request, so instances can be added, removed and replaced freely, and failures lose no session state. State moves to a shared store (database, cache, token).

Q4. How is QPS estimated from daily active users? Requests per user per day times users, divided by 86,400 for the average; multiplied by a peak factor of 2 to 3 for the peak. 10 million users at 20 requests per day give 2,315 requests per second on average and about 7,000 at peak.

Q5. What are the nines of availability? 99.9 percent allows 43.2 minutes of downtime in 30 days, 99.99 percent allows 4.3 minutes, 99.999 percent allows 26 seconds. Serial dependencies multiply: ten services at 99.9 percent give 99.0 percent.

Q6. Layer 4 versus layer 7 load balancing? Layer 4 forwards TCP connections without reading the request: fast and protocol-agnostic. Layer 7 parses HTTP and can route by path, header or cookie, terminate TLS and apply policy, at higher cost.

Q7. Which load balancing algorithms exist and when is each used? Round robin for uniform requests, least connections for uneven durations, consistent hashing for cache affinity, power of two choices for good balance with little state, weighted variants for heterogeneous nodes.

Q8. What is a CDN and what should it hold? A geographically distributed cache near users; it serves static and cacheable content, reduces origin load and latency, and can absorb attack traffic. Dynamic personalised responses are poor candidates unless they can be cached by a key.

Q9. What is the difference between a reverse proxy, a load balancer and an API gateway? A reverse proxy fronts servers; a load balancer distributes traffic across instances; an API gateway adds cross-cutting API concerns (authentication, rate limiting, routing, transformation). One product can play several roles.

Q10. What is graceful degradation? Continuing to serve the core function when a dependency fails by dropping optional features (recommendations, personalisation) or serving stale data, in place of failing the whole request.

Q11. What is backpressure? A signal from an overloaded consumer to producers to slow down, implemented with bounded queues, rejection with retry-after, or flow control. Without it, queues grow without limit and latency becomes unbounded.

Q12. What is idempotency and where is it required? An operation is idempotent if repeating it has the same effect as one execution. It is required wherever retries or duplicate delivery occur, typically implemented with a client-supplied idempotency key stored with the result.

Q13. How is pagination designed for large lists? Cursor (keyset) pagination, which continues after the last seen key, is stable and efficient; offset pagination scans and skips rows, becomes slow for deep pages and shows duplicates when data changes.

Q14. What is the 80/20 rule in caching and sizing? About 20 percent of keys receive about 80 percent of reads, so a cache holding the hot 20 percent of the data set captures most reads. It is a starting assumption to be verified with access logs.

Data and caching

Q15. How is SQL versus NoSQL chosen? By access pattern, consistency and scale: relational for joins, transactions and flexible queries; key-value or wide-column for known key access at very high volume; document for aggregates; search, graph and time-series stores for their specific query types. Many systems combine stores.

sheran

Q16. Explain cache-aside. The application reads the cache, on a miss reads the database and writes the value into the cache; on update it writes the database and invalidates (or updates) the cache. It is simple and tolerates cache loss, with a short window of staleness and a risk of races.

Q17. Write-through, write-back and write-around? Write-through writes cache and store together (consistent, slower writes). Write-back writes the cache and flushes later (fast, risks loss on failure). Write-around writes only the store and lets reads populate the cache (avoids caching rarely read data).

Q18. How is cache invalidation handled? By TTLs as a safety net, explicit invalidation on writes, versioned keys, or event-driven invalidation from change data capture. The hardest case is a race between a read refilling the cache with an old value and a concurrent update.

Q19. What are cache penetration, stampede and avalanche? Penetration is repeated requests for keys that do not exist (fixed with negative caching or a Bloom filter). Stampede is many concurrent misses on one hot key (fixed with coalescing). Avalanche is mass expiry or loss of the cache (fixed with TTL jitter and admission control).

Q20. LRU versus LFU? LRU evicts the least recently used entry and adapts quickly to changing access; LFU evicts the least frequently used and resists scan pollution but adapts slowly. Real caches often combine them.

Q21. How does replication lag affect users and what mitigates it? A user may not see a write immediately after making it. Mitigations: read your own writes from the leader for a short period, sticky routing to the leader after a write, or version tokens that replicas must have reached.

Q22. Range versus hash partitioning? Range supports range scans but creates hot partitions for monotonic keys. Hash spreads load evenly but loses range ordering. Directory-based and geographic schemes add flexibility with a lookup cost.

Q23. What is consistent hashing and why use virtual nodes? Keys and nodes are placed on a ring and each key belongs to the next node clockwise, so adding or removing a node moves only about 1/N of the keys. Virtual nodes give each physical node many ring positions, which evens the load and the movement.

Q24. How are hot keys handled? Detect them by per-key metrics, cache them locally in the application tier, replicate them, or split them (key plus random suffix) and aggregate on read.

Q25. How is a cross-shard transaction handled? Avoid it by choosing the partition key so that related data co-locates. Otherwise use a saga with compensations, or two-phase commit when atomicity is mandatory and the cost in latency and availability is acceptable.

Q26. How is resharding done without downtime? Create the new layout, copy data in the background, apply changes through double writes or change data capture until the copy is current, switch reads, then writes, and remove the old data. Directory or virtual-bucket indirection makes the move small.

Q27. When is a search engine needed beside a database? For full-text relevance, faceting and fuzzy matching that a relational index cannot serve efficiently. It is populated asynchronously from the primary store, which remains the source of truth.

Q28. How are large files stored? In object storage, uploaded in chunks with resumable uploads and checksums, addressed by a key that the database stores; metadata stays in the database, and a CDN serves downloads.

Distributed systems

Q29. State the CAP theorem correctly. During a network partition a system must choose between consistency (every read sees the latest write) and availability (every request receives a response). Without a partition both are possible; PACELC adds that without a partition there is still a trade-off between latency and consistency.

Q30. What is a quorum? With N replicas, a write acknowledged by W and a read from R replicas see the latest value when R plus W is greater than N. For N equal to 3, W and R equal to 2 tolerates one failure with overlap.

Q31. What is eventual consistency and when is it acceptable? Replicas converge when updates stop. It is acceptable where temporary staleness is harmless (like counts, feeds), and unacceptable for balances and inventory commitments unless compensated.

Q32. How does Raft elect a leader? A follower whose election timer expires becomes a candidate, increments the term and requests votes; a majority grants votes and the candidate becomes leader and replicates a log, committing an entry when a majority has stored it.

Q33. What is a fencing token? A monotonically increasing number issued with a lock or lease; the protected resource rejects requests carrying a lower number than one already seen, so a paused or partitioned old holder cannot corrupt data.

Q34. Why is a distributed lock with a TTL not sufficient for correctness? The holder can pause (garbage collection, scheduling) beyond the TTL while still believing it holds the lock. Correctness needs fencing at the resource; the lock only improves efficiency.

Q35. Two-phase commit versus saga? Two-phase commit gives atomicity with blocking coordination and holds locks during the protocol. A saga performs local transactions with compensating actions, which scales and stays available but exposes intermediate states and needs idempotent steps.

Q36. Which clocks are used in distributed systems? Wall clocks for display and coarse expiry, monotonic clocks for durations, Lamport and vector clocks for causality, hybrid logical clocks for time-like ordering with causality.

Q37. How are unique IDs generated at scale? Snowflake-style 64-bit IDs (timestamp, machine id, sequence) are roughly time ordered and need no coordination per ID; UUIDs need no coordination but are large and unordered; ULIDs are ordered; database sequences are simple but a bottleneck.

Q38. What causes split brain and how is it prevented? Two nodes both act as leader after a partition. Prevent it with majority quorums, leases with fencing, and failover policies that promote only when the old leader is fenced.

Q39. What is a Bloom filter and what are its limits? A bit array with several hash functions that answers "definitely not present" or "possibly present", with no false negatives and a tunable false-positive rate. Elements cannot be deleted from the plain version.

Q40. How does a service discover another? Through a registry with health checks (client-side or server-side lookup), DNS, or the platform service abstraction; clients cache results and tolerate stale entries.

Q41. How is a rate limiter implemented across many nodes? A shared counter or token bucket in a fast store updated atomically (a script or a single command), or local buckets with a periodic sync that accepts small overshoot. The trade-off is accuracy versus latency and availability of the shared store.

Q42. Fixed window versus token bucket? Fixed window counts per interval and permits double the limit at a boundary; token bucket refills at a constant rate and permits bursts up to the bucket size, which matches most API policies.

Messaging and asynchronous processing

Q43. Queue versus log? A queue delivers each message to one consumer and removes it after acknowledgement; a log retains messages for a period and lets many consumer groups read independently at their own offset, which also allows replay.

Q44. At-most-once, at-least-once and exactly-once? At-most-once may lose messages; at-least-once may duplicate them; exactly-once effects are achieved by at-least-once delivery plus idempotent processing or transactional writes, not by the network.

Q45. How is ordering preserved? Within a partition only. Messages with the same key go to the same partition, and a consumer processes a partition sequentially. Global ordering requires a single partition and limits throughput.

Q46. What is a dead-letter queue? A queue for messages that failed after the allowed retries, so that they do not block the stream and can be inspected, fixed and replayed.

Q47. What is the outbox pattern? The service writes the business change and an event row in one local transaction, and a relay publishes the rows to the broker. It avoids the dual-write inconsistency between a database and a broker.

Q48. How are consumer failures and retries handled? Retry with exponential backoff and jitter, cap attempts, move poison messages to the dead-letter queue, keep processing idempotent, and make visibility timeouts longer than the processing time.

Q49. What is the role of consumer groups? Partitions of a topic are divided among the consumers of one group, giving parallelism and failover; different groups each receive all messages.

Q50. Event sourcing and CQRS in brief? Event sourcing stores the sequence of state-changing events as the source of truth; CQRS separates the write model from read models built from the events. They give audit and flexible read views at the cost of complexity and eventual consistency.

Q51. Batch versus stream processing? Batch processes bounded data periodically with high throughput and simple recovery; stream processes events continuously with low latency and needs handling of late data and state.

Q52. How is backpressure applied in a pipeline? Bounded queues between stages, consumer lag monitoring, pausing or throttling producers, load shedding of low-priority messages, and autoscaling consumers on lag.

Scaling scenarios

Q53. A service latency doubled after a deployment. What is done? Compare the deployment time with the onset, roll back or disable the flag if correlated, then use traces to find the slow span, check resource saturation and recent dependency changes.

Q54. Traffic grows ten times in a year. How is the system prepared? Load test to find the current ceiling, identify the first bottleneck (usually the database), add caching and read replicas, partition the hot tables, move slow work to queues, and add autoscaling and capacity alarms.

Q55. The database is the bottleneck. What are the options in order? Fix queries and indexes, add caching, add read replicas, batch and queue writes, partition vertically by feature, then shard horizontally.

Q56. How is a flash sale handled? Pre-scale, put a virtual waiting room or queue in front, cache the product data, reserve stock atomically in a fast store, rate limit per user, make order creation asynchronous and idempotent, and degrade non-essential features.

Q57. How are millions of concurrent connections served? Event-driven or virtual-thread servers, WebSocket gateways that hold only connection state and are horizontally scaled, a pub-sub backplane to route messages between gateways, and a connection-count limit per node from load tests.

Q58. A cache outage doubles database load. What is done? Admit only the traffic the database can serve, coalesce misses, serve stale where allowed, and restore the cache warmed from a replica or snapshot.

Q59. How is a hot celebrity account handled in a feed? Fan-out on write is skipped for accounts above a follower threshold; their posts are fetched at read time and merged with the precomputed feed.

Q60. How are duplicate orders prevented? An idempotency key unique per client intent, stored with a unique constraint, and returned results on repeat; the database constraint, not application logic alone, is the guard.

Q61. How is search autocomplete served in under 100 ms? A trie or prefix index of top completions per prefix, cached at the edge and in memory, updated by an offline pipeline from query logs, with the top-k precomputed per prefix.

Q62. How is a multi-region design chosen? From the RPO and RTO and the latency need: active-passive for simplicity, active-active by user partition for latency and data residency, and conflict-free or last-write-wins data only where acceptable.

Q63. How is a zero-downtime migration of a large table done? Expand and contract with dual writes, backfill in small batches with throttling, verification by comparison, switch reads, then stop old writes and drop the old structure.

Q64. How is a service made to survive its dependency failure? Timeouts, a circuit breaker, a fallback (cache, default or reduced feature), bulkhead isolation, and a retry budget.

Low-level design and object-oriented design

Q65. What are the SOLID principles? Single responsibility, open for extension and closed for modification, Liskov substitution, interface segregation, dependency inversion. They reduce the cost of change by limiting coupling.

Q66. Composition or inheritance? Prefer composition: it allows behaviour to change at runtime and avoids fragile base classes. Use inheritance for a true is-a relationship with substitutable behaviour.

Q67. What steps are followed in a machine-coding round? Clarify requirements, list entities and relationships, define interfaces and behaviours, choose patterns where a requirement demands them, write compilable code with a main demo, then discuss concurrency and extension.

Q68. How is a parking lot modelled? Entities: lot, level, spot (by type), vehicle, ticket, payment. A strategy selects the spot, an enum or sealed type models vehicle and spot kinds, and tickets record entry time and price calculation is a strategy.

Q69. How is an elevator system modelled? Elevator with a state (idle, moving up, moving down, door open), a request scheduler (strategy such as nearest car or SCAN), and a controller that dispatches requests; the State pattern keeps transitions explicit.

Q70. How is a thread-safe LRU cache built? A hash map plus a doubly linked list gives O(1) get and put; thread safety through one lock (simple) or lock striping, or a concurrent map with a bounded eviction queue when exact LRU is not needed.

Q71. How is seat booking made safe against double booking? Make the claim atomic: a conditional update or a unique constraint on seat and show; hold a seat with an expiry before payment; release expired holds; never check availability and book in separate unprotected steps.

Q72. How is a vending machine modelled? The State pattern: idle, has money, dispensing, out of service; each state defines which actions are valid, and the machine delegates to the current state.

Q73. How is Splitwise simplified? Compute each person net balance (paid minus owed), then repeatedly match the largest creditor with the largest debtor using two heaps, which reduces the number of transactions.

Q74. How is a rate limiter class designed? An interface `RateLimiter.allow(key)` with token bucket and sliding window implementations, a factory or configuration for per-key limits, a clock abstraction for testing, and atomic updates for thread safety.

Q75. How is a logger framework designed? Log levels, appenders (console, file) as strategies, formatters, a chain of handlers filtered by level, and a configured singleton or registry for loggers; asynchronous appenders prevent logging from blocking requests.

Q76. How is an in-memory file system modelled? The Composite pattern: a node interface with file and directory implementations, directories holding children by name, path resolution by splitting on separators, and operations (create, list, read, delete) on nodes.

Q77. How is a pub-sub event bus designed? A topic registry mapping topics to subscribers, publish delivering to subscribers through an executor, subscriber isolation so that one failure does not affect others, and optional ordering per topic.

Q78. How is a connection pool designed? A bounded queue of connections, a borrow with timeout, validation on borrow, return on close, a maximum size and idle eviction, and leak detection for connections not returned.

Design patterns

Q79. Strategy versus State? Both delegate to an interchangeable object. Strategy lets the client choose an algorithm; State changes the delegate automatically as the object transitions, and states know their successors.

Q80. Where is the Observer pattern used? Event listeners, publish-subscribe, and in Spring application events: publishers emit events and registered listeners react without the publisher knowing them.

Q81. How is a thread-safe singleton written in Java? An enum singleton, a holder class (lazy initialisation by class loading), or a double-checked lock with a volatile field. The enum form is also safe against reflection and serialisation.

Q82. Factory versus Builder? A factory chooses which type to create; a builder constructs one complex object in steps with validation, which is useful for many optional parameters.

Q83. What does the Decorator pattern solve? Adding behaviour to an object dynamically by wrapping it with the same interface (Java I/O streams), avoiding a subclass for each combination.

Q84. Adapter versus Facade versus Proxy? Adapter converts one interface to another; Facade offers a simpler interface to a subsystem; Proxy has the same interface and controls access (lazy loading, security, caching, as in Spring AOP).

Q85. What is the Chain of Responsibility pattern? A request passes through a chain of handlers, each processing or passing it on; used in servlet filters, logging levels and approval flows.

Q86. What is the Command pattern used for? Encapsulating a request as an object so that it can be queued, logged, undone or retried, as in task queues, undo stacks and job schedulers.

Q87. Template Method in practice? A base class defines the algorithm skeleton and subclasses override steps; Spring JdbcTemplate fixes the resource handling and lets the caller supply the query and mapping.

Q88. When are patterns overused? When a pattern is added without a requirement that varies, which adds indirection and classes without benefit. The test is whether the pattern removes a concrete change cost.

Q89. What is dependency injection and why does it help design? Dependencies are supplied from outside through constructors, which keeps classes loosely coupled to interfaces, simplifies testing with fakes, and centralises object wiring.

Q90. What is the Law of Demeter? An object should talk only to its direct collaborators and not navigate through them (`a.getB().getC().doIt()`), which reduces coupling to internal structure.

Concurrency in design problems

Q91. How is a race condition on a counter prevented? Atomic types or locks for in-process state, and atomic increments or conditional updates in the shared store for distributed state.

Q92. Deadlock conditions and prevention in a transfer between accounts? Mutual exclusion, hold and wait, no preemption and circular wait must all hold. Prevent by acquiring locks in a global order (by account id) or with timeouts and retry.

Q93. `synchronized`**,** `ReentrantLock` **and atomics, when?** Synchronized for simple critical sections; ReentrantLock for timed or interruptible locking and conditions; atomics and compare-and-set for single-variable state.

Q94. How is a producer-consumer queue made safe? A blocking bounded queue, which provides both mutual exclusion and flow control; consumers block when empty and producers when full.

Q95. What is a memory leak in a cache and how is it avoided? Entries are never evicted or are strongly referenced by listeners. Bound the size, use time to live, use weak or soft references where suitable and remove listeners.

Q96. How is a delayed task scheduler built? A priority queue ordered by execution time with a worker that waits until the earliest time (a delay queue), a thread pool to run tasks, and cancellation support.

Q97. How are read-heavy shared structures made fast? Immutable snapshots replaced atomically (copy on write), read-write locks, or concurrent collections; measure before choosing.

Q98. Why use immutable objects in concurrent designs? They need no synchronisation and can be shared freely; records and unmodifiable collections make this easy in Java.

Q99. How are virtual threads relevant? They make blocking I/O cheap, so a thread-per-request design scales to many concurrent requests without callbacks; limits on downstream resources such as the connection pool are still required.

Q100. How is a design tested for concurrency? Stress tests with many threads, deterministic tests with a controllable clock and executor, invariant assertions (no double booking, balance totals preserved) and tools that detect races.

4.5.5 Twenty design prompts

Each prompt is paired with the approach in one paragraph; the case study chapters of the earlier parts treat most of them in full.

Prompt	Approach
`URL shortener`	Base62 of a unique ID (counter ranges or Snowflake), key-value store keyed by short code, cache for hot codes, 301 or 302 redirect, TTL and analytics asynchronously; read-heavy, so cache and read replicas.
`Rate limiter`	Token bucket per key held in a shared store with atomic updates; rules by endpoint and tier; fail open or closed by policy; headers report remaining quota.
`News feed`	Fan-out on write for ordinary users into per-follower feed lists, fan-out on read for accounts with very many followers, merge at read; cache feeds; rank asynchronously.
`Chat`	WebSocket gateways, a message store partitioned by conversation, per-conversation sequence numbers for ordering, acknowledgements for delivery and read receipts, offline sync from the store, push notification for offline users.
`Notification service`	Event intake, preference and deduplication check, priority queues per channel, workers with provider adapters, retries with backoff, dead-letter queue, delivery status store.
`Video streaming`	Upload to object storage, transcoding pipeline into adaptive bitrate ladders, segment files and manifest, CDN delivery, metadata in a database, view counts aggregated asynchronously.
`Ride hailing`	Driver locations in a geospatial index (geohash or quadtree) updated every few seconds, nearest-driver query with filtering, matching service with offer timeouts, trip state machine, surge from demand-supply ratio per cell.
`Ticket booking`	Seat hold with expiry using an atomic conditional update, payment with idempotency, release of expired holds, waiting room for peak sales, unique constraint on seat and show.
`E-commerce checkout`	Cart in a fast store, order saga across inventory, payment and shipping with compensations, idempotent order creation, inventory reservation with expiry, outbox for events.
`Search autocomplete`	Trie or prefix map with precomputed top-k, updated by a batch pipeline from query logs, cached at the edge, personalisation as a re-rank layer.
`File sync (Drive)`	Chunking with content hashes, deduplication by hash, metadata service with versions, change notifications to clients, conflict copies on concurrent edits, resumable upload.

Prompt	Approach
`Payment wallet`	Double-entry ledger in a relational store, idempotency keys, balance as derived data or guarded by a constraint, reconciliation with external providers, immutable audit trail.
`Distributed cache`	Consistent hashing with virtual nodes, replication for availability, eviction policy, client-side routing, hot key mitigation, persistence optional.
`Key-value store`	Partitioning by hash ring, replication with quorum reads and writes, hinted handoff, anti-entropy repair, vector clocks or last-write-wins for conflicts.
`Task scheduler`	Persistent job store, a time-ordered index of due jobs, leader or partitioned pollers, leases with fencing for exclusive execution, retries and a dead-letter state.
`Web crawler`	URL frontier with per-host politeness queues, fetcher workers, deduplication by URL normalisation and content hash, robots rules, storage of pages, re-crawl priority.
`Unique ID generator`	Snowflake IDs from timestamp, worker id and sequence; handling of clock moving backwards; worker id assignment through a coordination service or configuration.
`Pastebin`	Object storage for content, metadata and expiry in a database, short key generation, CDN for reads, cleanup job for expired items, size and abuse limits.
`Leaderboard`	Sorted set per period in an in-memory store, score updates atomic, top-k by rank range, sharding by period or region, periodic persistence and archival.
`Metrics and monitoring`	Agents push or are scraped, ingestion through a queue into a time-series store partitioned by series, downsampling for retention, an alert evaluator on rolling windows, dashboards querying the store.

Design prompts and approaches.

4.5.6 Preparation checklist

- Practise each phase of the walkthrough with a timer until the first three phases take no more than 18 minutes.
- Memorise the estimation constants: 86,400 seconds per day, 2.5 million seconds per month, powers of two, and the latency orders of magnitude in the cheat sheets.
- For each of the twenty prompts, be able to name the partition key, the consistency requirement, the main failure and the cache strategy.
- Write at least five machine-coding problems from scratch in a fixed time, compile and run them, and state the concurrency behaviour of each.
- Review the failure studies and prepare one example of each failure class with its prevention.

KEY POINTS

Interviewers reward structure, numbers that drive decisions, named trade-offs and failure thinking.

- Spend the first ten minutes on requirements and estimation; each later choice refers back to them.
- Explain mechanisms (partition key, delivery semantics, idempotency) rather than listing products.
- State the assumption behind each decision and what the alternative would cost.
- In machine coding, list entities first, use patterns only where a requirement varies, and state thread-safety explicitly.

4.6 Cheat Sheets and Study Plan

This chapter collects the reference tables used most often while designing and a six-week study plan. The tables summarise material developed in earlier chapters; each is intended for review shortly before an interview. Numbers are orders of magnitude and are labelled as such.

4.6.1 Latency and estimation reference

The latency figures follow the widely circulated "latency numbers every programmer should know" list attributed to Jeff Dean and popularised by Peter Norvig. They are approximate, date from older hardware generations in part and vary by platform; the relative orders of magnitude are what matters in estimation.

Operation	Time	Note
L1 cache reference	`0.5 ns`	reference unit
Main memory reference	`100 ns`	200 times an L1 hit

Operation	Time	Note
Compress 1 KB	3 µs	CPU bound
Send 1 KB over a 1 Gbps link	10 µs	transmission only
Random read 4 KB from SSD	150 µs	NVMe devices are faster
Read 1 MB sequentially from memory	250 µs	memory bandwidth bound
Round trip within one data center	500 µs	cross-zone adds roughly 1 ms or less
Read 1 MB sequentially from SSD	1 ms	four times a memory read
Disk seek (spinning)	10 ms	mechanical movement
Read 1 MB sequentially from disk	20 ms	sequential throughput bound
Round trip between continents	150 ms	bounded by the speed of light in fibre

Approximate latency (orders of magnitude).

Quantity	Formula
Average QPS	requests per day / 86,400 (about 100,000 for a quick estimate)
Peak QPS	average QPS × peak factor (2 to 3 for daily peaks; 5 to 10 for events)
Write QPS	total QPS × write fraction (from the read-to-write ratio)
Storage per day	writes per day × bytes per record × replication factor
Storage for the retention period	storage per day × days; add 20 to 30 percent for indexes and overhead
Bandwidth	QPS × bytes per response (outbound); QPS × bytes per request (inbound)
Cache size	daily unique hot data × 20 percent (80/20 rule); verify with access logs
Concurrent requests (Little's law)	arrival rate × mean latency
Instances	peak QPS / (capacity per instance × target utilisation), plus failure-domain headroom
Availability of serial dependencies	product of availabilities
Availability of redundant parallel components	1 minus the product of unavailabilities (assuming independent failures)
Month and year	1 month is about 2.6 million seconds; 1 year is about 31.5 million seconds

Estimation formulas.

Powers of two: 2^10 is about a thousand (KB), 2^20 a million (MB), 2^30 a billion (GB), 2^40 a trillion (TB). A 64-bit identifier occupies 8 bytes; one billion of them occupy 8 GB. A character of text is typically 1 to 4 bytes in UTF-8.

sreeram

4.6.2 Technology selection

Need	Typical choice	Reason and caveat
Transactions, joins, relational integrity	relational database (PostgreSQL, MySQL)	strong guarantees; scale with replicas, then partitioning
Simple key access at very high volume	key-value or wide-column store	horizontal scale; query flexibility is limited
Nested aggregates read as a unit	document store	flexible schema; cross-document transactions are limited
Low-latency cache, counters, leaderboards	in-memory store (Redis-style)	memory-bound; durability and cluster limits to check
Full-text relevance and facets	search engine (Lucene-based)	derived index; the primary store remains the source of truth
Large binary objects	object storage with CDN	cheap and durable; high latency per object, no partial update
Event streams, replay, many consumers	partitioned log (Kafka-style)	ordering per partition only; operational weight
Task queues with acknowledgements	message broker or cloud queue	simple; limited replay
Time series and metrics	time-series database	compression and downsampling; not for general queries
Relationship traversal	graph database	multi-hop queries; limited scale-out
Analytics over large history	column store or warehouse	scans are fast; point writes are slow

Choosing a technology by need.

4.6.3 Consistency, caching and rate limiting

Model	Guarantee	Typical use
Linearizable	operations appear atomic in real-time order	locks, leader election, balances
Sequential or causal	operations respect order or causality, not real time	collaborative applications, comments
Read your writes	a client sees its own updates	profile edits, posts
Eventual	replicas converge without new updates	likes, feeds, caches, DNS

Consistency models.

Strategy	Read path	Write path	Risk
Cache-aside	check cache, load on miss	write store, invalidate cache	stale reads, races
Read-through	cache loads on miss	write store	cache library coupling
Write-through	read cache	write cache and store together	slower writes
Write-back	read cache	write cache, flush later	loss on cache failure
Write-around	read cache after a miss	write store only	first read always misses

Caching strategies.

Algorithm	Behaviour	Memory per key	Weakness
Fixed window	count per interval	one counter	up to 2 times the limit across a boundary
Sliding window log	timestamps of requests	one entry per request	memory proportional to the limit
Sliding window counter	weighted sum of two windows	two counters	approximation
Token bucket	refill at a rate, burst up to capacity	tokens and time	tuning two parameters
Leaky bucket	constant outflow rate	queue or counter	adds delay; no bursts

Rate limiting algorithms.

4.6.4 Patterns and checklists

Pattern	Intent	Java or framework example
Strategy	interchangeable algorithms behind an interface	`Comparator`, pricing rules
Factory	create objects without naming the class	`List.of`, `Calendar` factories
Builder	construct complex objects in steps	`StringBuilder`, `HttpRequest.newBuilder()`
Singleton	one instance with global access	Spring beans (container-managed)
Observer	notify dependents of changes	`ApplicationEvent` listeners
Decorator	add behaviour by wrapping	`BufferedInputStream`
Adapter	convert one interface to another	`Arrays.asList`, `InputStreamReader`
Proxy	control access to an object	Spring AOP, JDK dynamic proxy
Command	request as an object	`Runnable`, job queues
State	behaviour depends on state	order or elevator state machines
Chain of Responsibility	pass a request along handlers	servlet filters
Template Method	fixed skeleton, variable steps	`JdbcTemplate`
Facade	simple interface to a subsystem	service layer over several repositories

Design patterns quick reference.

LLD checklist

1. Clarify the requirements and write the use cases, including what is out of scope.
2. List the entities (nouns) and their relationships (has-a, is-a, uses); draw a class diagram.
3. Define interfaces for the parts that vary; list behaviours (verbs) and the owner of each.
4. Choose patterns only where a requirement varies; name the requirement.
5. Write compilable code with immutable value types (records), enums for fixed sets and clear package-private visibility.
6. Write a `main` that demonstrates the main scenarios and one failure case.
7. State the concurrency model and the extension points; list what changes if a requirement is added.

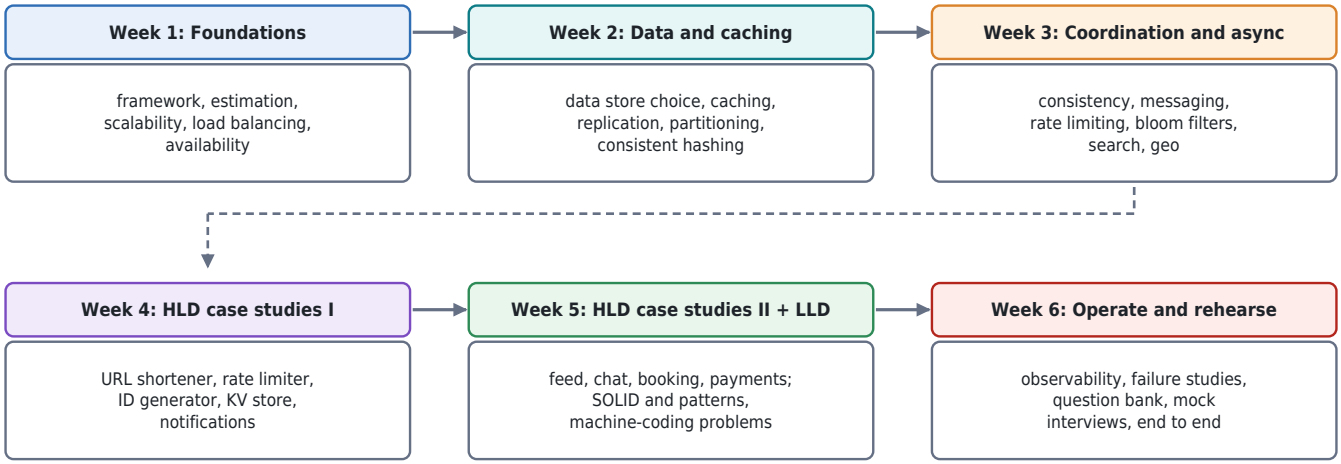
Concurrency checklist

1. Identify all shared mutable state and who writes it.
2. Prefer immutability, confinement to one thread, or message passing over locking.
3. Make check-then-act sequences atomic (one lock, compare-and-set, or a conditional update).
4. Acquire multiple locks in a fixed global order; hold locks for the shortest time; never call unknown code while holding a lock.
5. Use bounded queues and pools; define what happens when they are full.
6. Make operations idempotent where retries exist; record completion before side effects are repeated.
7. Provide a test with many threads and an invariant that must hold afterwards.

4.6.5 Six-week study plan

The plan assumes about ten hours per week and maps each week to chapters of this book by name. Each week ends with one timed 45-minute design and one 30-minute machine-coding problem, which are reviewed against the rubric in the interview chapter.

sreeram



Each week ends with one timed 45-minute mock design and one 30-minute machine-coding problem.

Figure 4.19. The plan moves from principles through data and coordination to case studies, then to low-level design and operations; the last week is rehearsal with the question bank.

Week	Focus	Chapters and activities	Output
1	Foundations	How to approach a system design interview; back-of-the-envelope estimation; scalability and performance; load balancing; availability and reliability. Rework the estimate in each chapter with new numbers.	one-page notes of formulas and two timed estimates
2	Data and caching	Choosing the data store; caching; replication, partitioning and consistent hashing. Run the simulations and change the parameters.	a decision table for stores and a cache design for one read-heavy system
3	Coordination and asynchronous processing	Consistency and coordination; asynchronous processing and messaging; common infrastructure components.	a diagram of a saga with an outbox and a rate limiter design
4	HLD case studies I	URL shortener, pastebin, rate limiter, unique ID generator, key-value store, distributed cache, notification system, web crawler, task scheduler. Do three from a blank page.	three 45-minute mock designs
5	HLD case studies II and LLD	News feed, chat, video streaming, ride hailing, ticket booking, checkout, autocomplete, file sync, payments; object-oriented principles and design patterns; first machine-coding problems.	three mock designs and three coded problems
6	Operate and rehearse	Observability and operations; security; architecture styles; failure studies; question bank; the end-to-end product chapter. Two full mock interviews.	answers to the question bank, two failure examples per class, final mock

Weekly plan.

KEY POINTS

Review tables rather than prose before an interview: latency orders, estimation formulas, technology fit, consistency and caching trade-offs, and the two checklists.

- Estimation constants: 86,400 seconds per day, about 2.6 million per month, peak factor 2 to 3.
- Choose stores by access pattern and consistency need; keep a source of truth and derive the rest.
- Use patterns to remove a concrete change cost, not as decoration.
- Practise under a timer: structure and numbers improve with repetition more than with reading.

4.7 Putting It Together: Designing a Complete Product End to End

The preceding chapters treat techniques one at a time. This chapter applies them in sequence to one small product, a food-delivery backend, from requirements to a running order service. The product is deliberately modest so that every step fits on a page, and every number is an assumption stated at the point of use. The sequence is the one used in an interview and in a design review: requirements, estimation, API, data model, high-level design, low-level design of the central service, failure handling, scaling steps.



Figure 4.20. Six lanes take the output of the previous lane as input: the estimate sizes the architecture, the architecture fixes the data model, the data model shapes the order service code, and the failure analysis tests all of them.

4.7.1 Requirements and scope

Functional requirements. A customer browses restaurants near a location, views a menu, places an order and pays; the restaurant accepts or rejects the order; a courier is assigned, collects and delivers it; the customer tracks the order status and the courier position; the customer or the system can cancel an order before pick-up, with a refund if it was paid.

Non-functional requirements. Checkout availability of 99.9 percent (assumed target); order placement latency under 500 ms at p99 excluding the payment provider; no double charge and no lost order under retries and failures; courier position visible to the customer within 5 seconds; data of a city retained for 12 months.

Out of scope. Promotions and pricing engines, restaurant onboarding, courier payouts, ratings and reviews, route optimisation. **Assumptions.** One country, one payment provider with idempotency-key support, restaurants confirm orders on a tablet.

4.7.2 Capacity estimation

Quantity	Arithmetic	Result
Orders per day	assumed	5,000,000
Average order rate	5,000,000 / 86,400	58 per second
Peak order rate (dinner window)	58 × 3	175 per second
API calls per order (browse, menu, track, order)	assumed 40	200,000,000 per day
Average API rate	200,000,000 / 86,400	2,315 per second
Peak API rate	2,315 × 3	about 7,000 per second
Online couriers at peak	assumed	200,000
Courier location updates	200,000 / 5 s	40,000 per second
Courier update bandwidth	40,000 × 100 B	4 MB per second
Order storage per day	5,000,000 × 2 KB (order, items, payment, outbox)	10 GB per day
Order storage for 12 months	10 GB × 365	3.65 TB
Courier trail storage per day	5,000,000 orders × 60 points × 50 B	15 GB per day
Concurrent active orders	175 per second × 40 min (2,400 s)	420,000
WebSocket connections for tracking	one per active order	420,000
Gateway nodes for tracking	420,000 / 50,000 per node (assumed)	9 nodes
Menu cache	500,000 restaurants × 50 KB	25 GB

Capacity estimate (every input is an assumption).

Three conclusions follow. First, the order write rate of 175 per second, at roughly 12 row writes per order (the order, its items, status changes and outbox rows), is about 2,100 row writes per second at peak, which one well-provisioned relational primary can sustain (assumed); the order store therefore starts unpartitioned and is partitioned by city later. Second, the courier location stream of 40,000 updates per second is the largest write load and must not go to the relational database: the latest position is held in an in-memory geospatial store, and only a sampled trail is archived. Third, the read load of about 7,000 requests per second is served mostly from caches and a search index, which makes the menu and restaurant data a caching problem and not a database problem.

4.7.3 API design

Endpoint	Behaviour	Notes
`POST /v1/orders`	create and pay for an order	header Idempotency-Key required; returns 201 with the order, or the existing order on repeat
`GET /v1/orders/{id}`	order status and courier summary	owner only; 404 for other users
`POST /v1/orders/{id}/cancel`	cancel before pick-up	idempotent; triggers a refund if paid
`GET /v1/restaurants?lat=&lng=&cursor=`	restaurants near a point	cursor pagination, served from the search index
`POST /v1/restaurant/orders/{id}/accept`	restaurant accepts or rejects	must occur within 90 seconds of notification
`PUT /v1/couriers/{id}/location`	courier position	high volume; no response body; last write wins per courier

Main endpoints.

4.7.4 Data model

The order store is relational because an order has a lifecycle with constraints, and correctness (no duplicate, no invalid status) matters more than raw scale at this volume. The idempotency key is protected by a unique constraint, and events leave the service through an outbox table that is written in the same transaction as the status change (the outbox pattern from the messaging chapter).

schema.sql (SQL common to PostgreSQL and SQLite) SQL

```sql
CREATE TABLE orders (
    order_id        BIGINT PRIMARY KEY,
    user_id         BIGINT NOT NULL,
    restaurant_id   BIGINT NOT NULL,
    city_id         INTEGER NOT NULL,
    status          VARCHAR(16) NOT NULL
        CHECK (status IN ('CREATED','PAID','ACCEPTED','PICKED_UP','DELIVERED','CANCELLED')),
    total_paise     BIGINT NOT NULL CHECK (total_paise > 0),
    idempotency_key VARCHAR(64) NOT NULL,
    created_at      TIMESTAMP NOT NULL,
    UNIQUE (user_id, idempotency_key)
);
CREATE INDEX idx_orders_user ON orders (user_id, created_at);
CREATE INDEX idx_orders_restaurant_active ON orders (restaurant_id, status);
CREATE TABLE order_items (
    order_id    BIGINT NOT NULL REFERENCES orders(order_id),
    line_no     INTEGER NOT NULL,
    item_id     BIGINT NOT NULL,
    quantity    INTEGER NOT NULL CHECK (quantity > 0),
    unit_paise  BIGINT NOT NULL,
    PRIMARY KEY (order_id, line_no)
);
CREATE TABLE outbox (
    event_id    BIGINT PRIMARY KEY,
    order_id    BIGINT NOT NULL,
    event_type  VARCHAR(32) NOT NULL,
    payload     VARCHAR(2000) NOT NULL,
    created_at  TIMESTAMP NOT NULL,
    published   INTEGER NOT NULL DEFAULT 0
);
CREATE INDEX idx_outbox_unpublished ON outbox (published, event_id);
```

The constraints are tested below with the SQLite engine of the Python standard library, which accepts this SQL subset. A repeated request with the same idempotency key is rejected by the unique constraint, and an invalid status is rejected by the check constraint; the service converts the first case into "return the existing order".

schema_run.py PYTHON

```python
import sqlite3
db = sqlite3.connect(":memory:")
db.executescript(open("schema.sql").read())
SQL = "INSERT INTO orders VALUES (%d, 42, 7, 3, '%s', 45000, '%s', '2026-10-19')"
cases = [("first, key-123", 1, "CREATED", "key-123"),
         ("duplicate, key-123", 2, "CREATED", "key-123"),
         ("status SHIPPED", 3, "SHIPPED", "key-124")]
for label, oid, status, key in cases:
    try:
        db.execute(SQL % (oid, status, key))
        print("OK     %-20s" % label)
    except sqlite3.IntegrityError as e:
        print("REJECT %-20s %s" % (label, str(e)[:66]))
print(db.execute("SELECT order_id, status, idempotency_key FROM orders").fetchall())
```

Run of schema_run.py (SQLite engine) TERMINAL

```
$ python3 schema_run.py
OK     first, key-123
REJECT duplicate, key-123   UNIQUE constraint failed: orders.user_id, orders.idempotency_key
REJECT status SHIPPED       CHECK constraint failed: status IN ('CREATED','PAID','ACCEPTED','P
[(1, 'CREATED', 'key-123')]
```

The partitioning decision is the following. Orders are looked up by order identifier, by user (history) and by restaurant (active orders). At 3.65 TB per year a single primary with monthly table partitions and read replicas is sufficient for the first stage. When the write rate or the size exceeds one node, the table is sharded by `city_id`, since an order, its

restaurant and its courier belong to one city and cross-city queries are rare; the cost is that a user who orders in two cities has history in two shards, which the history endpoint merges.

4.7.5 High-level architecture

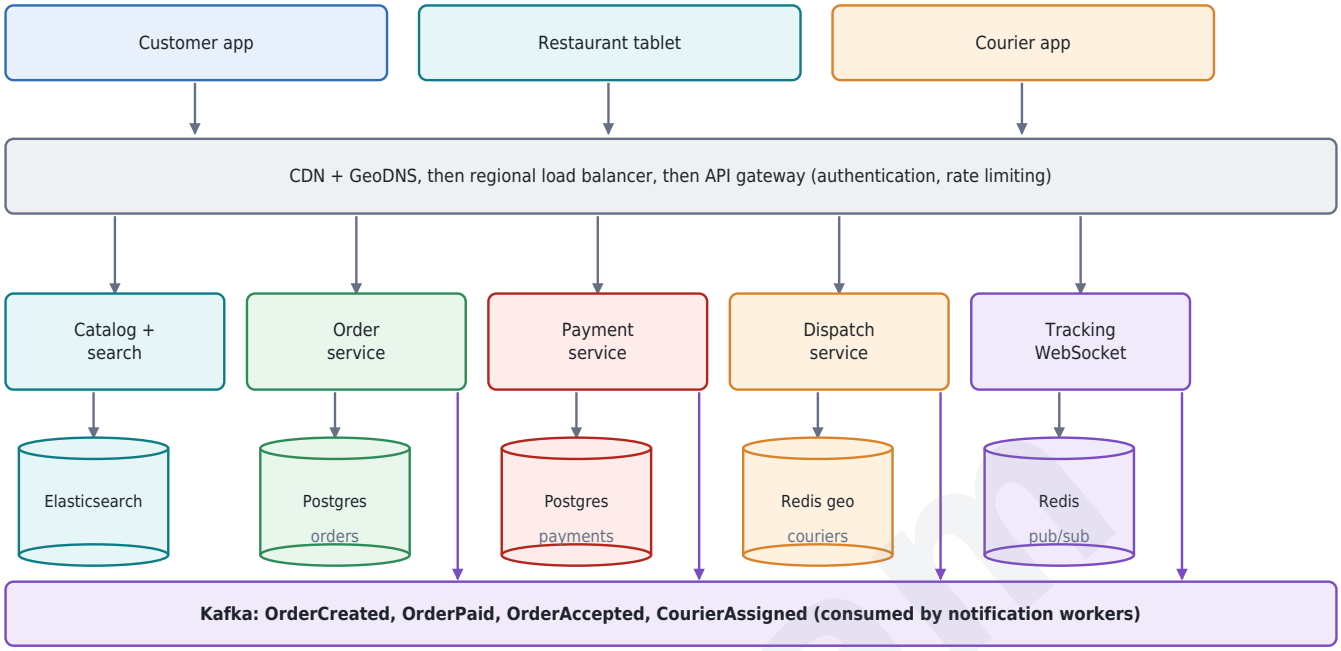


Figure 4.21. Clients enter through the CDN and gateway; stateless services each own a store suited to its access pattern, and domain events leave through Kafka for notification workers and other consumers.

The order flow, with the data touched in each step:

1. The customer application sends `POST /v1/orders` with an idempotency key. The gateway authenticates the token and applies the rate limit.
2. The order service reads the menu prices from the cache (never trusting prices from the client), inserts the order in `CREATED` status and an `OrderCreated` outbox row in one transaction.
3. The order service calls the payment service with a payment key derived from the order identifier. The payment service charges the provider with the same key, so a retry cannot charge twice.
4. On success the order moves to `PAID` and an `OrderPaid` outbox row is written in the same transaction.
5. The outbox relay publishes the event to Kafka; the restaurant notification consumer pushes the order to the restaurant tablet and starts a 90-second acceptance timer.
6. When the restaurant accepts, the status becomes `ACCEPTED` and the dispatch service queries the geospatial store for couriers near the restaurant, offers the order and records the assignment.
7. Courier position updates flow into the geospatial store; the tracking service pushes them to the customer connection over a WebSocket.

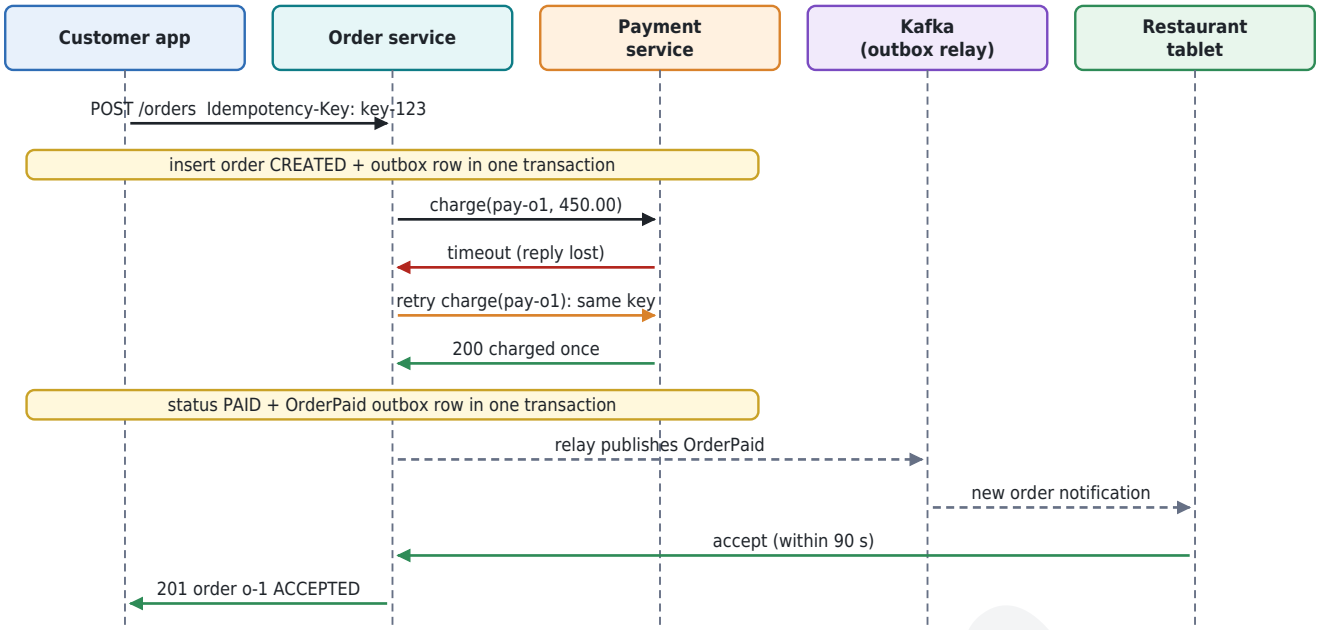


Figure 4.22. The critical flow: the order and its event are committed together, the payment call is retried with the same key after a lost reply, and the customer receives the final state only after the restaurant accepts.

4.7.6 Low-level design of the order service

The order service has four responsibilities: accept an order exactly once, drive the status through a controlled set of transitions, call payment safely, and emit events reliably. The status model is a state machine with a fixed set of legal transitions.

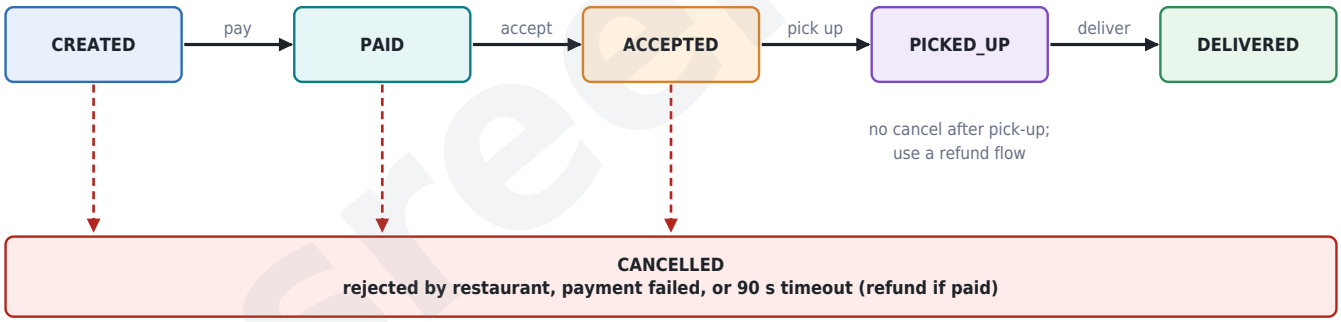


Figure 4.23. Legal transitions of an order: a forward path from creation to delivery, and a cancellation that is allowed before pick-up and ends the order.

The design decisions follow the principles of the object-oriented chapters. An `enum` with an explicit table of successors expresses the state machine, so an illegal transition is a single check and cannot be written by accident (single responsibility, open for new states by editing one table). The gateway to payment is an interface, so that tests use a fake and production uses the provider client (dependency inversion). The service is idempotent at two places: at order creation through the idempotency key, and at payment through a payment key derived from the order. Events are appended to an outbox in the same unit of work as the status change, which in the in-memory listing is a list and in production is the outbox table above.

The listing omits the `PAYMENT_UNKNOWN` parking state of the failure analysis below, which is a sub-state of `CREATED` handled by the reconciliation job, and cancels after three failed attempts. It is a single-file program that runs on Java 21. The fake gateway charges on the first call and then loses the reply, which is the hard case for payment.

FoodOrder.java JAVA

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class FoodOrder {
    enum Status {
        CREATED, PAID, ACCEPTED, PICKED_UP, DELIVERED, CANCELLED;
```

FoodOrder.java (continued) JAVA

```
        private static final Map<Status, Set<Status>> NEXT = Map.of(
            CREATED, Set.of(PAID, CANCELLED),
            PAID, Set.of(ACCEPTED, CANCELLED),
            ACCEPTED, Set.of(PICKED_UP, CANCELLED),
            PICKED_UP, Set.of(DELIVERED),
            DELIVERED, Set.of(),
            CANCELLED, Set.of());
        boolean canGo(Status to) { return NEXT.get(this).contains(to); }
    }

    record Event(String orderId, String type) {}

    static class Order {
        final String id; final String user; final long totalPaise; Status status = Status.CREATED;
        Order(String id, String user, long totalPaise) {
            this.id = id; this.user = user; this.totalPaise = totalPaise;
        }
    }

    /** Fake gateway: the first charge for a key succeeds server-side but the reply is lost. */
    static class Gateway {
        final Map<String, Long> charged = new HashMap<>();
        int calls = 0; boolean dropFirstReply = true;
        boolean charge(String key, long amount) {
            calls++;
            charged.putIfAbsent(key, amount);          // idempotent on the key
            if (dropFirstReply) { dropFirstReply = false; throw new RuntimeException("timeout"); }
            return true;
        }
        void refund(String key) { charged.remove(key); }
    }

    static class OrderService {
        final Map<String, Order> orders = new ConcurrentHashMap<>();
        final Map<String, String> byIdempotencyKey = new ConcurrentHashMap<>();
        final List<Event> outbox = new ArrayList<>();
        final Gateway gateway; int seq = 0;
        OrderService(Gateway g) { gateway = g; }

        synchronized Order place(String idemKey, String user, long total) {
            String existing = byIdempotencyKey.get(idemKey);
            if (existing != null) return orders.get(existing);          // duplicate request
            Order o = new Order("o-" + (++seq), user, total);
            orders.put(o.id, o); byIdempotencyKey.put(idemKey, o.id);
            outbox.add(new Event(o.id, "OrderCreated"));
            return o;
        }
        synchronized void pay(Order o) {
            for (int attempt = 1; attempt <= 3; attempt++) {
                try {
                    gateway.charge("pay-" + o.id, o.totalPaise);        // same key on every retry
                    move(o, Status.PAID, "OrderPaid"); return;
                } catch (RuntimeException e) {
                    System.out.println("  payment attempt " + attempt + " failed: "
                        + e.getMessage());
                }
            }
            move(o, Status.CANCELLED, "OrderCancelled");
        }
        synchronized void restaurantReply(Order o, boolean accept) {
            if (accept) move(o, Status.ACCEPTED, "OrderAccepted");
            else {
                gateway.refund("pay-" + o.id);
                move(o, Status.CANCELLED, "OrderRejectedRefunded");
            }
        }
        synchronized void move(Order o, Status to, String eventType) {
            if (!o.status.canGo(to))
                throw new IllegalStateException(o.status + " -> " + to + " not allowed");
            o.status = to;
            outbox.add(new Event(o.id, eventType));   // same transaction in a real DB
        }
    }

    public static void main(String[] args) {
```

FoodOrder.java (continued) JAVA

```
        Gateway gw = new Gateway();
        OrderService svc = new OrderService(gw);
        Order a = svc.place("key-123", "u1", 45_000);
        Order b = svc.place("key-123", "u1", 45_000);           // client retry of the same request
        System.out.println("same order returned for duplicate request: " + (a == b));
        svc.pay(a);
        System.out.println("gateway calls=" + gw.calls + ", distinct charges=" + gw.charged.size()
            + ", status=" + a.status);
        svc.restaurantReply(a, false);
        System.out.println("after rejection: status=" + a.status
            + ", charges left=" + gw.charged.size());
        try { svc.move(a, Status.DELIVERED, "x"); } catch (IllegalStateException e) {
            System.out.println("illegal transition blocked: " + e.getMessage());
        }
        Order c = svc.place("key-456", "u2", 30_000);
        svc.pay(c); svc.restaurantReply(c, true);
        svc.move(c, Status.PICKED_UP, "OrderPickedUp");
        svc.move(c, Status.DELIVERED, "OrderDelivered");
        System.out.println("order " + c.id + " final status=" + c.status);
        svc.outbox.forEach(e -> System.out.println("  outbox: " + e.orderId() + " " + e.type()));
    }
}
```

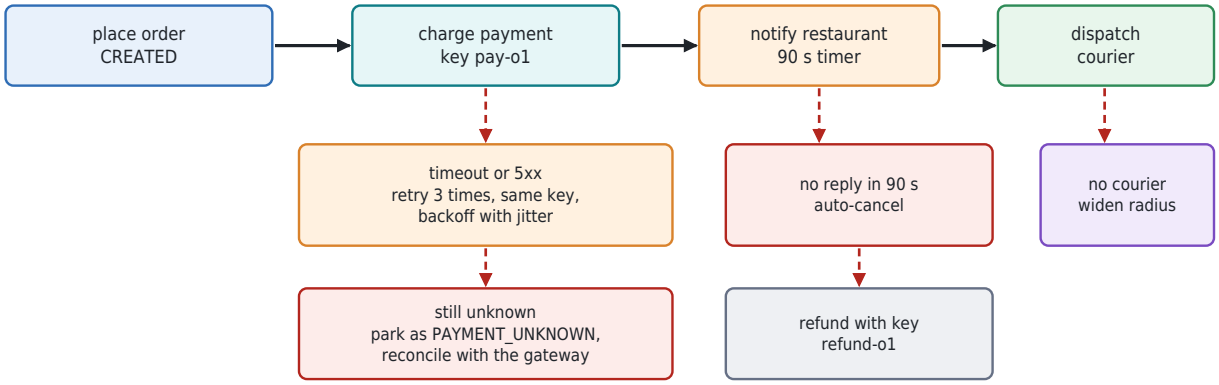
Run of FoodOrder TERMINAL

```
$ java FoodOrder.java
same order returned for duplicate request: true
  payment attempt 1 failed: timeout
gateway calls=2, distinct charges=1, status=PAID
after rejection: status=CANCELLED, charges left=0
illegal transition blocked: CANCELLED -> DELIVERED not allowed
order o-2 final status=DELIVERED
  outbox: o-1 OrderCreated
  outbox: o-1 OrderPaid
  outbox: o-1 OrderRejectedRefunded
  outbox: o-2 OrderCreated
  outbox: o-2 OrderPaid
  outbox: o-2 OrderAccepted
  outbox: o-2 OrderPickedUp
  outbox: o-2 OrderDelivered
```

What to notice in the run: the duplicate `place` call with `key-123` returns the same order object, so no second order exists. The first payment attempt fails with a timeout after the gateway has already recorded the charge; the second attempt uses the same key, and the gateway shows one distinct charge from two calls. When the restaurant rejects, the refund removes the charge and the order moves to `CANCELLED`; an attempted move from `CANCELLED` to `DELIVERED` raises an exception. The outbox lists the events in order, and the relay would publish them to Kafka at least once, so consumers must tolerate duplicates.

The listing uses `synchronized` methods for clarity. This serialises all orders and is acceptable only in a demonstration. In production the exclusion for one order is provided by the database: a unique constraint for the idempotency key and a conditional update for each transition (`UPDATE orders SET status = 'PAID' WHERE order_id = ? AND status = 'CREATED'`, succeeding only for one caller). The payment call must not be made inside a database transaction that holds locks, since the provider latency would hold the locks; the status `CREATED` committed first gives the retry logic and the reconciliation job a durable record to act on.

4.7.7 Failure handling



Every failure path ends in a defined state: retried, reconciled, refunded or cancelled.

Dashed red arrows mark failure transitions.

Figure 4.24. Each failure path leads to a defined state: payment uncertainty is retried with the same key and then reconciled, a silent restaurant causes cancellation and refund, and a missing courier widens the search.

Failure	Handling	Detection
Payment provider timeout	retry up to 3 times with jittered backoff and the same payment key; then mark the order `PAYMENT_UNKNOWN` and let a reconciliation job query the provider	payment error rate, count of unknown payments older than 5 minutes
Order service instance crashes after the commit	the committed state and outbox row survive; the relay publishes later	outbox lag metric
Duplicate Kafka event	consumers deduplicate by event identifier or are idempotent	duplicate counter per consumer
Restaurant does not respond	after 90 seconds cancel the order and refund	acceptance timeout count per restaurant
No courier available	widen the radius in steps, then offer a delay or cancel	unassigned orders older than 5 minutes
Database primary fails	automatic promotion of a replica in the same region; writes fail for tens of seconds (assumed); clients retry with idempotency keys	replication lag, failover events
Region failure	warm standby in a second region, replication lag assumed 1 to 5 seconds; failover decided by a human with a runbook	RPO measured as lag at failover, RTO from drills
Menu cache failure	serve from the search index with a small rate limit, admission control for the database	cache hit ratio

Failure analysis of the order flow.

The design applies the failure classes from the failure studies chapter: retries carry an idempotency key (retry storms and duplicates), the cache has a defined fallback (cold start), the region failover is manual (to avoid the automatic cross-region promotion that produced the split brain in the GitHub report), and every state has an exit (no order is left in an undefined state).

4.7.8 Scaling steps and operations

Stage	Trigger	Change
1	launch in one city	single primary with replicas, Redis, Kafka, three zones, autoscaled stateless services (45 instances for 7,000 requests per second as in the capacity example in the operations chapter)
2	order writes above 1,000 per second or the table above 5 TB	shard orders by city; separate payment database; move search to a dedicated cluster
3	second country	home region per country for data residency; regional deployments; a global directory of users
4	courier updates above 100,000 per second	partition the geospatial store by city; batch updates at the gateway; reduce frequency for stationary couriers

Scaling the product through stages (assumed thresholds).

Operations follow the earlier chapter: the SLO is checkout success (orders created and paid divided by attempts), alerted by multiwindow burn rate; traces carry the order identifier across the gateway, order, payment and dispatch services; deployments are canary with an automatic halt on the checkout error ratio; the new checkout flow is behind a flag with percentage rollout. Security follows the security chapter: token at the gateway, ownership check in the order service, the card data held only by the payment provider, and personal data (address, phone) masked in logs.

4.7.9 Trade-offs and extensions

- A single relational primary is the simplest design that meets the estimate; the alternative of starting with a sharded key-value store would add cost and remove joins that the lifecycle uses.
- The choice of synchronous payment inside order placement gives the customer an immediate result but couples availability of checkout to the provider. An asynchronous variant (accept the order, charge later, notify) improves availability and increases the cases to handle.
- Manual regional failover trades a longer recovery time for protection against divergence; a team with practised drills and fencing may choose automation.
- Courier assignment here is greedy by distance. A batched global matching improves efficiency and adds latency and complexity.
- The extension to scheduled orders adds a time-ordered job store and the task scheduler design; promotions add a pricing service whose result is stored with the order so that history is reproducible.

4.7.10 Questions to expect

How is a double charge prevented when the payment call times out?

Both the order creation and the payment carry idempotency keys; the payment key is derived from the order identifier, so every retry presents the same key and the provider returns the original result. If the outcome is still unknown after the retries, the order is parked and reconciled against the provider by a job.

Why is the payment not made inside the database transaction?

A transaction held open across a slow external call holds locks and connections for the duration, which under provider slowness exhausts the pool. The order is committed first in `CREATED` status, and the payment outcome is applied in a second short transaction guarded by the expected status.

How is an event guaranteed to be published when the order changes?

The status change and the event row are written in one transaction, and a relay publishes unpublished rows and marks them. Failure between the commit and the publish leaves the row for the next relay run, so delivery is at least once and consumers deduplicate.

What happens when the restaurant never responds?

A timer started when the notification is sent cancels the order after 90 seconds and issues a refund with its own idempotency key. The timer state is stored, not held in memory, so that it survives a restart.

How would the system be scaled to ten times the load?

Shard orders by city, partition the geospatial store by city, scale the stateless services and WebSocket gateways horizontally, expand the search index and caches, and verify each step by a load test that includes recovery from a temporary dip, as in the metastable failure study.

What would be monitored on the first day?

Checkout success ratio against the SLO, p99 latency per endpoint, payment error and unknown-payment counts, outbox lag, consumer lag, acceptance timeouts, unassigned orders, cache hit ratio, database replication lag and the saturation of the connection pools.

KEY POINTS

A complete design is a chain: requirements bound the estimate, the estimate selects the architecture, the architecture shapes the data model and code, and failure analysis tests each link.

- Estimate first: the courier location stream, not the orders, was the largest write load and set the store choice.
- Use idempotency keys at order creation and at payment, and an outbox for events.
- Model the lifecycle as an explicit state machine with a table of legal transitions.
- Give every failure a defined exit state, and keep regional failover deliberate and rehearsed.

sreeram